PHP — Linguagem

Manual oficial do PHP (php.net) — English

Fonte oficial: https://www.php.net/manual/
Gerado para o projeto doc-frog

Getting Started

Introduction

What is PHP?

PHP (recursive acronym for *PHP: Hypertext Preprocessor*) is a widely-used open source general-purpose scripting language that is especially suited for web development and can be embedded into HTML.

Nice, but what does that mean? An example:

Example #1 An introductory example

```
<!DOCTYPE html>
<html>
    <head>
        <title>Example</title>
    </head>
    <body>

        <?php
            echo "Hi, I'm a PHP script!";
        ?>

    </body>
</html>
```

Instead of lots of commands to output HTML (as seen in C or Perl), PHP pages contain HTML with embedded code that does something (in this case, output Hi, I'm a PHP script!). The PHP code is enclosed in special start and end processing instructions `<?php` and `?>` that allow jumping in and out of "PHP mode."

What distinguishes PHP from something like client-side JavaScript is that the code is executed on the server, generating HTML which is then sent to the client. The client would receive the results of running that script, but would not know what the underlying code was. A web server can even be configured to process all HTML files with PHP, and then there's no way that users can tell that PHP is being used.

The best part about using PHP is that it is extremely simple for a newcomer, but offers many advanced features for a professional programmer. Don't be afraid to read the long list of PHP's features. With PHP, almost anyone can get up and running and be writing simple scripts in no time at all.

Although PHP's development is focused on server-side scripting, much more can be done with it. Read on, and see more in the What can PHP do? section, or go right to the introductory tutorial to jump straight to learning about web programming.

What can PHP do?

Anything. PHP is mainly focused on server-side scripting, so it can do anything any other CGI program can do, such as collect form data, generate dynamic page content, or send and receive cookies. But PHP can do much more.

There are two main areas where PHP scripts are used.

- Server-side scripting. This is the most widely used and main target field for PHP. Three things are needed to make this work: the PHP parser (CGI or server module), a web server, and a web browser. All these can run on a local machine in order to just experiment with PHP programming. See the installation instructions section for more information.
- Command line scripting. A PHP script can be run without any server or browser, only the PHP parser is needed to use it this way. This type of usage is ideal for scripts regularly executed using **cron** (on Unix or macOS) or Task Scheduler (on Windows). These scripts can also be used for simple text processing tasks. See the section about Command line usage of PHP for more information.

PHP can be used on all major operating systems, including Linux, many Unix variants (including HP-UX, Solaris and OpenBSD), Microsoft Windows, macOS, RISC OS, and probably others. PHP also has support for most of the web servers today. This includes Apache, IIS, and many others. And this includes any web server that can utilize the FastCGI PHP binary, like lighttpd and nginx. PHP works as either a module, or as a CGI processor.

So with PHP, developers have the freedom of choosing an operating system and a web server. Furthermore, they also have the choice of using procedural programming or object-oriented programming (OOP), or a mixture of them both.

PHP is not limited to outputting HTML. PHP's abilities include outputting rich file types, such as images or PDF files, encrypting data, and sending emails. It can also output easily any text, such as JSON or XML. PHP can autogenerate these files, and save them in the file system, instead of printing it out, forming a server-side cache for dynamic content.

One of the strongest and most significant features in PHP is its support for a wide range of databases. Writing a database-enabled web page is incredibly simple using one of the database specific extensions (e.g., for mysql), or using an abstraction layer like PDO, or connect to any database supporting the Open Database Connection standard via the ODBC extension. Other databases may utilize cURL or sockets, like CouchDB.

PHP also has support for talking to other services using protocols such as LDAP, IMAP, SNMP, NNTP, POP3, HTTP, COM (on Windows) and countless others. It can also open raw network sockets and interact using any other protocol.

PHP has useful text processing features, which includes the Perl compatible regular expressions (PCRE), and many extensions and tools to parse and access XML documents. PHP standardizes all of the XML extensions on the solid base of libxml2, and extends the feature set adding SimpleXML, XMLReader and XMLWriter support.

And many other interesting extensions exist, which are categorized both alphabetically and by category. And there are additional PECL extensions that may or may not be documented within the PHP manual itself, like » XDebug.

This page is not enough to list all the features and benefits PHP can offer. Read on in the sections about installing PHP, and see the function reference part for explanation of the extensions mentioned here.

A simple tutorial

Table of Contents

Here we would like to show the very basics of PHP in a short, simple tutorial. This text only deals with dynamic web page creation with PHP, though PHP is not only capable of creating web pages. See the section titled What can PHP do for more information.

PHP-enabled web pages are treated just like regular HTML pages and you can create and edit them the same way you normally create regular HTML pages.

Your first PHP-enabled page

This tutorial assumes PHP is already installed. Installation instructions can be found on the » download page.

Create a file named *hello.php* with the following content:

Example #1 Our first PHP script: *hello.php*

```
<?php

echo "Hello World!";

?>
```

Using your terminal, navigate to the directory containing this file and start a development server with the following command:

```
php -S localhost:8000
```

Use your browser to access the file with your web server's URL, ending with the `/hello.php` file reference. According to the previous command executed, the URL will be `http://localhost:8000/hello.php`. If everything is configured correctly, this file will be parsed by PHP and you will see the "Hello World!" output displayed in your browser.

PHP can be embedded within a normal HTML web page. That means inside your HTML document you can write the PHP statements, as demonstrated in the following example:

```
<!DOCTYPE html>
<html>
    <head>
        <title>PHP Test</title>
    </head>
    <body>
        <?php echo '<p>Hello World</p>'; ?>
    </body>
</html>
```

This will result in the following output:

```
<!DOCTYPE html>
<html>
    <head>
        <title>PHP Test</title>
    </head>
    <body>
        <p>Hello World</p>
    </body>
</html>
```

This program is extremely simple and you really did not need to use PHP to create a page like this. All it does is display: `Hello World` using the PHP echo statement. Note that the file *does not need to be executable* or special in any way. The server finds out that this file needs to be interpreted by PHP because you used the ".php" extension, which the server is configured to pass on to PHP. Think of this as a normal HTML file which happens to have a set of special tags available to you that do a lot of interesting things.

The point of the example is to show the special PHP tag format. In this example we used `<?php` to indicate the start of a PHP tag. Then we put the PHP statement and left PHP mode by adding the closing tag, `?>`. You may jump in and out of PHP mode in an HTML file like this anywhere you want. For more details, read the manual section on the basic PHP syntax.

> **Note** : **A Note on Line Feeds**
>
> Line feeds have little meaning in HTML, however it is still a good idea to make your HTML look nice and clean by putting line feeds in. A linefeed that follows immediately after a closing `?>` will be removed by PHP. This can be extremely useful when you are putting in many blocks of PHP or include files containing PHP that aren't supposed to output anything. At the same time it can be a bit confusing. You can put a space after the closing `?>` to force a space and a line feed to be output, or you can put an explicit line feed in the last echo/print from within your PHP block.

> **Note** : **A Note on Text Editors**
>
> There are many text editors and Integrated Development Environments (IDEs) that you can use to create, edit and manage PHP files. A partial list of these tools is maintained at » PHP Editors List. If you wish to recommend an editor, please visit the above page and ask the page maintainer to add the editor to the list. Having an editor with syntax highlighting can be helpful.

> **Note** : **A Note on Word Processors**
>
> Word processors such as StarOffice Writer, Microsoft Word and Abiword are not optimal for editing PHP files. If you wish to use one for this test script, you must ensure that you save the file as *plain text* or PHP will not be able to read and execute the script.

Now that you have successfully created a working PHP script, it is time to create the most famous PHP script! Make a call to the phpinfo() function and you will see a lot of useful information about your system and setup such as available predefined variables, loaded PHP modules, and configuration settings. Take some time and review this important information.

Example #2 Get system information from PHP

```php
<?php

phpinfo();

?>
```

Something Useful

Let us do something more useful now. We are going to check what sort of browser the visitor is using. For that, we check the user agent string the browser sends as part of the HTTP request. This information is stored in a variable. Variables always start with a dollar-sign in PHP. The variable we are interested in right now is *$_SERVER['HTTP_USER_AGENT']*.

> **Note** :
>
> *$_SERVER* is a special reserved PHP variable that contains all web server information. It is known as a superglobal. See the related manual page on superglobals for more information.

To display this variable, you can simply do:

Example #1 Printing a variable (Array element)

```php
<?php

echo $_SERVER['HTTP_USER_AGENT'];

?>
```

A sample output of this script may be:

```
Mozilla/5.0 (Linux) Firefox/112.0
```

There are many types of variables available in PHP. In the above example we printed an element from an Array variable. Arrays can be very useful.

$_SERVER is just one variable that PHP automatically makes available to you. A list can be seen in the Reserved Variables section of the manual or you can get a complete list of them by looking at the output of the phpinfo() function used in the example in the previous section.

You can put multiple PHP statements inside a PHP tag and create little blocks of code that do more than just a single echo. For example, if you want to check for Firefox you can do this:

Example #2 Example using control structures and functions

```php
<?php

if (str_contains($_SERVER['HTTP_USER_AGENT'], 'Firefox')) {
    echo 'You are using Firefox.';
}

?>
```

A sample output of this script may be:

```
You are using Firefox.
```

Here we introduce a couple of new concepts. We have an if statement. If you are familiar with the basic syntax used by the C language, this should look logical to you. Otherwise, you should probably pick up an introductory PHP book and read the first couple of chapters, or read the Language Reference part of the manual.

The second concept we introduced was the str_contains() function call. str_contains() is a function built into PHP which determines if a given string contains another string. In this case we are looking for `'Firefox'` (so-called needle) inside *$_SERVER['HTTP_USER_AGENT']* (so-called haystack). If the needle is found inside the haystack, the function returns true. Otherwise, it returns **`false`**. If it returns **`true`**, the if expression evaluates to **`true`** and the code within its {braces} is executed. Otherwise, the code is not run. Feel free to create similar examples, with if, else, and other functions such as strtoupper() and strlen(). Each related manual page contains examples too. If you are unsure how to use functions, you will want to read both the manual page on how to read a function definition and the section about PHP functions.

We can take this a step further and show how you can jump in and out of PHP mode even in the middle of a PHP block:

Example #3 Mixing both HTML and PHP modes

```
<?php
if (str_contains($_SERVER['HTTP_USER_AGENT'], 'Firefox')) {
    ?>
    <h3>str_contains() returned true</h3>
    <p>You are using Firefox</p>
    <?php
} else {
    ?>
    <h3>str_contains() returned false</h3>
    <p>You are not using Firefox</p>
    <?php
}
?>
```

A sample output of this script may be:

```
<h3>str_contains() returned true</h3>
<p>You are using Firefox</p>
```

Instead of using a PHP echo statement to output something, we jumped out of PHP mode and just sent straight HTML. The important and powerful point to note here is that the logical flow of the script remains intact. Only one of the HTML blocks will end up getting sent to the viewer depending on the result of str_contains(). In other words, it depends on whether the string `Firefox` was found or not.

Dealing with Forms

One of the most powerful features of PHP is the way it handles HTML forms. The basic concept that is important to understand is that any form element will automatically be available to your PHP scripts. Please read the manual section on Variables from external sources for more information and examples on using forms with PHP. Here is an example HTML form:

Example #1 A simple HTML form

```
<form action="action.php" method="post">
    <label for="name">Your name:</label>
    <input name="name" id="name" type="text">

    <label for="age">Your age:</label>
    <input name="age" id="age" type="number">

    <button type="submit">Submit</button>
</form>
```

There is nothing special about this form. It is a straight HTML form with no special tags of any kind. When the user fills in this form and hits the submit button, the *action.php* page is called. In this file you would write something like this:

Example #2 Printing data from our form

```
Hi <?php echo htmlspecialchars($_POST['name']); ?>.
You are <?php echo (int) $_POST['age']; ?> years old.
```

A sample output of this script may be:

```
Hi Joe. You are 22 years old.
```

Apart from the htmlspecialchars() and `(int)` parts, it should be obvious what this does. htmlspecialchars() makes sure any characters that are special in html are properly encoded so people can't inject HTML tags or Javascript into your page. For the age field, since we know it is a number, we can just convert it to an int which will automatically get rid of any stray characters. You can also have PHP do this for you automatically by using the filter extension. The *$_POST['name']* and *$_POST['age']* variables are automatically set for you by PHP. Earlier we used the *$_SERVER* superglobal; above we just introduced the *$_POST* superglobal which contains all POST data. Notice how the *method* of our form is POST. If we used the method *GET* then our form information would live in the *$_GET* superglobal instead. You may also use the *$_REQUEST* superglobal, if you do not care about the source of your request data. It contains the merged information of GET, POST and COOKIE data.

What's next?

With your new knowledge you should be able to understand most of the manual.

In particular you may want to explore the following features:

- Reading and writing files with the filesystem functions
- Handling file uploads
- Fetching remote pages and files with Curl
- Storing and analyzing data in a database with PDO (SQLite can be used without running a database server)
- Persisting data across requests with Sessions

There's a trove of libraries and » frameworks for every occasion on the » Packagist repository, all installable via the Composer package manager.

For community help and advice, check out the » Help page.

For a variety of podcasts, presentations and other videos, check out the » community PeerTube.

Other community resources that will help you include "awesome lists" (curated directories of links) and "developer roadmaps" (lists of related topics).

When you're stuck with no idea where to start, try breaking down your project or problem into smaller parts, allowing you to more easily see what you already know how to do and what you need to learn. The list can be as deep / detailed as you need. For example, building a blog might be broken down into the following parts:

- Listing and viewing pages
    - Reading records (pages) from a database
- Creating pages
    - Handle form submission
    - Writing records (pages) to a database
- Admin login
    - Reading records (users) from a database
    - Handling passwords
    - Persisting data (user login) across requests / pages (sessions)

If there's nothing particular you want to build, you can try searching for coding exercises such as katas, challenges and "code golf". Even when not specifically aimed at PHP, most should be completable and will likely challenge your knowledge and thinking.

Language Reference

Basic syntax

Table of Contents

PHP tags

When PHP processes a file, it recognizes the opening and closing tags, `<?php` and `?>`, to define the boundaries of PHP code execution. Content outside these tags is ignored by the PHP parser, allowing PHP to seamlessly embed in various document types.

A whitespace character (space, tab, or newline) must follow `<?php` to ensure proper token separation. Omitting this whitespace will result in a syntax error.

PHP also includes the short echo tag `<?=`, which is shorthand for `<?php echo`.

Example #1 PHP Opening and Closing Tags

```
1.  <?php echo 'if you want to serve PHP code in XHTML or XML documents,
                use these tags'; ?>

2.  You can use the short echo tag to <?= 'print this string' ?>.
    It's equivalent to <?php echo 'print this string' ?>.

3.  <? echo 'this code is within short tags, but will only work '.
            'if short_open_tag is enabled'; ?>
```

Short tags (the third line in the example above) are available by default but can be disabled either via the short_open_tag *php.ini* configuration file directive, or are disabled by default if PHP is built with the **--disable-short-tags** configuration.

> **Note** :
>
> As short tags can be disabled it is recommended to only use the normal tags (`<?php ?>` and `<?= ?>`) to maximise compatibility.

If a file ends with PHP code, it is preferable to omit the PHP closing tag at the end of the file. This prevents accidental whitespace or new lines being added after the PHP closing tag, which may cause unwanted effects because PHP will start output buffering when there is no intention from the programmer to send any output at that point in the script.

Example #2 PHP Code Only File

```
<?php
echo "Hello world\n";

// ... more code
```

```php
echo "Last statement\n";

// the script ends here with no PHP closing tag
```

Escaping from HTML

Everything outside of a pair of opening and closing tags is ignored by the PHP parser which allows PHP files to have mixed content. This allows PHP to be embedded in HTML documents, for example to create templates.

Example #1 Embedding PHP in HTML

```php
<p>This is going to be ignored by PHP and displayed by the browser.</p>
<?php echo 'While this is going to be parsed.'; ?>
<p>This will also be ignored by PHP and displayed by the browser.</p>
```

This works as expected, because when the PHP interpreter hits the ?> closing tags, it simply starts outputting whatever it finds (except for the immediately following newline - see instruction separation) until it hits another opening tag unless in the middle of a conditional statement in which case the interpreter will determine the outcome of the conditional before making a decision of what to skip over. See the next example.

Using structures with conditions

Example #2 Advanced escaping using conditions

```php
<?php if (true): ?>
  This will show if the expression is true.
<?php else: ?>
  Otherwise this will show.
<?php endif; ?>
```

In this example PHP will skip the blocks where the condition is not met, even though they are outside of the PHP open/close tags; PHP skips them according to the condition since the PHP interpreter will jump over blocks contained within a condition that is not met.

For outputting large blocks of text, dropping out of PHP parsing mode is generally more efficient than sending all of the text through echo or print.

> **Note** :
>
> If PHP is embedded within XML or XHTML the normal PHP `<?php ?>` must be used to remain compliant with the standards.

Instruction separation

As in C or Perl, PHP requires instructions to be terminated with a semicolon at the end of each statement. The closing tag of a block of PHP code automatically implies a semicolon; you do not need to have a semicolon terminating the last line of a PHP block. The closing tag for the block will include the immediately trailing newline if one is present.

Example #1 Example showing the closing tag encompassing the trailing newline

```
<?php echo "Some text"; ?>
No newline
<?= "But newline now" ?>
```

The above example will output:

```
Some textNo newline
But newline now
```

Example #2 Examples of entering and exiting the PHP parser

```
<?php
    echo "This is a test\n";
?>

<?php echo "This is a test\n" ?>

<?php echo "We omitted the last closing tag\n";
```

> **Note** :
>
> The closing tag of a PHP block at the end of a file is optional, and in some cases omitting it is helpful when using include or require, so unwanted whitespace will not occur at the end of files, and you will still be able to add headers to the response later. It is also handy if you use output buffering, and would not like to see added unwanted whitespace at the end of the parts generated by the included files.

Comments

PHP supports 'C', 'C++' and Unix shell-style (Perl style) comments. For example:

Example #1 Comments

```
<?php
    echo "This is a test\n"; // This is a one-line c++ style comment
    /* This is a multi line comment
       yet another line of comment */
    echo "This is yet another test\n";
    echo "One Final Test\n"; # This is a one-line shell-style comment
?>
```

The "one-line" comment styles only comment to the end of the line or the current block of PHP code, whichever comes first. This means that HTML code after `// ... ?>` or `# ... ?>` WILL be printed: `?>` breaks out of PHP mode and returns to HTML mode, and `//` or `#` cannot influence that.

Example #2 One Line Comments

```
<h1>This is an <?php # echo 'simple';?> example</h1>
<p>The header above will say 'This is an  example'.</p>
```

'C' style comments end at the first `*/` encountered. Make sure you don't nest 'C' style comments. It is easy to make this mistake if you are trying to comment out a large block of code.

```
<?php
 /*
    echo 'This is a test'; /* This comment will cause a problem */
 */
?>
```

Types

Table of Contents

Introduction

Every single expression in PHP has one of the following built-in types depending on its value:

- null
- bool
- int
- float (floating-point number)
- string
- array
- object
- callable
- resource

PHP is a dynamically typed language, which means that by default there is no need to specify the type of a variable, as this will be determined at runtime. However, it is possible to statically type some aspect of the language via the use of type declarations. Different types that are supported by PHP's type system can be found at the type system page.

Types restrict the kind of operations that can be performed on them. However, if an expression/variable is used in an operation which its type does not support, PHP will attempt to type juggle the value into a type that supports the operation. This process depends on the context in which the value is used. For more information, see the section on Type Juggling.

> **Tip**
>
> The type comparison tables may also be useful, as various examples of comparison between values of different types are present.

> **Note** :
>
> It is possible to force an expression to be evaluated to a certain type by using a type cast. A variable can also be type cast in-place by using the settype() function on it.

To check the value and type of an expression, use the var_dump() function. To retrieve the type of an expression, use the get_debug_type() function. However, to check if an expression is of a certain type use the `is_type` functions instead.

Example #1 Different Types

```php
<?php
$a_bool = true;   // a bool
$a_str  = "foo";  // a string
$a_str2 = 'foo';  // a string
$an_int = 12;     // an int

echo get_debug_type($a_bool), "\n";
echo get_debug_type($a_str), "\n";

// If this is an integer, increment it by four
if (is_int($an_int)) {
    $an_int += 4;
}
var_dump($an_int);

// If $a_bool is a string, print it out
if (is_string($a_bool)) {
    echo "String: $a_bool";
}
?>
```

Output of the above example in PHP 8:

```
bool
string
int(16)
```

> **Note** :
>
> Prior to PHP 8.0.0, where the get_debug_type() is not available, the gettype() function can be used instead. However, it doesn't use the canonical type names.

Type System

PHP uses a nominal type system with a strong behavioral subtyping relation. The subtyping relation is checked at compile time whereas the verification of types is dynamically checked at run time.

PHP's type system supports various atomic types that can be composed together to create more complex types. Some of these types can be written as type declarations.

Atomic types

Some atomic types are built-in types which are tightly integrated with the language and cannot be reproduced with user-defined types.

The list of base types is:

- Built-in types
    - Scalar types:
        - bool type
        - int type
        - float type
        - string type
    - array type
    - object type
    - resource type
    - never type
    - void type
    - Relative class types: self, parent, and static
    - Singleton types
        - false
        - true
    - Unit types
        - null
- User-defined types (generally referred to as class-types)
    - Interfaces
    - Classes
    - Enumerations
- callable type

Scalar types

A value is considered scalar if it is of type int, float, string or bool.

User-defined types

It is possible to define custom types with interfaces, classes and enumerations. These are considered as user-defined types, or class-types. For example, a class called `Elephant` can be defined, then objects of type `Elephant` can be instantiated, and a function can request a parameter of type `Elephant`.

Composite types

It is possible to combine multiple atomic types into composite types. PHP allows types to be combined in the following ways:

- Intersection of class-types (interfaces and class names).
- Union of types.

Intersection types

An intersection type accepts values which satisfy multiple class-type declarations, rather than a single one. Individual types which form the intersection type are joined by the `&` symbol. Therefore, an intersection type comprised of the types `T`, `U`, and `V` will be written as `T&U&V`.

Union types

A union type accepts values of multiple different types, rather than a single one. Individual types which form the union type are joined by the `|` symbol. Therefore, a union type comprised of the types `T`, `U`, and `V` will be written as `T|U|V`. If one of the types is an intersection type, it needs to be bracketed with parentheses for it to be written in DNF: `T|(X&Y)`.

Type aliases

PHP supports two type aliases: mixed and iterable which correspond to the union type of `object|resource|array|string|float|int|bool|null` and `Traversable|array` respectively.

> **Note** :
>
> PHP does not support user-defined type aliases.

NULL

The null type is PHP's unit type, i.e. it has only one value: **`null`**.

Undefined and unset() variables resolve to the value **`null`**.

Syntax

There is only one value of type null, and that is the case-insensitive constant **`null`**.

```php
<?php
$var = NULL;
?>
```

Casting to `null`

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this feature is highly discouraged.

Casting a variable to null using `(unset) $var` will *not* remove the variable or unset its value. It will only return a `null` value.

See Also

- is_null()
- unset()

Booleans

The bool type only has two values, and is used to express a truth value. It can be either `true` or `false`.

Syntax

To specify a bool literal, use the constants `true` or `false`. Both are case-insensitive.

```
<?php

$foo = True; // assign the value TRUE to $foo

var_dump($foo); // bool(true)
?>
```

Typically, the result of an operator which returns a bool value is passed on to a control structure.

```
<?php

$action = "show_version";
$show_separators = true;

// == is an operator which tests
// equality and returns a boolean
if ($action == "show_version") {
    echo "The version is 1.23";
}

// this is not necessary...
if ($show_separators == TRUE) {
    echo "<hr>\n";
}

// ...because this can be used with exactly the same meaning:
if ($show_separators) {
    echo "<hr>\n";
}
?>
```

Converting to boolean

To explicitly convert a value to bool, use the `(bool)` cast. Generally this is not necessary because when a value is used in a logical context it will be automatically interpreted as a value of type bool. For more information see the Type Juggling page.

When converting to bool, the following values are considered **`false`**:

- the boolean **`false`** itself
- the integer `0` (zero)
- the floats `0.0` and `-0.0` (zero)
- the empty string `""`, and the string `"0"`
- an array with zero elements
- the unit type NULL (including unset variables)
- Internal objects that overload their casting behaviour to bool. For example, GMP objects representing the value `0`.

Every other value is considered **`true`** (including resource and **`NAN`**).

> **Warning**
>
> `-1` is considered **`true`**, like any other non-zero (whether negative or positive) number!

Example #1 Casting to Boolean

```php
<?php
var_dump((bool) "");        // bool(false)
var_dump((bool) "0");       // bool(false)
var_dump((bool) 1);         // bool(true)
var_dump((bool) -2);        // bool(true)
var_dump((bool) "foo");     // bool(true)
var_dump((bool) 2.3e5);     // bool(true)
var_dump((bool) array(12)); // bool(true)
var_dump((bool) array());   // bool(false)
var_dump((bool) "false");   // bool(true)
?>
```

Integers

An int is a number of the set $\mathbb{Z}$ = {..., -2, -1, 0, 1, 2, ...}.

See Also

- Floating-point numbers
- Arbitrary precision / BCMath
- Arbitrary length integer / GMP

Syntax

Ints can be specified in decimal (base 10), hexadecimal (base 16), octal (base 8) or binary (base 2) notation. The negation operator can be used to denote a negative int.

To use octal notation, precede the number with a `0` (zero). As of PHP 8.1.0, octal notation can also be preceded with `0o` or `0O`. To use hexadecimal notation, precede the number with `0x`. To use binary notation, precede the number with `0b`.

As of PHP 7.4.0, integer literals may contain underscores (_) between digits, for better readability of literals. These underscores are removed by PHP's scanner.

Example #1 Integer literals

```php
<?php
$a = 1234; // decimal number
$a = 0123; // octal number (equivalent to 83 decimal)
$a = 0o123; // octal number (as of PHP 8.1.0)
$a = 0x1A; // hexadecimal number (equivalent to 26 decimal)
$a = 0b11111111; // binary number (equivalent to 255 decimal)
$a = 1_234_567; // decimal number (as of PHP 7.4.0)
?>
```

Formally, the structure for int literals is as of PHP 8.1.0 (previously, the `0o` or `0O` octal prefixes were not allowed, and prior to PHP 7.4.0 the underscores were not allowed):

```
decimal     : [1-9][0-9]*(_[0-9]+)*
            | 0

hexadecimal : 0[xX][0-9a-fA-F]+(_[0-9a-fA-F]+)*

octal       : 0[oO]?[0-7]+(_[0-7]+)*

binary      : 0[bB][01]+(_[01]+)*

integer     : decimal
            | hexadecimal
            | octal
            | binary
```

The size of an int is platform-dependent, although a maximum value of about two billion is the usual value (that's 32 bits signed). 64-bit platforms usually have a maximum value of about 9E18. PHP does not support unsigned ints. int size can be determined using the constant **PHP_INT_SIZE**, maximum value using the constant **PHP_INT_MAX**, and minimum value using the constant **PHP_INT_MIN**.

Integer overflow

If PHP encounters a number beyond the bounds of the int type, it will be interpreted as a float instead. Also, an operation which results in a number beyond the bounds of the int type will return a float instead.

Example #2 Integer overflow

```php
<?php
$large_number = 50000000000000000000;
var_dump($large_number);         // float(5.0E+19)

var_dump(PHP_INT_MAX + 1);       // 32-bit system: float(2147483648)
```

```
                                    // 64-bit system: float(9.2233720368548E+18)
?>
```

Integer division

There is no int division operator in PHP. To achieve this, use the intdiv() function. `1/2` yields the float `0.5`. The value can be cast to an int to round it towards zero, or the round() function provides finer control over rounding.

Example #3 Divisions

```
<?php
var_dump(25/7);         // float(3.5714285714286)
var_dump((int) (25/7)); // int(3)
var_dump(round(25/7));  // float(4)
?>
```

Converting to integer

To explicitly convert a value to int, use the `(int)` cast. However, in most cases the cast is not needed, since a value will be automatically converted if an operator, function or control structure requires an int argument. A value can also be converted to int with the intval() function.

If a resource is converted to an int, then the result will be the unique resource number assigned to the resource by PHP at runtime.

See also Type Juggling.

From booleans

`false` will yield `0` (zero), and `true` will yield `1` (one).

From floating point numbers

When converting from float to int, the number will be rounded *towards zero*. As of PHP 8.1.0, a deprecation notice is emitted when implicitly converting a non-integral float to an int which loses precision.

Example #4 Casting from Float

```
<?php

function foo($value): int {
  return $value;
}

var_dump(foo(8.1)); // "Deprecated: Implicit conversion from float 8.1 to int loses precision" as of
PHP 8.1.0
var_dump(foo(8.1)); // 8 prior to PHP 8.1.0
var_dump(foo(8.0)); // 8 in both cases

var_dump((int) 8.1); // 8 in both cases
var_dump(intval(8.1)); // 8 in both cases
?>
```

If the float is beyond the boundaries of int (usually `+/- 2.15e+9 = 2^31` on 32-bit platforms and `+/- 9.22e+18 = 2^63` on 64-bit platforms), the result is undefined, since the float doesn't have enough precision to give an exact int result. As of PHP 8.5.0, an **E_WARNING** is emitted when a float that is not representable as an int is cast to one. Prior to PHP 8.5.0, no diagnostic at all was issued.

> **Note** :
>
> `NaN`, `Inf` and `-Inf` will always be zero when cast to int. As of PHP 8.5.0, casting them to int emits an **E_WARNING**, as does casting **NAN** to any other type.

> **Warning**
>
> Never cast an unknown fraction to int, as this can sometimes lead to unexpected results.
>
> ```php
> <?php
> echo (int) ( (0.1+0.7) * 10 ); // echoes 7!
> ?>
> ```
>
> See also the warning about float precision.

From strings

If the string is numeric or leading numeric then it will resolve to the corresponding integer value, otherwise it is converted to zero (`0`).

From NULL

null is always converted to zero (`0`).

From other types

> **Caution**
>
> The behaviour of converting to int is undefined for other types. Do *not* rely on any observed behaviour, as it can change without notice.

Floating-point numbers

Floating-point numbers (also known as "floats", "doubles", or "real numbers") can be specified using any of the following syntaxes:

```php
<?php
$a = 1.234;
$b = 1.2e3;
$c = 7E-10;
$d = 1_234.567; // as of PHP 7.4.0
?>
```

Formally as of PHP 7.4.0 (previously, underscores have not been allowed):

```
LNUM          [0-9]+(_[0-9]+)*
DNUM          ({LNUM}?"."{LNUM}) | ({LNUM}"."{LNUM}?)
EXPONENT_DNUM (({LNUM} | {DNUM}) [eE][+-]? {LNUM})
```

The size of a float is platform-dependent, although a maximum of approximately 1.8e308 with a precision of roughly 14 decimal digits is a common value (the 64 bit IEEE format).

> **Warning**
>
> ### Floating-point precision
>
> Floating-point numbers have limited precision. Although it depends on the system, PHP typically uses the IEEE 754 double precision format, which will give a maximum relative error due to rounding in the order of 1.11e-16. Non-elementary arithmetic operations may give larger errors, and, of course, error propagation must be considered when several operations are compounded.
>
> Additionally, rational numbers that are exactly representable as floating-point numbers in base 10, like `0.1` or `0.7`, do not have an exact representation as floating-point numbers in base 2, which is used internally, no matter the size of the mantissa. Hence, they cannot be converted into their internal binary counterparts without a small loss of precision. This can lead to confusing results: for example, `floor((0.1+0.7)*10)` will usually return `7` instead of the expected `8`, since the internal representation will be something like `7.9999999999999991118...`.
>
> So never trust floating-point number results to the last digit, and do not compare floating-point numbers directly for equality. If higher precision is necessary, the arbitrary precision math functions and gmp functions are available.
>
> For a "simple" explanation, see the » floating point guide that's also titled "Why don't my numbers add up?"

Converting to float

From strings

If the string is numeric or leading numeric then it will resolve to the corresponding float value, otherwise it is converted to zero (`0`).

From other types

For values of other types, the conversion is performed by converting the value to int first and then to float. See Converting to integer for more information.

> **Note** :
>
> As certain types have undefined behavior when converting to int, this is also the case when converting to float.

Comparing floats

As noted in the warning above, testing floating-point values for equality is problematic, due to the way that they are represented internally. However, there are ways to make comparisons of floating-point values that

work around these limitations.

To test floating-point values for equality, an upper bound on the relative error due to rounding is used. This value is known as the machine epsilon, or unit roundoff, and is the smallest acceptable difference in calculations.

$a and *$b* are equal to 5 digits of precision.

Example #1 Comparing Floats

```php
<?php
$a = 1.23456789;
$b = 1.23456780;
$epsilon = 0.00001;

if (abs($a - $b) < $epsilon) {
    echo "true";
}
?>
```

NaN

Some numeric operations can result in a value represented by the constant **NAN**. This result represents an undefined or unrepresentable value in floating-point calculations. Any loose or strict comparisons of this value against any other value, including itself, but except **true**, will have a result of **false**.

Because **NAN** represents any number of different values, **NAN** should not be compared to other values, including itself, and instead should be checked for using is_nan().

Strings

A string is a series of characters, where a character is the same as a byte. This means that PHP only supports a 256-character set, and hence does not offer native Unicode support. See details of the string type.

> **Note** :
>
> On 32-bit builds, a string can be up to 2GB (2147483647 bytes) in length.

Syntax

A string literal can be specified in four different ways:

- single quoted
- double quoted
- heredoc syntax
- nowdoc syntax

Single quoted

The simplest way to specify a string is to enclose it in single quotes (the character ').

To specify a literal single quote, escape it with a backslash (\). To specify a literal backslash, double it (\\). All other instances of backslash will be treated as a literal backslash: this means that the other escape sequences you might be used to, such as \r or \n, will be output literally as specified rather than having any special meaning.

> **Note** :
>
> Unlike the double-quoted and heredoc syntaxes, variables and escape sequences for special characters will *not* be expanded when they occur in single quoted strings.

Example #1 Syntax Variants

```php
<?php
echo 'this is a simple string', PHP_EOL;

echo 'You can also have embedded newlines in
strings this way as it is
okay to do', PHP_EOL;

// Outputs: Arnold once said: "I'll be back"
echo 'Arnold once said: "I\'ll be back"', PHP_EOL;

// Outputs: You deleted C:\*.*?
echo 'You deleted C:\\*.*?', PHP_EOL;

// Outputs: You deleted C:\*.*?
echo 'You deleted C:\*.*?', PHP_EOL;

// Outputs: This will not expand: \n a newline
echo 'This will not expand: \n a newline', PHP_EOL;

// Outputs: Variables do not $expand $either
echo 'Variables do not $expand $either', PHP_EOL;
?>
```

Double quoted

If the string is enclosed in double-quotes ("), PHP will interpret the following escape sequences for special characters:

Escaped characters

Sequence	Meaning
\n	linefeed (LF or 0x0A (10) in ASCII)
\r	carriage return (CR or 0x0D (13) in ASCII)
\t	horizontal tab (HT or 0x09 (9) in ASCII)
\v	vertical tab (VT or 0x0B (11) in ASCII)
\e	escape (ESC or 0x1B (27) in ASCII)
\f	form feed (FF or 0x0C (12) in ASCII)
\\	backslash

Sequence	Meaning
`\$`	dollar sign
`\"`	double-quote
`\[0-7]{1,3}`	Octal: the sequence of characters matching the regular expression `[0-7]{1,3}` is a character in octal notation (e.g. `"\101" === "A"`), which silently overflows to fit in a byte (e.g. `"\400" === "\000"`)
`\x[0-9A-Fa-f]{1,2}`	Hexadecimal: the sequence of characters matching the regular expression `[0-9A-Fa-f]{1,2}` is a character in hexadecimal notation (e.g. `"\x41" === "A"`)
`\u{[0-9A-Fa-f]+}`	Unicode: the sequence of characters matching the regular expression `[0-9A-Fa-f]+` is a Unicode codepoint, which will be output to the string as that codepoint's UTF-8 representation. The braces are required in the sequence. E.g. `"\u{41}" === "A"`

As in single quoted strings, escaping any other character will result in the backslash being printed too.

The most important feature of double-quoted strings is the fact that variable names will be expanded. See string interpolation for details.

Heredoc

A third way to delimit strings is the heredoc syntax: <<<. After this operator, an identifier is provided, then a newline. The string itself follows, and then the same identifier again to close the quotation.

The closing identifier may be indented with spaces or tabs. Whitespace equal to the closing identifier's indentation is then removed from the beginning of every line in the heredoc string. Prior to PHP 7.3.0, the closing identifier *must* begin in the first column of the line.

Also, the closing identifier must follow the same naming rules as any other label in PHP: it must contain only alphanumeric characters and underscores, and must start with a non-digit character or underscore.

Example #2 Basic Heredoc example as of PHP 7.3.0

```
<?php
// closing identifier not indented
echo <<<END
      a
     b
    c
\n
END;

// closing identifier indented with 3 spaces; 3 spaces stripped from each line
echo <<<END
      a
     b
    c
   END;
```

Output of the above example in PHP 7.3:

```
```

```
      a
     b
    c

   a
  b
 c
```

If the closing identifier is indented further than any lines of the body, then a ParseError will be thrown:

Example #3 Closing identifier must not be indented further than any lines of the body

```php
<?php
echo <<<END
  a
 b
c
   END;
```

Output of the above example in PHP 7.3:

```
Parse error: Invalid body indentation level (expecting an indentation level of at least 3) in
example.php on line 4
```

If the closing identifier is indented, tabs can be used as well. However, tabs and spaces *must not* be intermixed. The indentation of the body and the closing identifier must use one or the other consistently. If tabs and spaces are mixed, then a ParseError will be thrown. These whitespace constraints have been included because mixing tabs and spaces for indentation is harmful to legibility.

Example #4 Different indentation for body and closing identifier

```php
<?php
// All of the following code does not work.

// different indentation for body (spaces) and ending marker (tabs)
{
    echo <<<END
     a
        END;
}

// mixing spaces and tabs in body
{
    echo <<<END
        a
     END;
}
```

```
// mixing spaces and tabs in ending marker
{
    echo <<<END
          a
         END;
}
```

Output of the above example in PHP 7.3:

```
Parse error: Invalid indentation - tabs and spaces cannot be mixed in example.php line 8
```

The closing identifier for the body string is not required to be followed by a semicolon or newline. For example, the following code is allowed as of PHP 7.3.0:

Example #5 Continuing an expression after a closing identifier

```
<?php
$values = [<<<END
a
  b
    c
END, 'd e f'];
var_dump($values);
```

Output of the above example in PHP 7.3:

```
array(2) {
  [0] =>
  string(11) "a
  b
    c"
  [1] =>
  string(5) "d e f"
}
```

> **Warning**
>
> If the closing identifier is found at the start of a line, then regardless of whether it is a part of another word, it is considered as the closing identifier and causes a ParseError.
>
> **Example #6 Closing identifier in body of the string tends to cause ParseError**
>
> ```
> <?php
> $values = [<<<END
> a
> b
> ```

```
END ING
END, 'd e f'];
```

Output of the above example in PHP 7.3:

```
Parse error: syntax error, unexpected identifier "ING", expecting "]" in example.php on line
5
```

To avoid this problem, it is safe to follow the simple rule: *do not choose a word that appears in the body of the text as a closing identifier*.

> **Warning**
>
> Prior to PHP 7.3.0, it is very important to note that the line with the closing identifier must contain no other characters, except a semicolon (`;`). That means especially that the identifier *may not be indented*, and there may not be any spaces or tabs before or after the semicolon. It's also important to realize that the first character before the closing identifier must be a newline as defined by the local operating system. This is `\n` on UNIX systems, including macOS. The closing delimiter must also be followed by a newline.
>
> If this rule is broken and the closing identifier is not "clean", it will not be considered a closing identifier, and PHP will continue looking for one. If a proper closing identifier is not found before the end of the current file, a parse error will result at the last line.
>
> **Example #7 Invalid example, prior to PHP 7.3.0**
>
> ```
> <?php
> class foo {
>     public $bar = <<<EOT
> bar
>     EOT;
> }
> // Identifier must not be indented
> ?>
> ```
>
> **Example #8 Valid example, even prior to PHP 7.3.0**
>
> ```
> <?php
> class foo {
>     public $bar = <<<EOT
> bar
> EOT;
> }
> ?>
> ```
>
> Heredocs containing variables can not be used for initializing class properties.

Heredoc text behaves just like a double-quoted string, without the double quotes. This means that quotes in a heredoc do not need to be escaped, but the escape codes listed above can still be used. Variables are

expanded, but the same care must be taken when expressing complex variables inside a heredoc as with strings.

Example #9 Heredoc string quoting example

```php
<?php
$str = <<<EOD
Example of string
spanning multiple lines
using heredoc syntax.
EOD;

/* More complex example, with variables. */
class foo
{
    var $foo;
    var $bar;

    function __construct()
    {
        $this->foo = 'Foo';
        $this->bar = array('Bar1', 'Bar2', 'Bar3');
    }
}

$foo = new foo();
$name = 'MyName';

echo <<<EOT
My name is "$name". I am printing some $foo->foo.
Now, I am printing some {$foo->bar[1]}.
This should print a capital 'A': \x41
EOT;
?>
```

The above example will output:

```
My name is "MyName". I am printing some Foo.
Now, I am printing some Bar2.
This should print a capital 'A': A
```

It is also possible to use the Heredoc syntax to pass data to function arguments:

Example #10 Heredoc in arguments example

```php
<?php
var_dump(array(<<<EOD
foobar!
EOD
));
?>
```

It's possible to initialize static variables and class properties/constants using the Heredoc syntax:

Example #11 Using Heredoc to initialize static values

```
<?php
// Static variables
function foo()
{
    static $bar = <<<LABEL
Nothing in here...
LABEL;
}

// Class properties/constants
class foo
{
    const BAR = <<<FOOBAR
Constant example
FOOBAR;

    public $baz = <<<FOOBAR
Property example
FOOBAR;
}
?>
```

The opening Heredoc identifier may optionally be enclosed in double quotes:

Example #12 Using double quotes in Heredoc

```
<?php
echo <<<"FOOBAR"
Hello World!
FOOBAR;
?>
```

Nowdoc

Nowdocs are to single-quoted strings what heredocs are to double-quoted strings. A nowdoc is specified similarly to a heredoc, but *no string interpolation is done* inside a nowdoc. The construct is ideal for embedding PHP code or other large blocks of text without the need for escaping. It shares some features in common with the SGML `<![CDATA[ ]]>` construct, in that it declares a block of text which is not for parsing.

A nowdoc is identified with the same <<< sequence used for heredocs, but the identifier which follows is enclosed in single quotes, e.g. `<<<'EOT'`. All the rules for heredoc identifiers also apply to nowdoc identifiers, especially those regarding the appearance of the closing identifier.

Example #13 Nowdoc string quoting example

```
<?php
echo <<<'EOD'
Example of string spanning multiple lines
using nowdoc syntax. Backslashes are always treated literally,
e.g. \\ and \'.
EOD;
```

The above example will output:

```
Example of string spanning multiple lines
using nowdoc syntax. Backslashes are always treated literally,
e.g. \\ and \'.
```

Example #14 Nowdoc string quoting example with variables

```php
<?php
class foo
{
    public $foo;
    public $bar;

    function __construct()
    {
        $this->foo = 'Foo';
        $this->bar = array('Bar1', 'Bar2', 'Bar3');
    }
}

$foo = new foo();
$name = 'MyName';

echo <<<'EOT'
My name is "$name". I am printing some $foo->foo.
Now, I am printing some {$foo->bar[1]}.
This should not print a capital 'A': \x41
EOT;
?>
```

The above example will output:

```
My name is "$name". I am printing some $foo->foo.
Now, I am printing some {$foo->bar[1]}.
This should not print a capital 'A': \x41
```

Example #15 Static data example

```php
<?php
class foo {
    public $bar = <<<'EOT'
bar
EOT;
}
?>
```

String interpolation

When a string is specified in double quotes or with heredoc, variables can be substituted within it.

There are two types of syntax: a basic one and an advanced one. The basic syntax is the most common and convenient. It provides a way to embed a variable, an array value, or an object property in a string with a minimum of effort.

Basic syntax

If a dollar sign (`$`) is encountered, the characters that follow it which can be used in a variable name will be interpreted as such and substituted.

Example #16 String Interpolation

```php
<?php
$juice = "apple";

echo "He drank some $juice juice." . PHP_EOL;

?>
```

The above example will output:

```
He drank some apple juice.
```

Formally, the structure for the basic variable substitution syntax is as follows:

```
string-variable::
     variable-name   (offset-or-property)?
   | ${   expression   }

offset-or-property::
     offset-in-string
   | property-in-string

offset-in-string::
     [   name   ]
   | [   variable-name   ]
   | [   integer-literal   ]

property-in-string::
     ->  name

variable-name::
     $   name

name::
     [a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*
```

Warning

The `${ expression }` syntax is deprecated as of PHP 8.2.0, as it can be interpreted as variable variables:

```
<?php
const foo = 'bar';
$foo = 'foo';
$bar = 'bar';
var_dump("${foo}");
var_dump("${(foo)}");
?>
```

Output of the above example in PHP 8.2:

```
Deprecated: Using ${var} in strings is deprecated, use {$var} instead in file on line 6

Deprecated: Using ${expr} (variable variables) in strings is deprecated, use {${expr}}
instead in file on line 9
string(3) "foo"
string(3) "bar"
```

The above example will output:

```
string(3) "foo"
string(3) "bar"
```

The advanced string interpolation syntax should be used instead.

Note :

If it is not possible to form a valid name the dollar sign remains verbatim in the string:

```
<?php
echo "No interpolation $  has happened\n";
echo "No interpolation $\n has happened\n";
echo "No interpolation $2 has happened\n";
?>
```

The above example will output:

```
No interpolation $  has happened
No interpolation $
```

```
 has happened
No interpolation $2 has happened
```

Example #17 Interpolating the value of the first dimension of an array or property

```php
<?php
$juices = array("apple", "orange", "string_key" => "purple");

echo "He drank some $juices[0] juice.";
echo PHP_EOL;
echo "He drank some $juices[1] juice.";
echo PHP_EOL;
echo "He drank some $juices[string_key] juice.";
echo PHP_EOL;

class A {
    public $s = "string";
}

$o = new A();

echo "Object value: $o->s.";
?>
```

The above example will output:

```
He drank some apple juice.
He drank some orange juice.
He drank some purple juice.
Object value: string.
```

> **Note** :
>
> The array key must be unquoted, and it is therefore not possible to refer to a constant as a key with the basic syntax. Use the advanced syntax instead.

As of PHP 7.1.0, *negative* numeric indices are supported too.

Example #18 Negative numeric indices

```php
<?php
$string = 'string';
echo "The character at index -2 is $string[-2].", PHP_EOL;
$string[-3] = 'o';
echo "Changing the character at index -3 to o gives $string.", PHP_EOL;
?>
```

The above example will output:

```
The character at index -2 is n.
Changing the character at index -3 to o gives strong.
```

For anything more complex, the advanced syntax must be used.

Advanced (curly) syntax

The advanced syntax permits the interpolation of *variables* with arbitrary accessors.

Any scalar variable, array element or object property (static or not) with a string representation can be included via this syntax. The expression is written the same way as it would appear outside the string, and then wrapped in `{` and `}`. Since `{` can not be escaped, this syntax will only be recognised when the `$` immediately follows the `{`. Use `{\$` to get a literal `{$`. Some examples to make it clear:

Example #19 Curly Syntax

```php
<?php
const DATA_KEY = 'const-key';
$great = 'fantastic';
$arr = [
    '1',
    '2',
    '3',
    [41, 42, 43],
    'key' => 'Indexed value',
    'const-key' => 'Key with minus sign',
    'foo' => ['foo1', 'foo2', 'foo3']
];

// Won't work, outputs: This is { fantastic}
echo "This is { $great}" . PHP_EOL;

// Works, outputs: This is fantastic
echo "This is {$great}" . PHP_EOL;

// To show curly braces in the output:
echo "This is {{$great}}" . PHP_EOL;

class Square {
    public $width;

    public function __construct(int $width) { $this->width = $width; }
}

$square = new Square(5);

// Works
echo "This square is {$square->width}00 centimeters wide." . PHP_EOL;
```

```php
// Works, quoted keys only work using the curly brace syntax
echo "This works: {$arr['key']}" . PHP_EOL;


// Works
echo "This works: {$arr[3][2]}" . PHP_EOL;

echo "This works: {$arr[DATA_KEY]}" . PHP_EOL;

// When using multidimensional arrays, always use braces around arrays
// when inside of strings
echo "This works: {$arr['foo'][2]}" . PHP_EOL;

echo "This works: {$obj->values[3]->name}" . PHP_EOL;

echo "This works: {$obj->$staticProp}" . PHP_EOL;

// Won't work, outputs: C:\directory\{fantastic}.txt
echo "C:\directory\{$great}.txt" . PHP_EOL;

// Works, outputs: C:\directory\fantastic.txt
echo "C:\\directory\\{$great}.txt" . PHP_EOL;
?>
```

> **Note** :
>
> As this syntax allows arbitrary expressions, it is possible to use variable variables within the advanced syntax.

String access and modification by character

Characters within strings may be accessed and modified by specifying the zero-based offset of the desired character after the string using square array brackets, as in *$str[42]*. Think of a string as an array of characters for this purpose. The functions substr() and substr_replace() can be used when you want to extract or replace more than 1 character.

> **Note** :
>
> As of PHP 7.1.0, negative string offsets are also supported. These specify the offset from the end of the string. Formerly, negative offsets emitted **E_NOTICE** for reading (yielding an empty string) and **E_WARNING** for writing (leaving the string untouched).

> **Note** :
>
> Prior to PHP 8.0.0, strings could also be accessed using braces, as in *$str{42}*, for the same purpose. This curly brace syntax was deprecated as of PHP 7.4.0 and no longer supported as of PHP 8.0.0.

> **Warning**
>
> Writing to an out-of-range offset pads the string with spaces. Non-integer types are converted to an integer. An illegal offset type emits **E_WARNING**. Only the first character of an assigned string is used. As of

> PHP 7.1.0, assigning an empty string throws a fatal error. Formerly, it assigned a NULL byte.

> **Warning**
>
> Internally, PHP strings are byte arrays. As a result, accessing or modifying a string using array brackets is not multi-byte safe, and should only be done with strings that are in a single-byte encoding such as ISO-8859-1.

> **Note** :
>
> As of PHP 7.1.0, applying the empty index operator on an empty string throws a fatal error. Formerly, the empty string was silently converted to an array.

Example #20 Some string examples

```php
<?php
// Get the first character of a string
$str = 'This is a test.';
$first = $str[0];
var_dump($first);

// Get the third character of a string
$third = $str[2];
var_dump($third);

// Get the last character of a string.
$str = 'This is still a test.';
$last = $str[strlen($str)-1];
var_dump($last);

// Modify the last character of a string
$str = 'Look at the sea';
$str[strlen($str)-1] = 'e';
var_dump($str);
?>
```

String offsets must be either integers or integer-like strings. As of PHP 8.0.0, any other string throws a TypeError, except for a string starting with an integer followed by other characters, which emits `E_WARNING` and is interpreted as that leading integer. Prior to PHP 8.0.0, no TypeError was thrown: an illegal offset emitted `E_WARNING` instead.

Example #21 Example of Illegal String Offsets

```php
<?php
$str = 'abc';

$keys = [ '1', '1.0', 'x', '1x' ];

foreach ($keys as $keyToTry) {
    var_dump(isset($str[$keyToTry]));

    try {
```

```
        var_dump($str[$keyToTry]);
    } catch (TypeError $e) {
        echo $e->getMessage(), PHP_EOL;
    }

    echo PHP_EOL;
}
?>
```

Output of the above example in PHP 8:

```
bool(true)
string(1) "b"

bool(false)
Cannot access offset of type string on string

bool(false)
Cannot access offset of type string on string

bool(false)

Warning: Illegal string offset "1x" in example.php on line 10
string(1) "b"
```

Output of the above example in PHP 7:

```
bool(true)
string(1) "b"

bool(false)

Warning: Illegal string offset '1.0' in example.php on line 10
string(1) "b"

bool(false)

Warning: Illegal string offset 'x' in example.php on line 10
string(1) "a"

bool(false)

Notice: A non well formed numeric value encountered in example.php on line 10
string(1) "b"
```

> **Note** :

> Accessing variables of other types (not including arrays or objects implementing the appropriate interfaces) using `[]` or `{}` silently returns **`null`**.

> **Note** :
>
> Characters within string literals can be accessed using `[]` or `{}`.

> **Note** :
>
> Accessing characters within string literals using the `{}` syntax has been deprecated in PHP 7.4. This has been removed in PHP 8.0.

Useful functions and operators

Strings may be concatenated using the '.' (dot) operator. Note that the '+' (addition) operator will *not* work for this. See String operators for more information.

There are a number of useful functions for string manipulation.

See the string functions section for general functions, and the Perl-compatible regular expression functions for advanced find & replace functionality.

There are also functions for URL strings, and functions to encrypt/decrypt strings (Sodium and Hash).

Finally, see also the character type functions.

Converting to string

A value can be converted to a string using the `(string)` cast or the strval() function. String conversion is automatically done in the scope of an expression where a string is needed. This happens when using the echo or print functions, or when a variable is compared to a string. The sections on Types and Type Juggling will make the following clearer. See also the settype() function.

A bool **`true`** value is converted to the string `"1"`. bool **`false`** is converted to `""` (the empty string). This allows conversion back and forth between bool and string values.

An int or float is converted to a string representing the number textually (including the exponent part for floats). Floating-point numbers can be converted using exponential notation (`4.1E+6`).

> **Note** :
>
> As of PHP 8.0.0, the decimal point character is always a period (`"."`). Prior to PHP 8.0.0, the decimal point character is defined in the script's locale (category LC_NUMERIC). See the setlocale() function.

Arrays are always converted to the string `"Array"`; because of this, echo and print can not by themselves show the contents of an array. To view a single element, use a construction such as `echo $arr['foo']`. See below for tips on viewing the entire contents.

In order to convert objects to string, the magic method __toString must be used.

Resources are always converted to strings with the structure `"Resource id #1"`, where `1` is the resource number assigned to the resource by PHP at runtime. While the exact structure of this string should not be relied on and is subject to change, it will always be unique for a given resource within the lifetime of a script being executed (ie a Web request or CLI process) and won't be reused. To get a resource's type, use the get_resource_type() function.

`null` is always converted to an empty string.

As stated above, directly converting an array, object, or resource to a string does not provide any useful information about the value beyond its type. See the functions print_r() and var_dump() for more effective means of inspecting the contents of these types.

Most PHP values can also be converted to strings for permanent storage. This method is called serialization, and is performed by the serialize() function.

Details of the String Type

The string in PHP is implemented as an array of bytes and an integer indicating the length of the buffer. It has no information about how those bytes translate to characters, leaving that task to the programmer. There are no limitations on the values the string can be composed of; in particular, bytes with value `0` ("NUL bytes") are allowed anywhere in the string. However, a few functions, said in this manual not to be "binary safe", may hand off the strings to libraries that ignore data after a NUL byte.

This nature of the string type explains why there is no separate "byte" type in PHP – strings take this role. Functions that return no textual data – for instance, arbitrary data read from a network socket – will still return strings.

Given that PHP does not dictate a specific encoding for strings, one might wonder how string literals are encoded. For instance, is the string `"á"` equivalent to `"\xE1"` (ISO-8859-1), `"\xC3\xA1"` (UTF-8, C form), `"\x61\xCC\x81"` (UTF-8, D form) or any other possible representation? The answer is that the string will be encoded in whatever fashion it is encoded in the script file. Thus, if the script is written in ISO-8859-1, the string will be encoded in ISO-8859-1 and so on. However, this does not apply if Zend Multibyte is enabled; in that case, the script may be written in an arbitrary encoding (which is explicitly declared or is detected) and then converted to a certain internal encoding, which is then the encoding that will be used for the string literals. Note that there are some constraints on the encoding of the script (or on the internal encoding, should Zend Multibyte be enabled) – this almost always means that this encoding should be a compatible superset of ASCII, such as UTF-8 or ISO-8859-1. Note, however, that state-dependent encodings where the same byte values can be used in initial and non-initial shift states may be problematic.

Of course, in order to be useful, functions that operate on text may have to make some assumptions about how the string is encoded. Unfortunately, there is much variation on this matter throughout PHP's functions:

- Some functions assume that the string is encoded in some (any) single-byte encoding, but they do not need to interpret those bytes as specific characters. This is the case of, for instance, substr(), strpos(), strlen() or strcmp(). Another way to think of these functions is that they operate on memory buffers, i.e., they work with bytes and byte offsets.
- Other functions receive the encoding of the string as a parameter or assume a default when it's not provided. This is the case of htmlentities() and the majority of the functions in the mbstring extension.
- Others use the current locale (see setlocale()), but operate byte-by-byte.

- Finally, they may just assume the string is using a specific encoding, usually UTF-8. This is the case of most functions in the intl extension and in the PCRE extension (in the last case, only when the `u` modifier is used).

Ultimately, this means writing correct programs using Unicode depends on carefully avoiding functions that will not work and that most likely will corrupt the data and using instead the functions that do behave correctly, generally from the intl and mbstring extensions. However, using functions that can handle Unicode encodings is just the beginning. No matter the functions the language provides, it is essential to know the Unicode specification. For instance, a program that assumes there is only uppercase and lowercase is making a wrong assumption.

Numeric strings

A PHP string is considered numeric if it can be interpreted as an int or a float.

Formally as of PHP 8.0.0:

```
WHITESPACES      \s*
LNUM             [0-9]+
DNUM             ([0-9]*[\.]{LNUM}) | ({LNUM}[\.][0-9]*)
EXPONENT_DNUM    (({LNUM} | {DNUM}) [eE][+-]? {LNUM})
INT_NUM_STRING   {WHITESPACES} [+-]? {LNUM} {WHITESPACES}
FLOAT_NUM_STRING {WHITESPACES} [+-]? ({DNUM} | {EXPONENT_DNUM}) {WHITESPACES}
NUM_STRING       ({INT_NUM_STRING} | {FLOAT_NUM_STRING})
```

PHP also has a concept of *leading* numeric strings. This is simply a string which starts like a numeric string followed by any characters.

> **Note** :
>
> Any string that contains the letter `E` (case insensitive) bounded by numbers will be seen as a number expressed in scientific notation. This can produce unexpected results.
>
> **Example #1 Scientific Notation Comparisons**
>
> ```php
> <?php
> var_dump("0D1" == "000"); // false, "0D1" is not scientific notation
> var_dump("0E1" == "000"); // true, "0E1" is 0 * (10 ^ 1), or 0
> var_dump("2E1" == "020"); // true, "2E1" is 2 * (10 ^ 1), or 20
> ?>
> ```

Strings used in numeric contexts

When a string needs to be evaluated as a number (e.g. arithmetic operations, int type declaration, etc.) the following steps are taken to determine the outcome:

1. If the string is numeric, resolve to an int if the string is an integer numeric string and fits into the limits of the int type limits (as defined by **`PHP_INT_MAX`**), otherwise resolve to a float.
2. If the context allows leading numeric strings and the string is one, resolve to an int if the leading part of the string is an integer numeric string and fits into the limits of the int type limits (as defined by **`PHP_INT_MAX`**), otherwise resolve to a float. Additionally an error of level **`E_WARNING`** is raised.

3. If the string is not numeric, throw a TypeError.

Behavior prior to PHP 8.0.0

Prior to PHP 8.0.0, a string was considered numeric if it only had *leading* whitespaces. If it had *trailing* whitespaces then the string was considered to be leading numeric.

Prior to PHP 8.0.0, when a string was used in a numeric context it would perform the same steps as above with the following differences:

- The usage of a leading numeric string would raise an `E_NOTICE` instead of an `E_WARNING`.
- If the string was not numeric, an `E_WARNING` was raised and the value `0` would be returned.

Prior to PHP 7.1.0, neither `E_NOTICE` nor `E_WARNING` was raised.

```
<?php
$foo = 1 + "10.5";                // $foo is float (11.5)
$foo = 1 + "-1.3e3";              // $foo is float (-1299)
$foo = 1 + "bob-1.3e3";           // TypeError as of PHP 8.0.0, $foo is integer (1) previously
$foo = 1 + "bob3";                // TypeError as of PHP 8.0.0, $foo is integer (1) previously
$foo = 1 + "10 Small Pigs";       // $foo is integer (11) and an E_WARNING is raised in PHP 8.0.0,
E_NOTICE previously
$foo = 4 + "10.2 Little Piggies"; // $foo is float (14.2) and an E_WARNING is raised in PHP 8.0.0,
E_NOTICE previously
$foo = "10.0 pigs " + 1;          // $foo is float (11) and an E_WARNING is raised in PHP 8.0.0,
E_NOTICE previously
$foo = "10.0 pigs " + 1.0;        // $foo is float (11) and an E_WARNING is raised in PHP 8.0.0,
E_NOTICE previously
?>
```

Arrays

For a list of all array functions, see the array functions chapter in the documentation.

An array in PHP is actually an ordered map. A map is a type that associates *values* to *keys*. This type is optimized for several different uses; it can be treated as an array, list (vector), hash table (an implementation of a map), dictionary, collection, stack, queue, and probably more. As array values can be other arrays, trees and multidimensional arrays are also possible.

Explanation of those data structures is beyond the scope of this manual, but at least one example is provided for each of them. For more information, look towards the considerable literature that exists about this broad topic.

Syntax

Specifying with array()

An array can be created using the array() language construct. It takes any number of comma-separated `key => value` pairs as arguments.

```
array(
    key  => value,
    key2 => value2,
    key3 => value3,
```

```
    ...
)
```

The comma after the last array element is optional and can be omitted. This is usually done for single-line arrays, i.e. `array(1, 2)` is preferred over `array(1, 2, )`. For multi-line arrays on the other hand the trailing comma is commonly used, as it allows easier addition of new elements at the end.

> **Note** :
>
> A short array syntax exists which replaces `array()` with `[]`.

Example #1 A simple array

```php
<?php
$array1 = array(
    "foo" => "bar",
    "bar" => "foo",
);

// Using the short array syntax
$array2 = [
    "foo" => "bar",
    "bar" => "foo",
];

var_dump($array1, $array2);
?>
```

The key can either be an int or a string. The value can be of any type.

Additionally the following key casts will occur:

- Strings containing valid decimal ints, unless the number is preceded by a `+` sign, will be cast to the int type. E.g. the key `"8"` will actually be stored under `8`. On the other hand `"08"` will not be cast, as it isn't a valid decimal integer.
- Floats are also cast to ints, which means that the fractional part will be truncated. E.g. the key `8.7` will actually be stored under `8`.
- Bools are cast to ints, too, i.e. the key **`true`** will actually be stored under `1` and the key **`false`** under `0`.
- Null will be cast to the empty string, i.e. the key **`null`** will actually be stored under `""`.
- Arrays and objects *can not* be used as keys. Doing so will result in a warning: `Illegal offset type`.

If multiple elements in the array declaration use the same key, only the last one will be used as all others are overwritten.

Example #2 Type Casting and Overwriting example

```php
<?php
$array = array(
    1    => "a",
    "1"  => "b",
```

```php
    1.5  => "c",
    true => "d",
);
var_dump($array);
?>
```

The above example will output:

```
array(1) {
  [1]=>
  string(1) "d"
}
```

As all the keys in the above example are cast to `1`, the value will be overwritten on every new element and the last assigned value `"d"` is the only one left over.

PHP arrays can contain int and string keys at the same time as PHP does not distinguish between indexed and associative arrays.

Example #3 Mixed int and string keys

```php
<?php
$array = array(
    "foo" => "bar",
    "bar" => "foo",
    100   => -100,
    -100  => 100,
);
var_dump($array);
?>
```

The above example will output:

```
array(4) {
  ["foo"]=>
  string(3) "bar"
  ["bar"]=>
  string(3) "foo"
  [100]=>
  int(-100)
  [-100]=>
  int(100)
}
```

The key is optional. If it is not specified, PHP will use the increment of the largest previously used int key.

Example #4 Indexed arrays without key

```php
<?php
$array = array("foo", "bar", "hello", "world");
var_dump($array);
?>
```

The above example will output:

```
array(4) {
  [0]=>
  string(3) "foo"
  [1]=>
  string(3) "bar"
  [2]=>
  string(5) "hello"
  [3]=>
  string(5) "world"
}
```

It is possible to specify the key only for some elements and leave it out for others:

Example #5 Keys not on all elements

```php
<?php
$array = array(
         "a",
         "b",
    6 => "c",
         "d",
);
var_dump($array);
?>
```

The above example will output:

```
array(4) {
  [0]=>
  string(1) "a"
  [1]=>
  string(1) "b"
  [6]=>
  string(1) "c"
  [7]=>
  string(1) "d"
}
```

As you can see the last value `"d"` was assigned the key `7`. This is because the largest integer key before that was `6`.

Example #6 Complex Type Casting and Overwriting example

This example includes all variations of type casting of keys and overwriting of elements.

```php
<?php
$array = array(
    1    => 'a',
    '1'  => 'b', // the value "a" will be overwritten by "b"
    1.5  => 'c', // the value "b" will be overwritten by "c"
    -1 => 'd',
    '01'  => 'e', // as this is not an integer string it will NOT override the key for 1
    '1.5' => 'f', // as this is not an integer string it will NOT override the key for 1
    true => 'g', // the value "c" will be overwritten by "g"
    false => 'h',
    '' => 'i',
    null => 'j', // the value "i" will be overwritten by "j"
    'k', // value "k" is assigned the key 2. This is because the largest integer key before that was
1
    2 => 'l', // the value "k" will be overwritten by "l"
);

var_dump($array);
?>
```

The above example will output:

```
array(7) {
  [1]=>
  string(1) "g"
  [-1]=>
  string(1) "d"
  ["01"]=>
  string(1) "e"
  ["1.5"]=>
  string(1) "f"
  [0]=>
  string(1) "h"
  [""]=>
  string(1) "j"
  [2]=>
  string(1) "l"
}
```

Example #7 Negative index example

When assigning a negative integer key `n`, PHP will take care to assign the next key to `n+1`.

```php
<?php
$array = [];

$array[-5] = 1;
$array[] = 2;

var_dump($array);
?>
```

The above example will output:

```
array(2) {
  [-5]=>
  int(1)
  [-4]=>
  int(2)
}
```

> **Warning**
>
> Prior to PHP 8.3.0, assigning a negative integer key `n` would assign the next key to `0`, the previous example would therefore output:
>
> ```
> array(2) {
>   [-5]=>
>   int(1)
>   [0]=>
>   int(2)
> }
> ```

Accessing array elements with square bracket syntax

Array elements can be accessed using the `array[key]` syntax.

Example #8 Accessing array elements

```php
<?php
$array = array(
    "foo" => "bar",
    42    => 24,
    "multi" => array(
         "dimensional" => array(
             "array" => "foo"
         )
    )
);
```

```php
var_dump($array["foo"]);
var_dump($array[42]);
var_dump($array["multi"]["dimensional"]["array"]);
?>
```

The above example will output:

```
string(3) "bar"
int(24)
string(3) "foo"
```

> **Note** :
>
> Prior to PHP 8.0.0, square brackets and curly braces could be used interchangeably for accessing array elements (e.g. `$array[42]` and `$array{42}` would both do the same thing in the example above). The curly brace syntax was deprecated as of PHP 7.4.0 and no longer supported as of PHP 8.0.0.

Example #9 Array dereferencing

```php
<?php
function getArray() {
    return array(1, 2, 3);
}

$secondElement = getArray()[1];

var_dump($secondElement);
?>
```

> **Note** :
>
> Attempting to access an array key which has not been defined is the same as accessing any other undefined variable: an **E_WARNING**-level error message (**E_NOTICE**-level prior to PHP 8.0.0) will be issued, and the result will be **null**.

> **Note** :
>
> Array dereferencing a scalar value which is not a string yields **null**. Prior to PHP 7.4.0, that did not issue an error message. As of PHP 7.4.0, this issues **E_NOTICE**; as of PHP 8.0.0, this issues **E_WARNING**.

Creating/modifying with square bracket syntax

An existing array can be modified by explicitly setting values in it.

This is done by assigning values to the array, specifying the key in brackets. The key can also be omitted, resulting in an empty pair of brackets ([]).

```
$arr[key] = value;
$arr[] = value;
// key may be an int or string
// value may be any value of any type
```

If *$arr* doesn't exist yet or is set to **null** or **false**, it will be created, so this is also an alternative way to create an array. This practice is however discouraged because if *$arr* already contains some value (e.g. string from request variable) then this value will stay in place and [] may actually stand for string access operator. It is always better to initialize a variable by a direct assignment.

> **Note** :
>
> As of PHP 7.1.0, applying the empty index operator on a string throws a fatal error. Formerly, the string was silently converted to an array.

> **Note** :
>
> As of PHP 8.1.0, creating a new array from **false** value is deprecated. Creating a new array from **null** and undefined values is still allowed.

To change a certain value, assign a new value to that element using its key. To remove a key/value pair, call the unset() function on it.

Example #10 Using Square Brackets with Arrays

```
<?php
$arr = array(5 => 1, 12 => 2);

$arr[] = 56;    // This is the same as $arr[13] = 56;
                // at this point of the script

$arr["x"] = 42; // This adds a new element to
                // the array with key "x"

unset($arr[5]); // This removes the element from the array

var_dump($arr);

unset($arr);    // This deletes the whole array

var_dump($arr);
?>
```

> **Note** :
>
> If no key is specified, the key will be the maximum existing integer index + 1. If the array has no positive integer indices, the key will be 0. As of PHP 8.3.0, it can also be a negative integer.

Note that the maximum integer key used for this *need not currently exist in the array*. It need only have existed in the array at some time since the last time the array was re-indexed. The following example illustrates:

```php
<?php
// Create a simple array.
$array = array(1, 2, 3, 4, 5);
print_r($array);

// Now delete every item, but leave the array itself intact:
foreach ($array as $i => $value) {
    unset($array[$i]);
}
print_r($array);

// Append an item (note that the new key is 5, instead of 0).
$array[] = 6;
print_r($array);

// Re-index:
$array = array_values($array);
$array[] = 7;
print_r($array);
?>
```

The above example will output:

```
Array
(
    [0] => 1
    [1] => 2
    [2] => 3
    [3] => 4
    [4] => 5
)
Array
(
)
Array
(
    [5] => 6
)
Array
(
    [0] => 6
    [1] => 7
)
```

Array destructuring

Arrays can be destructured using the `[]` (as of PHP 7.1.0) or list() language constructs. These constructs can be used to destructure an array into distinct variables.

Example #11 Array Destructuring

```php
<?php
$source_array = ['foo', 'bar', 'baz'];

[$foo, $bar, $baz] = $source_array;

echo $foo, PHP_EOL;    // prints "foo"
echo $bar, PHP_EOL;    // prints "bar"
echo $baz, PHP_EOL;    // prints "baz"
?>
```

Array destructuring can be used in `foreach` to destructure a multi-dimensional array while iterating over it.

Example #12 Array Destructuring in Foreach

```php
<?php
$source_array = [
    [1, 'John'],
    [2, 'Jane'],
];

foreach ($source_array as [$id, $name]) {
    echo "{$id}: '{$name}'\n";
}
?>
```

Array elements will be ignored if the variable is not provided. Array destructuring always starts at index `0`.

Example #13 Ignoring Elements

```php
<?php
$source_array = ['foo', 'bar', 'baz'];

// Assign the element at index 2 to the variable $baz
[, , $baz] = $source_array;

echo $baz;    // prints "baz"
?>
```

As of PHP 7.1.0, associative arrays can be destructured too. This also allows for easier selection of the right element in numerically indexed arrays as the index can be explicitly specified.

Example #14 Destructuring Associative Arrays

```php
<?php
$source_array = ['foo' => 1, 'bar' => 2, 'baz' => 3];

// Assign the element at index 'baz' to the variable $three
['baz' => $three] = $source_array;

echo $three, PHP_EOL;  // prints 3
```

```php
$source_array = ['foo', 'bar', 'baz'];

// Assign the element at index 2 to the variable $baz
[2 => $baz] = $source_array;

echo $baz, PHP_EOL;    // prints "baz"
?>
```

Array destructuring can be used for easy swapping of two variables.

Example #15 Swapping Two Variable

```php
<?php
$a = 1;
$b = 2;

[$b, $a] = [$a, $b];

echo $a, PHP_EOL;    // prints 2
echo $b, PHP_EOL;    // prints 1
?>
```

> **Note** :
>
> The spread operator (`...`) is not supported in assignments.

> **Note** :
>
> Attempting to access an array key which has not been defined is the same as accessing any other undefined variable: an `E_WARNING`-level error message (`E_NOTICE`-level prior to PHP 8.0.0) will be issued, and the result will be `null`.

> **Note** :
>
> Destructuring a scalar value assigns `null` to all variables.

Useful functions

There are quite a few useful functions for working with arrays. See the array functions section.

> **Note** :
>
> The unset() function allows removing keys from an array. Be aware that the array will *not* be reindexed. If a true "remove and shift" behavior is desired, the array can be reindexed using the array_values() function.
>
> **Example #16 Unsetting Intermediate Elements**
>
> ```php
> <?php
> $a = array(1 => 'one', 2 => 'two', 3 => 'three');
```
```

```php
/* will produce an array that would have been defined as
   $a = array(1 => 'one', 3 => 'three');
   and NOT
   $a = array(1 => 'one', 2 =>'three');
*/
unset($a[2]);
var_dump($a);

$b = array_values($a);
// Now $b is array(0 => 'one', 1 =>'three')
var_dump($b);
?>
```

The `foreach` control structure exists specifically for arrays. It provides an easy way to traverse an array.

Array do's and don'ts

Why is `$foo[bar]` wrong?

Always use quotes around a string literal array index. For example, `$foo['bar']` is correct, while `$foo[bar]` is not. But why? It is common to encounter this kind of syntax in old scripts:

```php
<?php
$foo[bar] = 'enemy';
echo $foo[bar];
// etc
?>
```

This is wrong, but it works. The reason is that this code has an undefined constant (`bar`) rather than a string (`'bar'` - notice the quotes). It works because PHP automatically converts a *bare string* (an unquoted string which does not correspond to any known symbol) into a string which contains the bare string. For instance, if there is no defined constant named **`bar`**, then PHP will substitute in the string `'bar'` and use that.

> **Warning**
>
> The fallback to treat an undefined constant as bare string issues an error of level **`E_NOTICE`**. This has been deprecated as of PHP 7.2.0, and issues an error of level **`E_WARNING`**. As of PHP 8.0.0, it has been removed and throws an Error exception.

This does not mean to *always* quote the key. Do not quote keys which are constants or variables, as this will prevent PHP from interpreting them.

Example #17 Key Quoting

```php
<?php
error_reporting(E_ALL);
ini_set('display_errors', true);
ini_set('html_errors', false);

// Simple array:
$array = array(1, 2);
$count = count($array);
```

```php
for ($i = 0; $i < $count; $i++) {
    echo "\nChecking $i: \n";
    echo "Bad: " . $array['$i'] . "\n";
    echo "Good: " . $array[$i] . "\n";
    echo "Bad: {$array['$i']}\n";
    echo "Good: {$array[$i]}\n";
}
?>
```

The above example will output:

```
Checking 0:
Notice: Undefined index:  $i in /path/to/script.html on line 9
Bad:
Good: 1
Notice: Undefined index:  $i in /path/to/script.html on line 11
Bad:
Good: 1

Checking 1:
Notice: Undefined index:  $i in /path/to/script.html on line 9
Bad:
Good: 2
Notice: Undefined index:  $i in /path/to/script.html on line 11
Bad:
Good: 2
```

More examples to demonstrate this behaviour:

Example #18 More Examples

```php
<?php
// Show all errors
error_reporting(E_ALL);

$arr = array('fruit' => 'apple', 'veggie' => 'carrot');

// Correct
echo $arr['fruit'], PHP_EOL;  // apple
echo $arr['veggie'], PHP_EOL; // carrot

// Incorrect. This does not work and throws a PHP Error because
// of an undefined constant named fruit
//
// Error: Undefined constant "fruit"
try {
    echo $arr[fruit];
} catch (Error $e) {
    echo get_class($e), ': ', $e->getMessage(), PHP_EOL;
}

// This defines a constant to demonstrate what's going on.  The value 'veggie'
```

```php
// is assigned to a constant named fruit.
define('fruit', 'veggie');

// Notice the difference now
echo $arr['fruit'], PHP_EOL;  // apple
echo $arr[fruit], PHP_EOL;    // carrot

// The following is okay, as it's inside a string. Constants are not looked for
// within strings, so no error occurs here
echo "Hello $arr[fruit]", PHP_EOL;      // Hello apple

// With one exception: braces surrounding arrays within strings allows constants
// to be interpreted
echo "Hello {$arr[fruit]}", PHP_EOL;    // Hello carrot
echo "Hello {$arr['fruit']}", PHP_EOL;  // Hello apple

// Concatenation is another option
echo "Hello " . $arr['fruit'], PHP_EOL; // Hello apple
?>
```

```php
<?php
// This will not work, and will result in a parse error, such as:
// Parse error: parse error, expecting T_STRING' or T_VARIABLE' or T_NUM_STRING'
// This of course applies to using superglobals in strings as well
print "Hello $arr['fruit']";
print "Hello $_GET['foo']";
?>
```

As stated in the syntax section, what's inside the square brackets ('`[`' and '`]`') must be an expression. This means that code like this works:

```php
<?php
echo $arr[somefunc($bar)];
?>
```

This is an example of using a function return value as the array index. PHP also knows about constants:

```php
<?php
$error_descriptions[E_ERROR]   = "A fatal error has occurred";
$error_descriptions[E_WARNING] = "PHP issued a warning";
$error_descriptions[E_NOTICE]  = "This is just an informal notice";
?>
```

Note that `E_ERROR` is also a valid identifier, just like `bar` in the first example. But the last example is in fact the same as writing:

```php
<?php
$error_descriptions[1] = "A fatal error has occurred";
$error_descriptions[2] = "PHP issued a warning";
$error_descriptions[8] = "This is just an informal notice";
?>
```

because `E_ERROR` equals `1`, etc.

So why is it bad then?

At some point in the future, the PHP team might want to add another constant or keyword, or a constant in other code may interfere. For example, it is already wrong to use the words `empty` and `default` this way, since they are reserved keywords.

> **Note** :
>
> To reiterate, inside a double-quoted string, it's valid to not surround array indexes with quotes so `"$foo[bar]"` is valid. See the above examples for details on why as well as the section on variable parsing in strings.

Converting to array

For any of the types int, float, string, bool and resource, converting a value to an array results in an array with a single element with index zero and the value of the scalar which was converted. In other words, `(array) $scalarValue` is exactly the same as `array($scalarValue)`.

If an object is converted to an array, the result is an array whose elements are the object's properties. The keys are the member variable names, with a few notable exceptions: integer properties are inaccessible; private variables have the class name prepended to the variable name; protected variables have a '*' prepended to the variable name. These prepended values have `NUL` bytes on either side. Uninitialized typed properties are silently discarded.

Example #19 Converting to an Array

```php
<?php

class A {
    private $B;
    protected $C;
    public $D;
    function __construct()
    {
        $this->{1} = null;
    }
}

var_export((array) new A());
?>
```

The above example will output:

```
array (
  '' . "\0" . 'A' . "\0" . 'B' => NULL,
  '' . "\0" . '*' . "\0" . 'C' => NULL,
  'D' => NULL,
  1 => NULL,
)
```

These NUL can result in some unexpected behaviour:

Example #20 Casting an Object to an Array

```php
<?php

class A {
    private $A; // This will become '\0A\0A'
}

class B extends A {
    private $A; // This will become '\0B\0A'
    public $AA; // This will become 'AA'
}

var_dump((array) new B());
?>
```

Output of the above example in PHP 8.0:

```
array(3) {
  ["BA"]=>
  NULL
  ["AA"]=>
  NULL
  ["AA"]=>
  NULL
}
```

Output of the above example in PHP 8.1:

```
array(3) {
  ["AA"]=>
  NULL
  ["BA"]=>
  NULL
  ["AA"]=>
  NULL
}
```

The above will appear to have two keys named 'AA', although one of them is actually named '\0A\0A'.

Converting **null** to an array results in an empty array.

Comparing

It is possible to compare arrays with the array_diff() function and with array operators.

Array unpacking

An array prefixed by `...` will be expanded in place during array definition. Only arrays and objects which implement Traversable can be expanded. Array unpacking with `...` is available as of PHP 7.4.0. This is also called the spread operator.

It's possible to expand multiple times, and add normal elements before or after the `...` operator:

Example #21 Simple array unpacking

```php
<?php
// Using short array syntax.
// Also, works with array() syntax.
$arr1 = [1, 2, 3];
$arr2 = [...$arr1]; // [1, 2, 3]
$arr3 = [0, ...$arr1]; // [0, 1, 2, 3]
$arr4 = [...$arr1, ...$arr2, 111]; // [1, 2, 3, 1, 2, 3, 111]
$arr5 = [...$arr1, ...$arr1]; // [1, 2, 3, 1, 2, 3]

function getArr() {
  return ['a', 'b'];
}
$arr6 = [...getArr(), 'c' => 'd']; // ['a', 'b', 'c' => 'd']

var_dump($arr1, $arr2, $arr3, $arr4, $arr5, $arr6);
?>
```

Unpacking an array with the `...` operator follows the semantics of the array_merge() function. That is, later string keys overwrite earlier ones and integer keys are renumbered:

Example #22 Array unpacking with duplicate key

```php
<?php
// string key
$arr1 = ["a" => 1];
$arr2 = ["a" => 2];
$arr3 = ["a" => 0, ...$arr1, ...$arr2];
var_dump($arr3); // ["a" => 2]

// integer key
$arr4 = [1, 2, 3];
$arr5 = [4, 5, 6];
$arr6 = [...$arr4, ...$arr5];
var_dump($arr6); // [1, 2, 3, 4, 5, 6]
// Which is [0 => 1, 1 => 2, 2 => 3, 3 => 4, 4 => 5, 5 => 6]
// where the original integer keys have not been retained.
?>
```

> **Note** :
>
> Keys that are neither integers nor strings throw a TypeError. Such keys can only be generated by a Traversable object.

> **Note** :
>
> Prior to PHP 8.1, unpacking an array which has a string key is not supported:

```php
<?php

$arr1 = [1, 2, 3];
$arr2 = ['a' => 4];
$arr3 = [...$arr1, ...$arr2];
// Fatal error: Uncaught Error: Cannot unpack array with string keys in example.php:5

$arr4 = [1, 2, 3];
$arr5 = [4, 5];
$arr6 = [...$arr4, ...$arr5]; // works. [1, 2, 3, 4, 5]
?>
```

Examples

The array type in PHP is very versatile. Here are some examples:

Example #23 Array Versatility

```php
<?php
// This:
$a = array( 'color' => 'red',
            'taste' => 'sweet',
            'shape' => 'round',
            'name'  => 'apple',
            4        // key will be 0
          );

$b = array('a', 'b', 'c');

var_dump($a, $b);

// . . .is completely equivalent with this:
$a = array();
$a['color'] = 'red';
$a['taste'] = 'sweet';
$a['shape'] = 'round';
$a['name']  = 'apple';
$a[]        = 4;        // key will be 0

$b = array();
$b[] = 'a';
$b[] = 'b';
$b[] = 'c';

// After the above code is executed, $a will be the array
// array('color' => 'red', 'taste' => 'sweet', 'shape' => 'round',
// 'name' => 'apple', 0 => 4), and $b will be the array
// array(0 => 'a', 1 => 'b', 2 => 'c'), or simply array('a', 'b', 'c').

var_dump($a, $b);
?>
```

Example #24 Using array()

```php
<?php
// Array as (property-)map
$map = array( 'version'    => 4,
              'OS'         => 'Linux',
              'lang'       => 'english',
              'short_tags' => true
            );
var_dump($map);

// strictly numerical keys
// this is the same as array(0 => 7, 1 => 8, ...)
$array = array( 7,
                8,
                0,
                156,
                -10
              );
var_dump($array);

$switching = array(         10, // key = 0
                    5    =>  6,
                    3    =>  7,
                    'a'  =>  4,
                             11, // key = 6 (maximum of integer-indices was 5)
                    '8'  =>  2, // key = 8 (integer!)
                    '02' => 77, // key = '02'
                    0    => 12  // the value 10 will be overwritten by 12
                  );
var_dump($switching);

// empty array
$empty = array();
var_dump($empty);
?>
```

Example #25 Collection

```php
<?php
$colors = array('red', 'blue', 'green', 'yellow');

foreach ($colors as $color) {
    echo "Do you like $color?\n";
}

?>
```

The above example will output:

```
Do you like red?
Do you like blue?
```

```
Do you like green?
Do you like yellow?
```

Changing the values of the array directly is possible by passing them by reference.

Example #26 Changing element in the loop

```php
<?php
$colors = array('red', 'blue', 'green', 'yellow');

foreach ($colors as &$color) {
    $color = mb_strtoupper($color);
}
unset($color); /* ensure that following writes to
$color will not modify the last array element */

print_r($colors);
?>
```

The above example will output:

```
Array
(
    [0] => RED
    [1] => BLUE
    [2] => GREEN
    [3] => YELLOW
)
```

This example creates a one-based array.

Example #27 One-based index

```php
<?php
$firstquarter = array(1 => 'January', 'February', 'March');
print_r($firstquarter);
?>
```

The above example will output:

```
Array
(
    [1] => January
    [2] => February
    [3] => March
)
```

Example #28 Filling an array

```php
<?php
// fill an array with all items from a directory
$handle = opendir('.');
while (false !== ($file = readdir($handle))) {
    $files[] = $file;
}
closedir($handle);

var_dump($files);
?>
```

Arrays are ordered. The order can be changed using various sorting functions. See the array functions section for more information. The count() function can be used to count the number of items in an array.

Example #29 Sorting an array

```php
<?php
sort($files);
print_r($files);
?>
```

Because the value of an array can be anything, it can also be another array. This enables the creation of recursive and multi-dimensional arrays.

Example #30 Recursive and multi-dimensional arrays

```php
<?php
$fruits = array ( "fruits"  => array ( "a" => "orange",
                                       "b" => "banana",
                                       "c" => "apple"
                                     ),
                  "numbers" => array ( 1,
                                       2,
                                       3,
                                       4,
                                       5,
                                       6
                                     ),
                  "holes"   => array (      "first",
                                       5 => "second",
                                            "third"
                                     )
                );
var_dump($fruits);

// Some examples to address values in the array above
echo $fruits["holes"][5];    // prints "second"
echo $fruits["fruits"]["a"]; // prints "orange"
unset($fruits["holes"][0]);  // remove "first"

// Create a new multi-dimensional array
```

```
$juices["apple"]["green"] = "good";
var_dump($juices);
?>
```

Array assignment always involves value copying. Use the reference operator to copy an array by reference.

Example #31 Array Copying

```
<?php
$arr1 = array(2, 3);
$arr2 = $arr1;
$arr2[] = 4; // $arr2 is changed,
             // $arr1 is still array(2, 3)

$arr3 = &$arr1;
$arr3[] = 4; // now $arr1 and $arr3 are the same

var_dump($arr1, $arr2, $arr3);
?>
```

Objects

Object Initialization

To create a new object, use the `new` statement to instantiate a class:

Example #1 Object Construction

```
<?php
class foo
{
    function do_foo()
    {
        echo "Doing foo.";
    }
}

$bar = new foo;
$bar->do_foo();
?>
```

For a full discussion, see the Classes and Objects chapter.

Converting to object

If an object is converted to an object, it is not modified. If a value of any other type is converted to an object, a new instance of the stdClass built-in class is created. If the value was **`null`**, the new instance will be empty. An array converts to an object with properties named by keys and with their corresponding values. Prior to PHP 7.2.0, numeric keys would have been inaccessible unless iterated.

Example #2 Casting to an Object

```
<?php
$obj = (object) array('1' => 'foo');
var_dump(isset($obj->{'1'})); // outputs 'bool(true)'
```

```
// Deprecated as of PHP 8.1
var_dump(key($obj)); // outputs 'string(1) "1"'
?>
```

For any other value, a member variable named `scalar` will contain the value.

Example #3 `(object)` cast

```
<?php
$obj = (object) 'ciao';
echo $obj->scalar;  // outputs 'ciao'
?>
```

Enumerations

(PHP 8 >= 8.1.0)

Basic Enumerations

Enumerations are a restricting layer on top of classes and class constants, intended to provide a way to define a closed set of possible values for a type.

```
<?php
enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

function do_stuff(Suit $s)
{
    // ...
}

do_stuff(Suit::Spades);
?>
```

For a full discussion, see the Enumerations chapter.

Casting

If an enum is converted to an object, it is not modified. If an enum is converted to an array, an array with a single `name` key (for Pure enums) or an array with both `name` and `value` keys (for Backed enums) is created. All other cast types will result in an error.

Resources

A resource is a special variable, holding a reference to an external resource. Resources are created and used by special functions. See the appendix for a listing of all these functions and the corresponding resource types.

See also the get_resource_type() function.

Converting to resource

As resource variables hold special handles to opened files, database connections, image canvas areas and the like, converting to a resource makes no sense.

Freeing resources

Thanks to the reference-counting system being part of the Zend Engine, a resource with no more references to it is detected automatically, and it is freed by the garbage collector. For this reason, it is rarely necessary to free the memory manually.

> **Note** :
>
> Persistent database links are an exception to this rule. They are *not* destroyed by the garbage collector. See the persistent connections section for more information.

Callables

A callable is a reference to a function or method that is passed to another function as an argument. They are represented with the callable type declaration.

```php
<?php
function foo(callable $callback) {
    $callback();
}
?>
```

Some functions accept callback functions as a parameter, e.g. array_map(), usort(), or preg_replace_callback().

Creation of callables

A callable is a type that represents something that can be invoked. Callables can be passed as arguments to functions or methods which expect a callback parameter or they can be invoked directly. The callable type cannot be used as a type declaration for class properties. Instead, use a Closure type declaration.

Callables can be created in several different ways:

- Closure object
- string containing the name of a function or a method
- array containing a class name or an object in index 0 and the method name in index 1
- object implementing the __invoke() magic method

A Closure object can be created using anonymous function syntax, arrow function syntax, first-class callable syntax, or the Closure::fromCallable() method.

> **Note** :

> The first-class callable syntax is only available as of PHP 8.1.0.

Example #1 Callback example using a Closure

```php
<?php
// Using anonymous function syntax
$double1 = function ($a) {
    return $a * 2;
};

// Using first-class callable syntax
function double_function($a) {
    return $a * 2;
}
$double2 = double_function(...);

// Using arrow function syntax
$double3 = fn($a) => $a * 2;

// Using Closure::fromCallable
$double4 = Closure::fromCallable('double_function');

// Use the closure as a callback here to
// double the size of each element in our range
$new_numbers = array_map($double1, range(1, 5));
print implode(' ', $new_numbers) . PHP_EOL;

$new_numbers = array_map($double2, range(1, 5));
print implode(' ', $new_numbers) . PHP_EOL;

$new_numbers = array_map($double3, range(1, 5));
print implode(' ', $new_numbers) . PHP_EOL;

$new_numbers = array_map($double4, range(1, 5));
print implode(' ', $new_numbers);

?>
```

Output of the above example in PHP 8.1:

```
2 4 6 8 10
2 4 6 8 10
2 4 6 8 10
2 4 6 8 10
```

A callable can also be a string containing the name of a function or a static method. Any built-in or user-defined function can be used, except language constructs such as: array(), echo, empty(), eval(), isset(), list(), print or unset().

Static class methods can be used without instantiating an object of that class by either creating an array with the class name at index 0 and the method name at index 1, or by using the special syntax with the scope resolution operator `::`, as in `'ClassName::methodName'`.

A method of an instantiated object can be a callable when provided as an array with the object at index 0 and the method name at index 1.

The main difference between a Closure object and the callable type is that a Closure object is scope-independent and can always be invoked, whereas a callable type may be scope-dependent and may not be directly invoked. Closure is the preferred way to create callables. As of PHP 8.5.0, Closure is a proper subtype of callable.

> **Note** :
>
> While Closure objects are bound to the scope where they are created, callables referencing class methods as strings or arrays are resolved in the scope where they are called. To create a callable from a private or protected method, which can then be invoked from outside the class scope, use Closure::fromCallable() or the first-class callable syntax.

PHP allows the creation of callables which can be used as a callback argument but cannot be called directly. These are context-dependent callables which reference a class method in the inheritance hierarchy of a class, e.g. `'parent::method'` or `["static", "method"]`.

> **Note** :
>
> As of PHP 8.2.0, context-dependent callables are deprecated. Remove the context dependency by replacing `'parent::method'` with `parent::class . '::method'` or use the first-class callable syntax.

Example #2 Calling various types of callables with call_user_func()

```php
<?php

// An example callback function
function my_callback_function() {
    echo 'hello world!', PHP_EOL;
}

// An example callback method
class MyClass {
    static function myCallbackMethod() {
        echo 'Hello World!', PHP_EOL;
    }
}

// Type 1: Simple callback
call_user_func('my_callback_function');

// Type 2: Static class method call
call_user_func(['MyClass', 'myCallbackMethod']);

// Type 3: Object method call
```

```php
$obj = new MyClass();
call_user_func([$obj, 'myCallbackMethod']);

// Type 4: Static class method call
call_user_func('MyClass::myCallbackMethod');

// Type 5: Static class method call using ::class keyword
call_user_func([MyClass::class, 'myCallbackMethod']);

// Type 6: Relative static class method call
class A {
    public static function who() {
        echo 'A', PHP_EOL;
    }
}

class B extends A {
    public static function who() {
        echo 'B', PHP_EOL;
    }
}

call_user_func(['B', 'parent::who']); // deprecated as of PHP 8.2.0

// Type 7: Objects implementing __invoke can be used as callables
class C {
    public function __invoke($name) {
        echo 'Hello ', $name;
    }
}

$c = new C();
call_user_func($c, 'PHP!');
?>
```

The above example will output:

```
hello world!
Hello World!
Hello World!
Hello World!
Hello World!

Deprecated: Callables of the form ["B", "parent::who"] are deprecated in script on line 44
A
Hello PHP!
```

> **Note** :

> Callbacks registered with functions such as call_user_func() and call_user_func_array() will not be called if there is an uncaught exception thrown in a previous callback.

Mixed

The mixed type accepts every value. It is equivalent to the union type `object|resource|array|string|float|int|bool|null`. Available as of PHP 8.0.0.

mixed is, in type theory parlance, the top type, meaning every other type is a subtype of it.

Void

void is a return-only type declaration indicating the function does not return a value, but the function may still terminate. Therefore, it cannot be part of a union type declaration. Available as of PHP 7.1.0.

> **Note** :
>
> Even if a function has a return type of void, it will still return a value. This value is always `null`.

Discarding a value with `(void)`

The `(void)` syntax may be used to explicitly discard the result of an expression. This is useful to indicate that ignoring a return value is intentional, especially when calling a function or method marked with the NoDiscard attribute.

Unlike other casts, `(void)` does not convert the value to another type and does not produce a value. It is a statement and cannot be used as part of an expression.

Example #1 Discarding a return value

```php
<?php
#[\NoDiscard]
function process(): bool {
    return true;
}

(void) process(); // Explicitly discard the return value
?>
```

Never

never is a return-only type indicating the function does not terminate. This means that it either calls exit(), throws an exception, or is an infinite loop. Therefore, it cannot be part of a union type declaration. Available as of PHP 8.1.0.

never is, in type theory parlance, the bottom type, meaning it is the subtype of every other type and can replace any other return type during inheritance.

Relative class types

These type declarations can only be used within classes.

self

The value must be an `instanceof` the same class as the one in which the type declaration is used.

parent

The value must be an `instanceof` a parent of the class in which the type declaration is used.

static

static is a return-only type which requires that the value returned must be an `instanceof` the same class as the one the method is called in. Available as of PHP 8.0.0.

Singleton types

Singleton types are those which allow only one value. PHP has support for two singleton types: false as of PHP 8.0.0 and true as of PHP 8.2.0.

> **Warning**
>
> Prior to PHP 8.2.0, the false type could only be used as part of a union type.

> **Note** :
>
> It is not possible to define custom singleton types. Consider using enumerations instead.

Iterables

Iterable is a built-in compile-time type alias for `array|Traversable`. From its introduction in PHP 7.1.0 and prior to PHP 8.2.0, iterable was a built-in pseudo-type that acted as the aforementioned type alias and could be used as a type declaration. An iterable type can be used in `foreach` and with **yield from** within a generator.

> **Note** :
>
> Functions declaring iterable as a return type may also be generators.
>
> **Example #1 Iterable generator return type example**

```php
<?php

function gen(): iterable {
    yield 1;
    yield 2;
    yield 3;
}

foreach(gen() as $value) {
    echo $value, "\n";
}
?>
```

Type declarations

Type declarations can be added to function arguments, return values, as of PHP 7.4.0, class properties, and as of PHP 8.3.0, class constants. They ensure that the value is of the specified type at call time, otherwise a TypeError is thrown.

Every single type that PHP supports, with the exception of resource, can be used within a user-land type declaration. This page contains a changelog of availability of the different types and documentation about usage of them in type declarations.

> **Note** :
>
> When a class implements an interface method or reimplements a method which has already been defined by a parent class, it has to be compatible with the aforementioned definition. A method is compatible if it follows the variance rules.

Changelog

Version	Description
8.3.0	Support for class, interface, trait, and enum constant typing has been added.
8.2.0	Support for DNF types has been added.
8.2.0	Support for the literal type true has been added.
8.2.0	The types null and false can now be used standalone.
8.1.0	Support for intersection types has been added.
8.1.0	Returning by reference from a void function is now deprecated.
8.1.0	Support for the return only type never has been added.
8.0.0	Support for mixed has been added.
8.0.0	Support for the return only type static has been added.
8.0.0	Support for union types has been added.
7.4.0	Support for class properties typing has been added.
7.2.0	Support for object has been added.
7.1.0	Support for iterable has been added.
7.1.0	Support for void has been added.
7.1.0	Support for nullable types has been added.

Atomic Types Usage Notes

Atomic types have straightforward behaviour with some minor caveats which are described in this section.

Scalar types

> **Warning**
>
> Name aliases for scalar types (bool, int, float, string) are not supported. Instead, they are treated as class or interface names. For example, using `boolean` as a type declaration will require the value to be an

`instanceof` the class or interface `boolean`, rather than of type bool:

```
<?php
    function test(boolean $param) {}
    test(true);
?>
```

Output of the above example in PHP 8:

```
Warning: "boolean" will be interpreted as a class name. Did you mean "bool"? Write "\boolean"
to suppress this warning in /in/9YrUX on line 2

Fatal error: Uncaught TypeError: test(): Argument #1 ($param) must be of type boolean, bool
given, called in - on line 3 and defined in -:2
Stack trace:
#0 -(3): test(true)
#1 {main}
  thrown in - on line 2
```

void

> **Note** :
>
> Returning by reference from a void function is deprecated as of PHP 8.1.0, because such a function is contradictory. Previously, it already emitted the following `E_NOTICE` when called: Only variable references should be returned by reference.

```
<?php
function &test(): void {}
?>
```

Callable types

This type cannot be used as a class property type declaration.

> **Note** :
>
> It is not possible to specify the signature of the function.

Type declarations on pass-by-reference Parameters

If a pass-by-reference parameter has a type declaration, the type of the variable is *only* checked on function entry, at the beginning of the call, but not when the function returns. This means that a function can change the type of the variable passed by reference.

Example #1 Typed pass-by-reference Parameters

```php
<?php
function array_baz(array &$param)
{
    $param = 1;
}
$var = [];
array_baz($var);
var_dump($var);
array_baz($var);
?>
```

The above example will output something similar to:

```
int(1)

Fatal error: Uncaught TypeError: array_baz(): Argument #1 ($param) must be of type array, int
given, called in - on line 9 and defined in -:2
Stack trace:
#0 -(9): array_baz(1)
#1 {main}
  thrown in - on line 2
```

Composite Types Usage Notes

Composite type declarations are subject to a couple of restrictions and will perform a redundancy check at compile time to prevent simple bugs.

> **Caution**
>
> Prior to PHP 8.2.0, and the introduction of DNF types, it was not possible to combine intersection types with union types.

Union types

> **Warning**
>
> It is not possible to combine the two singleton types false and true together in a union type. Use bool instead.

> **Caution**
>
> Prior to PHP 8.2.0, as false and null could not be used as standalone types, a union type comprised of only these types was not permitted. This comprises the following types: false, `false|null`, and `?false`.

Nullable type syntactic sugar

A single base type declaration can be marked nullable by prefixing the type with a question mark (`?`). Thus `?T` and `T|null` are identical.

> **Note** :
>
> This syntax is supported as of PHP 7.1.0, and predates generalized union types support.

> **Note** :
>
> It is also possible to achieve nullable arguments by making `null` the default value. This is not recommended because if the default value is changed in a child class a type compatibility violation will be raised as the null type will need to be added to the type declaration. This behavior is also deprecated since PHP 8.4.
>
> **Example #2 Old way to make arguments nullable**
>
> ```php
> <?php
> class C {}
>
> function f(C $c = null) {
>     var_dump($c);
> }
>
> f(new C);
> f(null);
> ?>
> ```
>
> The above example will output:
>
> ```
> object(C)#1 (0) {
> }
> NULL
> ```

Duplicate and redundant types

To catch simple bugs in composite type declarations, redundant types that can be detected without performing class loading will result in a compile-time error. This includes:

- Each name-resolved type may only occur once. Types such as `int|string|INT` or `Countable&Traversable&COUNTABLE` result in an error.
- Using mixed or never results in an error.
- For union types:
  - If bool is used, false or true cannot be used additionally.
  - If object is used, class types cannot be used additionally.
  - If iterable is used, array and Traversable cannot be used additionally.

- For intersection types:
    - Using a type which is not a class-type results in an error.
    - Using either self, parent, or static results in an error.
- For DNF types:
    - If a more generic type is used, the more restrictive one is redundant.
    - Using two identical intersection types.

> **Note** :
>
> This does not guarantee that the type is "minimal", because doing so would require loading all used class types.

For example, if `A` and `B` are class aliases, then `A|B` remains a legal union type, even though it could be reduced to either `A` or `B`. Similarly, if class `B extends A {}`, then `A|B` is also a legal union type, even though it could be reduced to just `A`.

```
<?php
function foo(): int|INT {} // Disallowed
function foo(): bool|false {} // Disallowed
function foo(): int&Traversable {} // Disallowed
function foo(): self&Traversable {} // Disallowed

use A as B;
function foo(): A|B {} // Disallowed ("use" is part of name resolution)
function foo(): A&B {} // Disallowed ("use" is part of name resolution)

class_alias('X', 'Y');
function foo(): X|Y {} // Allowed (redundancy is only known at runtime)
function foo(): X&Y {} // Allowed (redundancy is only known at runtime)
?>
```

Examples

Example #3 Basic class type declaration

```
<?php
class C {}
class D extends C {}

// This doesn't extend C.
class E {}

function f(C $c) {
    echo get_class($c)."\n";
}

f(new C);
f(new D);
```

```
f(new E);
?>
```

Output of the above example in PHP 8:

```
C
D

Fatal error: Uncaught TypeError: f(): Argument #1 ($c) must be of type C, E given, called in
/in/gLonb on line 14 and defined in /in/gLonb:8
Stack trace:
#0 -(14): f(Object(E))
#1 {main}
  thrown in - on line 8
```

Example #4 Basic interface type declaration

```
<?php
interface I { public function f(); }
class C implements I { public function f() {} }

// This doesn't implement I.
class E {}

function f(I $i) {
    echo get_class($i)."\n";
}

f(new C);
f(new E);
?>
```

Output of the above example in PHP 8:

```
C

Fatal error: Uncaught TypeError: f(): Argument #1 ($i) must be of type I, E given, called in - on
line 13 and defined in -:8
Stack trace:
#0 -(13): f(Object(E))
#1 {main}
  thrown in - on line 8
```

Example #5 Basic return type declaration

```php
<?php
function sum($a, $b): float {
    return $a + $b;
}

// Note that a float will be returned.
var_dump(sum(1, 2));
?>
```

The above example will output:

```
float(3)
```

Example #6 Returning an object

```php
<?php
class C {}

function getC(): C {
    return new C;
}

var_dump(getC());
?>
```

The above example will output:

```
object(C)#1 (0) {
}
```

Example #7 Nullable argument type declaration

```php
<?php
class C {}

function f(?C $c) {
    var_dump($c);
}

f(new C);
f(null);
?>
```

The above example will output:

```
object(C)#1 (0) {
}
NULL
```

Example #8 Nullable return type declaration

```php
<?php
function get_item(): ?string {
    if (isset($_GET['item'])) {
        return $_GET['item'];
    } else {
        return null;
    }
}
?>
```

Example #9 Class property type declaration

```php
<?php
class User {
    public static string $foo = 'foo';

    public int $id;
    public string $username;

    public function __construct(int $id, string $username) {
        $this->id = $id;
        $this->username = $username;
    }
}
?>
```

Strict typing

By default, PHP will coerce values of the wrong type into the expected scalar type declaration if possible. For example, a function that is given an int for a parameter that expects a string will get a variable of type string.

It is possible to enable strict mode on a per-file basis. In strict mode, only a value corresponding exactly to the type declaration will be accepted, otherwise a TypeError will be thrown. The only exception to this rule is that an int value will pass a float type declaration.

> **Warning**
>
> Function calls from within internal functions will not be affected by the `strict_types` declaration.

To enable strict mode, the `declare` statement is used with the `strict_types` declaration:

> **Note** :

> Strict typing applies to function calls made from *within* the file with strict typing enabled, not to the functions declared within that file. If a file without strict typing enabled makes a call to a function that was defined in a file with strict typing, the caller's preference (coercive typing) will be respected, and the value will be coerced.

> **Note** :
>
> Strict typing is only defined for scalar type declarations.

Example #10 Strict typing for arguments values

```php
<?php
declare(strict_types=1);

function sum(int $a, int $b) {
    return $a + $b;
}

var_dump(sum(1, 2));
var_dump(sum(1.5, 2.5));
?>
```

Output of the above example in PHP 8:

```
int(3)

Fatal error: Uncaught TypeError: sum(): Argument #1 ($a) must be of type int, float given, called
in - on line 9 and defined in -:4
Stack trace:
#0 -(9): sum(1.5, 2.5)
#1 {main}
  thrown in - on line 4
```

Example #11 Coercive typing for argument values

```php
<?php
function sum(int $a, int $b) {
    return $a + $b;
}

var_dump(sum(1, 2));

// These will be coerced to integers: note the output below!
var_dump(sum(1.5, 2.5));
?>
```

The above example will output:

```
int(3)
int(3)
```

Example #12 Strict typing for return values

```php
<?php
declare(strict_types=1);

function sum($a, $b): int {
    return $a + $b;
}

var_dump(sum(1, 2));
var_dump(sum(1, 2.5));
?>
```

The above example will output:

```
int(3)

Fatal error: Uncaught TypeError: sum(): Return value must be of type int, float returned in -:5
Stack trace:
#0 -(9): sum(1, 2.5)
#1 {main}
  thrown in - on line 5
```

Type Juggling

PHP does not require explicit type definition in variable declaration. In this case, the type of a variable is determined by the value it stores. That is to say, if a string is assigned to variable *$var*, then *$var* is of type string. If afterwards an int value is assigned to *$var*, it will be of type int.

PHP may attempt to convert the type of a value to another automatically in certain contexts. The different contexts which exist are:

- Numeric
- String
- Logical
- Integral and string
- Comparative
- Function

> **Note** :
>
> When a value needs to be interpreted as a different type, the value itself does *not* change types.

To force a variable to be evaluated as a certain type, see the section on Type casting. To change the type of a variable, see the settype() function.

Numeric contexts

This is the context when using an arithmetical operator.

In this context, if either operand is a float (or not interpretable as an int), both operands are interpreted as floats, and the result will be a float. Otherwise, the operands will be interpreted as ints, and the result will also be an int. As of PHP 8.0.0, if one of the operands cannot be interpreted, a TypeError is thrown.

String contexts

This is the context when using echo, print, string interpolation, or the string concatenation operator.

In this context, the value will be interpreted as string. If the value cannot be interpreted, a TypeError is thrown. Prior to PHP 7.4.0, an `E_RECOVERABLE_ERROR` was raised.

Logical contexts

This is the context when using conditional statements, the ternary operator, or a logical operator.

In this context, the value will be interpreted as bool.

Integral and string contexts

This is the context when using bitwise operators.

In this context, if all operands are of type string, the result will also be a string. Otherwise, the operands will be interpreted as ints, and the result will also be an int. As of PHP 8.0.0, if one of the operands cannot be interpreted, a TypeError is thrown.

Comparative contexts

This is the context when using a comparison operator.

The type conversions which occur in this context are explained in the Comparison with Various Types table.

Function contexts

This is the context when a value is passed to a typed parameter, property, or returned from a function which declares a return type.

In this context, the value must be a value of the type. Two exceptions exist. The first one is: if the value is of type int and the declared type is float, then the integer is converted to a floating point number. The second one is: if the declared type is a *scalar* type, the value is convertible to a scalar type, and the coercive typing mode is active (the default), the value may be converted to an accepted scalar value. See below for a description of this behaviour.

> **Warning**
>
> Internal functions automatically coerce `null` to scalar types. This behaviour is *DEPRECATED* as of PHP 8.1.0.

Coercive typing with simple type declarations

- bool type declaration: value is interpreted as bool.
- int type declaration: value is interpreted as int if the conversion is well-defined. For example, the string is numeric.
- float type declaration: value is interpreted as float if the conversion is well-defined. For example, the string is numeric.
- string type declaration: value is interpreted as string.

Coercive typing with union types

When `strict_types` is not enabled, scalar type declarations are subject to limited implicit type coercions. If the exact type of the value is not part of the union, then the target type is chosen in the following order of preference:

1. int
2. float
3. string
4. bool

If the type exists in the union and the value can be coerced to the type under PHP's existing type-checking semantics, then the type is chosen. Otherwise, the next type is tried.

> **Caution**
>
> As an exception, if the value is a string and both int and float are part of the union, the preferred type is determined by the existing numeric string semantics. For example, for `"42"` int is chosen, while for `"42.0"` float is chosen.

> **Note** :
>
> Types that are not part of the above preference list are not eligible targets for implicit coercion. In particular, no implicit coercions to the null, false, and true types occur.

Example #1 Example of types being coerced into a type part of the union

```php
<?php
// int|string
42    --> 42          // exact type
"42"  --> "42"        // exact type
new ObjectWithToString --> "Result of __toString()"
                      // object never compatible with int, fall back to string
```

```
42.0  --> 42          // float compatible with int
42.1  --> 42          // float compatible with int
1e100 --> "1.0E+100"  // float too large for int type, fall back to string
INF   --> "INF"       // float too large for int type, fall back to string
true  --> 1           // bool compatible with int
[]    --> TypeError   // array not compatible with int or string

// int|float|bool
"45"    --> 45        // int numeric string
"45.0"  --> 45.0      // float numeric string

"45X"   --> true      // not numeric string, fall back to bool
""      --> false     // not numeric string, fall back to bool
"X"     --> true      // not numeric string, fall back to bool
[]      --> TypeError // array not compatible with int, float or bool
?>
```

Type Casting

Type casting converts the value to a chosen type by writing the type within parentheses before the value to convert.

Example #2 Type Casting

```
<?php
$foo = 10;          // $foo is an integer
$bar = (bool) $foo; // $bar is a boolean

var_dump($bar);
?>
```

The casts allowed are:

- `(int)` - cast to int
- `(bool)` - cast to bool
- `(float)` - cast to float
- `(string)` - cast to string
- `(array)` - cast to array
- `(object)` - cast to object
- `(unset)` - cast to NULL

The `(void)` cast is also available as of PHP 8.5.0, but it is not a value conversion. It is used as a statement to explicitly discard the result of an expression.

> **Warning**
>
> `(integer)` is an alias of the `(int)` cast. `(boolean)` is an alias of the `(bool)` cast. `(binary)` is an alias of the `(string)` cast. `(double)` and `(real)` are aliases of the `(float)` cast. These casts do not use the canonical type name and are deprecated as of PHP 8.5.0.

> **Warning**
>
> The `(real)` cast alias has been deprecated as of PHP 7.4.0 and removed as of PHP 8.0.0.

> **Warning**
>
> The `(unset)` cast has been deprecated as of PHP 7.2.0. Note that the `(unset)` cast is the same as assigning the value NULL to the variable or call. The `(unset)` cast is removed as of PHP 8.0.0.

> **Caution**
>
> The `(binary)` cast and `b` prefix exist for forward support. Currently `(binary)` and `(string)` are identical, however this may change and should not be relied upon.

> **Note** :
>
> Whitespaces are ignored within the parentheses of a cast. Therefore, the following two casts are equivalent:
>
> ```php
> <?php
> $foo = (int) $bar;
> $foo = ( int ) $bar;
> ?>
> ```

Casting literal strings and variables to binary strings:

```php
<?php
$binary = (binary) $string;
$binary = b"binary string";
?>
```

Instead of casting a variable to a string, it is also possible to enclose the variable in double quotes.

Example #3 Different Casting Mechanisms

```php
<?php
$foo = 10;            // $foo is an integer
$str = "$foo";        // $str is a string
$fst = (string) $foo; // $fst is also a string

// This prints out that "they are the same"
if ($fst === $str) {
    echo "they are the same", PHP_EOL;
}
?>
```

It may not be obvious exactly what will happen when casting between certain types. For more information, see these sections:

- Converting to boolean
- Converting to integer
- Converting to float
- Converting to string
- Converting to array
- Converting to object

- Converting to resource
- Converting to NULL
- Discarding a value with `(void)`
- The type comparison tables

> **Note** :
>
> Because PHP supports indexing into strings via offsets using the same syntax as array indexing, the following example holds true for all PHP versions:
>
> **Example #4 Using Array Offset with a String**
>
> ```php
> <?php
> $a    = 'car'; // $a is a string
> $a[0] = 'b';   // $a is still a string
> echo $a;       // bar
> ?>
> ```
>
> See the section titled String access by character for more information.

Variables

Table of Contents

Basics

Variables in PHP are represented by a dollar sign followed by the name of the variable. The variable name is case-sensitive.

A valid variable name starts with a letter (`A-Z`, `a-z`, or the bytes from 128 through 255) or underscore, followed by any number of letters, numbers, or underscores. As a regular expression, it would be expressed thus: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`

> **Note** :
>
> Variable names are compared as bytes, not as characters. PHP does not interpret or validate the encoding of the bytes from 128 through 255, and two names refer to the same variable only when their bytes are identical.
>
> One consequence is that names written in UTF-8 work, because every byte of a UTF-8 multi-byte sequence falls inside the accepted range. The same holds for any encoding whose non-ASCII bytes all fall in that range, and for byte sequences that are not valid text in any encoding. Encodings that use bytes below 128 within a multi-byte sequence, such as Shift-JIS or UTF-16, do not.
>
> Another is that two names which display identically may be distinct. `ä` encoded as U+00E4 and as `a` followed by the combining diaeresis U+0308 are different variables, as are a name with and without a trailing U+00A0 NO-BREAK SPACE.

> **Note** :
>
> `$this` is a special variable that can't be assigned. Prior to PHP 7.1.0, indirect assignment (e.g. by using variable variables) was possible.

> **Tip**
>
> See also the Userland Naming Guide.

Example #1 Valid variable names

```php
<?php
$var = 'Bob';
$Var = 'Joe';
echo "$var, $Var";      // outputs "Bob, Joe"

$_4site = 'not yet';    // valid; starts with an underscore
$täyte = 'mansikka';    // valid; 'ä' is (Extended) ASCII 228.
```

Example #2 Invalid variable names

```php
<?php
$4site = 'not yet';     // invalid; starts with a number
```

PHP accepts a sequence of any bytes as a variable name. Variable names that do not follow the above-mentioned naming rules can only be accessed dynamically at runtime. See variable variables for information on how to access them.

Example #3 Accessing obscure variable names

```php
<?php
${'invalid-name'} = 'bar';
$name = 'invalid-name';
echo ${'invalid-name'}, " ", $$name;
```

The above example will output:

```
bar bar
```

By default, variables are always assigned by value. That is to say, when an expression is assigned to a variable, the entire value of the original expression is copied into the destination variable. This means, for instance, that after assigning one variable's value to another, changing one of those variables will have no effect on the other. For more information on this kind of assignment, see the chapter on Expressions.

PHP also offers another way to assign values to variables: assign by reference. This means that the new variable simply references (in other words, "becomes an alias for" or "points to") the original variable. Changes

to the new variable affect the original, and vice versa.

To assign by reference, simply prepend an ampersand (&) to the beginning of the variable which is being assigned (the source variable). For instance, the following code snippet outputs '`My name is Bob`' twice:

```php
<?php
$foo = 'Bob';              // Assign the value 'Bob' to $foo
$bar = &$foo;              // Reference $foo via $bar.
$bar = "My name is $bar";  // Alter $bar...
echo $bar . PHP_EOL;
echo $foo . PHP_EOL;       // $foo is altered too.
```

One important thing to note is that only variables may be assigned by reference.

```php
<?php
$original = 25;
$ref1 = &$original;        // This is a valid assignment.
$ref2 = &(24 * 7);         // Invalid; references an unnamed expression.
```

```php
<?php
// Note the lack of & indicating a returned reference in the function declaration.
// i.e. the function does not return a reference, so the result cannot be assigned by reference
function test()
{
   $original = 25;
   return $original;
}

// Note: A notice is issued, but the non-reference value is assigned
$result1 = &test();        // Invalid because test() doesn't return a variable by reference.
var_dump($result1);

// This function is defined as returning a reference, but does not return a variable
function &test2()
{
    return 26;             // Invalid because the returned value isn't a variable reference.
}

// Note: The non-reference value is assigned
$result2 = &test2();
var_dump($result2);
```

It is not necessary to declare variables in PHP, however, it is a very good practice. Accessing an undefined variable will result in an **`E_WARNING`** (prior to PHP 8.0.0, **`E_NOTICE`**). An undefined variable has a default value of **`null`**. The isset() language construct can be used to detect if a variable has already been initialized.

Example #4 Default value of an uninitialized variable

```php
<?php
// Unset AND unreferenced (no use context) variable.
var_dump($unset_var);
```

The above example will output:

```
```

```
Warning: Undefined variable $unset_var in ...
NULL
```

PHP allows array autovivification (automatic creation of new arrays) from an undefined variable. Appending an element to an undefined variable will create a new array and will not generate a warning.

Example #5 Autovivification of an array from an undefined variable

```php
<?php
$unset_array[] = 'value'; // Does not generate a warning.
var_dump($unset_array);
```

> **Warning**
>
> Relying on the default value of an uninitialized variable is problematic when including one file in another which uses the same variable name.

A variable can be destroyed by using the unset() language construct.

For information on variable-related functions, see the Variable Functions Reference.

Predefined Variables

PHP provides a number of predefined variables. PHP also provides an additional set of predefined arrays containing variables from the web server (if applicable), the environment, and user input. These arrays are automatically available in every scope. For this reason, they are often known as "superglobals". (There is no mechanism in PHP for user-defined superglobals.) Refer to the list of superglobals for details.

> **Note** : **Variable variables**
>
> Superglobals cannot be used as variable variables inside functions or class methods.

If certain variables in variables_order are not set, their appropriate PHP predefined arrays are also left empty.

Variable scope

The scope of a variable is the context within which it is defined. PHP has a function scope and a global scope. Any variable defined outside a function is limited to the global scope. When a file is included, the code it contains inherits the variable scope of the line on which the include occurs.

Example #1 Example of global variable scope

```php
<?php
$a = 1;
include 'b.inc'; // Variable $a will be available within b.inc
```

Any variable created inside a named function or an anonymous function is limited to the scope of the function body. However, arrow functions bind variables from the parent scope to make them available inside the body.

If a file include occurs inside a function within the calling file, the variables contained in the called file will be available as if they had been defined inside the calling function.

Example #2 Example of local variable scope

```php
<?php
$a = 1; // global scope

function test()
{
    var_dump($a); // Variable $a is undefined as it refers to a local version of $a
}

test();
```

The example above will generate an undefined variable `E_WARNING` (or an `E_NOTICE` prior to PHP 8.0.0). This is because the echo statement refers to a local version of the *$a* variable, and it has not been assigned a value within this scope. Note that this is a little bit different from the C language in that global variables in C are automatically available to functions unless specifically overridden by a local definition. This can cause some problems in that people may inadvertently change a global variable. In PHP global variables must be declared global inside a function if they are going to be used in that function.

The `global` keyword

The `global` keyword is used to bind a variable from a global scope into a local scope. The keyword can be used with a list of variables or a single variable. A local variable will be created referencing the global variable of the same name. If the global variable does not exist, the variable will be created in global scope and assigned `null`.

Example #3 Using `global`

```php
<?php
$a = 1;
$b = 2;

function Sum()
{
    global $a, $b;

    $b = $a + $b;
}

Sum();
echo $b;
```

The above example will output:

```
3
```

By declaring *$a* and *$b* global within the function, all references to either variable will refer to the global version. There is no limit to the number of global variables that can be manipulated by a function.

A second way to access variables from the global scope is to use the special PHP-defined *$GLOBALS* array. The previous example can be rewritten as:

Example #4 Using *$GLOBALS* instead of global

```php
<?php
$a = 1;
$b = 2;

function Sum()
{
    $GLOBALS['b'] = $GLOBALS['a'] + $GLOBALS['b'];
}

Sum();
echo $b;
```

The *$GLOBALS* array is an associative array with the name of the global variable being the key and the contents of that variable being the value of the array element. Notice how *$GLOBALS* exists in any scope, this is because *$GLOBALS* is a superglobal. Here's an example demonstrating the power of superglobals:

Example #5 Example demonstrating superglobals and scope

```php
<?php
function test_superglobal()
{
    echo $_POST['name'];
}
```

> **Note** :
>
> Using `global` keyword outside a function is not an error. It can be used if the file is included from inside a function.

Using `static` variables

Another important feature of variable scoping is the *static* variable. A static variable exists only in a local function scope, but it does not lose its value when program execution leaves this scope. Consider the following example:

Example #6 Example demonstrating need for static variables

```php
<?php
function test()
{
    $a = 0;
    echo $a . PHP_EOL;
    $a++;
}
```

```
test();
test();
test();
```

This function is quite useless since every time it is called it sets *$a* to `0` and prints `0`. The *$a*++ which increments the variable serves no purpose since as soon as the function exits the *$a* variable disappears. To make a useful counting function which will not lose track of the current count, the *$a* variable is declared static:

Example #7 Example use of static variables

```
<?php
function test()
{
    static $a = 0;
    echo $a . PHP_EOL;
    $a++;
}

test();
test();
test();
```

Now, *$a* is initialized only in first call of function and every time the `test()` function is called it will print the value of *$a* and increment it.

Static variables also provide one way to deal with recursive functions. The following simple function recursively counts to 10, using the static variable *$count* to know when to stop:

Example #8 Static variables with recursive functions

```
<?php
function test()
{
    static $count = 0;

    $count++;
    echo $count . PHP_EOL;
    if ($count < 10) {
        test();
    }
    $count--;
}

test();
```

Prior to PHP 8.3.0, static variables could only be initialized using a constant expression. As of PHP 8.3.0, dynamic expressions (e.g. function calls) are also allowed:

Example #9 Declaring static variables

```
<?php
function foo(){
    static $int = 0;          // correct
    static $int = 1+2;        // correct
```

```
    static $int = sqrt(121);  // correct as of PHP 8.3.0

    $int++;
    echo $int;
}
```

Static variables inside anonymous functions persist only within that specific function instance. If the anonymous function is recreated on each call, the static variable will be reinitialized.

Example #10 Static variables in anonymous functions

```
<?php
function exampleFunction($input) {
    $result = (static function () use ($input) {
        static $counter = 0;
        $counter++;
        return "Input: $input, Counter: $counter\n";
    });

    return $result();
}

// Calls to exampleFunction will recreate the anonymous function, so the static
// variable does not retain its value.
echo exampleFunction('A'); // Outputs: Input: A, Counter: 1
echo exampleFunction('B'); // Outputs: Input: B, Counter: 1
```

As of PHP 8.1.0, when a method using static variables is inherited (but not overridden), the inherited method will now share static variables with the parent method. This means that static variables in methods now behave the same way as static properties.

As of PHP 8.3.0, static variables can be initialized with arbitrary expressions. This means that method calls, for example, can be used to initialize static variables.

Example #11 Usage of static Variables in Inherited Methods

```
<?php
class Foo {
    public static function counter() {
        static $counter = 0;
        $counter++;
        return $counter;
    }
}
class Bar extends Foo {}
var_dump(Foo::counter()); // int(1)
var_dump(Foo::counter()); // int(2)
var_dump(Bar::counter()); // int(3), prior to PHP 8.1.0 int(1)
var_dump(Bar::counter()); // int(4), prior to PHP 8.1.0 int(2)
```

References with `global` and `static` variables

PHP implements the static and global modifier for variables in terms of references. For example, a true global variable imported inside a function scope with the `global` statement actually creates a reference to the global variable. This can lead to unexpected behaviour which the following example addresses:

```php
<?php
function test_global_ref() {
    global $obj;
    $new = new stdClass;
    $obj = &$new;
}

function test_global_noref() {
    global $obj;
    $new = new stdClass;
    $obj = $new;
}

test_global_ref();
var_dump($obj);
test_global_noref();
var_dump($obj);
```

The above example will output:

```
NULL
object(stdClass)#1 (0) {
}
```

A similar behaviour applies to the `static` statement. References are not stored statically:

```php
<?php
function &get_instance_ref() {
    static $obj;

    echo 'Static object: ';
    var_dump($obj);
    if (!isset($obj)) {
        $new = new stdClass;
        // Assign a reference to the static variable
        $obj = &$new;
    }
    if (!isset($obj->property)) {
        $obj->property = 1;
    } else {
        $obj->property++;
    }
    return $obj;
}

function &get_instance_noref() {
    static $obj;

    echo 'Static object: ';
    var_dump($obj);
    if (!isset($obj)) {
```

```
        $new = new stdClass;
        // Assign the object to the static variable
        $obj = $new;
    }
    if (!isset($obj->property)) {
        $obj->property = 1;
    } else {
        $obj->property++;
    }
    return $obj;
}

$obj1 = get_instance_ref();
$still_obj1 = get_instance_ref();
echo "\n";
$obj2 = get_instance_noref();
$still_obj2 = get_instance_noref();
```

The above example will output:

```
Static object: NULL
Static object: NULL

Static object: NULL
Static object: object(stdClass)#3 (1) {
  ["property"]=>
  int(1)
}
```

This example demonstrates that when assigning a reference to a static variable, it is not *remembered* when the `&get_instance_ref()` function is called a second time.

Variable variables

Sometimes it is convenient to be able to have variable variable names. That is, a variable name which can be set and used dynamically. A normal variable is set with a statement such as:

```
<?php
$a = 'hello';
var_dump($a);
```

A variable variable takes the value of a variable and treats that as the name of a variable. In the above example, *hello*, can be used as the name of a variable by using two dollar signs. i.e.

```
<?php
$a = 'hello';
$$a = 'world';
var_dump($hello);
```

At this point two variables have been defined and stored in the PHP symbol tree: *$a* with contents "hello" and *$hello* with contents "world". Therefore, this statement:

```php
<?php
$a = 'hello';
$$a = 'world';
echo "$a {$$a}";
```

produces the exact same output as:

```php
<?php
$a = 'hello';
$$a = 'world';
echo "$a $hello";
```

i.e. they both produce: hello world.

In order to use variable variables with arrays, an ambiguity problem has to be resolved. That is, if the parser sees *$$a[1]* then it needs to know if *$a[1]* was meant to be used as a variable, or if *$$a* was wanted as the variable and then the `[1]` index from that variable. The syntax for resolving this ambiguity is: *${$a[1]}* for the first case and *${$a}[1]* for the second.

Class properties may also be accessed using variable property names. The variable property name will be resolved within the scope from which the call is made. For instance, if there is an expression such as *$foo->$bar*, then the local scope will be examined for *$bar* and its value will be used as the name of the property of *$foo*. This is also true if *$bar* is an array access.

Curly braces may also be used to clearly delimit the property name. They are most useful when accessing values within a property that contains an array, when the property name is made of multiple parts, or when the property name contains characters that are not otherwise valid (e.g. from json_decode() or SimpleXML).

Example #1 Variable property example

```php
<?php
class Foo {
    public $bar = 'I am bar.';
    public $arr = ['I am A.', 'I am B.', 'I am C.'];
    public $r   = 'I am r.';
}

$foo = new Foo();
$bar = 'bar';
$baz = ['foo', 'bar', 'baz', 'quux'];
echo $foo->$bar . "\n";
echo $foo->{$baz[1]} . "\n";

$start = 'b';
$end   = 'ar';
echo $foo->{$start . $end} . "\n";

$arr = 'arr';
echo $foo->{$arr[1]} . "\n";
echo $foo->{$arr}[1] . "\n";
```

The above example will output:

```
I am bar.
I am bar.
I am bar.
I am r.
I am B.
```

> **Warning**
>
> Please note that variable variables cannot be used with PHP's Superglobal arrays within functions or class methods. The variable `$this` is also a special variable that cannot be referenced dynamically.

Variables From External Sources

HTML Forms (GET and POST)

When a form is submitted to a PHP script, the information from that form is automatically made available to the script. There are few ways to access this information, for example:

Example #1 A simple HTML form

```
<form action="foo.php" method="post">
    Name:  <input type="text" name="username" /><br />
    Email: <input type="text" name="email" /><br />
    <input type="submit" name="submit" value="Submit me!" />
</form>
```

There are only two ways to access data from HTML forms. Currently available methods are listed below:

Example #2 Accessing data from a simple POST HTML form

```
<?php
echo $_POST['username'];
echo $_REQUEST['username'];
```

Using a GET form is similar except the appropriate GET predefined variable can be used instead. GET also applies to the `QUERY_STRING` (the information after the '?' in a URL). So, for example, `http://www.example.com/test.php?id=3` contains GET data which is accessible with *$_GET['id']*. See also *$_REQUEST*.

> **Note** :
>
> Dots and spaces in variable names are converted to underscores. For example `<input name="a.b" />` becomes `$_REQUEST["a_b"]`.

PHP also understands arrays in the context of form variables (see the related faq). For example, related variables may be grouped together, or this feature may be used to retrieve values from a multiple select input. For example, let's post a form to itself and upon submission display the data:

Example #3 More complex form variables

```
<?php
if ($_POST) {
    echo '<pre>';
    echo htmlspecialchars(print_r($_POST, true));
    echo '</pre>';
}
?>
<form action="" method="post">
    Name:  <input type="text" name="personal[name]" /><br />
    Email: <input type="text" name="personal[email]" /><br />
    Beer: <br />
    <select multiple name="beer[]">
        <option value="warthog">Warthog</option>
        <option value="guinness">Guinness</option>
        <option value="stuttgarter">Stuttgarter Schwabenbräu</option>
    </select><br />
    <input type="submit" value="submit me!" />
</form>
```

> **Note** :
>
> If an external variable name begins with a valid array syntax, trailing characters are silently ignored. For example, `<input name="foo[bar]baz">` becomes `$_REQUEST['foo']['bar']`.

IMAGE SUBMIT variable names

When submitting a form, it is possible to use an image instead of the standard submit button with a tag like:

```
<input type="image" src="image.gif" name="sub" />
```

When the user clicks somewhere on the image, the accompanying form will be transmitted to the server with two additional variables, *sub_x* and *sub_y*. These contain the coordinates of the user click within the image. The experienced may note that the actual variable names sent by the browser contains a period rather than an underscore, but PHP converts the period to an underscore automatically.

HTTP Cookies

PHP transparently supports HTTP cookies as defined by » RFC 6265. Cookies are a mechanism for storing data in the remote browser and thus tracking or identifying return users. It is possible to set cookies using the setcookie() function. Cookies are part of the HTTP header, so the SetCookie function must be called before any output is sent to the browser. This is the same restriction as for the header() function. Cookie data is then available in the appropriate cookie data arrays, such as *$_COOKIE* as well as in *$_REQUEST*. See the setcookie() manual page for more details and examples.

> **Note** :

> As of PHP 7.2.34, 7.3.23 and 7.4.11, respectively, the *names* of incoming cookies are no longer url-decoded for security reasons.

If multiple values should be assigned to a single cookie variable, they can be assigned as an array. For example:

```
<?php
setcookie("MyCookie[foo]", 'Testing 1', time()+3600);
setcookie("MyCookie[bar]", 'Testing 2', time()+3600);
```

That will create two separate cookies although *MyCookie* will now be a single array in the script. If just one cookie should be set with multiple values, consider using serialize() or explode() on the value first.

Note that a cookie will replace a previous cookie by the same name in the browser unless the path or domain is different. So, for a shopping cart application a counter may be kept, and passed along. I.e.

Example #4 A setcookie() example

```
<?php
if (isset($_COOKIE['count'])) {
    $count = $_COOKIE['count'] + 1;
} else {
    $count = 1;
}
setcookie('count', $count, time()+3600);
setcookie("Cart[$count]", $item, time()+3600);
```

Dots in incoming variable names

Typically, PHP does not alter the names of variables when they are passed into a script. However, it should be noted that the dot (period, full stop) is not a valid character in a PHP variable name. For the reason, look at it:

```
<?php
$varname.ext;  /* invalid variable name */
```

Now, what the parser sees is a variable named *$varname*, followed by the string concatenation operator, followed by the barestring (i.e. unquoted string which doesn't match any known key or reserved words) 'ext'. Obviously, this doesn't have the intended result.

For this reason, it is important to note that PHP will automatically replace any dots in incoming variable names with underscores.

Determining variable types

Because PHP determines the types of variables and converts them (generally) as needed, it is not always obvious what type a given variable is at any one time. PHP includes several functions which find out what type a variable is, such as: gettype(), is_array(), is_float(), is_int(), is_object(), and is_string(). See also the chapter on Types.

HTTP being a text protocol, most, if not all, content that comes in Superglobal arrays, like *$_POST* and *$_GET* will remain as strings. PHP will not try to convert values to a specific type. In the example below, *$_GET["var1"]* will contain the string "null" and *$_GET["var2"]*, the string "123".

```
/index.php?var1=null&var2=123
```

Changelog

Version	Description
7.2.34, 7.3.23, 7.4.11	The *names* of incoming cookies are no longer url-decoded for security reasons.

Constants

Table of Contents

A constant is an identifier (name) for a simple value. As the name suggests, that value cannot change during the execution of the script (except for magic constants, which aren't actually constants). Constants are case-sensitive. By convention, constant identifiers are always uppercase.

> **Note** :
>
> Prior to PHP 8.0.0, constants defined using the define() function may be case-insensitive.

> **Note** :
>
> As of PHP 8.5.0, redeclaring an already-defined constant is deprecated.

The name of a constant follows the same rules as any label in PHP. A valid constant name starts with a letter or underscore, followed by any number of letters, numbers, or underscores. As a regular expression, it would be expressed thusly: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`

It is possible to define() constants with reserved or even invalid names, whose value can only be retrieved with the constant() function. However, doing so is not recommended.

> **Tip**
>
> See also the Userland Naming Guide.

Example #1 Valid and invalid constant names

```php
<?php

// Valid constant names
define("FOO",     "something");
define("FOO2",    "something else");
define("FOO_BAR", "something more");

// Invalid constant names
define("2FOO",    "something");

// This is valid, but should be avoided:
// PHP may one day provide a magical constant
```

```
// that will break your script
define("__FOO__", "something");
```

> **Note** :
>
> For our purposes here, a letter is a-z, A-Z, and the ASCII characters from 128 through 255 (0x80-0xff).

Like superglobals, the scope of a constant is global. Constants can be accessed from anywhere in a script without regard to scope. For more information on scope, read the manual section on variable scope.

> **Note** :
>
> As of PHP 7.1.0, class constant may declare a visibility of protected or private, making them only available in the hierarchical scope of the class in which it is defined.

Syntax

Constants can be defined using the `const` keyword, or by using the define()-function. While define() allows a constant to be defined to an arbitrary expression, the `const` keyword has restrictions as outlined in the next paragraph. Once a constant is defined, it can never be changed or undefined.

When using the `const` keyword, only scalar (bool, int, float and string) expressions and constant arrays containing only scalar expressions are accepted. It is possible to define constants as a resource, but it should be avoided, as it can cause unexpected results.

The value of a constant is accessed simply by specifying its name. Unlike variables, a constant is *not* prepended with a `$`. It is also possible to use the constant() function to read a constant's value if the constant's name is obtained dynamically. Use get_defined_constants() to get a list of all defined constants.

> **Note** :
>
> Constants and (global) variables are in a different namespace. This implies that for example **`true`** and *$TRUE* are generally different.

If an undefined constant is used an Error is thrown. Prior to PHP 8.0.0, undefined constants would be interpreted as a bare word string, i.e. (CONSTANT vs "CONSTANT"). This fallback is deprecated as of PHP 7.2.0, and an error of level **`E_WARNING`** is issued when it happens. Prior to PHP 7.2.0, an error of level E_NOTICE has been issued instead. See also the manual entry on why $foo[bar] is wrong (unless `bar` is a constant). This does not apply to (fully) qualified constants, which will always raise a Error if undefined.

> **Note** :
>
> To check if a constant is set, use the defined() function.

These are the differences between constants and variables:

- Constants do not have a dollar sign (`$`) before them;

- Constants may be defined and accessed anywhere without regard to variable scoping rules;
- Constants may not be redefined or undefined once they have been set; and
- Constants may only evaluate to scalar values or arrays.

Example #1 Defining Constants

```php
<?php
define("CONSTANT", "Hello world.");
echo CONSTANT; // outputs "Hello world."
echo Constant; // Emits an Error: Undefined constant "Constant"
               // Prior to PHP 8.0.0, outputs "Constant" and issues a warning.
```

Example #2 Defining Constants using the `const` keyword

```php
<?php
// Simple scalar value
const CONSTANT = 'Hello World';

echo CONSTANT . "\n";

// Scalar expression
const ANOTHER_CONST = CONSTANT . '; Goodbye World';
echo ANOTHER_CONST . "\n";

const ANIMALS = array('dog', 'cat', 'bird');
echo ANIMALS[1] . "\n"; // outputs "cat"

// Constant arrays
define('ANIMALS_DEF', array(
    'dog',
    'cat',
    'bird'
));
echo ANIMALS_DEF[1] . "\n"; // outputs "cat"
```

> **Note** :
>
> As opposed to defining constants using define(), constants defined using the `const` keyword must be declared at the top-level scope because they are defined at compile-time. This means that they cannot be declared inside functions, loops, `if` statements or `try/catch` blocks.

Changelog

Version	Description
8.5.0	Added support for type casts (e.g. `(int)`, `(string)`) in constant expressions. Also added support for Closures and first-class callables in constant expressions.

See Also

- Class Constants

Predefined constants

PHP provides a large number of predefined constants to any script which it runs. Many of these constants, however, are created by various extensions, and will only be present when those extensions are available, either via dynamic loading or because they have been compiled in.

Magic constants

There are a few magical constants that change depending on where they are used. For example, the value of `__LINE__` depends on the line that it's used on in a script. All these "magical" constants are resolved at compile time, unlike regular constants, which are resolved at runtime. These special constants are case-insensitive and are as follows:

PHP's magic constants

Name	Description
`__LINE__`	The current line number of the file.
`__FILE__`	The full path and filename of the file with symlinks resolved. If used inside an include, the name of the included file is returned.
`__DIR__`	The directory of the file. If used inside an include, the directory of the included file is returned. This is equivalent to `dirname(__FILE__)`. This directory name does not have a trailing slash unless it is the root directory.
`__FUNCTION__`	The function name, or `{closure}` for anonymous functions.
`__CLASS__`	The class name. The class name includes the namespace it was declared in (e.g. `Foo\Bar`). When used inside a trait method, `__CLASS__` is the name of the class the trait is used in.
`__TRAIT__`	The trait name. The trait name includes the namespace it was declared in (e.g. `Foo\Bar`).
`__METHOD__`	The class method name.
`__PROPERTY__`	Only valid inside a property hook. It is equal to the name of the property.
`__NAMESPACE__`	The name of the current namespace.
`ClassName::class`	The fully qualified class name.

See Also

- ::class
- get_class()
- get_object_vars()
- file_exists()
- function_exists()

Expressions

Expressions are the most important building blocks of PHP. In PHP, almost anything you write is an expression. The simplest yet most accurate way to define an expression is "anything that has a value".

The most basic forms of expressions are constants and variables. When you type `$a = 5`, you're assigning `5` into *$a*. `5`, obviously, has the value 5, or in other words `5` is an expression with the value of 5 (in this case, `5` is an integer constant).

After this assignment, you'd expect *$a*'s value to be 5 as well, so if you wrote `$b = $a`, you'd expect it to behave just as if you wrote `$b = 5`. In other words, *$a* is an expression with the value of 5 as well. If everything works right, this is exactly what will happen.

Slightly more complex examples for expressions are functions. For instance, consider the following function:

```
<?php
function foo ()
{
    return 5;
}
?>
```

Assuming you're familiar with the concept of functions (if you're not, take a look at the chapter about functions), you'd assume that typing `$c = foo()` is essentially just like writing `$c = 5`, and you're right. Functions are expressions with the value of their return value. Since `foo()` returns 5, the value of the expression '`foo()`' is 5. Usually functions don't just return a static value but compute something.

Of course, values in PHP don't have to be integers, and very often they aren't. PHP supports four scalar value types: int values, floating point values (float), string values and bool values (scalar values are values that you can't 'break' into smaller pieces, unlike arrays, for instance). PHP also supports two composite (non-scalar) types: arrays and objects. Each of these value types can be assigned into variables or returned from functions.

PHP takes expressions much further, in the same way many other languages do. PHP is an expression-oriented language, in the sense that almost everything is an expression. Consider the example we've already dealt with, `$a = 5`. It's easy to see that there are two values involved here, the value of the integer constant `5`, and the value of *$a* which is being updated to 5 as well. But the truth is that there's one additional value involved here, and that's the value of the assignment itself. The assignment itself evaluates to the assigned value, in this case 5. In practice, it means that `$a = 5`, regardless of what it does, is an expression with the value 5. Thus, writing something like `$b = ($a = 5)` is like writing `$a = 5; $b = 5;` (a semicolon marks the end of a statement). Since assignments are parsed in a right to left order, you can also write `$b = $a = 5`.

Another good example of expression orientation is pre- and post-increment and decrement. Users of PHP and many other languages may be familiar with the notation of `variable++` and `variable--`. These are increment and decrement operators. In PHP, like in C, there are two types of increment - pre-increment and post-increment. Both pre-increment and post-increment essentially increment the variable, and the effect on the variable is identical. The difference is with the value of the increment expression. Pre-increment, which is written `++$variable`, evaluates to the incremented value (PHP increments the variable before reading its value, thus the name 'pre-increment'). Post-increment, which is written `$variable++` evaluates to the original value of *$variable*, before it was incremented (PHP increments the variable after reading its value, thus the name 'post-increment').

A very common type of expressions are comparison expressions. These expressions evaluate to either **`false`** or **`true`**. PHP supports > (bigger than), >= (bigger than or equal to), == (equal), != (not equal), < (smaller than) and <= (smaller than or equal to). The language also supports a set of strict equivalence operators: === (equal to and same type) and !== (not equal to or not same type). These expressions are most commonly used inside conditional execution, such as `if` statements.

The last example of expressions we'll deal with here is combined operator-assignment expressions. You already know that if you want to increment *$a* by 1, you can simply write `$a++` or `++$a`. But what if you want to add more than one to it, for instance 3? You could write `$a++` multiple times, but this is obviously not a very efficient or comfortable way. A much more common practice is to write `$a = $a + 3`. `$a + 3` evaluates to the

value of *$a* plus 3, and is assigned back into *$a*, which results in incrementing *$a* by 3. In PHP, as in several other languages like C, you can write this in a shorter way, which with time would become clearer and quicker to understand as well. Adding 3 to the current value of *$a* can be written `$a += 3`. This means exactly "take the value of *$a*, add 3 to it, and assign it back into *$a*". In addition to being shorter and clearer, this also results in faster execution. The value of `$a += 3`, like the value of a regular assignment, is the assigned value. Notice that it is NOT 3, but the combined value of *$a* plus 3 (this is the value that's assigned into *$a*). Any two-place operator can be used in this operator-assignment mode, for example `$a -= 5` (subtract 5 from the value of *$a*), `$b *= 7` (multiply the value of *$b* by 7), etc.

There is one more expression that may seem odd if you haven't seen it in other languages, the ternary conditional operator:

```
<?php
$first ? $second : $third
?>
```

If the value of the first subexpression is `true` (non-zero), then the second subexpression is evaluated, and that is the result of the conditional expression. Otherwise, the third subexpression is evaluated, and that is the value.

The following example should help you understand pre- and post-increment and expressions in general a bit better:

```
<?php
function double($i)
{
    return $i * 2;
}

$b = $a = 5;        /* assign the value five into the variable $a and $b */
$c = $a++;          /* post-increment, assign original value of $a
                       (5) to $c */
print "a = {$a}; b = {$b}; c = {$c}" . PHP_EOL;

$e = $d = ++$b;     /* pre-increment, assign the incremented value of
                       $b (6) to $d and $e */
print "b = {$b}; d = {$d}; e = {$e}" . PHP_EOL;

/* at this point, both $d and $e are equal to 6 */

$f = double($d++);  /* assign twice the value of $d before
                       the increment, 2*6 = 12 to $f */
print "d = {$d}; f = {$f}" . PHP_EOL;

$g = double(++$e);  /* assign twice the value of $e after
                       the increment, 2*7 = 14 to $g */
print "e = {$e}; g = {$g}" . PHP_EOL;

$h = $g += 10;      /* first, $g is incremented by 10 and ends with the
                       value of 24. the value of the assignment (24) is
                       then assigned into $h, and $h ends with the value
                       of 24 as well. */
print "g = {$g}; h = {$h}" . PHP_EOL;
?>
```

Some expressions can be considered as statements. In this case, a statement has the form of '`expr ;`' that is, an expression followed by a semicolon. In `$b = $a = 5;`, `$a = 5` is a valid expression, but it's not a statement by itself. `$b = $a = 5;`, however, is a valid statement.

One last thing worth mentioning is the truth value of expressions. In many events, mainly in conditional execution and loops, you're not interested in the specific value of the expression, but only care about whether it means **`true`** or **`false`**. The constants **`true`** and **`false`** (case-insensitive) are the two possible boolean values. When necessary, an expression is automatically converted to boolean. See the section about type-casting for details about how.

PHP provides a full and powerful implementation of expressions, and documenting it entirely goes beyond the scope of this manual. The above examples should give you a good idea about what expressions are and how you can construct useful expressions. Throughout the rest of this manual we'll write *expr* to indicate any valid PHP expression.

Operators

Table of Contents

An operator is something that takes one or more values (or expressions, in programming jargon) and yields another value (so that the construction itself becomes an expression).

Operators can be grouped according to the number of values they take. Unary operators take only one value, for example `!` (the logical not operator) or `++` (the increment operator). Binary operators take two values, such as the familiar arithmetical operators `+` (plus) and `-` (minus), and the majority of PHP operators fall into this category. Finally, there is a single ternary operator, `? :`, which takes three values; this is usually referred to simply as "the ternary operator" (although it could perhaps more properly be called the conditional operator).

A full list of PHP operators follows in the section Operator Precedence. The section also explains operator precedence and associativity, which govern exactly how expressions containing several different operators are evaluated.

There is a PECL extension that allows for overloading of some operators for objects. For more information, see the Operator Overloading for Objects section.

Operator Precedence

The precedence of an operator specifies how "tightly" it binds two expressions together. For example, in the expression `1 + 5 * 3`, the answer is `16` and not `18` because the multiplication ("*") operator has a higher precedence than the addition ("+") operator. Parentheses may be used to force precedence, if necessary. For instance: `(1 + 5) * 3` evaluates to `18`.

When operators have equal precedence their associativity decides how the operators are grouped. For example "-" is left-associative, so `1 - 2 - 3` is grouped as `(1 - 2) - 3` and evaluates to `-4`. "=" on the other hand is right-associative, so `$a = $b = $c` is grouped as `$a = ($b = $c)`.

Operators of equal precedence that are non-associative cannot be used next to each other, for example `1 < 2 > 1` is illegal in PHP. The expression `1 <= 1 == 1` on the other hand is legal, because the `==` operator has a lower precedence than the `<=` operator.

Associativity is only meaningful for binary (and ternary) operators. Unary operators are either prefix or postfix so this notion is not applicable. For example `!!$a` can only be grouped as `!(!$a)`.

Use of parentheses, even when not strictly necessary, can often increase readability of the code by making grouping explicit rather than relying on the implicit operator precedence and associativity.

The following table lists the operators in order of precedence, with the highest-precedence ones at the top. Operators on the same line have equal precedence, in which case associativity decides grouping.

Operator Precedence

Associativity	Operators	Additional Information
(n/a)	`clone new`	clone and new
right	`**`	arithmetic
(n/a)	`+ - ++ -- ~ (int) (float) (string) (array) (object) (bool) @`	arithmetic (unary + and -), increment/decrement, bitwise, type casting and error control
left	`instanceof`	type
(n/a)	`!`	logical
left	`* / %`	arithmetic
left	`+ - .`	arithmetic (binary + and -), array and string (`.` prior to PHP 8.0.0)
left	`<< >>`	bitwise
left	`.`	string (as of PHP 8.0.0)
left	`|>`	pipe
non-associative	`< <= > >=`	comparison
non-associative	`== != === !== <> <=>`	comparison
left	`&`	bitwise and references
left	`^`	bitwise
left	`|`	bitwise
left	`&&`	logical
left	`||`	logical
right	`??`	null coalescing
non-associative	`? :`	ternary (left-associative prior to PHP 8.0.0)
right	`= += -= *= **= /= .= %= &= |= ^= <<= >>= ??=`	assignment
(n/a)	`yield from`	yield from
(n/a)	`=>`	yield with keys
(n/a)	`yield`	yield
(n/a)	`print`	print
left	`and`	logical
left	`xor`	logical

Associativity	Operators	Additional Information
left	`or`	logical
(n/a)	`include include_once require require_once`	include, include_once, require and require_once
(n/a)	`fn (...) =>`	fn arrow disambiguity
(n/a)	`throw`	throw

> **Note** :
>
> The unary operators sharing the row with the cast operators (`~`, `@`, and unary `+`/`-`) bind more tightly than `instanceof`, whereas `!` binds less tightly. Consequently `(int) $x instanceof Foo` is grouped as `((int) $x) instanceof Foo`, while `!$x instanceof Foo` is grouped as `!($x instanceof Foo)`.

Example #1 Associativity

```php
<?php
$a = 3 * 3 % 5; // (3 * 3) % 5 = 4
var_dump($a);

$a = 1;
$b = 2;
$a = $b += 3; // $a = ($b += 3) -> $a = 5, $b = 5
var_dump($a, $b);
?>
```

The ternary operator specifically requires the use of parenthesis to disambiguate precedence.

Example #2 Explicit Precedence

```php
<?php
$a = true ? 0 : (true ? 1 : 2);
var_dump($a);

// this is not allowed since PHP 8
// $a = true ? 0 : true ? 1 : 2;
?>
```

Operator precedence and associativity only determine how expressions are grouped, they do not specify an order of evaluation. PHP does not (in the general case) specify in which order an expression is evaluated and code that assumes a specific order of evaluation should be avoided, because the behavior can change between versions of PHP or depending on the surrounding code.

Example #3 Undefined order of evaluation

```php
<?php
$a = 1;
echo $a + $a++; // may print either 2 or 3

$i = 1;
```

```php
$array[$i] = $i++; // may set either index 1 or 2
?>
```

Example #4 +, - and . precedence

```php
<?php
$x = 4;
// this line might result in unexpected output:
echo "x minus one equals " . $x-1 . ", or so I hope\n";

// the desired precedence can be enforced by using parentheses:
echo "x minus one equals " . ($x-1) . ", or so I hope\n";

// this is not allowed, and throws a TypeError:
echo (("x minus one equals " . $x) - 1) . ", or so I hope\n";
?>
```

The above example will output:

```
-1, or so I hope
-1, or so I hope
Fatal error: Uncaught TypeError: Unsupported operand types: string - int
```

Example #5 Prior to PHP 8, +, - and . had the same precedence

```php
<?php
$x = 4;
// this line might result in unexpected output:
echo "x minus one equals " . $x-1 . ", or so I hope\n";

// because it is evaluated like this line (prior to PHP 8.0.0):
echo (("x minus one equals " . $x) - 1) . ", or so I hope\n";

// the desired precedence can be enforced by using parentheses:
echo "x minus one equals " . ($x-1) . ", or so I hope\n";
?>
```

The above example will output:

```
-1, or so I hope
-1, or so I hope
x minus one equals 3, or so I hope
```

> **Note** :

> Although = has a lower precedence than most other operators, PHP will still allow expressions similar to the following: `if (!$a = foo())`, in which case the return value of `foo()` is put into *$a*.
>
> This is possible because the left-hand side of an assignment must be a variable, so the assignment is grouped with that variable rather than with the surrounding higher-precedence prefix operator. The same applies to the other prefix operators that take an expression operand, such as `clone`, the cast operators, `@` and ~: for instance `clone $a = $b` is grouped as `clone ($a = $b)`, not `(clone $a) = $b`.

Changelog

Version	Description
8.0.0	String concatenation (`.`) now has a lower precedence than arithmetic addition/subtraction (`+` and `-`) and bitwise shift left/right (`<<` and `>>`); previously it had the same precedence as `+` and `-` and a higher precedence than `<<` and `>>`.
8.0.0	The ternary operator (`? :`) is non-associative now; previously it was left-associative.
7.4.0	Relying on the precedence of string concatenation (`.`) relative to arithmetic addition/subtraction (`+` or `-`) or bitwise shift left/right (`<<` or `>>`), i.e. using them together in an unparenthesized expression, is deprecated.
7.4.0	Relying on left-associativity of the ternary operator (`? :`), i.e. nesting multiple unparenthesized ternary operators, is deprecated.

Arithmetic

Remember basic arithmetic from school? These work just like those.

Arithmetic Operators

Example	Name	Result
`+$a`	Identity	Conversion of *$a* to int or float as appropriate.
`-$a`	Negation	Opposite of *$a*.
`$a + $b`	Addition	Sum of *$a* and *$b*.
`$a - $b`	Subtraction	Difference of *$a* and *$b*.
`$a * $b`	Multiplication	Product of *$a* and *$b*.
`$a / $b`	Division	Quotient of *$a* and *$b*.
`$a % $b`	Modulo	Remainder of *$a* divided by *$b*.
`$a ** $b`	Exponentiation	Result of raising *$a* to the *$b*'th power.

The division operator `/` returns a float value unless the two operands are int (or numeric strings which are type juggled to int) and the numerator is a multiple of the divisor, in which case an integer value will be returned. For integer division, see intdiv().

Operands of modulo are converted to int before processing. For floating-point modulo, see fmod().

The result of the modulo operator `%` has the same sign as the dividend — that is, the result of `$a % $b` will have the same sign as *$a*. For example:

Example #1 The Modulo Operator

```php
<?php
var_dump(5 % 3);
var_dump(5 % -3);
var_dump(-5 % 3);
var_dump(-5 % -3);
?>
```

The above example will output:

```
int(2)
int(2)
int(-2)
int(-2)
```

See Also

- Math functions

Increment and Decrement

PHP supports pre- and post-increment and decrement operators. Those unary operators allow to increment or decrement the value by one.

Increment/decrement Operators

Example	Name	Effect
++$a	Pre-increment	Increments *$a* by one, then returns *$a*.
$a++	Post-increment	Returns *$a*, then increments *$a* by one.
--$a	Pre-decrement	Decrements *$a* by one, then returns *$a*.
$a--	Post-decrement	Returns *$a*, then decrements *$a* by one.

Example #1 Examples of Increment/Decrement

```php
<?php
echo 'Post-increment:', PHP_EOL;
$a = 5;
var_dump($a++);
var_dump($a);

echo 'Pre-increment:', PHP_EOL;
$a = 5;
var_dump(++$a);
var_dump($a);

echo 'Post-decrement:', PHP_EOL;
$a = 5;
var_dump($a--);
var_dump($a);
```

```php
echo 'Pre-decrement:', PHP_EOL;
$a = 5;
var_dump(--$a);
var_dump($a);
?>
```

The above example will output:

```
Post-increment:
int(5)
int(6)
Pre-increment:
int(6)
int(6)
Post-decrement:
int(5)
int(4)
Pre-decrement:
int(4)
int(4)
```

> **Warning**
>
> The increment and decrement operators have no effect on values of type bool. A **E_WARNING** is emitted as of PHP 8.3.0, because this will implicitly cast the value to int in the future.
>
> The decrement operator has no effect on values of type null. A **E_WARNING** is emitted as of PHP 8.3.0, because this will implicitly cast the value to int in the future.
>
> The decrement operator has no effect on non- numeric string. A **E_WARNING** is emitted as of PHP 8.3.0, because a TypeError will be thrown in the future.

> **Note** :
>
> Internal objects that support overloading addition and/or subtraction can also be incremented and/or decremented. One such internal object is GMP.

PERL string increment feature

> **Warning**
>
> This feature is soft-deprecated as of PHP 8.3.0. The str_increment() function should be used instead. As of PHP 8.5.0, incrementing non-numeric strings emits a deprecation warning.

It is possible to increment a non- numeric string in PHP. The string must be an alphanumerical ASCII string. Which increments letters up to the next letter, when reaching the letter z the increment is carried to the value on the left. For example, `$a = 'Z'; $a++;` turns *$a* into `'AA'`.

Example #2 PERL string increment example

```php
<?php
echo '== Alphabetic strings ==' . PHP_EOL;
$s = 'W';
for ($n=0; $n<6; $n++) {
    echo ++$s . PHP_EOL;
}
// Alphanumeric strings behave differently
echo '== Alphanumeric strings ==' . PHP_EOL;
$d = 'A8';
for ($n=0; $n<6; $n++) {
    echo ++$d . PHP_EOL;
}
$d = 'A08';
for ($n=0; $n<6; $n++) {
    echo ++$d . PHP_EOL;
}
?>
```

The above example will output:

```
== Alphabetic strings ==
X
Y
Z
AA
AB
AC
== Alphanumeric strings ==
A9
B0
B1
B2
B3
B4
A09
A10
A11
A12
A13
A14
```

> **Warning**
>
> If the alphanumerical string can be interpreted as a numeric string it will be cast to an int or float. This is particularly an issue with strings that look like a floating point numbers written in exponential notation. The str_increment() function does not suffer from these implicit type cast.
>
> **Example #3 Alphanumerical string converted to float**

```php
<?php
$s = "5d9";
var_dump(++$s);
var_dump(++$s);
?>
```

The above example will output:

```
string(3) "5e0"
float(6)
```

This is because the value `"5e0"` is interpreted as a float and cast to the value `5.0` before being incremented.

Assignment

The basic assignment operator is "=". Your first inclination might be to think of this as "equal to". Don't. It really means that the left operand gets set to the value of the expression on the right (that is, "gets set to").

The value of an assignment expression is the value assigned. That is, the value of `"$a = 3"` is 3. This allows you to do some tricky things:

Example #1 Nested Assignments

```php
<?php
$a = ($b = 4) + 5; // $a is equal to 9 now, and $b has been set to 4.
var_dump($a);
?>
```

In addition to the basic assignment operator, there are "combined operators" for all of the binary arithmetic, array union and string operators that allow you to use a value in an expression and then set its value to the result of that expression. For example:

Example #2 Combined Assignments

```php
<?php
$a = 3;
$a += 5; // sets $a to 8, as if we had said: $a = $a + 5;
$b = "Hello ";
$b .= "There!"; // sets $b to "Hello There!", just like $b = $b . "There!";

var_dump($a, $b);
?>
```

Note that the assignment copies the original variable to the new one (assignment by value), so changes to one will not affect the other. This may also have relevance if you need to copy something like a large array inside a tight loop.

An exception to the usual assignment by value behaviour within PHP occurs with objects, which are assigned by reference. Objects may be explicitly copied via the clone keyword.

Assignment by Reference

Assignment by reference is also supported, using the "$var = &$othervar;" syntax. Assignment by reference means that both variables end up pointing at the same data, and nothing is copied anywhere.

Example #3 Assigning by reference

```
<?php
$a = 3;
$b = &$a; // $b is a reference to $a

print "$a\n"; // prints 3
print "$b\n"; // prints 3

$a = 4; // change $a

print "$a\n"; // prints 4
print "$b\n"; // prints 4 as well, since $b is a reference to $a, which has
              // been changed
?>
```

The new operator returns a reference automatically, as such assigning the result of new by reference is an error.

Example #4 new Operator By-Reference

```
<?php
class C {}

$o = &new C;
?>
```

The above example will output:

```
Parse error: syntax error, unexpected token ";", expecting "("
```

More information on references and their potential uses can be found in the References Explained section of the manual.

Arithmetic Assignment Operators

Example	Equivalent	Operation
$a += $b	$a = $a + $b	Addition
$a -= $b	$a = $a - $b	Subtraction
$a *= $b	$a = $a * $b	Multiplication

Example	Equivalent	Operation
$a /= $b	$a = $a / $b	Division
$a %= $b	$a = $a % $b	Modulus
$a **= $b	$a = $a ** $b	Exponentiation

Bitwise Assignment Operators

Example	Equivalent	Operation
$a &= $b	$a = $a & $b	Bitwise And
$a |= $b	$a = $a | $b	Bitwise Or
$a ^= $b	$a = $a ^ $b	Bitwise Xor
$a <<= $b	$a = $a << $b	Left Shift
$a >>= $b	$a = $a >> $b	Right Shift

Other Assignment Operators

Example	Equivalent	Operation
$a .= $b	$a = $a . $b	String Concatenation
$a ??= $b	$a = $a ?? $b	Null Coalesce

See Also

- arithmetic operators
- bitwise operators
- null coalescing operator

Bitwise

Bitwise operators allow evaluation and manipulation of specific bits within an integer.

Bitwise Operators

Example	Name	Result
`$a & $b`	And	Bits that are set in both *$a* and *$b* are set.
`$a | $b`	Or (inclusive or)	Bits that are set in either *$a* or *$b* are set.
`$a ^ $b`	Xor (exclusive or)	Bits that are set in *$a* or *$b* but not both are set.
`~ $a`	Not	Bits that are set in *$a* are not set, and vice versa.
`$a << $b`	Shift left	Shift the bits of *$a* *$b* steps to the left (each step means "multiply by two")
`$a >> $b`	Shift right	Shift the bits of *$a* *$b* steps to the right (each step means "divide by two")

Bit shifting in PHP is arithmetic. Bits shifted off either end are discarded. Left shifts have zeros shifted in on the right while the sign bit is shifted out on the left, meaning the sign of an operand is not preserved. Right shifts have copies of the sign bit shifted in on the left, meaning the sign of an operand is preserved.

Use parentheses to ensure the desired precedence. For example, `$a & $b == true` evaluates the equivalency then the bitwise and; while `($a & $b) == true` evaluates the bitwise and then the equivalency.

If both operands for the &, | and ^ operators are strings, then the operation will be performed on the ASCII values of the characters that make up the strings and the result will be a string. In all other cases, both operands will be converted to integers and the result will be an integer.

If the operand for the ~ operator is a string, the operation will be performed on the ASCII values of the characters that make up the string and the result will be a string, otherwise the operand and the result will be treated as integers.

Both operands and the result for the << and >> operators are always treated as integers.

PHP's error_reporting ini setting uses bitwise values, providing a real-world demonstration of turning bits off. To show all errors, except for notices, the php.ini file instructions say to use: **`E_ALL & ~E_NOTICE`**

```
    This works by starting with E_ALL:
    00000000000000000111011111111111
    Then taking the value of E_NOTICE...
    00000000000000000000000000001000
    ... and inverting it via ~:
    11111111111111111111111111110111
    Finally, it uses AND (&) to find the bits turned
    on in both values:
    00000000000000000111011111110111
```

Another way to accomplish that is using XOR (^) to find bits that are on in only one value or the other: **`E_ALL ^ E_NOTICE`**

error_reporting can also be used to demonstrate turning bits on. The way to show just errors and recoverable errors is: **`E_ERROR | E_RECOVERABLE_ERROR`**

```
    This process combines E_ERROR
    00000000000000000000000000000001
    and
    00000000000000000001000000000000
    using the OR (|) operator
    to get the bits turned on in either value:
    00000000000000000001000000000001
```

Example #1 Bitwise AND, OR and XOR operations on integers

```php
<?php
/*
 * Ignore the top section,
 * it is just formatting to make output clearer.
 */

$format = '(%1$2d = %1$04b) = (%2$2d = %2$04b)'
        . ' %3$s (%4$2d = %4$04b)' . "\n";

echo <<<EOH
 ---------     ---------  -- ---------
 result        value      op test
 ---------     ---------  -- ---------
EOH;
```

```php
/*
 * Here are the examples.
 */

$values = array(0, 1, 2, 4, 8);
$test = 1 + 4;

echo "\n Bitwise AND \n";
foreach ($values as $value) {
    $result = $value & $test;
    printf($format, $result, $value, '&', $test);
}

echo "\n Bitwise Inclusive OR \n";
foreach ($values as $value) {
    $result = $value | $test;
    printf($format, $result, $value, '|', $test);
}

echo "\n Bitwise Exclusive OR (XOR) \n";
foreach ($values as $value) {
    $result = $value ^ $test;
    printf($format, $result, $value, '^', $test);
}
?>
```

The above example will output:

```
 ---------     ---------  -- ---------
 result        value      op test
 ---------     ---------  -- ---------
 Bitwise AND
( 0 = 0000) = ( 0 = 0000) & ( 5 = 0101)
( 1 = 0001) = ( 1 = 0001) & ( 5 = 0101)
( 0 = 0000) = ( 2 = 0010) & ( 5 = 0101)
( 4 = 0100) = ( 4 = 0100) & ( 5 = 0101)
( 0 = 0000) = ( 8 = 1000) & ( 5 = 0101)

 Bitwise Inclusive OR
( 5 = 0101) = ( 0 = 0000) | ( 5 = 0101)
( 5 = 0101) = ( 1 = 0001) | ( 5 = 0101)
( 7 = 0111) = ( 2 = 0010) | ( 5 = 0101)
( 5 = 0101) = ( 4 = 0100) | ( 5 = 0101)
(13 = 1101) = ( 8 = 1000) | ( 5 = 0101)

 Bitwise Exclusive OR (XOR)
( 5 = 0101) = ( 0 = 0000) ^ ( 5 = 0101)
( 4 = 0100) = ( 1 = 0001) ^ ( 5 = 0101)
( 7 = 0111) = ( 2 = 0010) ^ ( 5 = 0101)
( 1 = 0001) = ( 4 = 0100) ^ ( 5 = 0101)
(13 = 1101) = ( 8 = 1000) ^ ( 5 = 0101)
```

Example #2 Bitwise XOR operations on strings

```php
<?php
echo 12 ^ 9, PHP_EOL; // Outputs '5'

echo "12" ^ "9", PHP_EOL; // Outputs the Backspace character (ascii 8)
                          // ('1' (ascii 49)) ^ ('9' (ascii 57)) = #8

echo "hallo" ^ "hello", PHP_EOL; // Outputs the ascii values #0 #4 #0 #0 #0
                                 // 'a' ^ 'e' = #4

echo 2 ^ "3", PHP_EOL; // Outputs 1
                       // 2 ^ ((int) "3") == 1

echo "2" ^ 3, PHP_EOL; // Outputs 1
                       // ((int) "2") ^ 3 == 1
?>
```

Example #3 Bit shifting on integers

```php
<?php
/*
 * Here are the examples.
 */

echo "\n--- BIT SHIFT RIGHT ON POSITIVE INTEGERS ---\n";

$val = 4;
$places = 1;
$res = $val >> $places;
p($res, $val, '>>', $places, 'copy of sign bit shifted into left side');

$val = 4;
$places = 2;
$res = $val >> $places;
p($res, $val, '>>', $places);

$val = 4;
$places = 3;
$res = $val >> $places;
p($res, $val, '>>', $places, 'bits shift out right side');

$val = 4;
$places = 4;
$res = $val >> $places;
p($res, $val, '>>', $places, 'same result as above; can not shift beyond 0');


echo "\n--- BIT SHIFT RIGHT ON NEGATIVE INTEGERS ---\n";

$val = -4;
$places = 1;
$res = $val >> $places;
p($res, $val, '>>', $places, 'copy of sign bit shifted into left side');
```

```php
$val = -4;
$places = 2;
$res = $val >> $places;
p($res, $val, '>>', $places, 'bits shift out right side');

$val = -4;
$places = 3;
$res = $val >> $places;
p($res, $val, '>>', $places, 'same result as above; can not shift beyond -1');


echo "\n--- BIT SHIFT LEFT ON POSITIVE INTEGERS ---\n";

$val = 4;
$places = 1;
$res = $val << $places;
p($res, $val, '<<', $places, 'zeros fill in right side');

$val = 4;
$places = (PHP_INT_SIZE * 8) - 4;
$res = $val << $places;
p($res, $val, '<<', $places);

$val = 4;
$places = (PHP_INT_SIZE * 8) - 3;
$res = $val << $places;
p($res, $val, '<<', $places, 'sign bits get shifted out');

$val = 4;
$places = (PHP_INT_SIZE * 8) - 2;
$res = $val << $places;
p($res, $val, '<<', $places, 'bits shift out left side');


echo "\n--- BIT SHIFT LEFT ON NEGATIVE INTEGERS ---\n";

$val = -4;
$places = 1;
$res = $val << $places;
p($res, $val, '<<', $places, 'zeros fill in right side');

$val = -4;
$places = (PHP_INT_SIZE * 8) - 3;
$res = $val << $places;
p($res, $val, '<<', $places);

$val = -4;
$places = (PHP_INT_SIZE * 8) - 2;
$res = $val << $places;
p($res, $val, '<<', $places, 'bits shift out left side, including sign bit');


/*
 * Ignore this bottom section,
 * it is just formatting to make output clearer.
 */
```

```php
function p($res, $val, $op, $places, $note = '') {
    $format = '%0' . (PHP_INT_SIZE * 8) . "b\n";

    printf("Expression: %d = %d %s %d\n", $res, $val, $op, $places);

    echo " Decimal:\n";
    printf("  val=%d\n", $val);
    printf("  res=%d\n", $res);

    echo " Binary:\n";
    printf('  val=' . $format, $val);
    printf('  res=' . $format, $res);

    if ($note) {
        echo " NOTE: $note\n";
    }

    echo "\n\n";
}
?>
```

Output of the above example on 32 bit machines:

```
--- BIT SHIFT RIGHT ON POSITIVE INTEGERS ---
Expression: 2 = 4 >> 1
 Decimal:
  val=4
  res=2
 Binary:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000010
 NOTE: copy of sign bit shifted into left side

Expression: 1 = 4 >> 2
 Decimal:
  val=4
  res=1
 Binary:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000001

Expression: 0 = 4 >> 3
 Decimal:
  val=4
  res=0
 Binary:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000000
 NOTE: bits shift out right side

Expression: 0 = 4 >> 4
 Decimal:
  val=4
  res=0
```

```
 Binary:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000000
 NOTE: same result as above; can not shift beyond 0


--- BIT SHIFT RIGHT ON NEGATIVE INTEGERS ---
Expression: -2 = -4 >> 1
 Decimal:
  val=-4
  res=-2
 Binary:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111110
 NOTE: copy of sign bit shifted into left side

Expression: -1 = -4 >> 2
 Decimal:
  val=-4
  res=-1
 Binary:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111111
 NOTE: bits shift out right side

Expression: -1 = -4 >> 3
 Decimal:
  val=-4
  res=-1
 Binary:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111111
 NOTE: same result as above; can not shift beyond -1


--- BIT SHIFT LEFT ON POSITIVE INTEGERS ---
Expression: 8 = 4 << 1
 Decimal:
  val=4
  res=8
 Binary:
  val=00000000000000000000000000000100
  res=00000000000000000000000000001000
 NOTE: zeros fill in right side

Expression: 1073741824 = 4 << 28
 Decimal:
  val=4
  res=1073741824
 Binary:
  val=00000000000000000000000000000100
  res=01000000000000000000000000000000

Expression: -2147483648 = 4 << 29
 Decimal:
  val=4
  res=-2147483648
 Binary:
```

```
  val=00000000000000000000000000000100
  res=10000000000000000000000000000000
 NOTE: sign bits get shifted out

Expression: 0 = 4 << 30
 Decimal:
  val=4
  res=0
 Binary:
  val=00000000000000000000000000000100
  res=00000000000000000000000000000000
 NOTE: bits shift out left side


--- BIT SHIFT LEFT ON NEGATIVE INTEGERS ---
Expression: -8 = -4 << 1
 Decimal:
  val=-4
  res=-8
 Binary:
  val=11111111111111111111111111111100
  res=11111111111111111111111111111000
 NOTE: zeros fill in right side

Expression: -2147483648 = -4 << 29
 Decimal:
  val=-4
  res=-2147483648
 Binary:
  val=11111111111111111111111111111100
  res=10000000000000000000000000000000

Expression: 0 = -4 << 30
 Decimal:
  val=-4
  res=0
 Binary:
  val=11111111111111111111111111111100
  res=00000000000000000000000000000000
 NOTE: bits shift out left side, including sign bit
```

Output of the above example on 64 bit machines:

```


--- BIT SHIFT RIGHT ON POSITIVE INTEGERS ---
Expression: 2 = 4 >> 1
 Decimal:
  val=4
  res=2
 Binary:
  val=0000000000000000000000000000000000000000000000000000000000000100
  res=0000000000000000000000000000000000000000000000000000000000000010
```

```
 NOTE: copy of sign bit shifted into left side

Expression: 1 = 4 >> 2
 Decimal:
  val=4
  res=1
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000001

Expression: 0 = 4 >> 3
 Decimal:
  val=4
  res=0
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000000
 NOTE: bits shift out right side

Expression: 0 = 4 >> 4
 Decimal:
  val=4
  res=0
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000000
 NOTE: same result as above; can not shift beyond 0


--- BIT SHIFT RIGHT ON NEGATIVE INTEGERS ---
Expression: -2 = -4 >> 1
 Decimal:
  val=-4
  res=-2
 Binary:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111110
 NOTE: copy of sign bit shifted into left side

Expression: -1 = -4 >> 2
 Decimal:
  val=-4
  res=-1
 Binary:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111111
 NOTE: bits shift out right side

Expression: -1 = -4 >> 3
 Decimal:
  val=-4
  res=-1
 Binary:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111111
 NOTE: same result as above; can not shift beyond -1
```

```
--- BIT SHIFT LEFT ON POSITIVE INTEGERS ---
Expression: 8 = 4 << 1
 Decimal:
  val=4
  res=8
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000001000
 NOTE: zeros fill in right side

Expression: 4611686018427387904 = 4 << 60
 Decimal:
  val=4
  res=4611686018427387904
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=01000000000000000000000000000000000000000000000000000000000000000

Expression: -9223372036854775808 = 4 << 61
 Decimal:
  val=4
  res=-9223372036854775808
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=10000000000000000000000000000000000000000000000000000000000000000
 NOTE: sign bits get shifted out

Expression: 0 = 4 << 62
 Decimal:
  val=4
  res=0
 Binary:
  val=00000000000000000000000000000000000000000000000000000000000000100
  res=00000000000000000000000000000000000000000000000000000000000000000
 NOTE: bits shift out left side


--- BIT SHIFT LEFT ON NEGATIVE INTEGERS ---
Expression: -8 = -4 << 1
 Decimal:
  val=-4
  res=-8
 Binary:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=11111111111111111111111111111111111111111111111111111111111111000
 NOTE: zeros fill in right side

Expression: -9223372036854775808 = -4 << 61
 Decimal:
  val=-4
  res=-9223372036854775808
 Binary:
  val=11111111111111111111111111111111111111111111111111111111111111100
  res=10000000000000000000000000000000000000000000000000000000000000000

Expression: 0 = -4 << 62
 Decimal:
  val=-4
```

```
 res=0
Binary:
 val=11111111111111111111111111111111111111111111111111111111111111100
 res=00000000000000000000000000000000000000000000000000000000000000000
NOTE: bits shift out left side, including sign bit
```

> **Warning**
>
> Use functions from the gmp extension for bitwise manipulation on numbers beyond `PHP_INT_MAX`.

See Also

- pack()
- unpack()
- gmp_and()
- gmp_or()
- gmp_xor()
- gmp_testbit()
- gmp_clrbit()

Comparison

Comparison operators, as their name implies, allow you to compare two values. You may also be interested in viewing the type comparison tables, as they show examples of various type related comparisons.

Comparison Operators

Example	Name	Result
$a == $b	Equal	`true` if *$a* is equal to *$b* after type juggling.
$a === $b	Identical	`true` if *$a* is equal to *$b*, and they are of the same type.
$a != $b	Not equal	`true` if *$a* is not equal to *$b* after type juggling.
$a <> $b	Not equal	`true` if *$a* is not equal to *$b* after type juggling.
$a !== $b	Not identical	`true` if *$a* is not equal to *$b*, or they are not of the same type.
$a < $b	Less than	`true` if *$a* is strictly less than *$b*.
$a > $b	Greater than	`true` if *$a* is strictly greater than *$b*.
$a <= $b	Less than or equal to	`true` if *$a* is less than or equal to *$b*.
$a >= $b	Greater than or equal to	`true` if *$a* is greater than or equal to *$b*.
$a <=> $b	Spaceship	An int less than, equal to, or greater than zero when *$a* is less than, equal to, or greater than *$b*, respectively.

If both operands are numeric strings, or one operand is a number and the other one is a numeric string, then the comparison is done numerically. These rules also apply to the switch statement. The type conversion does not take place when the comparison is `===` or `!==` as this involves comparing the type as well as the value.

> **Warning**
>
> Prior to PHP 8.0.0, if a string is compared to a number or a numeric string then the string was converted to a number before performing the comparison. This can lead to surprising results as can be seen with the following example:

```php
<?php
var_dump(0 == "a");
var_dump("1" == "01");
var_dump("10" == "1e1");
var_dump(100 == "1e2");

switch ("a") {
case 0:
    echo "0";
    break;
case "a":
    echo "a";
    break;
}
?>
```

Output of the above example in PHP 7:

```
bool(true)
bool(true)
bool(true)
bool(true)
0
```

Output of the above example in PHP 8:

```
bool(false)
bool(true)
bool(true)
bool(true)
a
```

Example #1 Comparison Operators

```php
<?php
// Integers
echo 1 <=> 1, ' '; // 0
echo 1 <=> 2, ' '; // -1
echo 2 <=> 1, ' '; // 1
```

```php
// Floats
echo 1.5 <=> 1.5, ' '; // 0
echo 1.5 <=> 2.5, ' '; // -1
echo 2.5 <=> 1.5, ' '; // 1

// Strings
echo "a" <=> "a", ' '; // 0
echo "a" <=> "b", ' '; // -1
echo "b" <=> "a", ' '; // 1

echo "a" <=> "aa", ' ';  // -1
echo "zz" <=> "aa", ' '; // 1

// Arrays
echo [] <=> [], ' ';               // 0
echo [1, 2, 3] <=> [1, 2, 3], ' '; // 0
echo [1, 2, 3] <=> [], ' ';        // 1
echo [1, 2, 3] <=> [1, 2, 1], ' '; // 1
echo [1, 2, 3] <=> [1, 2, 4], ' '; // -1

// Objects
$a = (object) ["a" => "b"];
$b = (object) ["a" => "b"];
echo $a <=> $b, ' '; // 0

$a = (object) ["a" => "b"];
$b = (object) ["a" => "c"];
echo $a <=> $b, ' '; // -1

$a = (object) ["a" => "c"];
$b = (object) ["a" => "b"];
echo $a <=> $b, ' '; // 1

// not only values are compared; keys must match
$a = (object) ["a" => "b"];
$b = (object) ["b" => "b"];
echo $a <=> $b, ' '; // 1

?>
```

For various types, comparison is done according to the following table (in order).

Comparison with Various Types

Type of Operand 1	Type of Operand 2	Result
null or string	string	Convert `null` to "", numerical or lexical comparison
bool or null	anything	Convert both sides to bool, `false` < `true`
object	object	Built-in classes can define its own comparison, different classes are incomparable, same class see Object Comparison
string, resource, int or float	string, resource, int or float	Translate strings and resources to numbers, usual math

Type of Operand 1	Type of Operand 2	Result
array	array	Array with fewer members is smaller, if key from operand 1 is not found in operand 2 then arrays are incomparable, otherwise - compare value by value (see following example)
object	anything	object is always greater
array	anything	array is always greater

Example #2 Boolean/null comparison

```php
<?php
// Bool and null are compared as bool always
var_dump(1 == TRUE);  // TRUE - same as (bool) 1 == TRUE
var_dump(0 == FALSE); // TRUE - same as (bool) 0 == FALSE
var_dump(100 < TRUE); // FALSE - same as (bool) 100 < TRUE
var_dump(-10 < FALSE);// FALSE - same as (bool) -10 < FALSE
var_dump(min(-100, -10, NULL, 10, 100)); // NULL - (bool) NULL < (bool) -100 is FALSE < TRUE
?>
```

Example #3 Transcription of standard array comparison

```php
<?php
// Arrays are compared like this with standard comparison operators as well as the spaceship
operator.
function standard_array_compare($op1, $op2)
{
    if (count($op1) < count($op2)) {
        return -1; // $op1 < $op2
    } elseif (count($op1) > count($op2)) {
        return 1; // $op1 > $op2
    }
    foreach ($op1 as $key => $val) {
        if (!array_key_exists($key, $op2)) {
            return 1;
        } elseif ($val < $op2[$key]) {
            return -1;
        } elseif ($val > $op2[$key]) {
            return 1;
        }
    }
    return 0; // $op1 == $op2
}
?>
```

> **Warning**
>
> ### Comparison of floating point numbers
>
> Because of the way floats are represented internally, you should not test two floats for equality.
>
> See the documentation for float for more information.

> **Note** :
>
> Be aware that PHP's type juggling is not always obvious when comparing values of different types, particularly comparing ints to bools or ints to strings. It is therefore generally advisable to use `===` and `!==` comparisons rather than `==` and `!=` in most cases.

> **Note** :
>
> Encountering recursion during comparison, for example when comparing an array or object that contains a reference to itself, throws an Error as of PHP 8.4.0; previously, it resulted in a fatal error.

Incomparable Values

While identity comparison (`===` and `!==`) can be applied to arbitrary values, the other comparison operators should only be applied to comparable values. The result of comparing incomparable values is undefined, and should not be relied upon.

> **Note** :
>
> As of PHP 8.5.0, loosely comparing an object that is not comparable (such as an enum, CurlHandle, or another internal class) against a bool using `==` or `!=` consistently evaluates as if the object were cast to bool, that is `(bool) $object`. Because such an object always casts to **`true`**, it is equal to **`true`** and not equal to **`false`**. Prior to PHP 8.5.0, this only applied when comparing against a boolean literal (`$object == true`); comparing against a boolean value held in a variable always returned **`false`**. See the migration guide for details.

See Also

- strcasecmp()
- strcmp()
- Array operators
- Types

Ternary Operator

Another conditional operator is the "?:" (or ternary) operator.

Example #4 Assigning a default value

```php
<?php
// Example usage for: Ternary Operator
$action = (empty($_POST['action'])) ? 'default' : $_POST['action'];

// The above is identical to this if/else statement
if (empty($_POST['action'])) {
    $action = 'default';
} else {
    $action = $_POST['action'];
```

```
}
?>
```

The expression `(expr1) ? (expr2) : (expr3)` evaluates to expr2 if expr1 evaluates to `true`, and expr3 if expr1 evaluates to `false`.

It is possible to leave out the middle part of the ternary operator. Expression `expr1 ?: expr3` evaluates to the result of expr1 if expr1 evaluates to `true`, and expr3 otherwise. expr1 is only evaluated once in this case.

> **Note** :
>
> Please note that the ternary operator is an expression, and that it doesn't evaluate to a variable, but to the result of an expression. This is important to know if you want to return a variable by reference. The statement `return $var == 42 ? $a : $b;` in a return-by-reference function will therefore not work and a warning is issued.

> **Note** :
>
> It is recommended to avoid "stacking" ternary expressions. PHP's behaviour when using more than one unparenthesized ternary operator within a single expression is non-obvious compared to other languages. Indeed prior to PHP 8.0.0, ternary expressions were evaluated left-associative, instead of right-associative like most other programming languages. Relying on left-associativity is deprecated as of PHP 7.4.0. As of PHP 8.0.0, the ternary operator is non-associative.
>
> **Example #5 Non-obvious Ternary Behaviour**
>
> ```php
> <?php
> // on first glance, the following appears to output 'true'
> echo (true ? 'true' : false ? 't' : 'f');
>
> // however, the actual output of the above is 't' prior to PHP 8.0.0
> // this is because ternary expressions are left-associative
>
> // the following is a more obvious version of the same code as above
> echo ((true ? 'true' : false) ? 't' : 'f');
>
> // here, one can see that the first expression is evaluated to 'true', which
> // in turn evaluates to (bool) true, thus returning the true branch of the
> // second ternary expression.
> ?>
> ```

> **Note** :
>
> Chaining of short-ternaries (`?:`), however, is stable and behaves reasonably. It will evaluate to the first argument that evaluates to a non-falsy value. Note that undefined values will still raise a warning.
>
> **Example #6 Short-ternary chaining**
>
> ```php
> <?php
> echo 0 ?: 1 ?: 2 ?: 3, PHP_EOL; //1
> echo 0 ?: 0 ?: 2 ?: 3, PHP_EOL; //2
> ```

```
echo 0 ?: 0 ?: 0 ?: 3, PHP_EOL; //3
?>
```

Null Coalescing Operator

Another useful shorthand operator is the "??" (or null coalescing) operator.

Example #7 Assigning a default value

```
<?php
// Example usage for: Null Coalesce Operator
$action = $_POST['action'] ?? 'default';

// The above is identical to this if/else statement
if (isset($_POST['action'])) {
    $action = $_POST['action'];
} else {
    $action = 'default';
}
?>
```

The expression `(expr1) ?? (expr2)` evaluates to expr2 if expr1 is `null`, and expr1 otherwise.

In particular, this operator does not emit a notice or warning if the left-hand side value does not exist, just like isset(). This is especially useful on array keys.

> **Note** :
>
> Please note that the null coalescing operator is an expression, and that it doesn't evaluate to a variable, but to the result of an expression. This is important to know if you want to return a variable by reference. The statement `return $foo ?? $bar;` in a return-by-reference function will therefore not work and a warning is issued.

> **Note** :
>
> The null coalescing operator has low precedence. That means if mixing it with other operators (such as string concatenation or arithmetic operators) parentheses will likely be required.
>
> ```
> <?php
> // Raises a warning that $name is undefined.
> print 'Mr. ' . $name ?? 'Anonymous';
>
> // Prints "Mr. Anonymous"
> print 'Mr. ' . ($name ?? 'Anonymous');
> ?>
> ```

> **Note** :
>
> Please note that the null coalescing operator allows for simple nesting:
>
> **Example #8 Nesting null coalescing operator**

```php
<?php

$foo = null;
$bar = null;
$baz = 1;
$qux = 2;

echo $foo ?? $bar ?? $baz ?? $qux; // outputs 1

?>
```

Error Control

PHP supports one error control operator: the at sign (@). When prepended to an expression in PHP, any diagnostic error that might be generated by that expression will be suppressed.

If a custom error handler function is set with set_error_handler(), it will still be called even though the diagnostic has been suppressed.

> **Warning**
>
> Prior to PHP 8.0.0, the error_reporting() called inside the custom error handler always returned `0` if the error was suppressed by the `@` operator. As of PHP 8.0.0, it returns the value of this (bitwise) expression: `E_ERROR | E_CORE_ERROR | E_COMPILE_ERROR | E_USER_ERROR | E_RECOVERABLE_ERROR | E_PARSE`.

Any error message generated by the expression is available in the `"message"` element of the array returned by error_get_last(). The result of that function will change on each error, so it needs to be checked early.

Example #1 Intentional file error

```php
<?php
$my_file = @file ('non_existent_file') or
    die ("Failed opening file: error was '" . error_get_last()['message'] . "'");
?>
```

Example #2 Suppressing Expressions

```php
<?php
// this works for any expression, not just functions:
$value = @$cache[$key];
// will not issue a notice if the index $key doesn't exist.
?>
```

> **Note** :
>
> The @-operator works only on expressions. A simple rule of thumb is: if one can take the value of something, then one can prepend the `@` operator to it. For instance, it can be prepended to variables, functions calls, certain language construct calls (e.g. include), and so forth. It cannot be prepended to function or class definitions, or conditional structures such as `if` and `foreach`, and so forth.

> **Warning**

> Prior to PHP 8.0.0, it was possible for the `@` operator to disable critical errors that will terminate script execution. For example, prepending `@` to a call of a function which did not exist, by being unavailable or mistyped, would cause the script to terminate with no indication as to why.

See Also

- error_reporting()
- Error Handling and Logging functions

Execution

PHP supports one execution operator: backticks (``). Note that these are not single-quotes! PHP will attempt to execute the contents of the backticks as a shell command; the output will be returned (i.e., it won't simply be dumped to output; it can be assigned to a variable). Use of the backtick operator is identical to shell_exec().

Example #1 Backtick Operator

```
<?php
$output = `ls -al`;
echo "<pre>$output</pre>";
?>
```

> **Note** :
>
> The backtick operator is disabled when shell_exec() is disabled.

> **Note** :
>
> Unlike some other languages, backticks have no special meaning within double-quoted strings.

Changelog

Version	Description
8.5.0	The backtick operator as an alias for shell_exec() has been deprecated.

See Also

- Program Execution functions
- popen()
- proc_open()
- Using PHP from the commandline

Logic

Logical Operators

Example	Name	Result
$a and $b	And	`true` if both *$a* and *$b* are `true`.
$a or $b	Or	`true` if either *$a* or *$b* is `true`.

Example	Name	Result
$a xor $b	Xor	`true` if either *$a* or *$b* is `true`, but not both.
! $a	Not	`true` if *$a* is not `true`.
$a && $b	And	`true` if both *$a* and *$b* are `true`.
$a || $b	Or	`true` if either *$a* or *$b* is `true`.

The reason for the two different variations of "and" and "or" operators is that they operate at different precedences. (See Operator Precedence.)

Example #1 Logical operators illustrated

```php
<?php

// --------------------
// foo() will never get called as those operators are short-circuit

$a = (false && foo());
$b = (true  || foo());
$c = (false and foo());
$d = (true  or  foo());

// --------------------
// "||" has a greater precedence than "or"

// The result of the expression (false || true) is assigned to $e
// Acts like: ($e = (false || true))
$e = false || true;

// The constant false is assigned to $f before the "or" operation occurs
// Acts like: (($f = false) or true)
$f = false or true;

var_dump($e, $f);

// --------------------
// "&&" has a greater precedence than "and"

// The result of the expression (true && false) is assigned to $g
// Acts like: ($g = (true && false))
$g = true && false;

// The constant true is assigned to $h before the "and" operation occurs
// Acts like: (($h = true) and false)
$h = true and false;

var_dump($g, $h);
?>
```

The above example will output something similar to:

```
bool(true)
bool(false)
bool(false)
bool(true)
```

String

There are two string operators. The first is the concatenation operator ('.'), which returns the concatenation of its right and left arguments. The second is the concatenating assignment operator ('.='), which appends the argument on the right side to the argument on the left side. Please read Assignment Operators for more information.

Example #1 String Concatenating

```php
<?php
$a = "Hello ";
$b = $a . "World!"; // now $b contains "Hello World!"
var_dump($b);

$a = "Hello ";
$a .= "World!";     // now $a contains "Hello World!"
var_dump($a);
?>
```

See Also

- String type
- String functions

Array

Array Operators

Example	Name	Result
$a + $b	Union	Union of *$a* and *$b*.
$a == $b	Equality	`true` if *$a* and *$b* have the same key/value pairs.
$a === $b	Identity	`true` if *$a* and *$b* have the same key/value pairs in the same order and of the same types.
$a != $b	Inequality	`true` if *$a* is not equal to *$b*.
$a <> $b	Inequality	`true` if *$a* is not equal to *$b*.
$a !== $b	Non-identity	`true` if *$a* is not identical to *$b*.

The + operator returns the right-hand array appended to the left-hand array; for keys that exist in both arrays, the elements from the left-hand array will be used, and the matching elements from the right-hand array will be ignored.

Example #1 Array Append Operator

```php
<?php
$a = array("a" => "apple", "b" => "banana");
$b = array("a" => "pear", "b" => "strawberry", "c" => "cherry");

$c = $a + $b; // Union of $a and $b
echo "Union of \$a and \$b: \n";
var_dump($c);

$c = $b + $a; // Union of $b and $a
echo "Union of \$b and \$a: \n";
var_dump($c);

$a += $b; // Union of $a += $b is $a and $b
echo "Union of \$a += \$b: \n";
var_dump($a);
?>
```

The above example will output:

```
Union of $a and $b:
array(3) {
  ["a"]=>
  string(5) "apple"
  ["b"]=>
  string(6) "banana"
  ["c"]=>
  string(6) "cherry"
}
Union of $b and $a:
array(3) {
  ["a"]=>
  string(4) "pear"
  ["b"]=>
  string(10) "strawberry"
  ["c"]=>
  string(6) "cherry"
}
Union of $a += $b:
array(3) {
  ["a"]=>
  string(5) "apple"
  ["b"]=>
  string(6) "banana"
  ["c"]=>
  string(6) "cherry"
}
```

Elements of arrays are equal for the comparison if they have the same key and value.

Example #2 Comparing arrays

```php
<?php
$a = array("apple", "banana");
$b = array(1 => "banana", "0" => "apple");

var_dump($a == $b); // bool(true)
var_dump($a === $b); // bool(false)
?>
```

See Also

- Array type
- Array functions

Type

`instanceof` is used to determine whether a PHP variable is an instantiated object of a certain class:

Example #1 Using `instanceof` with classes

```php
<?php
class MyClass
{
}

class NotMyClass
{
}
$a = new MyClass;

var_dump($a instanceof MyClass);
var_dump($a instanceof NotMyClass);
?>
```

The above example will output:

```
bool(true)
bool(false)
```

`instanceof` can also be used to determine whether a variable is an instantiated object of a class that inherits from a parent class:

Example #2 Using `instanceof` with inherited classes

```php
<?php
class ParentClass
{
}

class MyClass extends ParentClass
{
```

```php
}

$a = new MyClass;

var_dump($a instanceof MyClass);
var_dump($a instanceof ParentClass);
?>
```

The above example will output:

```
bool(true)
bool(true)
```

To check if an object is *not* an instanceof a class, the logical `not` operator can be used.

Example #3 Using `instanceof` to check if object is *not* an instanceof a class

```php
<?php
class MyClass
{
}

$a = new MyClass;
var_dump(!($a instanceof stdClass));
?>
```

The above example will output:

```
bool(true)
```

Lastly, `instanceof` can also be used to determine whether a variable is an instantiated object of a class that implements an interface:

Example #4 Using `instanceof` with interfaces

```php
<?php
interface MyInterface
{
}

class MyClass implements MyInterface
{
}

$a = new MyClass;
```

```php
var_dump($a instanceof MyClass);
var_dump($a instanceof MyInterface);
?>
```

The above example will output:

```
bool(true)
bool(true)
```

Although `instanceof` is usually used with a literal classname, it can also be used with another object or a string variable:

Example #5 Using `instanceof` with other variables

```php
<?php
interface MyInterface
{
}

class MyClass implements MyInterface
{
}

$a = new MyClass;
$b = new MyClass;
$c = 'MyClass';
$d = 'NotMyClass';

var_dump($a instanceof $b); // $b is an object of class MyClass
var_dump($a instanceof $c); // $c is a string 'MyClass'
var_dump($a instanceof $d); // $d is a string 'NotMyClass'
?>
```

The above example will output:

```
bool(true)
bool(true)
bool(false)
```

instanceof does not throw any error if the variable being tested is not an object, it simply returns `false`. Constants, however, were not allowed prior to PHP 7.3.0.

Example #6 Using `instanceof` to test other variables

```php
<?php
$a = 1;
```

```php
$b = NULL;
$c = fopen('/tmp/', 'r');
var_dump($a instanceof stdClass); // $a is an integer
var_dump($b instanceof stdClass); // $b is NULL
var_dump($c instanceof stdClass); // $c is a resource
var_dump(FALSE instanceof stdClass);
?>
```

The above example will output:

```
bool(false)
bool(false)
bool(false)
PHP Fatal error:  instanceof expects an object instance, constant given
```

As of PHP 7.3.0, constants are allowed on the left-hand-side of the `instanceof` operator.

Example #7 Using `instanceof` to test constants

```php
<?php
var_dump(FALSE instanceof stdClass);
?>
```

Output of the above example in PHP 7.3:

```
bool(false)
```

As of PHP 8.0.0, `instanceof` can now be used with arbitrary expressions. The expression must be wrapped in parentheses and produce a string.

Example #8 Using `instanceof` with an arbitrary expression

```php
<?php

class ClassA extends \stdClass {}
class ClassB extends \stdClass {}
class ClassC extends ClassB {}
class ClassD extends ClassA {}

function getSomeClass(): string
{
    return ClassA::class;
}

var_dump(new ClassA instanceof ('std' . 'Class'));
var_dump(new ClassB instanceof ('Class' . 'B'));
```

```php
var_dump(new ClassC instanceof ('Class' . 'A'));
var_dump(new ClassD instanceof (getSomeClass()));
?>
```

Output of the above example in PHP 8:

```
bool(true)
bool(true)
bool(false)
bool(true)
```

The `instanceof` operator has a functional variant with the is_a() function.

See Also

- get_class()
- is_a()

Functional

PHP 8.5 and later supports one operator that works directly on callables. The `|>` operator, or "pipe," accepts a single-parameter callable on the right and passes the left-side value to it, evaluating to the callable's result. The callable on the right may be any valid PHP callable: a Closure, a first-class callable, an object that implements __invoke(), etc.

That means the following two lines are logically equivalent.

Example #1 Using `|>`

```php
<?php
$result = "Hello World" |> strlen(...);
echo $result, PHP_EOL;

$result = strlen("Hello World");
echo $result, PHP_EOL;
?>
```

The above example will output:

```
11
11
```

For a single call that is not especially useful. It becomes useful when multiple calls are chained together. That is, the following two code fragments are logically equivalent:

Example #2 Chaining |> calls

```php
<?php
$result = "PHP Rocks"
    |> htmlentities(...)
    |> str_split(...)
    |> (fn($x) => array_map(strtoupper(...), $x))
    |> (fn($x) => array_filter($x, fn($v) => $v != 'O'))
;
print_r($result);

$temp = "PHP Rocks";
$temp = htmlentities($temp);
$temp = str_split($temp);
$temp = array_map(strtoupper(...), $temp);
$temp = array_filter($temp, fn($v) => $v != 'O');
$result = $temp;
print_r($result);
?>
```

The above example will output:

```
Array
(
    [0] => P
    [1] => H
    [2] => P
    [3] =>
    [4] => R
    [6] => C
    [7] => K
    [8] => S
)
Array
(
    [0] => P
    [1] => H
    [2] => P
    [3] =>
    [4] => R
    [6] => C
    [7] => K
    [8] => S
)
```

The left-hand side of the pipe may be any value or expression. The right-hand side may be any valid PHP callable that takes a single parameter, or any expression that evaluates to such a callable. Functions with more than one required parameter are not allowed and will fail as if the function were called normally with insufficient arguments. Functions that take a variable by reference are not allowed. If the right-hand side does not evaluate to a valid callable it will throw an Error.

> **Note** :
>
> Be aware that, to avoid syntax ambiguity, arrow functions MUST be wrapped in parentheses when used with a pipe operator, as in the examples above. Failing to do so will result in a fatal error.

See Also

- Closure

Control Structures

Table of Contents

Introduction

Any PHP script is built out of a series of statements. A statement can be an assignment, a function call, a loop, a conditional statement or even a statement that does nothing (an empty statement). Statements usually end with a semicolon. In addition, statements can be grouped into a statement-group by encapsulating a group of statements with curly braces. A statement-group is a statement by itself as well. The various statement types are described in this chapter.

See Also

The following are also considered language constructs even though they are referenced under functions in the manual.

- list()
- array()
- echo
- eval()
- print

if

(PHP 4, PHP 5, PHP 7, PHP 8)

The `if` construct is one of the most important features of many languages, PHP included. It allows for conditional execution of code fragments. PHP features an `if` structure that is similar to that of C:

```
if (expr)
  statement
```

As described in the section about expressions, expression is evaluated to its Boolean value. If expression evaluates to **`true`**, PHP will execute statement, and if it evaluates to **`false`** - it'll ignore it. More information about what values evaluate to **`false`** can be found in the 'Converting to boolean' section.

The following example would display a is bigger than b if *$a* is bigger than *$b*:

```
<?php
$a = 3;
```

```php
$b = 1;
if ($a > $b)
  echo "a is bigger than b\n";
```

The above example will output:

```
a is bigger than b
```

Often you'd want to have more than one statement to be executed conditionally. Of course, there's no need to wrap each statement with an `if` clause. Instead, you can group several statements into a statement group. For example, this code would display a is bigger than b if *$a* is bigger than *$b*, and would then assign the value of *$a* into *$b*:

```php
<?php
$a = 3;
$b = 1;
if ($a > $b) {
  echo "a is bigger than b\n";
  $b = $a;
}
var_dump($b);
```

The above example will output:

```
a is bigger than b
int(3)
```

`If` statements can be nested infinitely within other `if` statements, which provides you with complete flexibility for conditional execution of the various parts of your program.

else

(PHP 4, PHP 5, PHP 7, PHP 8)

Often you'd want to execute a statement if a certain condition is met, and a different statement if the condition is not met. This is what `else` is for. `else` extends an `if` statement to execute a statement in case the expression in the `if` statement evaluates to **`false`**. For example, the following code would display a is greater than b if *$a* is greater than *$b*, and a is NOT greater than b otherwise:

```php
<?php
$a = 3;
$b = 5;
if ($a > $b) {
  echo "a is greater than b\n";
} else {
```

```
  echo "a is NOT greater than b\n";
}
```

The above example will output:

```
a is NOT greater than b
```

The `else` statement is only executed if the `if` expression evaluated to **`false`**, and if there were any `elseif` expressions - only if they evaluated to **`false`** as well (see elseif).

> **Note** : **Dangling else**
>
> In case of nested `if-else` statements, an `else` is always associated with the nearest `if`.
>
> ```
> <?php
> $a = false;
> $b = true;
> if ($a)
>     if ($b)
>         echo "b";
> else
>     echo "c";
> ```
>
> The above example will output:
>
> ```
> 
> ```
>
> Despite the indentation (which does not matter for PHP), the `else` is associated with the `if ($b)`, so the example does not produce any output. While relying on this behavior is valid, it is recommended to avoid it by using curly braces to resolve potential ambiguities.

elseif/else if

(PHP 4, PHP 5, PHP 7, PHP 8)

`elseif`, as its name suggests, is a combination of `if` and `else`. Like `else`, it extends an `if` statement to execute a different statement in case the original `if` expression evaluates to **`false`**. However, unlike `else`, it will execute that alternative expression only if the `elseif` conditional expression evaluates to **`true`**. For example, the following code would display a is bigger than b, a equal to b or a is smaller than b:

```
<?php
$a = 1;
$b = 1;
if ($a > $b) {
    echo "a is bigger than b";
} elseif ($a == $b) {
```

```
    echo "a is equal to b";
} else {
    echo "a is smaller than b";
}
```

The above example will output:

```
a is equal to b
```

There may be several `elseif`s within the same `if` statement. The first `elseif` expression (if any) that evaluates to **`true`** would be executed. In PHP, it's possible to write `else if` (in two words) and the behavior would be identical to that of `elseif` (in a single word). The syntactic meaning is slightly different (the same behavior as C) but the bottom line is that both would result in exactly the same behavior.

The `elseif` statement is only executed if the preceding `if` expression and any preceding `elseif` expressions evaluated to **`false`**, and the current `elseif` expression evaluated to **`true`**.

> **Note** :
>
> Note that `elseif` and `else if` will only be considered exactly the same when using curly brackets as in the above example. When using a colon to define `if`/`elseif` conditions, the use of `elseif` in a single word becomes necessary. PHP will fail with a parse error if `else if` is split into two words.

```
<?php
$a = 3;
$b = 1;

/* Incorrect Method: */
if ($a > $b):
    echo $a." is greater than ".$b;
else if ($a == $b): // Will not compile.
    echo "The above line causes a parse error.";
endif;
```

The above example will output:

```
Parse error: syntax error, unexpected token "if", expecting ":" in script on line 8
```

```
<?php
$a = 3;
$b = 1;

/* Correct Method: */
if ($a > $b):
```

```
    echo $a." is greater than ".$b;
elseif ($a == $b): // Note the combination of the words.
    echo $a." equals ".$b;
else:
    echo $a." is neither greater than or equal to ".$b;
endif;
```

The above example will output:

```
3 is greater than 1
```

Alternative syntax for control structures

(PHP 4, PHP 5, PHP 7, PHP 8)

PHP offers an alternative syntax for some of its control structures; namely, `if`, `while`, `for`, `foreach`, `switch`, and `declare`. In each case, the basic form of the alternate syntax is to change the opening brace to a colon (:) and the closing brace to `endif;`, `endwhile;`, `endfor;`, `endforeach;`, `endswitch;`, or `enddeclare;`, respectively.

```
<?php
$a = 5;
if ($a == 5): ?>
A is equal to 5
<?php endif; ?>
```

The above example will output:

```
A is equal to 5
```

In the above example, the HTML block "A is equal to 5" is nested within an `if` statement written in the alternative syntax. The HTML block would be displayed only if *$a* is equal to 5.

The alternative syntax applies to `else` and `elseif` as well. The following is an `if` structure with `elseif` and `else` in the alternative format:

```
<?php
$a = 5;
if ($a == 5):
    echo "a equals 5";
    echo "...";
elseif ($a == 6):
    echo "a equals 6";
    echo "!!!";
else:
```

```
    echo "a is neither 5 nor 6";
endif;
```

The above example will output:

```
a equals 5...
```

> **Note** :
>
> Mixing syntaxes in the same control block is not supported.

> **Warning**
>
> Any output (including whitespace) between a `switch` statement and the first `case` will result in a syntax error. For example, this is invalid:
>
> ```
> <?php
> $foo = 1;
> switch ($foo): ?>
>     <?php case 1: ?>
>     Foo is 1
> <?php endswitch; ?>
> ```
>
> The above example will output:
>
> ```
> Parse error: syntax error, unexpected T_INLINE_HTML "    ", expecting "endswitch" or "case"
> or "default" in script on line 4
> ```
>
> Whereas this is valid, as the trailing newline after the `switch` statement is considered part of the closing `?>` and hence nothing is output between the `switch` and `case`:
>
> ```
> <?php
> $foo = 1;
> switch ($foo): ?>
> <?php case 1: ?>
>     Foo is 1
> <?php endswitch; ?>
> ```
>
> The above example will output:
>
> ```
>     Foo is 1
> ```

See also while, for, and if for further examples.

while

(PHP 4, PHP 5, PHP 7, PHP 8)

`while` loops are the simplest type of loop in PHP. They behave just like their C counterparts. The basic form of a `while` statement is:

```
while (expr)
    statement
```

The meaning of a `while` statement is simple. It tells PHP to execute the nested statement(s) repeatedly, as long as the `while` expression evaluates to **`true`**. The value of the expression is checked each time at the beginning of the loop, so even if this value changes during the execution of the nested statement(s), execution will not stop until the end of the iteration (each time PHP runs the statements in the loop is one iteration). If the `while` expression evaluates to **`false`** from the very beginning, the nested statement(s) won't even be run once.

Like with the `if` statement, you can group multiple statements within the same `while` loop by surrounding a group of statements with curly braces, or by using the alternate syntax:

```
while (expr):
    statement
    ...
endwhile;
```

The following examples are identical, and both print the numbers 1 through 10:

```
<?php
$i = 1;
while ($i <= 10) {
    echo $i++ . "\n";  /* the printed value would be
                          $i before the increment
                          (post-increment) */
}
```

The above example will output:

```
1
2
3
4
5
6
7
8
9
10
```

```php
<?php
$i = 1;
while ($i <= 10):
    echo $i . "\n";
    $i++;
endwhile;
```

The above example will output:

```
1
2
3
4
5
6
7
8
9
10
```

do-while

(PHP 4, PHP 5, PHP 7, PHP 8)

`do-while` loops are very similar to `while` loops, except the truth expression is checked at the end of each iteration instead of at the beginning. The main difference from regular `while` loops is that the first iteration of a `do-while` loop is guaranteed to run (the truth expression is only checked at the end of the iteration), whereas it may not necessarily run with a regular `while` loop (the truth expression is checked at the beginning of each iteration; if it evaluates to `false` right from the beginning, the loop execution would end immediately).

There is just one syntax for `do-while` loops:

```php
<?php
$i = 0;
do {
    echo $i;
} while ($i > 0);
```

The above example will output:

```
0
```

The above loop would run one time exactly, since after the first iteration, when the truth expression is checked, it evaluates to `false` (*$i* is not bigger than 0) and the loop execution ends.

Advanced C users may be familiar with a different usage of the `do-while` loop, to allow stopping execution in the middle of code blocks, by encapsulating them with `do-while` (0), and using the `break` statement. The following code fragment demonstrates this:

```
<?php
$i = 50;
$factor = 0.5;
$minimum_limit = 20;
do {
    if ($i < 5) {
        echo "i is not big enough\n";
        break;
    }
    $i *= $factor;
    if ($i < $minimum_limit) {
        echo "i is less than minimum limit after factor\n";
        break;
    }
    echo $i ." is ok\n";

    /* process i */

} while (0);
```

The above example will output:

```
25 is ok
```

It is possible to use the `goto` operator instead of this hack.

for

(PHP 4, PHP 5, PHP 7, PHP 8)

`for` loops are the most complex loops in PHP. They behave like their C counterparts. The syntax of a `for` loop is:

```
for (expr1; expr2; expr3)
    statement
```

The first expression (*expr1*) is evaluated (executed) once unconditionally at the beginning of the loop.

At the beginning of each iteration, *expr2* is evaluated. If it evaluates to `true`, the loop continues and the nested statement(s) are executed. If it evaluates to `false`, the execution of the loop ends.

At the end of each iteration, *expr3* is evaluated (executed).

Each of the expressions can be empty or contain multiple expressions separated by commas. In *expr2*, all expressions separated by a comma are evaluated but the result is taken from the last part. *expr2* being empty means the loop should be run indefinitely (PHP implicitly considers it as `true`, like C). This may not be as useless as you might think, since often you'd want to end the loop using a conditional `break` statement instead of using the `for` truth expression.

Consider the following examples. All of them display the numbers 1 through 10:

```
<?php
for ($i = 1; $i <= 10; $i++) {
    echo $i . " ";
}
```

The above example will output:

```
1 2 3 4 5 6 7 8 9 10
```

```
<?php
for ($i = 1; ; $i++) {
    if ($i > 10) {
        break;
    }
    echo $i . " ";
}
```

The above example will output:

```
1 2 3 4 5 6 7 8 9 10
```

```
<?php
$i = 1;
for (; ; ) {
    if ($i > 10) {
        break;
    }
    echo $i . " ";
    $i++;
}
```

The above example will output:

```
1 2 3 4 5 6 7 8 9 10
```

```php
<?php
for ($i = 1, $j = 0; $i <= 10; $j += $i, print $i . " ", $i++);
```

The above example will output:

```
1 2 3 4 5 6 7 8 9 10
```

Of course, the first example appears to be the nicest one (or perhaps the fourth), but you may find that being able to use empty expressions in `for` loops comes in handy on many occasions.

PHP also supports the alternate "colon syntax" for `for` loops.

```
for (expr1; expr2; expr3):
    statement
    ...
endfor;
```

Expressions *expr2* and *expr3* are evaluated every iteration. It's advisable to use simple expressions in these places to avoid performance issues. For example, if the number of iterations is known in advance, it is better to use a variable instead of a function call in *expr2*:

```php
<?php

$people = ['Kalle', 'Pierre'];

// Bad practice: calling a function on every iteration
for($i = 0; $i < getIterationCount($people); ++$i) {
    $people[$i] .= ' is cool';
}

// Good practice: storing the count in a variable
for($i = 0, $peopleCount = getIterationCount($people); $i < $peopleCount; ++$i) {
    $people[$i] .= ' is cool';
}
```

In the example above, the function is called on every iteration of the first loop, even though it always returns the same value. Storing that value in a variable, as the second loop does, calls the function once. Note that the number of iterations is then fixed: if the array is modified inside the loop, the stored value no longer reflects its size.

> **Note** :
>
> The size of an array is stored with the array, which means that calling the built-in function count() does not require counting the elements and does not cause performance issues. This does not apply to

> **COUNT_RECURSIVE**, which walks the whole array. Care should also be taken when using count() on objects implementing Countable, as such calls can be more expensive.

> **Note** :
>
> The `for` loop is not the recommended way to iterate over arrays. The `foreach` loop is specifically designed for this purpose and is usually more convenient.

foreach

(PHP 4, PHP 5, PHP 7, PHP 8)

The `foreach` construct provides an easy way to iterate over arrays and objects. `foreach` will issue an error when used with a variable containing a different data type or with an uninitialized variable.

`foreach` can optionally get the `key` of each element:

```
foreach (iterable_expression as $value) {
    statement_list
}

foreach (iterable_expression as $key => $value) {
    statement_list
}
```

The first form traverses the value given by `iterable_expression`. On each iteration, the value of the current element is assigned to `$value`.

The second form will additionally assign the current element's key to the `$key` variable on each iteration.

Note that `foreach` does not modify the internal array pointer, which is used by functions such as current() and key().

By default, an object is traversed over the properties that are visible from the current scope. It is possible to customize object iteration.

Example #1 Common `foreach` usages

```
<?php

echo "Value only\n";
$array = [1, 2, 3, 17];

foreach ($array as $value) {
    echo "Current element of \$array: $value.\n";
}

echo "\n\nKey and value:\n";
$array = [
    "one" => 1,
    "two" => 2,
    "three" => 3,
    "seventeen" => 17
```

```php
];

foreach ($array as $key => $value) {
    echo "Key: $key => Value: $value\n";
}

echo "\n\nMulti-dimensional key-value arrays:\n";
$grid = [];
$grid[0][0] = "a";
$grid[0][1] = "b";
$grid[1][0] = "y";
$grid[1][1] = "z";

foreach ($grid as $y => $row) {
    foreach ($row as $x => $value) {
        echo "Value at position x=$x and y=$y: $value\n";
    }
}

echo "\n\nDynamic arrays:\n";
foreach (range(1, 5) as $value) {
    echo "$value\n";
}
```

The above example will output:

```
Value only
Current element of $array: 1.
Current element of $array: 2.
Current element of $array: 3.
Current element of $array: 17.


Key and value:
Key: one => Value: 1
Key: two => Value: 2
Key: three => Value: 3
Key: seventeen => Value: 17


Multi-dimensional key-value arrays:
Value at position x=0 and y=0: a
Value at position x=1 and y=0: b
Value at position x=0 and y=1: y
Value at position x=1 and y=1: z


Dynamic arrays:
1
2
3
4
5
```

> **Note** :
>
> `foreach` does not support the ability to suppress error messages using the `@`.

Unpacking nested arrays

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

It is possible to iterate over an array of arrays and unpack the nested array into loop variables by using either array destructuring via `[]` or by using the list() language construct as the value.

> **Note** :
>
> Please note that array destructuring via `[]` is only possible as of PHP 7.1.0.

In both of the following examples `$a` will be set to the first element of the nested array and `$b` will contain the second element:

```
<?php
$array = [
    [1, 2],
    [3, 4],
];

foreach ($array as [$a, $b]) {
    echo "A: $a; B: $b\n";
}

foreach ($array as list($a, $b)) {
    echo "A: $a; B: $b\n";
}
```

The above example will output:

```
A: 1; B: 2
A: 3; B: 4
A: 1; B: 2
A: 3; B: 4
```

When providing fewer variables than there are elements in the array, the remaining elements will be ignored. Similarly, elements can be skipped over by using a comma:

```
<?php
$array = [
    [1, 2, 5],
```

```
    [3, 4, 6],
];

foreach ($array as [$a, $b]) {
    // Note that there is no $c here.
    echo "$a $b\n";
}

foreach ($array as [, , $c]) {
    // Skipping over $a and $b
    echo "$c\n";
}
```

The above example will output:

```
1 2
3 4
5
6
```

A notice will be generated if there aren't enough array elements to fill the list():

```
<?php
$array = [
    [1, 2],
    [3, 4],
];

foreach ($array as [$a, $b, $c]) {
    echo "A: $a; B: $b; C: $c\n";
}
```

The above example will output:

```
Notice: Undefined offset: 2 in example.php on line 7
A: 1; B: 2; C:

Notice: Undefined offset: 2 in example.php on line 7
A: 3; B: 4; C:
```

foreach and references

It is possible to directly modify array elements within a loop by preceding `$value` with `&`. In that case the value will be assigned by reference.

```php
<?php
$arr = [1, 2, 3, 4];
foreach ($arr as &$value) {
    $value = $value * 2;
}
// $arr is now [2, 4, 6, 8]
var_dump($arr);
unset($value); // break the reference with the last element
```

The above example will output:

```
array(4) {
  [0]=>
  int(2)
  [1]=>
  int(4)
  [2]=>
  int(6)
  [3]=>
  &int(8)
}
```

> **Warning**
>
> A reference to the `$value` of the last array element remains even after the `foreach` loop. It is recommended to destroy it using unset(). Otherwise, the following behavior will occur:

```php
<?php
$arr = [1, 2, 3, 4];
foreach ($arr as &$value) {
    $value = $value * 2;
}
// $arr is now [2, 4, 6, 8]
var_dump($arr);

// without an unset($value), $value is still a reference to the last item: $arr[3]

echo "\nAnother loop:\n";
foreach ($arr as $key => $value) {
    // $arr[3] will be updated with each value from $arr...
    echo "{$key} => {$value}\n";
}
// ...until ultimately the second-to-last value is copied onto the last value
var_dump($arr);
```

The above example will output:

```
array(4) {
  [0]=>
```

```
  int(2)
  [1]=>
  int(4)
  [2]=>
  int(6)
  [3]=>
  &int(8)
}

Another loop:
0 => 2
1 => 4
2 => 6
3 => 6
array(4) {
  [0]=>
  int(2)
  [1]=>
  int(4)
  [2]=>
  int(6)
  [3]=>
  &int(6)
}
```

Example #2 Iterate a constant array's values by reference

```php
<?php
foreach ([1, 2, 3, 4] as &$value) {
    $value = $value * 2;
    echo $value . "\n";
}
```

The above example will output:

```
2
4
6
8
```

See Also

- array
- Traversable
- iterable
- list()

break

(PHP 4, PHP 5, PHP 7, PHP 8)

`break` ends execution of the current `for`, `foreach`, `while`, `do-while` or `switch` structure.

`break` accepts an optional numeric argument which tells it how many nested enclosing structures are to be broken out of. The default value is `1`, only the immediate enclosing structure is broken out of.

```php
<?php
$arr = array('one', 'two', 'three', 'four', 'stop', 'five');
foreach ($arr as $val) {
    if ($val == 'stop') {
        break;    /* You could also write 'break 1;' here. */
    }
    echo "$val\n";
}
```

The above example will output:

```
one
two
three
four
```

Using the optional argument:

```php
<?php
$i = 0;
while (++$i) {
    switch ($i) {
        case 5:
            echo "At 5\n";
            break 1;  /* Exit only the switch. */
        case 10:
            echo "At 10; quitting\n";
            break 2;  /* Exit the switch and the while. */
        default:
            break;
    }
    echo "$i\n";
}
```

The above example will output:

```
1
2
3
4
At 5
5
```

```
6
7
8
9
At 10; quitting
```

continue

(PHP 4, PHP 5, PHP 7, PHP 8)

`continue` is used within looping structures to skip the rest of the current loop iteration and continue execution at the condition evaluation and then the beginning of the next iteration.

> **Note** :
>
> In PHP the switch statement is considered a looping structure for the purposes of `continue`. `continue` behaves like `break` (when no arguments are passed) but will raise a warning as this is likely to be a mistake. If a `switch` is inside a loop, `continue 2` will continue with the next iteration of the outer loop.

`continue` accepts an optional numeric argument which tells it how many levels of enclosing loops it should skip to the end of. The default value is `1`, thus skipping to the end of the current loop.

```php
<?php
$arr = ['zero', 'one', 'two', 'three', 'four', 'five', 'six'];
foreach ($arr as $key => $value) {
    if (0 === ($key % 2)) { // skip members with even key
        continue;
    }
    echo $value . "\n";
}
```

The above examples will output:

```
one
three
five
```

```php
<?php
$i = 0;
while ($i++ < 5) {
    echo "Outer\n";
    while (1) {
        echo "Middle\n";
        while (1) {
            echo "Inner\n";
            continue 3;
        }
```

```
        echo "This never gets output.\n";
    }
    echo "Neither does this.\n";
}
```

The above examples will output:

```
Outer
Middle
Inner
Outer
Middle
Inner
Outer
Middle
Inner
Outer
Middle
Inner
Outer
Middle
Inner
```

Changelog for `continue`

Version	Description
7.3.0	`continue` within a `switch` that is attempting to act like a `break` statement for the `switch` will trigger an `E_WARNING`.

switch

(PHP 4, PHP 5, PHP 7, PHP 8)

The `switch` statement is similar to a series of `if` statements on the same expression. In situations where it is necessary to compare the same variable or expression with many different values, and execute a different piece of code depending on which value it equals, the `switch` statement is often more convenient and easier to read.

> **Note** :
>
> Note that unlike some other languages, the continue statement applies to `switch` and acts similarly to `break`. If you have a `switch` inside a loop and wish to continue to the next iteration of the outer loop, use `continue 2`.

> **Note** :
>
> Note that switch/case does a loose comparison.

In the following example, each code block is equivalent. One uses a series of `if` and `elseif` statements, and the other a `switch` statement. In each case, the output is the same.

Example #1 `switch` structure

```
<?php
$i = 1;

// This switch statement:

switch ($i) {
    case 0:
        echo "i equals 0\n";
        break;
    case 1:
        echo "i equals 1\n";
        break;
    case 2:
        echo "i equals 2\n";
        break;
}

// Is equivalent to:
if ($i == 0) {
    echo "i equals 0\n";
} elseif ($i == 1) {
    echo "i equals 1\n";
} elseif ($i == 2) {
    echo "i equals 2\n";
}
```

The above example will output:

```
i equals 1
i equals 1
```

It is important to understand how the `switch` statement is executed in order to avoid mistakes. The `switch` statement executes line by line (actually, statement by statement). In the beginning, no code is executed. Only when a `case` statement is found whose expression evaluates to a value that matches the value of the `switch` expression does PHP begin to execute the statements. PHP continues to execute the statements until the end of the `switch` block, or the first time it sees a `break` statement. If you don't write a `break` statement at the end of a case's statement list, PHP will go on executing the statements of the following case. For example:

```
<?php
$i = 1;

switch ($i) {
    case 0:
        echo "i equals 0\n";
    case 1:
        echo "i equals 1\n";
```

```
    case 2:
        echo "i equals 2\n";
}
```

The above example will output:

```
i equals 1
i equals 2
```

Here, if *$i* is equal to 0, PHP would execute all of the echo statements! If *$i* is equal to 1, PHP would execute the last two echo statements. You would get the expected behavior ('i equals 2' would be displayed) only if *$i* is equal to 2. Thus, it is important not to forget `break` statements (even though you may want to avoid supplying them on purpose under certain circumstances).

In a `switch` statement, the condition is evaluated only once and the result is compared to each `case` statement. In an `elseif` statement, the condition is evaluated again. If your condition is more complicated than a simple comparison and/or is in a tight loop, a `switch` may be faster.

The statement list for a case can also be empty, which simply passes control into the statement list for the next case.

```
<?php
$i = 1;

switch ($i) {
    case 0:
    case 1:
    case 2:
        echo "i is less than 3 but not negative";
        break;
    case 3:
        echo "i is 3";
}
```

The above example will output:

```
i is less than 3 but not negative
```

A special case is the `default` case. This case matches anything that wasn't matched by the other cases. For example:

```
<?php
$i = 4;

switch ($i) {
```

```
    case 0:
        echo "i equals 0";
        break;
    case 1:
        echo "i equals 1";
        break;
    case 2:
        echo "i equals 2";
        break;
    default:
       echo "i is not equal to 0, 1 or 2";
}
```

The above example will output:

```
i is not equal to 0, 1 or 2
```

> **Note** :
>
> Multiple default cases will raise an `E_COMPILE_ERROR` error.

> **Note** :
>
> Technically the `default` case may be listed in any order. It will only be used if no other case matches. However, by convention it is best to place it at the end as the last branch.

If no `case` branch matches, and there is no `default` branch, then no code will be executed, just as if no `if` statement was true.

A case value may be given as an expression. However, that expression will be evaluated on its own and then loosely compared with the switch value. That means it cannot be used for complex evaluations of the switch value. For example:

```
<?php
$target = 1;
$start = 3;

switch ($target) {
    case $start - 1:
        print "A";
        break;
    case $start - 2:
        print "B";
        break;
    case $start - 3:
        print "C";
        break;
    case $start - 4:
```

```
        print "D";
        break;
}
```

The above example will output:

```
B
```

For more complex comparisons, the value `true` may be used as the switch value. Or, alternatively, `if-else` blocks can be used instead of `switch`.

```
<?php
$offset = 1;
$start = 3;

switch (true) {
    case $start - $offset === 1:
        print "A";
        break;
    case $start - $offset === 2:
        print "B";
        break;
    case $start - $offset === 3:
        print "C";
        break;
    case $start - $offset === 4:
        print "D";
        break;
}
```

The above example will output:

```
B
```

The alternative syntax for control structures is supported with switches. For more information, see Alternative syntax for control structures.

```
<?php
$i = 1;

switch ($i):
    case 0:
        echo "i equals 0";
        break;
    case 1:
        echo "i equals 1";
```

```
        break;
    case 2:
        echo "i equals 2";
        break;
    default:
        echo "i is not equal to 0, 1 or 2";
endswitch;
```

The above example will output:

```
i equals 1
```

It's possible to use a semicolon instead of a colon after a case. As of PHP 8.5.0, this syntax is deprecated.

```
<?php
$beer = 'cola';

switch($beer)
{
    case 'tuborg';
    case 'carlsberg';
    case 'stella';
    case 'heineken';
        echo 'Good choice';
        break;
    default;
        echo 'Please make a new selection...';
        break;
}
```

The above example will output:

```
Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 6

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 7

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 8

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 9

Deprecated: Case statements followed by a semicolon (;) are deprecated, use a colon (:) instead
in script on line 12
Please make a new selection...
```

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.5.0. Relying on this feature is highly discouraged.

Changelog

Version	Description
8.5.0	The alternate syntax using a semicolon after `case` has been deprecated.

See Also

- match

match

(PHP 8)

The `match` expression branches evaluation based on an identity check of a value. Similarly to a `switch` statement, a `match` expression has a subject expression that is compared against multiple alternatives. Unlike `switch`, it will evaluate to a value much like ternary expressions. Unlike `switch`, the comparison is an identity check (`===`) rather than a weak equality check (`==`). Match expressions are available as of PHP 8.0.0.

Example #1 Structure of a `match` expression

```
<?php
$return_value = match (subject_expression) {
    single_conditional_expression => return_expression,
    conditional_expression1, conditional_expression2 => return_expression,
};
```

Example #2 Basic `match` usage

```
<?php
$food = 'cake';

$return_value = match ($food) {
    'apple' => 'This food is an apple',
    'bar' => 'This food is a bar',
    'cake' => 'This food is a cake',
};

var_dump($return_value);
```

The above example will output:

```
string(19) "This food is a cake"
```

Example #3 Example of using `match` with comparison operators

```php
<?php
$age = 18;

$output = match (true) {
    $age < 2 => "Baby",
    $age < 13 => "Child",
    $age <= 19 => "Teenager",
    $age >= 40 => "Adult",
    $age > 19 => "Young adult",
};

var_dump($output);
```

The above example will output:

```
string(8) "Teenager"
```

> **Note** :
>
> The result of a `match` expression does not need to be used.

> **Note** :
>
> When a `match` expression is used as a standalone expression, it *must* be terminated by a semicolon `;`.

The `match` expression is similar to a `switch` statement but has some key differences:

- A `match` arm compares values strictly (`===`) instead of loosely as the switch statement does.
- A `match` expression returns a value.
- `match` arms do not fall through to later cases the way `switch` statements do.
- A `match` expression must be exhaustive.

Like `switch` statements, `match` expressions are executed match arm by match arm. In the beginning, no code is executed. The conditional expressions are only evaluated if all previous conditional expressions failed to match the subject expression. Only the return expression corresponding to the matching conditional expression will be evaluated. For example:

```php
<?php
$result = match ($x) {
    foo() => 'value',
    $this->bar() => 'value', // $this->bar() isn't called if foo() === $x
    $this->baz => beep(), // beep() isn't called unless $x === $this->baz
```

```
    // etc.
};
```

`match` expression arms may contain multiple expressions separated by a comma. That is a logical OR, and is a shorthand for multiple match arms with the same right-hand side.

```
<?php
$result = match ($x) {
    // This match arm:
    $a, $b, $c => 5,
    // Is equivalent to these three match arms:
    $a => 5,
    $b => 5,
    $c => 5,
};
```

A special case is the `default` pattern. This pattern matches anything that wasn't previously matched. For example:

```
<?php
$expressionResult = match ($condition) {
    1, 2 => foo(),
    3, 4 => bar(),
    default => baz(),
};
```

> **Note** :
>
> Multiple default patterns will raise an **`E_FATAL_ERROR`** error.

A `match` expression must be exhaustive. If the subject expression is not handled by any match arm, an UnhandledMatchError is thrown.

Example #4 Example of an unhandled match expression

```
<?php
$condition = 5;

try {
    match ($condition) {
        1, 2 => foo(),
        3, 4 => bar(),
    };
} catch (\UnhandledMatchError $e) {
    var_dump($e);
}
```

The above example will output:

```
object(UnhandledMatchError)#1 (7) {
  ["message":protected]=>
```

```
  string(22) "Unhandled match case 5"
  ["string":"Error":private]=>
  string(0) ""
  ["code":protected]=>
  int(0)
  ["file":protected]=>
  string(6) "script"
  ["line":protected]=>
  int(5)
  ["trace":"Error":private]=>
  array(0) {
  }
  ["previous":"Error":private]=>
  NULL
}
```

Using match expressions to handle non-identity checks

It is possible to use a `match` expression to handle non-identity conditional cases by using `true` as the subject expression.

Example #5 Using a generalized match expression to branch on integer ranges

```php
<?php
$age = 23;

$result = match (true) {
    $age >= 65 => 'senior',
    $age >= 25 => 'adult',
    $age >= 18 => 'young adult',
    default => 'kid',
};

var_dump($result);
```

The above example will output:

```
string(11) "young adult"
```

Example #6 Using a generalized match expression to branch on string content

```php
<?php

$text = 'Bienvenue chez nous';

$result = match (true) {
    str_contains($text, 'Welcome'), str_contains($text, 'Hello') => 'en',
    str_contains($text, 'Bienvenue'), str_contains($text, 'Bonjour') => 'fr',
    // ...
```

```
};

var_dump($result);
```

The above example will output:

```
string(2) "fr"
```

declare

(PHP 4, PHP 5, PHP 7, PHP 8)

The `declare` construct is used to set execution directives for a block of code. The syntax of `declare` is similar to the syntax of other flow control constructs:

```
declare (directive)
    statement
```

The `directive` section allows the behavior of the `declare` block to be set. Currently only three directives are recognized:

- `ticks`
- `encoding`
- `strict_types`

As directives are handled as the file is being compiled, only literals may be given as directive values. Variables and constants cannot be used. To illustrate:

```
<?php
// This is valid:
declare(ticks=1);

// This is invalid:
const TICK_VALUE = 1;
declare(ticks=TICK_VALUE);
```

The `statement` part of the `declare` block will be executed - how it is executed and what side effects occur during execution may depend on the directive set in the `directive` block.

The `declare` construct can also be used in the global scope, affecting all code following it (however if the file with `declare` was included then it does not affect the parent file).

```
<?php
// these are the same:

// you can use this:
declare(ticks=1) {
    // entire script here
}
```

```
// or you can use this:
declare(ticks=1);
// entire script here
```

Ticks

A tick is an event that occurs for every *N* low-level tickable statements executed by the parser within the `declare` block. The value for *N* is specified using `ticks=N` within the `declare` block's `directive` section.

Not all statements are tickable. Typically, condition expressions and argument expressions are not tickable.

The event(s) that occur on each tick are specified using the register_tick_function(). See the example below for more details. Note that more than one event can occur for each tick.

Example #1 Tick usage example

```
<?php

declare(ticks=1);

// A function called on each tick event
function tick_handler()
{
    echo "tick_handler() called\n";
}

register_tick_function('tick_handler'); // causes a tick event

$a = 1; // causes a tick event

if ($a > 0) {
    $a += 2; // causes a tick event
    print $a . "\n"; // causes a tick event
}
```

The above example will output:

```
tick_handler() called
tick_handler() called
tick_handler() called
3
tick_handler() called
```

See also register_tick_function() and unregister_tick_function().

Encoding

A script's encoding can be specified per-script using the `encoding` directive.

Example #2 Declaring an encoding for the script

```php
<?php
declare(encoding='ISO-8859-1');
// code here
```

> **Caution**
>
> When combined with namespaces, the only legal syntax for declare is `declare(encoding='...');` where `...` is the encoding value. `declare(encoding='...') {}` will result in a parse error when combined with namespaces.

See also zend.script_encoding.

return

(PHP 4, PHP 5, PHP 7, PHP 8)

`return` returns program control to the calling module. Execution resumes at the expression following the called module's invocation.

If called from within a function, the `return` statement immediately ends execution of the current function, and returns its argument as the value of the function call. `return` also ends the execution of an eval() statement or script file.

If called from the global scope, then execution of the current script file is ended. If the current script file was included or required, then control is passed back to the calling file. Furthermore, if the current script file was included, then the value given to `return` will be returned as the value of the include call. If `return` is called from within the main script file, then script execution ends. If the current script file was named by the auto_prepend_file or auto_append_file configuration options in *php.ini*, then that script file's execution is ended.

For more information, see Returning values.

> **Note** :
>
> Note that since `return` is a language construct and not a function, the parentheses surrounding its argument are not required and their use is discouraged.

> **Note** :
>
> If no parameter is supplied, then the parentheses must be omitted and **`null`** will be returned. Calling `return` with parentheses but with no arguments will result in a parse error.

As of PHP 7.1.0, return statements without an argument in functions which declare a return type trigger **`E_COMPILE_ERROR`**, unless the return type is void, in which case return statements with an argument trigger that error.

require

(PHP 4, PHP 5, PHP 7, PHP 8)

`require` is identical to include except upon failure it will also produce an Error exception (**`E_COMPILE_ERROR`** level error prior to PHP 8.0.0) whereas include will only produce a warning (**`E_WARNING`** level error).

See the include documentation for how this works.

include

(PHP 4, PHP 5, PHP 7, PHP 8)

The `include` expression includes and evaluates the specified file.

The documentation below also applies to require.

Files are included based on the file path given or, if none is given, the include_path specified. If the file isn't found in the include_path, `include` will finally check in the calling script's own directory and the current working directory before failing. The `include` construct will emit an **`E_WARNING`** if it cannot find a file; this is different behavior from require, which will emit an **`E_ERROR`**.

Note that both `include` and `require` raise additional **`E_WARNING`**s, if the file cannot be accessed, before raising the final **`E_WARNING`** or **`E_ERROR`**, respectively.

If a path is defined — whether absolute (starting with a drive letter or `\` on Windows, or `/` on Unix/Linux systems) or relative to the current directory (starting with `.` or `..`) — the include_path will be ignored altogether. For example, if a filename begins with `../`, the parser will look in the parent directory to find the requested file.

For more information on how PHP handles including files and the include path, see the documentation for include_path.

When a file is included, the code it contains inherits the variable scope of the line on which the include occurs. Any variables available at that line in the calling file will be available within the called file, from that point forward. However, all functions and classes defined in the included file have the global scope.

Example #1 Basic `include` example

```
vars.php
<?php

$color = 'green';
$fruit = 'apple';

?>

test.php
<?php

echo "A $color $fruit"; // A

include 'vars.php';

echo "A $color $fruit"; // A green apple

?>
```

If the include occurs inside a function within the calling file, then all of the code contained in the called file will behave as though it had been defined inside that function. So, it will follow the variable scope of that function. An exception to this rule is magic constants which are evaluated by the parser before the include occurs.

Example #2 Including within functions

```php
<?php

function foo()
{
    global $color;

    include 'vars.php';

    echo "A $color $fruit";
}

/* vars.php is in the scope of foo() so     *
* $fruit is NOT available outside of this  *
* scope.  $color is because we declared it *
* as global.                               */

foo();                    // A green apple
echo "A $color $fruit";   // A green
```

When a file is included, parsing drops out of PHP mode and into HTML mode at the beginning of the target file, and resumes again at the end. For this reason, any code inside the target file which should be executed as PHP code must be enclosed within valid PHP start and end tags.

If "URL include wrappers" are enabled in PHP, you can specify the file to be included using a URL (via HTTP or other supported wrapper - see Supported Protocols and Wrappers for a list of protocols) instead of a local pathname. If the target server interprets the target file as PHP code, variables may be passed to the included file using a URL request string as used with HTTP GET. This is not strictly speaking the same thing as including the file and having it inherit the parent file's variable scope; the script is actually being run on the remote server and the result is then being included into the local script.

Example #3 `include` through HTTP

```php
<?php

/* This example assumes that www.example.com is configured to parse .php
* files and not .txt files. Also, 'Works' here means that the variables
* $foo and $bar are available within the included file. */

// Won't work; file.txt wasn't handled by www.example.com as PHP
include 'http://www.example.com/file.txt?foo=1&bar=2';

// Won't work; looks for a file named 'file.php?foo=1&bar=2' on the
// local filesystem.
include 'file.php?foo=1&bar=2';

// Works.
include 'http://www.example.com/file.php?foo=1&bar=2';
```

> **Warning**

> **Security warning**
>
> A remote file may be processed at the remote server (depending on the file extension and whether or not the remote server runs PHP) but it still has to produce a valid PHP script because it will be processed at the local server. If the file from the remote server should be processed there and outputted only, readfile() is a much better function to use. Otherwise, special care should be taken to secure the remote script to produce a valid and desired code.

See also Remote files, fopen() and file() for related information.

Handling Returns: `include` returns **`false`** on failure and raises a warning. Successful includes, unless overridden by the included file, return `1`. It is possible to execute a return statement inside an included file in order to terminate processing in that file and return to the script which called it. Also, it's possible to return values from included files. You can take the value of the include call as you would for a normal function. This is not, however, possible when including remote files unless the output of the remote file has valid PHP start and end tags (as with any local file). You can declare the needed variables within those tags and they will be introduced at whichever point the file was included.

Because `include` is a special language construct, parentheses are not needed around its argument. Take care when comparing the return value.

Example #4 Comparing return value of include

```
<?php
// won't work, evaluated as include(('vars.php') == TRUE), i.e. include('1')
if (include('vars.php') == TRUE) {
    echo 'OK';
}

// works
if ((include 'vars.php') == TRUE) {
    echo 'OK';
}
```

Example #5 `include` and the return statement

```
return.php
<?php

$var = 'PHP';

return $var;

?>

noreturn.php
<?php

$var = 'PHP';

?>

testreturns.php
<?php
```

```php
$foo = include 'return.php';

echo $foo; // prints 'PHP'

$bar = include 'noreturn.php';

echo $bar; // prints 1

?>
```

`$bar` is the value `1` because the include was successful. Notice the difference between the above examples. The first uses return within the included file while the other does not. If the file can't be included, **`false`** is returned and **`E_WARNING`** is issued.

If there are functions defined in the included file, they can be used in the main file regardless of whether they are before return or after. If the file is included twice, PHP will raise a fatal error because the functions were already declared. It is recommended to use include_once instead of checking if the file was already included and conditionally returning inside the included file.

Another way to "include" a PHP file into a variable is to capture the output by using the Output Control Functions with `include`. For example:

Example #6 Using output buffering to include a PHP file into a string

```php
<?php
$string = get_include_contents('somefile.php');

function get_include_contents($filename) {
    if (is_file($filename)) {
        ob_start();
        include $filename;
        return ob_get_clean();
    }
    return false;
}
```

In order to automatically include files within scripts, see also the auto_prepend_file and auto_append_file configuration options in *php.ini*.

> **Note** :
>
> Because this is a language construct and not a function, it cannot be called using variable functions, or named arguments.

See also require, require_once, include_once, get_included_files(), readfile(), virtual(), and include_path.

require_once

(PHP 4, PHP 5, PHP 7, PHP 8)

The `require_once` expression is identical to require except PHP will check if the file has already been included, and if so, not include (require) it again.

See the include_once documentation for information about the _once behaviour, and how it differs from its non _once siblings.

include_once

(PHP 4, PHP 5, PHP 7, PHP 8)

The include_once expression includes and evaluates the specified file during the execution of the script. This behavior is similar to the include expression, with the only difference being that if the code from a file has already been included, it will not be included again, and include_once returns **true**. As the name suggests, the file will be included just once.

include_once may be used in cases where the same file might be included and evaluated more than once during a particular execution of a script, so in this case it may help avoid problems such as function redefinitions, variable value reassignments, etc.

See the include documentation for information about how this function works.

goto

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

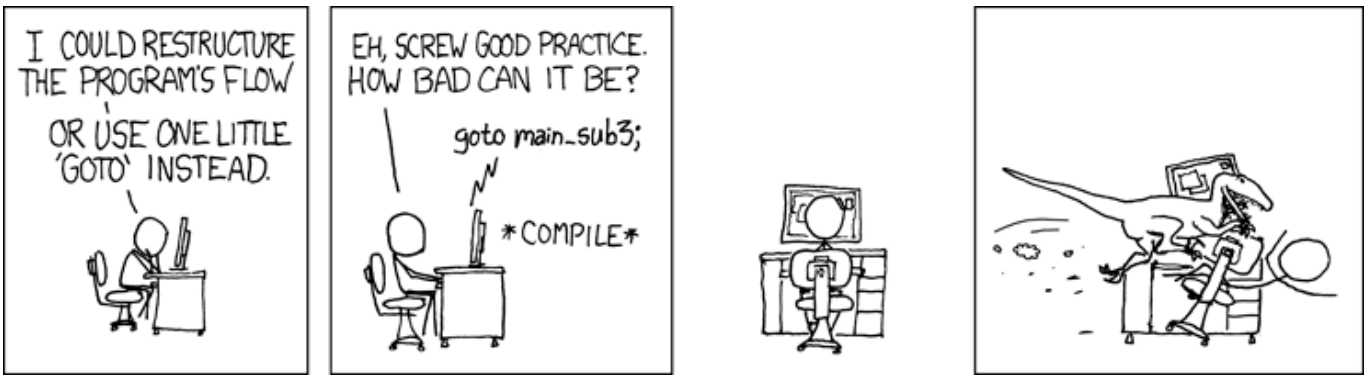


Image courtesy of » xkcd

The goto operator can be used to jump to another section in the program. The target point is specified by a *case-sensitive* label followed by a colon, and the instruction is given as goto followed by the desired target label. This is not a full unrestricted goto. The target label must be within the same file and context, meaning that you cannot jump out of a function or method, nor can you jump into one. You also cannot jump into any sort of loop or switch structure. You may jump out of these, and a common use is to use a goto in place of a multi-level break.

Example #1 goto example

```php
<?php
goto a;
echo 'Foo';

a:
echo 'Bar';
```

The above example will output:

```
Bar
```

Example #2 `goto` loop example

```php
<?php
for ($i = 0, $j = 50; $i < 100; $i++) {
    while ($j--) {
        if ($j == 17) {
            goto end;
        }
    }
}
echo "i = $i";
end:
echo 'j hit 17';
```

The above example will output:

```
j hit 17
```

Example #3 This will not work

```php
<?php
goto loop;
for ($i = 0, $j = 50; $i < 100; $i++) {
    while ($j--) {
        loop:
    }
}
echo "$i = $i";
```

The above example will output something similar to:

```
Fatal error: 'goto' into loop or switch statement is disallowed in script on line 2
```

Functions

Table of Contents

User-defined functions

A function is defined using the `function` keyword, a name, a list of parameters (which might be empty) separated by commas (`,`) enclosed in parentheses, followed by the body of the function enclosed in curly braces, such as the following:

Example #1 Declaring a new function named `foo`

```php
<?php
function foo($arg_1, $arg_2, /* ..., */ $arg_n)
{
    echo "Example function.\n";
    return $retval;
}
```

> **Note** :
>
> As of PHP 8.0.0, the list of parameters may have a trailing comma:
>
> ```php
> <?php
> function foo($arg_1, $arg_2,) { }
> ```

Any valid PHP code may appear inside the body of a function, even other functions and class definitions.

Function names follow the same rules as other labels in PHP. A valid function name starts with a letter or underscore, followed by any number of letters, numbers, or underscores. As a regular expression, it would be expressed thus: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`.

> **Tip**
>
> See also the Userland Naming Guide.

Functions need not be defined before they are referenced, *except* when a function is conditionally defined as shown in the two examples below.

When a function is defined in a conditional manner such as the two examples shown. Its definition must be processed *prior* to being called.

Example #2 Conditional functions

```php
<?php
$makefoo = true;

/* We can't call foo() from here
   since it doesn't exist yet,
   but we can call bar() */
if (function_exists('foo')) {
    foo();
} else {
    echo "Function foo does not exist (yet).\n";
}

bar();
```

```php
if ($makefoo) {
  function foo()
  {
    echo "I don't exist until program execution reaches me.\n";
  }
}

/* Now we can safely call foo()
   since $makefoo evaluated to true */

if ($makefoo) foo();

function bar()
{
  echo "I exist immediately upon program start.\n";
}
```

Example #3 Functions within functions

```php
<?php
function foo()
{
  function bar()
  {
    echo "I don't exist until foo() is called.\n";
  }
}

/* We can't call bar() yet
   since it doesn't exist. */
if (function_exists('bar')) {
    bar();
}

foo();

/* Now we can call bar(),
   foo()'s processing has
   made it accessible. */

bar();
```

All functions and classes in PHP have the global scope - they can be called outside a function even if they were defined inside and vice versa.

PHP does not support function overloading, nor is it possible to undefine or redefine previously-declared functions.

> **Note** :
>
> Function names are case-insensitive for the ASCII characters A to Z, though it is usually good form to call functions as they appear in their declaration.

Both variable number of arguments and default arguments are supported in functions. See also the function references for func_num_args(), func_get_arg(), and func_get_args() for more information.

It is possible to call recursive functions in PHP.

Example #4 Recursive functions

```php
<?php
function recursion($a)
{
    if ($a < 20) {
        echo "$a\n";
        recursion($a + 1);
    }
}

recursion(17);
```

> **Note** :
>
> Recursive function/method calls with over 100-200 recursion levels can smash the stack and cause a termination of the current script. Especially, infinite recursion is considered a programming error.

Function parameters and arguments

The function parameters are declared in the function signature. Information may be passed to functions via the argument list, which is a comma-delimited list of expressions. The arguments are evaluated from left to right and the result is assigned to the parameters of the function, before the function is actually called (*eager* evaluation).

PHP supports passing arguments by value (the default), passing by reference, and default argument values. Variable-length argument lists and Named Arguments are also supported.

> **Note** :
>
> As of PHP 7.3.0, it is possible to have a trailing comma in the argument list for a function calls:
>
> ```php
> <?php
> $v = foo(
>     $arg_1,
>     $arg_2,
> );
> ```

As of PHP 8.0.0, the list of function parameters may include a trailing comma, which will be ignored. That is particularly useful in cases where the list of parameters is long or contains long variable names, making it convenient to list parameters vertically.

Example #1 Function parameter list with trailing comma

```php
<?php
function takes_many_args(
```

```
    $first_arg,
    $second_arg,
    $a_very_long_argument_name,
    $arg_with_default = 5,
    $again = 'a default string', // This trailing comma was not permitted before 8.0.0.
)
{
    // ...
}
```

Passing arguments by reference

By default, function arguments are passed by value (so that if the value of the argument within the function is changed, it does not get changed outside of the function). To allow a function to modify its arguments, they must be passed by reference.

To have an argument to a function always passed by reference, prepend an ampersand (&) to the parameter name in the function definition:

Example #2 Passing function arguments by reference

```
<?php
function add_some_extra(&$string)
{
    $string .= 'and something extra.';
}
$str = 'This is a string, ';
add_some_extra($str);
echo $str;    // outputs 'This is a string, and something extra.'
```

It is an error to pass a constant expression as an argument to a parameter that expects to be passed by reference.

Default parameter values

A function may define default values for parameters using syntax similar to assigning a variable. The default is used only when the parameter's argument is not passed. Note that passing `null` does *not* assign the default value.

Example #3 Use of default parameters in functions

```
<?php
function makecoffee($type = "cappuccino")
{
    return "Making a cup of $type.\n";
}
echo makecoffee();
echo makecoffee(null);
echo makecoffee("espresso");
```

The above example will output:

```
```

```
Making a cup of cappuccino.
Making a cup of .
Making a cup of espresso.
```

Default parameter values may be scalar values, arrays, the special type `null`, and as of PHP 8.1.0, objects using the new ClassName() syntax.

Example #4 Using non-scalar types as default values

```php
<?php
function makecoffee($types = array("cappuccino"), $coffeeMaker = NULL)
{
    $device = is_null($coffeeMaker) ? "hands" : $coffeeMaker;
    return "Making a cup of ".join(", ", $types)." with $device.\n";
}
echo makecoffee();
echo makecoffee(array("cappuccino", "lavazza"), "teapot");
```

The above example will output:

```
Making a cup of cappuccino with hands.
Making a cup of cappuccino, lavazza with teapot.
```

Example #5 Using objects as default values (as of PHP 8.1.0)

```php
<?php
class DefaultCoffeeMaker {
    public function brew() {
        return "Making coffee.\n";
    }
}
class FancyCoffeeMaker {
    public function brew() {
        return "Crafting a beautiful coffee just for you.\n";
    }
}
function makecoffee($coffeeMaker = new DefaultCoffeeMaker)
{
    return $coffeeMaker->brew();
}
echo makecoffee();
echo makecoffee(new FancyCoffeeMaker);
```

The above example will output:

```
Making coffee.
Crafting a beautiful coffee just for you.
```

The default value must be a constant expression, not (for example) a variable, a class member or a function call.

Note that any optional parameters should be specified after any required parameters, otherwise they cannot be omitted from calls. Consider the following example:

Example #6 Incorrect usage of default function parameters

```php
<?php
function makeyogurt($container = "bowl", $flavour)
{
    return "Making a $container of $flavour yogurt.\n";
}

echo makeyogurt("raspberry"); // "raspberry" is $container, not $flavour
```

The above example will output:

```
Deprecated: makeyogurt(): Optional parameter $container declared before required parameter
$flavour is implicitly treated as a required parameter in script on line 2

Fatal error: Uncaught ArgumentCountError: Too few arguments to function makeyogurt(), 1 passed in
script on line 7 and exactly 2 expected in script:2
Stack trace:
#0 script(7): makeyogurt('raspberry')
#1 {main}
  thrown in script on line 2
```

Now, compare the above with this:

Example #7 Correct usage of default function parameters

```php
<?php
function makeyogurt($flavour, $container = "bowl")
{
    return "Making a $container of $flavour yogurt.\n";
}

echo makeyogurt("raspberry"); // "raspberry" is $flavour
```

The above example will output:

```
Making a bowl of raspberry yogurt.
```

As of PHP 8.0.0, named arguments can be used to skip over multiple optional parameters.

Example #8 Correct usage of default function parameters

```php
<?php
function makeyogurt($container = "bowl", $flavour = "raspberry", $style = "Greek")
{
    return "Making a $container of $flavour $style yogurt.\n";
}

echo makeyogurt(style: "natural");
```

The above example will output:

```
Making a bowl of raspberry natural yogurt.
```

As of PHP 8.0.0, declaring mandatory parameters after optional parameters is *deprecated*. This can generally be resolved by dropping the default value, since it will never be used. One exception to this rule are parameters of the form `Type $param = null`, where the `null` default makes the type implicitly nullable. This usage is deprecated as of PHP 8.4.0, and an explicit nullable type should be used instead.

Example #9 Declaring optional parameters after mandatory parameters

```php
<?php
function foo($a = [], $b) {}     // Default not used; deprecated as of PHP 8.0.0
function foo($a, $b) {}          // Functionally equivalent, no deprecation notice

function bar(A $a = null, $b) {} // As of PHP 8.1.0, $a is implicitly required
                                 // (because it comes before the required one),
                                 // but implicitly nullable (deprecated as of PHP 8.4.0),
                                 // because the default parameter value is null
function bar(?A $a, $b) {}       // Recommended
```

> **Note** :
>
> As of PHP 7.1.0, omitting a parameter which does not specify a default throws an ArgumentCountError; in previous versions it raised a Warning.

> **Note** :
>
> Parameters that expect the argument by reference may have a default value.

Variable-length argument lists

PHP has support for variable-length argument lists in user-defined functions by using the `...` token.

Parameter lists may include the `...` token to denote that the function accepts a variable number of arguments. The arguments will be passed into the given variable as an array:

Example #10 Using `...` to access variable arguments

```php
<?php
function sum(...$numbers) {
    $acc = 0;
    foreach ($numbers as $n) {
        $acc += $n;
    }
    return $acc;
}

echo sum(1, 2, 3, 4);
```

The above example will output:

```
10
```

`...` can also be used when calling functions to unpack an array or Traversable variable or literal into the argument list:

Example #11 Using `...` to provide arguments

```php
<?php
function add($a, $b) {
    return $a + $b;
}

echo add(...[1, 2])."\n";

$a = [1, 2];
echo add(...$a);
```

The above example will output:

```
3
3
```

You may specify normal positional parameters before the `...` token. In this case, only the trailing arguments that don't match a positional argument will be added to the array generated by `...`.

It is also possible to add a type declaration before the `...` token. If this is present, then all arguments captured by `...` must match that parameter type.

Example #12 Type declared variable arguments

```php
<?php
function total_intervals($unit, DateInterval ...$intervals) {
    $time = 0;
    foreach ($intervals as $interval) {
        $time += $interval->$unit;
    }
    return $time;
}

$a = new DateInterval('P1D');
$b = new DateInterval('P2D');
echo total_intervals('d', $a, $b).' days';

// This will fail, since null isn't a DateInterval object.
echo total_intervals('d', null);
```

The above example will output:

```
3 days
Fatal error: Uncaught TypeError: total_intervals(): Argument #2 must be of type DateInterval,
null given, called in script on line 15 and defined in script:2
Stack trace:
#0 script(15): total_intervals('d', NULL)
#1 {main}
  thrown in script on line 2
```

Finally, variable arguments can also be passed by reference by prefixing the `...` with an ampersand (`&`).

Named Arguments

PHP 8.0.0 introduced named arguments as an extension of the existing positional parameters. Named arguments allow passing arguments to a function based on the parameter name, rather than the parameter position. This makes the meaning of the argument self-documenting, makes the arguments order-independent and allows skipping default values arbitrarily.

Named arguments are passed by prefixing the value with the parameter name followed by a colon. Using reserved keywords as parameter names is allowed. The parameter name must be an identifier, specifying dynamically is not allowed.

Example #13 Named argument syntax

```php
<?php
myFunction(paramName: $value);
array_foobar(array: $value);
```

```
// NOT supported.
function_name($variableStoringParamName: $value);
```

Example #14 Positional arguments versus named arguments

```
<?php
// Using positional arguments:
array_fill(0, 100, 50);

// Using named arguments:
array_fill(start_index: 0, count: 100, value: 50);
```

The order in which the named arguments are passed does not matter.

Example #15 Same example as above with a different order of parameters

```
<?php
array_fill(value: 50, count: 100, start_index: 0);
```

Named arguments can be combined with positional arguments. In this case, the named arguments must come after the positional arguments. It is also possible to specify only some of the optional arguments of a function, regardless of their order.

Example #16 Combining named arguments with positional arguments

```
<?php
htmlspecialchars($string, double_encode: false);
// Same as
htmlspecialchars($string, ENT_QUOTES | ENT_SUBSTITUTE | ENT_HTML401, 'UTF-8', false);
```

Passing an argument to the same named parameter multiple times results in an Error exception.

Example #17 Error thrown when passing an argument to the same named parameter multiple times

```
<?php
function foo($param) { ... }

foo(param: 1, param: 2);
// Error: Named parameter $param overwrites previous argument

foo(1, param: 2);
// Error: Named parameter $param overwrites previous argument
```

As of PHP 8.1.0, it is possible to use named arguments after unpacking the arguments. A named argument *must not* override an already unpacked argument.

Example #18 Use named arguments after unpacking

```
<?php
function foo($a, $b, $c = 3, $d = 4) {
  return $a + $b + $c + $d;
}

var_dump(foo(...[1, 2], d: 40)); // 46
```

```php
var_dump(foo(...['b' => 2, 'a' => 1], d: 40)); // 46

var_dump(foo(...[1, 2], b: 20)); // Fatal error. Named parameter $b overwrites previous argument
```

Returning values

Values are returned by using the optional return statement. Any type may be returned, including arrays and objects. This causes the function to end its execution immediately and pass control back to the line from which it was called. See return for more information.

> **Note** :
>
> If the return is omitted the value `null` will be returned.

Use of return

Example #1 Use of return

```php
<?php
function square($num)
{
    return $num * $num;
}
echo square(4);   // outputs '16'.
```

A function can not return multiple values, but similar results can be obtained by returning an array.

Example #2 Returning an array to get multiple values

```php
<?php
function small_numbers()
{
    return [0, 1, 2];
}
// Array destructuring will collect each member of the array individually
[$zero, $one, $two] = small_numbers();
var_dump($zero, $one, $two);

// Prior to 7.1.0, the only equivalent alternative is using list() construct
list($zero, $one, $two) = small_numbers();
```

To return a reference from a function, use the reference operator & in both the function declaration and when assigning the returned value to a variable:

Example #3 Returning a reference from a function

```php
<?php
function &returns_reference()
{
    return $someref;
}
```

```
$newref =& returns_reference();
```

For more information on references, please check out References Explained.

Variable functions

PHP supports the concept of variable functions. This means that if a variable name has parentheses appended to it, PHP will look for a function with the same name as whatever the variable evaluates to, and will attempt to execute it. Among other things, this can be used to implement callbacks, function tables, and so forth.

Variable functions won't work with language constructs such as echo, print, unset(), isset(), empty(), include, require and the like. Utilize wrapper functions to make use of any of these constructs as variable functions.

Example #1 Variable function example

```
<?php
function foo() {
    echo "In foo()\n";
}

function bar($arg = '')
{
    echo "In bar(); argument was '$arg'.\n";
}

// This is a wrapper function around echo
function echoit($string)
{
    echo $string;
}

$func = 'foo';
$func();        // This calls foo()

$func = 'bar';
$func('test');  // This calls bar()

$func = 'echoit';
$func('test');  // This calls echoit()
```

Object methods can also be called with the variable functions syntax.

Example #2 Variable method example

```
<?php
class Foo
{
    function Variable()
    {
        $name = 'Bar';
        $this->$name(); // This calls the Bar() method
    }

    function Bar()
    {
```

```
        echo "This is Bar";
    }
}

$foo = new Foo();
$funcname = "Variable";
$foo->$funcname();  // This calls $foo->Variable()
```

When calling static methods, the function call is stronger than the static property operator:

Example #3 Variable method example with static properties

```
<?php
class Foo
{
    static $variable = 'static property';
    static function Variable()
    {
        echo "Method Variable called\n";
    }
}

echo Foo::$variable ."\n"; // This prints 'static property'. It does need a $variable in this scope.
$variable = "Variable";
Foo::$variable();  // This calls $foo->Variable() reading $variable in this scope.
```

The above example will output:

```
static property
Method Variable called
```

Example #4 Complex callables

```
<?php
class Foo
{
    static function bar()
    {
        echo "bar\n";
    }
    function baz()
    {
        echo "baz\n";
    }
}

$func = array("Foo", "bar");
$func(); // prints "bar"
$func = array(new Foo, "baz");
$func(); // prints "baz"
```

```php
$func = "Foo::bar";
$func(); // prints "bar"
```

See Also

- is_callable()
- call_user_func()
- function_exists()
- variable variables

Internal (built-in) functions

PHP comes standard with many functions and constructs. There are also functions that require specific PHP extensions compiled in, otherwise fatal "undefined function" errors will appear. For example, to use image functions such as imagecreatetruecolor(), PHP must be compiled with GD support. Or, to use mysqli_connect(), PHP must be compiled with MySQLi support. There are many core functions that are included in every version of PHP, such as the string and variable functions. A call to phpinfo() or get_loaded_extensions() will show which extensions are loaded into PHP. Also note that many extensions are enabled by default and that the PHP manual is split up by extension. See the configuration, installation, and individual extension chapters, for information on how to set up PHP.

Reading and understanding a function's prototype is explained within the manual section titled how to read a function definition. It's important to realize what a function returns or if a function works directly on a passed in value. For example, str_replace() will return the modified string while usort() works on the actual passed in variable itself. Each manual page also has specific information for each function like information on function parameters, behavior changes, return values for both success and failure, and availability information. Knowing these important (yet often subtle) differences is crucial for writing correct PHP code.

> **Note** :
>
> If the parameters given to a function are not what it expects, such as passing an array where a string is expected, the return value of the function is undefined. In this case it will likely return `null` but this is just a convention, and cannot be relied upon. As of PHP 8.0.0, a TypeError exception is supposed to be thrown in this case.

> **Note** :
>
> Scalar types for built-in functions are nullable by default in coercive mode. As of PHP 8.1.0, passing `null` to an internal function parameter that is not declared nullable is discouraged and emits a deprecation notice in coercive mode to align with the behavior of user-defined functions, where scalar types need to be marked as nullable explicitly.
>
> For example, strlen() function expects the parameter `$string` to be a non-nullable string. For historical reasons, PHP allows passing `null` for this parameter in coercive mode, and the parameter is implicitly cast to string, resulting in a `""` value. In contrast, a TypeError is emitted in strict mode.
>
> ```php
> <?php
> var_dump(strlen(null));
> // "Deprecated: Passing null to parameter #1 ($string) of type string is deprecated" as of PHP
> 8.1.0
> ```

```
// int(0)

var_dump(str_contains("foobar", null));
// "Deprecated: Passing null to parameter #2 ($needle) of type string is deprecated" as of PHP
8.1.0
// bool(true)
```

See Also

- function_exists()
- the function reference
- get_extension_funcs()
- dl()

Anonymous functions

Anonymous functions, also known as `closures`, allow the creation of functions which have no specified name. They are most useful as the value of callable parameters, but they have many other uses.

Anonymous functions are implemented using the Closure class.

Example #1 Anonymous function example

```
<?php
echo preg_replace_callback('~-([a-z])~', function ($match) {
    return strtoupper($match[1]);
}, 'hello-world');
// outputs helloWorld
```

Closures can also be used as the values of variables; PHP automatically converts such expressions into instances of the Closure internal class. Assigning a closure to a variable uses the same syntax as any other assignment, including the trailing semicolon:

Example #2 Anonymous function variable assignment example

```
<?php
$greet = function($name) {
    printf("Hello %s\r\n", $name);
};

$greet('World');
$greet('PHP');
```

Closures may also inherit variables from the parent scope. Any such variables must be passed to the `use` language construct. As of PHP 7.1, these variables must not include superglobals, *$this*, or variables with the same name as a parameter. A return type declaration of the function has to be placed *after* the `use` clause.

Example #3 Inheriting variables from the parent scope

```
<?php
// A variable defined in the parent scope
$message = 'hello';

// $message is not inherited and is undefined due to the
```

```php
// lack of a "use" clause
$example = function () {
    var_dump($message);
};
$example();

// $message is inherited from the parent scope due to the
// "use" clause
$example = function () use ($message) {
    var_dump($message);
};
$example();

// Inherited variable's value is from when the function
// is defined, not when called
$message = 'world';
$example();

// Reset $message
$message = 'hello';

// Inheriting by reference: a change made in the parent scope
// after the closure is defined is visible inside it
$example = function () use (&$message) {
    var_dump($message);
};
$example();

$message = 'world';
$example();

// Closures can also accept regular arguments
$example = function ($arg) use ($message) {
    var_dump($arg . ' ' . $message);
};
$example("hello");

// Closures can return values like any other function; the
// return type declaration comes after the use clause
$example = function () use ($message): array {
    return ["hello", $message];
};
var_dump($example());
```

The above example will output:

```
Warning: Undefined variable $message in script on line 8
NULL
string(5) "hello"
string(5) "hello"
string(5) "hello"
string(5) "world"
string(11) "hello world"
array(2) {
  [0]=>
```

```
  string(5) "hello"
  [1]=>
  string(5) "world"
}
```

As of PHP 8.0.0, the list of scope-inherited variables may include a trailing comma, which will be ignored.

Inheriting variables from the parent scope is *not* the same as using global variables. Global variables exist in the global scope, which is the same no matter what function is executing. The parent scope of a closure is the function in which the closure was declared (not necessarily the function it was called from). See the following example:

Example #4 Closures and scoping

```php
<?php
// A basic shopping cart which contains a list of added products
// and the quantity of each product. Includes a method which
// calculates the total price of the items in the cart using a
// closure as a callback.
class Cart
{
    const PRICE_BUTTER  = 1.00;
    const PRICE_MILK    = 3.00;
    const PRICE_EGGS    = 6.95;

    protected $products = array();

    public function add($product, $quantity)
    {
        $this->products[$product] = $quantity;
    }

    public function getQuantity($product)
    {
        return isset($this->products[$product]) ? $this->products[$product] :
               FALSE;
    }

    public function getTotal($tax)
    {
        $total = 0.00;

        $callback =
            function ($quantity, $product) use ($tax, &$total)
            {
                $pricePerItem = constant(__CLASS__ . "::PRICE_" .
                    strtoupper($product));
                $total += ($pricePerItem * $quantity) * ($tax + 1.0);
            };

        array_walk($this->products, $callback);
        return round($total, 2);
    }
}
```

```php
$my_cart = new Cart;

// Add some items to the cart
$my_cart->add('butter', 1);
$my_cart->add('milk', 3);
$my_cart->add('eggs', 6);

// Print the total with a 5% sales tax.
print $my_cart->getTotal(0.05) . "\n";
// The result is 54.29
```

Example #5 Automatic binding of `$this`

```php
<?php
class Test
{
    public function testing()
    {
        return function() {
            var_dump($this);
        };
    }
}

$object = new Test;
$function = $object->testing();
$function();
```

The above example will output:

```
object(Test)#1 (0) {
}
```

When declared in the context of a class, the current class is automatically bound to it, making `$this` available inside of the function's scope. If this automatic binding of the current class is not wanted, then static anonymous functions may be used instead.

Static anonymous functions

Anonymous functions may be declared statically. This prevents them from having the current class automatically bound to them. Objects may also not be bound to them at runtime.

Example #6 Attempting to use `$this` inside a static anonymous function

```php
<?php
class Foo
{
    function __construct()
    {
        $func = static function() {
```

```
            var_dump($this);
        };
        $func();
    }
};
new Foo();
```

The above example will output:

```
Fatal error: Uncaught Error: Using $this when not in object context in script:7
Stack trace:
#0 script(9): Foo::{closure:Foo::__construct():6}()
#1 script(12): Foo->__construct()
#2 {main}
  thrown in script on line 7
```

Example #7 Attempting to bind an object to a static anonymous function

```
<?php
$func = static function() {
    // function body
};
$func = $func->bindTo(new stdClass);
$func();
```

The above example will output:

```
Warning: Cannot bind an instance to a static closure, this will be an error in PHP 9 in script on
line 5

Fatal error: Uncaught Error: Value of type null is not callable in script:6
Stack trace:
#0 {main}
  thrown in script on line 6
```

Changelog

Version	Description
8.5.0	Closures can now be used in constant expressions (e.g. default property values, attribute parameters, constants).
8.3.0	Closures created from magic methods can accept named parameters.
7.1.0	Anonymous functions may not close over superglobals, *$this*, or any variable with the same name as a parameter.

Notes

> **Note** :
>
> It is possible to use func_num_args(), func_get_arg(), and func_get_args() from within a closure.

Arrow Functions

Arrow functions were introduced in PHP 7.4 as a more concise syntax for anonymous functions.

Both anonymous functions and arrow functions are implemented using the Closure class.

Arrow functions have the basic form `fn (argument_list) => expr`.

Arrow functions support the same features as anonymous functions, except that using variables from the parent scope is always automatic.

When a variable used in the expression is defined in the parent scope it will be implicitly captured by-value. In the following example, the functions *$fn1* and *$fn2* behave the same way.

Example #1 Arrow functions capture variables by value automatically

```php
<?php
$y = 1;

$fn1 = fn($x) => $x + $y;
// equivalent to using $y by value:
$fn2 = function ($x) use ($y) {
    return $x + $y;
};

var_export($fn1(3));
```

The above example will output:

```
4
```

This also works if the arrow functions are nested:

Example #2 Arrow functions capture variables by value automatically, even when nested

```php
<?php
$z = 1;
$fn = fn($x) => fn($y) => $x * $y + $z;
// Outputs 51
var_export($fn(5)(10));
```

Similarly to anonymous functions, the arrow function syntax allows arbitrary function signatures, including parameter and return types, default values, variadics, as well as by-reference passing and returning. All of the

following are valid examples of arrow functions:

Example #3 Examples of arrow functions

```php
<?php
fn(array $x) => $x;
static fn($x): int => $x;
fn($x = 42) => $x;
fn(&$x) => $x;
fn&($x) => $x;
fn($x, ...$rest) => $rest;
```

Arrow functions use by-value variable binding. This is roughly equivalent to performing a `use($x)` for every variable *$x* used inside the arrow function. A by-value binding means that it is not possible to modify any values from the outer scope. Anonymous functions can be used instead for by-ref bindings.

Example #4 Values from the outer scope cannot be modified by arrow functions

```php
<?php
$x = 1;
$fn = fn() => $x++; // Has no effect
$fn();
var_export($x);  // Outputs 1
```

Changelog

Version	Description
7.4.0	Arrow functions became available.

Notes

> **Note** :
>
> It is possible to use func_num_args(), func_get_arg(), and func_get_args() from within an arrow function.

First class callable syntax

The first class callable syntax is introduced as of PHP 8.1.0, as a way of creating anonymous functions from callable. It supersedes existing callable syntax using strings and arrays. The advantage of this syntax is that it is accessible to static analysis, and uses the scope at the point where the callable is acquired.

`CallableExpr(...)` syntax is used to create a Closure object from callable. `CallableExpr` accepts any expression that can be directly called in the PHP grammar:

Example #1 Simple first class callable syntax

```php
<?php
class Foo {
   public function method() {}
   public static function staticmethod() {}
   public function __invoke() {}
}
```

```php
$obj = new Foo();
$classStr = 'Foo';
$methodStr = 'method';
$staticmethodStr = 'staticmethod';


$f1 = strlen(...);
$f2 = $obj(...);  // invokable object
$f3 = $obj->method(...);
$f4 = $obj->$methodStr(...);
$f5 = Foo::staticmethod(...);
$f6 = $classStr::$staticmethodStr(...);

// traditional callable using string, array
$f7 = 'strlen'(...);
$f8 = [$obj, 'method'](...);
$f9 = [Foo::class, 'staticmethod'](...);
```

> **Note** :
>
> The `...` is part of the syntax, and not an omission.

`CallableExpr(...)` has the same semantics as Closure::fromCallable(). That is, unlike callable using strings and arrays, `CallableExpr(...)` respects the scope at the point where it is created:

Example #2 Scope comparison of `CallableExpr(...)` and traditional callable

```php
<?php
class Foo {
    public function getPrivateMethod() {
        return [$this, 'privateMethod'];
    }

    private function privateMethod() {
        echo __METHOD__, "\n";
    }
}

$foo = new Foo;
$privateMethod = $foo->getPrivateMethod();
$privateMethod();
// Fatal error: Call to private method Foo::privateMethod() from global scope
// This is because call is performed outside from Foo and visibility will be checked from this
point.
```

```php
<?php
class Foo1 {
    public function getPrivateMethod() {
        // Uses the scope where the callable is acquired.
        return $this->privateMethod(...); // identical to Closure::fromCallable([$this,
'privateMethod']);
    }

    private function privateMethod() {
```

```
        echo __METHOD__, "\n";
    }
}

$foo1 = new Foo1;
$privateMethod = $foo1->getPrivateMethod();
$privateMethod();  // Foo1::privateMethod
```

> **Note** :
>
> Object creation by this syntax (e.g `new Foo(...)`) is not supported, because `new Foo()` syntax is not considered a call.

> **Note** :
>
> The first-class callable syntax cannot be combined with the nullsafe operator. Both of the following result in a compile-time error:
>
> ```
> <?php
> $obj?->method(...);
> $obj?->prop->method(...);
> ```

Changelog

Version	Description
8.5.0	First-class callables can now be used in constant expressions (e.g. default property values, attribute parameters, constants).

Classes and Objects

Table of Contents

Introduction

PHP includes a complete object model. Some of its features are: visibility, abstract and final classes and methods, additional magic methods, interfaces, and cloning.

PHP treats objects in the same way as references or handles, meaning that each variable contains an object reference rather than a copy of the entire object. See Objects and References

> **Tip**
>
> See also the Userland Naming Guide.

The Basics

class

Basic class definitions begin with the keyword `class`, followed by a class name, followed by a pair of curly braces which enclose the definitions of the properties and methods belonging to the class.

The class name can be any valid label, provided it is not a PHP reserved word. As of PHP 8.4.0, using a single underscore _ as a class name is deprecated. A valid class name starts with a letter or underscore, followed by any number of letters, numbers, or underscores. As a regular expression, it would be expressed thus: `^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$`.

A class may contain its own constants, variables (called "properties"), and functions (called "methods").

Example #1 Simple Class definition

```php
<?php
class SimpleClass
{
    // property declaration
    public $var = 'a default value';

    // method declaration
    public function displayVar() {
        echo $this->var;
    }
}
?>
```

The pseudo-variable *$this* is available when a method is called from within an object context. *$this* is the value of the calling object.

> **Warning**
>
> Calling a non-static method statically throws an Error. Prior to PHP 8.0.0, this would generate a deprecation notice, and *$this* would be undefined.
>
> **Example #2 Some examples of the *$this* pseudo-variable**
>
> ```php
> <?php
> class A
> {
>     function foo()
>     {
>         if (isset($this)) {
>             echo '$this is defined (';
>             echo get_class($this);
>             echo ")\n";
>         } else {
>             echo "\$this is not defined.\n";
>         }
>     }
> }
>
> class B
> {
>     function bar()
>     {
>         A::foo();
>     }
> ```

```
}

$a = new A();
$a->foo();

A::foo();

$b = new B();
$b->bar();

B::bar();
?>
```

Output of the above example in PHP 7:

```
$this is defined (A)

Deprecated: Non-static method A::foo() should not be called statically in %s  on line 27
$this is not defined.

Deprecated: Non-static method A::foo() should not be called statically in %s  on line 20
$this is not defined.

Deprecated: Non-static method B::bar() should not be called statically in %s  on line 32

Deprecated: Non-static method A::foo() should not be called statically in %s  on line 20
$this is not defined.
```

Output of the above example in PHP 8:

```
$this is defined (A)

Fatal error: Uncaught Error: Non-static method A::foo() cannot be called statically in %s :27
Stack trace:
#0 {main}
  thrown in %s  on line 27
```

Readonly classes

As of PHP 8.2.0, a class can be marked with the readonly modifier. Marking a class as readonly will add the readonly modifier to every declared property, and prevent the creation of dynamic properties. Moreover, it is impossible to add support for them by using the AllowDynamicProperties attribute. Attempting to do so will trigger a compile-time error.

```
<?php
#[\AllowDynamicProperties]
```

```php
readonly class Foo {
}

// Fatal error: Cannot apply #[AllowDynamicProperties] to readonly class Foo
?>
```

As neither untyped nor static properties can be marked with the `readonly` modifier, readonly classes cannot declare them either:

```php
<?php
readonly class Foo
{
    public $bar;
}

// Fatal error: Readonly property Foo::$bar must have type
?>
```

```php
<?php
readonly class Foo
{
    public static int $bar;
}

// Fatal error: Readonly class Foo cannot declare static properties
?>
```

A readonly class can be extended if, and only if, the child class is also a readonly class.

new

To create an instance of a class, the `new` keyword must be used. An object will always be created unless the object has a constructor defined that throws an exception on error. Classes should be defined before instantiation (and in some cases this is a requirement).

If a variable containing a string with the name of a class is used with `new`, a new instance of that class will be created. If the class is in a namespace, its fully qualified name must be used when doing this.

> **Note** :
>
> If there are no arguments to be passed to the class's constructor, parentheses after the class name may be omitted.

Example #3 Creating an instance

```php
<?php
class SimpleClass {
}

$instance = new SimpleClass();
var_dump($instance);

// This can also be done with a variable:
$className = 'SimpleClass';
```

```php
$instance = new $className(); // new SimpleClass()
var_dump($instance);
?>
```

As of PHP 8.0.0, using `new` with arbitrary expressions is supported. This allows more complex instantiation if the expression produces a string. The expressions must be wrapped in parentheses.

Example #4 Creating an instance using an arbitrary expression

In the given example we show multiple examples of valid arbitrary expressions that produce a class name. This shows a call to a function, string concatenation, and the **`::class`** constant.

```php
<?php

class ClassA extends \stdClass {}
class ClassB extends \stdClass {}
class ClassC extends ClassB {}
class ClassD extends ClassA {}

function getSomeClass(): string
{
    return 'ClassA';
}

var_dump(new (getSomeClass()));
var_dump(new ('Class' . 'B'));
var_dump(new ('Class' . 'C'));
var_dump(new (ClassD::class));
?>
```

Output of the above example in PHP 8:

```
object(ClassA)#1 (0) {
}
object(ClassB)#1 (0) {
}
object(ClassC)#1 (0) {
}
object(ClassD)#1 (0) {
}
```

In the class context, it is possible to create a new object by `new self` and `new parent`.

When assigning an already created instance of a class to a new variable, the new variable will access the same instance as the object that was assigned. This behaviour is the same when passing instances to a function. A copy of an already created object can be made by cloning it.

Example #5 Object Assignment

```php
<?php
class SimpleClass {
    public string $var;
}

$instance = new SimpleClass();

$assigned   =  $instance;
$reference  =& $instance;

$instance->var = '$assigned will have this value';

$instance = null; // $instance and $reference become null

var_dump($instance);
var_dump($reference);
var_dump($assigned);
?>
```

The above example will output:

```
NULL
NULL
object(SimpleClass)#1 (1) {
  ["var"]=>
  string(30) "$assigned will have this value"
}
```

It's possible to create instances of an object in a couple of ways:

Example #6 Creating new objects

```php
<?php

class Test
{
    public static function getNew()
    {
        return new static();
    }
}

class Child extends Test {}

$obj1 = new Test(); // By the class name
$obj2 = new $obj1(); // Through the variable containing an object
var_dump($obj1 !== $obj2);

$obj3 = Test::getNew(); // By the class method
var_dump($obj3 instanceof Test);

$obj4 = Child::getNew(); // Through a child class method
```

```php
var_dump($obj4 instanceof Child);

?>
```

The above example will output:

```
bool(true)
bool(true)
bool(true)
```

It is possible to access a member of a newly created object in a single expression:

Example #7 Access member of newly created object

```php
<?php
echo (new DateTime())->format('Y'), PHP_EOL;

// surrounding parentheses are optional as of PHP 8.4.0
echo new DateTime()->format('Y'), PHP_EOL;
?>
```

The above example will output something similar to:

```
2025
2025
```

> **Note** :
>
> Prior to PHP 7.1, the arguments are not evaluated if there is no constructor function defined.

Properties and methods

Class properties and methods live in separate "namespaces", so it is possible to have a property and a method with the same name. Referring to both a property and a method has the same notation, and whether a property will be accessed or a method will be called, solely depends on the context, i.e. whether the usage is a variable access or a function call.

Example #8 Property access vs. method call

```php
<?php
class Foo
{
    public $bar = 'property';
```

```
    public function bar() {
        return 'method';
    }
}

$obj = new Foo();
echo $obj->bar, PHP_EOL, $obj->bar(), PHP_EOL;
```

The above example will output:

```
property
method
```

That means that calling an anonymous function which has been assigned to a property is not directly possible. Instead the property has to be assigned to a variable first, for instance. It is possible to call such a property directly by enclosing it in parentheses.

Example #9 Calling an anonymous function stored in a property

```
<?php
class Foo
{
    public $bar;

    public function __construct() {
        $this->bar = function() {
            return 42;
        };
    }
}

$obj = new Foo();

echo ($obj->bar)(), PHP_EOL;
```

The above example will output:

```
42
```

extends

A class can inherit the constants, methods, and properties of another class by using the keyword `extends` in the class declaration. It is not possible to extend multiple classes; a class can only inherit from one base class.

The inherited constants, methods, and properties can be overridden by redeclaring them with the same name defined in the parent class. However, if the parent class has defined a method or constant as final, they may

not be overridden. It is possible to access the overridden methods or static properties by referencing them with parent::.

> **Note** :
>
> As of PHP 8.1.0, constants may be declared as final.

Example #10 Simple Class Inheritance

```php
<?php
class SimpleClass
{
    function displayVar()
    {
        echo "Parent class\n";
    }
}

class ExtendClass extends SimpleClass
{
    // Redefine the parent method
    function displayVar()
    {
        echo "Extending class\n";
        parent::displayVar();
    }
}

$extended = new ExtendClass();
$extended->displayVar();
?>
```

The above example will output:

```
Extending class
Parent class
```

Signature compatibility rules

When overriding a method, its signature must be compatible with the parent method. Otherwise, a fatal error is emitted, or, prior to PHP 8.0.0, an **E_WARNING** level error is generated. A signature is compatible if it respects the variance rules, makes a mandatory parameter optional, adds only optional new parameters and doesn't restrict but only relaxes the visibility. This is known as the Liskov Substitution Principle, or LSP for short. The constructor, and `private` methods are exempt from these signature compatibility rules, and thus won't emit a fatal error in case of a signature mismatch.

Example #11 Compatible child methods

```php
<?php

class Base
{
    public function foo(int $a) {
        echo "Valid\n";
    }
}

class Extend1 extends Base
{
    function foo(int $a = 5)
    {
        parent::foo($a);
    }
}

class Extend2 extends Base
{
    function foo(int $a, $b = 5)
    {
        parent::foo($a);
    }
}

$extended1 = new Extend1();
$extended1->foo();
$extended2 = new Extend2();
$extended2->foo(1);
```

The above example will output:

```
Valid
Valid
```

The following examples demonstrate that a child method which removes a parameter, or makes an optional parameter mandatory, is not compatible with the parent method.

Example #12 Fatal error when a child method removes a parameter

```php
<?php

class Base
{
    public function foo(int $a = 5) {
        echo "Valid\n";
    }
}

class Extend extends Base
{
```

```
    function foo()
    {
        parent::foo(1);
    }
}
```

Output of the above example in PHP 8 is similar to:

```
Fatal error: Declaration of Extend::foo() must be compatible with Base::foo(int $a = 5) in
/in/evtlq on line 13
```

Example #13 Fatal error when a child method makes an optional parameter mandatory

```
<?php

class Base
{
    public function foo(int $a = 5) {
        echo "Valid\n";
    }
}

class Extend extends Base
{
    function foo(int $a)
    {
        parent::foo($a);
    }
}
```

Output of the above example in PHP 8 is similar to:

```
Fatal error: Declaration of Extend::foo(int $a) must be compatible with Base::foo(int $a = 5) in
/in/qJXVC on line 13
```

> **Warning**
>
> Renaming a method's parameter in a child class is not a signature incompatibility. However, this is discouraged as it will result in a runtime Error if named arguments are used.
>
> **Example #14 Error when using named arguments and parameters were renamed in a child class**
>
> ```
> <?php
>
> class A {
>     public function test($foo, $bar) {}
> ```

```php
}

class B extends A {
    public function test($a, $b) {}
}

$obj = new B;

// Pass parameters according to A::test() contract
$obj->test(foo: "foo", bar: "bar"); // ERROR!
```

The above example will output something similar to:

```
Fatal error: Uncaught Error: Unknown named parameter $foo in /in/XaaeN:14
Stack trace:
#0 {main}
  thrown in /in/XaaeN on line 14
```

::class

The `class` keyword is also used for class name resolution. To obtain the fully qualified name of a class `ClassName` use `ClassName::class`. This is particularly useful with namespaced classes.

Example #15 Class name resolution

```php
<?php
namespace NS {
    class ClassName {
    }

    echo ClassName::class;
}
?>
```

The above example will output:

```
NS\ClassName
```

> **Note** :
>
> The class name resolution using `::class` is a compile time transformation. That means at the time the class name string is created no autoloading has happened yet. As a consequence, class names are expanded even if the class does not exist. No error is issued in that case.
>
> **Example #16 Missing class name resolution**

```php
<?php
print Does\Not\Exist::class;
?>
```

The above example will output:

```
Does\Not\Exist
```

As of PHP 8.0.0, `::class` may also be used on objects. This resolution happens at runtime, not compile time. Its effect is the same as calling get_class() on the object.

Example #17 Object name resolution

```php
<?php
namespace NS {
    class ClassName {
    }

    $c = new ClassName();
    print $c::class;
}
?>
```

The above example will output:

```
NS\ClassName
```

Nullsafe methods and properties

As of PHP 8.0.0, properties and methods may also be accessed with the "nullsafe" operator instead: `?->`. The nullsafe operator works the same as property or method access as above, except that if the object being dereferenced is `null` then `null` will be returned rather than an exception thrown. If the dereference is part of a chain, the rest of the chain is skipped.

The effect is similar to wrapping each access in an is_null() check first, but more compact.

Example #18 Nullsafe Operator

```php
<?php

// As of PHP 8.0.0, this line:
$result = $repository?->getUser(5)?->name;

// Is equivalent to the following code block:
if (is_null($repository)) {
```

```
    $result = null;
} else {
    $user = $repository->getUser(5);
    if (is_null($user)) {
        $result = null;
    } else {
        $result = $user->name;
    }
}
?>
```

> **Note** :
>
> The nullsafe operator is best used when null is considered a valid and expected possible value for a property or method return. For indicating an error, a thrown exception is preferable.

Properties

Class member variables are called *properties*. They may be referred to using other terms such as *fields*, but for the purposes of this reference *properties* will be used. They are defined by using at least one modifier (such as Visibility, Static Keyword, or, as of PHP 8.1.0, readonly), optionally (except for `readonly` properties), as of PHP 7.4, followed by a type declaration, followed by a normal variable declaration. This declaration may include an initialization, but this initialization must be a constant value.

> **Note** :
>
> An obsolete way of declaring class properties is by using the `var` keyword instead of a modifier.

> **Note** :
>
> A property declared without a Visibility modifier will be declared as `public`.

Within class methods non-static properties may be accessed by using `->` (Object Operator): *$this->property* (where `property` is the name of the property). Static properties are accessed by using the `::` (Double Colon): *self::$property*. See Static Keyword for more information on the difference between static and non-static properties.

The pseudo-variable *$this* is available inside any class method when that method is called from within an object context. *$this* is the value of the calling object.

Example #1 Property declarations

```
<?php
class SimpleClass
{
   public $var1 = 'hello ' . 'world';
   public $var2 = <<<EOD
hello world
EOD;
   public $var3 = 1+2;
```

```
    // invalid property declarations:
    public $var4 = self::myStaticMethod();
    public $var5 = $myVar;

    // valid property declarations:
    public $var6 = myConstant;
    public $var7 = [true, false];

    public $var8 = <<<'EOD'
hello world
EOD;

    // Without visibility modifier:
    static $var9;
    readonly int $var10;
}
?>
```

> **Note** :
>
> There are various functions to handle classes and objects. See the Class/Object Functions reference.

Type declarations

As of PHP 7.4.0, property definitions can include Type declarations, with the exception of callable.

Example #2 Example of typed properties

```
<?php

class User
{
    public int $id;
    public ?string $name;

    public function __construct(int $id, ?string $name)
    {
        $this->id = $id;
        $this->name = $name;
    }
}

$user = new User(1234, null);

var_dump($user->id);
var_dump($user->name);

?>
```

The above example will output:

```

```

```
int(1234)
NULL
```

Typed properties must be initialized before accessing, otherwise an Error is thrown.

Example #3 Accessing properties

```php
<?php

class Shape
{
    public int $numberOfSides;
    public string $name;

    public function setNumberOfSides(int $numberOfSides): void
    {
        $this->numberOfSides = $numberOfSides;
    }

    public function setName(string $name): void
    {
        $this->name = $name;
    }

    public function getNumberOfSides(): int
    {
        return $this->numberOfSides;
    }

    public function getName(): string
    {
        return $this->name;
    }
}

$triangle = new Shape();
$triangle->setName("triangle");
$triangle->setNumberOfSides(3);
var_dump($triangle->getName());
var_dump($triangle->getNumberOfSides());

$circle = new Shape();
$circle->setName("circle");
var_dump($circle->getName());
var_dump($circle->getNumberOfSides());
?>
```

The above example will output:

```
string(8) "triangle"
int(3)
string(6) "circle"
```

```
Fatal error: Uncaught Error: Typed property Shape::$numberOfSides must not be accessed before
initialization
```

Readonly properties

As of PHP 8.1.0, a property can be declared with the `readonly` modifier, which prevents modification of the property after initialization. Prior to PHP 8.4.0 a `readonly` property is implicitly private-set, and may only be written to from the same class. As of PHP 8.4.0, `readonly` properties are implicitly `protected(set)`, so may be set from child classes. That may be overridden explicitly if desired.

Example #4 Example of readonly properties

```php
<?php

class Test {
   public readonly string $prop;

   public function __construct(string $prop) {
       // Legal initialization.
       $this->prop = $prop;
   }
}

$test = new Test("foobar");
// Legal read.
var_dump($test->prop); // string(6) "foobar"

// Illegal reassignment. It does not matter that the assigned value is the same.
$test->prop = "foobar";
// Error: Cannot modify readonly property Test::$prop
?>
```

> **Note** :
>
> The readonly modifier can only be applied to typed properties. A readonly property without type constraints can be created using the Mixed type.

> **Note** :
>
> Readonly static properties are not supported.

A readonly property can only be initialized once, and only from the scope where it has been declared. Any other assignment or modification of the property will result in an Error exception.

Example #5 Illegal initialization of readonly properties

```php
<?php
class Test1 {
    public readonly string $prop;
```

```
}

$test1 = new Test1;
// Illegal initialization outside of private scope.
$test1->prop = "foobar";
// Error: Cannot initialize readonly property Test1::$prop from global scope
?>
```

> **Note** :
>
> Readonly properties can only be initialized by a direct assignment. An uninitialized readonly property cannot be initialized by reference, not even from the scope where it has been declared: passing it to a function parameter that expects to be passed by reference, or assigning it by reference, results in an Error exception.

```
<?php

class Test {
    public readonly array $matches;

    public function __construct(string $subject) {
        // Illegal initialization by reference.
        preg_match('/\d+/', $subject, $this->matches);
        // Error: Cannot indirectly modify readonly property Test::$matches
    }
}

new Test('abc 42');
?>
```

> **Note** :
>
> Specifying an explicit default value on readonly properties is not allowed, because a readonly property with a default value is essentially the same as a constant, and thus not particularly useful.

```
<?php

class Test {
    // Fatal error: Readonly property Test::$prop cannot have default value
    public readonly int $prop = 42;
}
?>
```

> **Note** :
>
> Readonly properties cannot be unset() once they are initialized. However, it is possible to unset a readonly property prior to initialization, from the scope where the property has been declared.

Modifications are not necessarily plain assignments, all of the following will also result in an Error exception:

```php
<?php

class Test {
    public function __construct(
        public readonly int $i = 0,
        public readonly array $ary = [],
    ) {}
}

$test = new Test;
$test->i += 1;
$test->i++;
++$test->i;
$test->ary[] = 1;
$test->ary[0][] = 1;
unset($test->ary[0]);
$ref =& $test->i;
$test->i =& $ref;
byRef($test->i);
foreach ($test as &$prop);
?>
```

However, readonly properties do not preclude interior mutability. Objects (or resources) stored in readonly properties may still be modified internally:

```php
<?php

class Test {
    public function __construct(public readonly object $obj) {}
}

$test = new Test(new stdClass);
// Legal interior mutation.
$test->obj->foo = 1;
// Illegal reassignment.
$test->obj = new stdClass;
?>
```

As of PHP 8.3.0, readonly properties can be reinitialized when cloning an object using the __clone() method.

Example #6 Readonly properties and cloning

```php
<?php
class Test1 {
    public readonly ?string $prop;

    public function __clone() {
        $this->prop = null;
    }

    public function setProp(string $prop): void {
        $this->prop = $prop;
    }
}

$test1 = new Test1;
```

```php
$test1->setProp('foobar');

$test2 = clone $test1;
var_dump($test2->prop); // NULL
?>
```

> **Note** :
>
> As of PHP 8.4.0, indirect modification of a readonly property within __clone(), such as acquiring a reference to it (`$ref = &$this->prop`), is no longer allowed. This was already forbidden during readonly initialization.

Dynamic properties

If trying to assign to a non-existent property on an object, PHP will automatically create a corresponding property. This dynamically created property will *only* be available on this class instance.

> **Warning**
>
> Dynamic properties are deprecated as of PHP 8.2.0. It is recommended to declare the property instead. To handle arbitrary property names, the class should implement the magic methods __get() and __set(). As a last resort the class can be marked with the `#[\AllowDynamicProperties]` attribute.

Property Hooks

Property hooks, also known as "property accessors" in some other languages, are a way to intercept and override the read and write behavior of a property. This functionality serves two purposes:

1. It allows for properties to be used directly, without get- and set- methods, while leaving the option open to add additional behavior in the future. That renders most boilerplate get/set methods unnecessary, even without using hooks.
2. It allows for properties that describe an object without needing to store a value directly.

There are two hooks available on non-static properties: `get` and `set`. They allow overriding the read and write behavior of a property, respectively. Hooks are available for both typed and untyped properties.

A property may be "backed" or "virtual". A backed property is one that actually stores a value. Any property that has no hooks is backed. A virtual property is one that has hooks and those hooks do not interact with the property itself. In this case, the hooks are effectively the same as methods, and the object does not use any space to store a value for that property.

Property hooks are incompatible with `readonly` properties. If there is a need to restrict access to a `get` or `set` operation in addition to altering its behavior, use asymmetric property visibility.

> **Note** : **Version Information**
>
> Property hooks were introduced in PHP 8.4.

Basic Hook Syntax

The general syntax for declaring a hook is as follows.

Example #1 Property hooks (full version)

```php
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
        get {
            if ($this->modified) {
                return $this->foo . ' (modified)';
            }
            return $this->foo;
        }
        set(string $value) {
            $this->foo = strtolower($value);
            $this->modified = true;
        }
    }
}

$example = new Example();
$example->foo = 'changed';
print $example->foo;
?>
```

The *$foo* property ends in `{}`, rather than a semicolon. That indicates the presence of hooks. Both a `get` and `set` hook are defined, although it is allowed to define only one or the other. Both hooks have a body, denoted by `{}`, that may contain arbitrary code.

The `set` hook additionally allows specifying the type and name of an incoming value, using the same syntax as a method. The type must be either the same as the type of the property, or contravariant (wider) to it. For instance, a property of type string could have a `set` hook that accepts string|Stringable, but not one that only accepts array.

At least one of the hooks references `$this->foo`, the property itself. That means the property will be "backed". When calling `$example->foo = 'changed'`, the provided string will be first cast to lowercase, then saved to the backing value. When reading from the property, the previously saved value may conditionally be appended with additional text.

There are a number of shorthand syntax variants as well to handle common cases.

If the `get` hook is a single expression, then the `{}` may be omitted and replaced with an arrow expression.

Example #2 Property get expression

This example is equivalent to the previous.

```php
<?php
class Example
{
    private bool $modified = false;
```

```
    public string $foo = 'default value' {
        get => $this->foo . ($this->modified ? ' (modified)' : '');

        set(string $value) {
            $this->foo = strtolower($value);
            $this->modified = true;
        }
    }
}
?>
```

If the `set` hook's parameter type is the same as the property type (which is typical), it may be omitted. In that case, the value to set is automatically given the name *$value*.

Example #3 Property set defaults

This example is equivalent to the previous.

```
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
        get => $this->foo . ($this->modified ? ' (modified)' : '');

        set {
            $this->foo = strtolower($value);
            $this->modified = true;
        }
    }
}
?>
```

If the `set` hook is only setting a modified version of the passed in value, then it may also be simplified to an arrow expression. The value the expression evaluates to will be set on the backing value.

Example #4 Property set expression

```
<?php
class Example
{
    private bool $modified = false;

    public string $foo = 'default value' {
        get => $this->foo . ($this->modified ? ' (modified)' : '');
        set => strtolower($value);
    }
}
?>
```

This example is not quite equivalent to the previous, as it does not also modify `$this->modified`. If multiple statements are needed in the set hook body, use the braces version.

A property may implement zero, one, or both hooks as the situation requires. All shorthand versions are mutually-independent. That is, using a short-get with a long-set, a short-set with an explicit type, and so on is all valid.

On a backed property, omitting a `get` or `set` hook means the default read or write behavior will be used.

> **Note** :
>
> Hooks can be defined when using constructor property promotion. However, when doing so, values provided to the constructor must match the type associated with the property, regardless of what the `set` hook might allow.
>
> Consider the following:

```php
<?php
class Example
{
    public function __construct(
        public private(set) DateTimeInterface $created {
            set (string|DateTimeInterface $value) {
                if (is_string($value)) {
                    $value = new DateTimeImmutable($value);
                }
                $this->created = $value;
            }
        },
    ) {
    }
}
```

> Internally, the engine decomposes this to the following:

```php
<?php
class Example
{
    public private(set) DateTimeInterface $created {
        set (string|DateTimeInterface $value) {
            if (is_string($value)) {
                $value = new DateTimeImmutable($value);
            }
            $this->created = $value;
        }
    }

    public function __construct(
        DateTimeInterface $created,
    ) {
        $this->created = $created;
    }
}
```

> Any attempts to set the property outside the constructor will allow either string or DateTimeInterface values, but the constructor will only allow DateTimeInterface. This is because the defined type for the

> property (DateTimeInterface) is used as the parameter type within the constructor signature, regardless of what the `set` hook allows.
>
> If this kind of behavior is needed from the constructor, constructor property promotion cannot be used.

Virtual properties

Virtual properties are properties that have no backing value. A property is virtual if neither its `get` nor `set` hook references the property itself using exact syntax. That is, a property named `$foo` whose hook contains `$this->foo` will be backed. But the following is not a backed property, and will error:

Example #5 Invalid virtual property

```php
<?php
class Example
{
    public string $foo {
        get {
            $temp = __PROPERTY__;
            return $this->$temp; // Doesn't refer to $this->foo, so it doesn't count.
        }
    }
}
?>
```

For virtual properties, if a hook is omitted then that operation does not exist and trying to use it will produce an error. Virtual properties take up no memory space in an object. Virtual properties are suited for "derived" properties, such as those that are the combination of two other properties.

Example #6 Virtual property

```php
<?php
class Rectangle
{
    // A virtual property.
    public int $area {
        get => $this->h * $this->w;
    }

    public function __construct(public int $h, public int $w) {}
}

$s = new Rectangle(4, 5);
print $s->area; // prints 20
$s->area = 30; // Error, as there is no set operation defined.
?>
```

Defining both a `get` and `set` hook on a virtual property is also allowed.

Scoping

All hooks operate in the scope of the object being modified. That means they have access to all public, private, or protected methods of the object, as well as any public, private, or protected properties, including

properties that may have their own property hooks. Accessing another property from within a hook does not bypass the hooks defined on that property.

The most notable implication of this is that non-trivial hooks may sub-call to an arbitrarily complex method if they wish.

Example #7 Calling a method from a hook

```php
<?php
class Person {
    public string $phone {
        set => $this->sanitizePhone($value);
    }

    private function sanitizePhone(string $value): string {
        $value = ltrim($value, '+');
        $value = ltrim($value, '1');

        if (!preg_match('/\d\d\d\-\d\d\d\-\d\d\d\d/', $value)) {
            throw new \InvalidArgumentException();
        }
        return $value;
    }
}
?>
```

References

Because the presence of hooks intercepts the read and write process for properties, they cause issues when acquiring a reference to a property or with indirect modification, such as `$this->arrayProp['key'] = 'value';`. That is because any attempted modification of the value by reference would bypass a set hook, if one is defined.

In the rare case that getting a reference to a property that has hooks defined is necessary, the `get` hook may be prefixed with `&` to cause it to return by reference. Defining both `get` and `&get` on the same property is a syntax error.

Defining both `&get` and `set` hooks on a backed property is not allowed. As noted above, writing to the value returned by reference would bypass the `set` hook. On virtual properties, there is no necessary common value shared between the two hooks, so defining both is allowed.

Writing to an index of an array property also involves an implicit reference. For that reason, writing to a backed array property with hooks defined is allowed if and only if it defines only a `&get` hook. On a virtual property, writing to the array returned from either `get` or `&get` is legal, but whether that has any impact on the object depends on the hook implementation.

Overwriting the entire array property is fine, and behaves the same as any other property. Only working with elements of the array requires special care.

Inheritance

Final hooks

Hooks may also be declared final, in which case they may not be overridden.

Example #8 Final hooks

```php
<?php
class User
{
    public string $username {
        final set => strtolower($value);
    }
}

class Manager extends User
{
    public string $username {
        // This is allowed
        get => strtoupper($this->username);

        // But this is NOT allowed, because set is final in the parent.
        set => strtoupper($value);
    }
}
?>
```

A property may also be declared final. A final property may not be redeclared by a child class in any way, which precludes altering hooks or widening its access.

Declaring hooks final on a property that is declared final is redundant, and will be silently ignored. This is the same behavior as final methods.

A child class may define or redefine individual hooks on a property by redefining the property and just the hooks it wishes to override. A child class may also add hooks to a property that had none. This is essentially the same as if the hooks were methods.

Example #9 Hook inheritance

```php
<?php
class Point
{
    public int $x;
    public int $y;
}

class PositivePoint extends Point
{
    public int $x {
        set {
            if ($value < 0) {
                throw new \InvalidArgumentException('Too small');
            }
            $this->x = $value;
        }
    }
}
?>
```

Each hook overrides parent implementations independently of each other. If a child class adds hooks, any default value set on the property is removed, and must be redeclared. That is consistent with how inheritance works on hook-less properties.

Accessing parent hooks

A hook in a child class may access the parent class's property using the `parent::$prop` keyword, followed by the desired hook. For example, `parent::$propName::get()`. It may be read as "access the *prop* defined on the parent class, and then run its get operation" (or set operation, as appropriate).

If not accessed this way, the parent class's hook is ignored. This behavior is consistent with how all methods work. This also offers a way to access the parent class's storage, if any. If there is no hook on the parent property, its default get/set behavior will be used. Hooks may not access any other hook except their own parent on their own property.

The example above could be rewritten as follows, which would allow for the `Point` class to add its own `set` hook in the future without issues (in the previous example, a hook added to the parent class would be ignored in the child).

Example #10 Parent hook access (set)

```php
<?php
class Point
{
    public int $x;
    public int $y;
}

class PositivePoint extends Point
{
    public int $x {
        set {
            if ($value < 0) {
                throw new \InvalidArgumentException('Too small');
            }
            parent::$x::set($value);
        }
    }
}
?>
```

An example of overriding only a get hook could be:

Example #11 Parent hook access (get)

```php
<?php
class Strings
{
    public string $val;
}

class CaseFoldingStrings extends Strings
{
    public bool $uppercase = true;
```

```
    public string $val {
        get => $this->uppercase
            ? strtoupper(parent::$val::get())
            : strtolower(parent::$val::get());
    }
}
?>
```

Serialization

PHP has a number of different ways in which an object may be serialized, either for public consumption or for debugging purposes. The behavior of hooks varies depending on the use case. In some cases, the raw backing value of a property will be used, bypassing any hooks. In others, the property will be read or written "through" the hook, just like any other normal read/write action.

- var_dump(): Use raw value
- serialize(): Use raw value
- unserialize(): Use raw value
- __serialize()/__unserialize(): Custom logic, uses get/set hook
- Array casting: Use raw value
- var_export(): Use get hook
- json_encode(): Use get hook
- JsonSerializable: Custom logic, uses get hook
- get_object_vars(): Use get hook
- get_mangled_object_vars(): Use raw value

Class Constants

It is possible to define constants on a per-class basis. The default visibility of class constants is `public`.

> **Note** :
>
> Class constants can be redefined by a child class. As of PHP 8.1.0, class constants cannot be redefined by a child class if it is defined as final.

It's also possible for interfaces to have constants. Look at the interface documentation for examples.

It's possible to reference the class using a variable. The variable's value can not be a keyword (e.g. `self`, `parent` and `static`).

Note that class constants are allocated once per class, and not for each class instance.

As of PHP 8.3.0, class constants can have a scalar type such as `bool`, `int`, `float`, `string`, or even `array`. When using `array`, the contents can only be other scalar types.

Example #1 Defining and using a constant

```
<?php
class MyClass
{
    const CONSTANT = 'constant value';
```

```
    function showConstant() {
        echo  self::CONSTANT . "\n";
    }
}

echo MyClass::CONSTANT . "\n";

$classname = "MyClass";
echo $classname::CONSTANT . "\n";

$class = new MyClass();
$class->showConstant();

echo $class::CONSTANT."\n";
?>
```

The special **`::class`** constant allows for fully qualified class name resolution at compile time, this is useful for namespaced classes:

Example #2 Namespaced ::class example

```
<?php
namespace foo {
    class bar {
    }

    echo bar::class; // foo\bar
}
?>
```

Example #3 Class constant expression example

```
<?php
const ONE = 1;
class foo {
    const TWO = ONE * 2;
    const THREE = ONE + self::TWO;
    const SENTENCE = 'The value of THREE is '.self::THREE;
}
?>
```

Example #4 Class constant visibility modifiers, as of PHP 7.1.0

```
<?php
class Foo {
    public const BAR = 'bar';
    private const BAZ = 'baz';
}
echo Foo::BAR, PHP_EOL;
echo Foo::BAZ, PHP_EOL;
?>
```

Output of the above example in PHP 7.1:

```

```

```
bar

Fatal error: Uncaught Error: Cannot access private const Foo::BAZ in …
```

> **Note** :
>
> As of PHP 7.1.0 visibility modifiers are allowed for class constants.

Example #5 Class constant visibility variance check, as of PHP 8.3.0

```php
<?php

interface MyInterface
{
    public const VALUE = 42;
}

class MyClass implements MyInterface
{
    protected const VALUE = 42;
}
?>
```

Output of the above example in PHP 8.3:

```
Fatal error: Access level to MyClass::VALUE must be public (as in interface MyInterface) …
```

> **Note** :
>
> As of PHP 8.3.0 visibility variance is checked more strictly. Prior to this version, the visibility of a class constant could be different from the visibility of the constant in the implemented interface.

Example #6 Fetch class constant syntax, as of PHP 8.3.0

```php
<?php
class Foo {
    public const BAR = 'bar';
    private const BAZ = 'baz';
}

$name = 'BAR';
echo Foo::{$name}, PHP_EOL; // bar
?>
```

> **Note** :
>
> As of PHP 8.3.0, class constants can be fetched dynamically using a variable.

Example #7 Assigning types to class constants, as of PHP 8.3.0

```php
<?php

class MyClass {
    public const bool MY_BOOL = true;
    public const int MY_INT = 1;
    public const float MY_FLOAT = 1.01;
    public const string MY_STRING = 'one';
    public const array MY_ARRAY = [self::MY_BOOL, self::MY_INT, self::MY_FLOAT, self::MY_STRING];
}

var_dump(MyClass::MY_BOOL);
var_dump(MyClass::MY_INT);
var_dump(MyClass::MY_FLOAT);
var_dump(MyClass::MY_STRING);
var_dump(MyClass::MY_ARRAY);
?>
```

Output of the above example in PHP 8.3:

```
bool(true)
int(1)
float(1.01)
string(3) "one"
array(4) {
  [0]=>
  bool(true)
  [1]=>
  int(1)
  [2]=>
  float(1.01)
  [3]=>
  string(3) "one"
}
```

Autoloading Classes

Many developers writing object-oriented applications create one PHP source file per class definition. One of the biggest annoyances is having to write a long list of needed includes at the beginning of each script (one for each class).

The spl_autoload_register() function registers any number of autoloaders, enabling classes and interfaces to be automatically loaded if they are currently not defined. By registering autoloaders, PHP is given a last

chance to load the class or interface before it fails with an error.

Any class-like construct may be autoloaded the same way. That includes classes, interfaces, traits, and enumerations.

> **Caution**
>
> Prior to PHP 8.0.0, it was possible to use __autoload() to autoload classes and interfaces. However, it is a less flexible alternative to spl_autoload_register() and __autoload() is deprecated as of PHP 7.2.0, and removed as of PHP 8.0.0.

> **Note** :
>
> spl_autoload_register() may be called multiple times in order to register multiple autoloaders. Throwing an exception from an autoload function, however, will interrupt that process and not allow further autoload functions to run. For that reason, throwing exceptions from an autoload function is strongly discouraged.

Example #1 Autoload example

This example attempts to load the classes `MyClass1` and `MyClass2` from the files *MyClass1.php* and *MyClass2.php* respectively.

```php
<?php
spl_autoload_register(function ($class_name) {
    include $class_name . '.php';
});

$obj  = new MyClass1();
$obj2 = new MyClass2();
?>
```

Example #2 Autoload other example

This example attempts to load the interface `ITest`.

```php
<?php

spl_autoload_register(function ($name) {
    var_dump($name);
});

class Foo implements ITest {
}

/*
string(5) "ITest"

Fatal error: Interface 'ITest' not found in ...
*/
?>
```

Example #3 Using Composer's autoloader

» Composer generates a `vendor/autoload.php` which is set up to automatically load packages that are being managed by Composer. By including this file, those packages can be used without any additional work.

```php
<?php
require __DIR__ . '/vendor/autoload.php';

$uuid = Ramsey\Uuid\Uuid::uuid7();

echo "Generated new UUID -> ", $uuid->toString(), "\n";
?>
```

See Also

- unserialize()
- unserialize_callback_func
- unserialize_max_depth
- spl_autoload_register()
- spl_autoload()
- __autoload()

Constructors and Destructors

Constructor

function **__construct**(mixed `...$values = ""`): void

PHP allows developers to declare constructor methods for classes. Classes which have a constructor method call this method on each newly-created object, so it is suitable for any initialization that the object may need before it is used.

> **Note** :
>
> Parent constructors are not called implicitly if the child class defines a constructor. In order to run a parent constructor, a call to **parent::__construct()** within the child constructor is required. If the child does not define a constructor then it may be inherited from the parent class just like a normal class method (if it was not declared as private).

Example #1 Constructors in inheritance

```php
<?php
class BaseClass {
    function __construct() {
        print "In BaseClass constructor\n";
    }
}

class SubClass extends BaseClass {
    function __construct() {
        parent::__construct();
        print "In SubClass constructor\n";
    }
}
```

```php
class OtherSubClass extends BaseClass {
    // inherits BaseClass's constructor
}

// In BaseClass constructor
$obj = new BaseClass();

// In BaseClass constructor
// In SubClass constructor
$obj = new SubClass();

// In BaseClass constructor
$obj = new OtherSubClass();
?>
```

Unlike other methods, __construct() is exempt from the usual signature compatibility rules when being extended.

Constructors are ordinary methods which are called during the instantiation of their corresponding object. As such, they may define an arbitrary number of arguments, which may be required, may have a type, and may have a default value. Constructor arguments are called by placing the arguments in parentheses after the class name.

Example #2 Using constructor arguments

```php
<?php
class Point {
    protected int $x;
    protected int $y;

    public function __construct(int $x, int $y = 0) {
        $this->x = $x;
        $this->y = $y;
    }
}

// Pass both parameters.
$p1 = new Point(4, 5);
// Pass only the required parameter. $y will take its default value of 0.
$p2 = new Point(4);
// With named parameters (as of PHP 8.0):
$p3 = new Point(y: 5, x: 4);
?>
```

If a class has no constructor, or the constructor has no required arguments, the parentheses may be omitted.

Old-style constructors

Prior to PHP 8.0.0, classes in the global namespace will interpret a method named the same as the class as an old-style constructor. That syntax is deprecated, and will result in an **E_DEPRECATED** error but still call that function as a constructor. If both __construct() and a same-name method are defined, __construct() will be called.

In namespaced classes, or any class as of PHP 8.0.0, a method named the same as the class never has any special meaning.

Always use __construct() in new code.

Constructor Promotion

As of PHP 8.0.0, constructor parameters may also be promoted to correspond to an object property. It is very common for constructor parameters to be assigned to a property in the constructor but otherwise not operated upon. Constructor promotion provides a short-hand for that use case. The example above could be rewritten as the following.

Example #3 Using constructor property promotion

```php
<?php
class Point {
    public function __construct(protected int $x, protected int $y = 0) {
    }
}
```

When a constructor argument includes a modifier, PHP will interpret it as both an object property and a constructor argument, and assign the argument value to the property. The constructor body may then be empty or may contain other statements. Any additional statements will be executed after the argument values have been assigned to the corresponding properties.

Not all arguments need to be promoted. It is possible to mix and match promoted and not-promoted arguments, in any order. Promoted arguments have no impact on code calling the constructor.

> **Note** :
>
> Using a visibility modifier (`public`, `protected` or `private`) is the most likely way to apply property promotion, but any other single modifier (such as `readonly`) will have the same effect.

> **Note** :
>
> Object properties may not be typed callable due to engine ambiguity that this would introduce. Promoted arguments, therefore, may not be typed callable either. Any other type declaration is permitted, however.

> **Note** :
>
> As promoted properties are desugared to both a property as well as a function parameter, any and all naming restrictions for both properties as well as parameters apply.

> **Note** :
>
> Attributes placed on a promoted constructor argument will be replicated to both the property and argument. Default values on a promoted constructor argument will be replicated only to the argument and not the property.

New in initializers

As of PHP 8.1.0, objects can be used as default parameter values, static variables, and global constants, as well as in attribute arguments. Objects can also be passed to define() now.

> **Note** :
>
> The use of a dynamic or non-string class name or an anonymous class is not allowed. The use of argument unpacking is not allowed. The use of unsupported expressions as arguments is not allowed.

Example #4 Using new in initializers

```php
<?php

// All allowed:
static $x = new Foo;

const C = new Foo;

function test($param = new Foo) {}

#[AnAttribute(new Foo)]
class Test {
    public function __construct(
        public $prop = new Foo,
    ) {}
}

// All not allowed (compile-time error):
function test(
    $a = new (CLASS_NAME_CONSTANT)(), // dynamic class name
    $b = new class {}, // anonymous class
    $c = new A(...[]), // argument unpacking
    $d = new B($abc), // unsupported constant expression
) {}
?>
```

Static creation methods

PHP only supports a single constructor per class. In some cases, however, it may be desirable to allow an object to be constructed in different ways with different inputs. The recommended way to do so is by using static methods as constructor wrappers.

Example #5 Using static creation methods

```php
<?php
$some_json_string = '{ "id": 1004, "name": "Elephpant" }';
$some_xml_string = "<animal><id>1005</id><name>Elephpant</name></animal>";

class Product {

    private ?int $id;
    private ?string $name;

    private function __construct(?int $id = null, ?string $name = null) {
        $this->id = $id;
```

```
        $this->name = $name;
    }

    public static function fromBasicData(int $id, string $name): static {
        $new = new static($id, $name);
        return $new;
    }

    public static function fromJson(string $json): static {
        $data = json_decode($json, true);
        return new static($data['id'], $data['name']);
    }

    public static function fromXml(string $xml): static {
        $data = simplexml_load_string($xml);
        $new = new static();
        $new->id = (int) $data->id;
        $new->name = $data->name;
        return $new;
    }
}

$p1 = Product::fromBasicData(5, 'Widget');
$p2 = Product::fromJson($some_json_string);
$p3 = Product::fromXml($some_xml_string);

var_dump($p1, $p2, $p3);
```

The constructor may be made private or protected to prevent it from being called externally. If so, only a static method will be able to instantiate the class. Because they are in the same class definition they have access to private methods, even if not of the same object instance. The private constructor is optional and may or may not make sense depending on the use case.

The three public static methods then demonstrate different ways of instantiating the object.

- `fromBasicData()` takes the exact parameters that are needed, then creates the object by calling the constructor and returning the result.
- `fromJson()` accepts a JSON string and does some pre-processing on it itself to convert it into the format desired by the constructor. It then returns the new object.
- `fromXml()` accepts an XML string, preprocesses it, and then creates a bare object. The constructor is still called, but as all of the parameters are optional the method skips them. It then assigns values to the object properties directly before returning the result.

In all three cases, the `static` keyword is translated into the name of the class the code is in. In this case, `Product`.

Destructor

function **__destruct**(): void

PHP possesses a destructor concept similar to that of other object-oriented languages, such as C++. The destructor method will be called as soon as there are no other references to a particular object, or in any order during the shutdown sequence.

Example #6 Destructor Example

```php
<?php

class MyDestructableClass
{
    function __construct() {
        print "In constructor\n";
    }

    function __destruct() {
        print "Destroying " . __CLASS__ . "\n";
    }
}

$obj = new MyDestructableClass();
```

Like constructors, parent destructors will not be called implicitly by the engine. In order to run a parent destructor, one would have to explicitly call **parent::__destruct()** in the destructor body. Also like constructors, a child class may inherit the parent's destructor if it does not implement one itself.

The destructor will be called even if script execution is stopped using exit(). Calling exit() in a destructor will prevent the remaining shutdown routines from executing.

If a destructor creates new references to its object, it will not be called a second time when the reference count reaches zero again or during the shutdown sequence.

As of PHP 8.4.0, when cycle collection occurs during the execution of a Fiber, the destructors of objects scheduled for collection are executed in a separate Fiber, called the `gc_destructor_fiber`. If this Fiber is suspended, a new one will be created to execute any remaining destructors. The previous `gc_destructor_fiber` will no longer be referenced by the garbage collector and may be collected if it is not referenced elsewhere. Objects whose destructor is suspended will not be collected until the destructor returns or the Fiber itself is collected.

> **Note** :
>
> Destructors called during the script shutdown have HTTP headers already sent. The working directory in the script shutdown phase can be different with some SAPIs (e.g. Apache).

> **Note** :
>
> Attempting to throw an exception from a destructor (called in the time of script termination) causes a fatal error.

Visibility

The visibility of a property, a method or (as of PHP 7.1.0) a constant can be defined by prefixing the declaration with the keywords `public`, `protected` or `private`. Class members declared public can be accessed everywhere. Members declared protected can be accessed only within the class itself and by inheriting and parent classes. Members declared as private may only be accessed by the class that defines the member.

Property Visibility

Class properties may be defined as public, private, or protected. Properties declared without any explicit visibility keyword are defined as public.

Example #1 Property declaration

```php
<?php
/**
 * Define MyClass
 */
class MyClass
{
    public $public = 'Public';
    protected $protected = 'Protected';
    private $private = 'Private';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj = new MyClass();
echo $obj->public; // Works
echo $obj->protected; // Fatal Error
echo $obj->private; // Fatal Error
$obj->printHello(); // Shows Public, Protected and Private


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // We can redeclare the public and protected properties, but not private
    public $public = 'Public2';
    protected $protected = 'Protected2';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj2 = new MyClass2();
echo $obj2->public; // Works
echo $obj2->protected; // Fatal Error
echo $obj2->private; // Undefined
$obj2->printHello(); // Shows Public2, Protected2, Undefined

?>
```

Asymmetric Property Visibility

As of PHP 8.4, object properties may also have their visibility set asymmetrically, with a different scope for reading (`get`) and writing (`set`). Specifically, the `set` visibility may be specified separately, provided it is not more permissive than the default visibility.

Example #2 Asymmetric Property visibility

```php
<?php
class Book
{
    public function __construct(
        public private(set) string $title,
        public protected(set) string $author,
        protected private(set) int $pubYear,
    ) {}
}

class SpecialBook extends Book
{
    public function update(string $author, int $year): void
    {
        $this->author = $author; // OK
        $this->pubYear = $year; // Fatal Error
    }
}

$b = new Book('How to PHP', 'Peter H. Peterson', 2024);

echo $b->title; // Works
echo $b->author; // Works
echo $b->pubYear; // Fatal Error

$b->title = 'How not to PHP'; // Fatal Error
$b->author = 'Pedro H. Peterson'; // Fatal Error
$b->pubYear = 2023; // Fatal Error
?>
```

As of PHP 8.5, `set` visibility may also be applied to static properties of classes.

Example #3 Asymmetric Static Property Visibility

```php
<?php
class Manager
{
    public private(set) static int $calls = 0;

    public function doAThing(): string
    {
        self::$calls++;
        // Do other stuff.
        return "some string";
    }
}

$m = new Manager();

$m->doAThing(); // Works
```

```php
echo Manager::$calls; // Works
Manager::$calls = 5; // Fatal error
?>
```

The above example will output:

```
1
Fatal error: Uncaught Error: Cannot modify private(set) property Manager::$calls from global
scope in /some/file.php
```

There are a few caveats regarding asymmetric visibility:

- Only typed properties may have a separate `set` visibility.
- The `set` visibility must be the same as `get` or more restrictive. That is, `public protected(set)` and `protected protected(set)` are allowed, but `protected public(set)` will cause a syntax error.
- If a property is `public`, then the main visibility may be omitted. That is, `public private(set)` and `private(set)` will have the same result.
- A property with `private(set)` visibility is automatically `final`, and may not be redeclared in a child class.
- Obtaining a reference to a property follows `set` visibility, not `get`. That is because a reference may be used to modify the property value.
- Similarly, trying to write to an array property involves both a `get` and `set` operation internally, and therefore will follow the `set` visibility, as that is always the more restrictive.

> **Note** :
>
> Spaces are not allowed in the set-visibility declaration. `private(set)` is correct. `private( set )` is not correct and will result in a parse error.

When a class extends another, the child class may redefine any property that is not `final`. When doing so, it may widen either the main visibility or the `set` visibility, provided that the new visibility is the same or wider than the parent class. However, be aware that if a `private` property is overridden, it does not actually change the parent's property but creates a new property with a different internal name.

Example #4 Asymmetric Property inheritance

```php
<?php
class Book
{
    protected string $title;
    public protected(set) string $author;
    protected private(set) int $pubYear;
}

class SpecialBook extends Book
```

```
{
    public protected(set) string $title; // OK, as reading is wider and writing the same.
    public string $author; // OK, as reading is the same and writing is wider.
    public protected(set) int $pubYear; // Fatal Error. private(set) properties are final.
}
?>
```

Method Visibility

Class methods may be defined as public, private, or protected. Methods declared without any explicit visibility keyword are defined as public.

Example #5 Method Declaration

```
<?php
/**
 * Define MyClass
 */
class MyClass
{
    // Declare a public constructor
    public function __construct() { }

    // Declare a public method
    public function MyPublic() { }

    // Declare a protected method
    protected function MyProtected() { }

    // Declare a private method
    private function MyPrivate() { }

    // This is public
    function Foo()
    {
        $this->MyPublic();
        $this->MyProtected();
        $this->MyPrivate();
    }
}

$myclass = new MyClass;
$myclass->MyPublic(); // Works
$myclass->MyProtected(); // Fatal Error
$myclass->MyPrivate(); // Fatal Error
$myclass->Foo(); // Public, Protected and Private work


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // This is public
    function Foo2()
    {
        $this->MyPublic();
```

```
        $this->MyProtected();
        $this->MyPrivate(); // Fatal Error
    }
}

$myclass2 = new MyClass2;
$myclass2->MyPublic(); // Works
$myclass2->Foo2(); // Public and Protected work, not Private

class Bar
{
    public function test() {
        $this->testPrivate();
        $this->testPublic();
    }

    public function testPublic() {
        echo "Bar::testPublic\n";
    }

    private function testPrivate() {
        echo "Bar::testPrivate\n";
    }
}

class Foo extends Bar
{
    public function testPublic() {
        echo "Foo::testPublic\n";
    }

    private function testPrivate() {
        echo "Foo::testPrivate\n";
    }
}

$myFoo = new Foo();
$myFoo->test(); // Bar::testPrivate
                // Foo::testPublic
?>
```

Constant Visibility

As of PHP 7.1.0, class constants may be defined as public, private, or protected. Constants declared without any explicit visibility keyword are defined as public.

Example #6 Constant Declaration as of PHP 7.1.0

```
<?php
/**
 * Define MyClass
 */
class MyClass
{
    // Declare a public constant
    public const MY_PUBLIC = 'public';
```

```
    // Declare a protected constant
    protected const MY_PROTECTED = 'protected';

    // Declare a private constant
    private const MY_PRIVATE = 'private';

    public function foo()
    {
        echo self::MY_PUBLIC;
        echo self::MY_PROTECTED;
        echo self::MY_PRIVATE;
    }
}

$myclass = new MyClass();
MyClass::MY_PUBLIC; // Works
MyClass::MY_PROTECTED; // Fatal Error
MyClass::MY_PRIVATE; // Fatal Error
$myclass->foo(); // Public, Protected and Private work


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // This is public
    function foo2()
    {
        echo self::MY_PUBLIC;
        echo self::MY_PROTECTED;
        echo self::MY_PRIVATE; // Fatal Error
    }
}

$myclass2 = new MyClass2;
echo MyClass2::MY_PUBLIC; // Works
$myclass2->foo2(); // Public and Protected work, not Private
?>
```

Visibility from other objects

Objects of the same type will have access to each others private and protected members even though they are not the same instances. This is because the implementation specific details are already known when inside those objects.

Example #7 Accessing private members of the same object type

```
<?php
class Test
{
    private $foo;

    public function __construct($foo)
    {
        $this->foo = $foo;
    }
```

```
    private function bar()
    {
        echo 'Accessed the private method.';
    }

    public function baz(Test $other)
    {
        // We can change the private property:
        $other->foo = 'hello';
        var_dump($other->foo);

        // We can also call the private method:
        $other->bar();
    }
}

$test = new Test('test');

$test->baz(new Test('other'));
?>
```

The above example will output:

```
string(5) "hello"
Accessed the private method.
```

Object Inheritance

Inheritance is a well-established programming principle, and PHP makes use of this principle in its object model. This principle will affect the way many classes and objects relate to one another.

For example, when extending a class, the subclass inherits all of the public and protected methods, properties and constants from the parent class. Unless a class overrides those methods, they will retain their original functionality.

This is useful for defining and abstracting functionality, and permits the implementation of additional functionality in similar objects without the need to reimplement all of the shared functionality.

Private methods of a parent class are not accessible to a child class. As a result, child classes may reimplement a private method themselves without regard for normal inheritance rules. Prior to PHP 8.0.0, however, `final` and `static` restrictions were applied to private methods. As of PHP 8.0.0, the only private method restriction that is enforced is `private final` constructors, as that is a common way to "disable" the constructor when using static factory methods instead.

The visibility of methods, properties and constants can be relaxed, e.g. a `protected` method can be marked as `public`, but they cannot be restricted, e.g. marking a `public` property as `private`. One exception is constructors, whose visibility can be restricted, e.g. a `public` constructor can be marked as `private` in a child class.

> **Note** :
>
> Unless autoloading is used, the classes must be defined before they are used. If a class extends another, then the parent class must be declared before the child class structure. This rule applies to classes that inherit other classes and interfaces.

> **Note** :
>
> It is not allowed to override a read-write property with a readonly property or vice versa.

```php
<?php

class A {
    public int $prop;
}
class B extends A {
    // Illegal: read-write -> readonly
    public readonly int $prop;
}
?>
```

Example #1 Inheritance Example

```php
<?php

class Foo
{
    public function printItem($string)
    {
        echo 'Foo: ' . $string . PHP_EOL;
    }

    public function printPHP()
    {
        echo 'PHP is great.' . PHP_EOL;
    }
}

class Bar extends Foo
{
    public function printItem($string)
    {
        echo 'Bar: ' . $string . PHP_EOL;
    }
}

$foo = new Foo();
$bar = new Bar();
$foo->printItem('baz'); // Output: 'Foo: baz'
$foo->printPHP();       // Output: 'PHP is great.'
$bar->printItem('baz'); // Output: 'Bar: baz'
$bar->printPHP();       // Output: 'PHP is great.'
```

```
?>
```

Return Type Compatibility with Internal Classes

Prior to PHP 8.1, most internal classes or methods didn't declare their return types, and any return type was allowed when extending them.

As of PHP 8.1.0, most internal methods started to "tentatively" declare their return type, in that case the return type of methods should be compatible with the parent being extended; otherwise, a deprecation notice is emitted. Note that lack of an explicit return declaration is also considered a signature mismatch, and thus results in the deprecation notice.

If the return type cannot be declared for an overriding method due to PHP cross-version compatibility concerns, a ReturnTypeWillChange attribute can be added to silence the deprecation notice.

Example #2 The overriding method does not declare any return type

```php
<?php
class MyDateTime extends DateTime
{
    public function modify(string $modifier) { return false; }
}

// "Deprecated: Return type of MyDateTime::modify(string $modifier) should either be compatible with DateTime::modify(string $modifier): DateTime|false, or the #[\ReturnTypeWillChange] attribute should be used to temporarily suppress the notice" as of PHP 8.1.0
?>
```

Example #3 The overriding method declares a wrong return type

```php
<?php
class MyDateTime extends DateTime
{
    public function modify(string $modifier): ?DateTime { return null; }
}

// "Deprecated: Return type of MyDateTime::modify(string $modifier): ?DateTime should either be compatible with DateTime::modify(string $modifier): DateTime|false, or the #[\ReturnTypeWillChange] attribute should be used to temporarily suppress the notice" as of PHP 8.1.0
?>
```

Example #4 The overriding method declares a wrong return type without a deprecation notice

```php
<?php
class MyDateTime extends DateTime
{
    /**
     * @return DateTime|false
     */
    #[\ReturnTypeWillChange]
    public function modify(string $modifier) { return false; }
}
```

```
// No notice is triggered
?>
```

Scope Resolution Operator (::)

The Scope Resolution Operator (also called Paamayim Nekudotayim) or in simpler terms, the double colon, is a token that allows access to a constant, static property, or static method of a class or one of its parents. Moreover, static properties or methods can be overridden via late static binding.

When referencing these items from outside the class definition, use the name of the class.

It's possible to reference the class using a variable. The variable's value can not be a keyword (e.g. `self`, `parent` and `static`).

Paamayim Nekudotayim would, at first, seem like a strange choice for naming a double-colon. However, while writing the Zend Engine 0.5 (which powers PHP 3), that's what the Zend team decided to call it. It actually does mean double-colon - in Hebrew!

Example #1 :: from outside the class definition

```
<?php
class MyClass {
    const CONST_VALUE = 'A constant value';
}

$classname = 'MyClass';
echo $classname::CONST_VALUE;

echo MyClass::CONST_VALUE;
?>
```

Three special keywords *self*, *parent* and *static* are used to access properties or methods from inside the class definition.

Example #2 :: from inside the class definition

```
<?php
class MyClass {
    const CONST_VALUE = 'A constant value';
}

class OtherClass extends MyClass
{
    public static $my_static = 'static var';

    public static function doubleColon() {
        echo parent::CONST_VALUE . "\n";
        echo self::$my_static . "\n";
    }
}

$classname = 'OtherClass';
$classname::doubleColon();
```

```
OtherClass::doubleColon();
?>
```

When an extending class overrides the parent's definition of a method, PHP will not call the parent's method. It's up to the extended class whether or not the parent's method is called. This also applies to Constructors and Destructors, Overloading, and Magic method definitions.

Example #3 Calling a parent's method

```
<?php
class MyClass
{
    protected function myFunc() {
        echo "MyClass::myFunc()\n";
    }
}

class OtherClass extends MyClass
{
    // Override parent's definition
    public function myFunc()
    {
        // But still call the parent function
        parent::myFunc();
        echo "OtherClass::myFunc()\n";
    }
}

$class = new OtherClass();
$class->myFunc();
?>
```

See also some examples of static call trickery.

Static Keyword

> **Tip**
>
> This page describes the use of the `static` keyword to define static methods and properties. `static` can also be used to define static variables, define static anonymous functions and for late static bindings. Please refer to those pages for information on those meanings of `static`.

Declaring class properties or methods as static makes them accessible without needing an instantiation of the class. These can also be accessed statically within an instantiated class object.

Static methods

Because static methods are callable without an instance of the object created, the pseudo-variable *$this* is not available inside methods declared as static.

> **Warning**
>
> Calling non-static methods statically throws an Error.

> Prior to PHP 8.0.0, calling non-static methods statically was deprecated, and generated an **E_DEPRECATED** warning.

Example #1 Static method example

```php
<?php
class Foo {
    public static function aStaticMethod() {
        // ...
    }
}

Foo::aStaticMethod();
$classname = 'Foo';
$classname::aStaticMethod();
?>
```

Static properties

Static properties are accessed using the Scope Resolution Operator (`::`) and cannot be accessed through the object operator (`->`).

It's possible to reference the class using a variable. The variable's value cannot be a keyword (e.g. `self`, `parent` and `static`).

Example #2 Static property example

```php
<?php
class Foo
{
    public static $my_static = 'foo';

    public function staticValue() {
        return self::$my_static;
    }
}

class Bar extends Foo
{
    public function fooStatic() {
        return parent::$my_static;
    }
}


print Foo::$my_static . "\n";

$foo = new Foo();
print $foo->staticValue() . "\n";
print $foo->my_static . "\n";      // Undefined "Property" my_static

print $foo::$my_static . "\n";
$classname = 'Foo';
print $classname::$my_static . "\n";
```

```php
print Bar::$my_static . "\n";
$bar = new Bar();
print $bar->fooStatic() . "\n";
?>
```

Output of the above example in PHP 8 is similar to:

```
foo
foo

Notice: Accessing static property Foo::$my_static as non static in /in/V0Rvv on line 23

Warning: Undefined property: Foo::$my_static in /in/V0Rvv on line 23

foo
foo
foo
foo
```

Class Abstraction

PHP has abstract classes, methods, and properties. Classes defined as abstract cannot be instantiated, and any class that contains at least one abstract method or property must also be abstract. Methods defined as abstract simply declare the method's signature and whether it is public or protected; they cannot define the implementation. Properties defined as abstract may declare a requirement for `get` or `set` behavior, and may provide an implementation for one, but not both, operations.

When inheriting from an abstract class, all methods marked abstract in the parent's class declaration must be defined by the child class, and follow the usual inheritance and signature compatibility rules.

As of PHP 8.4, an abstract class may declare an abstract property, either public or protected. A protected abstract property may be satisfied by a property that is readable/writeable from either protected or public scope.

An abstract property may be satisfied either by a standard property or by a property with defined hooks, corresponding to the required operation.

Example #1 Abstract method example

```php
<?php

abstract class AbstractClass
{
    // Force extending class to define this method
    abstract protected function getValue();
    abstract protected function prefixValue($prefix);

    // Common method
    public function printOut()
    {
```

```
        print $this->getValue() . "\n";
    }
}

class ConcreteClass1 extends AbstractClass
{
    protected function getValue()
    {
        return "ConcreteClass1";
    }

    public function prefixValue($prefix)
    {
        return "{$prefix}ConcreteClass1";
    }
}

class ConcreteClass2 extends AbstractClass
{
    public function getValue()
    {
        return "ConcreteClass2";
    }

    public function prefixValue($prefix)
    {
        return "{$prefix}ConcreteClass2";
    }
}

$class1 = new ConcreteClass1();
$class1->printOut();
echo $class1->prefixValue('FOO_'), "\n";

$class2 = new ConcreteClass2();
$class2->printOut();
echo $class2->prefixValue('FOO_'), "\n";

?>
```

The above example will output:

```
ConcreteClass1
FOO_ConcreteClass1
ConcreteClass2
FOO_ConcreteClass2
```

Example #2 Abstract method example

```
<?php

abstract class AbstractClass
```

```php
{
    // An abstract method only needs to define the required arguments
    abstract protected function prefixName($name);
}

class ConcreteClass extends AbstractClass
{
    // A child class may define optional parameters which are not present in the parent's signature
    public function prefixName($name, $separator = ".")
    {
        if ($name == "Pacman") {
            $prefix = "Mr";
        } elseif ($name == "Pacwoman") {
            $prefix = "Mrs";
        } else {
            $prefix = "";
        }

        return "{$prefix}{$separator} {$name}";
    }
}

$class = new ConcreteClass();
echo $class->prefixName("Pacman"), "\n";
echo $class->prefixName("Pacwoman"), "\n";

?>
```

The above example will output:

```
Mr. Pacman
Mrs. Pacwoman
```

Example #3 Abstract property example

```php
<?php

abstract class A
{
    // Extending classes must have a publicly-gettable property
    abstract public string $readable {
        get;
    }

    // Extending classes must have a protected- or public-writeable property
    abstract protected string $writeable {
        set;
    }

    // Extending classes must have a protected or public symmetric property
    abstract protected string $both {
        get;
```

```
        set;
    }
}

class C extends A
{
    // This satisfies the requirement and also makes it settable, which is valid
    public string $readable;

    // This would NOT satisfy the requirement, as it is not publicly readable
    protected string $readable;

    // This satisfies the requirement exactly, so is sufficient.
    // It may only be written to, and only from protected scope
    protected string $writeable {
        set => $value;
    }

    // This expands the visibility from protected to public, which is fine
    public string $both;
}

?>
```

An abstract property on an abstract class may provide implementations for any hook, but must have either `get` or `set` declared but not defined (as in the example above).

Example #4 Abstract property with hooks example

```
<?php

abstract class A
{
    // This provides a default (but overridable) set implementation,
    // and requires child classes to provide a get implementation
    abstract public string $foo {
        get;

        set {
            $this->foo = $value;
        }
    }
}

?>
```

Object Interfaces

Object interfaces allow you to create code which specifies which methods and properties a class must implement, without having to define how these methods or properties are implemented. Interfaces share a namespace with classes, traits, and enumerations, so they may not use the same name.

Interfaces are defined in the same way as a class, but with the `interface` keyword replacing the `class` keyword and without any of the methods having their contents defined.

All methods declared in an interface must be public; this is the nature of an interface.

In practice, interfaces serve two complementary purposes:

- To allow developers to create objects of different classes that may be used interchangeably because they implement the same interface or interfaces. A common example is multiple database access services, multiple payment gateways, or different caching strategies. Different implementations may be swapped out without requiring any changes to the code that uses them.
- To allow a function or method to accept and operate on a parameter that conforms to an interface, while not caring what else the object may do or how it is implemented. These interfaces are often named like `Iterable`, `Cacheable`, `Renderable`, and so on to describe the significance of the behavior.

Interfaces may define magic methods to require implementing classes to implement those methods.

> **Note** :
>
> Although they are supported, including constructors in interfaces is strongly discouraged. Doing so significantly reduces the flexibility of the object implementing the interface. Additionally, constructors are not enforced by inheritance rules, which can cause inconsistent and unexpected behavior.

`implements`

To implement an interface, the `implements` operator is used. All methods in the interface must be implemented within a class; failure to do so will result in a fatal error. Classes may implement more than one interface if desired by separating each interface with a comma.

> **Warning**
>
> A class that implements an interface may use a different name for its parameters than the interface. However, as of PHP 8.0 the language supports named arguments, which means callers may rely on the parameter name in the interface. For that reason, it is strongly recommended that developers use the same parameter names as the interface being implemented.

> **Note** :
>
> Interfaces can be extended like classes using the extends operator.

> **Note** :
>
> The class implementing the interface must declare all methods in the interface with a compatible signature. A class can implement multiple interfaces which declare a method with the same name. In this case, the implementation must follow the signature compatibility rules for all the interfaces. So covariance and contravariance can be applied.

Constants

It's possible for interfaces to have constants. Interface constants work exactly like class constants. Prior to PHP 8.1.0, they cannot be overridden by a class/interface that inherits them.

Properties

As of PHP 8.4.0, interfaces may also declare properties. If they do, the declaration must specify if the property is to be readable, writeable, or both. The interface declaration applies only to public read and write access.

A class may satisfy an interface property in multiple ways. It may define a public property. It may define a public virtual property that implements only the corresponding hook. Or a read property may be satisfied by a `readonly` property. However, an interface property that is settable may not be `readonly`.

Example #1 Interface properties example

```php
<?php
interface I
{
    // An implementing class MUST have a publicly-readable property,
    // but whether or not it's publicly settable is unrestricted.
    public string $readable { get; }

    // An implementing class MUST have a publicly-writeable property,
    // but whether or not it's publicly readable is unrestricted.
    public string $writeable { set; }

    // An implementing class MUST have a property that is both publicly
    // readable and publicly writeable.
    public string $both { get; set; }
}

// This class implements all three properties as traditional, un-hooked
// properties. That's entirely valid.
class C1 implements I
{
    public string $readable;

    public string $writeable;

    public string $both;
}

// This class implements all three properties using just the hooks
// that are requested.  This is also entirely valid.
class C2 implements I
{
    private string $written = '';
    private string $all = '';

    // Uses only a get hook to create a virtual property.
    // This satisfies the "public get" requirement.
    // It is not writeable, but that is not required by the interface.
    public string $readable { get => strtoupper($this->writeable); }

    // The interface only requires the property be settable,
    // but also including get operations is entirely valid.
    // This example creates a virtual property, which is fine.
    public string $writeable {
        get => $this->written;
        set {
            $this->written = $value;
        }
    }
```

```
    // This property requires both read and write be possible,
    // so we need to either implement both, or allow it to have
    // the default behavior.
    public string $both {
        get => $this->all;
        set {
            $this->all = strtoupper($value);
        }
    }
}
?>
```

Examples

Example #2 Interface example

```
<?php

// Declare the interface 'Template'
interface Template
{
    public function setVariable($name, $var);
    public function getHtml($template);
}

// Implement the interface
// This will work
class WorkingTemplate implements Template
{
    private $vars = [];

    public function setVariable($name, $var)
    {
        $this->vars[$name] = $var;
    }

    public function getHtml($template)
    {
        foreach($this->vars as $name => $value) {
            $template = str_replace('{' . $name . '}', $value, $template);
        }

        return $template;
    }
}

// This will not work
// Fatal error: Class BadTemplate contains 1 abstract methods
// and must therefore be declared abstract (Template::getHtml)
class BadTemplate implements Template
{
    private $vars = [];

    public function setVariable($name, $var)
    {
        $this->vars[$name] = $var;
```

```
    }
}
?>
```

Example #3 Extendable Interfaces

```
<?php
interface A
{
    public function foo();
}

interface B extends A
{
    public function baz(Baz $baz);
}

// This will work
class C implements B
{
    public function foo()
    {
    }

    public function baz(Baz $baz)
    {
    }
}

// This will not work and result in a fatal error
class D implements B
{
    public function foo()
    {
    }

    public function baz(Foo $foo)
    {
    }
}
?>
```

Example #4 Variance compatibility with multiple interfaces

```
<?php
class Foo {}
class Bar extends Foo {}

interface A {
    public function myfunc(Foo $arg): Foo;
}

interface B {
    public function myfunc(Bar $arg): Bar;
}
```

```php
class MyClass implements A, B
{
    public function myfunc(Foo $arg): Bar
    {
        return new Bar();
    }
}
?>
```

Example #5 Multiple interface inheritance

```php
<?php
interface A
{
    public function foo();
}

interface B
{
    public function bar();
}

interface C extends A, B
{
    public function baz();
}

class D implements C
{
    public function foo()
    {
    }

    public function bar()
    {
    }

    public function baz()
    {
    }
}
?>
```

Example #6 Interfaces with constants

```php
<?php
interface A
{
    const B = 'Interface constant';
}

// Prints: Interface constant
echo A::B;


class B implements A
```

```
{
    const B = 'Class constant';
}

// Prints: Class constant
// Prior to PHP 8.1.0, this will however not work because it was not
// allowed to override constants.
echo B::B;
?>
```

Example #7 Interfaces with abstract classes

```
<?php
interface A
{
    public function foo(string $s): string;

    public function bar(int $i): int;
}

// An abstract class may implement only a portion of an interface.
// Classes that extend the abstract class must implement the rest.
abstract class B implements A
{
    public function foo(string $s): string
    {
        return $s . PHP_EOL;
    }
}

class C extends B
{
    public function bar(int $i): int
    {
        return $i * 2;
    }
}
?>
```

Example #8 Extending and implementing simultaneously

```
<?php

class One
{
    /* ... */
}

interface Usable
{
    /* ... */
}

interface Updatable
{
    /* ... */
```

```
}

// The keyword order here is important. 'extends' must come first.
class Two extends One implements Usable, Updatable
{
    /* ... */
}
?>
```

An interface, together with type declarations, provides a good way to make sure that a particular object contains particular methods. See instanceof operator and type declarations.

Traits

PHP implements a way to reuse code called Traits.

Traits are a mechanism for code reuse in single inheritance languages such as PHP. A Trait is intended to reduce some limitations of single inheritance by enabling a developer to reuse sets of methods freely in several independent classes living in different class hierarchies. The semantics of the combination of Traits and classes is defined in a way which reduces complexity, and avoids the typical problems associated with multiple inheritance and Mixins.

A Trait is similar to a class, but only intended to group functionality in a fine-grained and consistent way. It is not possible to instantiate a Trait on its own. It is an addition to traditional inheritance and enables horizontal composition of behavior; that is, the application of class members without requiring inheritance.

Example #1 Trait example

```
<?php

trait TraitA {
    public function sayHello() {
        echo 'Hello';
    }
}

trait TraitB {
    public function sayWorld() {
        echo 'World';
    }
}

class MyHelloWorld
{
    use TraitA, TraitB; // A class can use multiple traits

    public function sayHelloWorld() {
        $this->sayHello();
        echo ' ';
        $this->sayWorld();
        echo "!\n";
    }
}

$myHelloWorld = new MyHelloWorld();
$myHelloWorld->sayHelloWorld();
```

```
?>
```

The above example will output:

```
Hello World!
```

Precedence

An inherited member from a base class is overridden by a member inserted by a Trait. The precedence order is that members from the current class override Trait methods, which in turn override inherited methods.

> **Note** :
>
> As of PHP 8.5.0, traits are bound to a class before its parent class; previously they were bound after. This more closely reflects the copy and paste semantics of traits. In particular, when a trait and the parent class both declare a property or constant of the same name, the trait's definition now takes precedence over the inherited one, whereas earlier versions raised a fatal error about an incompatible composition. Method resolution is unaffected. See the migration guide for details.

Example #2 Precedence Order Example

An inherited method from a base class is overridden by the method inserted into MyHelloWorld from the SayWorld Trait. The behavior is the same for methods defined in the MyHelloWorld class. The precedence order is that methods from the current class override Trait methods, which in turn override methods from the base class.

```
<?php
class Base {
    public function sayHello() {
        echo 'Hello ';
    }
}

trait SayWorld {
    public function sayHello() {
        parent::sayHello();
        echo 'World!';
    }
}

class MyHelloWorld extends Base {
    use SayWorld;
}

$o = new MyHelloWorld();
$o->sayHello();
?>
```

The above example will output:

```
Hello World!
```

Example #3 Alternate Precedence Order Example

```php
<?php
trait HelloWorld {
    public function sayHello() {
        echo 'Hello World!';
    }
}

class TheWorldIsNotEnough {
    use HelloWorld;
    public function sayHello() {
        echo 'Hello Universe!';
    }
}

$o = new TheWorldIsNotEnough();
$o->sayHello();
?>
```

The above example will output:

```
Hello Universe!
```

Multiple Traits

Multiple Traits can be inserted into a class by listing them in the `use` statement, separated by commas.

Example #4 Multiple Traits Usage

```php
<?php
trait Hello {
    public function sayHello() {
        echo 'Hello ';
    }
}

trait World {
    public function sayWorld() {
        echo 'World';
    }
}
```

```php
class MyHelloWorld {
    use Hello, World;
    public function sayExclamationMark() {
        echo '!';
    }
}

$o = new MyHelloWorld();
$o->sayHello();
$o->sayWorld();
$o->sayExclamationMark();
?>
```

The above example will output:

```
Hello World!
```

Conflict Resolution

If two Traits insert a method with the same name, a fatal error is produced, if the conflict is not explicitly resolved.

To resolve naming conflicts between Traits used in the same class, the `insteadof` operator needs to be used to choose exactly one of the conflicting methods.

Since this only allows one to exclude methods, the `as` operator can be used to add an alias to one of the methods. Note the `as` operator does not rename the method and it does not affect any other method either.

Example #5 Conflict Resolution

In this example, Talker uses the traits A and B. Since A and B have conflicting methods, it specifies that it uses the variant of smallTalk from trait B, and the variant of bigTalk from trait A.

The Aliased_Talker makes use of the `as` operator to be able to use B's bigTalk implementation under an additional alias `talk`.

```php
<?php
trait A {
    public function smallTalk() {
        echo 'a';
    }
    public function bigTalk() {
        echo 'A';
    }
}

trait B {
    public function smallTalk() {
        echo 'b';
    }
```

```
    public function bigTalk() {
        echo 'B';
    }
}

class Talker {
    use A, B {
        B::smallTalk insteadof A;
        A::bigTalk insteadof B;
    }
}

class Aliased_Talker {
    use A, B {
        B::smallTalk insteadof A;
        A::bigTalk insteadof B;
        B::bigTalk as talk;
    }
}
?>
```

Changing Method Visibility

Using the `as` syntax, one can also adjust the visibility of the method in the exhibiting class.

Example #6 Changing Method Visibility

```
<?php
trait HelloWorld {
    public function sayHello() {
        echo 'Hello World!';
    }
}

// Change visibility of sayHello
class MyClass1 {
    use HelloWorld { sayHello as protected; }
}

// Alias method with changed visibility
// sayHello visibility not changed
class MyClass2 {
    use HelloWorld { sayHello as private myPrivateHello; }
}
?>
```

Traits Composed from Traits

Just as classes can make use of traits, so can other traits. By using one or more traits in a trait definition, it can be composed partially or entirely of the members defined in those other traits.

Example #7 Traits Composed from Traits

```
<?php
trait Hello {
    public function sayHello() {
```

```
        echo 'Hello ';
    }
}

trait World {
    public function sayWorld() {
        echo 'World!';
    }
}

trait HelloWorld {
    use Hello, World;
}

class MyHelloWorld {
    use HelloWorld;
}

$o = new MyHelloWorld();
$o->sayHello();
$o->sayWorld();
?>
```

The above example will output:

```
Hello World!
```

Abstract Trait Members

Traits support the use of abstract methods in order to impose requirements upon the exhibiting class. Public, protected, and private methods are supported. Prior to PHP 8.0.0, only public and protected abstract methods were supported.

> **Caution**
>
> As of PHP 8.0.0, the signature of a concrete method must follow the signature compatibility rules. Previously, its signature might be different.

Example #8 Express Requirements by Abstract Methods

```
<?php
trait Hello {
    public function sayHelloWorld() {
        echo 'Hello'.$this->getWorld();
    }
    abstract public function getWorld();
}

class MyHelloWorld {
    private $world;
```

```
    use Hello;
    public function getWorld() {
        return $this->world;
    }
    public function setWorld($val) {
        $this->world = $val;
    }
}
?>
```

Static Trait Members

Traits can define static variables, static methods and static properties.

> **Note** :
>
> As of PHP 8.1.0, calling a static method, or accessing a static property directly on a trait is deprecated. Static methods and properties should only be accessed on a class using the trait.

Example #9 Static Variables

```
<?php

trait Counter
{
    public function inc()
    {
        static $c = 0;
        $c = $c + 1;
        echo "$c\n";
    }
}

class C1
{
    use Counter;
}

class C2
{
    use Counter;
}

$o = new C1();
$o->inc();
$p = new C2();
$p->inc();

?>
```

The above example will output:

```

```

```
1
1
```

Example #10 Static Methods

```php
<?php

trait StaticExample
{
    public static function doSomething()
    {
        return 'Doing something';
    }
}

class Example
{
    use StaticExample;
}

echo Example::doSomething();

?>
```

The above example will output:

```
Doing something
```

Example #11 Static Properties

> **Caution**
>
> Prior to PHP 8.3.0, static properties defined in a trait were shared across all classes in the same inheritance hierarchy which used that trait. As of PHP 8.3.0, if a child class uses a trait with a static property, it will be considered distinct from the one defined in the parent class.

```php
<?php

trait T
{
    public static $counter = 1;
}

class A
{
    use T;
```

```
    public static function incrementCounter()
    {
        static::$counter++;
    }
}

class B extends A
{
    use T;
}

A::incrementCounter();

echo A::$counter, "\n";
echo B::$counter, "\n";

?>
```

Output of the above example in PHP 8.3:

```
2
1
```

Properties

Traits can also define properties.

Example #12 Defining Properties

```
<?php

trait PropertiesTrait
{
    public $x = 1;
}

class PropertiesExample
{
    use PropertiesTrait;
}

$example = new PropertiesExample();
$example->x;

?>
```

If a trait defines a property then a class can not define a property with the same name unless it is compatible (same visibility and type, readonly modifier, and initial value), otherwise a fatal error is issued.

Example #13 Conflict Resolution

```php
<?php
trait PropertiesTrait {
    public $same = true;
    public $different1 = false;
    public bool $different2;
    public bool $different3;
}

class PropertiesExample {
    use PropertiesTrait;
    public $same = true;
    public $different1 = true; // Fatal error
    public string $different2; // Fatal error
    readonly protected bool $different3; // Fatal error
}
?>
```

Constants

Traits can, as of PHP 8.2.0, also define constants.

Example #14 Defining Constants

```php
<?php
trait ConstantsTrait {
    public const FLAG_MUTABLE = 1;
    final public const FLAG_IMMUTABLE = 5;
}

class ConstantsExample {
    use ConstantsTrait;
}

$example = new ConstantsExample;
echo $example::FLAG_MUTABLE;
?>
```

The above example will output:

```
1
```

If a trait defines a constant then a class can not define a constant with the same name unless it is compatible (same visibility, initial value, and finality), otherwise a fatal error is issued.

Example #15 Conflict Resolution

```php
<?php
trait ConstantsTrait {
    public const FLAG_MUTABLE = 1;
    final public const FLAG_IMMUTABLE = 5;
}
```

```php
class ConstantsExample {
    use ConstantsTrait;
    public const FLAG_IMMUTABLE = 5; // Fatal error
}
?>
```

Final methods

As of PHP 8.3.0, the final modifier can be applied using the `as` operator to methods imported from traits. This can be used to prevent child classes from overriding the method. However, the class that uses the trait can still override the method.

Example #16 Defining a method coming from a trait as `final`

```php
<?php

trait CommonTrait
{
    public function method()
    {
        echo 'Hello';
    }
}

class FinalExampleA
{
    use CommonTrait {
        CommonTrait::method as final; // The 'final' prevents child classes from overriding the
method
    }
}

class FinalExampleB extends FinalExampleA
{
    public function method() {}
}

?>
```

The above example will output something similar to:

```
Fatal error: Cannot override final method FinalExampleA::method() in ...
```

Anonymous classes

Anonymous classes are useful when simple, one-off objects need to be created.

```php
<?php
```

```php
// Using an explicit class
class Logger
{
    public function log($msg)
    {
        echo $msg;
    }
}

$util->setLogger(new Logger());

// Using an anonymous class
$util->setLogger(new class {
    public function log($msg)
    {
        echo $msg;
    }
});
```

They can pass arguments through to their constructors, extend other classes, implement interfaces, and use traits just like a normal class can:

```php
<?php

class SomeClass {}
interface SomeInterface {}
trait SomeTrait {}

var_dump(new class(10) extends SomeClass implements SomeInterface {
    private $num;

    public function __construct($num)
    {
        $this->num = $num;
    }

    use SomeTrait;
});
```

The above example will output:

```
object(class@anonymous)#1 (1) {
  ["Command line code0x104c5b612":"class@anonymous":private]=>
  int(10)
}
```

Nesting an anonymous class within another class does not give it access to any private or protected methods or properties of that outer class. In order to use the outer class' protected properties or methods, the anonymous class can extend the outer class. To use the private properties of the outer class in the anonymous class, they must be passed through its constructor:

```php
<?php

class Outer
{
    private $prop = 1;
    protected $prop2 = 2;

    protected function func1()
    {
        return 3;
    }

    public function func2()
    {
        return new class($this->prop) extends Outer {
            private $prop3;

            public function __construct($prop)
            {
                $this->prop3 = $prop;
            }

            public function func3()
            {
                return $this->prop2 + $this->prop3 + $this->func1();
            }
        };
    }
}

echo (new Outer)->func2()->func3();
```

The above example will output:

```
6
```

All objects created by the same anonymous class declaration are instances of that very class.

```php
<?php
function anonymous_class()
{
    return new class {};
}

if (get_class(anonymous_class()) === get_class(anonymous_class())) {
    echo 'same class';
} else {
    echo 'different class';
}
```

The above example will output:

```
same class
```

> **Note** :
>
> Note that anonymous classes are assigned a name by the engine, as demonstrated in the following example. This name has to be regarded as an implementation detail, which should not be relied upon.
>
> ```
> <?php
> echo get_class(new class {});
> ```
>
> The above example will output something similar to:
>
> ```
> class@anonymous/in/oNi1A0x7f8636ad2021
> ```

Readonly anonymous classes

As of PHP 8.3.0, the `readonly` modifier can be applied to anonymous classes.

Example #1 Defining a readonly anonymous class

```
<?php
// Using an anonymous class
var_dump(new readonly class('[DEBUG]') {
    public function __construct(private string $prefix)
    {
    }

    public function log($msg)
    {
        echo $this->prefix . ' ' . $msg;
    }
});
```

Overloading

Overloading in PHP provides means to dynamically “create” properties and methods. These dynamic entities are processed via magic methods one can establish in a class for various action types.

The overloading methods are invoked when interacting with properties or methods that have not been declared or are not visible in the current scope. The rest of this section will use the terms “inaccessible properties” and “inaccessible methods” to refer to this combination of declaration and visibility.

All overloading methods must be defined as `public`.

> **Note** :
>
> None of the arguments of these magic methods can be passed by reference.

> **Note** :
>
> PHP's interpretation of "overloading" is different than most object-oriented languages. Overloading traditionally provides the ability to have multiple methods with the same name but different quantities and types of arguments.

Property overloading

public function **__set**(string `$name`, mixed `$value`): void
public function **__get**(string `$name`): mixed
public function **__isset**(string `$name`): bool
public function **__unset**(string `$name`): void

__set() is run when writing data to inaccessible (protected or private) or non-existing properties.

__get() is utilized for reading data from inaccessible (protected or private) or non-existing properties.

__isset() is triggered by calling isset() or empty() on inaccessible (protected or private) or non-existing properties.

__unset() is invoked when unset() is used on inaccessible (protected or private) or non-existing properties.

The *$name* argument is the name of the property being interacted with. The __set() method's *$value* argument specifies the value the *$name*'ed property should be set to.

Property overloading only works in object context. These magic methods will not be triggered in static context. Therefore these methods should not be declared static. A warning is issued if one of the magic overloading methods is declared `static`.

> **Note** :
>
> The return value of __set() is ignored because of the way PHP processes the assignment operator. Similarly, __get() is never called when chaining assignments together like this:
>
> ```
> $a = $obj->b = 8;
> ```

> **Note** :
>
> PHP will not call an overloaded method from within the same overloaded method. That means, for example, writing `return $this->foo` inside of __get() will return `null` and raise an `E_WARNING` if there is no `foo` property defined, rather than calling __get() a second time. However, overload methods may invoke other overload methods implicitly (such as __set() triggering __get()).

Example #1 Overloading properties via the __get(), __set(), __isset() and __unset() methods

```php
<?php
class PropertyTest
{
    /**  Location for overloaded data.  */
    private $data = array();

    /**  Overloading not used on declared properties.  */
    public $declared = 1;

    /**  Overloading only used on this when accessed outside the class.  */
    private $hidden = 2;

    public function __set($name, $value)
    {
        echo "Setting '$name' to '$value'\n";
        $this->data[$name] = $value;
    }

    public function __get($name)
    {
        echo "Getting '$name'\n";
        if (array_key_exists($name, $this->data)) {
            return $this->data[$name];
        }

        $trace = debug_backtrace();
        trigger_error(
            'Undefined property via __get(): ' . $name .
            ' in ' . $trace[0]['file'] .
            ' on line ' . $trace[0]['line'],
            E_USER_NOTICE);
        return null;
    }

    public function __isset($name)
    {
        echo "Is '$name' set?\n";
        return isset($this->data[$name]);
    }

    public function __unset($name)
    {
        echo "Unsetting '$name'\n";
        unset($this->data[$name]);
    }

    /**  Not a magic method, just here for example.  */
    public function getHidden()
    {
        return $this->hidden;
    }
}


$obj = new PropertyTest;

$obj->a = 1;
```

```php
echo $obj->a . "\n\n";

var_dump(isset($obj->a));
unset($obj->a);
var_dump(isset($obj->a));
echo "\n";

echo $obj->declared . "\n\n";

echo "Let's experiment with the private property named 'hidden':\n";
echo "Privates are visible inside the class, so __get() not used...\n";
echo $obj->getHidden() . "\n";
echo "Privates not visible outside of class, so __get() is used...\n";
echo $obj->hidden . "\n";
?>
```

The above example will output:

```
Setting 'a' to '1'
Getting 'a'
1

Is 'a' set?
bool(true)
Unsetting 'a'
Is 'a' set?
bool(false)

1

Let's experiment with the private property named 'hidden':
Privates are visible inside the class, so __get() not used...
2
Privates not visible outside of class, so __get() is used...
Getting 'hidden'

Notice: Undefined property via __get(): hidden in script on line 71 in script on line 27
```

Method overloading

public function **__call**(string `$name`, array `$arguments`): mixed
public static function **__callStatic**(string `$name`, array `$arguments`): mixed

__call() is triggered when invoking inaccessible methods in an object context.

__callStatic() is triggered when invoking inaccessible methods in a static context.

The *$name* argument is the name of the method being called. The *$arguments* argument is an enumerated array containing the parameters passed to the *$name*'ed method.

Example #2 Overloading methods via the __call() and __callStatic() methods

```php
<?php
class MethodTest
{
    public function __call($name, $arguments)
    {
        // Note: value of $name is case sensitive.
        echo "Calling object method '$name' "
             . implode(', ', $arguments). "\n";
    }

    public static function __callStatic($name, $arguments)
    {
        // Note: value of $name is case sensitive.
        echo "Calling static method '$name' "
             . implode(', ', $arguments). "\n";
    }
}

$obj = new MethodTest;
$obj->runTest('in object context');

MethodTest::runTest('in static context');
?>
```

The above example will output:

```
Calling object method 'runTest' in object context
Calling static method 'runTest' in static context
```

Object Iteration

PHP provides a way for objects to be defined so it is possible to iterate through a list of items with, for example, a `foreach` statement. By default, all visible properties will be used for the iteration.

Example #1 Simple Object Iteration

```php
<?php
class MyClass
{
    public $var1 = 'value 1';
    public $var2 = 'value 2';
    public $var3 = 'value 3';

    protected $protected = 'protected var';
    private   $private   = 'private var';

    function iterateVisible() {
       echo "MyClass::iterateVisible:\n";
       foreach ($this as $key => $value) {
           print "$key => $value\n";
       }
```

```
    }
}

$class = new MyClass();

foreach($class as $key => $value) {
    print "$key => $value\n";
}
echo "\n";


$class->iterateVisible();

?>
```

The above example will output:

```
var1 => value 1
var2 => value 2
var3 => value 3

MyClass::iterateVisible:
var1 => value 1
var2 => value 2
var3 => value 3
protected => protected var
private => private var
```

As the output shows, the `foreach` iterated through all of the visible properties that could be accessed.

See Also

- Generators
- Iterator
- IteratorAggregate
- SPL Iterators

Magic Methods

Magic methods are special methods which override PHP's default action when certain actions are performed on an object.

> **Caution**
>
> All method names starting with __ are reserved by PHP. Therefore, it is not recommended to use such method names unless overriding PHP's behavior.

The following method names are considered magical: __construct(), __destruct(), __call(), __callStatic(), __get(), __set(), __isset(), __unset(), __serialize(), __unserialize(), __sleep(), __wakeup(), __toString(), __invoke(),

__set_state(), __clone(), and __debugInfo().

> **Warning**
>
> All magic methods, with the exception of __construct(), __destruct(), and __clone(), *must* be declared as `public`, otherwise an **`E_WARNING`** is emitted. Prior to PHP 8.0.0, no diagnostic was emitted for the magic methods __sleep(), __wakeup(), __serialize(), __unserialize(), and __set_state().

> **Warning**
>
> If type declarations are used in the definition of a magic method, they must be identical to the signature described in this document. Otherwise, a fatal error is emitted. Prior to PHP 8.0.0, no diagnostic was emitted. However, __construct() and __destruct() must not declare a return type; otherwise a fatal error is emitted.

__serialize() and __unserialize()

public function **__serialize**(): array
public function **__unserialize**(array `$data`): void

serialize() checks if the class has a function with the magic name __serialize(). If so, that function is executed prior to any serialization. It must construct and return an associative array of key/value pairs that represent the serialized form of the object. If no array is returned a TypeError will be thrown.

> **Note** :
>
> If both __serialize() and __sleep() are defined in the same object, only __serialize() will be called. __sleep() will be ignored. If the object implements the Serializable interface, the interface's `serialize()` method will be ignored and __serialize() used instead.

The intended use of __serialize() is to define a serialization-friendly arbitrary representation of the object. Elements of the array may correspond to properties of the object but that is not required.

Conversely, unserialize() checks for the presence of a function with the magic name __unserialize(). If present, this function will be passed the restored array that was returned from __serialize(). It may then restore the properties of the object from that array as appropriate.

> **Note** :
>
> If both __unserialize() and __wakeup() are defined in the same object, only __unserialize() will be called. __wakeup() will be ignored.

> **Note** :
>
> This feature is available as of PHP 7.4.0.

Example #1 Serialize and unserialize

```php
<?php
class Connection
{
    protected $link;
    private $dsn, $username, $password;

    public function __construct($dsn, $username, $password)
    {
        $this->dsn = $dsn;
        $this->username = $username;
        $this->password = $password;
        $this->connect();
    }

    private function connect()
    {
        $this->link = new PDO($this->dsn, $this->username, $this->password);
    }

    public function __serialize(): array
    {
        return [
          'dsn' => $this->dsn,
          'user' => $this->username,
          'pass' => $this->password,
        ];
    }

    public function __unserialize(array $data): void
    {
        $this->dsn = $data['dsn'];
        $this->username = $data['user'];
        $this->password = $data['pass'];

        $this->connect();
    }
}?>
```

__sleep() and __wakeup()

> **Warning**
>
> This serialization mechanism is soft-deprecated as of PHP 8.5.0. It is maintained for backward compatibility. However, new and existing code should be migrated to use the __serialize() and __unserialize() magic methods instead.

public function **__sleep**(): array
public function **__wakeup**(): void

serialize() checks if the class has a function with the magic name __sleep(). If so, that function is executed prior to any serialization. It can clean up the object and is supposed to return an array with the names of all variables of that object that should be serialized. If the method doesn't return anything then `null` is serialized and `E_NOTICE` is issued.

> **Note** :
>
> It is not possible for __sleep() to return names of private properties in parent classes. Doing this will result in an **E_NOTICE** level error. Use __serialize() instead.

> **Note** :
>
> As of PHP 8.0.0, returning a value which is not an array from __sleep() generates a warning. Previously, it generated a notice.

The intended use of __sleep() is to commit pending data or perform similar cleanup tasks. Also, the function is useful if a very large object doesn't need to be saved completely.

Conversely, unserialize() checks for the presence of a function with the magic name __wakeup(). If present, this function can reconstruct any resources that the object may have.

The intended use of __wakeup() is to reestablish any database connections that may have been lost during serialization and perform other reinitialization tasks.

Example #2 Sleep and wakeup

```php
<?php
class Connection
{
    protected $link;
    private $dsn, $username, $password;

    public function __construct($dsn, $username, $password)
    {
        $this->dsn = $dsn;
        $this->username = $username;
        $this->password = $password;
        $this->connect();
    }

    private function connect()
    {
        $this->link = new PDO($this->dsn, $this->username, $this->password);
    }

    public function __sleep()
    {
        return array('dsn', 'username', 'password');
    }

    public function __wakeup()
    {
        $this->connect();
    }
}?>
```

__toString()

public function **__toString**(): string

The __toString() method allows a class to decide how it will react when it is treated like a string. For example, what `echo $obj;` will print.

> **Warning**
>
> As of PHP 8.0.0, the return value follows standard PHP type semantics, meaning it will be coerced into a string if possible and if strict typing is disabled.
>
> A Stringable object will *not* be accepted by a string type declaration if strict typing is enabled. If such behaviour is wanted the type declaration must accept Stringable and string via a union type.
>
> As of PHP 8.0.0, any class that contains a __toString() method will also implicitly implement the Stringable interface, and will thus pass type checks for that interface. Explicitly implementing the interface anyway is recommended.
>
> In PHP 7.4, the returned value *must* be a string, otherwise an Error is thrown.
>
> Prior to PHP 7.4.0, the returned value *must* be a string, otherwise a fatal **E_RECOVERABLE_ERROR** is emitted.

> **Warning**
>
> It was not possible to throw an exception from within a __toString() method prior to PHP 7.4.0. Doing so will result in a fatal error.

Example #3 Simple example

```php
<?php
// Declare a simple class
class TestClass
{
    public $foo;

    public function __construct($foo)
    {
        $this->foo = $foo;
    }

    public function __toString()
    {
        return $this->foo;
    }
}

$class = new TestClass('Hello');
echo $class;
?>
```

The above example will output:

```
```

```
Hello
```

__invoke()

function **__invoke**(...$values): mixed

The __invoke() method is called when a script tries to call an object as a function.

Example #4 Using __invoke()

```php
<?php
class CallableClass
{
    public function __invoke($x)
    {
        var_dump($x);
    }
}
$obj = new CallableClass;
$obj(5);
var_dump(is_callable($obj));
?>
```

The above example will output:

```
int(5)
bool(true)
```

Example #5 Using __invoke()

```php
<?php
class Sort
{
    private $key;

    public function __construct(string $key)
    {
        $this->key = $key;
    }

    public function __invoke(array $a, array $b): int
    {
        return $a[$this->key] <=> $b[$this->key];
    }
}

$customers = [
    ['id' => 1, 'first_name' => 'John', 'last_name' => 'Do'],
    ['id' => 3, 'first_name' => 'Alice', 'last_name' => 'Gustav'],
```

```
    ['id' => 2, 'first_name' => 'Bob', 'last_name' => 'Filipe']
];

// sort customers by first name
usort($customers, new Sort('first_name'));
print_r($customers);

// sort customers by last name
usort($customers, new Sort('last_name'));
print_r($customers);
?>
```

The above example will output:

```
Array
(
    [0] => Array
        (
            [id] => 3
            [first_name] => Alice
            [last_name] => Gustav
        )

    [1] => Array
        (
            [id] => 2
            [first_name] => Bob
            [last_name] => Filipe
        )

    [2] => Array
        (
            [id] => 1
            [first_name] => John
            [last_name] => Do
        )

)
Array
(
    [0] => Array
        (
            [id] => 1
            [first_name] => John
            [last_name] => Do
        )

    [1] => Array
        (
            [id] => 2
            [first_name] => Bob
            [last_name] => Filipe
        )

    [2] => Array
```

```
        (
            [id] => 3
            [first_name] => Alice
            [last_name] => Gustav
        )

)
```

__set_state()

static function **__set_state**(array `$properties`): object

This static method is called for classes exported by var_export().

The only parameter of this method is an array containing exported properties in the form `['property' => value, ...]`.

Example #6 Using __set_state()

```php
<?php

class A
{
    public $var1;
    public $var2;

    public static function __set_state($an_array)
    {
        $obj = new A;
        $obj->var1 = $an_array['var1'];
        $obj->var2 = $an_array['var2'];
        return $obj;
    }
}

$a = new A;
$a->var1 = 5;
$a->var2 = 'foo';

$b = var_export($a, true);
var_dump($b);
eval('$c = ' . $b . ';');
var_dump($c);
?>
```

The above example will output:

```
string(60) "A::__set_state(array(
   'var1' => 5,
   'var2' => 'foo',
))"
object(A)#2 (2) {
```

```
  ["var1"]=>
  int(5)
  ["var2"]=>
  string(3) "foo"
}
```

> **Note** :
>
> When exporting an object, var_export() does not check whether __set_state() is implemented by the object's class, so re-importing objects will result in an Error exception, if __set_state() is not implemented. Particularly, this affects some internal classes.
>
> It is the responsibility of the programmer to verify that only objects whose class implements **__set_state()** will be re-imported.

__debugInfo()

function **__debugInfo**(): array

This method is called by var_dump() when dumping an object to get the properties that should be shown. If the method isn't defined on an object, then all public, protected and private properties will be shown.

As of PHP 8.5.0, returning `null` from __debugInfo() is deprecated. Return an empty array instead.

Example #7 Using __debugInfo()

```php
<?php
class C {
    private $prop;

    public function __construct($val) {
        $this->prop = $val;
    }

    public function __debugInfo() {
        return [
            'propSquared' => $this->prop ** 2,
        ];
    }
}

var_dump(new C(42));
?>
```

The above example will output:

```
object(C)#1 (1) {
  ["propSquared"]=>
  int(1764)
}
```

Final Keyword

The final keyword prevents child classes from overriding a method, property, or constant by prefixing the definition with `final`. If the class itself is being defined final then it cannot be extended.

Example #1 Final methods example

```php
<?php
class BaseClass {
   public function test() {
       echo "BaseClass::test() called\n";
   }

   final public function moreTesting() {
       echo "BaseClass::moreTesting() called\n";
   }
}

class ChildClass extends BaseClass {
   public function moreTesting() {
       echo "ChildClass::moreTesting() called\n";
   }
}
// Results in Fatal error: Cannot override final method BaseClass::moreTesting()
?>
```

Example #2 Final class example

```php
<?php
final class BaseClass {
   public function test() {
       echo "BaseClass::test() called\n";
   }

   // As the class is already final, the final keyword is redundant
   final public function moreTesting() {
       echo "BaseClass::moreTesting() called\n";
   }
}

class ChildClass extends BaseClass {
}
// Results in Fatal error: Class ChildClass may not inherit from final class (BaseClass)
?>
```

Example #3 Final property example as of PHP 8.4.0

```php
<?php
class BaseClass {
   final protected string $test;
}

class ChildClass extends BaseClass {
```

```
    public string $test;
}
// Results in Fatal error: Cannot override final property BaseClass::$test
?>
```

Example #4 Final constants example as of PHP 8.1.0

```
<?php
class Foo
{
    final public const X = "foo";
}

class Bar extends Foo
{
    public const X = "bar";
}

// Fatal error: Bar::X cannot override final constant Foo::X
?>
```

> **Note** :
>
> As of PHP 8.0.0, private methods may not be declared final except for the constructor.

> **Note** :
>
> A property that is declared `private(set)` is implicitly `final`.

Object Cloning

Creating a copy of an object with fully replicated properties is not always the wanted behavior. A good example of the need for copy constructors is if you have an object which represents a GTK window and the object holds the resource of this GTK window, when you create a duplicate you might want to create a new window with the same properties and have the new object hold the resource of the new window. Another example is if your object holds a reference to another object which it uses and when you replicate the parent object you want to create a new instance of this other object so that the replica has its own separate copy.

An object copy is created by using the `clone` keyword (which calls the object's __clone() method if possible).

```
$copy_of_object = clone $object;
```

When an object is cloned, PHP will perform a shallow copy of all of the object's properties. Any properties that are references to other variables will remain references.

function **__clone**(): void

Once the cloning is complete, if a __clone() method is defined, then the newly created object's __clone() method will be called, to allow any necessary properties to be changed.

Example #1 Cloning an object

```php
<?php
class SubObject
{
    static $instances = 0;
    public $instance;

    public function __construct() {
        $this->instance = ++self::$instances;
    }

    public function __clone() {
        $this->instance = ++self::$instances;
    }
}

class MyCloneable
{
    public $object1;
    public $object2;

    function __clone()
    {
        // Force a copy of this->object, otherwise
        // it will point to same object.
        $this->object1 = clone $this->object1;
    }
}

$obj = new MyCloneable();

$obj->object1 = new SubObject();
$obj->object2 = new SubObject();

$obj2 = clone $obj;


print "Original Object:\n";
print_r($obj);

print "Cloned Object:\n";
print_r($obj2);

?>
```

The above example will output:

```
Original Object:
MyCloneable Object
(
    [object1] => SubObject Object
        (
            [instance] => 1
        )
```

```
    [object2] => SubObject Object
        (
            [instance] => 2
        )

)
Cloned Object:
MyCloneable Object
(
    [object1] => SubObject Object
        (
            [instance] => 3
        )

    [object2] => SubObject Object
        (
            [instance] => 2
        )

)
```

It is possible to access a member of a freshly cloned object in a single expression:

Example #2 Access member of freshly cloned object

```php
<?php
$dateTime = new DateTime();
echo (clone $dateTime)->format('Y');
?>
```

The above example will output something similar to:

```
2016
```

Cloning with property overrides

As of PHP 8.5.0, `clone` is also available as a function, whose second parameter `withProperties` takes an array mapping property names to the values the copy receives. Only the listed properties are overridden, every other property keeps the value it was copied with.

The overrides are applied after __clone() has run, and they honour the visibility of all properties. Readonly properties can be updated on the copy even if they were already set on the original object, which makes the function suited to methods returning a modified copy.

Example #3 Cloning an object with property overrides

```php
<?php
class Point {
```

```
    public function __construct(
        public readonly int $x,
        public readonly int $y,
    ) {}

    public function withX(int $x): static
    {
        return clone($this, ['x' => $x]);
    }
}

$p = new Point(1, 2);
$q = $p->withX(10);

echo $q->x, ' ', $q->y;
?>
```

The above example will output:

```
10 2
```

Comparing Objects

When using the comparison operator (==), object variables are compared in a simple manner, namely: Two object instances are equal if they have the same attributes and values (values are compared with ==), and are instances of the same class.

When using the identity operator (===), object variables are identical if and only if they refer to the same instance of the same class.

An example will clarify these rules.

Example #1 Example of object comparison

```
<?php
function bool2str($bool)
{
    if ($bool === false) {
        return 'FALSE';
    } else {
        return 'TRUE';
    }
}

function compareObjects(&$o1, &$o2)
{
    echo 'o1 == o2 : ' . bool2str($o1 == $o2) . "\n";
    echo 'o1 != o2 : ' . bool2str($o1 != $o2) . "\n";
    echo 'o1 === o2 : ' . bool2str($o1 === $o2) . "\n";
    echo 'o1 !== o2 : ' . bool2str($o1 !== $o2) . "\n";
}
```

```php
class Flag
{
    public $flag;

    function __construct($flag = true) {
        $this->flag = $flag;
    }
}

class OtherFlag
{
    public $flag;

    function __construct($flag = true) {
        $this->flag = $flag;
    }
}

$o = new Flag();
$p = new Flag();
$q = $o;
$r = new OtherFlag();

echo "Two instances of the same class\n";
compareObjects($o, $p);

echo "\nTwo references to the same instance\n";
compareObjects($o, $q);

echo "\nInstances of two different classes\n";
compareObjects($o, $r);
?>
```

The above example will output:

```
Two instances of the same class
o1 == o2 : TRUE
o1 != o2 : FALSE
o1 === o2 : FALSE
o1 !== o2 : TRUE

Two references to the same instance
o1 == o2 : TRUE
o1 != o2 : FALSE
o1 === o2 : TRUE
o1 !== o2 : FALSE

Instances of two different classes
o1 == o2 : FALSE
o1 != o2 : TRUE
o1 === o2 : FALSE
o1 !== o2 : TRUE
```

> **Note** :
>
> Extensions can define their own rules for comparing their objects (==).

Late Static Bindings

PHP implements a feature called late static bindings which can be used to reference the called class in the context of static inheritance.

More precisely, late static bindings work by storing the class named in the last "non-forwarding call". In case of static method calls, this is the class explicitly named (usually the one on the left of the `::` operator); in case of non static method calls, it is the class of the object. A "forwarding call" is a static one that is introduced by `self::`, `parent::`, `static::`, or, if going up in the class hierarchy, forward_static_call(). The function get_called_class() can be used to retrieve a string with the name of the called class and `static::` introduces its scope.

This feature was named "late static bindings" with an internal perspective in mind. "Late binding" comes from the fact that `static::` will not be resolved using the class where the method is defined but it will rather be computed using runtime information. It was also called a "static binding" as it can be used for (but is not limited to) static method calls.

Limitations of `self::`

Static references to the current class like `self::` or `__CLASS__` are resolved using the class in which the function belongs, as in where it was defined:

Example #1 `self::` usage

```php
<?php

class A
{
    public static function who()
    {
        echo __CLASS__;
    }

    public static function test()
    {
        self::who();
    }
}

class B extends A
{
    public static function who()
    {
        echo __CLASS__;
    }
}
```

```
B::test();

?>
```

The above example will output:

```
A
```

Late Static Bindings' usage

Late static bindings tries to solve that limitation by introducing a keyword that references the class that was initially called at runtime. Basically, a keyword that would allow referencing `B` from `test()` in the previous example. It was decided not to introduce a new keyword but rather use `static` that was already reserved.

Example #2 `static::` simple usage

```
<?php

class A
{
    public static function who()
    {
        echo __CLASS__;
    }

    public static function test()
    {
        static::who(); // Here comes Late Static Bindings
    }
}

class B extends A
{
    public static function who()
    {
        echo __CLASS__;
    }
}

B::test();

?>
```

The above example will output:

```
B
```

> **Note** :
>
> In non-static contexts, the called class will be the class of the object instance. Since `$this->` will try to call private methods from the same scope, using `static::` may give different results. Another difference is that `static::` can only refer to static properties.

Example #3 `static::` usage in a non-static context

```php
<?php

class A
{
    private function foo()
    {
        echo "Success!\n";
    }

    public function test()
    {
        $this->foo();
        static::foo();
    }
}

class B extends A
{
    /* foo() will be copied to B, hence its scope will still be A and
    * the call be successful */
}

class C extends A
{
    private function foo()
    {
        /* Original method is replaced; the scope of the new one is C */
    }
}

$b = new B();
$b->test();

$c = new C();
try {
    $c->test();
} catch (Error $e) {
    echo $e->getMessage();
}

?>
```

The above example will output:

```
Success!
Success!
Success!
Call to private method C::foo() from scope A
```

> **Note** :
>
> Late static bindings' resolution will stop at a fully resolved static call with no fallback. On the other hand, static calls using keywords like `parent::` or `self::` will forward the calling information.

Example #4 Forwarding and non-forwarding calls

```php
<?php

class A
{
    public static function foo()
    {
        static::who();
    }

    public static function who()
    {
        echo __CLASS__ . "\n";
    }
}

class B extends A
{
    public static function test()
    {
        A::foo();
        parent::foo();
        self::foo();
    }

    public static function who()
    {
        echo __CLASS__ . "\n";
    }
}

class C extends B
{
    public static function who()
    {
        echo __CLASS__ . "\n";
    }
}

C::test();

?>
```

The above example will output:

```
A
C
C
```

Objects and references

One of the key-points of PHP OOP that is often mentioned is that "objects are passed by reference by default". This is not completely true. This section rectifies that general thought using some examples.

A PHP reference is an alias, which allows two different variables to write to the same value. In PHP, an object variable doesn't contain the object itself as value. It only contains an object identifier which allows object accessors to find the actual object. When an object is sent by argument, returned or assigned to another variable, the different variables are not aliases: they hold a copy of the identifier, which points to the same object.

Example #1 References and Objects

```php
<?php
class A {
    public $foo = 1;
}

$a = new A;
$b = $a;     // $a and $b are copies of the same identifier
             // ($a) = ($b) = <id>
$b->foo = 2;
echo $a->foo."\n";


$c = new A;
$d = &$c;    // $c and $d are references
             // ($c,$d) = <id>

$d->foo = 2;
echo $c->foo."\n";


$e = new A;

function foo($obj) {
    // ($obj) = ($e) = <id>
    $obj->foo = 2;
}

foo($e);
echo $e->foo."\n";

?>
```

The above example will output:

```
2
2
2
```

Object Serialization

serialize() returns a string containing a byte-stream representation of any value that can be stored in PHP. unserialize() can use this string to recreate the original variable values. Using serialize to save an object will save all variables in an object. The methods in an object will not be saved, only the name of the class.

In order to be able to unserialize() an object, the class of that object needs to be defined. That is, if you have an object of class A and serialize this, you'll get a string that refers to class A and contains all values of variables contained in it. If you want to be able to unserialize an object of class A in another file, the definition of class A must be present in that file first. This can be done for example by storing the class definition of class A in an include file and including this file or making use of the spl_autoload_register() function.

```php
<?php
// A.php:

  class A {
      public $one = 1;

      public function show_one() {
          echo $this->one;
      }
  }

// page1.php:

  include "A.php";

  $a = new A;
  $s = serialize($a);
  // store $s somewhere where page2.php can find it.
  file_put_contents('store', $s);

// page2.php:

  // this is needed for the unserialize to work properly.
  include "A.php";

  $s = file_get_contents('store');
  $a = unserialize($s);

  // now use the function show_one() of the $a object.
  $a->show_one();
?>
```

It is strongly recommended that if an application serializes objects, for use later in the application, the application includes the class definition for that object throughout the application. Not doing so might result in an object being unserialized without a class definition, which will result in PHP giving the object a class of **__PHP_Incomplete_Class_Name**, which has no methods and would render the object useless.

So if in the example above *$a* became part of a session by adding a new key to the *$_SESSION* superglobal array, you should include the file `A.php` on all of your pages, not only *page1.php* and *page2.php*.

Beyond the above advice, note that you can also hook into the serialization and unserialization events on an object using the __sleep() and __wakeup() methods. Using __sleep() also allows you to only serialize a subset of the object's properties.

Covariance and Contravariance

In PHP 7.2.0, partial contravariance was introduced by removing type restrictions on parameters in a child method. As of PHP 7.4.0, full covariance and contravariance support was added.

Covariance allows a child's method to return a more specific type than the return type of its parent's method. Contravariance allows a parameter type to be less specific in a child method, than that of its parent.

A type declaration is considered more specific in the following case:

- A type is removed from a union type
- A type is added to an intersection type
- A class type is changed to a child class type
- iterable is changed to array or Traversable

A type declaration is considered less specific if the opposite is true.

Covariance

To illustrate how covariance works, a simple abstract parent class, *Animal* is created. *Animal* will be extended by child classes, *Cat*, and *Dog*.

```php
<?php

abstract class Animal
{
    protected string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    abstract public function speak();
}

class Dog extends Animal
{
    public function speak()
    {
```

```
        echo $this->name . " barks";
    }
}

class Cat extends Animal
{
    public function speak()
    {
        echo $this->name . " meows";
    }
}
```

Note that there aren't any methods which return values in this example. A few factories will be added which return a new object of class type *Animal*, *Cat*, or *Dog*.

```
<?php

interface AnimalShelter
{
    public function adopt(string $name): Animal;
}

class CatShelter implements AnimalShelter
{
    public function adopt(string $name): Cat // instead of returning class type Animal, it can
return class type Cat
    {
        return new Cat($name);
    }
}

class DogShelter implements AnimalShelter
{
    public function adopt(string $name): Dog // instead of returning class type Animal, it can
return class type Dog
    {
        return new Dog($name);
    }
}

$kitty = (new CatShelter)->adopt("Ricky");
$kitty->speak();
echo "\n";

$doggy = (new DogShelter)->adopt("Mavrick");
$doggy->speak();
```

The above example will output:

```
Ricky meows
Mavrick barks
```

> **Note** :
>
> As of PHP 8.5.0, when a parent method declares static as its return type, an overriding method in a final class may declare self or the class's own type as its return type instead, since these are equivalent to static in a class that cannot be extended further. See the migration guide for details.

Contravariance

Continuing with the previous example with the classes *Animal*, *Cat*, and *Dog*, a class called *Food* and *AnimalFood* will be included, and a method *eat(AnimalFood $food)* is added to the *Animal* abstract class.

```php
<?php

class Food {}

class AnimalFood extends Food {}

abstract class Animal
{
    protected string $name;

    public function __construct(string $name)
    {
        $this->name = $name;
    }

    public function eat(AnimalFood $food)
    {
        echo $this->name . " eats " . get_class($food);
    }
}
```

In order to see the behavior of contravariance, the *eat* method is overridden in the *Dog* class to allow any *Food* type object. The *Cat* class remains unchanged.

```php
<?php

class Dog extends Animal
{
    public function eat(Food $food) {
        echo $this->name . " eats " . get_class($food);
    }
}
```

The next example will show the behavior of contravariance.

```php
<?php

$kitty = (new CatShelter)->adopt("Ricky");
$catFood = new AnimalFood();
$kitty->eat($catFood);
echo "\n";
```

```php
$doggy = (new DogShelter)->adopt("Mavrick");
$banana = new Food();
$doggy->eat($banana);
```

The above example will output:

```
Ricky eats AnimalFood
Mavrick eats Food
```

But what happens if *$kitty* tries to **eat()** the *$banana*?

```php
$kitty->eat($banana);
```

The above example will output:

```
Fatal error: Uncaught TypeError: Argument 1 passed to Animal::eat() must be an instance of
AnimalFood, instance of Food given
```

Property variance

By default, properties are neither covariant nor contravariant, hence invariant. That is, their type may not change in a child class at all. The reason for that is "get" operations must be covariant, and "set" operations must be contravariant. The only way for a property to satisfy both requirements is to be invariant.

As of PHP 8.4.0, with the addition of abstract properties (on an interface or abstract class) and virtual properties, it is possible to declare a property that has only a get or set operation. As a result, abstract properties or virtual properties that have only a "get" operation required may be covariant. Similarly, an abstract property or virtual property that has only a "set" operation required may be contravariant.

Once a property has both a get and set operation, however, it is no longer covariant or contravariant for further extension. That is, it is now invariant.

Example #1 Property type variance

```php
<?php
class Animal {}
class Dog extends Animal {}
class Poodle extends Dog {}

interface PetOwner
{
    // Only a get operation is required, so this may be covariant.
    public Animal $pet { get; }
}
```

```php
class DogOwner implements PetOwner
{
    // This may be a more restrictive type since the "get" side
    // still returns an Animal.  However, as a native property
    // children of this class may not change the type anymore.
    public Dog $pet;
}

class PoodleOwner extends DogOwner
{
    // This is NOT ALLOWED, because DogOwner::$pet has both
    // get and set operations defined and required.
    public Poodle $pet;
}
?>
```

Lazy Objects

A lazy object is an object whose initialization is deferred until its state is observed or modified. Some use-case examples include dependency injection components that provide lazy services fully initialized only if needed, ORMs providing lazy entities that hydrate themselves from the database only when accessed, or a JSON parser that delays parsing until elements are accessed.

Two lazy object strategies are supported: Ghost Objects and Virtual Proxies, hereafter referred to as "lazy ghosts" and "lazy proxies". In both strategies, the lazy object is attached to an initializer or factory that is called automatically when its state is observed or modified for the first time. From an abstraction point of view, lazy ghost objects are indistinguishable from non-lazy ones: they can be used without knowing they are lazy, allowing them to be passed to and used by code that is unaware of laziness. Lazy proxies are similarly transparent, but care must be taken when their identity is used, as the proxy and its real instance have different identities.

> **Note** : **Version Information**
>
> Lazy objects were introduced in PHP 8.4.

Creating Lazy Objects

It is possible to create a lazy instance of any user defined class or the stdClass class (other internal classes are not supported), or to reset an instance of these classes to make it lazy. The entry points for creating a lazy object are the ReflectionClass::newLazyGhost() and ReflectionClass::newLazyProxy() methods.

Both methods accept a function that is called when the object requires initialization. The function's expected behavior varies depending on the strategy in use, as described in the reference documentation for each method.

Example #1 Creating a Lazy Ghost

```php
<?php
class Example
{
    public function __construct(public int $prop)
    {
```

```
        echo __METHOD__, "\n";
    }
}

$reflector = new ReflectionClass(Example::class);
$lazyObject = $reflector->newLazyGhost(function (Example $object) {
    // Initialize object in-place
    $object->__construct(1);
});

var_dump($lazyObject);
var_dump(get_class($lazyObject));

// Triggers initialization
var_dump($lazyObject->prop);
?>
```

The above example will output:

```
lazy ghost object(Example)#3 (0) {
  ["prop"]=>
  uninitialized(int)
}
string(7) "Example"
Example::__construct
int(1)
```

Example #2 Creating a Lazy Proxy

```
<?php
class Example
{
    public function __construct(public int $prop)
    {
        echo __METHOD__, "\n";
    }
}

$reflector = new ReflectionClass(Example::class);
$lazyObject = $reflector->newLazyProxy(function (Example $object) {
    // Create and return the real instance
    return new Example(1);
});

var_dump($lazyObject);
var_dump(get_class($lazyObject));

// Triggers initialization
var_dump($lazyObject->prop);
?>
```

The above example will output:

```
lazy proxy object(Example)#3 (0) {
  ["prop"]=>
  uninitialized(int)
}
string(7) "Example"
Example::__construct
int(1)
```

Any access to properties of a lazy object triggers its initialization (including via ReflectionProperty). However, certain properties might be known in advance and should not trigger initialization when accessed:

Example #3 Initializing Properties Eagerly

```php
<?php
class BlogPost
{
    public function __construct(
        public int $id,
        public string $title,
        public string $content,
    ) { }
}

$reflector = new ReflectionClass(BlogPost::class);

$post = $reflector->newLazyGhost(function ($post) {
    $data = fetch_from_store($post->id);
    $post->__construct($data['id'], $data['title'], $data['content']);
});

// Without this line, the following call to ReflectionProperty::setValue() would
// trigger initialization.
$reflector->getProperty('id')->skipLazyInitialization($post);
$reflector->getProperty('id')->setValue($post, 123);

// Alternatively, one can use this directly:
$reflector->getProperty('id')->setRawValueWithoutLazyInitialization($post, 123);

// The id property can be accessed without triggering initialization
var_dump($post->id);
?>
```

The ReflectionProperty::skipLazyInitialization() and ReflectionProperty::setRawValueWithoutLazyInitialization() methods offer ways to bypass lazy-initialization when accessing a property.

About Lazy Object Strategies

Lazy ghosts are objects that initialize in-place and, once initialized, are indistinguishable from an object that was never lazy. This strategy is suitable when we control both the instantiation and initialization of the object, making it unsuitable if either of these is managed by another party.

Lazy proxies, once initialized, act as proxies to a real instance: any operation on an initialized lazy proxy is forwarded to the real instance. The creation of the real instance can be delegated to another party, making this strategy useful in cases where lazy ghosts are unsuitable. Although lazy proxies are nearly as transparent as lazy ghosts, caution is needed when their identity is used, as the proxy and its real instance have distinct identities.

Lifecycle of Lazy Objects

Objects can be made lazy at instantiation time using ReflectionClass::newLazyGhost() or ReflectionClass::newLazyProxy(), or after instantiation by using ReflectionClass::resetAsLazyGhost() or ReflectionClass::resetAsLazyProxy(). Following this, a lazy object can become initialized through one of the following operations:

- Interacting with the object in a way that triggers automatic initialization. See Initialization triggers.
- Marking all its properties as non-lazy using ReflectionProperty::skipLazyInitialization() or ReflectionProperty::setRawValueWithoutLazyInitialization().
- Calling explicitly ReflectionClass::initializeLazyObject() or ReflectionClass::markLazyObjectAsInitialized().

As lazy objects become initialized when all their properties are marked non-lazy, the above methods will not mark an object as lazy if no properties could be marked as lazy.

Initialization Triggers

Lazy objects are designed to be fully transparent to their consumers, so normal operations that observe or modify the object's state will automatically trigger initialization before the operation is performed. This includes, but is not limited to, the following operations:

- Reading or writing a property.
- Testing if a property is set or unsetting it.
- Accessing or modifying a property via ReflectionProperty::getValue(), ReflectionProperty::getRawValue(), ReflectionProperty::setValue(), or ReflectionProperty::setRawValue().
- Listing properties with **ReflectionObject::getProperties()**, **ReflectionObject::getProperty()**, get_object_vars().
- Iterating over properties of an object that does not implement Iterator or IteratorAggregate using foreach.
- Serializing the object with serialize(), json_encode(), etc.
- Cloning the object.

Method calls that do not access the object state will not trigger initialization. Similarly, interactions with the object that invoke magic methods or hook functions will not trigger initialization if these methods or functions do not access the object's state.

Non-Triggering Operations

The following specific methods or low-level operations allow access or modification of lazy objects without triggering initialization:

- Marking properties as non-lazy with ReflectionProperty::skipLazyInitialization() or ReflectionProperty::setRawValueWithoutLazyInitialization().
- Retrieving the internal representation of properties using get_mangled_object_vars() or by casting the object to an array.

- Using serialize() when `ReflectionClass::SKIP_INITIALIZATION_ON_SERIALIZE` is set, unless __serialize() or __sleep() trigger initialization.
- Calling **ReflectionObject::__toString()**.
- Using var_dump() or debug_zval_dump(), unless __debugInfo() triggers initialization.

Initialization Sequence

This section outlines the sequence of operations performed when initialization is triggered, based on the strategy in use.

Ghost Objects

- The object is marked as non-lazy.
- Properties not initialized with ReflectionProperty::skipLazyInitialization() or ReflectionProperty::setRawValueWithoutLazyInitialization() are set to their default values, if any. At this stage, the object resembles one created with ReflectionClass::newInstanceWithoutConstructor(), except for already initialized properties.
- The initializer function is then called with the object as its first parameter. The function is expected, but not required, to initialize the object state, and must return `null` or no value. The object is no longer lazy at this point, so the function can access its properties directly.

After initialization, the object is indistinguishable from an object that was never lazy.

Proxy Objects

- The object is marked as non-lazy.
- Unlike ghost objects, the properties of the object are not modified at this stage.
- The factory function is called with the object as its first parameter and must return a non-lazy instance of a compatible class (see ReflectionClass::newLazyProxy()).
- The returned instance is referred to as the *real instance* and is attached to the proxy.
- The proxy's property values are discarded as though unset() was called.

After initialization, accessing any property on the proxy will yield the same result as accessing the corresponding property on the real instance; all property accesses on the proxy are forwarded to the real instance, including declared, dynamic, non-existing, or properties marked with ReflectionProperty::skipLazyInitialization() or ReflectionProperty::setRawValueWithoutLazyInitialization().

The proxy object itself is *not* replaced or substituted for the real instance.

While the factory receives the proxy as its first parameter, it is not expected to modify it (modifications are allowed but will be lost during the final initialization step). However, the proxy can be used for decisions based on the values of initialized properties, the class, the object itself, or its identity. For instance, the initializer might use an initialized property's value when creating the real instance.

Common Behavior

The scope and *$this* context of the initializer or factory function remains unchanged, and usual visibility constraints apply.

After successful initialization, the initializer or factory function is no longer referenced by the object and may be released if it has no other references.

If the initializer throws an exception, the object state is reverted to its pre-initialization state and the object is marked as lazy again. In other words, all effects on the object itself are reverted. Other side effects, such as effects on other objects, are not reverted. This prevents exposing a partially initialized instance in case of failure.

Cloning

Cloning a lazy object triggers its initialization before the clone is created, resulting in an initialized object.

For proxy objects, both the proxy and its real instance are cloned, and the clone of the proxy is returned. The `__clone` method is called on the real instance, not on the proxy. The cloned proxy and real instance are linked as they are during initialization, so accesses to the proxy clone are forwarded to the real instance clone.

This behavior ensures that the clone and the original object maintain separate states. Changes to the original object or its initializer's state after cloning do not affect the clone. Cloning both the proxy and its real instance, rather than returning a clone of the real instance alone, ensures that the clone operation consistently returns an object of the same class.

Destructors

For lazy ghosts, the destructor is only called if the object has been initialized. For proxies, the destructor is only called on the real instance, if one exists.

The ReflectionClass::resetAsLazyGhost() and ReflectionClass::resetAsLazyProxy() methods may invoke the destructor of the object being reset.

OOP Changelog

Changes to the PHP OOP model are logged here. Descriptions and other notes regarding these features are documented within the OOP model documentation.

Version	Description
8.5.0	Changed: A method that returns static may now be overridden in a final class to declare self or the class's own type as its return type, as these are equivalent to static in a class that cannot be extended further.
8.5.0	Changed: Traits are now bound to a class before its parent class.
8.4.0	Added: Support for Property Hooks.
8.4.0	Added: Support for Lazy Objects.
8.1.0	Added: Support for the final modifier for class constants. Also, interface constants become overridable by default.
8.0.0	Added: Support for the nullsafe operator ?-> to access properties and methods on objects that may be null.
7.4.0	Changed: It is now possible to throw an exception within **__toString()**.
7.4.0	Added: Support for limited return type covariance and argument type contravariance. Full variance support is only available if autoloading is used. Inside a single file only non-cyclic type references are possible.
7.4.0	Added: It is now possible to type class properties.

Version	Description
7.3.0	Incompatibility: Argument unpacking of Traversables with non-int keys is no longer supported. This behaviour was not intended and thus has been removed.
7.3.0	Incompatibility: In previous versions it was possible to separate the static properties by assigning a reference. This has been removed.
7.3.0	Changed: The instanceof operator now allows literals as the first operand, in which case the result is always `false`.
7.2.0	Deprecated: The __autoload() method has been deprecated in favour of spl_autoload_register().
7.2.0	Changed: The following name cannot be used to name classes, interfaces, or traits: `object`.
7.2.0	Changed: A trailing comma can now be added to the group-use syntax for namespaces.
7.2.0	Changed: Parameter type widening. Parameter types from overridden methods and from interface implementations may now be omitted.
7.2.0	Changed: Abstract methods can now be overridden when an abstract class extends another abstract class.
7.1.0	Changed: The following names cannot be used to name classes, interfaces, or traits: `void` and `iterable`.
7.1.0	Added: It is now possible to specify the visibility of class constants.
7.0.0	Deprecated: Static calls to methods that are not declared static.
7.0.0	Deprecated: PHP 4 style constructor. I.e. methods that have the same name as the class they are defined in.
7.0.0	Added: Group *use* declaration: classes, functions and constants being imported from the same namespace can now be grouped together in a single use statement.
7.0.0	Added: Support for anonymous classes has been added via `new class`.
7.0.0	Incompatibility: Iterating over a non-Traversable object will now have the same behaviour as iterating over by-reference arrays.
7.0.0	Changed: Defining (compatible) properties in two used traits no longer triggers an error.
5.6.0	Added: The __debugInfo() method.
5.5.0	Added: The ::class magic constant.
5.5.0	Added: finally to handle exceptions.
5.4.0	Added: traits.
5.4.0	Changed: If an abstract class defines a signature for the constructor it will now be enforced.
5.3.3	Changed: Methods with the same name as the last element of a namespaced class name will no longer be treated as constructor. This change doesn't affect non-namespaced classes.
5.3.0	Changed: Classes that implement interfaces with methods that have default values in the prototype are no longer required to match the interface's default value.
5.3.0	Changed: It's now possible to reference the class using a variable (e.g., `echo $classname::constant;`). The variable's value can not be a keyword (e.g., `self`, `parent` or `static`).
5.3.0	Changed: An `E_WARNING` level error is issued if the magic overloading methods are declared static. It also enforces the public visibility requirement.

Version	Description
5.3.0	Changed: Prior to 5.3.0, exceptions thrown in the __autoload() function could not be caught in the catch block, and would result in a fatal error. Exceptions now thrown in the __autoload function can be caught in the catch block, with one provision. If throwing a custom exception, then the custom exception class must be available. The __autoload function may be used recursively to autoload the custom exception class.
5.3.0	Added: The __callStatic method.
5.3.0	Added: heredoc and nowdoc support for class *const* and property definitions. Note: heredoc values must follow the same rules as double-quoted strings, (e.g. no variables within).
5.3.0	Added: Late Static Bindings.
5.3.0	Added: The __invoke() method.
5.2.0	Changed: The __toString() method was only called when it was directly combined with echo or print. But now, it is called in any string context (e.g. in printf() with `%s` modifier) but not in other type contexts (e.g. with `%d` modifier). As of PHP 5.2.0, converting objects without a __toString method to string emits a **`E_RECOVERABLE_ERROR`** level error.
5.1.3	Changed: In previous versions of PHP 5, the use of `var` was considered deprecated and would issue an **`E_STRICT`** level error. It's no longer deprecated, therefore does not emit the error.
5.1.0	Changed: The __set_state() static method is now called for classes exported by var_export().
5.1.0	Added: The __isset() and __unset() methods.

Namespaces

Table of Contents

Overview

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

What are namespaces? In the broadest definition namespaces are a way of encapsulating items. This can be seen as an abstract concept in many places. For example, in any operating system directories serve to group related files, and act as a namespace for the files within them. As a concrete example, the file `foo.txt` can exist in both directory `/home/greg` and in `/home/other`, but two copies of `foo.txt` cannot co-exist in the same directory. In addition, to access the `foo.txt` file outside of the `/home/greg` directory, we must prepend the directory name to the file name using the directory separator to get `/home/greg/foo.txt`. This same principle extends to namespaces in the programming world.

In the PHP world, namespaces are designed to solve two problems that authors of libraries and applications encounter when creating re-usable code elements such as classes or functions:

1. Name collisions between code you create, and internal PHP classes/functions/constants or third-party classes/functions/constants.
2. Ability to alias (or shorten) Extra_Long_Names designed to alleviate the first problem, improving readability of source code.

PHP Namespaces provide a way in which to group related classes, interfaces, functions and constants. Here is an example of namespace syntax in PHP:

Example #1 Namespace syntax example

```php
<?php
namespace my\name; // see "Defining Namespaces" section

class MyClass {}
function myfunction() {}
const MYCONST = 1;

$a = new MyClass;
print $a::class . "\n";
$c = new \my\name\MyClass; // see "Global Space" section
print $c::class . "\n";

$a = strlen('hi'); // see "Using namespaces: fallback to global
                   // function/constant" section
print $a . "\n";

$d = namespace\MYCONST; // see "namespace operator and __NAMESPACE__
                        // constant" section
print $d . "\n";
$d = __NAMESPACE__ . '\MYCONST';
echo constant($d); // see "Namespaces and dynamic language features" section
```

> **Note** :
>
> Namespace names are case-insensitive.

> **Note** :
>
> The Namespace name `PHP`, and compound names starting with this name (like `PHP\Classes`) are reserved for internal language use and should not be used in the userspace code.

Namespaces

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Although any valid PHP code can be contained within a namespace, only the following types of code are affected by namespaces: classes (including abstract classes, traits and enums), interfaces, functions and constants.

Namespaces are declared using the `namespace` keyword. A file containing a namespace must declare the namespace at the top of the file before any other code - with one exception: the declare keyword.

Example #1 Declaring a single namespace

```php
<?php
namespace MyProject;

const CONNECT_OK = 1;
class Connection { /* ... */ }
function connect() { /* ... */ }
```

> **Note** :
>
> Fully qualified names (i.e. names starting with a backslash) are not allowed in namespace declarations, because such constructs are interpreted as relative namespace expressions.

The only code construct allowed before a namespace declaration is the `declare` statement, for defining encoding of a source file. In addition, no non-PHP code may precede a namespace declaration, including extra whitespace:

Example #2 Declaring a single namespace

```
<html>
<?php
namespace MyProject; // fatal error - namespace must be the first statement in the script
```

In addition, unlike any other PHP construct, the same namespace may be defined in multiple files, allowing splitting up of a namespace's contents across the filesystem.

Sub-namespaces

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Much like directories and files, PHP namespaces also contain the ability to specify a hierarchy of namespace names. Thus, a namespace name can be defined with sub-levels:

Example #1 Declaring a single namespace with hierarchy

```
<?php
namespace MyProject\Sub\Level;

const CONNECT_OK = 1;
class Connection { /* ... */ }
function connect() { /* ... */  }
```

The above example creates constant `MyProject\Sub\Level\CONNECT_OK`, class `MyProject\Sub\Level\Connection` and function `MyProject\Sub\Level\connect`.

Defining multiple namespaces in the same file

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Multiple namespaces may also be declared in the same file. There are two allowed syntaxes.

Example #1 Declaring multiple namespaces, simple combination syntax

```
<?php
namespace MyProject;

const CONNECT_OK = 1;
class Connection { /* ... */ }
function connect() { /* ... */  }

namespace AnotherProject;
```

```
const CONNECT_OK = 1;
class Connection { /* ... */ }
function connect() { /* ... */  }
```

This syntax is not recommended for combining namespaces into a single file. Instead it is recommended to use the alternate bracketed syntax.

Example #2 Declaring multiple namespaces, bracketed syntax

```
<?php
namespace MyProject {

const CONNECT_OK = 1;
class Connection { /* ... */ }
function connect() { /* ... */  }
}

namespace AnotherProject {

const CONNECT_OK = 1;
class Connection { /* ... */ }
function connect() { /* ... */  }
}
```

It is strongly discouraged as a coding practice to combine multiple namespaces into the same file. The primary use case is to combine multiple PHP scripts into the same file.

To combine global non-namespaced code with namespaced code, only bracketed syntax is supported. Global code should be encased in a namespace statement with no namespace name as in:

Example #3 Declaring multiple namespaces and unnamespaced code

```
<?php
namespace MyProject {

const CONNECT_OK = 1;
class Connection {
    public static function start() {
        print __METHOD__ . "\n";
    }
}

function connect() {
    print __FUNCTION__ . "\n";
}
}

namespace { // global code
print strlen("hi") . "\n";
MyProject\connect();
MyProject\Connection::start();
}
```

No PHP code may exist outside of the namespace brackets except for an opening declare statement.

Example #4 Declaring multiple namespaces and unnamespaced code

```
<?php
declare(strict_types=1);
namespace MyProject {

const CONNECT_OK = 1;
class Connection {
    public static function start() {
        print __METHOD__ . "\n";
    }
}

function connect() {
    print __FUNCTION__ . "\n";
}
}

namespace { // global code
print strlen("hi") . "\n";
MyProject\connect();
MyProject\Connection::start();
}
```

Basics

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Before discussing the use of namespaces, it is important to understand how PHP knows which namespaced element your code is requesting. A simple analogy can be made between PHP namespaces and a filesystem. There are three ways to access a file in a file system:

1. Relative file name like `foo.txt`. This resolves to `currentdirectory/foo.txt` where `currentdirectory` is the directory currently occupied. So if the current directory is `/home/foo`, the name resolves to `/home/foo/foo.txt`.

2. Relative path name like `subdirectory/foo.txt`. This resolves to `currentdirectory/subdirectory/foo.txt`.

3. Absolute path name like `/main/foo.txt`. This resolves to `/main/foo.txt`.

The same principle can be applied to namespaced elements in PHP. For example, a class name can be referred to in three ways:

1. Unqualified name, or an unprefixed class name like `$a = new foo();` or `foo::staticmethod();`. If the current namespace is `currentnamespace`, this resolves to `currentnamespace\foo`. If the code is global, non-namespaced code, this resolves to `foo`.

   One caveat: unqualified names for functions and constants will resolve to global functions and constants if the namespaced function or constant is not defined. See Using namespaces: fallback to global function/constant for details.

2. Qualified name, or a prefixed class name like `$a = new subnamespace\foo();` or `subnamespace\foo::staticmethod();`. If the current namespace is `currentnamespace`, this resolves to `currentnamespace\subnamespace\foo`. If the code is global, non-namespaced code, this resolves to `subnamespace\foo`.

3. Fully qualified name, or a prefixed name with global prefix operator like `$a = new \currentnamespace\foo();` or `\currentnamespace\foo::staticmethod();`. This always resolves to the literal name specified in the code, `currentnamespace\foo`.

Here is an example of the three kinds of syntax in actual code:

file1.php

```
<?php
namespace Foo\Bar\subnamespace;

const FOO = 1;
function foo() {}
class foo
{
    static function staticmethod() {}
}
```

file2.php

```
<?php
namespace Foo\Bar;
include 'file1.php';

const FOO = 2;
function foo() {}
class foo
{
    static function staticmethod() {}
}

/* Unqualified name */
foo(); // resolves to function Foo\Bar\foo
foo::staticmethod(); // resolves to class Foo\Bar\foo, method staticmethod
echo FOO; // resolves to constant Foo\Bar\FOO

/* Qualified name */
subnamespace\foo(); // resolves to function Foo\Bar\subnamespace\foo
subnamespace\foo::staticmethod(); // resolves to class Foo\Bar\subnamespace\foo,
                                  // method staticmethod
echo subnamespace\FOO; // resolves to constant Foo\Bar\subnamespace\FOO

/* Fully qualified name */
\Foo\Bar\foo(); // resolves to function Foo\Bar\foo
\Foo\Bar\foo::staticmethod(); // resolves to class Foo\Bar\foo, method staticmethod
echo \Foo\Bar\FOO; // resolves to constant Foo\Bar\FOO
```

Note that to access any global class, function or constant, a fully qualified name can be used, such as \strlen() or \Exception or \INI_ALL.

Example #1 Accessing global classes, functions and constants from within a namespace

```
<?php
namespace Foo;

function strlen() {}
```

```
const INI_ALL = 3;
class Exception {}

print \strlen('hi') . "\n"; // calls global function strlen
print \INI_ALL . "\n"; // accesses global constant INI_ALL
$c = new \Exception('error'); // instantiates global class Exception
print $c::class . "\n";
```

Namespaces and dynamic language features

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

PHP's implementation of namespaces is influenced by its dynamic nature as a programming language. Thus, to convert code like the following example into namespaced code:

Example #1 Dynamically accessing elements

example1.php:

```
<?php
class classname
{
    function __construct()
    {
        echo __METHOD__,"\n";
    }
}
function funcname()
{
    echo __FUNCTION__,"\n";
}
const constname = "global";

$a = 'classname';
$obj = new $a; // prints classname::__construct
$b = 'funcname';
$b(); // prints funcname
echo constant('constname'), "\n"; // prints global
```

One must use the fully qualified name (class name with namespace prefix). Note that because there is no difference between a qualified and a fully qualified Name inside a dynamic class name, function name, or constant name, the leading backslash is not necessary.

Example #2 Dynamically accessing namespaced elements

```
<?php
namespace namespacename;
class classname
{
    function __construct()
    {
        echo __METHOD__,"\n";
    }
}
function funcname()
{
```

```
    echo __FUNCTION__,"\n";
}
const constname = "namespaced";

/* note that if using double quotes, "\\namespacename\\classname" must be used */
$a = '\namespacename\classname';
$obj = new $a; // prints namespacename\classname::__construct
$a = 'namespacename\classname';
$obj = new $a; // also prints namespacename\classname::__construct
$b = 'namespacename\funcname';
$b(); // prints namespacename\funcname
$b = '\namespacename\funcname';
$b(); // also prints namespacename\funcname
echo constant('\namespacename\constname'), "\n"; // prints namespaced
echo constant('namespacename\constname'), "\n"; // also prints namespaced
```

Be sure to read the note about escaping namespace names in strings.

namespace keyword and __NAMESPACE__

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

PHP supports two ways of abstractly accessing elements within the current namespace, the `__NAMESPACE__` magic constant, and the `namespace` keyword.

The value of `__NAMESPACE__` is a string that contains the current namespace name. In global, un-namespaced code, it contains an empty string.

Example #1 __NAMESPACE__ example, namespaced code

```
<?php
namespace MyProject;

echo '"', __NAMESPACE__, '"'; // outputs "MyProject"
```

Example #2 __NAMESPACE__ example, global code

```
<?php
echo '"', __NAMESPACE__, '"'; // outputs ""
```

The `__NAMESPACE__` constant is useful for dynamically constructing names, for instance:

Example #3 using __NAMESPACE__ for dynamic name construction

```
<?php
namespace MyProject;

function get($classname)
{
    $a = __NAMESPACE__ . '\\' . $classname;
    return new $a;
}
```

The `namespace` keyword can be used to explicitly request an element from the current namespace or a sub-namespace. It is the namespace equivalent of the `self` operator for classes.

Example #4 the namespace operator, inside a namespace

```php
<?php
namespace MyProject;

use blah\blah as mine; // see "Using namespaces: Aliasing/Importing"

blah\mine(); // calls function MyProject\blah\mine()
namespace\blah\mine(); // calls function MyProject\blah\mine()

namespace\func(); // calls function MyProject\func()
namespace\sub\func(); // calls function MyProject\sub\func()
namespace\cname::method(); // calls static method "method" of class MyProject\cname
$a = new namespace\sub\cname(); // instantiates object of class MyProject\sub\cname
$b = namespace\CONSTANT; // assigns value of constant MyProject\CONSTANT to $b
```

Example #5 the namespace operator, in global code

```php
<?php

namespace\func(); // calls function func()
namespace\sub\func(); // calls function sub\func()
namespace\cname::method(); // calls static method "method" of class cname
$a = new namespace\sub\cname(); // instantiates object of class sub\cname
$b = namespace\CONSTANT; // assigns value of constant CONSTANT to $b
```

Aliasing and Importing

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

The ability to refer to an external fully qualified name with an alias, or importing, is an important feature of namespaces. This is similar to the ability of unix-based filesystems to create symbolic links to a file or to a directory.

PHP can alias(/import) constants, functions, classes, interfaces, traits, enums and namespaces.

Aliasing is accomplished with the `use` operator. Here is an example showing all 5 kinds of importing:

Example #1 importing/aliasing with the use operator

```php
<?php
namespace foo;
use My\Full\Classname as Another;

// this is the same as use My\Full\NSname as NSname
use My\Full\NSname;

// importing a global class
use ArrayObject;

// importing a function
use function My\Full\functionName;

// aliasing a function
use function My\Full\functionName as func;
```

```
// importing a constant
use const My\Full\CONSTANT;

$obj = new namespace\Another; // instantiates object of class foo\Another
$obj = new Another; // instantiates object of class My\Full\Classname
NSname\subns\func(); // calls function My\Full\NSname\subns\func
$a = new ArrayObject(array(1)); // instantiates object of class ArrayObject
// without the "use ArrayObject" we would instantiate an object of class foo\ArrayObject
func(); // calls function My\Full\functionName
echo CONSTANT; // echoes the value of My\Full\CONSTANT
```

Note that for namespaced names (fully qualified namespace names containing namespace separator, such as `Foo\Bar` as opposed to global names that do not, such as `FooBar`), the leading backslash is unnecessary and not recommended, as import names must be fully qualified, and are not processed relative to the current namespace.

PHP additionally supports a convenience shortcut to place multiple use statements on the same line

Example #2 importing/aliasing with the use operator, multiple use statements combined

```
<?php
use My\Full\Classname as Another, My\Full\NSname;

$obj = new Another; // instantiates object of class My\Full\Classname
NSname\subns\func(); // calls function My\Full\NSname\subns\func
```

Importing is performed at compile-time, and so does not affect dynamic class, function or constant names.

Example #3 Importing and dynamic names

```
<?php
use My\Full\Classname as Another, My\Full\NSname;

$obj = new Another; // instantiates object of class My\Full\Classname
$a = 'Another';
$obj = new $a;      // instantiates object of class Another
```

In addition, importing only affects unqualified and qualified names. Fully qualified names are absolute, and unaffected by imports.

Example #4 Importing and fully qualified names

```
<?php
use My\Full\Classname as Another, My\Full\NSname;

$obj = new Another; // instantiates object of class My\Full\Classname
$obj = new \Another; // instantiates object of class Another
$obj = new Another\thing; // instantiates object of class My\Full\Classname\thing
$obj = new \Another\thing; // instantiates object of class Another\thing
```

Scoping rules for importing

The `use` keyword must be declared in the outermost scope of a file (the global scope) or inside namespace declarations. This is because the importing is done at compile time and not runtime, so it cannot be block

scoped. The following example will show an illegal use of the `use` keyword:

Example #5 Illegal importing rule

```php
<?php
namespace Languages;

function toGreenlandic()
{
    use Languages\Danish;

    // ...
}
```

> **Note** :
>
> Importing rules are per file basis, meaning included files will *NOT* inherit the parent file's importing rules.

Group `use` declarations

Classes, functions and constants being imported from the same `namespace` can be grouped together in a single `use` statement.

```php
<?php

use some\namespace\ClassA;
use some\namespace\ClassB;
use some\namespace\ClassC as C;

use function some\namespace\fn_a;
use function some\namespace\fn_b;
use function some\namespace\fn_c;

use const some\namespace\ConstA;
use const some\namespace\ConstB;
use const some\namespace\ConstC;

// is equivalent to the following grouped use declaration
use some\namespace\{ClassA, ClassB, ClassC as C};
use function some\namespace\{fn_a, fn_b, fn_c};
use const some\namespace\{ConstA, ConstB, ConstC};
```

Global space

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Without any namespace definition, all class and function definitions are placed into the global space - as it was in PHP before namespaces were supported. Prefixing a name with `\` will specify that the name is required from the global space even in the context of the namespace.

Example #1 Using global space specification

```php
<?php
namespace A\B\C;

/* This function is A\B\C\fopen */
function fopen() {
     /* ... */
     $f = \fopen(...); // call global fopen
     return $f;
}
```

Fallback to global space

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Inside a namespace, when PHP encounters an unqualified Name in a class name, function or constant context, it resolves these with different priorities. Class names always resolve to the current namespace name. Thus to access internal or non-namespaced user classes, one must refer to them with their fully qualified Name as in:

Example #1 Accessing global classes inside a namespace

```php
<?php
namespace A\B\C;
class Exception extends \Exception {}

$a = new Exception('hi'); // $a is an object of class A\B\C\Exception
var_dump($a::class);
$b = new \Exception('hi'); // $b is an object of class Exception
var_dump($b::class);

$c = new ArrayObject; // fatal error, class A\B\C\ArrayObject not found
```

For functions and constants, PHP will fall back to global functions or constants if a namespaced function or constant does not exist.

Example #2 global functions/constants fallback inside a namespace

```php
<?php
namespace A\B\C;

const E_ERROR = 45;
function strlen($str)
{
    return \strlen($str) - 1;
}

echo E_ERROR, "\n"; // prints "45"
echo INI_ALL, "\n"; // prints "7" - falls back to global INI_ALL

echo strlen('hi'), "\n"; // prints "1"
if (is_array('hi')) { // prints "is not array"
    echo "is array\n";
} else {
    echo "is not array\n";
}
```

Name resolution rules

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

For the purposes of these resolution rules, here are some important definitions:

Namespace name definitions
Unqualified name

> This is an identifier without a namespace separator, such as `Foo`

Qualified name

> This is an identifier with a namespace separator, such as `Foo\Bar`

Fully qualified name

> This is an identifier with a namespace separator that begins with a namespace separator, such as `\Foo\Bar`. The namespace `\Foo` is also a fully qualified name.

Relative name

> This is an identifier starting with `namespace`, such as `namespace\Foo\Bar`.

Names are resolved following these resolution rules:

1. Fully qualified names always resolve to the name without leading namespace separator. For instance `\A\B` resolves to `A\B`.
2. Relative names always resolve to the name with `namespace` replaced by the current namespace. If the name occurs in the global namespace, the `namespace\` prefix is stripped. For example `namespace\A` inside namespace `X\Y` resolves to `X\Y\A`. The same name inside the global namespace resolves to `A`.
3. For qualified names the first segment of the name is translated according to the current class/namespace import table. For example, if the namespace `A\B\C` is imported as `C`, the name `C\D\E` is translated to `A\B\C\D\E`.
4. For qualified names, if no import rule applies, the current namespace is prepended to the name. For example, the name `C\D\E` inside namespace `A\B`, resolves to `A\B\C\D\E`.
5. For unqualified names, the name is translated according to the current import table for the respective symbol type. This means that class-like names are translated according to the class/namespace import table, function names according to the function import table and constants according to the constant import table. For example, after `use A\B\C;` a usage such as `new C()` resolves to the name `A\B\C()`. Similarly, after `use function A\B\foo;` a usage such as `foo()` resolves to the name `A\B\foo`.
6. For unqualified names, if no import rule applies and the name refers to a class-like symbol, the current namespace is prepended. For example `new C()` inside namespace `A\B` resolves to name `A\B\C`.
7. For unqualified names, if no import rule applies and the name refers to a function or constant and the code is outside the global namespace, the name is resolved at runtime. Assuming the code is in namespace `A\B`, here is how a call to function `foo()` is resolved:
    1. It looks for a function from the current namespace: `A\B\foo()`.

2. It tries to find and call the *global* function `foo()`.

Example #1 Name resolutions illustrated

```php
<?php
namespace A;
use B\D, C\E as F;

// function calls

foo();      // first tries to call "foo" defined in namespace "A"
            // then calls global function "foo"

\foo();     // calls function "foo" defined in global scope

my\foo();   // calls function "foo" defined in namespace "A\my"

F();        // first tries to call "F" defined in namespace "A"
            // then calls global function "F"

// class references

new B();    // creates object of class "B" defined in namespace "A"
            // if not found, it tries to autoload class "A\B"

new D();    // using import rules, creates object of class "D" defined in namespace "B"
            // if not found, it tries to autoload class "B\D"

new F();    // using import rules, creates object of class "E" defined in namespace "C"
            // if not found, it tries to autoload class "C\E"

new \B();   // creates object of class "B" defined in global scope
            // if not found, it tries to autoload class "B"

new \D();   // creates object of class "D" defined in global scope
            // if not found, it tries to autoload class "D"

new \F();   // creates object of class "F" defined in global scope
            // if not found, it tries to autoload class "F"

// static methods/namespace functions from another namespace

B\foo();    // calls function "foo" from namespace "A\B"

B::foo();   // calls method "foo" of class "B" defined in namespace "A"
            // if class "A\B" not found, it tries to autoload class "A\B"

D::foo();   // using import rules, calls method "foo" of class "D" defined in namespace "B"
            // if class "B\D" not found, it tries to autoload class "B\D"

\B\foo();   // calls function "foo" from namespace "B"

\B::foo();  // calls method "foo" of class "B" from global scope
            // if class "B" not found, it tries to autoload class "B"

// static methods/namespace functions of current namespace
```

```
A\B::foo();   // calls method "foo" of class "B" from namespace "A\A"
              // if class "A\A\B" not found, it tries to autoload class "A\A\B"

\A\B::foo();  // calls method "foo" of class "B" from namespace "A"
              // if class "A\B" not found, it tries to autoload class "A\B"
```

FAQ

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

This FAQ is split into two sections: common questions, and some specifics of implementation that are helpful to understand fully.

First, the common questions.

1. If I don't use namespaces, should I care about any of this?
2. How do I use internal or global classes in a namespace?
3. How do I use namespaces classes functions, or constants in their own namespace?
4. How does a name like `\my\name` or `\name` resolve?
5. How does a name like `my\name` resolve?
6. How does an unqualified class name like `name` resolve?
7. How does an unqualified function name or unqualified constant name like `name` resolve?

There are a few implementation details of the namespace implementations that are helpful to understand.

1. Import names must not conflict with classes defined in the same file.
2. Nested namespaces are not allowed.
3. Dynamic namespace names (quoted identifiers) should escape backslash.
4. Undefined Constants referenced using any backslash die with fatal error
5. Cannot override special constants **`null`**, **`true`** or **`false`**

If I don't use namespaces, should I care about any of this?

No. Namespaces do not affect any existing code in any way, or any as-yet-to-be-written code that does not contain namespaces. You can write this code if you wish:

Example #1 Accessing global classes outside a namespace

```
<?php
$a = new \stdClass;
print $a::class;
```

This is functionally equivalent to:

Example #2 Accessing global classes outside a namespace

```php
<?php
$a = new stdClass;
print $a::class;
```

How do I use internal or global classes in a namespace?

Example #3 Accessing internal classes in namespaces

```php
<?php
namespace foo;
$a = new \stdClass;
print $a::class . "\n";

function test(?\ArrayObject $parameter_type_example = null) {}

$a = \FilesystemIterator::CURRENT_AS_FILEINFO;
print $a . "\n";

// extending an internal or global class
class MyException extends \Exception {}
print MyException::class . "\n";
```

How do I use namespaces classes, functions, or constants in their own namespace?

Example #4 Accessing internal classes, functions or constants in namespaces

```php
<?php
namespace foo;

class MyClass {}

// using a class from the current namespace as a parameter type
function test(?MyClass $parameter_type_example = null) {}
// another way to use a class from the current namespace as a parameter type
function test2(?\foo\MyClass $parameter_type_example = null) {}

// extending a class from the current namespace
class Extended extends MyClass {}
print Extended::class . "\n";

// accessing a global function
$a = \strlen("test");
print $a . "\n";

// accessing a global constant
$b = \INI_ALL;
print $b . "\n";
```

How does a name like `\my\name` or `\name` resolve?

Names that begin with a `\` always resolve to what they look like, so `\my\name` is in fact `my\name`, and `\Exception` is `Exception`.

Example #5 Fully Qualified names

```php
<?php
namespace foo;
$a = new \my\name(); // instantiates "my\name" class
echo \strlen('hi'); // calls function "strlen"
$a = \INI_ALL; // $a is set to the value of constant "INI_ALL"
```

How does a name like `my\name` resolve?

Names that contain a backslash but do not begin with a backslash like `my\name` can be resolved in 2 different ways.

If there is an import statement that aliases another name to `my`, then the import alias is applied to the `my` in `my\name`.

Otherwise, the current namespace name is prepended to `my\name`.

Example #6 Qualified names

```php
<?php
namespace foo;
use blah\blah as foo;

$a = new my\name(); // instantiates "foo\my\name" class
foo\bar::name(); // calls static method "name" in class "blah\blah\bar"
my\bar(); // calls function "foo\my\bar"
$a = my\BAR; // sets $a to the value of constant "foo\my\BAR"
```

How does an unqualified class name like `name` resolve?

Class names that do not contain a backslash like `name` can be resolved in 2 different ways.

If there is an import statement that aliases another name to `name`, then the import alias is applied.

Otherwise, the current namespace name is prepended to `name`.

Example #7 Unqualified class names

```php
<?php
namespace foo;
use blah\blah as foo;

$a = new name(); // instantiates "foo\name" class
foo::name(); // calls static method "name" in class "blah\blah"
```

How does an unqualified function name or unqualified constant name like `name` resolve?

Function or constant names that do not contain a backslash like `name` can be resolved in 2 different ways.

First, the current namespace name is prepended to `name`.

Finally, if the constant or function `name` does not exist in the current namespace, a global constant or function `name` is used if it exists.

Example #8 Unqualified function or constant names

```php
<?php
namespace foo;
use blah\blah as foo;

const FOO = 1;

function my() {
    print __FUNCTION__ . "\n";
}
function foo() {
    print __FUNCTION__ . "\n";
}
function sort(&$a)
{
    \sort($a); // calls the global function "sort"
    $a = array_flip($a);
    return $a;
}

my(); // calls "foo\my"
print strlen('hi') . "\n"; // calls global function "strlen" because "foo\strlen" does not exist
$arr = array(1,3,2);
$b = sort($arr); // calls function "foo\sort"
var_dump($b);
foo(); // calls function "foo\foo" - import is not applied

$a = FOO; // sets $a to value of constant "foo\FOO" - import is not applied
print $a;
$b = INI_ALL; // sets $b to value of global constant "INI_ALL"
print $b;
```

Import names must not conflict with classes defined in the same file.

The following script combinations are legal:

file1.php

```php
<?php
namespace my\stuff;
class MyClass {}
```

another.php

```php
<?php
namespace another;
class thing {}
```

file2.php

```php
<?php
namespace my\stuff;
include 'file1.php';
include 'another.php';
```

```php
use another\thing as MyClass;
$a = new MyClass; // instantiates class "thing" from namespace another
```

There is no name conflict, even though the class `MyClass` exists within the `my\stuff` namespace, because the MyClass definition is in a separate file. However, the next example causes a fatal error on name conflict because MyClass is defined in the same file as the use statement.

```php
<?php
namespace my\stuff;
use another\thing as MyClass;
class MyClass {} // fatal error: MyClass conflicts with import statement
$a = new MyClass;
```

Nested namespaces are not allowed.

PHP does not allow nesting namespaces

```php
<?php
namespace my\stuff {
    namespace nested {
        class foo {}
    }
}
```

However, it is easy to simulate nested namespaces like so:

```php
<?php
namespace my\stuff\nested {
    class foo {}
}
```

Dynamic namespace names (quoted identifiers) should escape backslash

It is very important to realize that because the backslash is used as an escape character within strings, it should always be doubled when used inside a string. Otherwise there is a risk of unintended consequences:

Example #9 Dangers of using namespaced names inside a double-quoted string

```php
<?php
$a = "dangerous\name"; // \n is a newline inside double quoted strings!
$obj = new $a;

$a = 'not\at\all\dangerous'; // no problems here.
$obj = new $a;
```

Inside a single-quoted string, the backslash escape sequence is much safer to use, but it is still recommended practice to escape backslashes in all strings as a best practice.

Undefined Constants referenced using any backslash die with fatal error

Any undefined constant that is unqualified like `FOO` will produce a notice explaining that PHP assumed `FOO` was the value of the constant. Any constant, qualified or fully qualified, that contains a backslash will produce a fatal error if not found.

Example #10 Undefined constants

```php
<?php
namespace bar;
$a = FOO; // produces notice - undefined constants "FOO" assumed "FOO";
$a = \FOO; // fatal error, undefined namespace constant FOO
$a = Bar\FOO; // fatal error, undefined namespace constant bar\Bar\FOO
$a = \Bar\FOO; // fatal error, undefined namespace constant Bar\FOO
```

Cannot override special constants `null`, `true` or `false`

Any attempt to define a namespaced constant that is a special, built-in constant results in a fatal error

Example #11 Undefined constants

```php
<?php
namespace bar;
const NULL = 0; // fatal error;
const true = 'stupid'; // also fatal error;
// etc.
```

Enumerations

Table of Contents

Enumerations overview

(PHP 8 >= 8.1.0)

Enumerations, or "Enums", allow a developer to define a custom type that is limited to one of a discrete number of possible values. That can be especially helpful when defining a domain model, as it enables "making invalid states unrepresentable."

Enums appear in many languages with a variety of different features. In PHP, Enums are a special kind of object. The Enum itself is a class, and its possible cases are all single-instance objects of that class. That means Enum cases are valid objects and may be used anywhere an object may be used, including type checks.

The most popular example of enumerations is the built-in boolean type, which is an enumerated type with legal values `true` and `false`. Enums allows developers to define their own arbitrarily robust enumerations.

Basic enumerations

Enums are similar to classes, and share the same namespaces as classes, interfaces, and traits. They are also autoloadable the same way. An Enum defines a new type, which has a fixed, limited number of possible legal values.

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}
```

This declaration creates a new enumerated type named `Suit`, which has four and only four legal values: `Suit::Hearts`, `Suit::Diamonds`, `Suit::Clubs`, and `Suit::Spades`. Variables may be assigned to one of those legal values. A function may be type checked against an enumerated type, in which case only values of that type may be passed.

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

function pick_a_card(Suit $suit)
{
    var_dump($suit);
}

$val = Suit::Diamonds;

// OK
pick_a_card($val);

// OK
pick_a_card(Suit::Clubs);

// TypeError: pick_a_card(): Argument #1 ($suit) must be of type Suit, string given
pick_a_card('Spades');
```

An Enumeration may have zero or more `case` definitions, with no maximum. A zero-case enum is syntactically valid, if rather useless.

For Enumeration cases, the same syntax rules apply as to any label in PHP, see Constants.

By default, cases are not intrinsically backed by a scalar value. That is, `Suit::Hearts` is not equal to `"0"`. Instead, each case is backed by a singleton object of that name. That means that:

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

$a = Suit::Spades;
$b = Suit::Spades;

if ($a === $b) {
    print "Suits match using ===\n";
}
```

```php
if ($a instanceof Suit) {
    print "Suits match using instanceof\n";
}

if ($a !== 'Spades') {
    print "Suit does not match the string\n";
}
```

It also means that enum values are never < or > each other, since those comparisons are not meaningful on objects. Those comparisons will always return `false` when working with enum values.

This type of case, with no related data, is called a "Pure Case." An Enum that contains only Pure Cases is called a Pure Enum.

All Pure Cases are implemented as instances of their enum type. The enum type is represented internally as a class.

All Cases have a read-only property, `name`, that is the case-sensitive name of the case itself.

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

print Suit::Spades->name;
// prints "Spades"
```

It is also possible to use the defined() and constant() functions to check for the existence of or read an enum case if the name is obtained dynamically. This is, however, discouraged as using Backed enums should work for most use cases.

Backed enumerations

By default, Enumerated Cases have no scalar equivalent. They are simply singleton objects. However, there are ample cases where an Enumerated Case needs to be able to round-trip to a database or similar datastore, so having a built-in scalar (and thus trivially serializable) equivalent defined intrinsically is useful.

To define a scalar equivalent for an Enumeration, the syntax is as follows:

```php
<?php

enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}
```

A case that has a scalar equivalent is called a Backed Case, as it is "Backed" by a simpler value. An Enum that contains all Backed Cases is called a "Backed Enum." A Backed Enum may contain only Backed Cases. A Pure Enum may contain only Pure Cases.

A Backed Enum may be backed by types of `int` or `string`, and a given enumeration supports only a single type at a time (that is, no union of `int|string`). If an enumeration is marked as having a scalar equivalent, then all cases must have a unique scalar equivalent defined explicitly. There are no auto-generated scalar equivalents (e.g., sequential integers). Backed cases must be unique; two backed enum cases may not have the same scalar equivalent. However, a constant may refer to a case, effectively creating an alias. See Enumeration constants.

The equivalent values may be a constant scalar expression. Prior to PHP 8.2.0, the equivalent values had to be literals or literal expressions. This means that constants and constant expressions were not supported. That is, `1 + 1` was allowed, but `1 + SOME_CONST` was not.

Backed Cases have an additional read-only property, `value`, which is the value specified in the definition.

```php
<?php

enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

print Suit::Clubs->value;
// Prints "C"
```

In order to enforce the `value` property as read-only, a variable cannot be assigned as a reference to it. That is, the following throws an error:

```php
<?php

enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

$suit = Suit::Clubs;
$ref = &$suit->value;
// Fatal Error: Cannot indirectly modify readonly property Suit::$value
```

Backed enums implement an internal BackedEnum interface, which exposes two additional methods:

- `from(int|string): self` will take a scalar and return the corresponding Enum Case. If one is not found, it will throw a ValueError. This is mainly useful in cases where the input scalar is trusted and a missing enum value should be considered an application-stopping error.
- `tryFrom(int|string): ?self` will take a scalar and return the corresponding Enum Case. If one is not found, it will return `null`. This is mainly useful in cases where the input scalar is untrusted and the caller

wants to implement their own error handling or default-value logic.

The `from()` and `tryFrom()` methods follow standard weak/strong typing rules. In weak typing mode, passing an integer or string is acceptable and the system will coerce the value accordingly. Passing a float will also work and be coerced. In strict typing mode, passing an integer to `from()` on a string-backed enum (or vice versa) will result in a TypeError, as will a float in all circumstances. All other parameter types will throw a TypeError in both modes.

```php
<?php

enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

function get_stuff_from_database($id) {
    return [
        'suit' => 'S',
    ];
}

$record = get_stuff_from_database(42);
print $record['suit'] . "\n";

$suit = Suit::tryFrom('A') ?? Suit::Spades;
// Invalid data returns null, so Suit::Spades is used instead.
print $suit->value . "\n";

$suit =  Suit::from('X');
// Invalid data throws a ValueError: "X" is not a valid backing scalar value for enum Suit
print $suit->value . "\n";
```

Manually defining a `from()` or `tryFrom()` method on a Backed Enum will result in a fatal error.

Enumeration methods

Enums (both Pure Enums and Backed Enums) may contain methods, and may implement interfaces. If an Enum implements an interface, then any type check for that interface will also accept all cases of that Enum.

```php
<?php

interface Colorful
{
    public function color(): string;
}

enum Suit implements Colorful
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
```

```
    // Fulfills the interface contract.
    public function color(): string
    {
        return match ($this) {
            Suit::Hearts, Suit::Diamonds => 'Red',
            Suit::Clubs, Suit::Spades => 'Black',
        };
    }

    // Not part of an interface; that's fine.
    public function shape(): string
    {
        return "Rectangle";
    }
}

function paint(Colorful $c)
{
   print $c->color() . "\n";
}

paint(Suit::Clubs);  // Works

print Suit::Diamonds->shape(); // prints "Rectangle"
```

In this example, all four instances of `Suit` have two methods, `color()` and `shape()`. As far as calling code and type checks are concerned, they behave exactly the same as any other object instance.

On a Backed Enum, the interface declaration goes after the backing type declaration.

```
<?php

interface Colorful
{
    public function color(): string;
}

enum Suit: string implements Colorful
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';

    // Fulfills the interface contract.
    public function color(): string
    {
        return match ($this) {
            Suit::Hearts, Suit::Diamonds => 'Red',
            Suit::Clubs, Suit::Spades => 'Black',
        };
    }
}
```

Inside a method, the `$this` variable is defined and refers to the Case instance.

Methods may be arbitrarily complex, but in practice will usually return a static value or match on `$this` to provide different results for different cases.

Note that in this case it would be a better data modeling practice to also define a `SuitColor` Enum Type with values Red and Black and return that instead. However, that would complicate this example.

The above hierarchy is logically similar to the following class structure (although this is not the actual code that runs):

```php
<?php

interface Colorful
{
    public function color(): string;
}

final class Suit implements UnitEnum, Colorful
{
    public const Hearts = new self('Hearts');
    public const Diamonds = new self('Diamonds');
    public const Clubs = new self('Clubs');
    public const Spades = new self('Spades');

    private function __construct(public readonly string $name) {}

    public function color(): string
    {
        return match ($this) {
            Suit::Hearts, Suit::Diamonds => 'Red',
            Suit::Clubs, Suit::Spades => 'Black',
        };
    }

    public function shape(): string
    {
        return "Rectangle";
    }

    public static function cases(): array
    {
        // Illegal method, because manually defining a cases() method on an Enum is disallowed.
        // See also "Value listing" section.
    }
}
```

Methods may be public, private, or protected, although in practice private and protected are equivalent as inheritance is not allowed.

Enumeration static methods

Enumerations may also have static methods. The use for static methods on the enumeration itself is primarily for alternative constructors. E.g.:

```php
<?php
```

```php
enum Size
{
    case Small;
    case Medium;
    case Large;

    public static function fromLength(int $cm): self
    {
        return match (true) {
            $cm < 50 => self::Small,
            $cm < 100 => self::Medium,
            default => self::Large,
        };
    }
}

var_dump(Size::fromLength(50));
```

Static methods may be public, private, or protected, although in practice private and protected are equivalent as inheritance is not allowed.

Enumeration constants

Enumerations may include constants, which may be public, private, or protected, although in practice private and protected are equivalent as inheritance is not allowed.

An enum constant may refer to an enum case:

```php
<?php

enum Size
{
    case Small;
    case Medium;
    case Large;

    public const Huge = self::Large;
}

var_dump(Size::Huge);
```

Traits

Enumerations may leverage traits, which will behave the same as on classes. The caveat is that traits `used` in an enum must not contain properties. They may only include methods, static methods, and constants. A trait with properties will result in a fatal error.

```php
<?php

interface Colorful
{
    public function color(): string;
}

trait Rectangle
```

```
{
    public function shape(): string
    {
        return "Rectangle";
    }
}

enum Suit implements Colorful
{
    use Rectangle;

    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;

    public function color(): string
    {
        return match ($this) {
            Suit::Hearts, Suit::Diamonds => 'Red',
            Suit::Clubs, Suit::Spades => 'Black',
        };
    }
}

$suit = Suit::Spades;
var_dump($suit->color());
var_dump($suit->shape());
```

Enum values in constant expressions

Because cases are represented as constants on the enum itself, they may be used as static values in most constant expressions: property defaults, static variable defaults, parameter defaults, global and class constant values. They may not be used in other enum case values, but normal constants may refer to an enum case.

However, implicit magic method calls such as ArrayAccess on enums are not allowed in static or constant definitions as we cannot absolutely guarantee that the resulting value is deterministic or that the method invocation is free of side effects. Function calls, method calls, and property access continue to be invalid operations in constant expressions.

```
<?php

// This is an entirely legal Enum definition.
enum Direction implements ArrayAccess
{
    case Up;
    case Down;

    public function offsetExists($offset): bool
    {
        return false;
    }

    public function offsetGet($offset): mixed
    {
        return null;
```

```
    }

    public function offsetSet($offset, $value): void
    {
        throw new Exception();
    }

    public function offsetUnset($offset): void
    {
        throw new Exception();
    }
}

class Foo
{
    // This is allowed.
    const DOWN = Direction::Down;

    // This is disallowed, as it may not be deterministic.
    const UP = Direction::Up['short'];
    // Fatal error: Cannot use [] on enums in constant expression
}

// This is entirely legal, because it's not a constant expression.
$x = Direction::Up['short'];
var_dump("\$x is " . var_export($x, true));

$foo = new Foo();
```

Differences from objects

Although Enums are built on classes and objects, they do not support all object-related functionality. In particular, Enum cases are forbidden from having state.

- Constructors and Destructors are forbidden.
- Inheritance is not supported. Enums may not extend or be extended.
- Static or object properties are not allowed.
- Cloning an Enum case is not supported, as cases must be singleton instances.
- Magic methods, except for those listed below, are disallowed.
- Enums must always be declared before they are used.

The following object functionality is available, and behaves just as it does on any other object:

- Public, private, and protected methods.
- Public, private, and protected static methods.
- Public, private, and protected constants.
- Enums may implement any number of interfaces.
- Enums and cases may have attributes attached to them. The **`TARGET_CLASS`** target filter includes Enums themselves. The **`TARGET_CLASS_CONST`** target filter includes Enum Cases.
- __call, __callStatic, and __invoke magic methods
- **`__CLASS__`** and **`__FUNCTION__`** constants behave as normal

The `::class` magic constant on an Enum type evaluates to the type name including any namespace, exactly the same as an object. The `::class` magic constant on a Case instance also evaluates to the Enum type, as it is

an instance of that type.

Additionally, enum cases may not be instantiated directly with `new`, nor with ReflectionClass::newInstanceWithoutConstructor() in reflection. Both will result in an error.

```php
<?php

$clovers = new Suit();
// Error: Cannot instantiate enum Suit

$horseshoes = (new ReflectionClass(Suit::class))->newInstanceWithoutConstructor()
// Error: Cannot instantiate enum Suit
```

Value listing

Both Pure Enums and Backed Enums implement an internal interface named UnitEnum. `UnitEnum` includes a static method `cases()`. `cases()` returns a packed array of all defined Cases in the order of declaration.

```php
<?php

enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

var_dump(Suit::cases());

enum SuitBacked: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

var_dump(SuitBacked::cases());
```

Manually defining a `cases()` method on an Enum will result in a fatal error.

Serialization

Enumerations are serialized differently from objects. Specifically, they have a new serialization code, `"E"`, that specifies the name of the enum case. The deserialization routine is then able to use that to set a variable to the existing singleton value. That ensures that:

```php
<?php

enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
```

```
    case Spades = 'S';
}

Suit::Hearts === unserialize(serialize(Suit::Hearts));

print serialize(Suit::Hearts);
// E:11:"Suit:Hearts";
```

On deserialization, if an enum and case cannot be found to match a serialized value a warning will be issued and `false` returned.

The `allowed_classes` option of unserialize() does not affect Enumerations.

If a Pure Enum is serialized to JSON, an error will be thrown. If a Backed Enum is serialized to JSON, it will be represented by its scalar value only, in the appropriate type. The behavior of both may be overridden by implementing JsonSerializable.

For print_r(), the output of an enum case is slightly different from objects to minimize confusion.

```
<?php

enum Foo
{
    case Bar;
}

enum Baz: int
{
    case Beep = 5;
}

print_r(Foo::Bar);
print_r(Baz::Beep);

/* Produces

Foo Enum (
    [name] => Bar
)
Baz Enum:int {
    [name] => Beep
    [value] => 5
}
*/
```

Why enums aren't extendable

Classes have contracts on their methods:

```
<?php

class A {}
class B extends A {}

function foo(A $a) {}
```

```
function bar(B $b)
{
    foo($b);
}
```

This code is type-safe, as B follows the contract of A, and through the magic of co/contra-variance, any expectation one may have of the methods will be preserved, exceptions excepted.

Enums have contracts on their cases, not methods:

```
<?php

enum ErrorCode
{
    case SOMETHING_BROKE;
}

function quux(ErrorCode $errorCode)
{
    // When written, this code appears to cover all cases
    match ($errorCode) {
        ErrorCode::SOMETHING_BROKE => true,
    };
}
```

The match statement in the function `quux` can be static analyzed to cover all of the cases in ErrorCode.

But imagine it was allowed to extend enums:

```
<?php

// Thought experiment code where enums are not final.
// Note, this won't actually work in PHP.
enum MoreErrorCode extends ErrorCode
{
    case PEBKAC;
}

function fot(MoreErrorCode $errorCode)
{
    quux($errorCode);
}

fot(MoreErrorCode::PEBKAC);
```

Under normal inheritance rules, a class that extends another will pass the type check.

The problem would be that the match statement in `quux()` no longer covers all the cases. Because it doesn't know about `MoreErrorCode::PEBKAC` the match will throw an exception.

Because of this enums are final and can't be extended.

Examples

Example #1 Basic limited values

```php
<?php

enum SortOrder
{
    case Asc;
    case Desc;
}

function query($fields, $filter, SortOrder $order = SortOrder::Asc)
{
     /* ... */
}
```

The `query()` function can now proceed safe in the knowledge that `$order` is guaranteed to be either `SortOrder::Asc` or `SortOrder::Desc`. Any other value would have resulted in a TypeError, so no further error checking or testing is needed.

Example #2 Advanced exclusive values

```php
<?php

enum UserStatus: string
{
    case Pending = 'P';
    case Active = 'A';
    case Suspended = 'S';
    case CanceledByUser = 'C';

    public function label(): string
    {
        return match ($this) {
            self::Pending => 'Pending',
            self::Active => 'Active',
            self::Suspended => 'Suspended',
            self::CanceledByUser => 'Canceled by user',
        };
    }
}

$status = UserStatus::Suspended;
var_dump($status->label());
```

In this example, a user's status may be one of, and exclusively, `UserStatus::Pending`, `UserStatus::Active`, `UserStatus::Suspended`, or `UserStatus::CanceledByUser`. A function can type a parameter against `UserStatus` and then only accept those four values, period.

All four values have a `label()` method, which returns a human-readable string. That string is independent of the "machine name" scalar equivalent string, which can be used in, for example, a database field or an HTML select box.

```php
<?php

enum UserStatus: string
{
    case Pending = 'P';
```

```
    case Active = 'A';
    case Suspended = 'S';
    case CanceledByUser = 'C';

    public function label(): string
    {
        return match($this) {
            self::Pending => 'Pending',
            self::Active => 'Active',
            self::Suspended => 'Suspended',
            self::CanceledByUser => 'Canceled by user',
        };
    }
}

foreach (UserStatus::cases() as $case) {
    printf(
        "<option value=\"%s\">%s</option>\n",
        htmlentities($case->value),
        htmlentities($case->label())
    );
}
```

Errors

Table of Contents

Introduction

Sadly, no matter how careful we are when writing our code, errors are a fact of life. PHP will report errors, warnings and notices for many common coding and runtime problems, and knowing how to detect and handle these errors will make debugging much easier.

Basics

PHP reports errors in response to a number of internal error conditions. These may be used to signal a number of different conditions, and can be displayed and/or logged as required.

Every error that PHP generates includes a type. A list of these error types is available, along with a short description of their behaviour and how they can be caused.

Handling errors with PHP

If no error handler is set, then PHP will handle any errors that occur according to its configuration. Which errors are reported and which are ignored is controlled by the `error_reporting` php.ini directive, or at runtime by calling error_reporting(). It is strongly recommended that the configuration directive be set, however, as some errors can occur before execution of your script begins.

In a development environment, you should always set `error_reporting` to **`E_ALL`**, as you need to be aware of and fix the issues raised by PHP. In production, you may wish to set this to a less verbose level such as `E_ALL & ~E_NOTICE & ~E_DEPRECATED`, but in many cases **`E_ALL`** is also appropriate, as it may provide early warning of potential issues.

What PHP does with these errors depends on two further php.ini directives. `display_errors` controls whether the error is shown as part of the script's output. This should always be disabled in a production environment, as it can include confidential information such as database passwords, but is often useful to enable in development, as it ensures immediate reporting of issues.

In addition to displaying errors, PHP can log errors when the `log_errors` directive is enabled. This will log any errors to the file or syslog defined by `error_log`. This can be extremely useful in a production environment, as you can log errors that occur and then generate reports based on those errors.

User error handlers

If PHP's default error handling is inadequate, you can also handle many types of error with your own custom error handler by installing it with set_error_handler(). While some error types cannot be handled this way, those that can be handled can then be handled in the way that your script sees fit: for example, this can be used to show a custom error page to the user and then report more directly than via a log, such as by sending an e-mail.

Changelog

Version	Description
8.5.0	Fatal errors (such as an exceeded maximum execution time) now include a stack trace in their error message.

Errors in PHP 7

PHP 7 changes how most errors are reported by PHP. Instead of reporting errors through the traditional error reporting mechanism used by PHP 5, most errors are now reported by throwing Error exceptions.

As with normal exceptions, these Error exceptions will bubble up until they reach the first matching `catch` block. If there are no matching blocks, then any default exception handler installed with set_exception_handler() will be called, and if there is no default exception handler, then the exception will be converted to a fatal error and will be handled like a traditional error.

As the Error hierarchy does not inherit from Exception, code that uses `catch (Exception $e) { ... }` blocks to handle uncaught exceptions in PHP 5 will find that these Errors are not caught by these blocks. Either a `catch (Error $e) { ... }` block or a set_exception_handler() handler is required.

Error hierarchy

- Throwable
  - Error
    - ArithmeticError
      - DivisionByZeroError
    - AssertionError
    - CompileError
      - ParseError
    - TypeError

        - ArgumentCountError
    - ValueError
    - UnhandledMatchError
    - FiberError
    - RequestParseBodyException
- Exception
    - ...

Exceptions

Table of Contents

> **Note** :
>
> See also the Exception class

PHP has an exception model similar to that of other programming languages. An exception can be `throw`n, and caught ("`catch`ed") within PHP. Code may be surrounded in a `try` block, to facilitate the catching of potential exceptions. Each `try` must have at least one corresponding `catch` or `finally` block.

If an exception is thrown and its current function scope has no `catch` block, the exception will "bubble up" the call stack to the calling function until it finds a matching `catch` block. All `finally` blocks it encounters along the way will be executed. If the call stack is unwound all the way to the global scope without encountering a matching `catch` block, the program will terminate with a fatal error unless a global exception handler has been set.

The thrown object must be an `instanceof` Throwable. Trying to throw an object that is not will result in a PHP Fatal Error.

As of PHP 8.0.0, the `throw` keyword is an expression and may be used in any expression context. In prior versions it was a statement and was required to be on its own line.

`catch`

A `catch` block defines how to respond to a thrown exception. A `catch` block defines one or more types of exception or error it can handle, and optionally a variable to which to assign the exception. (The variable was required prior to PHP 8.0.0.) The first `catch` block a thrown exception or error encounters that matches the type of the thrown object will handle the object.

Multiple `catch` blocks can be used to catch different classes of exceptions. Normal execution (when no exception is thrown within the `try` block) will continue after that last `catch` block defined in sequence. Exceptions can be `throw`n (or re-thrown) within a `catch` block. If not, execution will continue after the `catch` block that was triggered.

When an exception is thrown, code following the statement will not be executed, and PHP will attempt to find the first matching `catch` block. If an exception is not caught, a PHP Fatal Error will be issued with an "`Uncaught`

`Exception ...`" message, unless a handler has been defined with set_exception_handler().

As of PHP 7.1.0, a `catch` block may specify multiple exceptions using the pipe (`|`) character. This is useful for when different exceptions from different class hierarchies are handled the same.

As of PHP 8.0.0, the variable name for a caught exception is optional. If not specified, the `catch` block will still execute but will not have access to the thrown object.

`finally`

A `finally` block may also be specified after or instead of `catch` blocks. Code within the `finally` block will always be executed after the `try` and `catch` blocks, regardless of whether an exception has been thrown, and before normal execution resumes.

One notable interaction is between the `finally` block and a `return` statement. If a `return` statement is encountered inside either the `try` or the `catch` blocks, the `finally` block will still be executed. Moreover, the `return` statement is evaluated when encountered, but the result will be returned after the `finally` block is executed. Additionally, if the `finally` block also contains a `return` statement, the value from the `finally` block is returned.

Another notable interaction is between an exception thrown from within a `try` block, and an exception thrown from within a `finally` block. If both blocks throw an exception, then the exception thrown from the `finally` block will be the one that is propagated, and the exception thrown from the `try` block will be used as its previous exception.

Global exception handler

If an exception is allowed to bubble up to the global scope, it may be caught by a global exception handler if set. The set_exception_handler() function can set a function that will be called in place of a `catch` block if no other block is invoked. The effect is essentially the same as if the entire program were wrapped in a `try-catch` block with that function as the `catch`.

Notes

> **Note** :
>
> Internal PHP functions mainly use Error reporting, only modern Object-oriented extensions use exceptions. However, errors can be easily translated to exceptions with ErrorException. This technique only works with non-fatal errors, however.
>
> **Example #1 Converting error reporting to exceptions**
>
> ```php
> <?php
> function exceptions_error_handler($severity, $message, $filename, $lineno) {
>     throw new ErrorException($message, 0, $severity, $filename, $lineno);
> }
>
> set_error_handler('exceptions_error_handler');
> ```

> **Tip**
>
> The Standard PHP Library (SPL) provides a good number of built-in exceptions.

Examples

Example #2 Throwing an Exception

```php
<?php
function inverse($x) {
    if (!$x) {
        throw new Exception('Division by zero.');
    }
    return 1/$x;
}

try {
    echo inverse(5) . "\n";
    echo inverse(0) . "\n";
} catch (Exception $e) {
    echo 'Caught exception: ',  $e->getMessage(), "\n";
}

// Continue execution
echo "Hello World\n";
```

The above example will output:

```
0.2
Caught exception: Division by zero.
Hello World
```

Example #3 Exception handling with a `finally` block

```php
<?php
function inverse($x) {
    if (!$x) {
        throw new Exception('Division by zero.');
    }
    return 1/$x;
}

try {
    echo inverse(5) . "\n";
} catch (Exception $e) {
    echo 'Caught exception: ',  $e->getMessage(), "\n";
} finally {
    echo "First finally.\n";
}

try {
    echo inverse(0) . "\n";
} catch (Exception $e) {
    echo 'Caught exception: ',  $e->getMessage(), "\n";
} finally {
```

```
    echo "Second finally.\n";
}

// Continue execution
echo "Hello World\n";
```

The above example will output:

```
0.2
First finally.
Caught exception: Division by zero.
Second finally.
Hello World
```

Example #4 Interaction between the `finally` block and `return`

```
<?php

function test() {
    try {
        throw new Exception('foo');
    } catch (Exception $e) {
        return 'catch';
    } finally {
        return 'finally';
    }
}

echo test();
```

The above example will output:

```
finally
```

Example #5 Nested Exception

```
<?php

class MyException extends Exception { }

class Test {
    public function testing() {
        try {
            try {
                throw new MyException('foo!');
            } catch (MyException $e) {
```

```
                // rethrow it
                throw $e;
            }
        } catch (Exception $e) {
            var_dump($e->getMessage());
        }
    }
}

$foo = new Test;
$foo->testing();
```

The above example will output:

```
string(4) "foo!"
```

Example #6 Multi catch exception handling

```
<?php

class MyException extends Exception { }

class MyOtherException extends Exception { }

class Test {
    public function testing() {
        try {
            throw new MyException();
        } catch (MyException | MyOtherException $e) {
            var_dump(get_class($e));
        }
    }
}

$foo = new Test;
$foo->testing();
```

The above example will output:

```
string(11) "MyException"
```

Example #7 Omitting the caught variable

Only permitted in PHP 8.0.0 and later.

```php
<?php

class SpecificException extends Exception {}

function test() {
    throw new SpecificException('Oopsie');
}

try {
    test();
} catch (SpecificException) {
    print "A SpecificException was thrown, but we don't care about the details.";
}
```

The above example will output:

```
A SpecificException was thrown, but we don't care about the details.
```

Example #8 Throw as an expression

Only permitted in PHP 8.0.0 and later.

```php
<?php

function test() {
    do_something_risky() or throw new Exception('It did not work');
}

function do_something_risky() {
    return false; // Simulate failure
}

try {
    test();
} catch (Exception $e) {
    print $e->getMessage();
}
```

The above example will output:

```
It did not work
```

Example #9 Exception in try and in finally

```php
<?php
```

```php
try {
    try {
        throw new Exception(message: 'Third', previous: new Exception('Fourth'));
    } finally {
        throw new Exception(message: 'First', previous: new Exception('Second'));
    }
} catch (Exception $e) {
    var_dump(
        $e->getMessage(),
        $e->getPrevious()->getMessage(),
        $e->getPrevious()->getPrevious()->getMessage(),
        $e->getPrevious()->getPrevious()->getPrevious()->getMessage(),
    );
}
```

The above example will output:

```
string(5) "First"
string(6) "Second"
string(5) "Third"
string(6) "Fourth"
```

Extending Exceptions

A User defined Exception class can be defined by extending the built-in Exception class. The members and properties below, show what is accessible within the child class that derives from the built-in Exception class.

Example #1 The Built in Exception class

```php
<?php
class Exception implements Throwable
{
    protected $message = '';                    // exception message
    private   $string;                          // __toString cache
    protected $code = 0;                        // user defined exception code
    protected $file;                            // source filename of exception
    protected $line;                            // source line of exception
    private   $trace;                           // backtrace
    private   $previous;                        // previous exception if nested exception

    public function __construct($message = '', $code = 0, ?Throwable $previous = null);

    private function __clone();                 // Inhibits cloning of exceptions.

    final public  function getMessage();        // message of exception
    final public  function getCode();           // code of exception
    final public  function getFile();           // source filename
    final public  function getLine();           // source line
    final public  function getTrace();          // an array of the backtrace()
    final public  function getPrevious();       // previous exception
    final public  function getTraceAsString();  // formatted string of trace
```

```
    // Overridable
    public function __toString();               // formatted string for display
}
```

If a class extends the built-in Exception class and re-defines the constructor, it is highly recommended that it also call parent::__construct() to ensure all available data has been properly assigned. The __toString() method can be overridden to provide a custom output when the object is presented as a string.

> **Note** :
>
> Exceptions cannot be cloned. Attempting to clone an exception throws an Error, even if a subclass declares its own __clone() method.

Example #2 Extending the Exception class

```
<?php
/**
 * Define a custom exception class
 */
class MyException extends Exception
{
    // Redefine the exception so message isn't optional
    public function __construct($message, $code = 0, ?Throwable $previous = null) {
        // some code

        // make sure everything is assigned properly
        parent::__construct($message, $code, $previous);
    }

    // custom string representation of object
    public function __toString() {
        return __CLASS__ . ": [{$this->code}]: {$this->message}\n";
    }

    public function customFunction() {
        echo "A custom function for this type of exception\n";
    }
}


/**
 * Create a class to test the exception
 */
class TestException
{
    public $var;

    const THROW_NONE    = 0;
    const THROW_CUSTOM  = 1;
    const THROW_DEFAULT = 2;

    function __construct($avalue = self::THROW_NONE) {

        switch ($avalue) {
```

```
            case self::THROW_CUSTOM:
                // throw custom exception
                throw new MyException('1 is an invalid parameter', 5);
                break;

            case self::THROW_DEFAULT:
                // throw default one.
                throw new Exception('2 is not allowed as a parameter', 6);
                break;

            default:
                // No exception, object will be created.
                $this->var = $avalue;
                break;
        }
    }
}


echo "# Example 1\n";
try {
    $o = new TestException(TestException::THROW_CUSTOM);
} catch (MyException $e) {      // Will be caught
    echo "Caught my exception\n", $e;
    $e->customFunction();
} catch (Exception $e) {        // Skipped
    echo "Caught Default Exception\n", $e;
}

// Continue execution
var_dump($o); // Null


echo "\n\n# Example 2\n";
try {
    $o = new TestException(TestException::THROW_DEFAULT);
} catch (MyException $e) {      // Doesn't match this type
    echo "Caught my exception\n", $e;
    $e->customFunction();
} catch (Exception $e) {        // Will be caught
    echo "Caught Default Exception\n", $e;
}

// Continue execution
var_dump($o); // Null


echo "\n\n# Example 3\n";
try {
    $o = new TestException(TestException::THROW_CUSTOM);
} catch (Exception $e) {        // Will be caught
    echo "Default Exception caught\n", $e;
}

// Continue execution
var_dump($o); // Null
```

```php
echo "\n\n# Example 4\n";
try {
    $o = new TestException();
} catch (Exception $e) {        // Skipped, no exception
    echo "Default Exception caught\n", $e;
}

// Continue execution
var_dump($o); // TestException
```

Fibers

Fibers overview

(PHP 8 >= 8.1.0)

Fibers represent full-stack, interruptible functions. Fibers may be suspended from anywhere in the call-stack, pausing execution within the fiber until the fiber is resumed at a later time.

Fibers pause the entire execution stack, so the direct caller of the function does not need to change how it invokes the function.

Execution may be interrupted anywhere in the call stack using Fiber::suspend() (that is, the call to Fiber::suspend() may be in a deeply nested function or not even exist at all).

Unlike stack-less Generators, each Fiber has its own call stack, allowing them to be paused within deeply nested function calls. A function declaring an interruption point (that is, calling Fiber::suspend()) need not change its return type, unlike a function using `yield` which must return a Generator instance.

Fibers can be suspended in any function call, including those called from within the PHP VM, such as functions provided to array_map() or methods called by `foreach` on an Iterator object.

Once suspended, execution of the fiber may be resumed with any value using Fiber::resume() or by throwing an exception into the fiber using Fiber::throw(). The value is returned (or exception thrown) from Fiber::suspend().

> **Note** :
>
> Prior to PHP 8.4.0, switching fibers during the execution of an object destructor was not allowed.

Example #1 Basic usage

```php
<?php
$fiber = new Fiber(function (): void {
   $value = Fiber::suspend('fiber');
   echo "Value used to resume fiber: ", $value, PHP_EOL;
});

$value = $fiber->start();

echo "Value from fiber suspending: ", $value, PHP_EOL;
```

```php
$fiber->resume('test');
?>
```

The above example will output:

```
Value from fiber suspending: fiber
Value used to resume fiber: test
```

Generators

Table of Contents

Generators overview

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generators provide an easy way to implement simple iterators without the overhead or complexity of implementing a class that implements the Iterator interface.

A generator offers a convenient way to provide data to `foreach` loops without having to build an array in memory ahead of time, which may cause the program to exceed a memory limit or require a considerable amount of processing time to generate. Instead, a generator function can be used, which is the same as a normal function, except that instead of returning once, a generator can `yield` as many times as it needs to in order to provide the values to be iterated over. Like with iterators, random data access is not possible.

A simple example of this is to reimplement the range() function as a generator. The standard range() function has to generate an array with every value in it and return it, which can result in large arrays: for example, calling **range(0, 1000000)** will result in well over 10 MB of memory being used.

As an alternative, we can implement an `xrange()` generator, which will only ever need enough memory to create an Iterator object and track the current state of the generator internally, which turns out to be less than 1 kilobyte.

Example #1 Implementing range() as a generator

```php
<?php
function xrange($start, $limit, $step = 1) {
    if ($start <= $limit) {
        if ($step <= 0) {
            throw new LogicException('Step must be positive');
        }

        for ($i = $start; $i <= $limit; $i += $step) {
            yield $i;
        }
    } else {
        if ($step >= 0) {
            throw new LogicException('Step must be negative');
        }
```

```
        for ($i = $start; $i >= $limit; $i += $step) {
            yield $i;
        }
    }
}

/*
 * Note that both range() and xrange() result in the same
 * output below.
 */

echo 'Single digit odd numbers from range():  ';
foreach (range(1, 9, 2) as $number) {
    echo "$number ";
}
echo "\n";

echo 'Single digit odd numbers from xrange(): ';
foreach (xrange(1, 9, 2) as $number) {
    echo "$number ";
}
```

The above example will output:

```
Single digit odd numbers from range():  1 3 5 7 9
Single digit odd numbers from xrange(): 1 3 5 7 9
```

Generator objects

When a generator function is called, a new object of the internal Generator class is returned. This object implements the Iterator interface in much the same way as a forward-only iterator object would, and provides methods that can be called to manipulate the state of the generator, including sending values to and returning values from it.

Generator syntax

A generator function looks just like a normal function, except that instead of returning a value, a generator `yield`s as many values as it needs to. Any function containing `yield` is a generator function.

When a generator function is called, it returns an object that can be iterated over. When you iterate over that object (for instance, via a `foreach` loop), PHP will call the object's iteration methods each time it needs a value, then saves the state of the generator when the generator yields a value so that it can be resumed when the next value is required.

Once there are no more values to be yielded, then the generator can simply return, and the calling code continues just as if an array has run out of values.

> **Note** :

> A generator can return values, which can be retrieved using Generator::getReturn().

yield keyword

The heart of a generator function is the **yield** keyword. In its simplest form, a yield statement looks much like a return statement, except that instead of stopping execution of the function and returning, yield instead provides a value to the code looping over the generator and pauses execution of the generator function.

Example #1 A simple example of yielding values

```
<?php
function gen_one_to_three() {
    for ($i = 1; $i <= 3; $i++) {
        // Note that $i is preserved between yields.
        yield $i;
    }
}

$generator = gen_one_to_three();
foreach ($generator as $value) {
    echo "$value\n";
}
```

The above example will output:

```
1
2
3
```

> **Note** :
>
> Internally, sequential integer keys will be paired with the yielded values, just as with a non-associative array.

Yielding values with keys

PHP also supports associative arrays, and generators are no different. In addition to yielding simple values, as shown above, you can also yield a key at the same time.

The syntax for yielding a key/value pair is very similar to that used to define an associative array, as shown below.

Example #2 Yielding a key/value pair

```
<?php
/*
 * The input is semi-colon separated fields, with the first
 * field being an ID to use as a key.
```

```
 */

$input = <<<'EOF'
1;PHP;Likes dollar signs
2;Python;Likes whitespace
3;Ruby;Likes blocks
EOF;

function input_parser($input) {
    foreach (explode("\n", $input) as $line) {
        $fields = explode(';', $line);
        $id = array_shift($fields);

        yield $id => $fields;
    }
}

foreach (input_parser($input) as $id => $fields) {
    echo "$id:\n";
    echo "    $fields[0]\n";
    echo "    $fields[1]\n";
}
```

The above example will output:

```
1:
    PHP
    Likes dollar signs
2:
    Python
    Likes whitespace
3:
    Ruby
    Likes blocks
```

Yielding null values

Yield can be called without an argument to yield a `null` value with an automatic key.

Example #3 Yielding `null`s

```
<?php
function gen_three_nulls() {
    foreach (range(1, 3) as $i) {
        yield;
    }
}

var_dump(iterator_to_array(gen_three_nulls()));
```

The above example will output:

```
array(3) {
  [0]=>
  NULL
  [1]=>
  NULL
  [2]=>
  NULL
}
```

Yielding by reference

Generator functions are able to yield values by reference as well as by value. This is done in the same way as returning references from functions: by prepending an ampersand to the function name.

Example #4 Yielding values by reference

```php
<?php
function &gen_reference() {
    $value = 3;

    while ($value > 0) {
        yield $value;
    }
}

/*
 * Note that we can change $number within the loop, and
 * because the generator is yielding references, $value
 * within gen_reference() changes.
 */
foreach (gen_reference() as &$number) {
    echo (--$number).'... ';
}
```

The above example will output:

```
2... 1... 0...
```

Generator delegation via **yield from**

Generator delegation allows you to yield values from another generator, Traversable object, or array by using the **yield from** keyword. The outer generator will then yield all values from the inner generator, object, or array until that is no longer valid, after which execution will continue in the outer generator.

If a generator is used with **yield from**, the **yield from** expression will also return any value returned by the inner generator.

> **Caution**
>
> ### Storing into an array (e.g. with iterator_to_array())
>
> **yield from** does not reset the keys. It preserves the keys returned by the Traversable object, or array. Thus some values may share a common key with another **yield** or **yield from**, which, upon insertion into an array, will overwrite former values with that key.
>
> A common case where this matters is iterator_to_array() returning a keyed array by default, leading to possibly unexpected results. iterator_to_array() has a second parameter `preserve_keys` which can be set to `false` to collect all the values while ignoring the keys returned by the Generator.
>
> **Example #5 yield from with iterator_to_array()**

```php
<?php
function inner() {
    yield 1; // key 0
    yield 2; // key 1
    yield 3; // key 2
}
function gen() {
    yield 0; // key 0
    yield from inner(); // keys 0-2
    yield 4; // key 1
}
// pass false as second parameter to get an array [0, 1, 2, 3, 4]
var_dump(iterator_to_array(gen()));
```

> The above example will output:

```
array(3) {
  [0]=>
  int(1)
  [1]=>
  int(4)
  [2]=>
  int(3)
}
```

Example #6 Basic use of yield from

```php
<?php
function count_to_ten() {
    yield 1;
    yield 2;
    yield from [3, 4];
    yield from new ArrayIterator([5, 6]);
    yield from seven_eight();
    yield 9;
    yield 10;
```

```
}

function seven_eight() {
    yield 7;
    yield from eight();
}

function eight() {
    yield 8;
}

foreach (count_to_ten() as $num) {
    echo "$num ";
}
```

The above example will output:

```
1 2 3 4 5 6 7 8 9 10
```

Example #7 yield from and return values

```
<?php
function count_to_ten() {
    yield 1;
    yield 2;
    yield from [3, 4];
    yield from new ArrayIterator([5, 6]);
    yield from seven_eight();
    return yield from nine_ten();
}

function seven_eight() {
    yield 7;
    yield from eight();
}

function eight() {
    yield 8;
}

function nine_ten() {
    yield 9;
    return 10;
}

$gen = count_to_ten();
foreach ($gen as $num) {
    echo "$num ";
}
echo $gen->getReturn();
```

The above example will output:

```
1 2 3 4 5 6 7 8 9 10
```

Comparing generators with Iterator objects

The primary advantage of generators is their simplicity. Much less boilerplate code has to be written compared to implementing an Iterator class, and the code is generally much more readable. For example, the following function and class are equivalent:

```php
<?php
function getLinesFromFile($fileName) {
    if (!$fileHandle = fopen($fileName, 'r')) {
        return;
    }

    while (false !== $line = fgets($fileHandle)) {
        yield $line;
    }

    fclose($fileHandle);
}

// versus...

class LineIterator implements Iterator {
    protected $fileHandle;

    protected $line;
    protected $i;

    public function __construct($fileName) {
        if (!$this->fileHandle = fopen($fileName, 'r')) {
            throw new RuntimeException('Couldn\'t open file "' . $fileName . '"');
        }
    }

    public function rewind() {
        fseek($this->fileHandle, 0);
        $this->line = fgets($this->fileHandle);
        $this->i = 0;
    }

    public function valid() {
        return false !== $this->line;
    }

    public function current() {
        return $this->line;
    }

    public function key() {
        return $this->i;
```

```
    }

    public function next() {
        if (false !== $this->line) {
            $this->line = fgets($this->fileHandle);
            $this->i++;
        }
    }

    public function __destruct() {
        fclose($this->fileHandle);
    }
}
```

This flexibility does come at a cost, however: generators are forward-only iterators, and cannot be rewound once iteration has started. This also means that the same generator can't be iterated over multiple times: the generator will need to be rebuilt by calling the generator function again.

See Also

- Object Iteration

Attributes

Table of Contents

Attributes overview

(PHP 8)

PHP attributes provide structured, machine-readable metadata for classes, methods, functions, parameters, properties, and constants. They can be inspected at runtime via the Reflection API, enabling dynamic behavior without modifying code. Attributes provide a declarative way to annotate code with metadata.

Attributes enable the decoupling of a feature's implementation from its usage. While interfaces define structure by enforcing methods, attributes provide metadata across multiple elements, including methods, functions, properties, and constants. Unlike interfaces, which enforce method implementations, attributes annotate code without altering its structure.

Attributes can complement or replace optional interface methods by providing metadata instead of enforced structure. Consider an `ActionHandler` interface that represents an operation in an application. Some implementations may require a setup step while others do not. Instead of forcing all classes implementing `ActionHandler` to define a `setUp()` method, an attribute can indicate setup requirements. This approach increases flexibility, allowing attributes to be applied multiple times when necessary.

Example #1 Implementing optional methods of an interface with Attributes

```
<?php
interface ActionHandler
{
    public function execute();
}

#[Attribute]
```

```php
class SetUp {}

class CopyFile implements ActionHandler
{
    public string $fileName;
    public string $targetDirectory;

    #[SetUp]
    public function fileExists()
    {
        if (!file_exists($this->fileName)) {
            throw new RuntimeException("File does not exist");
        }
    }

    #[SetUp]
    public function targetDirectoryExists()
    {
        if (!file_exists($this->targetDirectory)) {
            mkdir($this->targetDirectory);
        } elseif (!is_dir($this->targetDirectory)) {
            throw new RuntimeException("Target directory $this->targetDirectory is not a
directory");
        }
    }

    public function execute()
    {
        copy($this->fileName, $this->targetDirectory . '/' . basename($this->fileName));
    }
}

function executeAction(ActionHandler $actionHandler)
{
    $reflection = new ReflectionObject($actionHandler);

    foreach ($reflection->getMethods() as $method) {
        $attributes = $method->getAttributes(SetUp::class);

        if (count($attributes) > 0) {
            $methodName = $method->getName();

            $actionHandler->$methodName();
        }
    }

    $actionHandler->execute();
}

$copyAction = new CopyFile();
$copyAction->fileName = "/tmp/foo.jpg";
$copyAction->targetDirectory = "/home/user";

executeAction($copyAction);
```

Attribute syntax

Attribute syntax consists of several key components. An attribute declaration starts with `#[` and ends with `]`. Inside, one or more attributes can be listed, separated by commas. The attribute name can be unqualified, qualified, or fully-qualified, as described in Using Namespaces Basics. Arguments to the attribute are optional and enclosed in parentheses `()`. Arguments can only be literal values or constant expressions. Both positional and named argument syntax are supported.

Attribute names and their arguments are resolved to a class, and the arguments are passed to its constructor when an instance of the attribute is requested through the Reflection API. Therefore, it is recommended to introduce a class for each attribute.

Example #1 Attribute Syntax

```php
<?php
// a.php
namespace MyExample;

use Attribute;

#[Attribute]
class MyAttribute
{
    const VALUE = 'value';

    private $value;

    public function __construct($value = null)
    {
        $this->value = $value;
    }
}

// b.php

namespace Another;

use MyExample\MyAttribute;

#[MyAttribute]
#[\MyExample\MyAttribute]
#[MyAttribute(1234)]
#[MyAttribute(value: 1234)]
#[MyAttribute(MyAttribute::VALUE)]
#[MyAttribute(array("key" => "value"))]
#[MyAttribute(100 + 200)]
class Thing
{
}

#[MyAttribute(1234), MyAttribute(5678)]
class AnotherThing
{
}
```

Reading Attributes with the Reflection API

To access attributes from classes, methods, functions, parameters, properties, and class constants, use the **getAttributes()** method provided by the Reflection API. This method returns an array of ReflectionAttribute instances. These instances can be queried for the attribute name, arguments, and can be used to instantiate an instance of the represented attribute.

Separating the reflected attribute representation from its actual instance provides more control over error handling, such as missing attribute classes, mistyped arguments, or missing values. Objects of the attribute class are instantiated only after calling ReflectionAttribute::newInstance(), ensuring that argument validation occurs at that point.

Example #1 Reading Attributes using Reflection API

```php
<?php

#[Attribute]
class MyAttribute
{
    public $value;

    public function __construct($value)
    {
        $this->value = $value;
    }
}

#[MyAttribute(value: 1234)]
class Thing
{
}

function dumpAttributeData($reflection) {
    $attributes = $reflection->getAttributes();

    foreach ($attributes as $attribute) {
       var_dump($attribute->getName());
       var_dump($attribute->getArguments());
       var_dump($attribute->newInstance());
    }
}

dumpAttributeData(new ReflectionClass(Thing::class));
```

The above example will output:

```
string(11) "MyAttribute"
array(1) {
  ["value"]=>
  int(1234)
}
object(MyAttribute)#3 (1) {
  ["value"]=>
  int(1234)
}
```

Instead of iterating over all attributes on the reflection instance, you can retrieve only those of a specific attribute class by passing the attribute class name as an argument.

Example #2 Reading Specific Attributes using Reflection API

```php
<?php
#[Attribute]
class MyAttribute
{
    public $value;

    public function __construct($value)
    {
        $this->value = $value;
    }
}

#[MyAttribute(value: 1234)]
class Thing
{
}

function dumpMyAttributeData($reflection) {
    $attributes = $reflection->getAttributes(MyAttribute::class);

    foreach ($attributes as $attribute) {
       var_dump($attribute->getName());
       var_dump($attribute->getArguments());
       var_dump($attribute->newInstance());
    }
}

dumpMyAttributeData(new ReflectionClass(Thing::class));
```

Declaring Attribute Classes

It is recommended to define a separate class for each attribute. In the simplest case, an empty class with the `#[Attribute]` declaration is sufficient. The attribute can be imported from the global namespace using a `use` statement.

Example #1 Simple Attribute Class

```php
<?php

namespace Example;

use Attribute;

#[Attribute]
class MyAttribute
{
}
```

To restrict the types of declarations an attribute can be applied to, pass a bitmask as the first argument to the `#[Attribute]` declaration.

Example #2 Using target specification to restrict where attributes can be used

```php
<?php

namespace Example;

use Attribute;

#[Attribute(Attribute::TARGET_METHOD | Attribute::TARGET_FUNCTION)]
class MyAttribute
{
}
```

Declaring **MyAttribute** on another type will now throw an exception during the call to ReflectionAttribute::newInstance()

The following targets can be specified:

- `Attribute::TARGET_CLASS`
- `Attribute::TARGET_FUNCTION`
- `Attribute::TARGET_METHOD`
- `Attribute::TARGET_PROPERTY`
- `Attribute::TARGET_CLASS_CONSTANT`
- `Attribute::TARGET_PARAMETER`
- `Attribute::TARGET_ALL`

By default, an attribute can only be used once per declaration. To allow an attribute to be repeatable, specify it in the bitmask of the `#[Attribute]` declaration using the `Attribute::IS_REPEATABLE` flag.

Example #3 Using IS_REPEATABLE to allow attribute on a declaration multiple times

```php
<?php

namespace Example;

use Attribute;

#[Attribute(Attribute::TARGET_METHOD | Attribute::TARGET_FUNCTION | Attribute::IS_REPEATABLE)]
class MyAttribute
{
}
```

References Explained

Table of Contents

What References Are

References in PHP are a means to access the same variable content by different names. They are not like C pointers; for instance, you cannot perform pointer arithmetic using them, they are not actual memory

addresses, and so on. See What References Are Not for more information. Instead, they are symbol table aliases. Note that in PHP, variable name and variable content are different, so the same content can have different names. The closest analogy is with Unix filenames and files - variable names are directory entries, while variable content is the file itself. References can be likened to hardlinking in Unix filesystem.

What References Do

There are three basic operations performed using references: assigning by reference, passing by reference, and returning by reference. This section will give an introduction to these operations, with links for further reading.

Assign By Reference

In the first of these, PHP references allow you to make two variables refer to the same content. Meaning, when you do:

```
<?php
$a =& $b;
```

it means that *$a* and *$b* point to the same content.

> **Note** :
>
> *$a* and *$b* are completely equal here. *$a* is not pointing to *$b* or vice versa. *$a* and *$b* are pointing to the same place.

> **Note** :
>
> If you assign, pass, or return an undefined variable by reference, it will get created.
>
> **Example #1 Using references with undefined variables**
>
> ```
> <?php
>
> function foo(&$var) {}
>
> foo($a); // $a is "created" and assigned to null
>
> $b = array();
> foo($b['b']);
> var_dump(array_key_exists('b', $b)); // bool(true)
>
> $c = new stdClass();
> foo($c->d);
> var_dump(property_exists($c, 'd')); // bool(true)
> ```

The same syntax can be used with functions that return references:

```
<?php
$foo =& find_var($bar);
```

Using the same syntax with a function that does *not* return by reference will give an error, as will using it with the result of the new operator. Although objects are passed around as pointers, these are not the same as references, as explained under Objects and references.

> **Warning**
>
> If you assign a reference to a variable declared `global` inside a function, the reference will be visible only inside the function. You can avoid this by using the *$GLOBALS* array.
>
> **Example #2 Referencing global variables inside functions**

```php
<?php
$var1 = "Example variable";
$var2 = "";

function global_references($use_globals)
{
    global $var1, $var2;

    if (!$use_globals) {
        $var2 =& $var1; // visible only inside the function
    } else {
        $GLOBALS["var2"] =& $var1; // visible also in global context
    }
}

global_references(false);
echo "var2 is set to '$var2'\n"; // var2 is set to ''

global_references(true);
echo "var2 is set to '$var2'\n"; // var2 is set to 'Example variable'
```

> Think about `global $var;` as a shortcut to `$var =& $GLOBALS['var'];`. Thus assigning another reference to `$var` only changes the local variable's reference.

> **Note** :
>
> If you assign a value to a variable with references in a `foreach` statement, the references are modified too.
>
> **Example #3 References and foreach statement**

```php
<?php
$ref = 0;
$row =& $ref;

foreach (array(1, 2, 3) as $row) {
    // Do something
}

echo $ref; // 3 - last element of the iterated array
```

While not being strictly an assignment by reference, expressions created with the language construct `array()` can also behave as such by prefixing `&` to the array element to add. Example:

```php
<?php
$a = 1;
$b = array(2, 3);

$arr = array(&$a, &$b[0], &$b[1]);
$arr[0]++;
$arr[1]++;
$arr[2]++;
/* $a == 2, $b == array(3, 4); */
var_dump($a);
var_dump($b);
```

Note, however, that references inside arrays are potentially dangerous. Doing a normal (not by reference) assignment with a reference on the right side does not turn the left side into a reference, but references inside arrays are preserved in these normal assignments. This also applies to function calls where the array is passed by value. Example:

```php
<?php

/* Assignment of scalar variables */
$a = 1;
$b =& $a;
$c = $b;
$c = 7; // $c is not a reference; no change to $a or $b
print "a = {$a}; b = {$b}; c = {$c}\n\n";

/* Assignment of array variables */
$arr = array(1);
$a =& $arr[0]; // $a and $arr[0] are in the same reference set
$arr2 = $arr; // Not an assignment-by-reference!
$arr2[0]++;
/* $a == 2, $arr == array(2) */
/* The contents of $arr are changed even though it's not a reference! */
print "a = {$a}\n";
var_dump($arr);
var_dump($arr2);
```

In other words, the reference behavior of arrays is defined in an element-by-element basis; the reference behavior of individual elements is dissociated from the reference status of the array container.

Pass By Reference

The second thing references do is to pass variables by reference. This is done by making a local variable in a function and a variable in the calling scope referencing the same content. Example:

```php
<?php
function foo(&$var)
{
    $var++;
}

$a = 5;
foo($a);
print $a;
```

will make *$a* to be 6. This happens because in the function *foo* the variable *$var* refers to the same content as *$a*. For more information on this, read the passing by reference section.

Return By Reference

The third thing references can do is return by reference.

What References Are Not

As said before, references are not pointers. That means, the following construct won't do what you expect:

```
<?php
function foo(&$var)
{
    $var =& $GLOBALS["baz"];
}

foo($bar);
```

What happens is that *$var* in *foo* will be bound with *$bar* in the caller, but then re-bound with *$GLOBALS["baz"]*. There's no way to bind *$bar* in the calling scope to something else using the reference mechanism, since *$bar* is not available in the function *foo* (it is represented by *$var*, but *$var* has only variable contents and not name-to-value binding in the calling symbol table). You can use returning references to reference variables selected by the function.

Passing by Reference

You can pass a variable by reference to a function so the function can modify the variable. The syntax is as follows:

```
<?php
function foo(&$var)
{
    $var++;
}

$a = 5;
foo($a);
print $a; // $a is 6 here
```

> **Note** :
>
> There is no reference sign on a function call - only on function definitions. Function definitions alone are enough to correctly pass the argument by reference.

The following things can be passed by reference:

- Variables, i.e. `foo($a)`
- References returned from functions, i.e.:

```php
<?php

function foo(&$var)
{
    $var++;
    print $var;
}

function &bar()
{
    $a = 5;
    return $a;
}

foo(bar());
```

See more about returning by reference.

No other expressions should be passed by reference, as the result is undefined. For example, the following examples of passing by reference are invalid:

```php
<?php

function foo(&$var)
{
    $var++;
}

function bar() // Note the missing &
{
    $a = 5;
    return $a;
}

foo(bar()); // Produces a notice

foo($a = 5); // Expression, not variable
foo(5); // Produces fatal error

class Foobar {}

foo(new Foobar()) // Produces a notice as of PHP 7.0.7
                  // Notice: Only variables should be passed by reference
```

Returning References

Returning by reference is useful when you want to use a function to find to which variable a reference should be bound. Do *not* use return-by-reference to increase performance. The engine will automatically optimize this on its own. Only return references when you have a valid technical reason to do so. To return references, use this syntax:

```php
<?php

class Foo
{
    public $value = 42;
```

```
    public function &getValue()
    {
        return $this->value;
    }
}

$obj = new Foo();
$myValue = &$obj->getValue(); // $myValue is a reference to $obj->value, which is 42
$obj->value = 2;
echo $myValue;                // Prints the new value of $obj->value, i.e. 2
```

In this example, the property of the object returned by the *getValue* function would be set, not the copy, as it would be without using reference syntax.

> **Note** :
>
> Unlike parameter passing, here you have to use `&` in both places - to indicate that you want to return by reference, not a copy, and to indicate that reference binding, rather than usual assignment, should be done for *$myValue*.

> **Note** :
>
> If you try to return a reference from a function with the syntax: `return ($this->value);` this will *not* work as you are attempting to return the result of an *expression*, and not a variable, by reference. You can only return variables by reference from a function - nothing else.

To use the returned reference, you must use reference assignment:

```
<?php

function &collector()
{
    static $collection = array();
    return $collection;
}

$collection = &collector();
// Now the $collection is a referenced variable that references the static array inside the function

$collection[] = 'foo';

print_r(collector());
```

The above example will output:

```
Array
(
    [0] => foo
)
```

> **Note** :
>
> If the assignment is done without the `&` symbol, e.g. `$collection = collector();`, the *$collection* variable will receive a copy of the value, not the reference returned by the function.

To pass the returned reference to another function expecting a reference you can use this syntax:

```
<?php

function &collector()
{
    static $collection = array();
    return $collection;
}

array_push(collector(), 'foo');
```

> **Note** :
>
> Note that `array_push(&collector(), 'foo');` will *not* work, it results in a fatal error.

Unsetting References

When you unset the reference, you just break the binding between variable name and variable content. This does not mean that variable content will be destroyed. For example:

```
<?php

$a = 1;
$b =& $a;
unset($a);
var_dump($a);
var_dump($b);
```

won't unset *$b*, just *$a*.

Again, it might be useful to think about this as analogous to the Unix **unlink** call.

Spotting References

Many syntax constructs in PHP are implemented via referencing mechanisms, so everything mentioned herein about reference binding also applies to these constructs. Some constructs, like passing and returning by reference, are mentioned above. Other constructs that use references are:

global References

When you declare a variable as **global $var** you are in fact creating reference to a global variable. That means, this is the same as:

```
<?php
```

```php
$var =& $GLOBALS["var"];
```

This also means that unsetting *$var* won't unset the global variable.

Predefined Variables

PHP provides predefined variables that represent external variables, built-in environment variables, and other information about the execution environment, such as the number and values of the arguments passed to the script in the CLI environment.

Table of Contents

Superglobals

Superglobals

Superglobals — Built-in variables that are always available in all scopes

Description

Several predefined variables in PHP are "superglobals", which means they are available in all scopes throughout a script. There is no need to do **global $variable;** to access them within functions or methods.

These superglobal variables are:

- *$GLOBALS*
- *$_SERVER*
- *$_GET*
- *$_POST*
- *$_FILES*
- *$_COOKIE*
- *$_SESSION*
- *$_REQUEST*
- *$_ENV*

Notes

> **Note** : **Variable availability**
>
> By default, all of the superglobals are available but there are directives that affect this availability. For further information, refer to the documentation for variables_order.

> **Note** : **Variable variables**
>
> Superglobals cannot be used as variable variables inside functions or class methods.

See Also

- variable scope
- The variables_order directive

- The filter extension

$GLOBALS

$GLOBALS

(PHP 4, PHP 5, PHP 7, PHP 8)

$GLOBALS — References all variables available in global scope

Description

An associative array containing references to all variables which are currently defined in the global scope of the script. The variable names are the keys of the array.

Examples

Example #1 *$GLOBALS* example

```php
<?php

function test()
{
    $foo = "local variable";

    echo '$foo in global scope: ' . $GLOBALS["foo"] . "\n";
    echo '$foo in current scope: ' . $foo . "\n";
}

$foo = "Example content";
test();

?>
```

The above example will output something similar to:

```
$foo in global scope: Example content
$foo in current scope: local variable
```

> **Warning**
>
> As of PHP 8.1.0, write access to the entire *$GLOBALS* array is no longer supported:
>
> **Example #2 writing entire *$GLOBALS* will result in error.**
>
> ```php
> <?php
>
> // Generates compile-time error:
> $GLOBALS = [];
> $GLOBALS += [];
> ```

```php
$GLOBALS =& $x;
$x =& $GLOBALS;
unset($GLOBALS);
array_pop($GLOBALS);
// ...and any other write/read-write operation on $GLOBALS

?>
```

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

> **Note** : **Variable availability**
>
> Unlike all of the other superglobals, *$GLOBALS* has essentially always been available in PHP.

> **Note** :
>
> As of PHP 8.1.0, *$GLOBALS* is now a read-only copy of the global symbol table. That is, global variables cannot be modified via its copy. Previously, *$GLOBALS* array is excluded from the usual by-value behavior of PHP arrays and global variables can be modified via its copy.

```php
<?php

// Before PHP 8.1.0
$a = 1;

$globals = $GLOBALS; // Ostensibly by-value copy
$globals['a'] = 2;
var_dump($a); // int(2)

// As of PHP 8.1.0
// this no longer modifies $a. The previous behavior violated by-value semantics.
$globals = $GLOBALS;
$globals['a'] = 1;

// To restore the previous behavior, iterate its copy and assign each property back to $GLOBALS.
foreach ($globals as $key => $value) {
    $GLOBALS[$key] = $value;
}

?>
```

$_SERVER

$_SERVER

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_SERVER — Server and execution environment information

Description

$_SERVER is an array containing information such as headers, paths, and script locations. The entries in this array are created by the web server, therefore there is no guarantee that every web server will provide any of these; servers may omit some, or provide others not listed here. However, most of these variables are accounted for in the » CGI/1.1 specification, and are likely to be defined.

> **Note** :
>
> When running PHP on the command line most of these entries will not be available or have any meaning.

In addition to the elements listed below, PHP will create additional elements with values from request headers. These entries will be named `HTTP_` followed by the header name, capitalized and with underscores instead of hyphens. For example, the `Accept-Language` header would be available as `$_SERVER['HTTP_ACCEPT_LANGUAGE']`.

Indices

'*PHP_SELF*'

The filename of the currently executing script, relative to the document root. For instance, *$_SERVER['PHP_SELF']* in a script at the address *http://example.com/foo/bar.php* would be */foo/bar.php*. The __FILE__ constant contains the full path and filename of the current (i.e. included) file.

If PHP is running as a command-line processor this variable contains the script name.

'argv'

Array of arguments passed to the script. When the script is run on the command line, this gives C-style access to the command line parameters. When called via the GET method, this will contain the query string.

'argc'

Contains the number of command line parameters passed to the script (if run on the command line).

'*GATEWAY_INTERFACE*'

What revision of the CGI specification the server is using; e.g. `'CGI/1.1'`.

'*SERVER_ADDR*'

The IP address of the server under which the current script is executing.

'*SERVER_NAME*'

The name of the server host under which the current script is executing. If the script is running on a virtual host, this will be the value defined for that virtual host.

> **Note** :

> Under Apache 2, `UseCanonicalName = On` and `ServerName` must be set. Otherwise, this value reflects the hostname supplied by the client, which can be spoofed. It is not safe to rely on this value in security-dependent contexts.

'*SERVER_SOFTWARE*'

Server identification string, given in the headers when responding to requests.

'*SERVER_PROTOCOL*'

Name and revision of the information protocol via which the page was requested; e.g. `'HTTP/1.0'`;

'*REQUEST_METHOD*'

Which request method was used to access the page; e.g. `'GET'`, `'HEAD'`, `'POST'`, `'PUT'`.

> **Note** :
>
> PHP script is terminated after sending headers (it means after producing any output without output buffering) if the request method was `HEAD`.

'*REQUEST_TIME*'

The timestamp when PHP started processing the request.

'*REQUEST_TIME_FLOAT*'

The timestamp when PHP started processing the request, with microsecond precision.

'*QUERY_STRING*'

The query string, if any, via which the page was accessed.

'*DOCUMENT_ROOT*'

The document root directory under which the current script is executing, as defined in the server's configuration file.

'*HTTPS*'

Set to a non-empty value if the script was queried through the HTTPS protocol.

'*REMOTE_ADDR*'

The IP address from which the user is viewing the current page.

'*REMOTE_HOST*'

The Host name from which the user is viewing the current page. The reverse dns lookup is based on the *REMOTE_ADDR* of the user.

> **Note** :

> The web server must be configured to create this variable. For example in Apache `HostnameLookups On` must be set inside *httpd.conf* for it to exist. See also gethostbyaddr().

'*REMOTE_PORT*'

The port being used on the user's machine to communicate with the web server.

'*REMOTE_USER*'

The authenticated user.

'*REDIRECT_REMOTE_USER*'

The authenticated user if the request is internally redirected.

'*SCRIPT_FILENAME*'

The absolute pathname of the currently executing script.

> **Note** :
>
> If a script is executed with the CLI, as a relative path, such as *file.php* or *../file.php*, *$_SERVER['SCRIPT_FILENAME']* will contain the relative path specified by the user.

'*SERVER_ADMIN*'

The value given to the SERVER_ADMIN (for Apache) directive in the web server configuration file. If the script is running on a virtual host, this will be the value defined for that virtual host.

'*SERVER_PORT*'

The port on the server machine being used by the web server for communication. For default setups, this will be `'80'`; using SSL, for instance, will change this to whatever your defined secure HTTP port is.

> **Note** :
>
> Under Apache 2, `UseCanonicalName = On`, as well as `UseCanonicalPhysicalPort = On` must be set in order to get the physical (real) port, otherwise, this value can be spoofed, and it may or may not return the physical port value. It is not safe to rely on this value in security-dependent contexts.

'*SERVER_SIGNATURE*'

String containing the server version and virtual host name which are added to server-generated pages, if enabled.

'*PATH_TRANSLATED*'

Filesystem- (not document root-) based path to the current script, after the server has done any virtual-to-real mapping.

> **Note** :

> Apache 2 users may use `AcceptPathInfo = On` inside *httpd.conf* to define *PATH_INFO*.

'*SCRIPT_NAME*'

Contains the current script's path. This is useful for pages which need to point to themselves. The __FILE__ constant contains the full path and filename of the current (i.e. included) file.

'*REQUEST_URI*'

The URI which was given in order to access this page; for instance, '`/index.html`'.

'*PHP_AUTH_DIGEST*'

When doing Digest HTTP authentication this variable is set to the 'Authorization' header sent by the client (which you should then use to make the appropriate validation).

'*PHP_AUTH_USER*'

When doing HTTP authentication this variable is set to the username provided by the user.

'*PHP_AUTH_PW*'

When doing HTTP authentication this variable is set to the password provided by the user.

'*AUTH_TYPE*'

When doing HTTP authentication this variable is set to the authentication type.

'*PATH_INFO*'

Contains any client-provided pathname information trailing the actual script filename but preceding the query string, if available. For instance, if the current script was accessed via the URI *http://www.example.com/php/path_info.php/some/stuff?foo=bar*, then *$_SERVER['PATH_INFO']* would contain `/some/stuff`.

'*ORIG_PATH_INFO*'

Original version of '*PATH_INFO*' before processed by PHP.

Examples

Example #1 *$_SERVER* example

```php
<?php
echo $_SERVER['SERVER_NAME'];
?>
```

The above example will output something similar to:

```
www.example.com
```

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

See Also

- The filter extension

$_GET

$_GET

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_GET — Query string variables

Description

An associative array of variables passed to the current script via the URL parameters (also known as the query string). Note that this array is populated whenever a query string is present, regardless of the HTTP request method.

Examples

Example #1 *$_GET* example

```
<?php
echo 'Hello ' . htmlspecialchars($_GET["name"]) . '!';
?>
```

Assuming the user entered `http://example.com/?name=Hannes`.

The above example will output something similar to:

```
Hello Hannes!
```

Notes

> **Note** :

> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

> **Note** :
>
> The values in *$_GET* are automatically passed through urldecode().

See Also

- Handling external variables
- The filter extension

$_POST

$_POST

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_POST — Form data from HTTP POST requests

Description

An associative array of variables passed to the current script via the HTTP POST method when using `application/x-www-form-urlencoded` or `multipart/form-data` as the HTTP Content-Type in the request.

Examples

Example #1 *$_POST* example

```php
<?php
echo 'Hello ' . htmlspecialchars($_POST["name"]) . '!';
?>
```

Assuming the user sent a POST request with `name=Hannes` in the body.

The above example will output something similar to:

```
Hello Hannes!
```

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

> **Note** :
>
> To read POST data sent with other content types (e.g. `application/json` or `application/xml`) *php://input* must be used. Unlike *$_POST*, which only works with `application/x-www-form-urlencoded` and `multipart/form-data`, *php://input* provides direct access to the raw data from the body of the request.

See Also

- Handling external variables
- The filter extension

$_FILES

$_FILES

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_FILES — HTTP File Upload variables

Description

An associative array of items uploaded to the current script via the HTTP POST method. The structure of this array is outlined in the POST method uploads section.

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

See Also

- move_uploaded_file() - Moves an uploaded file to a new location
- Handling File Uploads

$_REQUEST

$_REQUEST

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_REQUEST — HTTP Request variables

Description

An associative array that by default contains the contents of *$_GET*, *$_POST* and *$_COOKIE*.

Notes

> **Note** :

> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

> **Note** :
>
> When running on the command line , this will *not* include the argv and argc entries; these are present in the *$_SERVER* array.

> **Note** :
>
> The variables in *$_REQUEST* are provided to the script via the GET, POST, and COOKIE input mechanisms and therefore could be modified by the remote user and cannot be trusted. The presence and order of variables listed in this array is defined according to the PHP request_order, and variables_order configuration directives.

See Also

- Handling external variables
- The filter extension

$_SESSION

$_SESSION

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_SESSION — Session variables

Description

An associative array containing session variables available to the current script. See the Session functions documentation for more information on how this is used.

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

See Also

- session_start() - Start new or resume existing session

$_ENV

$_ENV

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_ENV — Environment variables

Description

An associative array of variables passed to the current script via the environment method.

These variables are imported into PHP's global namespace from the environment under which the PHP parser is running. Many are provided by the shell under which PHP is running and different systems are likely running different kinds of shells, a definitive list is impossible. Please see your shell's documentation for a list of defined environment variables.

Other environment variables include the CGI variables, placed there regardless of whether PHP is running as a server module or CGI processor.

Examples

Example #1 *$_ENV* example

```php
<?php
echo 'My username is ' .$_ENV["USER"] . '!';
?>
```

Assuming "bjori" executes this script

The above example will output something similar to:

```
My username is bjori!
```

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

See Also

- getenv() - Gets the value of a single or all environment variables
- The filter extension

$_COOKIE

$_COOKIE

(PHP 4 >= 4.1.0, PHP 5, PHP 7, PHP 8)

$_COOKIE — HTTP Cookies

Description

An associative array of variables passed to the current script via HTTP Cookies.

Examples

Example #1 *$_COOKIE* example

```php
<?php
echo 'Hello ' . htmlspecialchars($_COOKIE["name"]) . '!';
?>
```

Assuming the "name" cookie has been set earlier

The above example will output something similar to:

```
Hello Hannes!
```

Notes

> **Note** :
>
> This is a 'superglobal', or automatic global, variable. This simply means that it is available in all scopes throughout a script. There is no need to do **global $variable;** to access it within functions or methods.

See Also

- setcookie() - Send a cookie
- Handling external variables
- The filter extension

$php_errormsg

$php_errormsg

(PHP 4, PHP 5, PHP 7)

$php_errormsg — The previous error message

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 7.2.0, and *REMOVED* as of PHP 8.0.0. Relying on this feature is highly discouraged.

Use error_get_last() instead.

Description

$php_errormsg is a variable containing the text of the last error message generated by PHP. This variable will only be available within the scope in which the error occurred, and only if the track_errors configuration option is turned on (it defaults to off).

> **Warning**
>
> If a user defined error handler (set_error_handler()) is set *$php_errormsg* is only set if the error handler returns `false`.

Changelog

Version	Description
8.0.0	Directive track_errors which caused *$php_errormsg* to be available has been removed.
7.2.0	Directive track_errors which caused *$php_errormsg* to be available has been deprecated.

Examples

Example #1 *$php_errormsg* example

```
<?php
@strpos();
echo $php_errormsg;
?>
```

The above example will output something similar to:

```
Wrong parameter count for strpos()
```

See Also

- error_get_last() - Get the last occurred error

$http_response_header

$http_response_header

(PHP 4 >= 4.0.4, PHP 5, PHP 7, PHP 8)

$http_response_header — HTTP response headers

> **Warning**
>
> This feature has been *DEPRECATED* as of PHP 8.5.0. Relying on this feature is highly discouraged.

Use http_get_last_response_headers() instead.

Description

The *$http_response_header* array is similar to the get_headers() function. When using the HTTP wrapper, *$http_response_header* will be populated with the HTTP response headers. *$http_response_header* will be created in the local scope.

Examples

Example #1 *$http_response_header* example

```php
<?php
function get_contents() {
  file_get_contents("http://example.com");
  var_dump($http_response_header); // variable is populated in the local scope
}
get_contents();
var_dump($http_response_header); // a call to get_contents() does not populate the variable outside
the function scope
?>
```

The above example will output something similar to:

```
array(9) {
  [0]=>
  string(15) "HTTP/1.1 200 OK"
  [1]=>
  string(35) "Date: Sat, 12 Apr 2008 17:30:38 GMT"
  [2]=>
  string(29) "Server: Apache/2.2.3 (CentOS)"
  [3]=>
  string(44) "Last-Modified: Tue, 15 Nov 2005 13:24:10 GMT"
  [4]=>
  string(27) "ETag: "280100-1b6-80bfd280""
  [5]=>
  string(20) "Accept-Ranges: bytes"
  [6]=>
  string(19) "Content-Length: 438"
  [7]=>
  string(17) "Connection: close"
  [8]=>
  string(38) "Content-Type: text/html; charset=UTF-8"
}

Warning: Undefined variable $http_response_header
NULL
```

See Also

- http_get_last_response_headers() - Retrieve last HTTP response headers
- http_clear_last_response_headers() - Clears the stored HTTP response headers

$argc

$argc

(PHP 4, PHP 5, PHP 7, PHP 8)

$argc — The number of arguments passed to script

Description

Contains the number of arguments passed to the current script when running from the command line.

> **Note** :
>
> The script's filename is always passed as an argument to the script, therefore the minimum value of *$argc* is 1.

> **Note** :
>
> This variable is not available when register_argc_argv is disabled.

Examples

Example #1 *$argc* example

```
<?php
var_dump($argc);
?>
```

When executing the example with: php script.php arg1 arg2 arg3

The above example will output something similar to:

```
int(4)
```

Notes

> **Note** :
>
> This is also available as *$_SERVER['argc']*.

See Also

- getopt() - Gets options from the command line argument list
- $argv

$argv

$argv

(PHP 4, PHP 5, PHP 7, PHP 8)

$argv — Array of arguments passed to script

Description

Contains an array of all the arguments passed to the script when running from the command line.

> **Note** :
>
> The first argument *$argv[0]* is always the name that was used to run the script.

> **Note** :
>
> This variable is not available when register_argc_argv is disabled.

> **Warning**
>
> To test if a script is being run from the command line, php_sapi_name() should be used instead of checking whether *$argv* or *$_SERVER['argv']* is set.

Examples

Example #1 *$argv* example

```
<?php
var_dump($argv);
?>
```

When executing the example with: php script.php arg1 arg2 arg3

The above example will output something similar to:

```
array(4) {
  [0]=>
  string(10) "script.php"
  [1]=>
  string(4) "arg1"
  [2]=>
  string(4) "arg2"
```

```
  [3]=>
  string(4) "arg3"
}
```

Notes

> **Note** :
>
> This is also available as *$_SERVER['argv']*.

See Also

- getopt() - Gets options from the command line argument list
- $argc

Predefined Exceptions

Table of Contents

See also the SPL Exceptions.

Exception

(PHP 5, PHP 7, PHP 8)

Introduction

Exception is the base class for all user exceptions.

Class synopsis

```
class Exception implements Throwable {
/* Properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Methods */
public function __construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function getMessage(): string
final public function getPrevious(): ?Throwable
final public function getCode(): int
final public function getFile(): string
final public function getLine(): int
final public function getTrace(): array
```

final public function getTraceAsString(): string
public function __toString(): string
private function __clone(): void
}

Properties

message

The exception message

code

The exception code

file

The filename where the exception was created

line

The line where the exception was created

previous

The previously thrown exception

string

The string representation of the stack trace

trace

The stack trace as an array

Table of Contents

Exception::__construct

Exception::__construct

(PHP 5, PHP 7, PHP 8)

Exception::__construct — Construct the exception

Description

public function **Exception::__construct**(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)

Constructs the Exception.

Parameters

`message`

The Exception message to throw.

`code`

The Exception code.

`previous`

The previous exception used for the exception chaining.

> **Note** :
>
> Calling the constructor of class Exception from a subclass ignores the default arguments, if the properties $code and $message are already set.

Notes

> **Note** :
>
> The `message` is *NOT* binary safe.

Exception::getMessage

Exception::getMessage

(PHP 5, PHP 7, PHP 8)

Exception::getMessage — Gets the Exception message

Description

final public function **Exception::getMessage**(): string

Returns the Exception message.

Parameters

This function has no parameters.

Return Values

Returns the Exception message as a string.

Examples

Example #1 **Exception::getMessage()** example

```
<?php
try {
    throw new Exception("Some error message");
} catch(Exception $e) {
    echo $e->getMessage();
}
?>
```

The above example will output something similar to:

```
Some error message
```

See Also

- Throwable::getMessage() - Gets the message

Exception::getPrevious

Exception::getPrevious

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Exception::getPrevious — Returns previous Throwable

Description

final public function **Exception::getPrevious**(): ?Throwable

Returns previous Throwable (which had been passed as the third parameter of Exception::__construct()).

Parameters

This function has no parameters.

Return Values

Returns the previous Throwable if available or `null` otherwise.

Examples

Example #1 Exception::getPrevious() example

Looping over, and printing out, exception trace.

```php
<?php
class MyCustomException extends Exception {}

function doStuff() {
    try {
        throw new InvalidArgumentException("You are doing it wrong!", 112);
    } catch(Exception $e) {
        throw new MyCustomException("Something happened", 911, $e);
    }
}


try {
    doStuff();
} catch(Exception $e) {
```

```
    do {
        printf("%s:%d %s (%d) [%s]\n", $e->getFile(), $e->getLine(), $e->getMessage(), $e-
>getCode(), get_class($e));
    } while($e = $e->getPrevious());
}
?>
```

The above example will output something similar to:

```
/home/bjori/ex.php:8 Something happened (911) [MyCustomException]
/home/bjori/ex.php:6 You are doing it wrong! (112) [InvalidArgumentException]
```

See Also

- Throwable::getPrevious() - Returns the previous Throwable

Exception::getCode

Exception::getCode

(PHP 5, PHP 7, PHP 8)

Exception::getCode — Gets the Exception code

Description

final public function **Exception::getCode**(): int

Returns the Exception code.

Parameters

This function has no parameters.

Return Values

Returns the exception code as int in Exception but possibly as other type in Exception descendants (for example as string in PDOException).

Examples

Example #1 **Exception::getCode()** example

```
<?php
try {
    throw new Exception("Some error message", 30);
} catch(Exception $e) {
    echo "The exception code is: " . $e->getCode();
}
?>
```

The above example will output something similar to:

```
The exception code is: 30
```

See Also

- Throwable::getCode() - Gets the exception code

Exception::getFile

Exception::getFile

(PHP 5, PHP 7, PHP 8)

Exception::getFile — Gets the file in which the exception was created

Description

final public function **Exception::getFile**(): string

Get the name of the file in which the exception was created.

Parameters

This function has no parameters.

Return Values

Returns the filename in which the exception was created.

Examples

Example #1 Exception::getFile() example

```php
<?php
try {
    throw new Exception;
} catch(Exception $e) {
    echo $e->getFile();
}
?>
```

The above example will output something similar to:

```
/home/bjori/tmp/ex.php
```

See Also

- Throwable::getFile() - Gets the file in which the object was created

Exception::getLine

Exception::getLine

(PHP 5, PHP 7, PHP 8)

Exception::getLine — Gets the line in which the exception was created

Description

final public function **Exception::getLine**(): int

Get line number where the exception was created.

Parameters

This function has no parameters.

Return Values

Returns the line number where the exception was created.

Examples

Example #1 Exception::getLine() example

```php
<?php
try {
    throw new Exception("Some error message");
} catch(Exception $e) {
    echo "The exception was created on line: " . $e->getLine();
}
?>
```

The above example will output something similar to:

```
The exception was created on line: 3
```

See Also

- Throwable::getLine() - Gets the line on which the object was instantiated

Exception::getTrace

Exception::getTrace

(PHP 5, PHP 7, PHP 8)

Exception::getTrace — Gets the stack trace

Description

final public function **Exception::getTrace**(): array

Returns the Exception stack trace.

Parameters

This function has no parameters.

Return Values

Returns the Exception stack trace as an array.

Examples

Example #1 Exception::getTrace() example

```php
<?php
function test() {
 throw new Exception;
}

try {
 test();
} catch(Exception $e) {
 var_dump($e->getTrace());
}
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["file"]=>
    string(22) "/home/bjori/tmp/ex.php"
    ["line"]=>
    int(7)
    ["function"]=>
    string(4) "test"
    ["args"]=>
    array(0) {
    }
  }
}
```

See Also

- Throwable::getTrace() - Gets the stack trace

Exception::getTraceAsString

Exception::getTraceAsString

(PHP 5, PHP 7, PHP 8)

Exception::getTraceAsString — Gets the stack trace as a string

Description

final public function **Exception::getTraceAsString**(): string

Returns the Exception stack trace as a string.

Parameters

This function has no parameters.

Return Values

Returns the Exception stack trace as a string.

Examples

Example #1 **Exception::getTraceAsString()** example

```php
<?php
function test() {
    throw new Exception;
}

try {
    test();
} catch(Exception $e) {
    echo $e->getTraceAsString();
}
?>
```

The above example will output something similar to:

```
#0 /home/bjori/tmp/ex.php(7): test()
#1 {main}
```

See Also

- Throwable::getTraceAsString() - Gets the stack trace as a string

Exception::__toString

Exception::__toString

(PHP 5, PHP 7, PHP 8)

Exception::__toString — String representation of the exception

Description

public function **Exception::__toString**(): string

Returns the string representation of the exception.

Parameters

This function has no parameters.

Return Values

Returns the string representation of the exception.

Examples

Example #1 **Exception::__toString()** example

```
<?php
try {
    throw new Exception("Some error message");
} catch(Exception $e) {
    echo $e;
}
?>
```

The above example will output something similar to:

```
exception 'Exception' with message 'Some error message' in /home/bjori/tmp/ex.php:3
Stack trace:
#0 {main}
```

See Also

- Throwable::__toString() - Gets a string representation of the thrown object

Exception::__clone

Exception::__clone

(PHP 5, PHP 7, PHP 8)

Exception::__clone — Clone the exception

Description

private function **Exception::__clone**(): void

Exceptions do not support cloning; this method exists only to prevent it.

> **Warning**
>
> Exceptions cannot be cloned, whether they are instances of Exception or of a subclass. Attempting to clone an exception throws an Error, which may be caught.

Parameters

This function has no parameters.

Return Values

No value is returned.

Errors/Exceptions

Exceptions are *not* clonable.

Changelog

Version	Description
8.1.0	**Exception::__clone()** is no longer declared final, because declaring a private method final emits a warning as of PHP 8.0.0. Exceptions remain uncloneable.

ErrorException

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

An Error Exception.

Class synopsis

```
class ErrorException extends Exception {
/* Properties */
protected int $severity = E_ERROR;
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
```

```
/* Methods */
public function __construct(
    string $message = "",
    int $code = 0,
    int $severity = E_ERROR,
    ?string $filename = null,
    ?int $line = null,
    ?Throwable $previous = null
)
final public function getSeverity(): int
/* Inherited methods */
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}
```

Properties

severity

The severity of the exception

Examples

Example #1 Use set_error_handler() to change error messages into ErrorException

```php
<?php

set_error_handler(function (int $errno, string $errstr, string $errfile, int $errline) {
    if (!(error_reporting() & $errno)) {
        // This error code is not included in error_reporting.
        return;
    }

    if ($errno === E_DEPRECATED || $errno === E_USER_DEPRECATED) {
        // Do not throw an Exception for deprecation warnings as new or unexpected
        // deprecations would break the application.
        return;
    }

    throw new \ErrorException($errstr, 0, $errno, $errfile, $errline);
});

// Unserializing broken data triggers a warning which will be turned into an
// ErrorException by the error handler.
unserialize('broken data');
```

```
?>
```

The above example will output something similar to:

```
Fatal error: Uncaught ErrorException: unserialize(): Error at offset 0 of 11 bytes in test.php:16
Stack trace:
#0 [internal function]: {closure}(2, 'unserialize(): ...', 'test.php', 16)
#1 test.php(16): unserialize('broken data')
#2 {main}
  thrown in test.php on line 16
```

Table of Contents

ErrorException::__construct

ErrorException::__construct

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ErrorException::__construct — Constructs the exception

Description

public function **ErrorException::__construct**(
    string $message = "",
    int $code = 0,
    int $severity = E_ERROR,
    ?string $filename = null,
    ?int $line = null,
    ?Throwable $previous = null
)

Constructs the Exception.

Parameters

`message`

The Exception message to throw.

`code`

The Exception code.

`severity`

The severity level of the exception.

> **Note** :

> While the severity can be any int value, it is intended that the error constants be used.

`filename`

The filename where the exception is thrown.

`line`

The line number where the exception is thrown.

`previous`

The previous exception used for the exception chaining.

Changelog

Version	Description
8.0.0	`filename` and `line` are nullable now. Previously, their defaults were `__FILE__` and `__LINE__`, respectively.

ErrorException::getSeverity

ErrorException::getSeverity

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

ErrorException::getSeverity — Gets the exception severity

Description

final public function **ErrorException::getSeverity**(): int

Returns the severity of the exception.

Parameters

This function has no parameters.

Return Values

Returns the severity level of the exception.

Examples

Example #1 ErrorException::getSeverity() example

```php
<?php
try {
    throw new ErrorException("Exception message", 0, E_USER_ERROR);
} catch(ErrorException $e) {
    echo "This exception severity is: " . $e->getSeverity();
    var_dump($e->getSeverity() === E_USER_ERROR);
}
?>
```

The above example will output something similar to:

```
This exception severity is: 256
bool(true)
```

ClosedGeneratorException

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introduction

A **ClosedGeneratorException** is thrown when trying to retrieve a value from a closed Generator.

Class synopsis

class **ClosedGeneratorException** extends Exception {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Exception::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Error

(PHP 7, PHP 8)

Introduction

Error is the base class for all internal PHP errors.

Class synopsis

class **Error** implements Throwable {
/* Properties */

```
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Methods */
public function __construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function getMessage(): string
final public function getPrevious(): ?Throwable
final public function getCode(): int
final public function getFile(): string
final public function getLine(): int
final public function getTrace(): array
final public function getTraceAsString(): string
public function __toString(): string
private function __clone(): void
}
```

Properties

message

: The error message

code

: The error code

file

: The filename where the error happened

line

: The line where the error happened

previous

: The previously thrown exception

string

: The string representation of the stack trace

trace

: The stack trace as an array

Table of Contents

Error::__construct

Error::__construct

(PHP 7, PHP 8)

Error::__construct — Construct the error object

Description

public function **Error::__construct**(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = `null`)

Constructs the Error.

Parameters

`message`

The error message.

`code`

The error code.

`previous`

The previous throwable used for the exception chaining.

Notes

> **Note** :
>
> The `message` is *NOT* binary safe.

Error::getMessage

Error::getMessage

(PHP 7, PHP 8)

Error::getMessage — Gets the error message

Description

final public function **Error::getMessage**(): string

Returns the error message.

Parameters

This function has no parameters.

Return Values

Returns the error message as a string.

Examples

Example #1 **Error::getMessage()** example

```php
<?php
try {
    throw new Error("Some error message");
} catch(Error $e) {
    echo $e->getMessage();
}
?>
```

The above example will output something similar to:

```
Some error message
```

See Also

- Throwable::getMessage() - Gets the message

Error::getPrevious

Error::getPrevious

(PHP 7, PHP 8)

Error::getPrevious — Returns previous Throwable

Description

final public function **Error::getPrevious**(): ?Throwable

Returns previous Throwable (the third parameter of Error::__construct()).

Parameters

This function has no parameters.

Return Values

Returns the previous Throwable if available or `null` otherwise.

Examples

Example #1 **Error::getPrevious()** example

Looping over, and printing out, error trace.

```php
<?php
class MyCustomError extends Error {}

function doStuff() {
```

```
    try {
        throw new TypeError("You are doing it wrong!", 112);
    } catch(Error $e) {
        throw new MyCustomError("Something happened", 911, $e);
    }
}


try {
    doStuff();
} catch(Error $e) {
    do {
        printf("%s:%d %s (%d) [%s]\n", $e->getFile(), $e->getLine(), $e->getMessage(), $e-
>getCode(), get_class($e));
    } while($e = $e->getPrevious());
}
?>
```

The above example will output something similar to:

```
/home/bjori/ex.php:8 Something happened (911) [MyCustomError]
/home/bjori/ex.php:6 You are doing it wrong! (112) [TypeError]
```

See Also

- Throwable::getPrevious() - Returns the previous Throwable

Error::getCode

Error::getCode

(PHP 7, PHP 8)

Error::getCode — Gets the error code

Description

final public function **Error::getCode**(): int

Returns the error code.

Parameters

This function has no parameters.

Return Values

Returns the error code as int

Examples

Example #1 **Error::getCode()** example

```php
<?php
try {
    throw new Error("Some error message", 30);
} catch(Error $e) {
    echo "The Error code is: " . $e->getCode();
}
?>
```

The above example will output something similar to:

```
The Error code is: 30
```

See Also

- Throwable::getCode() - Gets the exception code

Error::getFile

Error::getFile

(PHP 7, PHP 8)

Error::getFile — Gets the file in which the error occurred

Description

final public function **Error::getFile**(): string

Get the name of the file the error occurred.

Parameters

This function has no parameters.

Return Values

Returns the filename in which the error occurred.

Examples

Example #1 **Error::getFile()** example

```php
<?php
try {
    throw new Error;
} catch(Error $e) {
    echo $e->getFile();
```

```
}
?>
```

The above example will output something similar to:

```
/home/bjori/tmp/ex.php
```

See Also

- Throwable::getFile() - Gets the file in which the object was created

Error::getLine

Error::getLine

(PHP 7, PHP 8)

Error::getLine — Gets the line in which the error occurred

Description

final public function **Error::getLine**(): int

Get line number where the error occurred.

Parameters

This function has no parameters.

Return Values

Returns the line number where the error occurred.

Examples

Example #1 Error::getLine() example

```
<?php
try {
    throw new Error("Some error message");
} catch(Error $e) {
    echo "The error was created on line: " . $e->getLine();
}
?>
```

The above example will output something similar to:

```
The error was created on line: 3
```

See Also

- Throwable::getLine() - Gets the line on which the object was instantiated

Error::getTrace

Error::getTrace

(PHP 7, PHP 8)

Error::getTrace — Gets the stack trace

Description

final public function **Error::getTrace**(): array

Returns the stack trace.

Parameters

This function has no parameters.

Return Values

Returns the stack trace as an array.

Examples

Example #1 Error::getTrace() example

```
<?php
function test() {
 throw new Error;
}

try {
 test();
} catch(Error $e) {
 var_dump($e->getTrace());
}
?>
```

The above example will output something similar to:

```
array(1) {
  [0]=>
  array(4) {
    ["file"]=>
    string(22) "/home/bjori/tmp/ex.php"
    ["line"]=>
    int(7)
```

```
    ["function"]=>
    string(4) "test"
    ["args"]=>
    array(0) {
    }
  }
}
```

See Also

- Throwable::getTrace() - Gets the stack trace

Error::getTraceAsString

Error::getTraceAsString

(PHP 7, PHP 8)

Error::getTraceAsString — Gets the stack trace as a string

Description

final public function **Error::getTraceAsString**(): string

Returns the stack trace as a string.

Parameters

This function has no parameters.

Return Values

Returns the stack trace as a string.

Examples

Example #1 Error::getTraceAsString() example

```php
<?php
function test() {
    throw new Error;
}

try {
    test();
} catch(Error $e) {
    echo $e->getTraceAsString();
}
?>
```

The above example will output something similar to:

```
```

```
#0 /home/bjori/tmp/ex.php(7): test()
#1 {main}
```

See Also

- Throwable::getTraceAsString() - Gets the stack trace as a string

Error::__toString

Error::__toString

(PHP 7, PHP 8)

Error::__toString — String representation of the error

Description

public function **Error::__toString**(): string

Returns the string representation of the error.

Parameters

This function has no parameters.

Return Values

Returns the string representation of the error.

Examples

Example #1 Error::__toString() example

```php
<?php
try {
    throw new Error("Some error message");
} catch(Error $e) {
    echo $e;
}
?>
```

The above example will output something similar to:

```
Error: Some error message in /home/bjori/tmp/ex.php:3
Stack trace:
#0 {main}
```

See Also

- Throwable::__toString() - Gets a string representation of the thrown object

Error::__clone

Error::__clone

(PHP 7, PHP 8)

Error::__clone — Clone the error

Description

private function **Error::__clone**(): void

Error can not be cloned, so this method results in fatal error.

Parameters

This function has no parameters.

Return Values

No value is returned.

Errors/Exceptions

Errors are *not* clonable.

Changelog

Version	Description
8.1.0	**Error::__clone()** is no longer final.

ArgumentCountError

(PHP 7 >= PHP 7.1.0, PHP 8)

Introduction

ArgumentCountError is thrown when too few arguments are passed to a user-defined function or method.

This error is also thrown when too many arguments are passed to a non-variadic built-in function.

Class synopsis

```
class ArgumentCountError extends TypeError {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
```

private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

ArithmeticError

(PHP 7, PHP 8)

Introduction

ArithmeticError is thrown when an error occurs while performing mathematical operations. These errors include attempting to perform a bitshift by a negative amount, and any call to intdiv() that would result in a value outside the possible bounds of an int.

Class synopsis

class **ArithmeticError** extends Error {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

AssertionError

(PHP 7, PHP 8)

Introduction

AssertionError is thrown when an assertion made via assert() fails.

Class synopsis

class **AssertionError** extends Error {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

DivisionByZeroError

(PHP 7, PHP 8)

Introduction

DivisionByZeroError is thrown when an attempt is made to divide a number by zero.

Class synopsis

class **DivisionByZeroError** extends ArithmeticError {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int

final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

CompileError

(PHP 7 > 7.3.0, PHP 8)

Introduction

CompileError is thrown for some compilation errors, which formerly issued a fatal error.

Class synopsis

class **CompileError** extends Error {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

ParseError

(PHP 7, PHP 8)

Introduction

ParseError is thrown when an error occurs while parsing PHP code, such as when eval() is called.

> **Note** :
>
> **ParseError** extends CompileError as of PHP 7.3.0. Formerly, it extended Error.

Class synopsis

```
class ParseError extends CompileError {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

TypeError

(PHP 7, PHP 8)

Introduction

A **TypeError** may be thrown when:

- The value being set for a class property does not match the property's corresponding declared type.
- The argument type being passed to a function does not match its corresponding declared parameter type.
- A value being returned from a function does not match the declared function return type.

Class synopsis

```
class TypeError extends Error {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
```

final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

Changelog

Version	Description
7.1.0	In strict mode, passing an incorrect number of arguments to a built-in PHP function no longer results in a generic **TypeError**. Instead, a more specific ArgumentCountError, which extends **TypeError**, is thrown.

ValueError

(PHP 8)

Introduction

A **ValueError** is thrown when the type of an argument is correct but its value is incorrect. For example, passing a negative integer when the function expects a positive one, or passing an empty string/array when the function expects it to not be empty.

Class synopsis

class **ValueError** extends Error {
/* Inherited properties */
protected string *$message* = "";
private string *$string* = "";
protected int *$code*;
protected string *$file* = "";
protected int *$line*;
private array *$trace* = [];
private ?Throwable *$previous* = null;
/* Inherited methods */
public function Error::__construct(string `$message` = "", int `$code` = 0, ?Throwable `$previous` = **`null`**)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}

UnhandledMatchError

(PHP 8)

Introduction

An **UnhandledMatchError** is thrown when the subject passed to a match expression is not handled by any arm of the match expression.

Class synopsis

```
class UnhandledMatchError extends Error {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Inherited methods */
public function Error::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

FiberError

(PHP 8 >= 8.1.0)

Introduction

FiberError is thrown when an invalid operation is performed on a Fiber.

Class synopsis

```
final class FiberError extends Error {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
/* Methods */
public function __construct()
```

```
/* Inherited methods */
final public function Error::getMessage(): string
final public function Error::getPrevious(): ?Throwable
final public function Error::getCode(): int
final public function Error::getFile(): string
final public function Error::getLine(): int
final public function Error::getTrace(): array
final public function Error::getTraceAsString(): string
public function Error::__toString(): string
private function Error::__clone(): void
}
```

Table of Contents

FiberError::__construct

FiberError::__construct

(PHP 8 >= 8.1.0)

FiberError::__construct — Constructor to disallow direct instantiation

Description

public function **FiberError::__construct**()

Parameters

This function has no parameters.

Errors/Exceptions

Throws an Error exception when called.

RequestParseBodyException

(PHP 8 >= 8.4.0)

Introduction

A **RequestParseBodyException** is thrown in request_parse_body() when the request body is invalid, according to the `Content-Type` header.

Class synopsis

```
class RequestParseBodyException extends Exception {
/* Inherited properties */
protected string $message = "";
private string $string = "";
protected int $code;
protected string $file = "";
protected int $line;
private array $trace = [];
private ?Throwable $previous = null;
```

/* Inherited methods */
public function Exception::__construct(string $message = "", int $code = 0, ?Throwable $previous = null)
final public function Exception::getMessage(): string
final public function Exception::getPrevious(): ?Throwable
final public function Exception::getCode(): int
final public function Exception::getFile(): string
final public function Exception::getLine(): int
final public function Exception::getTrace(): array
final public function Exception::getTraceAsString(): string
public function Exception::__toString(): string
private function Exception::__clone(): void
}

Predefined Interfaces and Classes

Table of Contents

See also the SPL Interfaces and reserved classes.

Traversable

(PHP 5, PHP 7, PHP 8)

Introduction

Interface to detect if a class is traversable using `foreach`.

Abstract base interface that cannot be implemented alone. Instead, it must be implemented by either IteratorAggregate or Iterator.

Interface synopsis

interface **Traversable** {
}

This interface has no methods; its only purpose is to be the base interface for all traversable classes.

Changelog

Version	Description
8.0.0	The **Traversable** interface can now be implemented by abstract classes. Extending classes must implement Iterator or IteratorAggregate.

Notes

> **Note** :
>
> Internal (built-in) classes that implement this interface can be used in a `foreach` construct and do not need to implement IteratorAggregate or Iterator.

> **Note** :

> Prior to PHP 8.0.0, this internal engine interface couldn't be directly implemented in PHP scripts — not even by abstract classes. Either IteratorAggregate or Iterator must be used instead.

Iterator

(PHP 5, PHP 7, PHP 8)

Introduction

Interface for external iterators or objects that can be iterated themselves internally.

Interface synopsis

interface **Iterator** extends Traversable {
/* Methods */
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
}

Predefined iterators

PHP already provides a number of iterators for many day-to-day tasks. See SPL iterators for a list.

Examples

Example #1 Basic usage

This example demonstrates in which order methods are called when using `foreach` with an iterator.

```php
<?php
class myIterator implements Iterator {
    private $position = 0;
    private $array = array(
        "firstelement",
        "secondelement",
        "lastelement",
    );

    public function __construct() {
        $this->position = 0;
    }

    public function rewind(): void {
        var_dump(__METHOD__);
        $this->position = 0;
    }

    #[\ReturnTypeWillChange]
    public function current() {
        var_dump(__METHOD__);
        return $this->array[$this->position];
    }
```

```
    #[\ReturnTypeWillChange]
    public function key() {
        var_dump(__METHOD__);
        return $this->position;
    }

    public function next(): void {
        var_dump(__METHOD__);
        ++$this->position;
    }

    public function valid(): bool {
        var_dump(__METHOD__);
        return isset($this->array[$this->position]);
    }
}

$it = new myIterator;

foreach($it as $key => $value) {
    var_dump($key, $value);
    echo "\n";
}
?>
```

The above example will output something similar to:

```
string(18) "myIterator::rewind"
string(17) "myIterator::valid"
string(19) "myIterator::current"
string(15) "myIterator::key"
int(0)
string(12) "firstelement"

string(16) "myIterator::next"
string(17) "myIterator::valid"
string(19) "myIterator::current"
string(15) "myIterator::key"
int(1)
string(13) "secondelement"

string(16) "myIterator::next"
string(17) "myIterator::valid"
string(19) "myIterator::current"
string(15) "myIterator::key"
int(2)
string(11) "lastelement"

string(16) "myIterator::next"
string(17) "myIterator::valid"
```

See Also

See also object iteration.

Table of Contents

Iterator::current

Iterator::current

(PHP 5, PHP 7, PHP 8)

Iterator::current — Return the current element

Description

public function **Iterator::current**(): mixed

Returns the current element.

Parameters

This function has no parameters.

Return Values

Can return any type.

Iterator::key

Iterator::key

(PHP 5, PHP 7, PHP 8)

Iterator::key — Return the key of the current element

Description

public function **Iterator::key**(): mixed

Returns the key of the current element.

Parameters

This function has no parameters.

Return Values

Returns scalar on success, or `null` on failure.

Errors/Exceptions

Issues `E_NOTICE` on failure.

Iterator::next

Iterator::next

(PHP 5, PHP 7, PHP 8)

Iterator::next — Move forward to next element

Description

public function **Iterator::next**(): void

Moves the current position to the next element.

> **Note** :
>
> This method is called *after* each `foreach` loop.

Parameters

This function has no parameters.

Return Values

Any returned value is ignored.

Iterator::rewind

Iterator::rewind

(PHP 5, PHP 7, PHP 8)

Iterator::rewind — Rewind the Iterator to the first element

Description

public function **Iterator::rewind**(): void

Rewinds back to the first element of the Iterator.

> **Note** :
>
> This is the *first* method called when starting a `foreach` loop. It will *not* be executed *after* `foreach` loops.
>
> As `foreach` always calls this method before starting iteration, manually advancing the iterator position (for example via SplFileObject::seek()) will be reset.
>
> To iterate without rewinding the iterator, wrap it in a NoRewindIterator.

Parameters

This function has no parameters.

Return Values

Any returned value is ignored.

Iterator::valid

Iterator::valid

(PHP 5, PHP 7, PHP 8)

Iterator::valid — Checks if current position is valid

Description

public function **Iterator::valid**(): bool

This method is called after Iterator::rewind() and Iterator::next() to check if the current position is valid.

Parameters

This function has no parameters.

Return Values

The return value will be casted to bool and then evaluated. Returns `true` on success or `false` on failure.

IteratorAggregate

(PHP 5, PHP 7, PHP 8)

Introduction

Interface to create an external Iterator.

Interface synopsis

interface **IteratorAggregate** extends Traversable {
/* Methods */
public function getIterator(): Traversable
}

Examples

Example #1 Basic usage

```php
<?php

class myData implements IteratorAggregate
{
    public function getIterator(): Traversable
    {
        return new ArrayIterator([
            "key one" => "item one",
            "key two" => "item two",
            "key three" => "item three"
        ]);
    }
}

$obj = new myData();

foreach ($obj as $key => $value) {
    var_dump($key, $value);
```

```
    echo "\n";
}
```

The above example will output something similar to:

```
string(7) "key one"
string(8) "item one"

string(7) "key two"
string(8) "item two"

string(9) "key three"
string(10) "item three"
```

Table of Contents

IteratorAggregate::getIterator

IteratorAggregate::getIterator

(PHP 5, PHP 7, PHP 8)

IteratorAggregate::getIterator — Retrieve an external iterator or traversable

Description

public function **IteratorAggregate::getIterator**(): Traversable

Returns an external iterator or traversable.

Parameters

This function has no parameters.

Return Values

An instance of an object implementing Iterator or Traversable

Errors/Exceptions

Throws an Exception on failure.

InternalIterator

(PHP 8)

Introduction

Class to ease implementing IteratorAggregate for *internal* classes.

Class synopsis

final class **InternalIterator** implements Iterator {
/* Methods */
private function __construct()
public function current(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function valid(): bool
}

Table of Contents

InternalIterator::__construct

InternalIterator::__construct

(PHP 8)

InternalIterator::__construct — Private constructor to disallow direct instantiation

Description

private function **InternalIterator::__construct**()

Parameters

This function has no parameters.

InternalIterator::current

InternalIterator::current

(PHP 8)

InternalIterator::current — Return the current element

Description

public function **InternalIterator::current**(): mixed

Returns the current element.

Parameters

This function has no parameters.

Return Values

Returns the current element.

InternalIterator::key

InternalIterator::key

(PHP 8)

InternalIterator::key — Return the key of the current element

Description

public function **InternalIterator::key**(): mixed

Returns the key of the current element.

Parameters

This function has no parameters.

Return Values

Returns the key of the current element.

InternalIterator::next

InternalIterator::next

(PHP 8)

InternalIterator::next — Move forward to next element

Description

public function **InternalIterator::next**(): void

Moves the current position to the next element.

Parameters

This function has no parameters.

Return Values

No value is returned.

InternalIterator::rewind

InternalIterator::rewind

(PHP 8)

InternalIterator::rewind — Rewind the Iterator to the first element

Description

public function **InternalIterator::rewind**(): void

Rewinds back to the first element of the Iterator.

Parameters

This function has no parameters.

Return Values

No value is returned.

InternalIterator::valid

InternalIterator::valid

(PHP 8)

InternalIterator::valid — Check if current position is valid

Description

public function **InternalIterator::valid**(): bool

Checks if current position is valid.

Parameters

This function has no parameters.

Return Values

Returns whether the current position is valid.

Throwable

(PHP 7, PHP 8)

Introduction

Throwable is the base interface for any object that can be thrown via a `throw` statement, including Error and Exception.

> **Note** :
>
> PHP classes cannot implement the **Throwable** interface directly, and must instead extend Exception.

Interface synopsis

```
interface Throwable extends Stringable {
/* Methods */
public function getMessage(): string
public function getCode(): int
public function getFile(): string
public function getLine(): int
public function getTrace(): array
public function getTraceAsString(): string
public function getPrevious(): ?Throwable
public function __toString(): string
/* Inherited methods */
public function Stringable::__toString(): string
}
```

Changelog

Version	Description
8.0.0	**Throwable** implements Stringable now.

Table of Contents

Throwable::getMessage

Throwable::getMessage

(PHP 7, PHP 8)

Throwable::getMessage — Gets the message

Description

public function **Throwable::getMessage**(): string

Returns the message associated with the thrown object.

Parameters

This function has no parameters.

Return Values

Returns the message associated with the thrown object.

See Also

- Exception::getMessage() - Gets the Exception message

Throwable::getCode

Throwable::getCode

(PHP 7, PHP 8)

Throwable::getCode — Gets the exception code

Description

public function **Throwable::getCode**(): int

Returns the error code associated with the thrown object.

Parameters

This function has no parameters.

Return Values

Returns the exception code as int in Exception but possibly as other type in Exception descendants (for example as string in PDOException).

See Also

- Exception::getCode() - Gets the Exception code

Throwable::getFile

Throwable::getFile

(PHP 7, PHP 8)

Throwable::getFile — Gets the file in which the object was created

Description

public function **Throwable::getFile**(): string

Get the name of the file in which the thrown object was created.

Parameters

This function has no parameters.

Return Values

Returns the filename in which the thrown object was created.

See Also

- Exception::getFile() - Gets the file in which the exception was created

Throwable::getLine

Throwable::getLine

(PHP 7, PHP 8)

Throwable::getLine — Gets the line on which the object was instantiated

Description

public function **Throwable::getLine**(): int

Returns the line number where the thrown object was instantiated.

Parameters

This function has no parameters.

Return Values

Returns the line number where the thrown object was instantiated.

See Also

- Exception::getLine() - Gets the line in which the exception was created

Throwable::getTrace

Throwable::getTrace

(PHP 7, PHP 8)

Throwable::getTrace — Gets the stack trace

Description

public function **Throwable::getTrace**(): array

Returns the stack trace as an array.

Parameters

This function has no parameters.

Return Values

Returns the stack trace as an array in the same format as debug_backtrace().

See Also

- Exception::getTrace() - Gets the stack trace

Throwable::getTraceAsString

Throwable::getTraceAsString

(PHP 7, PHP 8)

Throwable::getTraceAsString — Gets the stack trace as a string

Description

public function **Throwable::getTraceAsString**(): string

Parameters

This function has no parameters.

Return Values

Returns the stack trace as a string.

See Also

- Exception::getTraceAsString() - Gets the stack trace as a string

Throwable::getPrevious

Throwable::getPrevious

(PHP 7, PHP 8)

Throwable::getPrevious — Returns the previous Throwable

Description

public function **Throwable::getPrevious**(): ?Throwable

Returns any previous Throwable (for example, one provided as the third parameter to Exception::__construct()).

Parameters

This function has no parameters.

Return Values

Returns the previous Throwable if available, or `null` otherwise.

See Also

- Exception::getPrevious() - Returns previous Throwable

Throwable::__toString

Throwable::__toString

(PHP 7, PHP 8)

Throwable::__toString — Gets a string representation of the thrown object

Description

public function **Throwable::__toString**(): string

Parameters

This function has no parameters.

Return Values

Returns the string representation of the thrown object.

See Also

- Exception::__toString() - String representation of the exception

Countable

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Classes implementing **Countable** can be used with the count() function.

Interface synopsis

```
interface Countable {
/* Methods */
public function count(): int
}
```

Table of Contents

Countable::count

Countable::count

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Countable::count — Count elements of an object

Description

public function **Countable::count**(): int

This method is executed when the `value` for count() is an object implementing Countable.

Parameters

This function has no parameters.

Return Values

The custom count as an int.

Examples

Example #1 Countable::count() example

```php
<?php

class Counter implements Countable
{
    private $count = 0;

    public function count(): int
    {
        return ++$this->count;
    }
}

$counter = new Counter;

for ($i = 0; $i < 10; ++$i) {
    echo "I have been count()ed " . count($counter) . " times\n";
}

?>
```

The above example will output something similar to:

```
I have been count()ed 1 times
I have been count()ed 2 times
I have been count()ed 3 times
I have been count()ed 4 times
I have been count()ed 5 times
I have been count()ed 6 times
I have been count()ed 7 times
I have been count()ed 8 times
```

```
I have been count()ed 9 times
I have been count()ed 10 times
```

ArrayAccess

(PHP 5, PHP 7, PHP 8)

Introduction

Interface to provide access to objects as arrays.

Interface synopsis

interface **ArrayAccess** {
/* Methods */
public function offsetExists(mixed `$offset`): bool
public function offsetGet(mixed `$offset`): mixed
public function offsetSet(mixed `$offset`, mixed `$value`): void
public function offsetUnset(mixed `$offset`): void
}

Examples

Example #1 Basic usage

```php
<?php
class Obj implements ArrayAccess {
    public $container = [
        "one"   => 1,
        "two"   => 2,
        "three" => 3,
    ];

    public function offsetSet($offset, $value): void {
        if (is_null($offset)) {
            $this->container[] = $value;
        } else {
            $this->container[$offset] = $value;
        }
    }

    public function offsetExists($offset): bool {
        return isset($this->container[$offset]);
    }

    public function offsetUnset($offset): void {
        unset($this->container[$offset]);
    }

    public function offsetGet($offset): mixed {
        return isset($this->container[$offset]) ? $this->container[$offset] : null;
    }
```

```
}

$obj = new Obj;

var_dump(isset($obj["two"]));
var_dump($obj["two"]);
unset($obj["two"]);
var_dump(isset($obj["two"]));
$obj["two"] = "A value";
var_dump($obj["two"]);
$obj[] = 'Append 1';
$obj[] = 'Append 2';
$obj[] = 'Append 3';
print_r($obj);
```

The above example will output something similar to:

```
bool(true)
int(2)
bool(false)
string(7) "A value"
Obj Object
(
    [container] => Array
        (
            [one] => 1
            [three] => 3
            [two] => A value
            [0] => Append 1
            [1] => Append 2
            [2] => Append 3
        )

)
```

Table of Contents

ArrayAccess::offsetExists

ArrayAccess::offsetExists

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetExists — Whether an offset exists

Description

public function **ArrayAccess::offsetExists**(mixed `$offset`): bool

Whether or not an offset exists.

This method is executed when using isset() or empty() on objects implementing ArrayAccess.

> **Note** :
>
> When using empty() ArrayAccess::offsetGet() will be called and checked if empty only if **ArrayAccess::offsetExists()** returns `true`.

Parameters

`offset`

An offset to check for.

Return Values

Returns `true` on success or `false` on failure.

> **Note** :
>
> The return value will be casted to bool if non-boolean was returned.

Examples

Example #1 ArrayAccess::offsetExists() example

```php
<?php
class obj implements ArrayAccess {
    public function offsetSet($offset, $value): void {
        var_dump(__METHOD__);
    }
    public function offsetExists($var): bool {
        var_dump(__METHOD__);
        if ($var == "foobar") {
            return true;
        }
        return false;
    }
    public function offsetUnset($var): void {
        var_dump(__METHOD__);
    }
    #[\ReturnTypeWillChange]
    public function offsetGet($var) {
        var_dump(__METHOD__);
        return "value";
    }
}

$obj = new obj;

echo "Runs obj::offsetExists()\n";
var_dump(isset($obj["foobar"]));

echo "\nRuns obj::offsetExists() and obj::offsetGet()\n";
var_dump(empty($obj["foobar"]));
```

```
echo "\nRuns obj::offsetExists(), *not* obj:offsetGet() as there is nothing to get\n";
var_dump(empty($obj["foobaz"]));
?>
```

The above example will output something similar to:

```
Runs obj::offsetExists()
string(17) "obj::offsetExists"
bool(true)

Runs obj::offsetExists() and obj::offsetGet()
string(17) "obj::offsetExists"
string(14) "obj::offsetGet"
bool(false)

Runs obj::offsetExists(), *not* obj:offsetGet() as there is nothing to get
string(17) "obj::offsetExists"
bool(true)
```

ArrayAccess::offsetGet

ArrayAccess::offsetGet

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetGet — Offset to retrieve

Description

public function **ArrayAccess::offsetGet**(mixed `$offset`): mixed

Returns the value at specified offset.

This method is executed when checking if offset is empty().

Parameters

`offset`

The offset to retrieve.

Return Values

Can return all value types.

Notes

> **Note** :

> It's possible for implementations of this method to return by reference. This makes indirect modifications to the overloaded array dimensions of ArrayAccess objects possible.
>
> A direct modification is one that replaces completely the value of the array dimension, as in `$obj[6] = 7`. An indirect modification, on the other hand, only changes part of the dimension, or attempts to assign the dimension by reference to another variable, as in `$obj[6][7] = 7` or `$var =& $obj[6]`. Increments with `++` and decrements with `--` are also implemented in a way that requires indirect modification.
>
> While direct modification triggers a call to ArrayAccess::offsetSet(), indirect modification triggers a call to **ArrayAccess::offsetGet()**. In that case, the implementation of **ArrayAccess::offsetGet()** must be able to return by reference, otherwise an `E_NOTICE` message is raised.

See Also

- ArrayAccess::offsetExists() - Whether an offset exists

ArrayAccess::offsetSet

ArrayAccess::offsetSet

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetSet — Assign a value to the specified offset

Description

public function **ArrayAccess::offsetSet**(mixed `$offset`, mixed `$value`): void

Assigns a value to the specified offset.

Parameters

`offset`

The offset to assign the value to.

`value`

The value to set.

Return Values

No value is returned.

Notes

> **Note** :
>
> The `offset` parameter will be set to `null` if another value is not available, like in the following example.

```php
<?php
$arrayaccess[] = "first value";
$arrayaccess[] = "second value";
```

```
print_r($arrayaccess);
?>
```

The above example will output:

```
Array
(
    [0] => first value
    [1] => second value
)
```

> **Note** :
>
> This function is not called in assignments by reference and otherwise indirect changes to array dimensions overloaded with ArrayAccess (indirect in the sense they are made not by changing the dimension directly, but by changing a sub-dimension or sub-property or assigning the array dimension by reference to another variable). Instead, ArrayAccess::offsetGet() is called. The operation will only be successful if that method returns by reference.

ArrayAccess::offsetUnset

ArrayAccess::offsetUnset

(PHP 5, PHP 7, PHP 8)

ArrayAccess::offsetUnset — Unset an offset

Description

public function **ArrayAccess::offsetUnset**(mixed `$offset`): void

Unsets an offset.

> **Note** :
>
> This method will *not* be called when type-casting to (unset)

Parameters

`offset`

The offset to unset.

Return Values

No value is returned.

Serializable

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Introduction

Interface for customized serializing.

Classes that implement this interface no longer support __sleep() and __wakeup(). The serialize() method is called whenever an instance needs to be serialized. This does not invoke **__destruct()** or have any other side effect unless programmed inside the method. When the data is unserialized the class is known and the appropriate unserialize() method is called as a constructor instead of calling **__construct()**. The constructor can be called from within the unserialize() method if desired.

> **Warning**
>
> As of PHP 8.1.0, a class which implements **Serializable** without also implementing __serialize() and __unserialize() will generate a deprecation warning.

New code should use the __serialize() and __unserialize() magic methods instead, available as of PHP 7.4.0. When a class declares both them and this interface, serialize() always uses __serialize() and never reaches Serializable::serialize(). Unserializing is decided by the format of the data instead: a payload written by PHP 7.4.0 or later is read by __unserialize(), one written before it is still read by Serializable::unserialize(). Implementing **Serializable** therefore remains of use for reading such older payloads, and for satisfying a **Serializable** type declaration; declaring all four methods covers every case, and avoids the deprecation notice.

> **Note** :
>
> The magic methods have no interface of their own, so a class which declares them without implementing **Serializable** is not an instance of it. method_exists() is the way to detect them.

Interface synopsis

interface **Serializable** {
/* Methods */
public function serialize(): ?string
public function unserialize(string `$data`): void
}

Examples

Example #1 Supporting PHP 7.1.0 through 7.3.0

The magic methods carry the serialized form, and the interface methods delegate to them, so that a single representation serves both.

```php
<?php
class Task implements Serializable
{
    private $label;

    public function __construct($label)
```

```
    {
        $this->label = $label;
    }

    public function __serialize(): array
    {
        return ['label' => $this->label];
    }

    public function __unserialize(array $data): void
    {
        $this->label = $data['label'];
    }

    // Never called as of PHP 7.4.0
    public function serialize()
    {
        return serialize($this->__serialize());
    }

    // Still called as of PHP 7.4.0, for data written before it
    public function unserialize($data)
    {
        $this->__unserialize(unserialize($data));
    }
}

var_dump(serialize(new Task('deploy')));
?>
```

The above example will output:

```
string(40) "O:4:"Task":1:{s:5:"label";s:6:"deploy";}"
```

Prior to PHP 7.4.0, the same code serializes through the interface instead, and outputs `string(47) "C:4:"Task":31:{a:1:{s:5:"label";s:6:"deploy";}}"`.

Table of Contents

Serializable::serialize

Serializable::serialize

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Serializable::serialize — String representation of object

Description

public function **Serializable::serialize**(): ?string

Should return the string representation of the object.

Parameters

This function has no parameters.

Return Values

Returns the string representation of the object or `null`

Errors/Exceptions

Throws Exception when returning other types than strings and `null`.

See Also

- __sleep()
- __serialize()

Serializable::unserialize

Serializable::unserialize

(PHP 5 >= 5.1.0, PHP 7, PHP 8)

Serializable::unserialize — Constructs the object

Description

public function **Serializable::unserialize**(string `$data`): void

Called during unserialization of the object.

> **Note** :
>
> This method acts as the constructor of the object. The __construct() method will *not* be called after this method.

Parameters

`data`

The string representation of the object.

Return Values

The return value from this method is ignored.

See Also

- __wakeup()
- __unserialize()

Closure

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Introduction

Class used to represent anonymous functions.

Anonymous functions yield objects of this type. This class has methods that allow further control of the anonymous function after it has been created.

Besides the methods listed here, this class also has an `__invoke` method. This is for consistency with other classes that implement calling magic, as this method is not used for calling the function.

Class synopsis

final class **Closure** {
/* Methods */
private function __construct()
public static function bind(Closure `$closure`, ?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure
public function bindTo(?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure
public function call(object `$newThis`, mixed `...$args`): mixed
public static function fromCallable(callable `$callback`): Closure
public static function getCurrent(): Closure
}

Changelog

Version	Description
8.4.0	The output of **Closure::__debugInfo()** now includes the name, line, and file of the closure.

Notes

> **Note** :
>
> **Closure** objects cannot be serialized as closures may contain bound variables and a specific execution context. Attempting to serialize a closure will throw an Exception.

Table of Contents

Closure::__construct

Closure::__construct

(PHP 5 >= 5.3.0, PHP 7, PHP 8)

Closure::__construct — Constructor that disallows instantiation

Description

private function **Closure::__construct**()

This method exists only to disallow instantiation of the Closure class. Objects of this class are created in the fashion described on the anonymous functions page.

Parameters

This function has no parameters.

See Also

- Anonymous functions

Closure::bind

Closure::bind

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Closure::bind — Duplicates a closure with a specific bound object and class scope

Description

public static function **Closure::bind**(Closure `$closure`, ?object `$newThis`, object|string|null `$newScope = "static"`): ?Closure

This method is a static version of Closure::bindTo(). See the documentation of that method for more information.

Parameters

`closure`

The anonymous functions to bind.

`newThis`

The object to which the given anonymous function should be bound, or `null` for the closure to be unbound.

`newScope`

The class scope to which the closure is to be associated, or 'static' to keep the current one. If an object is given, the type of the object will be used instead. This determines the visibility of protected and private methods of the bound object. It is not allowed to pass (an object of) an internal class as this parameter.

Return Values

Returns a new Closure object, or `null` on failure.

Changelog

Version	Description
8.5.0	Binding an instance to a static closure, or binding a method from an incompatible class hierarchy, is now deprecated (these cases already emitted an `E_WARNING`).

Examples

Example #1 **Closure::bind()** example

```
<?php
class A {
    private static $sfoo = 1;
    private $ifoo = 2;
```

```
}
$cl1 = static function() {
    return A::$sfoo;
};
$cl2 = function() {
    return $this->ifoo;
};

$bcl1 = Closure::bind($cl1, null, 'A');
$bcl2 = Closure::bind($cl2, new A(), 'A');
echo $bcl1(), "\n";
echo $bcl2(), "\n";
?>
```

The above example will output something similar to:

```
1
2
```

See Also

- Anonymous functions
- Closure::bindTo() - Duplicates the closure with a new bound object and class scope

Closure::bindTo

Closure::bindTo

(PHP 5 >= 5.4.0, PHP 7, PHP 8)

Closure::bindTo — Duplicates the closure with a new bound object and class scope

Description

public function **Closure::bindTo**(?object `$newThis`, object|string|null `$newScope` = "static"): ?Closure

Create and return a new anonymous function with the same body and bound variables as this one, but possibly with a different bound object and a new class scope.

The “bound object” determines the value `$this` will have in the function body and the “class scope” represents a class which determines which private and protected members the anonymous function will be able to access. Namely, the members that will be visible are the same as if the anonymous function were a method of the class given as value of the `newScope` parameter.

Static closures cannot have any bound object (the value of the parameter `newThis` should be `null`), but this method can nevertheless be used to change their class scope.

This method will ensure that for a non-static closure, having a bound instance will imply being scoped and vice-versa. To this end, non-static closures that are given a scope but a `null` instance are made static and non-static non-scoped closures that are given a non-null instance are scoped to an unspecified class.

> **Note** :
>
> If you only want to duplicate the anonymous functions, you can use cloning instead.

Parameters

`newThis`

The object to which the given anonymous function should be bound, or `null` for the closure to be unbound.

`newScope`

The class scope to which the closure is to be associated, or 'static' to keep the current one. If an object is given, the type of the object will be used instead. This determines the visibility of protected and private methods of the bound object. It is not allowed to pass (an object of) an internal class as this parameter.

Return Values

Returns the newly created Closure object or `null` on failure.

Changelog

Version	Description
8.5.0	Binding an instance to a static closure, or binding a method from an incompatible class hierarchy, is now deprecated (these cases already emitted an `E_WARNING`).

Examples

Example #1 **Closure::bindTo()** example

```php
<?php

class A
{
    private $val;

    public function __construct($val)
    {
        $this->val = $val;
    }

    public function getClosure()
    {
        // Returns closure bound to this object and scope
        return function() {
            return $this->val;
        };
    }
}

$ob1 = new A(1);
$ob2 = new A(2);
```

```php
$cl = $ob1->getClosure();
echo $cl(), "\n";

$cl = $cl->bindTo($ob2);
echo $cl(), "\n";

?>
```

The above example will output something similar to:

```
1
2
```

See Also

- Anonymous functions
- Closure::bind() - Duplicates a closure with a specific bound object and class scope

Closure::call

Closure::call

(PHP 7, PHP 8)

Closure::call — Binds and calls the closure

Description

public function **Closure::call**(object `$newThis`, mixed `...$args`): mixed

Temporarily binds the closure to `newThis`, and calls it with any given parameters.

Parameters

`newThis`

The object to bind the closure to for the duration of the call.

`args`

Zero or more parameters, which will be given as parameters to the closure.

Return Values

Returns the return value of the closure.

Examples

Example #1 Closure::call() example

```php
<?php
class Value {
    protected $value;

    public function __construct($value) {
        $this->value = $value;
    }

    public function getValue() {
        return $this->value;
    }
}

$three = new Value(3);
$four = new Value(4);

$closure = function ($delta) { var_dump($this->getValue() + $delta); };
$closure->call($three, 4);
$closure->call($four, 4);
?>
```

The above example will output:

```
int(7)
int(8)
```

Closure::fromCallable

Closure::fromCallable

(PHP 7 >= 7.1.0)

Closure::fromCallable — Converts a callable into a closure

Description

public static function **Closure::fromCallable**(callable `$callback`): Closure

Create and return a new anonymous function from given `callback` using the current scope. This method checks if the `callback` is callable in the current scope and throws a TypeError if it is not.

> **Note** :
>
> As of PHP 8.1.0, First class callable syntax has the same semantics as this method.

Parameters

`callback`

The callable to convert.

Return Values

Returns the newly created Closure or throws a TypeError if the `callback` is not callable in the current scope.

See Also

- Anonymous functions
- First class callable syntax

Closure::getCurrent

Closure::getCurrent

(PHP 8 >= 8.5.0)

Closure::getCurrent — Returns the currently executing closure

Description

public static function **Closure::getCurrent**(): Closure

Returns the currently executing closure. This method is primarily useful for implementing recursive closures without needing to capture a reference to the closure variable using the `use` keyword.

This method must be called from within a closure; calling it outside of a closure context will result in `Error: Current function is not a closure.`

Parameters

This function has no parameters.

Return Values

Returns the currently executing Closure instance.

Errors/Exceptions

Throws an Error if called outside of a closure context.

Examples

Example #1 Closure::getCurrent() example

Using **Closure::getCurrent()** to implement a recursive Fibonacci function:

```php
<?php
$fibonacci = function (int $n) {
    if ($n === 0 || $n === 1) {
        return $n;
    }

    $fn = Closure::getCurrent();
    return $fn($n - 1) + $fn($n - 2);
};
```

```
echo $fibonacci(10); // Outputs: 55
?>
```

Example #2 Comparison with traditional approach

Prior to PHP 8.5, implementing recursive closures required capturing a reference to the closure variable using the `use` keyword:

```
<?php
// Traditional approach (still works in PHP 8.5)
$fibonacci = function (int $n) use (&$fibonacci) {
    if ($n === 0) return 0;
    if ($n === 1) return 1;
    return $fibonacci($n - 1) + $fibonacci($n - 2);
};

echo $fibonacci(10); // Outputs: 55
?>
```

The **Closure::getCurrent()** approach eliminates the need to declare the variable with a reference in the `use` clause, making the code cleaner and less error-prone.

stdClass

(PHP 4, PHP 5, PHP 7, PHP 8)

Introduction

A generic empty class with dynamic properties.

Objects of this class can be instantiated with the new operator or created by typecasting to object. Several PHP functions also create instances of this class, e.g. json_decode(), mysqli_fetch_object() or PDOStatement::fetchObject().

Despite not implementing __get()/__set() magic methods, this class allows dynamic properties and does not require the `#[\AllowDynamicProperties]` attribute.

This is not a base class as PHP does not have a concept of a universal base class. However, it is possible to create a custom class that extends **stdClass** and as a result inherits the functionality of dynamic properties.

Class synopsis

#[\AllowDynamicProperties]
class **stdClass** {
}

This class has no methods or default properties.

Examples

Example #1 Created as a result of typecasting to object

```
<?php
$obj = (object) array('foo' => 'bar');
var_dump($obj);
```

The above example will output:

```
object(stdClass)#1 (1) {
  ["foo"]=>
  string(3) "bar"
}
```

Example #2 Created as a result of json_decode()

```
<?php
$json = '{"foo":"bar"}';
var_dump(json_decode($json));
```

The above example will output:

```
object(stdClass)#1 (1) {
  ["foo"]=>
  string(3) "bar"
}
```

Example #3 Declaring dynamic properties

```
<?php
$obj = new stdClass();
$obj->foo = 42;
$obj->{1} = 42;
var_dump($obj);
```

The above example will output:

```
object(stdClass)#1 (2) {
  ["foo"]=>
  int(42)
  ["1"]=>
  int(42)
}
```

Generator

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Introduction

Generator objects are returned from generators.

> **Caution**
>
> **Generator** objects cannot be instantiated via new.

Class synopsis

```
final class Generator implements Iterator {
/* Methods */
public function current(): mixed
public function getReturn(): mixed
public function key(): mixed
public function next(): void
public function rewind(): void
public function send(mixed $value): mixed
public function throw(Throwable $exception): mixed
public function valid(): bool
public function __wakeup(): void
}
```

See Also

See also object iteration.

Table of Contents

Generator::current

Generator::current

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::current — Get the yielded value

Description

public function **Generator::current**(): mixed

Parameters

This function has no parameters.

Return Values

Returns the yielded value.

Generator::getReturn

Generator::getReturn

(PHP 7, PHP 8)

Generator::getReturn — Get the return value of a generator

Description

public function **Generator::getReturn**(): mixed

Parameters

This function has no parameters.

Return Values

Returns the generator's return value once it has finished executing.

Examples

Example #1 Generator::getReturn() example

```php
<?php

$gen = (function() {
    yield 1;
    yield 2;

    return 3;
})();

foreach ($gen as $val) {
    echo $val, PHP_EOL;
}

echo $gen->getReturn(), PHP_EOL;
```

The above example will output:

```
1
2
3
```

Generator::key

Generator::key

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::key — Get the yielded key

Description

public function **Generator::key**(): mixed

Gets the key of the yielded value.

Parameters

This function has no parameters.

Return Values

Returns the yielded key.

Examples

Example #1 **Generator::key()** example

```php
<?php

function Gen()
{
    yield 'key' => 'value';
}

$gen = Gen();

echo "{$gen->key()} => {$gen->current()}";
```

The above example will output:

```
key => value
```

Generator::next

Generator::next

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::next — Resume execution of the generator

Description

public function **Generator::next**(): void

Calling **Generator::next()** has the same effect as calling Generator::send() with `null` as argument.

Parameters

This function has no parameters.

Return Values

No value is returned.

Generator::rewind

Generator::rewind

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::rewind — Execute the generator up to and including the first yield

Description

public function **Generator::rewind**(): void

Executes the generator up to and including the *first* `yield`. If the generator is already at the *first* `yield`, no action will be taken. If the generator has ever advanced beyond a `yield` expression, this method will throw an Exception.

> **Note** :
>
> This is the *first* method called when starting a `foreach` loop. It will *not* be executed *after* `foreach` loops.

Parameters

This function has no parameters.

Return Values

No value is returned.

Examples

Example #1 Generator::rewind() example

```php
<?php

function generator(): Generator
{
    echo "I'm a generator!\n";

    for ($i = 1; $i <= 3; $i++) {
        yield $i;
    }
}

// Initialize the generator
$generator = generator();

// Rewind the generator to the beginning of the first yield expression,
// if it's not already there
$generator->rewind(); // I'm a generator!

// Nothing happens here; the generator is already rewound
$generator->rewind(); // No output (NULL)

// This rewinds the generator to the first yield expression,
// if it's not already there, and iterates over the generator
```

```php
foreach ($generator as $value) {
    // After yielding the first value, the generator remains at
    // the first yield expression until it resumes execution and advances to the next yield
    echo $value, PHP_EOL; // 1

    break;
}

// Resume and rewind again. No error occurs because the generator has not advanced beyond the first
yield
$generator->rewind();

echo $generator->current(), PHP_EOL; // 1

// No error occurs, the generator is still at the first yield
$generator->rewind();

// This advances the generator to the second yield expression
$generator->next();

try {
    // This will throw an Exception,
    // because the generator has already advanced to the second yield
    $generator->rewind(); // Fatal error: Uncaught Exception: Cannot rewind a generator that was
already run
} catch (Exception $e) {
    echo $e->getMessage();
}

?>
```

The above example will output:

```
I'm a generator!
1
1
Cannot rewind a generator that was already run
```

Generator::send

Generator::send

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::send — Send a value to the generator

Description

public function **Generator::send**(mixed `$value`): mixed

Sends the given value to the generator as the result of the current `yield` expression and resumes execution of the generator.

If the generator is not at a `yield` expression when this method is called, it will first be let to advance to the first `yield` expression before sending the value. As such it is not necessary to "prime" PHP generators with a Generator::next() call (like it is done in Python).

Parameters

`value`

Value to send into the generator. This value will be the return value of the `yield` expression the generator is currently at.

Return Values

Returns the yielded value.

Examples

Example #1 Using **Generator::send()** to inject values

```
<?php
function printer() {
    echo "I'm printer!".PHP_EOL;
    while (true) {
        $string = yield;
        echo $string.PHP_EOL;
    }
}

$printer = printer();
$printer->send('Hello world!');
$printer->send('Bye world!');
?>
```

The above example will output:

```
I'm printer!
Hello world!
Bye world!
```

Generator::throw

Generator::throw

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::throw — Throw an exception into the generator

Description

public function **Generator::throw**(Throwable `$exception`): mixed

Throws an exception into the generator and resumes execution of the generator. The behavior will be the same as if the current `yield` expression was replaced with a `throw $exception` statement.

If the generator is already closed when this method is invoked, the exception will be thrown in the caller's context instead.

Parameters

`exception`

Exception to throw into the generator.

Return Values

Returns the yielded value.

Examples

Example #1 Throwing an exception into a generator

```php
<?php
function gen() {
    echo "Foo\n";
    try {
        yield;
    } catch (Exception $e) {
        echo "Exception: {$e->getMessage()}\n";
    }
    echo "Bar\n";
}

$gen = gen();
$gen->rewind();
$gen->throw(new Exception('Test'));
?>
```

The above example will output:

```
Foo
Exception: Test
Bar
```

Generator::valid

Generator::valid

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::valid — Check if the iterator has been closed

Description

public function **Generator::valid**(): bool

Parameters

This function has no parameters.

Return Values

Returns `false` if the iterator has been closed. Otherwise returns `true`.

Generator::__wakeup

Generator::__wakeup

(PHP 5 >= 5.5.0, PHP 7, PHP 8)

Generator::__wakeup — Serialize callback

Description

public function **Generator::__wakeup**(): void

Throws an exception as generators can't be serialized.

Parameters

This function has no parameters.

Return Values

No value is returned.

Fiber

(PHP 8 >= 8.1.0)

Introduction

Fibers represent full-stack, interruptible functions. Fibers may be suspended from anywhere in the call-stack, pausing execution within the fiber until the fiber is resumed at a later time.

Class synopsis

final class **Fiber** {
/* Methods */
public function __construct(callable `$callback`)
public function start(mixed `...$args`): mixed
public function resume(mixed `$value = null`): mixed
public function throw(Throwable `$exception`): mixed
public function getReturn(): mixed
public function isStarted(): bool

public function isSuspended(): bool
public function isRunning(): bool
public function isTerminated(): bool
public static function suspend(mixed $value = null): mixed
public static function getCurrent(): ?Fiber
}

See Also

Fibers overview

Table of Contents

Fiber::__construct

Fiber::__construct

(PHP 8 >= 8.1.0)

Fiber::__construct — Creates a new Fiber instance

Description

public function **Fiber::__construct**(callable $callback)

Parameters

`callback`

The callable to invoke when starting the fiber. Arguments given to Fiber::start() will be provided as arguments to the given callable.

Fiber::start

Fiber::start

(PHP 8 >= 8.1.0)

Fiber::start — Start execution of the fiber

Description

public function **Fiber::start**(mixed ...$args): mixed

A variadic list of arguments to provide to the callable used when constructing the fiber.

If the fiber has already been started when this method is called, a FiberError will be thrown.

Parameters

`args`

The arguments to use when invoking the callable given to the fiber constructor.

Return Values

The value provided to the first call to Fiber::suspend() or `null` if the fiber returns. If the fiber throws an exception before suspending, it will be thrown from the call to this method.

Fiber::resume

Fiber::resume

(PHP 8 >= 8.1.0)

Fiber::resume — Resumes execution of the fiber with a value

Description

public function **Fiber::resume**(mixed `$value = null`): mixed

Resumes the fiber using the given value as the result of the current Fiber::suspend() call.

If the fiber is not suspended when this method is called, a FiberError will be thrown.

Parameters

`value`

The value to resume the fiber. This value will be the return value of the current Fiber::suspend() call.

Return Values

The value provided to the next call to Fiber::suspend() or `null` if the fiber returns. If the fiber throws an exception before suspending, it will be thrown from the call to this method.

Fiber::throw

Fiber::throw

(PHP 8 >= 8.1.0)

Fiber::throw — Resumes execution of the fiber with an exception

Description

public function **Fiber::throw**(Throwable `$exception`): mixed

Resumes the fiber by throwing the given exception from the current Fiber::suspend() call.

If the fiber is not suspended when this method is called, a FiberError will be thrown.

Parameters

`exception`

The exception to throw into the fiber from the current Fiber::suspend() call.

Return Values

The value provided to the next call to Fiber::suspend() or `null` if the fiber returns. If the fiber throws an exception before suspending, it will be thrown from the call to this method.

Examples

```php
<?php

$fiber = new Fiber(function () {
   try {
       // Suspend execution of the fiber declaring an interruption point
       Fiber::suspend();
   } catch (Throwable $e) {
       echo $e->getMessage();
   }
});

$fiber->start();

// Resumes execution of the fiber with
// passing the Exception to throw at the interruption point
$fiber->throw(new Exception('Message of an exception thrown at the current interrupt point'));

?>
```

The above example will output something similar to:

```
Message of an exception thrown at the current interrupt point
```

Fiber::getReturn

Fiber::getReturn

(PHP 8 >= 8.1.0)

Fiber::getReturn — Gets the value returned by the Fiber

Description

public function **Fiber::getReturn**(): mixed

Parameters

This function has no parameters.

Return Values

Returns the value returned by the callable provided to Fiber::__construct(). If the fiber has not returned a value, either because it has not been started, has not terminated, or threw an exception, a FiberError will be thrown.

Fiber::isStarted

Fiber::isStarted

(PHP 8 >= 8.1.0)

Fiber::isStarted — Determines if the fiber has started

Description

public function **Fiber::isStarted**(): bool

Parameters

This function has no parameters.

Return Values

Returns `true` only after the fiber has been started; otherwise `false` is returned.

Fiber::isSuspended

Fiber::isSuspended

(PHP 8 >= 8.1.0)

Fiber::isSuspended — Determines if the fiber is suspended

Description

public function **Fiber::isSuspended**(): bool

Parameters

This function has no parameters.

Return Values

Returns `true` if the fiber is currently suspended; otherwise `false` is returned.

Fiber::isRunning

Fiber::isRunning

(PHP 8 >= 8.1.0)

Fiber::isRunning — Determines if the fiber is running

Description

public function **Fiber::isRunning**(): bool

Parameters

This function has no parameters.

Return Values

Returns `true` only if the fiber is running. A fiber is considered running after a call to Fiber::start(), Fiber::resume(), or Fiber::throw() that has not yet returned. Returns `false` if the fiber is not running.

Fiber::isTerminated

Fiber::isTerminated

(PHP 8 >= 8.1.0)

Fiber::isTerminated — Determines if the fiber has terminated

Description

public function **Fiber::isTerminated**(): bool

Parameters

This function has no parameters.

Return Values

Returns `true` only after the fiber has terminated, either by returning or throwing an exception; otherwise `false` is returned.

Fiber::suspend

Fiber::suspend

(PHP 8 >= 8.1.0)

Fiber::suspend — Suspends execution of the current fiber

Description

public static function **Fiber::suspend**(mixed `$value = null`): mixed

Suspends execution of the current fiber. The value provided to this method will be returned from the call to Fiber::start(), Fiber::resume(), or Fiber::throw() that switched execution into the current fiber.

When the fiber is resumed, this method returns the value provided to Fiber::resume(). If the fiber is resumed using Fiber::throw(), the exception given to that method will be thrown from the call to this method.

If this method is called from outside a fiber, a FiberError will be thrown.

Parameters

`value`

> The value to return from the call to Fiber::start(), Fiber::resume(), or Fiber::throw() that switched execution into the current fiber.

Return Values

The value provided to Fiber::resume().

Fiber::getCurrent

Fiber::getCurrent

(PHP 8 >= 8.1.0)

Fiber::getCurrent — Gets the currently executing Fiber instance

Description

public static function **Fiber::getCurrent**(): ?Fiber

Parameters

This function has no parameters.

Return Values

Returns the currently executing Fiber instance or `null` if this method is called from outside a fiber.

WeakReference

(PHP 7 >= 7.4.0, PHP 8)

Introduction

Weak references allow the programmer to retain a reference to an object without preventing the object from being destroyed. They are useful for implementing cache-like structures. If the original object has been destroyed, `null` will be returned when calling the WeakReference::get() method. The original object will be destroyed when the refcount for it drops to zero; creating weak references does not increase the `refcount` of the object being referenced.

WeakReferences cannot be serialized.

Class synopsis

final class **WeakReference** {
/* Methods */
public function __construct()
public static function create(object `$object`): WeakReference
public function get(): ?object
}

WeakReference Examples

Example #1 Basic WeakReference Usage

```php
<?php

$obj = new stdClass();
$weakref = WeakReference::create($obj);

var_dump($weakref->get());

unset($obj);

var_dump($weakref->get());

?>
```

The above example will output something similar to:

```
```

```
object(stdClass)#1 (0) {
}
NULL
```

Changelog

Version	Description
8.4.0	The output of **WeakReference::__debugInfo()** now includes the referenced object, or `null` if the reference is no longer valid.

Table of Contents

WeakReference::__construct

WeakReference::__construct

(PHP 7 >= 7.4.0, PHP 8)

WeakReference::__construct — Constructor that disallows instantiation

Description

public function **WeakReference::__construct**()

This method exists only to disallow instantiation of the WeakReference class. Weak references are to be instantiated with the factory method WeakReference::create().

Parameters

This function has no parameters.

See Also

- WeakReference::create() - Create a new weak reference

WeakReference::create

WeakReference::create

(PHP 7 >= 7.4.0, PHP 8)

WeakReference::create — Create a new weak reference

Description

public static function **WeakReference::create**(object `$object`): WeakReference

Creates a new WeakReference.

Parameters

`object`

The object to be weakly referenced.

Return Values

Returns a new WeakReference, or the existing instance if there was already a WeakReference to the same object.

WeakReference::get

WeakReference::get

(PHP 7 >= 7.4.0, PHP 8)

WeakReference::get — Get a weakly referenced Object

Description

public function **WeakReference::get**(): ?object

Gets a weakly referenced object. If the object has already been destroyed, `null` is returned.

Parameters

This function has no parameters.

Return Values

Returns the referenced object, or `null` if the object has been destroyed.

WeakMap

(PHP 8)

Introduction

A **WeakMap** is a map (or dictionary) that accepts objects as keys. However, unlike the otherwise similar SplObjectStorage, an object in a key of a **WeakMap** does not contribute toward the object's reference count. That is, if at any point the only remaining reference to an object is the key of a **WeakMap**, the object will be garbage collected and removed from the **WeakMap**. Its primary use case is for building caches of data derived from an object that do not need to live longer than the object.

WeakMap implements ArrayAccess, Traversable (via IteratorAggregate), and Countable, so in most cases it can be used in the same fashion as an associative array.

Class synopsis

final class **WeakMap** implements ArrayAccess, Countable, IteratorAggregate {
/* Methods */
public function count(): int
public function getIterator(): Iterator
public function offsetExists(object `$object`): bool
public function offsetGet(object `$object`): mixed
public function offsetSet(object `$object`, mixed `$value`): void
public function offsetUnset(object `$object`): void
}

Examples

Example #1 WeakMap usage example

```php
<?php
$wm = new WeakMap();

$o = new stdClass;

class A {
    public function __destruct() {
        echo "Dead!\n";
    }
}

$wm[$o] = new A;

var_dump(count($wm));
echo "Unsetting...\n";
unset($o);
echo "Done\n";
var_dump(count($wm));
```

The above example will output:

```
int(1)
Unsetting...
Dead!
Done
int(0)
```

Table of Contents

WeakMap::count

WeakMap::count

(PHP 8)

WeakMap::count — Counts the number of live entries in the map

Description

public function **WeakMap::count**(): int

Counts the number of live entries in the map.

Parameters

This function has no parameters.

Return Values

Returns the number of live entries in the map.

WeakMap::getIterator

WeakMap::getIterator

(PHP 8)

WeakMap::getIterator — Retrieve an external iterator

Description

public function **WeakMap::getIterator**(): Iterator

Returns an external iterator.

Parameters

This function has no parameters.

Return Values

An instance of an object implementing Iterator or Traversable

Errors/Exceptions

Throws an Exception on failure.

WeakMap::offsetExists

WeakMap::offsetExists

(PHP 8)

WeakMap::offsetExists — Checks whether a certain object is in the map

Description

public function **WeakMap::offsetExists**(object `$object`): bool

Checks whether the passed object is referenced in the map.

Parameters

`object`

Object to check for.

Return Values

Returns `true` if the object is contained in the map, `false` otherwise.

WeakMap::offsetGet

WeakMap::offsetGet

(PHP 8)

WeakMap::offsetGet — Returns the value pointed to by a certain object

Description

public function **WeakMap::offsetGet**(object `$object`): mixed

Returns the value pointed to by a certain object.

Parameters

`object`

Some object contained as key in the map.

Return Values

Returns the value associated to the object passed as argument.

Errors/Exceptions

Throws an Error on failure.

WeakMap::offsetSet

WeakMap::offsetSet

(PHP 8)

WeakMap::offsetSet — Updates the map with a new key-value pair

Description

public function **WeakMap::offsetSet**(object `$object`, mixed `$value`): void

Updates the map with a new key-value pair. If the key already existed in the map, the old value is replaced with the new.

Parameters

`object`

The object serving as key of the key-value pair.

`value`

The arbitrary data serving as value of the key-value pair.

Return Values

No value is returned.

WeakMap::offsetUnset

WeakMap::offsetUnset

(PHP 8)

WeakMap::offsetUnset — Removes an entry from the map

Description

public function **WeakMap::offsetUnset**(object `$object`): void

Removes an entry from the map.

Parameters

`object`

The key object to remove from the map.

Return Values

No value is returned.

Stringable

(PHP 8)

Introduction

The **Stringable** interface denotes a class as having a __toString() method. Unlike most interfaces, **Stringable** is implicitly present on any class that has the magic __toString() method defined, although it can and should be declared explicitly.

Its primary value is to allow functions to type check against the union type `string|Stringable` to accept either a string primitive or an object that can be cast to a string.

Interface synopsis

interface **Stringable** {
/* Methods */
public function __toString(): string
}

Stringable Examples

Example #1 Basic Stringable Usage

This uses constructor property promotion.

```php
<?php
class IPv4Address implements Stringable {
    public function __construct(
        private string $oct1,
        private string $oct2,
        private string $oct3,
        private string $oct4,
    ) {}

    public function __toString(): string {
        return "$this->oct1.$this->oct2.$this->oct3.$this->oct4";
    }
}

function showStuff(string|Stringable $value) {
    // For a Stringable, this will implicitly call __toString().
```

```
    print $value;
}

$ip = new IPv4Address('123', '234', '42', '9');

showStuff($ip);
?>
```

The above example will output something similar to:

```
123.234.42.9
```

Table of Contents

Stringable::__toString

Stringable::__toString

(PHP 8)

Stringable::__toString — Gets a string representation of the object

Description

public function **Stringable::__toString**(): string

Parameters

This function has no parameters.

Return Values

Returns the string representation of the object.

See Also

- __toString()

UnitEnum

(PHP 8 >= 8.1.0)

Introduction

The **UnitEnum** interface is automatically applied to all enumerations by the engine. It may not be implemented by user-defined classes. Enumerations may not override its methods, as default implementations are provided by the engine. It is available only for type checks.

Interface synopsis

interface **UnitEnum** {
/* Methods */

public static function cases(): array
}

Table of Contents

UnitEnum::cases

UnitEnum::cases

(PHP 8 >= 8.1.0)

UnitEnum::cases — Generates a list of cases on an enum

Description

public static function **UnitEnum::cases**(): array

This method will return a packed array of all cases in an enumeration, in order of declaration.

Parameters

This function has no parameters.

Return Values

An array of all defined cases of this enumeration, in order of declaration.

Examples

Example #1 Basic usage

The following example illustrates how enum cases are returned.

```php
<?php
enum Suit
{
    case Hearts;
    case Diamonds;
    case Clubs;
    case Spades;
}

var_dump(Suit::cases());
```

The above example will output:

```
array(4) {
  [0]=>
  enum(Suit::Hearts)
  [1]=>
  enum(Suit::Diamonds)
  [2]=>
  enum(Suit::Clubs)
  [3]=>
```

```
    enum(Suit::Spades)
}
```

BackedEnum

(PHP 8 >= 8.1.0)

Introduction

The **BackedEnum** interface is automatically applied to backed enumerations by the engine. It may not be implemented by user-defined classes. Enumerations may not override its methods, as default implementations are provided by the engine. It is available only for type checks.

Interface synopsis

interface **BackedEnum** extends UnitEnum {
/* Methods */
public static function from(int|string `$value`): static
public static function tryFrom(int|string `$value`): ?static
/* Inherited methods */
public static function UnitEnum::cases(): array
}

Table of Contents

BackedEnum::from

BackedEnum::from

(PHP 8 >= 8.1.0)

BackedEnum::from — Maps a scalar to an enum instance

Description

public static function **BackedEnum::from**(int|string `$value`): static

The **from()** method translates a string or int into the corresponding Enum case, if any. If there is no matching case defined, it will throw a ValueError.

Parameters

`value`

The scalar value to map to an enum case.

Return Values

A case instance of this enumeration.

Examples

Example #1 Basic usage

The following example illustrates how enum cases are returned.

```php
<?php
enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

$h = Suit::from('H');

var_dump($h);

$b = Suit::from('B');
?>
```

The above example will output:

```
enum(Suit::Hearts)

Fatal error: Uncaught ValueError: "B" is not a valid backing value for enum "Suit" in
/file.php:15
```

See Also

- UnitEnum::cases() - Generates a list of cases on an enum
- BackedEnum::tryFrom() - Maps a scalar to an enum instance or null

BackedEnum::tryFrom

BackedEnum::tryFrom

(PHP 8 >= 8.1.0)

BackedEnum::tryFrom — Maps a scalar to an enum instance or null

Description

public static function **BackedEnum::tryFrom**(int|string `$value`): ?static

The **tryFrom()** method translates a string or int into the corresponding Enum case, if any. If there is no matching case defined, it will return null.

Parameters

`value`

The scalar value to map to an enum case.

Return Values

A case instance of this enumeration, or null if not found.

Examples

Example #1 Basic usage

The following example illustrates how enum cases are returned.

```php
<?php
enum Suit: string
{
    case Hearts = 'H';
    case Diamonds = 'D';
    case Clubs = 'C';
    case Spades = 'S';
}

$h = Suit::tryFrom('H');

var_dump($h);

$b = Suit::tryFrom('B') ?? Suit::Spades;

var_dump($b);
?>
```

The above example will output:

```
enum(Suit::Hearts)
enum(Suit::Spades)
```

See Also

- UnitEnum::cases() - Generates a list of cases on an enum
- BackedEnum::from() - Maps a scalar to an enum instance

SensitiveParameterValue

(PHP 8 >= 8.2.0)

Introduction

The **SensitiveParameterValue** class allows wrapping sensitive values to protect them against accidental exposure.

Values of parameters having the SensitiveParameter attribute will automatically be wrapped inside of a **SensitiveParameterValue** object within stack traces.

Class synopsis

final class **SensitiveParameterValue** {
/* Properties */
private readonly mixed *$value*;
/* Methods */
public function __construct(mixed `$value`)
public function __debugInfo(): array
public function getValue(): mixed
}

Properties

value

The sensitive value to be protected against accidental exposure.

Table of Contents

SensitiveParameterValue::__construct

SensitiveParameterValue::__construct

(PHP 8 >= 8.2.0)

SensitiveParameterValue::__construct — Constructs a new SensitiveParameterValue object

Description

public function **SensitiveParameterValue::__construct**(mixed `$value`)

Construct a SensitiveParameterValue holding a sensitive value.

Parameters

`value`

An arbitrary value that should be stored inside the SensitiveParameterValue object.

SensitiveParameterValue::__debugInfo

SensitiveParameterValue::__debugInfo

(PHP 8 >= 8.2.0)

SensitiveParameterValue::__debugInfo — Protects the sensitive value against accidental exposure

Description

public function **SensitiveParameterValue::__debugInfo**(): array

Returns an empty array to protect the sensitive value against accidental exposure when using var_dump().

Parameters

This function has no parameters.

Return Values

An empty array.

Examples

Example #1 Passing a SensitiveParameterValue object to var_dump()

```
<?php
$s = new \SensitiveParameterValue('secret');

var_dump($s);
?>
```

The above example will output:

```
object(SensitiveParameterValue)#1 (0) {
}
```

SensitiveParameterValue::getValue

SensitiveParameterValue::getValue

(PHP 8 >= 8.2.0)

SensitiveParameterValue::getValue — Returns the sensitive value

Description

public function **SensitiveParameterValue::getValue**(): mixed

Get the sensitive value.

Parameters

This function has no parameters.

Return Values

The sensitive value.

Examples

Example #1 SensitiveParameterValue::getValue() example

```
<?php
$s = new \SensitiveParameterValue('secret');
```

```php
echo "The protected value is: ", $s->getValue(), "\n";
?>
```

The above example will output:

```
The protected value is: secret
```

__PHP_Incomplete_Class

(PHP 4 >=4.0.1, PHP 5, PHP 7, PHP 8)

Introduction

Created by unserialize() when trying to unserialize an undefined class or a class that is not listed in the `allowed_classes` of unserialize()'s `options` array.

Prior to PHP 7.2.0, using is_object() on the **__PHP_Incomplete_Class** class would return `false`. As of PHP 7.2.0, `true` will be returned.

Class synopsis

#[\AllowDynamicProperties]
final class **__PHP_Incomplete_Class** {
}

This class has no default properties or methods. When created by unserialize(), in addition to all unserialized properties and values the object will have a `__PHP_Incomplete_Class_Name` property which will contain the name of the unserialized class.

Changelog

Version	Description
8.0.0	This class is now `final`.

Examples

Example #1 Created by unserialize()

```php
<?php

class MyClass
{
    public string $property = "myValue";
}

$myObject = new MyClass;

$foo = serialize($myObject);
```

```php
// unserializes all objects into __PHP_Incomplete_Class objects
$disallowed = unserialize($foo, ["allowed_classes" => false]);

var_dump($disallowed);

// unserializes all objects into __PHP_Incomplete_Class objects except those of MyClass2 and
MyClass3
$disallowed2 = unserialize($foo, ["allowed_classes" => ["MyClass2", "MyClass3"]]);

var_dump($disallowed2);

// unserializes undefined class into __PHP_Incomplete_Class object
$undefinedClass = unserialize('O:16:"MyUndefinedClass":0:{}');

var_dump($undefinedClass);
```

The above example will output:

```
object(__PHP_Incomplete_Class)#2 (2) {
  ["__PHP_Incomplete_Class_Name"]=>
  string(7) "MyClass"
  ["property"]=>
  string(7) "myValue"
}
object(__PHP_Incomplete_Class)#3 (2) {
  ["__PHP_Incomplete_Class_Name"]=>
  string(7) "MyClass"
  ["property"]=>
  string(7) "myValue"
}
object(__PHP_Incomplete_Class)#4 (1) {
  ["__PHP_Incomplete_Class_Name"]=>
  string(16) "MyUndefinedClass"
}
```

Predefined Attributes

Table of Contents

PHP provides some predefined attributes that can be used.

Attribute

(PHP 8)

Introduction

Attributes offer the ability to add structured, machine-readable metadata information on declarations in code: Classes, methods, functions, parameters, properties and class constants can be the target of an attribute. The metadata defined by attributes can then be inspected at runtime using the Reflection APIs. Attributes could therefore be thought of as a configuration language embedded directly into code.

Class synopsis

```
#[\Attribute(\Attribute::TARGET_CLASS)]
final class Attribute {
/* Constants */
const int TARGET_CLASS;
const int TARGET_FUNCTION;
const int TARGET_METHOD;
const int TARGET_PROPERTY;
const int TARGET_CLASS_CONSTANT;
const int TARGET_PARAMETER;
const int TARGET_CONSTANT;
const int TARGET_ALL;
const int IS_REPEATABLE;
/* Properties */
public int $flags;
/* Methods */
public function __construct(int $flags = Attribute::TARGET_ALL)
}
```

Predefined Constants

Attribute::TARGET_CLASS

Attribute::TARGET_FUNCTION

Attribute::TARGET_METHOD

Attribute::TARGET_PROPERTY

Attribute::TARGET_CLASS_CONSTANT

Attribute::TARGET_PARAMETER

Attribute::TARGET_CONSTANT

Attribute::TARGET_ALL

Attribute::IS_REPEATABLE

Properties

flags

Changelog

Version	Description
8.5.0	Added `Attribute::TARGET_CONSTANT`. Applying `#[\Attribute]` to an abstract class, enum, interface, or trait now triggers a compile-time error (previously it was only reported at runtime when

Version	Description
	ReflectionAttribute::newInstance() was called).

See Also

Attributes overview

Table of Contents

Attribute::__construct

Attribute::__construct

(PHP 8)

Attribute::__construct — Construct a new Attribute instance

Description

public function **Attribute::__construct**(int `$flags = Attribute::TARGET_ALL`)

Constructs a new Attribute instance.

Parameters

`flags`

AllowDynamicProperties

(PHP 8 >= 8.2.0)

Introduction

This attribute is used to mark classes that allow dynamic properties.

> **Note** :
>
> Although attributes themselves are not inherited, the effect of the `AllowDynamicProperties` attribute *is* inherited. Child classes of a class marked with this attribute will also allow dynamic properties, even if they do not explicitly declare the attribute.

Class synopsis

```
#[\Attribute(\Attribute::TARGET_CLASS)]
final class AllowDynamicProperties {
/* Methods */
public function __construct()
}
```

Examples

Dynamic properties are deprecated as of PHP 8.2.0, thus using them without marking the class with this attribute will emit a deprecation notice.

Example #1 AllowDynamicProperties with non-existing property

```php
<?php
class DefaultBehaviour { }

#[\AllowDynamicProperties]
class ClassAllowsDynamicProperties { }

$o1 = new DefaultBehaviour();
$o2 = new ClassAllowsDynamicProperties();

$o1->nonExistingProp = true;
$o2->nonExistingProp = true;
?>
```

Output of the above example in PHP 8.2:

```
Deprecated: Creation of dynamic property DefaultBehaviour::$nonExistingProp is deprecated in file
on line 10
```

Example #2 AllowDynamicProperties with non-existing property in inherited class

```php
<?php
class DefaultBehaviour { }

#[\AllowDynamicProperties]
class ClassAllowsDynamicProperties { }

class InheritedClassAllowsDynamicProperties extends ClassAllowsDynamicProperties { }

$o1 = new DefaultBehaviour();
$o2 = new InheritedClassAllowsDynamicProperties();

$o1->nonExistingProp = true;
$o2->nonExistingProp = true;
?>
```

Output of the above example in PHP 8.2:

```
Deprecated: Creation of dynamic property DefaultBehaviour::$nonExistingProp is deprecated in file
on line 12
```

See Also

Attributes overview

Table of Contents

AllowDynamicProperties::__construct

AllowDynamicProperties::__construct

(PHP 8 >= 8.2.0)

AllowDynamicProperties::__construct — Construct a new AllowDynamicProperties attribute instance

Description

public function **AllowDynamicProperties::__construct**()

Constructs a new AllowDynamicProperties instance.

Parameters

This function has no parameters.

DelayedTargetValidation

(PHP 8 >= 8.5.0)

Introduction

This attribute delays target validation errors for internal attributes from compile time to when the attribute is instantiated via the Reflection API.

When applied to a declaration, any invalid usage of internal attributes on the same target will not trigger a compile time error. Instead, the validation is deferred and performed when the attribute is instantiated via ReflectionAttribute::newInstance().

This is primarily intended for forward compatibility, allowing code to use attributes that may gain additional valid targets in future PHP versions without breaking on older versions.

Class synopsis

#[\Attribute(\Attribute::TARGET_ALL)]
final class **DelayedTargetValidation** {
}

Examples

Example #1 Delaying validation of an invalid target

```php
<?php

class Base {
    protected function foo(): void {}
}

class Child extends Base {

    #[\DelayedTargetValidation]
    #[\Override]
```

```
    public const NAME = 'child';

    #[\Override]
    protected function foo(): void {}
}
```

On PHP versions where Override is not allowed on class constants, this does not produce a compile time error.

Example #2 Validation occurs during reflection

```
<?php

$reflection = new ReflectionClassConstant(Child::class, 'NAME');

foreach ($reflection->getAttributes() as $attribute) {
    $attribute->newInstance(); // May throw if invalid
}
```

When any attribute applied to the same target (other than DelayedTargetValidation itself) is instantiated via reflection using ReflectionAttribute::newInstance(), target validation is performed and an exception may be thrown if the attribute is used on an unsupported target.

Notes

This attribute only affects target validation of internal attributes.

It does not suppress functional validation performed by those attributes. For example, Override will still emit an error if a method does not actually override a parent method.

See Also

- Attributes overview
- Override

Deprecated

(PHP 8 >= 8.4.0)

Introduction

This attribute is used to mark functionality as deprecated. Using deprecated functionality will cause an `E_USER_DEPRECATED` error to be emitted.

As of PHP 8.5.0, this attribute can also be applied to traits and to compile-time constants declared outside of a class.

Class synopsis

```
#[\Attribute(
    \Attribute::TARGET_METHOD
    | \Attribute::TARGET_FUNCTION
    | \Attribute::TARGET_CLASS_CONSTANT
    | \Attribute::TARGET_CONSTANT
    | \Attribute::TARGET_CLASS,
```

```
)]
final class Deprecated {
/* Properties */
public readonly ?string $message;
public readonly ?string $since;
/* Methods */
public function __construct(?string $message = null, ?string $since = null)
}
```

Properties

message

> An optional message explaining the reason for the deprecation and possible replacement functionality. Will be included in the emitted deprecation message.

since

> An optional string indicating since when the functionality is deprecated. The contents are not validated by PHP and may contain a version number, a date or any other value that is considered appropriate. Will be included in the emitted deprecation message.
>
> Functionality that is part of PHP will use Major.Minor as the *since* value, for example `'8.4'`.

Examples

```php
<?php

#[\Deprecated(message: "use safe_replacement() instead", since: "1.5")]
function unsafe_function()
{
   echo "This is unsafe", PHP_EOL;
}

unsafe_function();

?>
```

Output of the above example in PHP 8.4 is similar to:

```
Deprecated: Function unsafe_function() is deprecated since 1.5, use safe_replacement() instead in example.php on line 9
This is unsafe
```

Changelog

Version	Description
8.5.0	**Deprecated** can now be applied to traits and to compile-time constants declared outside of a class, the latter through the new `Attribute::TARGET_CONSTANT` target.

See Also

- Attributes overview
- ReflectionFunctionAbstract::isDeprecated()
- ReflectionClassConstant::isDeprecated()
- `E_USER_DEPRECATED`

Table of Contents

Deprecated::__construct

Deprecated::__construct

(PHP 8 >= 8.4.0)

Deprecated::__construct — Construct a new Deprecated attribute instance

Description

public function **Deprecated::__construct**(?string `$message = null`, ?string `$since = null`)

Constructs a new Deprecated instance.

Parameters

`message`

The value of the message property.

`since`

The value of the since property.

NoDiscard

(PHP 8 >= 8.5.0)

Introduction

This attribute can be used to indicate that the return value of a function or a method should not be discarded. If the return value is not used in any way, a warning will be emitted.

This is useful for functions where not checking the return value is likely to be a bug.

To intentionally discard the return value of such a function, use (void) cast to suppress the warning.

> **Note** :
>
> Since attributes are designed to be backwards compatible, `#[\NoDiscard]` can be added to functions and methods even when PHP 8.4 or below are supported, it just won't do anything. On PHP 8.5 and above a warning will be emitted if the result is unused. To suppress the warning without using `(void)`, which is not supported before PHP 8.5, consider using a variable like `$_`.

> **Note** :
>
> `#[\NoDiscard]` applies to the specific function or method declaration it is written on, and the warning is emitted based on the declaration that is actually called. As a result, adding `#[\NoDiscard]` to an interface method or an abstract method does not emit a warning, because the method that is invoked is the implementing or overriding method. Likewise, a method that overrides a `#[\NoDiscard]` method does not emit the warning unless it is itself marked with the attribute. In contrast, a method imported from a trait keeps the attribute, because the trait method is copied into the using class as if declared there.

Class synopsis

```
#[\Attribute(
    \Attribute::TARGET_METHOD
    | \Attribute::TARGET_FUNCTION,
)]
final class NoDiscard {
/* Properties */
public readonly ?string $message;
/* Methods */
public function __construct(?string $message = null)
}
```

Properties

message

An optional message explaining why the return value should not be discarded.

Examples

Example #1 Basic usage

```php
<?php

/**
 * Processes all the given items and returns an array with the results of the
 * operation for each item. `null` indicates success and an Exception indicates
 * an error. The keys of the result array match the keys of the $items array.
 *
 * @param array<string> $items
 * @return array<null|Exception>
 */
#[\NoDiscard("as processing might fail for individual items")]
function bulk_process(array $items): array {
    $results = [];

    foreach ($items as $key => $item) {
        if (\random_int(0, 9999) < 9999) {
            // Pretend to do something useful with $item,
            // which will succeed in 99.99% of cases.
            echo "Processing {$item}", PHP_EOL;
            $error = null;
        } else {
```

```
            $error = new \Exception("Failed to process {$item}.");
        }

        $results[$key] = $error;
    }

    return $results;
}

bulk_process($items);

?>
```

Output of the above example in PHP 8.5 is similar to:

```
Warning: The return value of function bulk_process() should either be used or intentionally
ignored by casting it as (void), as processing might fail for individual items
```

Example #2 Intentionally discarding the return value

```
<?php

#[\NoDiscard]
function some_command(): int {
    return 1;
}

// Suppress the warning using (void) - PHP 8.5+
(void) some_command();

// For compatibility with PHP versions before 8.5, use a temporary variable
$_ = some_command();

?>
```

See Also

- Attributes overview

Table of Contents

NoDiscard::__construct

NoDiscard::__construct

(PHP 8 >= 8.5.0)

NoDiscard::__construct — Construct a new NoDiscard attribute instance

Description

public function **NoDiscard::__construct**(?string `$message` = `null`)

Constructs a new NoDiscard instance.

Parameters

`message`

The value of the message property.

Override

(PHP 8 >= 8.3.0)

Introduction

This attribute is used to indicate that a method or a property is intended to override a method or a property of a parent class or that it implements a method or a property defined in an interface.

If no method or property with the same name exists in a parent class or in an implemented interface a compile-time error will be emitted.

The attribute cannot be used on the __construct() method, which is exempt from signature checks.

Class synopsis

```
#[\Attribute(
    \Attribute::TARGET_METHOD
    | \Attribute::TARGET_PROPERTY,
)]
final class Override {
/* Methods */
public function __construct()
}
```

Changelog

Version	Description
8.5.0	**Override** can be applied to properties.

Examples

Example #1 Usage with methods

```php
<?php

class Base {
    protected function foo(): void {}
}

final class Extended extends Base {
    #[\Override]
    protected function boo(): void {}
}

?>
```

Output of the above example in PHP 8.3 is similar to:

```
Fatal error: Extended::boo() has #[\Override] attribute, but no matching parent method exists
```

Example #2 Usage with properties

```php
<?php

class Base {
    protected string $foo;
}

final class Extended extends Base {
    #[\Override]
    protected string $boo;
}

?>
```

Output of the above example in PHP 8.5 is similar to:

```
Fatal error: Extended::$boo has #[\Override] attribute, but no matching parent property exists
```

See Also

- Attributes overview

Table of Contents

Override::__construct

Override::__construct

(PHP 8 >= 8.3.0)

Override::__construct — Construct a new Override attribute instance

Description

public function **Override::__construct**()

Constructs a new Override instance.

Parameters

This function has no parameters.

ReturnTypeWillChange

(PHP 8 >= 8.1.0)

Introduction

As of PHP 8.1.0, a tentative phase of transitioning to return type declarations for internal class methods began.

Most non-final internal methods now require overriding methods to declare a compatible return type. Otherwise, a deprecation notice is emitted during inheritance validation warning that the signature violates covariance rules.

In a future PHP version, method signature checking will become strict, and mismatches will cause a fatal error.

In case the return type cannot be declared for an overriding method due to PHP cross-version compatibility concerns, or the overriding method declares an incompatible return type, a `#[\ReturnTypeWillChange]` attribute can be added to silence the deprecation notice.

> **Warning**
>
> When overriding methods defined in user-defined classes, return types are checked strictly. A fatal error is emitted on mismatch, even if the overriding method is marked with the attribute.
>
> The **ReturnTypeWillChange** attribute suppresses deprecation warnings *only* during the tentative return type phase. After strict type checking is enforced, the attribute will stop working and a mismatch in the overriding method signature of an internal class will cause a fatal error.

Class synopsis

```
#[\Attribute(\Attribute::TARGET_METHOD)]
final class ReturnTypeWillChange {
/* Methods */
public function __construct()
}
```

See Also

Attributes overview

Table of Contents

ReturnTypeWillChange::__construct

ReturnTypeWillChange::__construct

(PHP 8 >= 8.1.0)

ReturnTypeWillChange::__construct — Construct a new ReturnTypeWillChange attribute instance

Description

public function **ReturnTypeWillChange::__construct**()

Constructs a new ReturnTypeWillChange instance.

Parameters

This function has no parameters.

SensitiveParameter

(PHP 8 >= 8.2.0)

Introduction

This attribute is used to mark a parameter that is sensitive and should have its value redacted if present in a stack trace.

Class synopsis

#[\Attribute(\Attribute::TARGET_PARAMETER)]
final class **SensitiveParameter** {
/* Methods */
public function __construct()
}

Examples

```php
<?php

function defaultBehavior(
    string $secret,
    string $normal
) {
    throw new Exception('Error!');
}

function sensitiveParametersWithAttribute(
    #[\SensitiveParameter]
    string $secret,
    string $normal
) {
    throw new Exception('Error!');
}

try {
    defaultBehavior('password', 'normal');
} catch (Exception $e) {
    echo $e, PHP_EOL, PHP_EOL;
}

try {
    sensitiveParametersWithAttribute('password', 'normal');
} catch (Exception $e) {
    echo $e, PHP_EOL, PHP_EOL;
}

?>
```

Output of the above example in PHP 8.2 is similar to:

```
Exception: Error! in example.php:7
Stack trace:
#0 example.php(19): defaultBehavior('password', 'normal')
#1 {main}

Exception: Error! in example.php:15
Stack trace:
#0 example.php(25): sensitiveParametersWithAttribute(Object(SensitiveParameterValue), 'normal')
#1 {main}
```

See Also

- Attributes overview
- SensitiveParameterValue

Table of Contents

SensitiveParameter::__construct

SensitiveParameter::__construct

(PHP 8 >= 8.2.0)

SensitiveParameter::__construct — Construct a new SensitiveParameter attribute instance

Description

public function **SensitiveParameter::__construct**()

Constructs a new SensitiveParameter instance.

Parameters

This function has no parameters.

Context options and parameters

PHP offers various context options and parameters which can be used with all filesystem and stream wrappers. The context is created with stream_context_create(). Options are set with stream_context_set_option() and parameters with stream_context_set_params().

Table of Contents

Socket context options

Socket context options

Socket context options — Socket context option listing

Description

Socket context options are available for all wrappers that work over sockets, like `tcp`, `http` and `ftp`.

Options

`bindto`

Used to specify the IP address (either IPv4 or IPv6) and/or the port number that PHP will use to access the network. The syntax is `ip:port` for IPv4 addresses, and `[ip]:port` for IPv6 addresses. Setting the IP or the port to `0` will let the system choose the IP and/or port.

> **Note** :
>
> As FTP creates two socket connections during normal operation, the port number cannot be specified using this option.

`backlog`

Used to limit the number of outstanding connections in the socket's listen queue.

> **Note** :
>
> This is only applicable to stream_socket_server().

`ipv6_v6only`

Overrides the OS default regarding mapping IPv4 into IPv6.

> **Note** :
>
> This is important in particular when trying to listen on IPv4 addresses separately while there exists a binding on `[::]`.
>
> This is only applicable to stream_socket_server().

`so_reuseport`

Allows multiple bindings to a same ip:port pair, even from separate processes.

> **Note** :
>
> This is only applicable to stream_socket_server().

`so_broadcast`

Enables sending and receiving data to/from broadcast addresses.

> **Note** :

> This is only applicable to stream_socket_server().

`tcp_nodelay`

Setting this option to **`true`** will set `SOL_TCP,NO_DELAY=1` appropriately, thus disabling the TCP Nagle algorithm.

Changelog

Version	Description
7.1.0	Added `tcp_nodelay`.
7.0.1	Added `ipv6_v6only`.

Examples

Example #1 Basic `bindto` usage example

```php
<?php
// connect to the internet using the '192.168.0.100' IP
$opts = array(
    'socket' => array(
        'bindto' => '192.168.0.100:0',
    ),
);


// connect to the internet using the '192.168.0.100' IP and port '7000'
$opts = array(
    'socket' => array(
        'bindto' => '192.168.0.100:7000',
    ),
);


// connect to the internet using the '2001:db8::1' IPv6 address
// and port '7000'
$opts = array(
    'socket' => array(
        'bindto' => '[2001:db8::1]:7000',
    ),
);


// connect to the internet using port '7000'
$opts = array(
    'socket' => array(
        'bindto' => '0:7000',
    ),
);


// create the context...
$context = stream_context_create($opts);

// ...and use it to fetch the data
```

```php
echo file_get_contents('http://www.example.com', false, $context);

?>
```

HTTP context options

HTTP context options

HTTP context options — HTTP context option listing

Description

Context options for `http://` and `https://` transports.

Options

`method` string

> **`GET`**, **`POST`**, or any other HTTP method supported by the remote server.
>
> Defaults to **`GET`**.

`header` array or string

> Additional headers to be sent during request. Values in this option will override other values (such as `User-agent:`, `Host:`, and `Authentication:`), even when following `Location:` redirects. Thus it is not recommended to set a `Host:` header, if `follow_location` is enabled.
>
> String value should be `Key: value` pairs delimited by `\r\n`, e.g. `"Content-Type: application/json\r\nConnection: close"`. Array value should be a list of `Key: value` pairs, e.g. `["Content-Type: application/json", "Connection: close"]`.

`user_agent` string

> Value to send with `User-Agent:` header. This value will only be used if user-agent is *not* specified in the `header` context option above.
>
> By default the user_agent *php.ini* setting is used.

`content` string

> Additional data to be sent after the headers. Typically used with POST or PUT requests.

`proxy` string

> URI specifying address of proxy server (e.g. `tcp://proxy.example.com:5100`).

`request_fulluri` bool

> When set to **`true`**, the entire URI will be used when constructing the request (e.g. `GET http://www.example.com/path/to/file.html HTTP/1.0`). While this is a non-standard request format, some proxy servers require it.
>
> Defaults to **`false`**.

`follow_location` int

Follow `Location` header redirects. Set to `0` to disable.

When this option is not set, redirects are only followed for the `300`, `301`, `302`, `303`, `307` and `308` response codes. Setting it to `1` is not equivalent: any response carrying a `Location` header is then followed, whatever its response code.

`max_redirects` int

The max number of redirects to follow. Value `1` or less means that no redirects are followed.

Defaults to `20`.

`protocol_version` float

HTTP protocol version.

Defaults to `1.1` as of PHP 8.0.0; prior to that version the default was `1.0`.

`timeout` float

Read timeout in seconds, specified by a float (e.g. `10.5`).

By default the default_socket_timeout *php.ini* setting is used.

`ignore_errors` bool

Fetch the content even on failure status codes.

Defaults to **`false`**.

`uri_parser_class` string or null

The class to use for parsing URIs. When **`null`**, the legacy behavior based on parse_url() is used. Accepted values are Uri\Rfc3986\Uri (RFC 3986 compliant parser) and Uri\WhatWg\Url (WHATWG URL Standard parser). Custom userland implementations are not supported.

Defaults to **`null`**.

Changelog

Version	Description
8.5.0	The `uri_parser_class` option was added.

Examples

Example #1 Fetch a page and send POST data

```php
<?php

$postdata = http_build_query(
    [
        'var1' => 'some content',
        'var2' => 'doh',
    ]
);
```

```php
$opts = [
    'http' => [
        'method'  => 'POST',
        'header'  => 'Content-type: application/x-www-form-urlencoded',
        'content' => $postdata,
    ]
];

$context = stream_context_create($opts);

$result = file_get_contents('http://example.com/submit.php', false, $context);

?>
```

Example #2 Ignore redirects but fetch headers and content

```php
<?php

$url = "http://www.example.org/header.php";

$opts = [
    'http' => [
        'method'        => 'GET',
        'max_redirects' => '0',
        'ignore_errors' => '1',
    ]
];

$context = stream_context_create($opts);
$stream = fopen($url, 'r', false, $context);

// header information as well as meta data
// about the stream
var_dump(stream_get_meta_data($stream));

// actual data at $url
var_dump(stream_get_contents($stream));
fclose($stream);
?>
```

Notes

> **Note** : **Underlying socket stream context options**
>
> Additional context options may be supported by the underlying transport For `http://` streams, refer to context options for the `tcp://` transport. For `https://` streams, refer to context options for the `ssl://` transport.

> **Note** : **HTTP status line**
>
> When this stream wrapper follows a redirect, the `wrapper_data` returned by stream_get_meta_data() might not necessarily contain the HTTP status line that actually applies to the content data at index `0`.

```
array (
  'wrapper_data' =>
  array (
    0 => 'HTTP/1.0 301 Moved Permanently',
    1 => 'Cache-Control: no-cache',
    2 => 'Connection: close',
    3 => 'Location: http://example.com/foo.jpg',
    4 => 'HTTP/1.1 200 OK',
    ...
```

The first request returned a `301` (permanent redirect), so the stream wrapper automatically followed the redirect to get a `200` response (index = `4`).

See Also

- http://
- Socket context options
- SSL context options

FTP context options

FTP context options

FTP context options — FTP context option listing

Description

Context options for `ftp://` and `ftps://` transports.

Options

`overwrite` bool

Allow overwriting of already existing files on remote server. Applies to write mode (uploading) only.

Defaults to **`false`**.

`resume_pos` int

File offset at which to begin transfer. Applies to read mode (downloading) only.

Defaults to `0` (Beginning of File).

`proxy` string

Proxy FTP request via http proxy server. Applies to file read operations only. Ex: `tcp://squid.example.com:8000`.

Notes

> **Note** : **Underlying socket stream context options**
>
> Additional context options may be supported by the underlying transport. For `ftp://` streams, refer to context options for the `tcp://` transport. For `ftps://` streams, refer to context options for the `ssl://` transport.

See Also

- ftp://
- Socket context options
- SSL context options

SSL context options

SSL context options

SSL context options — SSL context option listing

Description

Context options for `ssl://` and `tls://` transports.

Options

`peer_name` string

> Peer name to be used. If this value is not set, then the name is guessed based on the hostname used when opening the stream.

`verify_peer` bool

> Require verification of SSL certificate used.
>
> Defaults to **`true`**.

`verify_peer_name` bool

> Require verification of peer name.
>
> Defaults to **`true`**.

`allow_self_signed` bool

> Allow self-signed certificates. Requires `verify_peer`.
>
> Defaults to **`false`**

`cafile` string

> Location of Certificate Authority file on local filesystem which should be used with the `verify_peer` context option to authenticate the identity of the remote peer.

`capath` string

If `cafile` is not specified or if the certificate is not found there, the directory pointed to by `capath` is searched for a suitable certificate. `capath` must be a correctly hashed certificate directory.

`local_cert` string

Path to local certificate file on filesystem. It must be a PEM encoded file which contains your certificate and private key. It can optionally contain the certificate chain of issuers. The private key also may be contained in a separate file specified by `local_pk`.

`local_pk` string

Path to local private key file on filesystem in case of separate files for certificate (`local_cert`) and private key.

`passphrase` string

Passphrase with which your `local_cert` file was encoded.

`verify_depth` int

Abort if the certificate chain is too deep.

Defaults to no verification.

`ciphers` string

Sets the list of available ciphers. The format of the string is described in » ciphers(1).

Defaults to `DEFAULT`.

`capture_peer_cert` bool

If set to **`true`** a `peer_certificate` context option will be created containing the peer certificate.

`capture_peer_cert_chain` bool

If set to **`true`** a `peer_certificate_chain` context option will be created containing the certificate chain.

`SNI_enabled` bool

If set to **`true`** server name indication will be enabled. Enabling SNI allows multiple certificates on the same IP address.

`SNI_server_certs` array

An array of server names and their corresponding certificates to be used for SNI. The keys are the server names and the values are the paths to the certificate files on the local filesystem. The certificate files must be PEM encoded and contain both the certificate and private key.

Available as of PHP 5.6.0.

`alpn_protocols` array

An array of application layer protocol names to be used for ALPN (Application-Layer Protocol Negotiation). The values are the protocol names as strings (e.g. "http/1.1", "h2").

Available as of PHP 7.0.0 and OpenSSL 1.0.2.

`no_ticket` bool

If set, disable TLS session tickets. This can help to enhance security by providing Perfect Forward Secrecy (PFS).

Available as of PHP 5.3.6.

`disable_compression` bool

If set, disable TLS compression. This can help mitigate the CRIME attack vector.

`peer_fingerprint` string | array

Aborts when the remote certificate digest doesn't match the specified hash.

When a string is used, the length will determine which hashing algorithm is applied, either "md5" (32) or "sha1" (40).

When an array is used, the keys indicate the hashing algorithm name and each corresponding value is the expected digest.

`security_level` int

Sets the security level. If not specified the library default security level is used. The security levels are described in » SSL_CTX_get_security_level(3).

Available as of PHP 7.2.0 and OpenSSL 1.1.0.

`min_proto_version` int

Sets the minimum protocol version allowed. If not specified the library default minimum protocol version is used. The protocol versions are described in » SSL_CTX_set_min_proto_version(3).

Available as of PHP 7.3.0 and OpenSSL 1.1.0.

`max_proto_version` int

Sets the maximum protocol version allowed. If not specified the library default maximum protocol version is used. The protocol versions are described in » SSL_CTX_set_max_proto_version(3).

Available as of PHP 7.3.0 and OpenSSL 1.1.0.

Changelog

Version	Description
7.3.0	Added `min_proto_version` and `max_proto_version`. Requires OpenSSL >= 1.1.0.
7.2.0	Added `security_level`. Requires OpenSSL >= 1.1.0.
7.0.0	Added `alpn_protocols`. Requires OpenSSL >= 1.0.2.
5.6.0	Added `SNI_server_certs`.
5.3.6	Added `no_ticket`.

Notes

> **Note** :
>
> Because `ssl://` is the underlying transport for the `https://` and `ftps://` wrappers, any context options which apply to `ssl://` also apply to `https://` and `ftps://`.

> **Note** :
>
> For SNI (Server Name Indication) to be available, then PHP must be compiled with OpenSSL 0.9.8j or greater. Use the **OPENSSL_TLSEXT_SERVER_NAME** to determine whether SNI is supported.

See Also

- Socket context options

Phar context options

Phar context options

Phar context options — Phar context option listing

Description

Context options for `phar://` wrapper.

Options

`compress` int

One of Phar compression constants.

`metadata` mixed

Phar metadata. See Phar::setMetadata().

See Also

- phar://
- Phar stream wrapper

Context parameters

Context parameters

Context parameters — Context parameter listing

Description

These `parameters` can be set on a `context` using the stream_context_set_params() function.

Parameters

`notification` callable

A callable to be called when an event occurs on a stream.

See stream_notification_callback for more details.

Zip context options

Zip context options

Zip context options — Zip context option listing

Description

Zip context options are available for `zip` wrappers.

Options

`password`

Used to specify password used for encrypted archive.

Changelog

Version	Description
7.2.0, PECL zip 1.14.0	Added `password`.

Examples

Example #1 Basic `password` usage example

```php
<?php
// Read encrypted archive
$opts = array(
    'zip' => array(
        'password' => 'secret',
    ),
);
// create the context...
$context = stream_context_create($opts);

// ...and use it to fetch the data
echo file_get_contents('zip://test.zip#test.txt', false, $context);

?>
```

Zlib context options

Zlib context options

Zlib context options — Zlib context option listing

Description

Zlib context options are available for `zlib` wrappers.

Options

`level`

Used to specify the compression level (`0` - `9`).

Changelog

Version	Description
7.3.0	`level` has been added.

Supported Protocols and Wrappers

PHP comes with many built-in wrappers for various URL-style protocols for use with the filesystem functions such as fopen(), copy(), file_exists() and filesize(). In addition to these wrappers, it is possible to register custom wrappers using the stream_wrapper_register() function.

> **Note** :
>
> The URL syntax used to describe a wrapper only supports the `scheme://...` syntax. The `scheme:/` and `scheme:` syntaxes are not supported.

Table of Contents

file://

file://

file:// — Accessing local filesystem

Description

`file://` is the default wrapper used with PHP and represents the local filesystem. When a relative path is specified (a path which does not begin with `/`, `\`, `\\`, or a Windows drive letter) the path provided will be applied against the current working directory. In many cases this is the directory in which the script resides unless it has been changed. Using the CLI SAPI, this defaults to the directory from which the script was called.

With some functions, such as fopen() and file_get_contents(), `include_path` may be optionally searched for relative paths as well.

Usage

- */path/to/file.ext*
- *relative/path/to/file.ext*
- *fileInCwd.ext*
- *C:/path/to/winfile.ext*
- *C:\path\to\winfile.ext*
- *\\smbserver\share\path\to\winfile.ext*

- *file:///path/to/file.ext*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Allows Reading	Yes
Allows Writing	Yes
Allows Appending	Yes
Allows Simultaneous Reading and Writing	Yes
Supports stat()	Yes
Supports unlink()	Yes
Supports rename()	Yes
Supports mkdir()	Yes
Supports rmdir()	Yes

http://

http://

https://

http:// -- https:// — Accessing HTTP(s) URLs

Description

Allows read-only access to files/resources via HTTP. By default, a HTTP 1.0 GET is used. A `Host:` header is sent with the request to handle name-based virtual hosts. If you have configured a user_agent string using your *php.ini* file or the stream context, it will also be included in the request.

The stream allows access to the *body* of the resource; the headers are stored in the *$http_response_header* variable.

If it's important to know the URL of the resource where your document came from (after all redirects have been processed), you'll need to process the series of response headers returned by the stream.

The from directive will be used for the `From:` header if set and not overwritten by the Context options and parameters.

Usage

- *http://example.com*
- *http://example.com/file.php?var1=val1&var2=val2*
- *http://user:password@example.com*
- *https://example.com*

- *https://example.com/file.php?var1=val1&var2=val2*
- *https://user:password@example.com*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	Yes
Allows Reading	Yes
Allows Writing	No
Allows Appending	No
Allows Simultaneous Reading and Writing	N/A
Supports stat()	No
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No

Examples

Example #1 Detecting which URL we ended up on after redirects

```php
<?php
$url = 'http://www.example.com/redirecting_page.php';

$fp = fopen($url, 'r');

$meta_data = stream_get_meta_data($fp);
foreach ($meta_data['wrapper_data'] as $response) {

    /* Were we redirected? */
    if (strtolower(substr($response, 0, 10)) == 'location: ') {

        /* update $url with where we were redirected to */
        $url = substr($response, 10);
    }

}

?>
```

Notes

> **Note** :
>
> HTTPS is only supported when the openssl extension is enabled.

HTTP connections are read-only; writing data or copying files to an HTTP resource is not supported.

Sending *POST* and *PUT* requests, for example, can be done with the help of HTTP Contexts.

See Also

- HTTP context options
- *$http_response_header*
- stream_get_meta_data() - Retrieves header/meta data from streams/file pointers

ftp://

ftp://

ftps://

ftp:// -- ftps:// — Accessing FTP(s) URLs

Description

Allows read access to existing files and creation of new files via FTP. If the server does not support passive mode ftp, the connection will fail.

You can open files for either reading or writing, but not both simultaneously. If the remote file already exists on the ftp server and you attempt to open it for writing but have not specified the context option `overwrite`, the connection will fail. If you need to overwrite existing files over ftp, specify the `overwrite` option in the context and open the file for writing. Alternatively, you can use the FTP extension.

If you have set the from directive in *php.ini*, then this value will be sent as the anonymous FTP password.

Usage

- *ftp://example.com/pub/file.txt*
- *ftp://user:password@example.com/pub/file.txt*
- *ftps://example.com/pub/file.txt*
- *ftps://user:password@example.com/pub/file.txt*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	Yes
Allows Reading	Yes
Allows Writing	Yes (new files/existing files with `overwrite`)
Allows Appending	Yes
Allows Simultaneous Reading and Writing	No
Supports stat()	filesize(), filemtime(), filetype(), file_exists(), is_file(), and is_dir() elements only.
Supports unlink()	Yes

Attribute	Supported
Supports rename()	Yes
Supports mkdir()	Yes
Supports rmdir()	Yes

Notes

> **Note** :
>
> FTPS is only supported when the openssl extension is enabled.
>
> If the server does not support SSL, then the connection falls back to regular unencrypted ftp.

> **Note** : **Appending**
>
> Files may be appended via the `ftp://` URL wrapper.

See Also

- FTP context options

php://

php://

php:// — Accessing various I/O streams

Description

PHP provides a number of miscellaneous I/O streams that allow access to PHP's own input and output streams, the standard input, output and error file descriptors, in-memory and disk-backed temporary file streams, and filters that can manipulate other file resources as they are read from and written to.

php://stdin, php://stdout and php://stderr

php://stdin, *php://stdout* and *php://stderr* allow direct access to the corresponding input or output stream of the PHP process. The stream references a duplicate file descriptor, so if you open *php://stdin* and later close it, you close only your copy of the descriptor-the actual stream referenced by **STDIN** is unaffected. It is recommended that you simply use the constants **STDIN**, **STDOUT** and **STDERR** instead of manually opening streams using these wrappers.

php://stdin is read-only, whereas *php://stdout* and *php://stderr* are write-only.

php://input

php://input is a read-only stream that allows you to read raw data from the request body. *php://input* is not available in POST requests with `enctype="multipart/form-data"` if enable_post_data_reading option is enabled.

php://output

php://output is a write-only stream that allows you to write to the output buffer mechanism in the same way as print and echo.

php://fd

php://fd allows direct access to the given file descriptor. For example, *php://fd/3* refers to file descriptor 3.

php://memory and php://temp

php://memory and *php://temp* are read-write streams that allow temporary data to be stored in a file-like wrapper. One difference between the two is that *php://memory* will always store its data in memory, whereas *php://temp* will use a temporary file once the amount of data stored hits a predefined limit (the default is 2 MB). The location of this temporary file is determined in the same way as the sys_get_temp_dir() function.

The memory limit of *php://temp* can be controlled by appending `/maxmemory:NN`, where `NN` is the maximum amount of data to keep in memory before using a temporary file, in bytes.

> **Caution**
>
> Some PHP extensions may require a standard IO stream, and may attempt to cast a given stream to a standard IO stream. This cast can fail for memory streams as it requires the C **fopencookie()** function to be available. This C function is *not* available on Windows.

php://filter

php://filter is a kind of meta-wrapper designed to permit the application of filters to a stream at the time of opening. This is useful with all-in-one file functions such as readfile(), file(), and file_get_contents() where there is otherwise no opportunity to apply a filter to the stream prior the contents being read.

The *php://filter* target takes the following parameters as part of its path. Multiple filter chains can be specified on one path. Please refer to the examples for specifics on using these parameters.

php://filter parameters

Name	Description
`resource=<stream to be filtered>`	This parameter is required. It specifies the stream that you would like to filter.
`read=<filter list to apply to read chain>`	This parameter is optional. One or more filter names can be provided here, separated by the pipe character (`|`).
`write=<filter list to apply to write chain>`	This parameter is optional. One or more filter names can be provided here, separated by the pipe character (`|`).
`<filter list to apply to both chains>`	Any filter lists which are not prefixed by `read=` or `write=` will be applied to both the read and write chains as appropriate.

Options

Wrapper Summary (for `php://filter`, refer to the summary of the wrapper being filtered)

Attribute	Supported
Restricted by allow_url_fopen	No
Restricted by allow_url_include	`php://input`, `php://stdin`, `php://memory` and `php://temp` only.

Attribute	Supported
Allows Reading	`php://stdin`, `php://input`, `php://fd`, `php://memory` and `php://temp` only.
Allows Writing	`php://stdout`, `php://stderr`, `php://output`, `php://fd`, `php://memory` and `php://temp` only.
Allows Appending	`php://stdout`, `php://stderr`, `php://output`, `php://fd`, `php://memory` and `php://temp` only. (Equivalent to writing)
Allows Simultaneous Reading and Writing	`php://fd`, `php://memory` and `php://temp` only.
Supports stat()	No. However, `php://memory` and `php://temp` support fstat().
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No
Supports stream_select()	`php://stdin`, `php://stdout`, `php://stderr`, `php://fd` and `php://temp` only.

Examples

Example #1 php://temp/maxmemory

This optional parameter allows setting the memory limit before *php://temp* starts using a temporary file.

```
<?php
// Set the limit to 5 MB.
$fiveMBs = 5 * 1024 * 1024;
$fp = fopen("php://temp/maxmemory:$fiveMBs", 'r+');

fputs($fp, "hello\n");

// Read what we have written.
rewind($fp);
echo stream_get_contents($fp);
?>
```

Example #2 php://filter/resource=<stream to be filtered>

This parameter must be located at the end of your *php://filter* specification and should point to the stream which you want filtered.

```
<?php
/* This is equivalent to simply:
  readfile("http://www.example.com");
  since no filters are actually specified */

readfile("php://filter/resource=http://www.example.com");
?>
```

Example #3 php://filter/read=<filter list to apply to read chain>

This parameter takes one or more filternames separated by the pipe character |.

```php
<?php
/* This will output the contents of
  www.example.com entirely in uppercase */
readfile("php://filter/read=string.toupper/resource=http://www.example.com");

/* This will do the same as above
  but will also ROT13 encode it */
readfile("php://filter/read=string.toupper|string.rot13/resource=http://www.example.com");
?>
```

Example #4 php://filter/write=<filter list to apply to write chain>

This parameter takes one or more filternames separated by the pipe character |.

```php
<?php
/* This will filter the string "Hello World"
  through the rot13 filter, then write to
  example.txt in the current directory */
file_put_contents("php://filter/write=string.rot13/resource=example.txt","Hello World");
?>
```

Example #5 php://memory and php://temp are not reusable

php://memory and *php://temp* are not reusable, i.e. after the streams have been closed there is no way to refer to them again.

```php
<?php
file_put_contents('php://memory', 'PHP');
echo file_get_contents('php://memory'); // prints nothing
```

Example #6 php://input to read JSON data from the request body

This example demonstrates how to read raw JSON data from POST, PUT and PATCH requests using *php://input*.

```php
<?php
$input = file_get_contents("php://input");
$json_array = json_decode(
  json: $input,
  associative: true,
  flags: JSON_THROW_ON_ERROR
);

echo "Received JSON data: ";
print_r($json_array);
?>
```

zlib://

zlib://

bzip2://

zip://

zlib:// -- bzip2:// -- zip:// — Compression Streams

Description

compress.zlib:// and *compress.bzip2://*

zlib: works like gzopen(), except that the stream can be used with fread() and the other filesystem functions. This is deprecated due to ambiguities with filenames containing ':' characters; use *compress.zlib://* instead.

compress.zlib:// and *compress.bzip2://* are equivalent to gzopen() and bzopen() respectively, and operate even on systems that do not support fopencookie.

ZIP extension registers *zip:* wrapper. As of PHP 7.2.0 and libzip 1.2.0+, support for the passwords for encrypted archives were added, allowing passwords to be supplied by stream contexts. Passwords can be set using the `'password'` stream context option.

Usage

- *compress.zlib://file.gz*
- *compress.bzip2://file.bz2*
- *zip://archive.zip#dir/file.txt*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Allows Reading	Yes
Allows Writing	Yes (except `zip://`)
Allows Appending	Yes (except `zip://`)
Allows Simultaneous Reading and Writing	No
Supports stat()	No, use the normal `file://` wrapper to stat compressed files.
Supports unlink()	No, use the normal `file://` wrapper to unlink compressed files.
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No

See Also

- Zlib context options

data://

data://

data:// — Data (RFC 2397)

Description

The *data:* (» RFC 2397) stream wrapper.

Usage

- *data://text/plain; base64,*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	Yes
Restricted by allow_url_include	Yes
Allows Reading	Yes
Allows Writing	No
Allows Appending	No
Allows Simultaneous Reading and Writing	No
Supports stat()	No
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No

Examples

Example #1 Print data:// contents

```
<?php
// prints "I love PHP"
echo file_get_contents('data://text/plain; base64,SSBsb3ZlIFBIUAo=');
?>
```

Example #2 Fetch the media type

```
<?php
$fp   = fopen('data://text/plain; base64,', 'r');
$meta = stream_get_meta_data($fp);

// prints "text/plain"
echo $meta['mediatype'];
?>
```

glob://

glob://

glob:// — Find pathnames matching pattern

Description

The *glob:* stream wrapper.

Usage

- *glob://*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Restricted by allow_url_include	No
Allows Reading	No
Allows Writing	No
Allows Appending	No
Allows Simultaneous Reading and Writing	No
Supports stat()	No
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No

Examples

Example #1 Basic usage

```php
<?php
// Loop over all *.php files in ext/spl/examples/ directory
// and print the filename and its size
$it = new DirectoryIterator("glob://ext/spl/examples/*.php");
foreach($it as $f) {
    printf("%s: %.1FK\n", $f->getFilename(), $f->getSize()/1024);
}
?>
```

```
tree.php: 1.0K
findregex.php: 0.6K
findfile.php: 0.7K
dba_dump.php: 0.9K
nocvsdir.php: 1.1K
phar_from_dir.php: 1.0K
ini_groups.php: 0.9K
directorytree.php: 0.9K
dba_array.php: 1.1K
class_tree.php: 1.8K
```

phar://

phar://

phar:// — PHP Archive

Description

The *phar://* stream wrapper. See Phar stream wrapper for a detailed description.

Usage

- *phar://*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Restricted by allow_url_include	No
Allows Reading	Yes
Allows Writing	Yes
Allows Appending	No
Allows Simultaneous Reading and Writing	Yes
Supports stat()	Yes
Supports unlink()	Yes
Supports rename()	Yes
Supports mkdir()	Yes
Supports rmdir()	Yes

See Also

- Phar context options

ssh2://

ssh2://

ssh2:// — Secure Shell 2

Description

ssh2.shell:// ssh2.exec:// ssh2.tunnel:// ssh2.sftp:// ssh2.scp:// (PECL)

> **Note** : **This wrapper is not enabled by default**

> In order to use the *ssh2.*://* wrappers, the » SSH2 extension available from » PECL must be installed.

In addition to accepting traditional URI login details, the ssh2 wrappers will also reuse open connections by passing the connection resource in the host portion of the URL.

Usage

- *ssh2.shell://user:pass@example.com:22/xterm*
- *ssh2.exec://user:pass@example.com:22/usr/local/bin/somecmd*
- *ssh2.tunnel://user:pass@example.com:22/192.168.0.1:14*
- *ssh2.sftp://user:pass@example.com:22/path/to/filename*

Options

Wrapper Summary

Attribute	ssh2.shell	ssh2.exec	ssh2.tunnel	ssh2.sftp	ssh2.scp
Restricted by allow_url_fopen	Yes	Yes	Yes	Yes	Yes
Allows Reading	Yes	Yes	Yes	Yes	Yes
Allows Writing	Yes	Yes	Yes	Yes	No
Allows Appending	No	No	No	Yes (When supported by server)	No
Allows Simultaneous Reading and Writing	Yes	Yes	Yes	Yes	No
Supports stat()	No	No	No	Yes	No
Supports unlink()	No	No	No	Yes	No
Supports rename()	No	No	No	Yes	No
Supports mkdir()	No	No	No	Yes	No
Supports rmdir()	No	No	No	Yes	No

Context options

Name	Usage	Default
`session`	Preconnected ssh2 resource to be reused	
`sftp`	Preallocated sftp resource to be reused	
`methods`	Key exchange, hostkey, cipher, compression, and MAC methods to use	
`callbacks`		
`username`	Username to connect as	
`password`	Password to use with password authentication	
`pubkey_file`	Name of public key file to use for authentication	
`privkey_file`	Name of private key file to use for authentication	
`env`	Associate array of environment variables to set	

Name	Usage	Default
`term`	Terminal emulation type to request when allocating a pty	
`term_width`	Width of terminal requested when allocating a pty	
`term_height`	Height of terminal requested when allocating a pty	
`term_units`	Units to use with term_width and term_height	`SSH2_TERM_UNIT_CHARS`

Examples

Example #1 Opening a stream from an active connection

```
<?php
$session = ssh2_connect('example.com', 22);
ssh2_auth_pubkey_file($session, 'username', '/home/username/.ssh/id_rsa.pub',
                                            '/home/username/.ssh/id_rsa', 'secret');
$stream = fopen("ssh2.tunnel://$session/remote.example.com:1234", 'r');
?>
```

Example #2 This *$session* variable must be kept available!

In order to use the *ssh2.*://$session* wrappers, the *$session* resource variable must be kept. The code below will not have the desired effect:

```
<?php
$session = ssh2_connect('example.com', 22);
ssh2_auth_pubkey_file($session, 'username', '/home/username/.ssh/id_rsa.pub',
                                            '/home/username/.ssh/id_rsa', 'secret');
$connection_string = "ssh2.sftp://$session/";
unset($session);
$stream = fopen($connection_string . "path/to/file", 'r');
?>
```

unset() closes the session, because *$connection_string* does not hold a reference to the *$session* variable, just a string cast derived from it. This also happens when the unset() is implicit because of leaving scope (like in a function).

rar://

rar://

rar:// — RAR

Description

The wrapper takes the url encoded path to the RAR archive (relative or absolute), an optional asterisk (*), an optional number sign (#) and an optional url encoded entry name, as stored in the archive. Specifying an entry name requires the number sign; a leading forward slash in the entry name is optional.

This wrapper can open both files and directories. When opening directories, the asterisk sign forces the directory entries names to be returned unencoded. If it's not specified, they will be returned url encoded – the reason for this is to allow the wrapper to be correctly used with built-in functionality like the RecursiveDirectoryIterator in the presence of file names that seem like url encoded data.

If the pound sign and the entry name part are not included, the root of the archive will be displayed. This differs from regular directories in that the resulting stream will not contain information such as the modification time, as the root directory is not stored in an individual entry in the archive. The usage of the wrapper with RecursiveDirectoryIterator requires the number sign to be included in the URL when accessing the root, so that the URLs of the children may be constructed correctly.

> **Note** : **This wrapper is not enabled by default**
>
> In order to use the *rar://* wrapper, the » rar extension available from » PECL must be installed.

rar:// Available since PECL rar 3.0.0

Usage

- *rar://<url encoded archive name>[*][#[<url encoded entry name>]]*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Restricted by allow_url_include	No
Allows Reading	Yes
Allows Writing	No
Allows Appending	No
Allows Simultaneous Reading and Writing	No
Supports stat()	Yes
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No

Context options

Name	Usage	Default
`open_password`	The password used to encrypt the headers of the archive, if any. WinRAR will encrypt all the files with the same password as the headers password when the later is present, so for archives with encrypted headers, `file_password` will be ignored.	
`file_password`	The password used to encrypt a file, if any. If the headers are also encrypted, this option will be ignored in favor of `open_password`. The reason there are two options is to cover the possibility of supporting archives with different headers and file passwords, should those archives arise. Note that if the archive does not have its headers encrypted, `open_password` will be ignored and this option must be used instead.	

Name	Usage	Default
volume_callback	A callback to determine the path of missing volumes. See RarArchive::open() for more information.	

Examples

Example #1 Traversing a RAR archive

```
<?php

class MyRecDirIt extends RecursiveDirectoryIterator {
    function current() {
        return rawurldecode($this->getSubPathName()) .
            (is_dir(parent::current())?" [DIR]":"");
    }
}

$f = "rar://" . rawurlencode(dirname(__FILE__)) .
    DIRECTORY_SEPARATOR . 'dirs_and_extra_headers.rar#';

$it = new RecursiveTreeIterator(new MyRecDirIt($f));

foreach ($it as $s) {
    echo $s, "\n";
}
?>
```

The above example will output something similar to:

```
|-allow_everyone_ni [DIR]
|-file1.txt
|-file2_ℵ.txt
|-with_streams.txt
\-ℵ [DIR]
  |-%2\ℵempty%2E [DIR]
  | \-%2\ℵempty%2E\file7.txt
  |-ℵ\empty [DIR]
  |-ℵ\file3.txt
  |-ℵ\file4_ℵ.txt
  \-2_ℵ\ℵ [DIR]
    |-2_ℵ\ℵ\file5.txt
    \-2_ℵ\ℵ\file6_ℵ.txt
```

Example #2 Opening an encrypted file (header encryption)

```
<?php
$stream = fopen("rar://" .
    rawurlencode(dirname(__FILE__)) . DIRECTORY_SEPARATOR .
    'encrypted_headers.rar' . '#encfile1.txt', "r", false,
```

```
    stream_context_create(
        array(
            'rar' =>
                array(
                    'open_password' => 'samplepassword'
                )
            )
        )
    );
var_dump(stream_get_contents($stream));
/* creation and last access date is opt-in in WinRAR, hence most
 * files don't have them */
var_dump(fstat($stream));
?>
```

The above example will output something similar to:

```
string(26) "Encrypted file 1 contents."
Array
(
    [0] => 0
    [1] => 0
    [2] => 33206
    [3] => 1
    [4] => 0
    [5] => 0
    [6] => 0
    [7] => 26
    [8] => 0
    [9] => 1259550052
    [10] => 0
    [11] => -1
    [12] => -1
    [dev] => 0
    [ino] => 0
    [mode] => 33206
    [nlink] => 1
    [uid] => 0
    [gid] => 0
    [rdev] => 0
    [size] => 26
    [atime] => 0
    [mtime] => 1259550052
    [ctime] => 0
    [blksize] => -1
    [blocks] => -1
)
```

ogg://

ogg://

ogg:// — Audio streams

Description

Files opened for reading via the *ogg://* wrapper are treated as compressed audio encoded using the `OGG/Vorbis` codec. Similarly, files opened for writing or appending via the *ogg://* wrapper are written as compressed audio data. stream_get_meta_data(), when used on an `OGG/Vorbis` file opened for reading will return various details about the stream including the `vendor` tag, any included `comments`, the number of `channels`, the sampling `rate`, and the encoding rate range described by: `bitrate_lower`, `bitrate_upper`, `bitrate_nominal`, and `bitrate_window`.

ogg:// (PECL)

> **Note** : **This wrapper is not enabled by default**
>
> In order to use the *ogg://* wrapper, the » OGG/Vorbis extension available from » PECL must be installed.

Usage

- *ogg://soundfile.ogg*
- *ogg:///path/to/soundfile.ogg*
- *ogg://http://www.example.com/path/to/soundstream.ogg*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Allows Reading	Yes
Allows Writing	Yes
Allows Appending	Yes
Allows Simultaneous Reading and Writing	No
Supports stat()	No
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No
Supports rmdir()	No

Context options

Name	Usage	Default	Mode
`pcm_mode`	PCM encoding to apply while reading, one of: **`OGGVORBIS_PCM_U8`**, **`OGGVORBIS_PCM_S8`**, **`OGGVORBIS_PCM_U16_BE`**, **`OGGVORBIS_PCM_S16_BE`**,	OGGVORBIS_PCM_S16_LE	Read

Name	Usage	Default	Mode
	OGGVORBIS_PCM_U16_LE, and **OGGVORBIS_PCM_S16_LE**. (8 vs 16 bit, signed or unsigned, big or little endian)		
rate	Sampling rate of input data, expressed in Hz	44100	Write/Append
bitrate	When given as an integer, the fixed bitrate at which to encode. (16000 to 131072) When given as a float, the variable bitrate quality to use. (-1.0 to 1.0)	128000	Write/Append
channels	The number of audio channels to encode, typically 1 (Mono), or 2 (Stereo). May range as high as 16.	2	Write/Append
comments	An array of string values to encode into the track header.		Write/Append

expect://

expect://

expect:// — Process Interaction Streams

Description

Streams opened via the *expect://* wrapper provide access to process's stdin, stdout and stderr via PTY.

> **Note** : **This wrapper is not enabled by default**
>
> In order to use the *expect://* wrapper, the » Expect extension available from » PECL must be installed.

expect:// (PECL)

Usage

- *expect://command*

Options

Wrapper Summary

Attribute	Supported
Restricted by allow_url_fopen	No
Allows Reading	Yes
Allows Writing	Yes
Allows Appending	Yes
Allows Simultaneous Reading and Writing	No
Supports stat()	No
Supports unlink()	No
Supports rename()	No
Supports mkdir()	No

Attribute	Supported
Supports rmdir()	No

List of Reserved Words

Table of Contents

The following is a listing of predefined identifiers in PHP. None of the identifiers listed here should be used as identifiers in any of your scripts unless explicitly noted otherwise. These lists include keywords and predefined variables, constant, and class names. These lists are neither exhaustive nor complete.

List of Keywords

These words have special meaning in PHP. Some of them represent things which look like functions, some look like constants, and so on - but they're not, really: they are language constructs. The following words cannot be used as constants, class names, or function names. They are, however, allowed as property, constant, and method names of classes, interfaces, traits and enums, except that `class` may not be used as constant name.

PHP Keywords

__halt_compiler()	abstract	and	array()	as
break	callable	case	catch	class
clone	const	continue	declare	default
die()	do	echo	else	elseif
empty()	enddeclare	endfor	endforeach	endif
endswitch	endwhile	eval()	exit()	extends
final	finally	fn (as of PHP 7.4)	for	foreach
function	global	goto	if	implements
include	include_once	instanceof	insteadof	interface
isset()	list()	match (as of PHP 8.0)	namespace	new
or	print	private	protected	public
readonly (as of PHP 8.1.0) *	require	require_once	return	static
switch	throw	trait	try	unset()
use	var	while	xor	yield
yield from				

* `readonly` may be used as function name.

Compile-time constants

__CLASS__	__DIR__	__FILE__	__FUNCTION__	__LINE__
__METHOD__	__PROPERTY__	__NAMESPACE__	__TRAIT__	

Predefined Classes

This section lists standard predefined classes. Miscellaneous extensions define other classes which are described in their reference.

Standard Defined Classes

These classes are defined in the standard set of functions included in the PHP build.

Directory

: Created by dir().

stdClass

: A generic empty class created as a result of typecasting to object or various standard functions.

__PHP_Incomplete_Class

: Possibly created by unserialize().

Exception

ErrorException

php_user_filter

Closure

: The predefined final class Closure is used for representing anonymous functions.

Generator

: The predefined final class Generator is used for representing generators.

ArithmeticError

AssertionError

DivisionByZeroError

Error

Throwable

ParseError

TypeError

Special classes

Following identifiers may not be used as a class name as they have special purpose.

self

: Current class.

static

: Current class in runtime.

parent

Parent class.

Predefined Constants

Core Predefined Constants

These constants are defined by the PHP core. This includes PHP, the Zend engine, and SAPI modules.

PHP_VERSION (string)

The current PHP version as a string in "major.minor.release[extra]" notation.

PHP_MAJOR_VERSION (int)

The current PHP "major" version as an integer (e.g., int(5) from version "5.2.7-extra").

PHP_MINOR_VERSION (int)

The current PHP "minor" version as an integer (e.g., int(2) from version "5.2.7-extra").

PHP_RELEASE_VERSION (int)

The current PHP "release" version as an integer (e.g., int(7) from version "5.2.7-extra").

PHP_VERSION_ID (int)

The current PHP version as an integer, useful for version comparisons (e.g., int(50207) from version "5.2.7-extra").

PHP_EXTRA_VERSION (string)

The current PHP "extra" version as a string (e.g., '-extra' from version "5.2.7-extra"). Often used by distribution vendors to indicate a package version.

PHP_BUILD_DATE (string)

The date and time when PHP was built, in `"M d Y H:i:s"` format. Available as of PHP 8.5.0.

PHP_BUILD_PROVIDER (string)

The provider who built PHP. Available as of PHP 8.5.0.

> **Note** :
>
> This constant may not be available in all builds of PHP. (e.g., not set when building from source without specifying a value during configuration).

ZEND_THREAD_SAFE (bool)

Indicates whether the current build of PHP is thread safe.

ZEND_DEBUG_BUILD (bool)

Indicates whether the current build of PHP is a debug build.

PHP_ZTS (bool) Alias of **ZEND_THREAD_SAFE**

Indicates whether the current build of PHP is thread safe.

PHP_DEBUG (bool) Alias of **ZEND_DEBUG_BUILD**

Indicates whether the current build of PHP is a debug build.

DEBUG_BACKTRACE_PROVIDE_OBJECT (int)

Populate the "object" index.

DEBUG_BACKTRACE_IGNORE_ARGS (int)

Don't include the argument information for functions in the stack trace.

PHP_MAXPATHLEN (int)

The maximum length of filenames (including path) supported by this build of PHP.

PHP_OS (string)

The operating system PHP was built for.

PHP_OS_FAMILY (string)

The operating system family PHP was built for. One of `'Windows'`, `'BSD'`, `'Darwin'`, `'Solaris'`, `'Linux'` or `'Unknown'`. Available as of PHP 7.2.0.

PHP_SAPI (string)

The Server API for this build of PHP. See also php_sapi_name().

PHP_EOL (string)

The correct 'End Of Line' symbol for this platform.

PHP_INT_MAX (int)

The largest integer supported in this build of PHP. Usually int(2147483647) in 32 bit systems and int(9223372036854775807) in 64 bit systems.

PHP_INT_MIN (int)

The smallest integer supported in this build of PHP. Usually int(-2147483648) in 32 bit systems and int(-9223372036854775808) in 64 bit systems. Usually, PHP_INT_MIN === ~PHP_INT_MAX.

PHP_INT_SIZE (int)

The size of an integer in bytes in this build of PHP.

PHP_FLOAT_DIG (int)

Number of decimal digits that can be rounded into a float and back without precision loss. Available as of PHP 7.2.0.

PHP_FLOAT_EPSILON (float)

Smallest representable positive number x, so that `x + 1.0 != 1.0`. Available as of PHP 7.2.0.

PHP_FLOAT_MIN (float)

Smallest representable *positive* floating point number. If you need the smallest representable *negative* floating point number, use `-` `PHP_FLOAT_MAX`. Available as of PHP 7.2.0.

PHP_FLOAT_MAX (float)

Largest representable floating point number. Available as of PHP 7.2.0.

DEFAULT_INCLUDE_PATH (string)

PEAR_INSTALL_DIR (string)

PEAR_EXTENSION_DIR (string)

PHP_EXTENSION_DIR (string)

The default directory where to look for dynamically loadable extensions (unless overridden by extension_dir). Defaults to **PHP_PREFIX** (or `PHP_PREFIX . "\\ext"` on Windows).

PHP_PREFIX (string)

The value **--prefix** was set to at configure. On Windows, it is the value **--with-prefix** was set to at configure.

PHP_BINDIR (string)

The value **--bindir** was set to at configure. On Windows, it is the value **--with-prefix** was set to at configure.

PHP_SBINDIR (string)

The value **--sbindir** was set to at configure. On Windows, it is the value **--with-prefix** was set to at configure. Available as of PHP 8.4.0.

PHP_BINARY (string)

Specifies the PHP binary path during script execution.

PHP_MANDIR (string)

Specifies where the manpages were installed into.

> **Note** :
>
> This constant is not present on Windows builds of PHP.

PHP_LIBDIR (string)

PHP_DATADIR (string)

PHP_SYSCONFDIR (string)

PHP_LOCALSTATEDIR (string)

PHP_CONFIG_FILE_PATH (string)

PHP_CONFIG_FILE_SCAN_DIR (string)

PHP_SHLIB_SUFFIX (string)

The build-platform's shared library suffix, such as "so" (most Unixes) or "dll" (Windows).

PHP_FD_SETSIZE (int)

The maximum number of file descriptors for select system calls. Available as of PHP 7.1.0.

E_ERROR (int)
E_WARNING (int)
E_PARSE (int)
E_NOTICE (int)
E_CORE_ERROR (int)
E_CORE_WARNING (int)
E_COMPILE_ERROR (int)
E_COMPILE_WARNING (int)
E_USER_ERROR (int)
E_USER_WARNING (int)
E_USER_NOTICE (int)
E_RECOVERABLE_ERROR (int)
E_DEPRECATED (int)
E_USER_DEPRECATED (int)
E_ALL (int)

Error reporting constants.

E_STRICT (int)

Error reporting constants. This constant is unused, and has been deprecated as of PHP 8.4.0.

__COMPILER_HALT_OFFSET__ (int)

true (bool)

See Booleans.

false (bool)

See Booleans.

null (null)

See Null.

PHP_WINDOWS_EVENT_CTRL_C (int)

A Windows CTRL +C event. Available as of PHP 7.4.0 (Windows only).

PHP_WINDOWS_EVENT_CTRL_BREAK (int)

A Windows CTRL +BREAK event. Available as of PHP 7.4.0 (Windows only).

PHP_CLI_PROCESS_TITLE (bool)

Indicates whether the setting and getting of the process title is available. Available only under the CLI SAPI.

STDERR (resource)

An already opened stream to `stderr`. Available only under the CLI SAPI.

STDIN (resource)

An already opened stream to `stdin`. Available only under the CLI SAPI.

STDOUT (resource)

An already opened stream to `stdout`. Available only under the CLI SAPI.

See also: Magic constants.

Standard Predefined Constants

All constants from core extensions are defined in PHP by default.

List of other reserved words

The following words cannot be used to name a class, interface, trait or enum. Prior to PHP 8.0, they are also prohibited from being used in namespaces.

Reserved words

parent	self	int	float
bool	string	true	false
null	void (as of PHP 7.1)	iterable (as of PHP 7.1)	object (as of PHP 7.2)
mixed (as of PHP 8.0)	never (as of PHP 8.1)	array (as of PHP 8.5)	callable (as of PHP 8.5)

The following list of words have had soft reservations placed on them. Whilst they may still be used as class, interface, trait and enum names, usage of them is highly discouraged since they may be used in future versions of PHP.

Soft reserved words

enum	resource	numeric	

Security

Introduction

PHP is a powerful language and the interpreter, whether included in a web server as a module or executed as a separate CGI binary, is able to access files, execute commands and open network connections on the server. These properties make anything run on a web server insecure by default. PHP is designed specifically to be a more secure language for writing CGI programs than Perl or C, and with correct selection of compile-time and runtime configuration options, and proper coding practices, it can give you exactly the combination of freedom and security you need.

As there are many different ways of utilizing PHP, there are many configuration options controlling its behaviour. A large selection of options guarantees you can use PHP for a lot of purposes, but it also means there are combinations of these options and server configurations that result in an insecure setup.

The configuration flexibility of PHP is equally rivalled by the code flexibility. PHP can be used to build complete server applications, with all the power of a shell user, or it can be used for simple server-side includes with little risk in a tightly controlled environment. How you build that environment, and how secure it is, is largely up to the PHP developer.

This chapter starts with some general security advice, explains the different configuration option combinations and the situations they can be safely used, and describes different considerations in coding for different levels of security.

General considerations

A completely secure system is a virtual impossibility, so an approach often used in the security profession is one of balancing risk and usability. If every variable submitted by a user required two forms of biometric validation (such as a retinal scan and a fingerprint), you would have an extremely high level of accountability. It would also take half an hour to fill out a fairly complex form, which would tend to encourage users to find ways of bypassing the security.

The best security is often unobtrusive enough to suit the requirements without the user being prevented from accomplishing their work, or over-burdening the code author with excessive complexity. Indeed, some security attacks are merely exploits of this kind of overly built security, which tends to erode over time.

A phrase worth remembering: A system is only as good as the weakest link in a chain. If all transactions are heavily logged based on time, location, transaction type, etc. but the user is only verified based on a single cookie, the validity of tying the users to the transaction log is severely weakened.

When testing, keep in mind that you will not be able to test all possibilities for even the simplest of pages. The input you may expect will be completely unrelated to the input given by a disgruntled employee, a cracker with months of time on their hands, or a housecat walking across the keyboard. This is why it's best to look at the code from a logical perspective, to discern where unexpected data can be introduced, and then follow how it is modified, reduced, or amplified.

The Internet is filled with people trying to make a name for themselves by breaking your code, crashing your site, posting inappropriate content, and otherwise making your day interesting. It doesn't matter if you have a

small or large site, you are a target by simply being online, by having a server that can be connected to. Many cracking programs do not discern by size, they simply trawl massive IP blocks looking for victims. Try not to become one.

Installed as CGI binary

Table of Contents

Possible attacks

Using PHP as a CGI binary is an option for setups that for some reason do not wish to integrate PHP as a module into server software (like Apache), or will use PHP with different kinds of CGI wrappers to create safe **chroot** and **setuid** environments for scripts. This setup usually involves installing executable **php** binary to the web server *cgi-bin* directory. CERT advisory » CA-96.11 recommends against placing any interpreters into *cgi-bin*. Even if the **php** binary can be used as a standalone interpreter, PHP is designed to prevent the attacks this setup makes possible:

- Accessing system files: *http://my.host/cgi-bin/php?/etc/passwd*

  The query information in a URL after the question mark (`?`) is passed as command line arguments to the interpreter by the CGI interface. Usually interpreters open and execute the file specified as the first argument on the command line.

  When invoked as a CGI binary, **php** refuses to interpret the command line arguments.

- Accessing any web document on server: *http://my.host/cgi-bin/php/secret/doc.html*

  The path information part of the URL after the PHP binary name, */secret/doc.html* is conventionally used to specify the name of the file to be opened and interpreted by the CGI program. Usually some web server configuration directives (Apache: `Action`) are used to redirect requests to documents like *http://my.host/secret/script.php* to the PHP interpreter. With this setup, the web server first checks the access permissions to the directory */secret*, and after that creates the redirected request *http://my.host/cgi-bin/php/secret/script.php*. Unfortunately, if the request is originally given in this form, no access checks are made by web server for file */secret/script.php*, but only for the */cgi-bin/php* file. This way any user able to access */cgi-bin/php* is able to access any protected document on the web server.

  In PHP, runtime configuration directives cgi.force_redirect, doc_root and user_dir can be used to prevent this attack, if the server document tree has any directories with access restrictions. See below for the full explanation of the different combinations.

Case 1: only public files served

If your server does not have any content that is not restricted by password or IP based access control, there is no need for these configuration options. If your web server does not allow you to do redirects, or the server does not have a way to communicate to the PHP binary that the request is a safely redirected request, you can enable the cgi.force_redirect ini directive. You still have to make sure your PHP scripts do not rely on one or another way of calling the script, neither by directly *http://my.host/cgi-bin/php/dir/script.php* nor by redirection *http://my.host/dir/script.php*.

Redirection can be configured in Apache by using `AddHandler` and `Action` directives (see below).

Case 2: using cgi.force_redirect

The configuration directive cgi.force_redirect prevents anyone from calling **php** directly with a URL like *http://my.host/cgi-bin/php/secretdir/script.php*. Instead, PHP will only parse in this mode if it has gone through a web server redirect rule.

Usually the redirection in the Apache configuration is done with the following directives:

```
Action php-script /cgi-bin/php
AddHandler php-script .php
```

This option has only been tested with the Apache web server, and relies on Apache to set the non-standard CGI environment variable *REDIRECT_STATUS* on redirected requests. If your web server does not support any way of telling if the request is direct or redirected, you cannot use this option and you must use one of the other ways of running the CGI version documented here.

Case 3: setting doc_root or user_dir

To include active content, like scripts and executables, in the web server document directories is sometimes considered an insecure practice. If, because of some configuration mistake, the scripts are not executed but displayed as regular HTML documents, this may result in leakage of intellectual property or security information like passwords. Therefore many sysadmins will prefer setting up another directory structure for scripts that are accessible only through the PHP CGI, and therefore always interpreted and not displayed as such.

Also if the method for making sure the requests are not redirected, as described in the previous section, is not available, it is necessary to set up a script doc_root that is different from web document root.

You can set the PHP script document root by the configuration directive doc_root in the configuration file, or you can set the environment variable *PHP_DOCUMENT_ROOT*. If it is set, the CGI version of PHP will always construct the file name to open with this `doc_root` and the path information in the request, so you can be sure no script is executed outside this directory (except for `user_dir` below).

Another option usable here is user_dir. When `user_dir` is unset, only thing controlling the opened file name is `doc_root`. Opening a URL like *http://my.host/~user/doc.php* does not result in opening a file under users home directory, but a file called *~user/doc.php* under `doc_root` (yes, a directory name starting with a tilde [~]).

If `user_dir` is set to for example *public_php*, a request like *http://my.host/~user/doc.php* will open a file called *doc.php* under the directory named *public_php* under the home directory of the user. If the home of the user is */home/user*, the file executed is */home/user/public_php/doc.php*.

`user_dir` expansion happens regardless of the `doc_root` setting, so you can control the document root and user directory access separately.

Case 4: PHP parser outside of web tree

A very secure option is to put the PHP parser binary somewhere outside of the web tree of files. In */usr/local/bin*, for example. The only real downside to this option is that you will now have to put a line similar to:

```
#!/usr/local/bin/php
```

as the first line of any file containing PHP tags. You will also need to make the file executable. That is, treat it exactly as you would treat any other CGI script written in Perl or sh or any other common scripting language which uses the `#!` shell-escape mechanism for launching itself.

To get PHP to handle *PATH_INFO* and *PATH_TRANSLATED* information correctly with this setup, the cgi.discard_path ini directive has to be enabled.

Installed as an Apache module

When PHP is used as an Apache module it inherits Apache's user permissions (typically those of the "nobody" user). This has several impacts on security and authorization. For example, if you are using PHP to access a database, unless that database has built-in access control, you will have to make the database accessible to the "nobody" user. This means a malicious script could access and modify the database, even without a username and password. It's entirely possible that a web spider could stumble across a database administrator's web page, and drop all of your databases. You can protect against this with Apache authorization, or you can design your own access model using LDAP, *.htaccess* files, etc. and include that code as part of your PHP scripts.

Often, once security is established to the point where the PHP user (in this case, the apache user) has very little risk attached to it, it is discovered that PHP is now prevented from writing any files to user directories. Or perhaps it has been prevented from accessing or changing databases. It has equally been secured from writing good and bad files, or entering good and bad database transactions.

A frequent security mistake made at this point is to allow apache root permissions, or to escalate apache's abilities in some other way.

Escalating the Apache user's permissions to root is extremely dangerous and may compromise the entire system, so sudo'ing, chroot'ing, or otherwise running as root should not be considered by those who are not security professionals.

There are some simpler solutions. By using open_basedir you can control and restrict what directories are allowed to be used for PHP. You can also set up apache-only areas, to restrict all web based activity to non-user, or non-system, files.

Session Security

It is important to keep your HTTP session management secure. Session related security is described in the Session Security section of Session module reference.

Filesystem Security

Table of Contents

PHP is subject to the security built into most server systems with respect to permissions on a file and directory basis. This allows you to control which files in the filesystem may be read. Care should be taken with any files which are world readable to ensure that they are safe for reading by all users who have access to that filesystem.

Since PHP was designed to allow user level access to the filesystem, it's entirely possible to write a PHP script that will allow you to read system files such as */etc/passwd*, modify your ethernet connections, send massive

printer jobs out, etc. This has some obvious implications, in that you need to ensure that the files that you read from and write to are the appropriate ones.

Consider the following script, where a user indicates that they'd like to delete a file in their home directory. This assumes a situation where a PHP web interface is regularly used for file management, so the Apache user is allowed to delete files in the user home directories.

Example #1 Poor variable checking leads to....

```
<?php

// Remove a file from the user's home directory
$username = $_POST['user_submitted_name'];
$userfile = $_POST['user_submitted_filename'];
$homedir  = "/home/$username";

unlink("$homedir/$userfile");

echo "The file has been deleted!";

?>
```

Since the username and the filename are postable from a user form, they can submit a username and a filename belonging to someone else, and delete it even if they're not supposed to be allowed to do so. In this case, you'd want to use some other form of authentication. Consider what could happen if the variables submitted were `"../etc/"` and `"passwd"`. The code would then effectively read:

Example #2 ... A filesystem attack

```
<?php

// Removes a file from anywhere on the hard drive that
// the PHP user has access to. If PHP has root access:
$username = $_POST['user_submitted_name']; // "../etc"
$userfile = $_POST['user_submitted_filename']; // "passwd"
$homedir  = "/home/$username"; // "/home/../etc"

unlink("$homedir/$userfile"); // "/home/../etc/passwd"

echo "The file has been deleted!";

?>
```

There are two important measures you should take to prevent these issues.

- Only allow limited permissions to the PHP web user binary.
- Check all variables which are submitted.

Here is an improved script:

Example #3 More secure file name checking

```
<?php

// Removes a file from the hard drive that
// the PHP user has access to.
```

```php
$username = $_SERVER['REMOTE_USER']; // using an authentication mechanism
$userfile = basename($_POST['user_submitted_filename']);
$homedir  = "/home/$username";

$filepath = "$homedir/$userfile";

if (file_exists($filepath) && unlink($filepath)) {
    $logstring = "Deleted $filepath\n";
} else {
    $logstring = "Failed to delete $filepath\n";
}

$fp = fopen("/home/logging/filedelete.log", "a");
fwrite($fp, $logstring);
fclose($fp);

echo htmlentities($logstring, ENT_QUOTES);

?>
```

However, even this is not without its flaws. If your authentication system allowed users to create their own user logins, and a user chose the login `"../etc/"`, the system is once again exposed. For this reason, you may prefer to write a more customized check:

Example #4 More secure file name checking

```php
<?php

$username     = $_SERVER['REMOTE_USER']; // using an authentication mechanisim
$userfile     = $_POST['user_submitted_filename'];
$homedir      = "/home/$username";

$filepath     = "$homedir/$userfile";

if (!ctype_alnum($username) || !preg_match('/^(?:[a-z0-9_-]|\.(?!\.))+$/iD', $userfile)) {
    die("Bad username/filename");
}

// etc.

?>
```

Depending on your operating system, there are a wide variety of files which you should be concerned about, including device entries (*/dev/* or *COM1*), configuration files (*/etc/* files and the `.ini` files), well known file storage areas (*/home/*, *My Documents*), etc. For this reason, it's usually easier to create a policy where you forbid everything except for what you explicitly allow.

Null bytes related issues

As PHP uses the underlying C functions for filesystem related operations, it may handle null bytes in a quite unexpected way. As null bytes denote the end of a string in C, strings containing them won't be considered entirely but rather only until a null byte occurs. The following example shows a vulnerable code that demonstrates this problem:

Example #1 Script vulnerable to null bytes

```php
<?php

$file = $_GET['file']; // "../../etc/passwd\0"

if (file_exists('/home/wwwrun/' . $file . '.php')) {
    // File_exists will return true as the file /home/wwwrun/../../etc/passwd exists
    include '/home/wwwrun/' . $file . '.php';

    // The file /etc/passwd will be included
}

?>
```

Therefore, any tainted string that is used in a filesystem operation should always be validated properly. Here is a better version of the previous example:

Example #2 Correctly validating the input

```php
<?php

$file = $_GET['file'];

// Whitelisting possible values
switch ($file) {
    case 'main':
    case 'foo':
    case 'bar':
        include '/home/wwwrun/include/' . $file . '.php';
        break;
    default:
        include '/home/wwwrun/include/main.php';
}

?>
```

Database Security

Table of Contents

Nowadays, databases are cardinal components of any web based application by enabling websites to provide varying dynamic content. Since very sensitive or secret information can be stored in a database, you should strongly consider protecting your databases.

To retrieve or to store any information you need to connect to the database, send a legitimate query, fetch the result, and close the connection. Nowadays, the commonly used query language in this interaction is the Structured Query Language (SQL). See how an attacker can tamper with an SQL query.

As you can surmise, PHP cannot protect your database by itself. The following sections aim to be an introduction into the very basics of how to access and manipulate databases within PHP scripts.

Keep in mind this simple rule: defense in depth. The more places you take action to increase the protection of your database, the less probability of an attacker succeeding in exposing or abusing any stored information. Good design of the database schema and the application deals with your greatest fears.

Designing Databases

The first step is always to create the database, unless you want to use one from a third party. When a database is created, it is assigned to an owner, who executed the creation statement. Usually, only the owner (or a superuser) can do anything with the objects in that database, and in order to allow other users to use it, privileges must be granted.

Applications should never connect to the database as its owner or a superuser, because these users can execute any query at will, for example, modifying the schema (e.g. dropping tables) or deleting its entire content.

You may create different database users for every aspect of your application with very limited rights to database objects. The most required privileges should be granted only, and avoid that the same user can interact with the database in different use cases. This means that if intruders gain access to your database using your applications credentials, they can only effect as many changes as your application can.

Connecting to Database

You may want to establish the connections over SSL to encrypt client/server communications for increased security, or you can use ssh to encrypt the network connection between clients and the database server. If either of these is used, then monitoring your traffic and gaining information about your database will be difficult for a would-be attacker.

Encrypted Storage Model

SSL/SSH protects data travelling from the client to the server: SSL/SSH does not protect persistent data stored in a database. SSL is an on-the-wire protocol.

Once an attacker gains access to your database directly (bypassing the webserver), stored sensitive data may be exposed or misused, unless the information is protected by the database itself. Encrypting the data is a good way to mitigate this threat, but very few databases offer this type of data encryption.

The easiest way to work around this problem is to first create your own encryption package, and then use it from within your PHP scripts. PHP can assist you in this with several extensions, such as OpenSSL and Sodium, covering a wide variety of encryption algorithms. The script encrypts the data before inserting it into the database, and decrypts it when retrieving. See the references for further examples of how encryption works.

Hashing

In the case of truly hidden data, if its raw representation is not needed (i.e. will not be displayed), hashing should be taken into consideration. The well-known example for hashing is storing the cryptographic hash of a password in a database, instead of the password itself.

The password functions provide a convenient way to hash sensitive data and work with these hashes.

password_hash() is used to hash a given string using the strongest algorithm currently available and password_verify() checks whether the given password matches the hash stored in database.

Example #1 Hashing password field

```
<?php

// storing password hash
```

```php
$query  = sprintf("INSERT INTO users(name,pwd) VALUES('%s','%s');",
            pg_escape_string($username),
            password_hash($password, PASSWORD_DEFAULT));
$result = pg_query($connection, $query);

// querying if user submitted the right password
$query = sprintf("SELECT pwd FROM users WHERE name='%s';",
            pg_escape_string($username));
$row = pg_fetch_assoc(pg_query($connection, $query));

if ($row && password_verify($password, $row['pwd'])) {
    echo 'Welcome, ' . htmlspecialchars($username) . '!';
} else {
    echo 'Authentication failed for ' . htmlspecialchars($username) . '.';
}

?>
```

SQL Injection

SQL injection is a technique where an attacker exploits flaws in application code responsible for building dynamic SQL queries. The attacker can gain access to privileged sections of the application, retrieve all information from the database, tamper with existing data, or even execute dangerous system-level commands on the database host. The vulnerability occurs when developers concatenate or interpolate arbitrary input in their SQL statements.

Example #1 Splitting the result set into pages ... and making superusers (PostgreSQL)

In the following example, user input is directly interpolated into the SQL query allowing the attacker to gain a superuser account in the database.

```php
<?php

$offset = $_GET['offset']; // beware, no input validation!
$query  = "SELECT id, name FROM products ORDER BY name LIMIT 20 OFFSET $offset;";
$result = pg_query($conn, $query);

?>
```

Normal users click on the 'next', 'prev' links where the *$offset* is encoded into the URL. The script expects that the incoming *$offset* is a number. However, what if someone tries to break in by appending the following to the URL

```
0;
insert into pg_shadow(usename,usesysid,usesuper,usecatupd,passwd)
    select 'crack', usesysid, 't','t','crack'
    from pg_shadow where usename='postgres';
--
```

If it happened, the script would present a superuser access to the attacker. Note that `0;` is to supply a valid offset to the original query and to terminate it.

> **Note** :

> It is a common technique to force the SQL parser to ignore the rest of the query written by the developer with `--` which is the comment sign in SQL.

A feasible way to gain passwords is to circumvent your search result pages. The only thing the attacker needs to do is to see if there are any submitted variables used in SQL statements which are not handled properly. These filters can be set commonly in a preceding form to customize `WHERE`, `ORDER BY`, `LIMIT` and `OFFSET` clauses in `SELECT` statements. If your database supports the `UNION` construct, the attacker may try to append an entire query to the original one to list passwords from an arbitrary table. It is strongly recommended to store only secure hashes of passwords instead of the passwords themselves.

Example #2 Listing out articles ... and some passwords (any database server)

```
<?php

$query  = "SELECT id, name, inserted, size FROM products
           WHERE size = '$size'";
$result = odbc_exec($conn, $query);

?>
```

The static part of the query can be combined with another `SELECT` statement which reveals all passwords:

```
'
union select '1', concat(uname||'-'||passwd) as name, '1971-01-01', '0' from usertable;
--
```

`UPDATE` and `INSERT` statements are also susceptible to such attacks.

Example #3 From resetting a password ... to gaining more privileges (any database server)

```
<?php
$query = "UPDATE usertable SET pwd='$pwd' WHERE uid='$uid';";
?>
```

If a malicious user submits the value `' or uid like'%admin%` to *$uid* to change the admin's password, or simply sets *$pwd* to `hehehe', trusted=100, admin='yes` to gain more privileges, then the query will be twisted:

```
<?php

// $uid: ' or uid like '%admin%
$query = "UPDATE usertable SET pwd='...' WHERE uid='' or uid like '%admin%';";

// $pwd: hehehe', trusted=100, admin='yes
$query = "UPDATE usertable SET pwd='hehehe', trusted=100, admin='yes' WHERE
...;";

?>
```

While it remains obvious that an attacker must possess at least some knowledge of the database architecture to conduct a successful attack, obtaining this information is often very simple. For example, the code may be part of an open-source software and be publicly available. This information may also be divulged by closed-source code - even if it's encoded, obfuscated, or compiled - and even by your own code through the display of error messages. Other methods include the use of typical table and column names. For example, a login form that uses a 'users' table with column names 'id', 'username', and 'password'.

Example #4 Attacking the database host operating system (MSSQL Server)

A frightening example of how operating system-level commands can be accessed on some database hosts.

```
<?php

$query  = "SELECT * FROM products WHERE id LIKE '%$prod%'";
$result = mssql_query($query);

?>
```

If attacker submits the value `a%' exec master..xp_cmdshell 'net user test testpass /ADD' --` to *$prod*, then the *$query* will be:

```
<?php

$query  = "SELECT * FROM products
           WHERE id LIKE '%a%'
           exec master..xp_cmdshell 'net user test testpass /ADD' --%'";
$result = mssql_query($query);

?>
```

MSSQL Server executes the SQL statements in the batch including a command to add a new user to the local accounts database. If this application were running as `sa` and the MSSQLSERVER service was running with sufficient privileges, the attacker would now have an account with which to access this machine.

> **Note** :
>
> The **mssql_query()** function belonged to the `mssql` extension, which was removed in PHP 7.0.0. It is used here only to illustrate the attack; current code connecting to MSSQL Server should use the sqlsrv or pdo_sqlsrv extensions instead.

> **Note** :
>
> Some examples above are tied to a specific database server, but it does not mean that a similar attack is impossible against other products. Your database server may be similarly vulnerable in another manner.

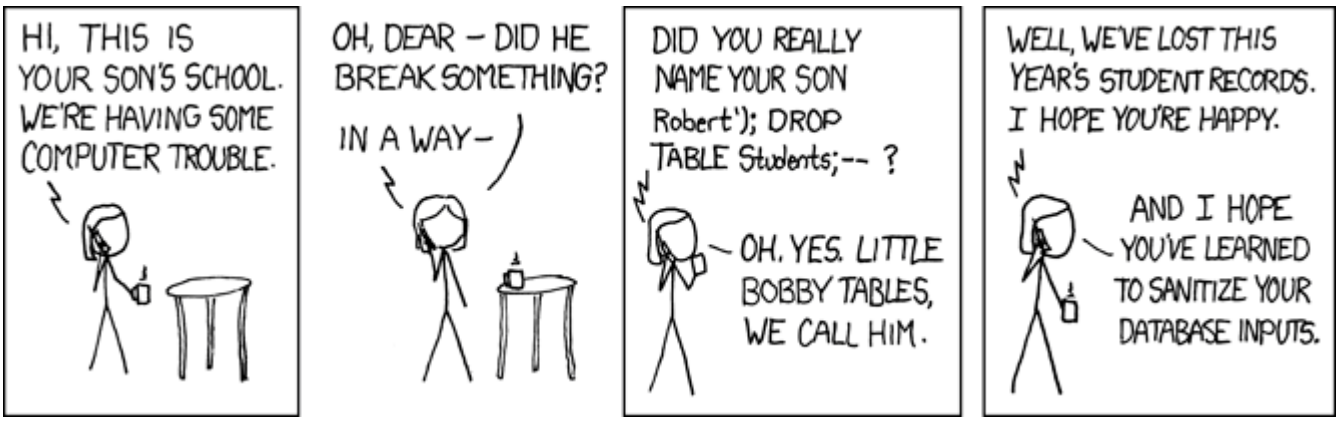


Image courtesy of » xkcd

Avoidance Techniques

The recommended way to avoid SQL injection is by binding all data via prepared statements. Using parameterized queries isn't enough to entirely avoid SQL injection, but it is the easiest and safest way to provide input to SQL statements. All dynamic data literals in `WHERE`, `SET`, and `VALUES` clauses must be replaced with placeholders. The actual data will be bound during the execution and sent separately from the SQL command.

Parameter binding can only be used for data. Other dynamic parts of the SQL query must be filtered against a known list of allowed values.

Example #5 Avoiding SQL injection by using PDO prepared statements

```php
<?php

// The dynamic SQL part is validated against expected values
$sortingOrder = $_GET['sortingOrder'] === 'DESC' ? 'DESC' : 'ASC';
$productId = $_GET['productId'];
// The SQL is prepared with a placeholder
$stmt = $pdo->prepare("SELECT * FROM products WHERE id LIKE ? ORDER BY price {$sortingOrder}");
// The value is provided with LIKE wildcards
$stmt->execute(["%{$productId}%"]);

?>
```

Prepared statements are provided by PDO, by MySQLi, and by other database libraries.

SQL injection attacks are mainly based on exploiting the code not being written with security in mind. Never trust any input, especially from the client side, even though it comes from a select box, a hidden input field, or a cookie. The first example shows that such a simple query can cause disasters.

A defense-in-depth strategy involves several good coding practices:

- Never connect to the database as a superuser or as the database owner. Use always customized users with minimal privileges.
- Check if the given input has the expected data type. PHP has a wide range of input validating functions, from the simplest ones found in Variable Functions and in Character Type Functions (e.g. is_numeric(), ctype_digit() respectively) and onwards to the Perl Compatible Regular Expressions support.
- If the application expects numerical input, consider verifying data with ctype_digit(), silently change its type using settype(), or use its numeric representation by sprintf().
- If the database layer doesn't support binding variables then quote each non-numeric user-supplied value that is passed to the database with the database-specific string escape function (e.g. mysql_real_escape_string(), **sqlite_escape_string()**, etc.). Generic functions like addslashes() are useful only in a very specific environment (e.g. MySQL in a single-byte character set with disabled *NO_BACKSLASH_ESCAPES*), so it is better to avoid them.
- Do not print out any database-specific information, especially about the schema, by fair means or foul. See also Error Reporting and Error Handling and Logging Functions.

Besides these, you benefit from logging queries either within your script or by the database itself, if it supports logging. Obviously, the logging is unable to prevent any harmful attempt, but it can be helpful to trace back which application has been circumvented. The log is not useful by itself but through the information it contains. More detail is generally better than less.

Error Reporting

With PHP security, there are two sides to error reporting. One is beneficial to increasing security, the other is detrimental.

A standard attack tactic involves profiling a system by feeding it improper data, and checking for the kinds, and contexts, of the errors which are returned. This allows the system cracker to probe for information about the server, to determine possible weaknesses. For example, if an attacker had gleaned information about a page based on a prior form submission, they may attempt to override variables, or modify them:

Example #1 Attacking Variables with a custom HTML page

```
<form method="post" action="attacktarget?username=badfoo&amp;password=badfoo">
<input type="hidden" name="username" value="badfoo" />
<input type="hidden" name="password" value="badfoo" />
</form>
```

The PHP errors which are normally returned can be quite helpful to a developer who is trying to debug a script, indicating such things as the function or file that failed, the PHP file it failed in, and the line number which the failure occurred in. This is all information that can be exploited. It is not uncommon for a php developer to use show_source(), highlight_string(), or highlight_file() as a debugging measure, but in a live site, this can expose hidden variables, unchecked syntax, and other dangerous information. Especially dangerous is running code from known sources with built-in debugging handlers, or using common debugging techniques. If the attacker can determine what general technique you are using, they may try to brute-force a page, by sending various common debugging strings:

Example #2 Exploiting common debugging variables

```
<form method="post" action="attacktarget?errors=Y&amp;showerrors=1&amp;debug=1">
<input type="hidden" name="errors" value="Y" />
<input type="hidden" name="showerrors" value="1" />
<input type="hidden" name="debug" value="1" />
</form>
```

Regardless of the method of error handling, the ability to probe a system for errors leads to providing an attacker with more information.

For example, the very style of a generic PHP error indicates a system is running PHP. If the attacker was looking at an `.html` page, and wanted to probe for the back-end (to look for known weaknesses in the system), by feeding it the wrong data they may be able to determine that a system was built with PHP.

A function error can indicate whether a system may be running a specific database engine, or give clues as to how a web page or programmed or designed. This allows for deeper investigation into open database ports, or to look for specific bugs or weaknesses in a web page. By feeding different pieces of bad data, for example, an attacker can determine the order of authentication in a script, (from the line number errors) as well as probe for exploits that may be exploited in different locations in the script.

A filesystem or general PHP error can indicate what permissions the web server has, as well as the structure and organization of files on the web server. Developer written error code can aggravate this problem, leading to easy exploitation of formerly "hidden" information.

There are three major solutions to this issue. The first is to scrutinize all functions, and attempt to compensate for the bulk of the errors. The second is to disable error reporting entirely on the running code. The third is to use PHP's custom error handling functions to create your own error handler. Depending on your security policy, you may find all three to be applicable to your situation.

One way of catching this issue ahead of time is to make use of PHP's own error_reporting(), to help you secure your code and find variable usage that may be dangerous. By testing your code, prior to deployment, with **E_ALL**, you can quickly find areas where your variables may be open to poisoning or modification in other ways. Once you are ready for deployment, you should either disable error reporting completely by setting error_reporting() to 0, or turn off the error display using the *php.ini* option `display_errors`, to insulate your code from probing. If you choose to do the latter, you should also define the path to your log file using the `error_log` ini directive, and turn `log_errors` on.

Example #3 Finding dangerous variables with E_ALL

```
<?php
if ($username) {  // Not initialized or checked before usage
    $good_login = 1;
}
if ($good_login == 1) { // If above test fails, not initialized or checked before usage
    readfile ("/highly/sensitive/data/index.html");
}
?>
```

User Submitted Data

The greatest weakness in many PHP programs is not inherent in the language itself, but merely an issue of code not being written with security in mind. For this reason, you should always take the time to consider the implications of a given piece of code, to ascertain the possible damage if an unexpected variable is submitted to it.

Example #1 Dangerous Variable Usage

```
<?php
// remove a file from the user's home directory... or maybe
// somebody else's?
unlink ($evil_var);

// Write logging of their access... or maybe an /etc/passwd entry?
fwrite ($fp, $evil_var);

// Execute something trivial.. or rm -rf *?
system ($evil_var);
exec ($evil_var);

?>
```

You should always carefully examine your code to make sure that any variables being submitted from a web browser are being properly checked, and ask yourself the following questions:

- Will this script only affect the intended files?
- Can unusual or undesirable data be acted upon?

- Can this script be used in unintended ways?
- Can this be used in conjunction with other scripts in a negative manner?
- Will any transactions be adequately logged?

By adequately asking these questions while writing the script, rather than later, you prevent an unfortunate re-write when you need to increase your security. By starting out with this mindset, you won't guarantee the security of your system, but you can help improve it.

Improve security by disabling convenience settings that obscure input data's origin, validity, or integrity. Implicit variable creation and unchecked input can lead to vulnerabilities like injection attacks and data manipulation.

Features like `register_globals` and `magic_quotes` (both removed in PHP 5.4.0) once contributed to these risks by automatically creating variables from user input and escaping data inconsistently. While no longer in PHP, similar risks persist if input handling is mismanaged.

Enable error_reporting(E_ALL) to help detect uninitialized variables and validate input. Use strict types (declare(strict_types=1), introduced in PHP 7) to enforce type safety, prevent unintended type conversions, and improving overall security.

Hiding PHP

In general, security by obscurity is one of the weakest forms of security. But in some cases, every little bit of extra security is desirable.

A few simple techniques can help to hide PHP, possibly slowing down an attacker who is attempting to discover weaknesses in your system. By setting expose_php to `off` in your *php.ini* file, you reduce the amount of information available to them.

Another tactic is to configure web servers such as apache to parse different filetypes through PHP, either with an *.htaccess* directive, or in the apache configuration file itself. You can then use misleading file extensions:

Example #1 Hiding PHP as another language

```
# Make PHP code look like other code types
AddType application/x-httpd-php .asp .py .pl
```

Or obscure it completely:

Example #2 Using unknown types for PHP extensions

```
# Make PHP code look like unknown types
AddType application/x-httpd-php .bop .foo .133t
```

Or hide it as HTML code, which has a slight performance hit because all HTML will be parsed through the PHP engine:

Example #3 Using HTML types for PHP extensions

```
# Make all PHP code look like HTML
AddType application/x-httpd-php .htm .html
```

For this to work effectively, you must rename your PHP files with the above extensions. While it is a form of security through obscurity, it's a minor preventative measure with few drawbacks.

Keeping Current

PHP, like any other large system, is under constant scrutiny and improvement. Each new version will often include both major and minor changes to enhance security and repair any flaws, configuration mishaps, and other issues that will affect the overall security and stability of your system.

Like other system-level scripting languages and programs, the best approach is to update often, and maintain awareness of the latest versions and their changes.

Features

HTTP authentication with PHP

It is possible to use the header() function to send an `"Authentication Required"` message to the client browser causing it to pop up a Username/Password input window. Once the user has filled in a username and a password, the URL containing the PHP script will be called again with the predefined variables *PHP_AUTH_USER*, *PHP_AUTH_PW*, and *AUTH_TYPE* set to the user name, password and authentication type respectively. These predefined variables are found in the *$_SERVER* array. *Only* the "Basic" authentication method is supported. See the header() function for more information.

An example script fragment which would force client authentication on a page is as follows:

Example #1 Basic HTTP Authentication example

```php
<?php
if (!isset($_SERVER['PHP_AUTH_USER'])) {
    header('HTTP/1.1 401 Unauthorized');
    header('WWW-Authenticate: Basic realm="My Realm"');
    echo 'Text to send if user hits Cancel button';
    exit;
} else {
    echo "<p>Hello {$_SERVER['PHP_AUTH_USER']}.</p>";
    echo "<p>You entered {$_SERVER['PHP_AUTH_PW']} as your password.</p>";
}
?>
```

> **Note** : **Compatibility**
>
> Please be careful when coding the HTTP header lines. In order to guarantee maximum compatibility with all clients, the keyword "Basic" should be written with an uppercase "B", the realm string must be enclosed in double (not single) quotes, and exactly one space should precede the *401* code in the *HTTP/1.1 401* header line. Authentication parameters have to be comma-separated.

Instead of simply printing out *PHP_AUTH_USER* and *PHP_AUTH_PW*, as done in the above example, you may want to check the username and password for validity. Perhaps by sending a query to a database, or by looking up the user in a dbm file.

> **Note** : **Apache Configuration**
>
> PHP uses the presence of an `AuthType` directive to determine whether external authentication is in effect.

Note, however, that the above does not prevent someone who controls a non-authenticated URL from stealing passwords from authenticated URLs on the same server.

> **Note** : **Browser behavior**

> HTTP Basic authentication really is basic, and it wasn't designed to support logouts. Because HTTP is a stateless protocol, most browsers will cache the provided credentials as soon as a `2xx` status code is seen, and will send them in every request, until the browser is closed. There is no defined way for a server to request a new prompt for credentials. Over the years, various workarounds for this have spread as advice on the internet, but they all depend on how different browsers have chosen to handle undefined edge cases (or even violations of the HTTP standard). It is best to avoid such workarounds and not use Basic authentication for anything serious.

> **Note** : **IIS Configuration**
>
> In order to get HTTP Authentication to work on IIS server with the CGI version of PHP, the php.ini directive cgi.rfc2616_headers must be set to `0` (the default value), and you must edit your IIS configuration "`Directory Security`". Click on "`Edit`" and only check "`Anonymous Access`", all other fields should be left unchecked.

Cookies

PHP transparently supports HTTP cookies. Cookies are a mechanism for storing data in the remote browser and thus tracking or identifying return users. You can set cookies using the setcookie() or setrawcookie() function. Cookies are part of the HTTP header, so setcookie() must be called before any output is sent to the browser. This is the same limitation that header() has. You can use the output buffering functions to delay the script output until you have decided whether or not to set any cookies or send any headers.

Any cookies sent to server from the client will automatically be included into a *$_COOKIE* auto-global array if variables_order contains "C". If you wish to assign multiple values to a single cookie, just add `[]` to the cookie name.

For more details, including notes on browser bugs, see the setcookie() and setrawcookie() function.

Sessions

Session support in PHP consists of a way to preserve certain data across subsequent accesses. This enables you to build more customized applications and increase the appeal of your web site. All information is in the Session reference section.

Handling file uploads

Table of Contents

POST method uploads

This feature lets people upload both text and binary files. With PHP's authentication and file manipulation functions, you have full control over who is allowed to upload and what is to be done with the file once it has been uploaded.

PHP is capable of receiving file uploads from any RFC-1867 compliant browser.

> **Note** : **Related Configurations Note**
>
> See also the file_uploads, upload_max_filesize, upload_tmp_dir, post_max_size and max_input_time directives in *php.ini*

PHP also supports PUT-method file uploads as used by Netscape Composer and W3C's Amaya clients. See the PUT Method Support for more details.

Example #1 File Upload Form

A file upload screen can be built by creating a special form which looks something like this:

```
<!-- The data encoding type, enctype, MUST be specified as below -->
<form enctype="multipart/form-data" action="__URL__" method="POST">
    <!-- MAX_FILE_SIZE must precede the file input field -->
    <input type="hidden" name="MAX_FILE_SIZE" value="30000" />
    <!-- Name of input element determines name in $_FILES array -->
    Send this file: <input name="userfile" type="file" />
    <input type="submit" value="Send File" />
</form>
```

The `__URL__` in the above example should be replaced, and point to a PHP file.

The `MAX_FILE_SIZE` hidden field (measured in bytes) must precede the file input field, and its value is the maximum filesize accepted by PHP. This form element should always be used as it saves users the trouble of waiting for a big file being transferred only to find that it was too large and the transfer failed. Keep in mind: fooling this setting on the browser side is quite easy, so never rely on files with a greater size being blocked by this feature. It is merely a convenience feature for users on the client side of the application. The PHP settings (on the server side) for maximum-size, however, cannot be fooled.

> **Note** :
>
> Be sure your file upload form has attribute `enctype="multipart/form-data"` otherwise the file upload will not work.

The global *$_FILES* will contain all the uploaded file information. Its contents from the example form is as follows. Note that this assumes the use of the file upload name *userfile*, as used in the example script above. This can be any name.

$_FILES['userfile']['name']

The original name of the file on the client machine.

$_FILES['userfile']['type']

The mime type of the file, if the browser provided this information. An example would be `"image/gif"`. This mime type is however not checked on the PHP side and therefore don't take its value for granted.

$_FILES['userfile']['size']

The size, in bytes, of the uploaded file.

$_FILES['userfile']['tmp_name']

The temporary filename of the file in which the uploaded file was stored on the server.

$_FILES['userfile']['error']

The error code associated with this file upload.

$_FILES['userfile']['full_path']

The full path as submitted by the browser. This value does not always contain a real directory structure, and cannot be trusted. Available as of PHP 8.1.0.

Files will, by default be stored in the server's default temporary directory, unless another location has been given with the upload_tmp_dir directive in *php.ini*. The server's default directory can be changed by setting the environment variable *TMPDIR* in the environment in which PHP runs. Setting it using putenv() from within a PHP script will not work. This environment variable can also be used to make sure that other operations are working on uploaded files, as well.

Example #2 Validating file uploads

See also the function entries for is_uploaded_file() and move_uploaded_file() for further information. The following example will process the file upload that came from a form.

```
<?php
$uploaddir = '/var/www/uploads/';
$uploadfile = $uploaddir . basename($_FILES['userfile']['name']);

echo '<pre>';
if (move_uploaded_file($_FILES['userfile']['tmp_name'], $uploadfile)) {
    echo "File is valid, and was successfully uploaded.\n";
} else {
    echo "Possible file upload attack!\n";
}

echo 'Here is some more debugging info:';
print_r($_FILES);

print "</pre>";

?>
```

The PHP script which receives the uploaded file should implement whatever logic is necessary for determining what should be done with the uploaded file. You can, for example, use the *$_FILES['userfile']['size']* variable to throw away any files that are either too small or too big. You could use the *$_FILES['userfile']['type']* variable to throw away any files that didn't match a certain type criteria, but use this only as first of a series of checks, because this value is completely under the control of the client and not checked on the PHP side. Also, you could use *$_FILES['userfile']['error']* and plan your logic according to the error codes. Whatever the logic, you should either delete the file from the temporary directory or move it elsewhere.

If no file is selected for upload in your form, PHP will return *$_FILES['userfile']['size']* as 0, and *$_FILES['userfile']['tmp_name']* as none.

The file will be deleted from the temporary directory at the end of the request if it has not been moved away or renamed.

Example #3 Uploading array of files

PHP supports HTML array feature even with files.

```
<form action="" method="post" enctype="multipart/form-data">
<p>Pictures:
<input type="file" name="pictures[]" />
<input type="file" name="pictures[]" />
<input type="file" name="pictures[]" />
<input type="submit" value="Send" />
</p>
</form>
```

```
<?php
foreach ($_FILES["pictures"]["error"] as $key => $error) {
    if ($error == UPLOAD_ERR_OK) {
        $tmp_name = $_FILES["pictures"]["tmp_name"][$key];
        // basename() may prevent filesystem traversal attacks;
        // further validation/sanitation of the filename may be appropriate
        $name = basename($_FILES["pictures"]["name"][$key]);
        move_uploaded_file($tmp_name, "data/$name");
    }
}
?>
```

File upload progress bar can be implemented using Session Upload Progress.

Error Messages Explained

PHP returns an appropriate error code along with the file array. The error code can be found in the `error` segment of the file array that is created during the file upload by PHP. In other words, the error might be found in *$_FILES['userfile']['error']*.

The value of this error code is one of the **`UPLOAD_ERR_*`** constants.

Common Pitfalls

The `MAX_FILE_SIZE` item cannot specify a file size greater than the file size that has been set in the upload_max_filesize in the *php.ini* file. The default is 2 megabytes.

If a memory limit is enabled, a larger memory_limit may be needed. Make sure you set memory_limit large enough.

If max_execution_time is set too small, script execution may be exceeded by the value. Make sure you set `max_execution_time` large enough.

> **Note** :
>
> max_execution_time only affects the execution time of the script itself. Any time spent on activity that happens outside the execution of the script such as system calls using system(), the sleep() function, database queries, time taken by the file upload process, etc. is not included when determining the maximum time that the script has been running.

> **Warning**
>
> max_input_time sets the maximum time, in seconds, the script is allowed to receive input; this includes file uploads. For large or multiple files, or users on slower connections, the default of `60` seconds may be exceeded.

If post_max_size is set too small, large files cannot be uploaded. Make sure you set `post_max_size` large enough.

The max_file_uploads configuration setting controls the maximum number of files that can uploaded in one request. If more files are uploaded than the limit, then *$_FILES* will stop processing files once the limit is reached. For example, if max_file_uploads is set to `10`, then *$_FILES* will never contain more than 10 items.

Not validating which file you operate on may mean that users can access sensitive information in other directories.

Due to the large amount of directory listing styles we cannot guarantee that files with exotic names (like containing spaces) are handled properly.

A developer may not mix normal `input` fields and file upload fields in the same form variable (by using an `input` name like `foo[]`).

Uploading multiple files

Multiple files can be uploaded using different `name` for `input`.

It is also possible to upload multiple files simultaneously and have the information organized automatically in arrays for you. To do so, you need to use the same array submission syntax in the HTML form as you do with multiple selects and checkboxes:

Example #1 Uploading multiple files

```
<form action="file-upload.php" method="post" enctype="multipart/form-data">
  Send these files:<br />
  <input name="userfile[]" type="file" /><br />
  <input name="userfile[]" type="file" /><br />
  <input type="submit" value="Send files" />
</form>
```

When the above form is submitted, the arrays *$_FILES['userfile']*, *$_FILES['userfile']['name']*, and *$_FILES['userfile']['size']* will be initialized.

For instance, assume that the filenames */home/test/review.html* and */home/test/xwp.out* are submitted. In this case, *$_FILES['userfile']['name'][0]* would contain the value *review.html*, and *$_FILES['userfile']['name'][1]* would contain the value *xwp.out*. Similarly, *$_FILES['userfile']['size'][0]* would contain *review.html*'s file size, and so forth.

$_FILES['userfile']['name'][0], *$_FILES['userfile']['tmp_name'][0]*, *$_FILES['userfile']['size'][0]*, and *$_FILES['userfile']['type'][0]* are also set.

> **Warning**

> The max_file_uploads configuration setting acts as a limit on the number of files that can be uploaded in one request. You will need to ensure that your form does not try to upload more files in one request than this limit.

Example #2 Uploading an entire directory

In HTML file upload fields, it is possible to upload an entire directory with the `webkitdirectory` attribute. This feature is supported in most modern browsers.

With the `full_path` information, it is possible to store the relative paths, or reconstruct the same directory in the server.

```
<form action="file-upload.php" method="post" enctype="multipart/form-data">
  Send this directory:<br />
  <input name="userfile[]" type="file" webkitdirectory multiple />
  <input type="submit" value="Send files" />
</form>
```

> **Warning**
>
> The `webkitdirectory` attribute is non-standard and is not on a standards track. Do not use it on production sites facing the Web: it will not work for every user. There may also be large incompatibilities between implementations and the behavior may change in the future.
>
> PHP only parses the relative path information submitted by the browser/user-agent, and passes that information to the *$_FILES* array. There is no guarantee that the values in the `full_path` array contains a real directory structure, and the PHP application must not trust this information.

PUT method support

PHP provides support for the HTTP PUT method used by some clients to store files on a server. PUT requests are much simpler than a file upload using POST requests and they look something like this:

```
PUT /path/filename.html HTTP/1.1
```

This would normally mean that the remote client would like to save the content that follows as: */path/filename.html* in your web tree. It is obviously not a good idea for Apache or PHP to automatically let everybody overwrite any files in your web tree. So, to handle such a request you have to first tell your web server that you want a certain PHP script to handle the request. In Apache you do this with the *Script* directive. It can be placed almost anywhere in your Apache configuration file. A common place is inside a `<Directory>` block or perhaps inside a `<VirtualHost>` block. A line like this would do the trick:

```
Script PUT /put.php
```

This tells Apache to send all PUT requests for URIs that match the context in which you put this line to the *put.php* script. This assumes, of course, that you have PHP enabled for the *.php* extension and PHP is active. The destination resource for all PUT requests to this script has to be the script itself, not a filename the uploaded file should have.

With PHP you would then do something like the following in your put.php. This would copy the contents of the uploaded file to the file *myputfile.ext* on the server. You would probably want to perform some checks and/or authenticate the user before performing this file copy.

Example #1 Saving HTTP PUT files

```
<?php
/* PUT data comes in on the stdin stream */
$putdata = fopen("php://input", "r");

/* Open a file for writing */
$fp = fopen("myputfile.ext", "w");

/* Read the data 1 KB at a time
   and write to the file */
while ($data = fread($putdata, 1024))
  fwrite($fp, $data);

/* Close the streams */
fclose($fp);
fclose($putdata);
?>
```

See Also

- Filesystem Security

Using remote files

As long as **allow_url_fopen** is enabled in *php.ini*, you can use HTTP and FTP URLs with most of the functions that take a filename as a parameter. In addition, URLs can be used with the include, include_once, require and require_once statements (**allow_url_include** must be enabled for these). See Supported Protocols and Wrappers for more information about the protocols supported by PHP.

For example, you can use this to open a file on a remote web server, parse the output for the data you want, and then use that data in a database query, or simply to output it in a style matching the rest of your website.

Example #1 Getting the title of a remote page

```
<?php
$file = fopen ("http://www.example.com/", "r");
if (!$file) {
    echo "<p>Unable to open remote file.\n";
    exit;
}
while (!feof ($file)) {
    $line = fgets ($file, 1024);
    /* This only works if the title and its tags are on one line */
    if (preg_match ("@\<title\>(.*)\</title\>@i", $line, $out)) {
        $title = $out[1];
        break;
    }
}
```

```
fclose($file);
?>
```

You can also write to files on an FTP server (provided that you have connected as a user with the correct access rights). You can only create new files using this method; if you try to overwrite a file that already exists, the fopen() call will fail.

To connect as a user other than 'anonymous', you need to specify the username (and possibly password) within the URL, such as '`ftp://user:password@ftp.example.com/path/to/file`'. (You can use the same sort of syntax to access files via HTTP when they require Basic authentication.)

Example #2 Storing data on a remote server

```
<?php
$file = fopen ("ftp://ftp.example.com/incoming/outputfile", "w");
if (!$file) {
    echo "<p>Unable to open remote file for writing.\n";
    exit;
}
/* Write the data here. */
fwrite ($file, $_SERVER['HTTP_USER_AGENT'] . "\n");
fclose ($file);
?>
```

> **Note** :
>
> You might get the idea from the example above that you can use this technique to write to a remote log file. Unfortunately that would not work because the fopen() call will fail if the remote file already exists. To do distributed logging like that, you should take a look at syslog().

Connection handling

Internally in PHP a connection status is maintained. There are 4 possible states:

- 0 - NORMAL
- 1 - ABORTED
- 2 - TIMEOUT
- 3 - ABORTED and TIMEOUT

When a PHP script is running normally, the NORMAL state is active. If the remote client disconnects, the ABORTED state flag is turned on. A remote client disconnect is usually caused by users hitting their STOP button. If the PHP-imposed time limit (see set_time_limit()) is hit, the TIMEOUT state flag is turned on.

You can decide whether or not you want a client disconnect to cause your script to be aborted. Sometimes it is handy to always have your scripts run to completion even if there is no remote browser receiving the output. The default behaviour is however for your script to be aborted when the remote client disconnects. This behaviour can be set via the ignore_user_abort *php.ini* directive as well as through the corresponding `php_value ignore_user_abort` Apache *httpd.conf* directive or with the ignore_user_abort() function. If you do not tell PHP to ignore a user abort and the user aborts, your script will terminate. The one exception is if you

have registered a shutdown function using register_shutdown_function(). With a shutdown function, when the remote user hits his STOP button, the next time your script tries to output something PHP will detect that the connection has been aborted and the shutdown function is called. This shutdown function will also get called at the end of your script terminating normally, so to do something different in case of a client disconnect you can use the connection_aborted() function. This function will return `true` if the connection was aborted.

Your script can also be terminated by the built-in script timer. The default timeout is 30 seconds. It can be changed using the **max_execution_time** *php.ini* directive or the corresponding `php_value max_execution_time` Apache *httpd.conf* directive as well as with the set_time_limit() function. When the timer expires the script will be aborted and as with the above client disconnect case, if a shutdown function has been registered it will be called. Within this shutdown function you can check to see if a timeout caused the shutdown function to be called by calling the connection_status() function. This function will return 2 if a timeout caused the shutdown function to be called.

One thing to note is that both the ABORTED and the TIMEOUT states can be active at the same time. This is possible if you tell PHP to ignore user aborts. PHP will still note the fact that a user may have broken the connection, but the script will keep running. If it then hits the time limit it will be aborted and your shutdown function, if any, will be called. At this point you will find that connection_status() returns 3.

Persistent Database Connections

What are Persistent Connections?

Persistent connections are links that do not close when the execution of the script ends. When a persistent connection is requested, PHP checks whether an identical persistent connection (that remained open from earlier) already exists; if one does, it is reused, and if not, a new link is created. An 'identical' connection is one opened to the same host with the same username and password (where applicable).

There is no way to request a specific connection, or to guarantee whether the returned connection will be an existing one or a brand new one (if all existing connections are in use, or the request is being served by a different worker, which has a separate pool of connections).

PHP's persistent connections therefore cannot be used to, for example:

- assign a specific database session to a specific web user
- create a large transaction across multiple requests
- initiate a query on one request and collect the results on another

Persistent connections do not provide *any* functionality that was not possible with non-persistent connections.

Web Requests

There are two ways in which a web server can utilize PHP to generate web pages:

The first method is to use PHP as a CGI "wrapper". When run this way, an instance of the PHP interpreter is created and destroyed for every page request (for a PHP page) to the web server. Because it is destroyed after every request, any resources that it acquires (such as a link to an SQL database server) are closed when it is destroyed. In this case, there is nothing to be gained from using persistent connections - they simply do not persist.

The second, and most popular, method is to run PHP-FPM, or PHP as a module in a multiprocess web server (currently only Apache). These setups typically have one process (the parent) which coordinates a set of

processes (its children) that actually do the work of serving up web pages. When a request comes in from a client, it is handed off to one of the children that is not already serving another client. This means that when the same client makes a second request to the server, it may be served by a different child process than the first time. Once a persistent connection has been opened, any subsequent page served by the same child process can reuse the already established connection to the SQL server.

> **Note** :
>
> The method in use can be checked by looking at the value of "Server API" in the output of phpinfo() or the value of `PHP_SAPI`, run from a web request.
>
> If the Server API is "Apache 2 Handler" or "FPM/FastCGI", then persistent connections will be used across requests served by the same worker. For any other value, persistent connections will not persist after each request.

Command-line Processes

As command-line PHP uses a new process for each script, persistent connections are not shared between command-line scripts, so there is no value in using them in transient scripts such as crons or commands. However, they may be useful, for example, in a long-running application server that serves many requests or tasks, each of which may need its own database connection.

Why Use Them?

Persistent connections are beneficial when the overhead of creating a link to an SQL server is high. Whether this overhead is significant depends on many factors, such as the type of database, whether it resides on the same machine as the web server, and how loaded that machine is. When the connection overhead is high, persistent connections can help considerably: each child process connects only once for its entire lifespan, rather than every time it processes a page that requires a connection to the SQL server. This means every child that opens a persistent connection will maintain its own connection to the server. For example, if 20 different child processes each run a script that makes a persistent connection to the SQL server, there will be 20 separate connections to that server, one from each child.

Potential Drawbacks: Connection Limits

Note, however, that this can have drawbacks when using a database with connection limits that are exceeded by persistent child connections. If the database has a limit of 16 simultaneous connections, and during a busy server session 17 child processes attempt to connect, one of them will fail. If there are bugs in the scripts that prevent connections from shutting down (such as infinite loops), a database with only 16 connections may be rapidly swamped.

Persistent connections will usually increase the number of connections open at any given time, because idle workers still hold on to the connections they opened for previous requests. If a large number of workers are spun up to handle a spike in traffic, the connections they opened will remain until the worker is terminated or the database server closes the connection.

Ensure that the maximum number of connections allowed by the database server is greater than the maximum number of web request workers (plus any other usage such as crons or administrative connections).

Check the database documentation for information on handling abandoned or idle connections (timeouts). Long timeouts may significantly increase the number of persistent connections open at any one time.

Potential Drawbacks: Maintaining Connection State

Some database extensions perform automatic cleanup when the connection is reused; others leave this task at the discretion of the application developer. Depending on the chosen database extension and the application design, manual cleanup may be needed before the script exits. Changes that may leave connections in an unexpected state include:

- Selected / default database
- Table locks
- Uncommitted transactions
- Temporary tables
- Connection specific settings or features such as profiling

Table locks and transactions that are not cleaned up or closed may cause other queries to be blocked indefinitely and/or cause subsequent reuse of the connection to cause unexpected changes.

Having the wrong database selected will cause subsequent reuse of the connection to be unable to execute queries as expected (or execute them on the wrong database if schemas are similar enough).

If temporary tables are not cleaned up, subsequent requests will not be able to recreate the same table.

Cleanup can be implemented using class destructors or register_shutdown_function(). Dedicated connection pooling proxies that include this as part of their functionality may also be considered.

Final Words

Given their behavior and potential drawbacks described above, persistent connections should not be used without careful consideration. They should not be used without implementing additional changes to the application and careful configuration of the database server and web server and/or PHP-FPM.

Consider alternative solutions such as investigating and fixing the causes of connection creation overheads (for example, disabling reverse DNS lookups on the database server), or dedicated connection pooling proxies.

For high volume web APIs, consider using alternative runtimes or long-running application servers.

See Also

- ibase_pconnect()
- oci_pconnect()
- odbc_pconnect()
- pfsockopen()
- pg_connect()
- MySQLi and Persistent Connections
- PDO Connection Management

Command line usage

Table of Contents

Introduction

The main focus of CLI SAPI is for developing shell applications with PHP. There are quite a few differences between the CLI SAPI and other SAPIs which are explained in this chapter. It is worth mentioning that CLI and CGI are different SAPIs although they do share many of the same behaviors.

The CLI SAPI is enabled by default using **--enable-cli**, but may be disabled using the **--disable-cli** option when running **./configure**.

The name, location and existence of the CLI/CGI binaries will differ depending on how PHP is installed on your system. By default when executing **make**, both the CGI and CLI are built and placed as *sapi/cgi/php-cgi* and *sapi/cli/php* respectively, in your PHP source directory. You will note that both are named *php*. What happens during **make install** depends on your configure line. If a module SAPI is chosen during configure, such as apxs, or the **--disable-cgi** option is used, the CLI is copied to *{PREFIX}/bin/php* during **make install** otherwise the CGI is placed there. So, for example, if **--with-apxs** is in your configure line then the CLI is copied to *{PREFIX}/bin/php* during **make install**. If you want to override the installation of the CGI binary, use **make install-cli** after **make install**. Alternatively you can specify **--disable-cgi** in your configure line.

> **Note** :
>
> Because both **--enable-cli** and **--enable-cgi** are enabled by default, simply having **--enable-cli** in your configure line does not necessarily mean the CLI will be copied as *{PREFIX}/bin/php* during **make install**.

The CLI binary is distributed in the main folder as *php.exe* on Windows. The CGI version is distributed as *php-cgi.exe*. Additionally, a *php-win.exe* is distributed if PHP is configured using **--enable-cli-win32**. This does the same as the CLI version, except that it doesn't output anything and thus provides no console.

> **Note** : **What SAPI do I have?**
>
> From a shell, typing **php -v** will tell you whether *php* is CGI or CLI. See also the function php_sapi_name() and the constant `PHP_SAPI`.

> **Note** :
>
> A Unix `manual` page is available by typing **man php** in the shell environment.

Differences to other SAPIs

Remarkable differences of the CLI SAPI compared to other SAPIs:

- Unlike the CGI SAPI, no headers are written to the output.

  Though the CGI SAPI provides a way to suppress HTTP headers, there's no equivalent switch to enable them in the CLI SAPI.

  CLI is started up in quiet mode by default, though the **-q** and **--no-header** switches are kept for compatibility so that it is possible to use older CGI scripts.

  It does not change the working directory to that of the script. (**-C** and **--no-chdir** switches kept for compatibility)

Plain text error messages (no HTML formatting).

- There are certain *php.ini* directives which are overridden by the CLI SAPI because they do not make sense in shell environments:

Overridden *php.ini* directives

Directive	CLI SAPI default value	Comment
html_errors	`false`	Defaults to `false`, as it can be quite hard to read error messages in the shell environment when they are cluttered up with uninterpreted HTML tags.
implicit_flush	`true`	In a shell environment, it is usually desirable for output, such as from print, echo and friends, to be displayed immediately, and not held in a buffer. Nonetheless, it is still possible to use output buffering to defer or manipulate standard output.
max_execution_time	0 (unlimited)	PHP in a shell environment tends to be used for a much more diverse range of purposes than typical Web-based scripts, and as these can be very long-running, the maximum execution time is set to unlimited.

register_argc_argv	`true`	Setting this to `true` means that scripts executed via the CLI SAPI always have access to *argc* (number of arguments passed to the application) and *argv* (array of the actual arguments).
		
		The PHP variables *$argc* and *$argv* are automatically set to the appropriate values when using the CLI SAPI. These values can also be found in the *$_SERVER* array, for example: *$_SERVER['argv']*.
		
		Warning
		
		The presence of *$argv* or *$_SERVER['argv']* is not a reliable indication that a script is being run from the command line because they may be set in other contexts when register_argc_argv is enabled. The value returned by php_sapi_name() should be checked instead.
		
		`<?php`
		
		`if (php_sapi_name() === 'cli') {`
		`    echo "This is being run from the command line!\n";`
		`}`
		
		Warning
		
		This feature has been *DEPRECATED* as of PHP 8.5.0. Relying on this feature is highly discouraged.

Directive	CLI SAPI default value	Comment
output_buffering	`false`	Although the *php.ini* setting is hardcoded to `false`, the Output buffering functions are available.
max_input_time	`false`	The PHP CLI does not support GET, POST or file uploads.

> **Note** :
>
> These directives cannot be initialized with another value from the configuration file *php.ini* or a custom one (if specified). This limitation is because the values are applied after all configuration files have been parsed. However, their values can be changed during runtime (although this is not sensible for all of them, such as register_argc_argv).

> **Note** :
>
> It is recommended to set ignore_user_abort for command line scripts. See ignore_user_abort() for more information.

- To ease working in the shell environment, a number of constants are defined for I/O streams .
- The CLI SAPI does **not** change the current directory to the directory of the executed script.

  Example #1 Example showing the difference to the CGI SAPI:

  ```
  <?php
  // Our simple test application named test.php
  echo getcwd(), "\n";
  ?>
  ```

  When using the CGI version, the output is:

  ```
  $ pwd
  /tmp

  $ php -q another_directory/test.php
  /tmp/another_directory
  ```

  This clearly shows that PHP changes its current directory to the one of the executed script.

  Using the CLI SAPI yields:

```
$ pwd
/tmp

$ php -f another_directory/test.php
/tmp
```

This allows greater flexibility when writing shell tools in PHP.

> **Note** :
>
> The CGI SAPI supports this CLI SAPI behaviour by means of the **-C** switch when run from the command line.

Options

The list of command line options provided by the PHP binary can be queried at any time by running PHP with the **-h** switch:

```
Usage: php [options] [-f] <file> [--] [args...]
   php [options] -r <code> [--] [args...]
   php [options] [-B <begin_code>] -R <code> [-E <end_code>] [--] [args...]
   php [options] [-B <begin_code>] -F <file> [-E <end_code>] [--] [args...]
   php [options] -- [args...]
   php [options] -a

  -a               Run interactively
  -c <path>|<file> Look for php.ini file in this directory
  -n               No php.ini file will be used
  -d foo[=bar]     Define INI entry foo with value 'bar'
  -e               Generate extended information for debugger/profiler
  -f <file>        Parse and execute <file>.
  -h               This help
  -i               PHP information
  -l               Syntax check only (lint)
  -m               Show compiled in modules
  -r <code>        Run PHP <code> without using script tags <?..?>
  -B <begin_code>  Run PHP <begin_code> before processing input lines
  -R <code>        Run PHP <code> for every input line
  -F <file>        Parse and execute <file> for every input line
  -E <end_code>    Run PHP <end_code> after processing all input lines
  -H               Hide any passed arguments from external tools.
  -S <addr>:<port> Run with built-in web server.
  -t <docroot>     Specify document root <docroot> for built-in web server.
  -s               Output HTML syntax highlighted source.
  -v               Version number
  -w               Output source with stripped comments and whitespace.
```

```
  args...          Arguments passed to script. Use -- args when first argument
                   starts with - or script is read from stdin

  --ini            Show configuration file names

  --rf <name>      Show information about function <name>.
  --rc <name>      Show information about class <name>.
  --re <name>      Show information about extension <name>.
  --rz <name>      Show information about Zend extension <name>.
  --ri <name>      Show configuration for extension <name>.
```

Command line options

Option	Long Option	Description
-a	--interactive	Run PHP interactively. For more information, see the Interactive shell section.
-b	--bindpath	Bind Path for external FASTCGI Server mode (CGI only).
-C	--no-chdir	Do not chdir to the script's directory (CGI only).
-q	--no-header	Quiet-mode. Suppress HTTP header output (CGI only).
-T	--timing	Measure execution time of script repeated *count* times (CGI only).

-c	--php-ini	Specifies either a directory in which to look for *php.ini*, or a custom `INI` file (which does not need to be named *php.ini*), e.g.:
		`$ php -c /custom/directory/ my_script.php`
		`$ php -c /custom/directory/custom-file.ini my_script.php`
		If this option is not specified, *php.ini* is searched for in the default locations.

-n	--no-php-ini	Ignore *php.ini* completely.
-d	--define	Set a custom value for any of the configuration directives allowed in *php.ini*. The syntax is:

Option	Long Option	Description

		`-d configuration_directive[=value]`
		
		Example #1 Example of using `-d` to set an INI setting
		
		``
		# Omitting the value part will set the given configuration directive to "1"
		$ php -d max_execution_time
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(1) "1"
		
		# Passing an empty value part will set the configuration directive to ""
		php -d max_execution_time=
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(0) ""
		
		# The configuration directive will be set to anything passed after the '=' character
		$ php -d max_execution_time=20
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(2) "20"
		$ php
		        -d max_execution_time=doesntmakesense
		        -r '$foo = ini_get("max_execution_time"); var_dump($foo);'
		string(15) "doesntmakesense"
		``

-e	--profile-info	Activate the extended information mode, to be used by a debugger/profiler.
-f	--file	Parse and execute the specified file. The **-f** is optional and may be omitted - providing just the filename to execute is sufficient.
-h and -?	--help and --usage	Output a list of command line options with one line descriptions of what they do.
-i	--info	Calls phpinfo(), and prints out the results. If PHP is not working correctly, it is advisable to use the command **php -i** and see whether any error messages are printed out before or in place of the information tables. Beware that when using the CGI mode the output is in HTML and therefore very large.
-l	--syntax-check	Syntax check but do not execute the given PHP code. The input from standard input will be processed if no filenames are specified, otherwise each filename will be checked. On success, the text `No syntax errors detected in <filename>` is written to standard output. On failure, the text `Errors parsing`

Option	Long Option	Description
		`<filename>` is written to standard output in addition to the internal parser error. If any failures are found in the specified files (or standard input), the shell return code is set to `-1`, otherwise the shell return code is set to `0`. This option won't find fatal errors (like undefined functions) that require executing the code. > **Note** : Prior to PHP 8.3.0, it was only possible to specify one filename to be checked. > **Note** : This option does not work together with the **-r** option.

-m	--modules	**Example #2 Printing built in (and loaded) PHP and Zend modules**
		
		``
		$ php -m
		[PHP Modules]
		xml
		tokenizer
		standard
		session
		posix
		pcre
		overload
		mysql
		mbstring
		ctype
		
		[Zend Modules]
		``

-r	--run	Allows execution of PHP included directly on the command line. The PHP start and end tags (`<?php` and `?>`) are **not needed** and will cause a parse error if present. > **Note** : Care must be taken when using this form of PHP not to collide with command line variable substitution done by the shell. **Example #3 Getting a syntax error when using double quotes**

Option	Long Option	Description

```
$ php -r "$foo = get_defined_constants();"
PHP Parse error:  syntax error, unexpected '=' in Command line
code on line 1

Parse error: syntax error, unexpected '=' in Command line code
on line 1
```

The problem here is that sh/bash performs variable substitution even when using double quotes ". Since the variable *$foo* is unlikely to be defined, it expands to nothing which results in the code passed to PHP for execution actually reading:

```
$ php -r " = get_defined_constants();"
```

The correct way would be to use single quotes '. Variables in single-quoted strings are not expanded by sh/bash.

Example #4 Using single quotes to prevent the shell's variable substitution

```
$ php -r '$foo = get_defined_constants(); var_dump($foo);'
array(370) {
  ["E_ERROR"]=>
  int(1)
  ["E_WARNING"]=>
  int(2)
  ["E_PARSE"]=>
  int(4)
  ["E_NOTICE"]=>
  int(8)
  ["E_CORE_ERROR"]=>
  [...]
```

If using a shell other than sh/bash, further issues might be experienced - if appropriate, a bug report should be opened at » https://github.com/php/php-src/issues. It is still easy to run into trouble when trying to use variables (shell or PHP) in command-line code, or using

Option	Long Option	Description
		backslashes for escaping, so take great care when doing so. You have been warned! **Note** : **-r** is available in the CLI SAPI, but not in the *CGI* SAPI. **Note** : This option is only intended for very basic code, so some configuration directives (such as auto_prepend_file and auto_append_file) are ignored in this mode.
-B	--process-begin	PHP code to execute before processing stdin.
-R	--process-code	PHP code to execute for every input line. There are two special variables available in this mode: *$argn* and *$argi*. *$argn* will contain the line PHP is processing at that moment, while *$argi* will contain the line number.
-F	--process-file	PHP file to execute for every input line.

-E	--process-end	PHP code to execute after processing the input.
		
		Example #5 Using the -B, -R and -E options to count the number of lines of a project.
		
		`$ find my_proj | php -B '$l=0;' -R '$l += count(@file($argn));' -E 'echo "Total Lines: $l\n";'`
		`Total Lines: 37328`

-S	--server	Starts built-in web server.
-t	--docroot	Specifies document root for built-in web server.
-s	--syntax-highlight and --syntax-highlighting	Display colour syntax highlighted source. This option uses the internal mechanism to parse the file and writes an HTML highlighted version of it to standard output. Note that all it does is generate a

Option	Long Option	Description
		block of `<code> [...] </code>` HTML tags, no HTML headers. > **Note** : > This option does not work together with the **-r** option.

-v	--version	**Example #6 Using -v to get the SAPI name and the version of PHP and Zend**
		
		``
		$ php -v
		PHP 5.3.1 (cli) (built: Dec 11 2009 19:55:07)
		Copyright (c) 1997-2009 The PHP Group
		Zend Engine v2.3.0, Copyright (c) 1998-2009 Zend Technologies
		``

-w	--strip	Display source with comments and whitespace stripped. > **Note** : > This option does not work together with the **-r** option.

	--ini	Show configuration file names and scanned directories. Optionally, pass `--ini=diff` to show the differences between the loaded configuration files and the default configuration.
		
		Example #7 `--ini` example
		
		``
		$ php --ini
		Configuration File (php.ini) Path: /usr/dev/php/5.2/lib
		Loaded Configuration File: /usr/dev/php/5.2/lib/php.ini
		Scan for additional .ini files in: (none)
		Additional .ini files parsed: (none)
		``
		
		> **Note** :
		> The `--ini=diff` option is not available prior to PHP 8.5.0.

--rf	--rfunction	Show information about the given function or class method (e.g. number and name of the parameters). This option is only available if PHP was compiled with Reflection support.

Option	Long Option	Description

		Example #8 basic `--rf` usage
		
		`$ php --rf var_dump`
		`Function [ <internal> public function var_dump ] {`
		
		`  - Parameters [2] {`
		`  Parameter #0 [ <required> $var ]`
		`  Parameter #1 [ <optional> $... ]`
		`  }`
		`}`

--rc	--rclass	Show information about the given class (list of constants, properties and methods).
		
		This option is only available if PHP was compiled with Reflection support.
		
		Example #9 `--rc` example
		
		`$ php --rc Directory`
		`Class [ <internal:standard> class Directory ] {`
		
		`  - Constants [0] {`
		`  }`
		
		`  - Static properties [0] {`
		`  }`
		
		`  - Static methods [0] {`
		`  }`
		
		`  - Properties [0] {`
		`  }`
		
		`  - Methods [3] {`
		`  Method [ <internal> public method close ] {`
		`  }`
		
		`  Method [ <internal> public method rewind ] {`
		`  }`
		
		`  Method [ <internal> public method read ] {`
		`  }`
		`  }`
		`}`

--re	--rextension	Show information about the given extension (list of *php.ini* options, defined functions, constants and classes). This option is only available if PHP was compiled with Reflection support. **Example #10 `--re` example**

Option	Long Option	Description

		`$ php --re json`
		`Extension [ <persistent> extension #19 json version 1.2.1 ] {`
		
		`  - Functions {`
		`  Function [ <internal> function json_encode ] {`
		`  }`
		`  Function [ <internal> function json_decode ] {`
		`  }`
		`  }`
		`}`

--rz	--rzendextension	Show the configuration information for the given Zend extension (the same information that is returned by phpinfo()).

--ri	--rextinfo	Show the configuration information for the given extension (the same information that is returned by phpinfo()). The core configuration information is available using "main" as extension name.
		
		Example #11 `--ri` example
		
		`$ php --ri date`
		
		`date`
		
		`date/time support => enabled`
		`"Olson" Timezone Database Version => 2009.20`
		`Timezone Database => internal`
		`Default timezone => Europe/Oslo`
		
		`Directive => Local Value => Master Value`
		`date.timezone => Europe/Oslo => Europe/Oslo`
		`date.default_latitude => 59.930972 => 59.930972`
		`date.default_longitude => 10.776699 => 10.776699`
		`date.sunset_zenith => 90.583333 => 90.583333`
		`date.sunrise_zenith => 90.583333 => 90.583333`

> **Note** :
>
> Options `-rBRFEH`, `--ini` and `--r[fcezi]` are available only in CLI.

Usage

There are three different ways of supplying the CLI SAPI with PHP code to be executed:

1. Tell PHP to execute a certain file.

```
$ php my_script.php

$ php -f my_script.php
```

Both ways (whether using the **-f** switch or not) execute the file *my_script.php*. Note that there is no restriction on which files can be executed; in particular, the filename is not required have a `.php` extension.

2. Pass the PHP code to execute directly on the command line.

```
$ php -r 'print_r(get_defined_constants());'
```

Special care has to be taken with regard to shell variable substitution and usage of quotes.

> **Note** :
>
> Read the example carefully: there are no beginning or ending tags! The **-r** switch simply does not need them, and using them will lead to a parse error.

3. Provide the PHP code to execute via standard input (`stdin`).

This gives the powerful ability to create PHP code dynamically and feed it to the binary, as shown in this (fictional) example:

```
$ some_application | some_filter | php | sort -u > final_output.txt
```

You cannot combine any of the three ways to execute code.

As with every shell application, the PHP binary accepts a number of arguments; however, the PHP script can also receive further arguments. The number of arguments that can be passed to your script is not limited by PHP (and although the shell has a limit to the number of characters which can be passed, this is not in general likely to be hit). The arguments passed to the script are available in the global array *$argv*. The first index (zero) always contains the name of the script as called from the command line. Note that, if the code is executed in-line using the command line switch **-r**, the value of *$argv[0]* will be `"Standard input code"`; prior to PHP 7.2.0, it was a dash (`"-"`) instead. The same is true if the code is executed via a pipe from **STDIN**.

A second global variable, *$argc*, contains the number of elements in the *$argv* array (**not** the number of arguments passed to the script).

As long as the arguments to be passed to the script do not start with the `-` character, there's nothing special to watch out for. Passing an argument to the script which starts with a `-` will cause trouble because the PHP interpreter thinks it has to handle it itself, even before executing the script. To prevent this, use the argument list separator `--`. After this separator has been parsed by PHP, every following argument is passed untouched to the script.

```
# This will not execute the given code but will show the PHP usage
$ php -r 'var_dump($argv);' -h
Usage: php [options] [-f] <file> [args...]
[...]

# This will pass the '-h' argument to the script and prevent PHP from showing its usage
$ php -r 'var_dump($argv);' -- -h
array(2) {
  [0]=>
  string(1) "-"
  [1]=>
  string(2) "-h"
}
```

However, on Unix systems there's another way of using PHP for shell scripting: make the first line of the script start with `#!/usr/bin/php` (or whatever the path to your PHP CLI binary is if different). The rest of the file should contain normal PHP code within the usual PHP starting and end tags. Once the execution attributes of the file are set appropriately (e.g. **chmod +x test**), the script can be executed like any other shell or perl script:

Example #1 Execute PHP script as shell script

```
#!/usr/bin/php
<?php
var_dump($argv);
?>
```

Assuming this file is named *test* in the current directory, it is now possible to do the following:

```
$ chmod +x test
$ ./test -h -- foo
array(4) {
  [0]=>
  string(6) "./test"
  [1]=>
  string(2) "-h"
  [2]=>
  string(2) "--"
  [3]=>
  string(3) "foo"
}
```

As can be seen, in this case no special care needs to be taken when passing parameters starting with `-`.

The PHP executable can be used to run PHP scripts absolutely independent of the web server. On Unix systems, the special `#!` (or "shebang") first line should be added to PHP scripts so that the system can automatically tell which program should run the script. On Windows platforms, it's possible to associate *php.exe* with the double click option of the `.php` extension, or a batch file can be created to run scripts through PHP. The special shebang first line for Unix does no harm on Windows (as it's formatted as a PHP comment), so cross platform programs can be written by including it. A simple example of writing a command line PHP program is shown below.

Example #2 Script intended to be run from command line (script.php)

```
#!/usr/bin/php
<?php

if ($argc != 2 || in_array($argv[1], array('--help', '-help', '-h', '-?'))) {
?>

This is a command line PHP script with one option.

  Usage:
  <?php echo $argv[0]; ?> <option>

  <option> can be some word you would like
  to print out. With the --help, -help, -h,
  or -? options, you can get this help.

<?php
} else {
    echo $argv[1];
}
?>
```

The script above includes the Unix shebang first line to indicate that this file should be run by PHP. We are working with a CLI version here, so no HTTP headers will be output.

The program first checks that there is the required one argument (in addition to the script name, which is also counted). If not, or if the argument was **--help**, **-help**, **-h** or **-?**, the help message is printed out, using *$argv[0]* to dynamically print the script name as typed on the command line. Otherwise, the argument is echoed out exactly as received.

To run the above script on Unix, it must be made executable, and called simply as **script.php echothis** or **script.php -h**. On Windows, a batch file similar to the following can be created for this task:

Example #3 Batch file to run a command line PHP script (script.bat)

```
@echo OFF
"C:\php\php.exe" script.php %*
```

Assuming the above program is named *script.php*, and the CLI *php.exe* is in *C:\php\php.exe*, this batch file will run it, passing on all appended options: **script.bat echothis** or **script.bat -h**.

See also the Readline extension documentation for more functions which can be used to enhance command line applications in PHP.

On Windows, PHP can be configured to run without the need to supply the *C:\php\php.exe* or the `.php` extension, as described in Command Line PHP on Microsoft Windows.

> **Note** :
>
> On Windows it is recommended to run PHP under an actual user account. When running under a network service certain operations will fail, because "No mapping between account names and security IDs was done".

I/O streams

The CLI SAPI defines a few constants for I/O streams to make programming for the command line a bit easier.

CLI specific Constants

Constant	Description

STDIN	An already opened stream to `stdin`. This saves opening it with
	`<?php`
	`$stdin = fopen('php://stdin', 'r');`
	`?>`
	If you want to read single line from `stdin`, you can use
	`<?php`
	`$line = trim(fgets(STDIN)); // reads one line from STDIN`
	`fscanf(STDIN, "%d\n", $number); // reads number from STDIN`
	`?>`

STDOUT	An already opened stream to `stdout`. This saves opening it with
	`<?php`
	`$stdout = fopen('php://stdout', 'w');`
	`?>`

STDERR	An already opened stream to `stderr`. This saves opening it with
	`<?php`
	`$stderr = fopen('php://stderr', 'w');`
	`?>`

Given the above, you don't need to open e.g. a stream for `stderr` yourself but simply use the constant instead of the stream resource:

```
php -r 'fwrite(STDERR, "stderr\n");'
```

You do not need to explicitly close these streams, as they are closed automatically by PHP when your script ends.

> **Note** :
>
> These constants are not available if reading the PHP script from `stdin`.

Interactive shell

The CLI SAPI provides an interactive shell using the **-a** option if PHP is compiled with the **--with-readline** option. As of PHP 7.1.0 the interactive shell is also available on Windows, if the readline extension is enabled.

Using the interactive shell you are able to type PHP code and have it executed directly.

Example #1 Executing code using the interactive shell

```
$ php -a
Interactive shell

php > echo 5+8;
13
php > function addTwo($n)
php > {
php { return $n + 2;
php { }
php > var_dump(addtwo(2));
int(4)
php >
```

The interactive shell also features tab completion for functions, constants, class names, variables, static method calls and class constants.

Example #2 Tab completion

Pressing the tab key twice when there are multiple possible completions will result in a list of these completions:

```
php > strp[TAB][TAB]
strpbrk   strpos    strptime
php > strp
```

When there is only one possible completion, pressing tab once will complete the rest on the same line:

```
php > strpt[TAB]ime(
```

Completion will also work for names that have been defined during the current interactive shell session:

```
php > $fooThisIsAReallyLongVariableName = 42;
php > $foo[TAB]ThisIsAReallyLongVariableName
```

The interactive shell stores your history which can be accessed using the up and down keys. The history is saved in the *~/.php_history* file. As of PHP 8.4.0, the path to the history file can be set using the *PHP_HISTFILE* environment variable.

The CLI SAPI provides the *php.ini* settings `cli.pager` and `cli.prompt`. The `cli.pager` setting allows an external program (such as *less*) to act as a pager for the output instead of being displayed directly on the screen. The `cli.prompt` setting makes it possible to change the `php >` prompt.

It is also possible to set *php.ini* settings in the interactive shell using a shorthand notation.

Example #3 Setting *php.ini* settings in the interactive shell

The `cli.prompt` setting:

```
php > #cli.prompt=hello world :>
hello world :>
```

Using backticks it is possible to have PHP code executed in the prompt:

```
php > #cli.prompt=`echo date('H:i:s');` php >
15:49:35 php > echo 'hi';
hi
15:49:43 php > sleep(2);
15:49:45 php >
```

Setting the pager to *less*:

```
php > #cli.pager=less
php > phpinfo();
(output displayed in less)
php >
```

The `cli.prompt` setting supports a few escape sequences:

`cli.prompt` escape sequences

Sequence	Description
`\e`	Used for adding colors to the prompt. An example could be `\e[032m\v \e[031m\b \e[34m\> \e[0m`
`\v`	The PHP version.
`\b`	Indicates which block PHP is in. For instance `/*` to indicate being inside a multi-line comment. The outer scope is denoted by `php`.
`\>`	Indicates the prompt character. By default this is `>`, but changes when the shell is inside an unterminated block or string. Possible characters are: `' " { ( >`

> **Note** :
>
> Files included through auto_prepend_file and auto_append_file are parsed in this mode but with some restrictions - e.g. functions have to be defined before called.

Interactive mode

If the readline extension is not available, prior to PHP 8.1.0, invoking the CLI SAPI with the **-a** option provided the interactive mode. In this mode, a complete PHP script is supposed to be given via STDIN, and after

termination with CTRL +D (POSIX) or CTRL +Z followed by +ENTER (Windows), this script is evaluated. This is basically the same as invoking the CLI SAPI without the **-a** option.

As of PHP 8.1.0, invoking the CLI SAPI with the **-a** option fails, if the readline extension is not available.

Built-in web server

> **Warning**
>
> This web server is designed to aid application development. It may also be useful for testing purposes or for application demonstrations that are run in controlled environments. It is not intended to be a full-featured web server. It should not be used on a public network.

The CLI SAPI provides a built-in web server.

The web server runs only one single-threaded process, so PHP applications will stall if a request is blocked.

URI requests are served from the current working directory where PHP was started, unless the -t option is used to specify an explicit document root. If a URI request does not specify a file, then either index.php or index.html in the given directory are returned. If neither file exists, the lookup for index.php and index.html will be continued in the parent directory and so on until one is found or the document root has been reached. If an index.php or index.html is found, it is returned and $_SERVER['PATH_INFO'] is set to the trailing part of the URI. Otherwise a 404 response code is returned.

> **Note** :
>
> As of PHP 8.4.0, this index file lookup is performed even when the requested path looks like a file, i.e. its last path component contains a period, but cannot be located. Previously, such requests skipped the lookup and immediately returned a 404 response.

If a PHP file is given on the command line when the web server is started it is treated as a "router" script. The script is run at the start of each HTTP request. If this script returns `false`, then the requested resource is returned as-is. Otherwise the script's output is returned to the browser.

Standard MIME types are returned for files with extensions: `.3gp`, `.apk`, `.avi`, `.bmp`, `.css`, `.csv`, `.doc`, `.docx`, `.flac`, `.gif`, `.gz`, `.gzip`, `.htm`, `.html`, `.ics`, `.jpe`, `.jpeg`, `.jpg`, `.js`, `.kml`, `.kmz`, `.m4a`, `.mov`, `.mp3`, `.mp4`, `.mpeg`, `.mpg`, `.odp`, `.ods`, `.odt`, `.oga`, `.ogg`, `.ogv`, `.pdf`, `.png`, `.pps`, `.pptx`, `.qt`, `.svg`, `.swf`, `.tar`, `.text`, `.tif`, `.txt`, `.wav`, `.webm`, `.wmv`, `.xls`, `.xlsx`, `.xml`, `.xsl`, `.xsd`, `.zip` .

As of PHP 7.4.0, the built-in webserver can be configured to fork multiple workers in order to test code that requires multiple concurrent requests to the built-in webserver. Set the *PHP_CLI_SERVER_WORKERS* environment variable to the number of desired workers before starting the server.

> **Note** :
>
> Multiple workers are not supported on Windows.

> **Note** :

> For information on PHP commandline usage and options, run **php --help** or **man php**. Not all options will apply when running the web server.

> **Warning**
>
> This *experimental* feature is *not* intended for production usage. Generally, the built-in Web Server is *not* intended for production usage.

Example #1 Starting the web server

```
$ cd ~/public_html
$ php -S localhost:8000
```

The terminal will show:

```
PHP 5.4.0 Development Server started at Thu Jul 21 10:43:28 2011
Listening on localhost:8000
Document root is /home/me/public_html
Press Ctrl-C to quit
```

After URI requests for http://localhost:8000/ and http://localhost:8000/myscript.html the terminal will show something similar to:

```
PHP 5.4.0 Development Server started at Thu Jul 21 10:43:28 2011
Listening on localhost:8000
Document root is /home/me/public_html
Press Ctrl-C to quit.
[Thu Jul 21 10:48:48 2011] ::1:39144 GET /favicon.ico - Request read
[Thu Jul 21 10:48:50 2011] ::1:39146 GET / - Request read
[Thu Jul 21 10:48:50 2011] ::1:39147 GET /favicon.ico - Request read
[Thu Jul 21 10:48:52 2011] ::1:39148 GET /myscript.html - Request read
[Thu Jul 21 10:48:52 2011] ::1:39149 GET /favicon.ico - Request read
```

Note that prior to PHP 7.4.0, symlinked statical resources have not been accessible on Windows, unless the router script would handle these.

Example #2 Starting with a specific document root directory

```
$ cd ~/public_html
$ php -S localhost:8000 -t foo/
```

The terminal will show:

```
PHP 5.4.0 Development Server started at Thu Jul 21 10:50:26 2011
Listening on localhost:8000
Document root is /home/me/public_html/foo
Press Ctrl-C to quit
```

Example #3 Using a Router Script

In this example, requests for images will display them, but requests for HTML files will display "Welcome to PHP":

```php
<?php
// router.php
if (preg_match('/\.(?:png|jpg|jpeg|gif)$/', $_SERVER["REQUEST_URI"])) {
    return false;    // serve the requested resource as-is.
} else {
    echo "<p>Welcome to PHP</p>";
}
?>
```

```
$ php -S localhost:8000 router.php
```

Example #4 Checking for CLI Web Server Use

To reuse a framework router script during development with the CLI web server and later also with a production web server:

```php
<?php
// router.php
if (php_sapi_name() == 'cli-server') {
    /* route static assets and return false */
}
/* go on with normal index.php operations */
?>
```

```
$ php -S localhost:8000 router.php
```

Example #5 Handling Unsupported File Types

If you need to serve a static resource whose MIME type is not handled by the CLI web server, use:

```php
<?php
// router.php
$path = pathinfo($_SERVER["SCRIPT_FILENAME"]);
if ($path["extension"] == "el") {
    header("Content-Type: text/x-script.elisp");
    readfile($_SERVER["SCRIPT_FILENAME"]);
}
else {
    return FALSE;
```

```
}
?>
```

```
$ php -S localhost:8000 router.php
```

Example #6 Accessing the CLI Web Server From Remote Machines

You can make the web server accessible on port 8000 to any interface with:

```
$ php -S 0.0.0.0:8000
```

> **Warning**
>
> The built-in Web Server should not be used on a public network.

INI settings

CLI SAPI Configuration Options

Name	Default	Changeable	Changelog
cli_server.color	"0"	INI_ALL	

Here's a short explanation of the configuration directives.

`cli_server.color` bool

Enable the built-in development web server to use ANSI color coding in terminal output.

Garbage Collection

Table of Contents

This section explains the merits of the new Garbage Collection (also known as GC) mechanism that is part of PHP 5.3.

Reference Counting Basics

A PHP variable is stored in a container called a "zval". A zval container contains, besides the variable's type and value, two additional bits of information. The first is called "is_ref" and is a boolean value indicating whether or not the variable is part of a "reference set". With this bit, PHP's engine knows how to differentiate between normal variables and references. Since PHP allows user-land references, as created by the & operator, a zval container also has an internal reference counting mechanism to optimize memory usage. This second piece of additional information, called "refcount", contains how many variable names (also called symbols) point to this one zval container. All symbols are stored in a symbol table, of which there is one per scope. There is a scope for the main script (i.e., the one requested through the browser), as well as one for every function or method.

A zval container is created when a new variable is created with a constant value, such as:

Example #1 Creating a new zval container

```php
<?php
$a = "new string";
?>
```

In this case, the new symbol name, `a`, is created in the current scope, and a new variable container is created with the type string and the value `new string`. The "is_ref" bit is by default set to **`false`** because no user-land reference has been created. The "refcount" is set to `1` as there is only one symbol that makes use of this variable container. Note that references (i.e. "is_ref" is **`true`**) with "refcount" `1`, are treated as if they are not references (i.e. as "is_ref" was **`false`**). If you have » Xdebug installed, you can display this information by calling **xdebug_debug_zval()**.

Example #2 Displaying zval information

```php
<?php
$a = "new string";
xdebug_debug_zval('a');
?>
```

The above example will output:

```
a: (refcount=1, is_ref=0)='new string'
```

Assigning this variable to another variable name will increase the refcount.

Example #3 Increasing refcount of a zval

```php
<?php
$a = "new string";
$b = $a;
xdebug_debug_zval( 'a' );
?>
```

The above example will output:

```
a: (refcount=2, is_ref=0)='new string'
```

The refcount is `2` here, because the same variable container is linked with both *a* and *b*. PHP is smart enough not to copy the actual variable container when it is not necessary. Variable containers get destroyed when the "refcount" reaches zero. The "refcount" gets decreased by one when any symbol linked to the variable container leaves the scope (e.g. when the function ends) or when a symbol is unassigned (e.g. by calling unset()). The following example shows this:

Example #4 Decreasing zval refcount

```php
<?php
$a = "new string";
$c = $b = $a;
xdebug_debug_zval( 'a' );
$b = 42;
xdebug_debug_zval( 'a' );
unset( $c );
xdebug_debug_zval( 'a' );
?>
```

The above example will output:

```
a: (refcount=3, is_ref=0)='new string'
a: (refcount=2, is_ref=0)='new string'
a: (refcount=1, is_ref=0)='new string'
```

If we now call `unset($a);`, the variable container, including the type and value, will be removed from memory.

Compound Types

Things get a tad more complex with compound types such as arrays and objects. As opposed to scalar values, arrays and objects store their properties in a symbol table of their own. This means that the following example creates three zval containers:

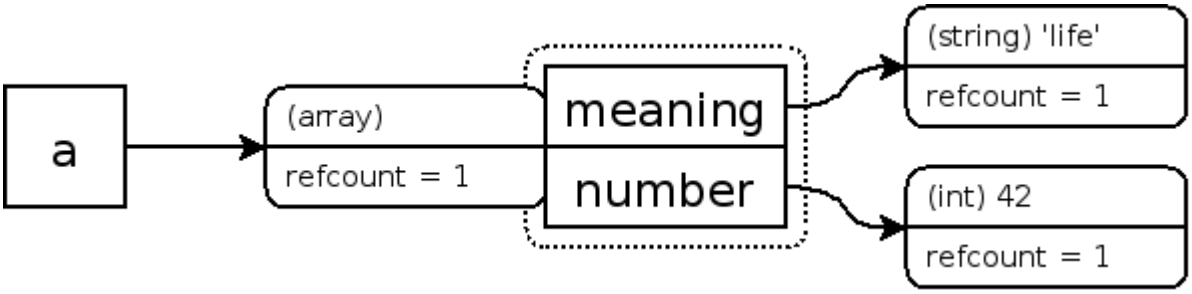
Example #5 Creating a array zval

```php
<?php
$a = array( 'meaning' => 'life', 'number' => 42 );
xdebug_debug_zval( 'a' );
?>
```

The above example will output something similar to:

```
a: (refcount=1, is_ref=0)=array (
   'meaning' => (refcount=1, is_ref=0)='life',
   'number' => (refcount=1, is_ref=0)=42
)
```

Or graphically



The three zval containers are: *a*, *meaning*, and *number*. Similar rules apply for increasing and decreasing "refcounts". Below, we add another element to the array, and set its value to the contents of an already existing element:

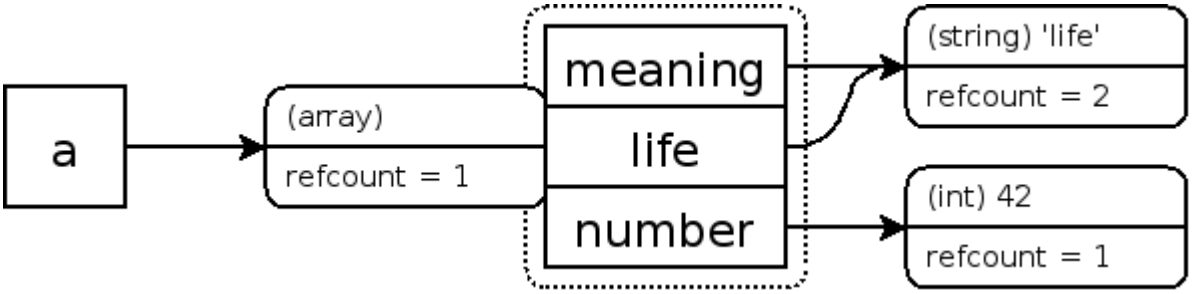
Example #6 Adding already existing element to an array

```
<?php
$a = array( 'meaning' => 'life', 'number' => 42 );
$a['life'] = $a['meaning'];
xdebug_debug_zval( 'a' );
?>
```

The above example will output something similar to:

```
a: (refcount=1, is_ref=0)=array (
   'meaning' => (refcount=2, is_ref=0)='life',
   'number' => (refcount=1, is_ref=0)=42,
   'life' => (refcount=2, is_ref=0)='life'
)
```

Or graphically



From the above Xdebug output, we see that both the old and new array elements now point to a zval container whose "refcount" is 2. Although Xdebug's output shows two zval containers with value `'life'`, they are the same one. The **xdebug_debug_zval()** function does not show this, but you could see it by also displaying the memory pointer.

Removing an element from the array is like removing a symbol from a scope. By doing so, the "refcount" of a container that an array element points to is decreased. Again, when the "refcount" reaches zero, the variable container is removed from memory. Again, an example to show this:

Example #7 Removing an element from an array

```php
<?php
$a = array( 'meaning' => 'life', 'number' => 42 );
$a['life'] = $a['meaning'];
unset( $a['meaning'], $a['number'] );
xdebug_debug_zval( 'a' );
?>
```

The above example will output something similar to:

```
a: (refcount=1, is_ref=0)=array (
   'life' => (refcount=1, is_ref=0)='life'
)
```

Now, things get interesting if we add the array itself as an element of the array, which we do in the next example, in which we also sneak in a reference operator, since otherwise PHP would create a copy:

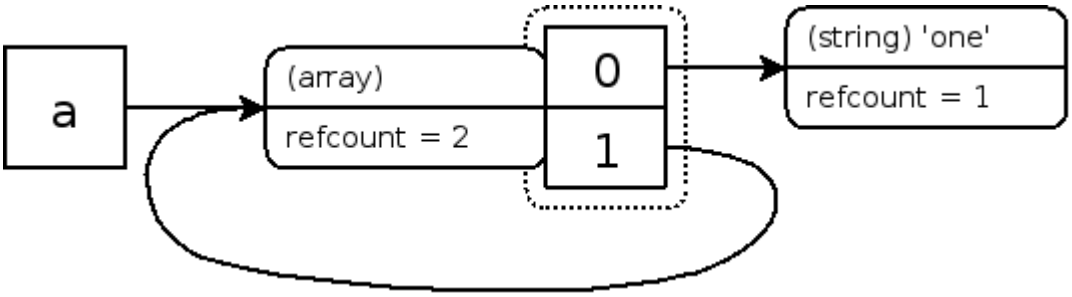
Example #8 Adding the array as an element of itself

```php
<?php
$a = array( 'one' );
$a[] =& $a;
xdebug_debug_zval( 'a' );
?>
```

The above example will output something similar to:

```
a: (refcount=2, is_ref=1)=array (
   0 => (refcount=1, is_ref=0)='one',
   1 => (refcount=2, is_ref=1)=...
)
```

Or graphically



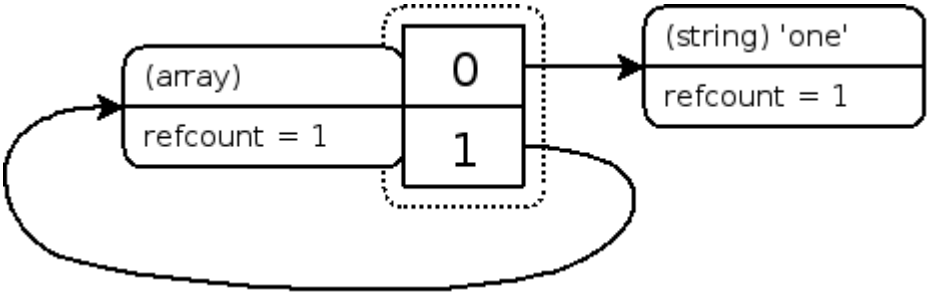
You can see that the array variable (*a*) as well as the second element (*1*) now point to a variable container that has a "refcount" of 2. The "..." in the display above shows that there is recursion involved, which, of course, in this case means that the "..." points back to the original array.

Just like before, unsetting a variable removes the symbol, and the reference count of the variable container it points to is decreased by one. So, if we unset variable *$a* after running the above code, the reference count of the variable container that *$a* and element "1" point to gets decreased by one, from "2" to "1". This can be represented as:

Example #9 Unsetting *$a*

```
(refcount=1, is_ref=1)=array (
   0 => (refcount=1, is_ref=0)='one',
   1 => (refcount=1, is_ref=1)=...
)
```

Or graphically



Cleanup Problems

Although there is no longer a symbol in any scope pointing to this structure, it cannot be cleaned up because the array element "1" still points to this same array. Because there is no external symbol pointing to it, there is no way for a user to clean up this structure; thus you get a memory leak. Fortunately, PHP will clean up this data structure at the end of the request, but before then, this is taking up valuable space in memory. This situation happens often if you're implementing parsing algorithms or other things where you have a child point back at a "parent" element. The same situation can also happen with objects of course, where it actually is more likely to occur, as objects are always implicitly used "by reference".

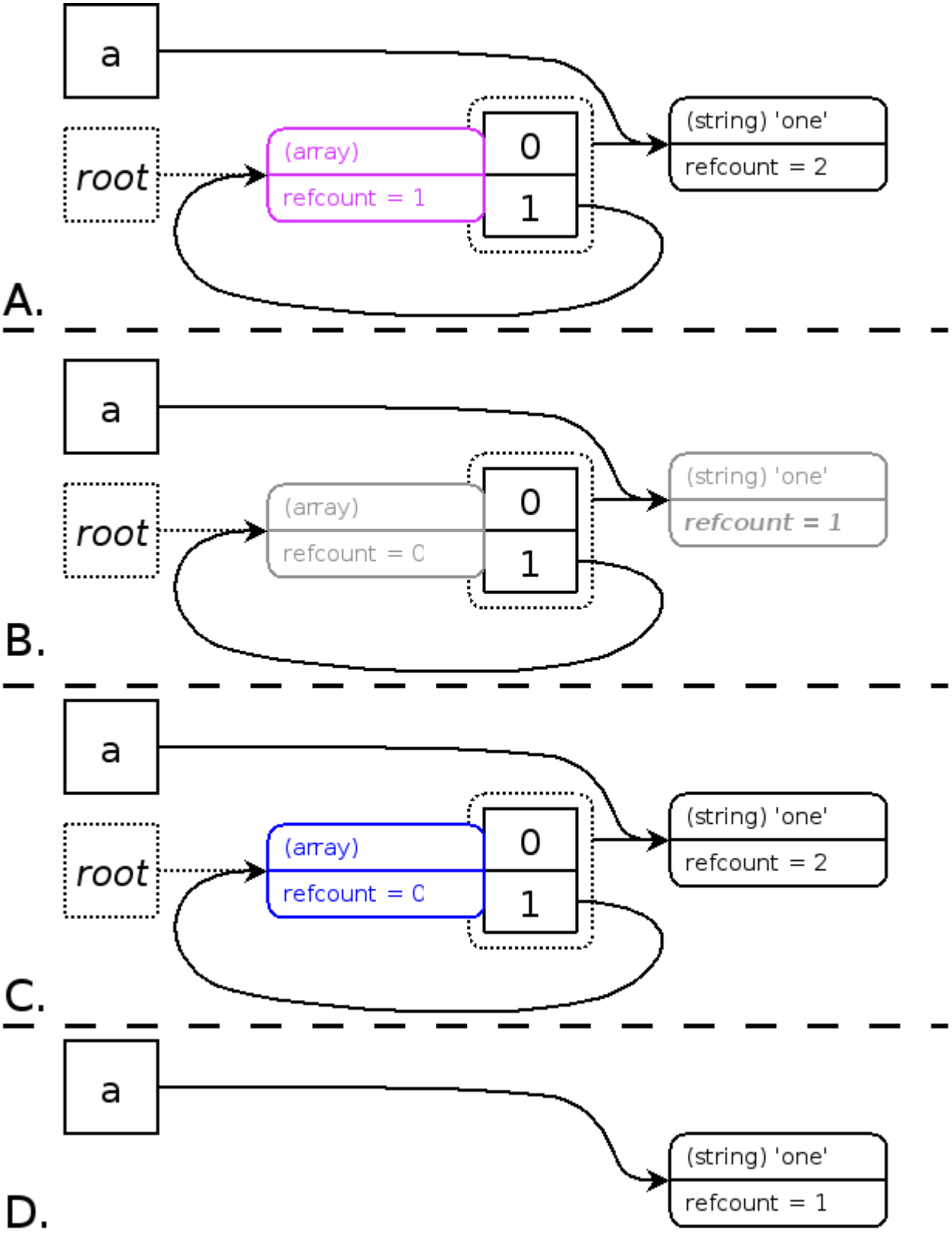
This might not be a problem if this only happens once or twice, but if there are thousands, or even millions, of these memory losses, this obviously starts being a problem. This is especially problematic in long running scripts, such as daemons where the request basically never ends, or in large sets of unit tests. The latter caused problems while running the unit tests for the Template component of the eZ Components library. In some cases, it would require over 2 GB of memory, which the test server didn't quite have.

Collecting Cycles

Traditionally, reference counting memory mechanisms, such as that used previously by PHP, fail to address circular reference memory leaks; however, as of 5.3.0, PHP implements the synchronous algorithm from the » Concurrent Cycle Collection in Reference Counted Systems paper which addresses that issue.

A full explanation of how the algorithm works would be slightly beyond the scope of this section, but the basics are explained here. First of all, we have to establish a few ground rules. If a refcount is increased, it's still in use and therefore, not garbage. If the refcount is decreased and hits zero, the zval can be freed. This means that garbage cycles can only be created when a refcount argument is decreased to a non-zero value.

Secondly, in a garbage cycle, it is possible to discover which parts are garbage by checking whether it is possible to decrease their refcount by one, and then checking which of the zvals have a refcount of zero.



To avoid having to call the checking of garbage cycles with every possible decrease of a refcount, the algorithm instead puts all possible roots (zvals) in the "root buffer" (marking them "purple"). It also makes sure that each possible garbage root ends up in the buffer only once. Only when the root buffer is full does the collection mechanism start for all the different zvals inside. See step A in the figure above.

In step B, the algorithm runs a depth-first search on all possible roots to decrease by one the refcounts of each zval it finds, making sure not to decrease a refcount on the same zval twice (by marking them as "grey"). In step C, the algorithm again runs a depth-first search from each root node, to check the refcount of each zval again. If it finds that the refcount is zero, the zval is marked "white" (blue in the figure). If it's larger than zero, it reverts the decreasing of the refcount by one with a depth-first search from that point on, and they are marked "black" again. In the last step (D), the algorithm walks over the root buffer removing the zval roots from there, and meanwhile, checks which zvals have been marked "white" in the previous step. Every zval marked as "white" will be freed.

Now that you have a basic understanding of how the algorithm works, we will look back at how this integrates with PHP. By default, PHP's garbage collector is turned on. There is, however, a *php.ini* setting that allows you to change this: zend.enable_gc.

When the garbage collector is turned on, the cycle-finding algorithm as described above is executed whenever the root buffer runs full. The root buffer has a fixed size of 10,000 possible roots (although you can alter this by changing the **`GC_THRESHOLD_DEFAULT`** constant in `Zend/zend_gc.c` in the PHP source code, and re-compiling PHP). When the garbage collector is turned off, the cycle-finding algorithm will never run. However, possible roots will always be recorded in the root buffer, no matter whether the garbage collection mechanism has been activated with this configuration setting.

If the root buffer becomes full with possible roots while the garbage collection mechanism is turned off, further possible roots will simply not be recorded. Those possible roots that are not recorded will never be analyzed by the algorithm. If they were part of a circular reference cycle, they would never be cleaned up and would create a memory leak.

The reason why possible roots are recorded even if the mechanism has been disabled is because it's faster to record possible roots than to have to check whether the mechanism is turned on every time a possible root could be found. The garbage collection and analysis mechanism itself, however, can take a considerable amount of time.

Besides changing the zend.enable_gc configuration setting, it is also possible to turn the garbage collecting mechanism on and off by calling gc_enable() or gc_disable() respectively. Calling those functions has the same effect as turning on or off the mechanism with the configuration setting. It is also possible to force the collection of cycles even if the possible root buffer is not full yet. For this, you can use the gc_collect_cycles() function. This function will return how many cycles were collected by the algorithm.

The rationale behind the ability to turn the mechanism on and off, and to initiate cycle collection yourself, is that some parts of your application could be highly time-sensitive. In those cases, you might not want the garbage collection mechanism to kick in. Of course, by turning off the garbage collection for certain parts of your application, you do risk creating memory leaks because some possible roots might not fit into the limited root buffer. Therefore, it is probably wise to call gc_collect_cycles() just before you call gc_disable() to free up the memory that could be lost through possible roots that are already recorded in the root buffer. This then leaves an empty buffer so that there is more space for storing possible roots while the cycle collecting mechanism is turned off.

Performance Considerations

We have already mentioned in the previous section that simply collecting the possible roots had a very tiny performance impact, but this is when you compare PHP 5.2 against PHP 5.3. Although the recording of possible roots compared to not recording them at all, like in PHP 5.2, is slower, other changes to the PHP runtime in PHP 5.3 prevented this particular performance loss from even showing.

There are two major areas in which performance is affected. The first area is reduced memory usage, and the second area is run-time delay when the garbage collection mechanism performs its memory cleanups. We will look at both of those issues.

Reduced Memory Usage

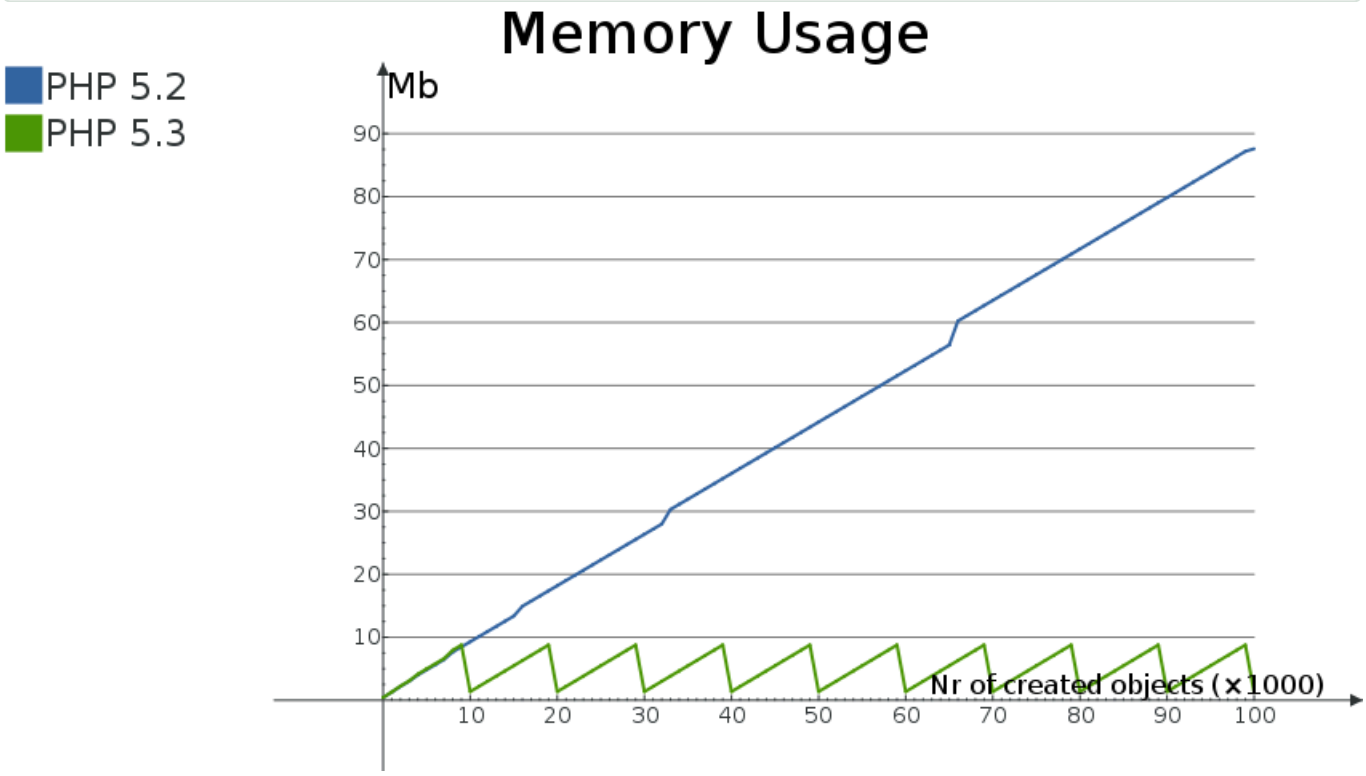
First of all, the whole reason for implementing the garbage collection mechanism is to reduce memory usage by cleaning up circular-referenced variables as soon as the prerequisites are fulfilled. In PHP's implementation, this happens as soon as the root-buffer is full, or when the function gc_collect_cycles() is called. In the graph below, we display the memory usage of the script below, in both PHP 5.2 and PHP 5.3, excluding the base memory that PHP itself uses when starting up.

Example #1 Memory usage example

```php
<?php
class Foo
{
    public $var = '3.14159265359';
    public $self;
}

$baseMemory = memory_get_usage();

for ( $i = 0; $i <= 100000; $i++ )
{
    $a = new Foo;
    $a->self = $a;
    if ( $i % 500 === 0 )
    {
        echo sprintf( '%8d: ', $i ), memory_get_usage() - $baseMemory, "\n";
    }
}
?>
```



In this very academic example, we are creating an object in which a property is set to point back to the object itself. When the *$a* variable in the script is re-assigned in the next iteration of the loop, a memory leak would typically occur. In this case, two zval-containers are leaked (the object zval, and the property zval), but only one possible root is found: the variable that was unset. When the root-buffer is full after 10,000 iterations (with a total of 10,000 possible roots), the garbage collection mechanism kicks in and frees the memory associated with those possible roots. This can very clearly be seen in the jagged memory-usage graph for PHP 5.3. After each 10,000 iterations, the mechanism kicks in and frees the memory associated with the circular referenced variables. The mechanism itself does not have to do a whole lot of work in this example, because the structure that is leaked is extremely simple. From the diagram, you see that the maximum memory usage in PHP 5.3 is about 9 Mb, whereas in PHP 5.2 the memory usage keeps increasing.

Run-Time Slowdowns

The second area where the garbage collection mechanism influences performance is the time taken when the garbage collection mechanism kicks in to free the "leaked" memory. In order to see how much this is, we slightly modify the previous script to allow for a larger number of iterations and the removal of the intermediate memory usage figures. The second script is here:

Example #2 GC performance influences

```php
<?php
class Foo
{
    public $var = '3.14159265359';
    public $self;
}

for ( $i = 0; $i <= 1000000; $i++ )
{
    $a = new Foo;
    $a->self = $a;
}

echo memory_get_peak_usage(), "\n";
?>
```

We will run this script two times, once with the zend.enable_gc setting turned on, and once with it turned off:

Example #3 Running the above script

```
time php -dzend.enable_gc=0 -dmemory_limit=-1 -n example2.php
# and
time php -dzend.enable_gc=1 -dmemory_limit=-1 -n example2.php
```

On my machine, the first command seems to take consistently about 10.7 seconds, whereas the second command takes about 11.4 seconds. This is a slowdown of about 7%. However, the maximum amount of memory used by the script is reduced by 98% from 931Mb to 10Mb. This benchmark is not very scientific, or even representative of real-life applications, but it does demonstrate the memory usage benefits that this garbage collection mechanism provides. The good thing is that the slowdown is always the same 7%, for this particular script, while the memory saving capabilities save more and more memory as more circular references are found during script execution.

PHP's Internal GC Statistics

It is possible to coax a little bit more information about how the garbage collection mechanism is run from within PHP. But in order to do so, you will have to re-compile PHP to enable the benchmark and data-collecting code. You will have to set the `CFLAGS` environment variable to `-DGC_BENCH=1` prior to running `./configure` with your desired options. The following sequence should do the trick:

Example #4 Recompiling PHP to enable GC benchmarking

```
export CFLAGS=-DGC_BENCH=1
./config.nice
make clean
make
```

When you run the above example code again with the newly built PHP binary, you will see the following being shown after PHP has finished execution:

Example #5 GC statistics

```
GC Statistics
-------------
Runs:               110
Collected:          2072204
Root buffer length: 0
Root buffer peak:   10000

      Possible            Remove from  Marked
        Root    Buffered     buffer     grey
      --------  --------  -----------  ------
ZVAL   7175487   1491291    1241690   3611871
ZOBJ  28506264   1527980     677581   1025731
```

The most informative statistics are displayed in the first block. You can see here that the garbage collection mechanism ran 110 times, and in total, more than 2 million memory allocations were freed during those 110 runs. As soon as the garbage collection mechanism has run at least one time, the "Root buffer peak" is always 10000.

Conclusion

In general the garbage collector in PHP will only cause a slowdown when the cycle collecting algorithm actually runs, whereas in normal (smaller) scripts there should be no performance hit at all.

However, in the cases where the cycle collection mechanism does run for normal scripts, the memory reduction it will provide allows more of those scripts to run concurrently on your server, since not so much memory is used in total.

The benefits are most apparent for longer-running scripts, such as lengthy test suites or daemon scripts. Also, for » PHP-GTK applications that generally tend to run longer than scripts for the Web, the new mechanism should make quite a bit of a difference regarding memory leaks creeping in over time.

DTrace Dynamic Tracing

Table of Contents

Introduction to PHP and DTrace

DTrace is an always-available, low overhead, tracing framework available on a number of platforms including Solaris, macOS, Oracle Linux and BSD. DTrace can trace operating system behavior and user program execution. It can display argument values and be used to infer performance statistics. Probes are monitored by user created scripts written in the DTrace D scripting language. This allows efficient analysis of data points.

PHP probes that are not being actively monitored by a user's DTrace D script do not contain instrumented code so there is no performance degradation during normal application execution. Probes that are being monitored incur an overhead low enough to generally allow DTrace monitoring on live production systems.

PHP incorporates "User-level Statically Defined Tracing" (USDT) probes that are triggered at runtime. For example, when a D script is monitoring PHP's `function-entry` probe, then, every time a PHP script function is called, this probe is fired and the associated D script action code is executed. This action code could, for example, print probe arguments such as the source file location of the PHP function. Or the action could aggregate data such as the number of times each function is called.

Only the PHP USDT probes are described here. Refer to external general and operating system-specific DTrace literature to see how DTrace can be used to trace arbitrary functions, and how it can be used to trace operating system behavior. Note not all DTrace features are available in all DTrace implementations.

The static DTrace probes in PHP can alternatively be used with the SystemTap facility on some Linux distributions.

Using PHP and DTrace

PHP can be configured with DTrace static probes on platforms that support DTrace Dynamic Tracing.

Configuring PHP for DTrace Static Probes

Refer to external platform specific documentation for enabling operating system DTrace support. For example, on Oracle Linux boot a UEK3 kernel and do:

```
# modprobe fasttrap
# chmod 666 /dev/dtrace/helper
```

Instead of using `chmod`, you could instead use an ACL package rule to limit device access to a specific user.

Build PHP with the `--enable-dtrace` configuration parameter:

```
# ./configure --enable-dtrace ...
# make
# make install
```

This makes the static probes available in core PHP. Any PHP extensions that provide their own probes should be built separately as shared extensions.

To enable probes, set **USE_ZEND_DTRACE=1** environment variable to the target PHP processes.

DTrace Static Probes in Core PHP

The following static probes are available in PHP

Probe Name	Probe Description	Probe Arguments
`request-startup`	Fires when a request starts.	char **file*, char **request_uri*, char **request_method*

Probe Name	Probe Description	Probe Arguments
`request-shutdown`	Fires when a request shutdown.	char **file*, char **request_uri*, char **request_method*
`compile-file-entry`	Fires when the compilation of a script starts.	char **compile_file*, char **compile_file_translated*
`compile-file-return`	Fires when the compilation of a script finishes.	char **compile_file*, char **compile_file_translated*
`execute-entry`	Fires when an opcode array is to be executed. For example, it fires on function calls, includes, and generator resumes.	char **request_file*, int *lineno*
`execute-return`	Fires after execution of an opcode array.	char **request_file*, int *lineno*
`function-entry`	Fires when the PHP engine enters a PHP function or method call.	char **function_name*, char **request_file*, int *lineno*, char **classname*, char **scope*
`function-return`	Fires when the PHP engine returns from a PHP function or method call.	char **function_name*, char **request_file*, int *lineno*, char **classname*, char **scope*
`exception-thrown`	Fires when an exception is thrown.	char **classname*
`exception-caught`	Fires when an exception is caught.	char **classname*
`error`	Fires when an error occurs, regardless of the error_reporting level.	char **errormsg*, char **request_file*, int *lineno*

PHP extensions may also have additional static probes.

Listing DTrace Static Probes in PHP

To list available probes, start a PHP process and then run:

```
# dtrace -l
```

The output will be similar to:

```
   ID   PROVIDER            MODULE                          FUNCTION NAME
   [ . . . ]
    4   php15271               php               dtrace_compile_file compile-file-entry
    5   php15271               php               dtrace_compile_file compile-file-return
    6   php15271               php                        zend_error error
    7   php15271               php  ZEND_CATCH_SPEC_CONST_CV_HANDLER exception-caught
    8   php15271               php     zend_throw_exception_internal exception-thrown
    9   php15271               php                  dtrace_execute_ex execute-entry
   10   php15271               php           dtrace_execute_internal execute-entry
   11   php15271               php                  dtrace_execute_ex execute-return
   12   php15271               php           dtrace_execute_internal execute-return
   13   php15271               php                  dtrace_execute_ex function-entry
   14   php15271               php                  dtrace_execute_ex function-return
   15   php15271               php              php_request_shutdown request-shutdown
   16   php15271               php               php_request_startup request-startup
```

The Provider column values consist of `php` and the process id of the currently running PHP process.

If the Apache web server is running, the module name might be, for example, *libphp5.so*, and there would be multiple blocks of listings, one per running Apache process.

The Function column refers to PHP's internal C implementation function names where each provider is located.

If a PHP process is not running, then no PHP probes will be shown.

DTrace with PHP Example

This example shows the basics of the DTrace D scripting language.

Example #1 *all_probes.d* for tracing all PHP Static Probes with DTrace

```
#!/usr/sbin/dtrace -Zs

#pragma D option quiet

php*:::compile-file-entry
{
    printf("PHP compile-file-entry\n");
    printf("  compile_file              %s\n", copyinstr(arg0));
    printf("  compile_file_translated   %s\n", copyinstr(arg1));
}

php*:::compile-file-return
{
    printf("PHP compile-file-return\n");
    printf("  compile_file              %s\n", copyinstr(arg0));
    printf("  compile_file_translated   %s\n", copyinstr(arg1));
}

php*:::error
{
    printf("PHP error\n");
    printf("  errormsg                  %s\n", copyinstr(arg0));
    printf("  request_file              %s\n", copyinstr(arg1));
    printf("  lineno                    %d\n", (int)arg2);
}

php*:::exception-caught
{
    printf("PHP exception-caught\n");
    printf("  classname                 %s\n", copyinstr(arg0));
}

php*:::exception-thrown
{
    printf("PHP exception-thrown\n");
    printf("  classname                 %s\n", copyinstr(arg0));
}

php*:::execute-entry
{
    printf("PHP execute-entry\n");
```

```
    printf("  request_file              %s\n", copyinstr(arg0));
    printf("  lineno                    %d\n", (int)arg1);
}

php*:::execute-return
{
    printf("PHP execute-return\n");
    printf("  request_file              %s\n", copyinstr(arg0));
    printf("  lineno                    %d\n", (int)arg1);
}

php*:::function-entry
{
    printf("PHP function-entry\n");
    printf("  function_name             %s\n", copyinstr(arg0));
    printf("  request_file              %s\n", copyinstr(arg1));
    printf("  lineno                    %d\n", (int)arg2);
    printf("  classname                 %s\n", copyinstr(arg3));
    printf("  scope                     %s\n", copyinstr(arg4));
}

php*:::function-return
{
    printf("PHP function-return\n");
    printf("  function_name             %s\n", copyinstr(arg0));
    printf("  request_file              %s\n", copyinstr(arg1));
    printf("  lineno                    %d\n", (int)arg2);
    printf("  classname                 %s\n", copyinstr(arg3));
    printf("  scope                     %s\n", copyinstr(arg4));
}

php*:::request-shutdown
{
    printf("PHP request-shutdown\n");
    printf("  file                      %s\n", copyinstr(arg0));
    printf("  request_uri               %s\n", copyinstr(arg1));
    printf("  request_method            %s\n", copyinstr(arg2));
}

php*:::request-startup
{
    printf("PHP request-startup\n");
    printf("  file                      %s\n", copyinstr(arg0));
    printf("  request_uri               %s\n", copyinstr(arg1));
    printf("  request_method            %s\n", copyinstr(arg2));
}
```

This script uses the `-z` option to *dtrace*, allowing it to be run when there is no PHP process executing. If this option were omitted the script would immediately terminate because it knows none of the probes to be monitored are in existence.

The script traces all core PHP static probe points throughout the duration of a running PHP script. Run the D script:

```
# ./all_probes.d
```

Run a PHP script or application. The monitoring D script will output each probe's arguments as it fires.

When monitoring is complete, the D script can be terminated with a CTRL+C.

On multi-CPU machines the probe ordering might not appear to be sequential. This depends on which CPU was processing the probes, and how threads migrate across CPUs. Displaying probe time stamps will help reduce confusion, for example:

```
php*:::function-entry
{
      printf("%lld: PHP function-entry ", walltimestamp);
      [ . . .]
}
```

See Also

- OCI8 and DTrace Dynamic Tracing

Using SystemTap with PHP DTrace Static Probes

On some Linux distributions, the SystemTap tracing utility can be used to trace PHP's static DTrace probes. This is available with PHP 5.4.20 and PHP 5.5.

Installing PHP with SystemTap

Install the SystemTap SDT development package:

```
# yum install systemtap-sdt-devel
```

Install PHP with the DTrace probes enabled:

```
# ./configure --enable-dtrace ...
# make
```

Listing Static Probes with SystemTap

The static probes in PHP can be listed using *stap*:

```
# stap -l 'process.provider("php").mark("*")' -c 'sapi/cli/php -i'
```

This outputs:

```
process("sapi/cli/php").provider("php").mark("compile__file__entry")
process("sapi/cli/php").provider("php").mark("compile__file__return")
process("sapi/cli/php").provider("php").mark("error")
process("sapi/cli/php").provider("php").mark("exception__caught")
process("sapi/cli/php").provider("php").mark("exception__thrown")
process("sapi/cli/php").provider("php").mark("execute__entry")
process("sapi/cli/php").provider("php").mark("execute__return")
process("sapi/cli/php").provider("php").mark("function__entry")
process("sapi/cli/php").provider("php").mark("function__return")
process("sapi/cli/php").provider("php").mark("request__shutdown")
process("sapi/cli/php").provider("php").mark("request__startup")
```

SystemTap with PHP Example

Example #1 *all_probes.stp* for tracing all PHP Static Probes with SystemTap

```
probe process("sapi/cli/php").provider("php").mark("compile__file__entry") {
    printf("Probe compile__file__entry\n");
    printf("  compile_file %s\n", user_string($arg1));
    printf("  compile_file_translated %s\n", user_string($arg2));
}
probe process("sapi/cli/php").provider("php").mark("compile__file__return") {
    printf("Probe compile__file__return\n");
    printf("  compile_file %s\n", user_string($arg1));
    printf("  compile_file_translated %s\n", user_string($arg2));
}
probe process("sapi/cli/php").provider("php").mark("error") {
    printf("Probe error\n");
    printf("  errormsg %s\n", user_string($arg1));
    printf("  request_file %s\n", user_string($arg2));
    printf("  lineno %d\n", $arg3);
}
probe process("sapi/cli/php").provider("php").mark("exception__caught") {
    printf("Probe exception__caught\n");
    printf("  classname %s\n", user_string($arg1));
}
probe process("sapi/cli/php").provider("php").mark("exception__thrown") {
    printf("Probe exception__thrown\n");
    printf("  classname %s\n", user_string($arg1));
}
probe process("sapi/cli/php").provider("php").mark("execute__entry") {
    printf("Probe execute__entry\n");
    printf("  request_file %s\n", user_string($arg1));
    printf("  lineno %d\n", $arg2);
}
probe process("sapi/cli/php").provider("php").mark("execute__return") {
    printf("Probe execute__return\n");
    printf("  request_file %s\n", user_string($arg1));
    printf("  lineno %d\n", $arg2);
}
probe process("sapi/cli/php").provider("php").mark("function__entry") {
    printf("Probe function__entry\n");
    printf("  function_name %s\n", user_string($arg1));
    printf("  request_file %s\n", user_string($arg2));
    printf("  lineno %d\n", $arg3);
    printf("  classname %s\n", user_string($arg4));
    printf("  scope %s\n", user_string($arg5));
}
probe process("sapi/cli/php").provider("php").mark("function__return") {
    printf("Probe function__return: %s\n", user_string($arg1));
    printf(" function_name %s\n", user_string($arg1));
    printf("  request_file %s\n", user_string($arg2));
    printf("  lineno %d\n", $arg3);
    printf("  classname %s\n", user_string($arg4));
    printf("  scope %s\n", user_string($arg5));
}
probe process("sapi/cli/php").provider("php").mark("request__shutdown") {
    printf("Probe request__shutdown\n");
    printf("  file %s\n", user_string($arg1));
```

```
    printf("  request_uri %s\n", user_string($arg2));
    printf("  request_method %s\n", user_string($arg3));
}
probe process("sapi/cli/php").provider("php").mark("request__startup") {
    printf("Probe request__startup\n");
    printf("  file %s\n", user_string($arg1));
    printf("  request_uri %s\n", user_string($arg2));
    printf("  request_method %s\n", user_string($arg3));
}
```

The above script will trace all core PHP static probe points throughout the duration of a running PHP script:

```
# stap -c 'sapi/cli/php test.php' all_probes.stp
```

Using bpftrace with PHP DTrace Static Probes

On Linux distributions with a kernel that supports eBPF, the bpftrace utility can attach to PHP's DTrace USDT probes directly, without requiring SystemTap.

Installing bpftrace

Install bpftrace using the distribution's package manager. For example, on Oracle Linux, RHEL, or Fedora:

```
# dnf install bpftrace
```

Or, on Debian or Ubuntu:

```
# apt install bpftrace
```

The examples below assume the target PHP binary is installed at */usr/bin/php*.

The same USDT probes are also exposed by other SAPIs built from the same source tree, so the probe target may instead be the Apache module (*libphp.so*) or the FastCGI Process Manager binary (*php-fpm*); substitute the appropriate path or attach by PID with `-p` as needed.

Make sure the target binary is built with DTrace and the environment variable is configured properly. See Configuring PHP for DTrace Static Probes for details.

The static probes in PHP can be listed using **bpftrace**:

```
# bpftrace -l 'usdt:/usr/bin/php:php:*'
```

This outputs:

```
usdt:/usr/bin/php:php:compile__file__entry
usdt:/usr/bin/php:php:compile__file__return
usdt:/usr/bin/php:php:error
usdt:/usr/bin/php:php:exception__caught
usdt:/usr/bin/php:php:exception__thrown
usdt:/usr/bin/php:php:execute__entry
usdt:/usr/bin/php:php:execute__return
usdt:/usr/bin/php:php:function__entry
usdt:/usr/bin/php:php:function__return
```

```
usdt:/usr/bin/php:php:request__shutdown
usdt:/usr/bin/php:php:request__startup
```

Example #1 *all_probes.bt* for tracing all PHP Static Probes with bpftrace

```
#!/usr/bin/env bpftrace

usdt:/usr/bin/php:php:compile__file__entry
{
    printf("Probe compile__file__entry\n");
    printf("  compile_file %s\n", str(arg0));
    printf("  compile_file_translated %s\n", str(arg1));
}
usdt:/usr/bin/php:php:compile__file__return
{
    printf("Probe compile__file__return\n");
    printf("  compile_file %s\n", str(arg0));
    printf("  compile_file_translated %s\n", str(arg1));
}
usdt:/usr/bin/php:php:error
{
    printf("Probe error\n");
    printf("  errormsg %s\n", str(arg0));
    printf("  request_file %s\n", str(arg1));
    printf("  lineno %d\n", (int32)arg2);
}
usdt:/usr/bin/php:php:exception__caught
{
    printf("Probe exception__caught\n");
    printf("  classname %s\n", str(arg0));
}
usdt:/usr/bin/php:php:exception__thrown
{
    printf("Probe exception__thrown\n");
    printf("  classname %s\n", str(arg0));
}
usdt:/usr/bin/php:php:execute__entry
{
    printf("Probe execute__entry\n");
    printf("  request_file %s\n", str(arg0));
    printf("  lineno %d\n", (int32)arg1);
}
usdt:/usr/bin/php:php:execute__return
{
    printf("Probe execute__return\n");
    printf("  request_file %s\n", str(arg0));
    printf("  lineno %d\n", (int32)arg1);
}
usdt:/usr/bin/php:php:function__entry
{
    printf("Probe function__entry\n");
    printf("  function_name %s\n", str(arg0));
    printf("  request_file %s\n", str(arg1));
    printf("  lineno %d\n", (int32)arg2);
    printf("  classname %s\n", str(arg3));
    printf("  scope %s\n", str(arg4));
}
```

```
usdt:/usr/bin/php:php:function__return
{
    printf("Probe function__return\n");
    printf("  function_name %s\n", str(arg0));
    printf("  request_file %s\n", str(arg1));
    printf("  lineno %d\n", (int32)arg2);
    printf("  classname %s\n", str(arg3));
    printf("  scope %s\n", str(arg4));
}
usdt:/usr/bin/php:php:request__shutdown
{
    printf("Probe request__shutdown\n");
    printf("  file %s\n", str(arg0));
    printf("  request_uri %s\n", str(arg1));
    printf("  request_method %s\n", str(arg2));
}
usdt:/usr/bin/php:php:request__startup
{
    printf("Probe request__startup\n");
    printf("  file %s\n", str(arg0));
    printf("  request_uri %s\n", str(arg1));
    printf("  request_method %s\n", str(arg2));
}
```

The above script will trace all core PHP static probe points throughout the duration of a running PHP script. bpftrace requires root privileges:

```
# USE_ZEND_DTRACE=1 bpftrace -c '/usr/bin/php test.php' all_probes.bt
```

To trace an already-running PHP process (for instance, a *php-fpm* worker or an Apache process loading *libphp.so*), attach by PID:

```
# bpftrace -p $PID all_probes.bt
```

The `usdt:` target path in the script must match the binary of the running process; adjust `usdt:/usr/bin/php` to the *php-fpm* binary or *libphp.so* as appropriate.

Frequently Asked Questions

General Information

This section holds the most general questions about PHP: what it is and what it does.

1. What is PHP?
2. What does PHP stand for?
3. What is the relation between the versions?
4. Can I run several versions of PHP at the same time?
5. I think I found a bug! Who should I tell?

What is PHP?

> From the preface of the manual:
>
> PHP is an HTML-embedded scripting language. Much of its syntax is borrowed from C, Java and Perl with a couple of unique PHP-specific features thrown in. The goal of the language is to allow web developers to write dynamically generated pages quickly.

What does PHP stand for?

> PHP stands for *PHP: Hypertext Preprocessor*. This confuses many people because the first word of the acronym is the acronym. This type of acronym is called a recursive acronym. For more information, the curious can visit » Free On-Line Dictionary of Computing or the » Wikipedia entry on recursive acronyms.

What is the relation between the versions?

> PHP/FI 2.0 is an early and no longer supported version of PHP. PHP 3 is the successor to PHP/FI 2.0 and is a lot nicer. PHP 8 is the current generation of PHP, which uses the Zend engine 4 which, among other things, offers many additional OOP features.

Can I run several versions of PHP at the same time?

> Yes. See the *INSTALL* file that is included in the PHP source distribution.

I think I found a bug! Who should I tell?

> You should go to the PHP Bug Database and make sure the bug isn't a known bug. If you don't see it in the database, use the reporting form to report the bug. It is important to use the bug database instead of just sending an email to one of the mailing lists because the bug will have a tracking number assigned and it will then be possible for you to go back later and check on the status of the bug. The bug database can be found at » https://github.com/php/php-src/issues.

Mailing lists

This section holds questions about how to get in touch with the PHP community. The best way is the mailing lists.

1. Are there any PHP mailing lists?
2. Are there any other communities?
3. Can I start my own PHP mailing list?
4. Help! I can't seem to subscribe to/unsubscribe from one of the mailing lists!
5. Is there an archive of the mailing lists anywhere?
6. What can I ask the mailing list?
7. What information should I include when posting to the mailing list?

Are there any PHP mailing lists?

Of course! There are many mailing lists for several subjects. The most common community mailing lists can be found on our » mailing list page.

The most general mailing list is `php-general`. To subscribe, send a blank email message to » php-general+subscribe@lists.php.net. To unsubscribe, send a blank email to » php-general+unsubscribe@lists.php.net.

You can also subscribe and unsubscribe using the web interface on our » mailing list page, and unsubscribe instructions are included in the footer of every mailing list message.

Are there any other communities?

There are countless PHP-centric communities around the world, and we have links to some of these resources (and information on how to learn more) on our » Support page.

Can I start my own PHP mailing list?

Absolutely! In fact, we not only freely allow it, we encourage it! Helping others by sharing your knowledge and experience with PHP helps to foster the growth and development of the global PHP community and the language itself.

Help! I can't seem to subscribe to/unsubscribe from one of the mailing lists!

The most common reason folks have a hard time unsubscribing from our mailing lists is due to the use of mail forwarders. For example, if your email address is `elephpant@example.com`, but you subscribed to the mailing list with the forwarder `php-lists@example.com` and forward that to `elephpant@example.com`, attempting to unsubscribe `elephpant@example.com` will not work, as that address is not even known to our systems. Instead, you will need to unsubscribe the address to which the mail is being sent, by sending an email from that address — in this example, `php-lists@example.com`.

Is there an archive of the mailing lists anywhere?

Yes, you will find links to some of the archive sites on the » mailing list page. You will also find dozens of sites that archive and/or syndicate our mailing list content by using your favorite Internet search engine and searching for "php mailing list archives".

All mailing list articles are also archived as news messages. You can access the news server at » news://news.php.net/ with a news client. There is also an experimental web interface for the news server at » http://news.php.net/

What can I ask the mailing list?

Since PHP is growing more and more popular by the day, the traffic on the php-general mailing list can be very heavy, ranging from 75 to 200 or more posts per day. Because of this, it is in everyone's interest

that you use the list as a last resort, when you have exhausted all other avenues.

Before you post to the list please have a look in this FAQ and the manual to see if you can find the help there. If there is nothing to be found there try out the mailing list archives (see above). If you're having problem with installing or configuring PHP, please read through all included documentation and README's. If you still can't find any information that helps you out, you're more than welcome to use the appropriate mailing list.

To ensure that you receive the best responses (and to reduce the likelihood of frustrating your fellow developers), please be sure to post your question to the appropriate list. For example, if you are having difficulties installing PHP, you should send your question to the `php-install` mailing list. A caveat: some lists have similar names and completely different uses; a question regarding PHP scripts on Windows should be directed to the Windows PHP users list, *not* to the Windows Internals list.

Before asking questions, you may want to read the paper on » How To Ask Questions The Smart Way as this is a good idea for everyone.

What information should I include when posting to the mailing list?

Posts like "I can't get PHP up and running! Help me! What is wrong?" are of absolutely no use to anyone. If you're having problems getting PHP up and running you must include the operating system being used, the version of PHP you're trying to use, how you got it (pre-compiled, Git, RPM, and so on), what you have done so far, and the exact error message or present result.

This goes for any other problem as well: you must include an explanation of what you have done, what you're trying to do, and any error messages encountered in the process. If you're having problems with your source code, you should include the part of the code that isn't working, indicating which line is the problem by an appropriately-placed code comment. Do not include more code than necessary though! It makes the post difficult to read, causing many to ignore your request. If you're unsure about how much information should be included, post what you believe is immediately relevant to your present issue, and be prepared to provide more information upon request.

Another important thing to remember is to summarize your problem on the subject line. A subject like "HELP MEEEE!!!" or "What is the problem here?" will be ignored by the majority of the readers.

And lastly, you're encouraged to read the paper on » How To Ask Questions The Smart Way as this will be a great help for everyone, especially yourself.

Obtaining PHP

This section has details about PHP download locations, and OS issues.

1. Where can I obtain PHP?
2. Are pre-compiled binary versions available?
3. Where can I get libraries needed to compile some of the optional PHP extensions?
4. How do I get these libraries to work?
5. I got the latest version of the PHP source code from the Git repository on my Windows machine, what do I need to compile it?
6. Where do I find the Browser Capabilities File?
7. What does thread safety mean when downloading PHP?

Where can I obtain PHP?

You can download PHP from any of the members of the PHP network of sites. These can be found at » https://www.php.net/. You can also use anonymous Git to get the absolute latest version of the source. For more information, go to » https://www.php.net/git.php.

Are pre-compiled binary versions available?

We only distribute precompiled binaries for Windows systems, as we are not able to compile PHP for every major Linux/Unix platform with every extension combination. Also note, that many Linux distributions come with PHP built in these days. Windows binaries can be downloaded from our » Downloads page, for Linux binaries, please visit your distribution's website.

Where can I get libraries needed to compile some of the optional PHP extensions?

> **Note** :
>
> Those marked with a * are to the best of our knowledge not thread safe; they are not recommended for use in a multi-threaded environment.

- » LDAP (Unix).
- » LDAP (Unix/Win) : Mozilla Directory (LDAP) SDK
- » free LDAP server.
- » Berkeley DB2 (Unix/Win) : http://www.sleepycat.com/.
- » SNMP* (Unix): .
- » GD (Unix/Win).
- » mSQL* (Unix).
- » PostgreSQL (Unix).
- » IMAP* (Win/Unix).
- » Sybase-CT* (Linux, libc5) : Available locally.
- » FreeType (libttf):.
- » ZLib (Unix/Win32).
- » expat XML parser (Unix/Win32).
- » PDFLib.
- » mcrypt.
- » mhash.
- » t1lib.
- » dmalloc.
- » aspell.

- » libedit.

How do I get these libraries to work?

You will need to follow instructions provided with the library. Some of these libraries are detected automatically when you run the 'configure' script of PHP (such as the GD library), and others you will have to enable using '`--with-EXTENSION`' options to '`configure`'. Run '`configure --help`' for a listing of these.

I got the latest version of the PHP source code from the Git repository on my Windows machine, what do I need to compile it?

See the PHP Wiki for the latest instructions: » Step by Step Build Instructions

Where do I find the Browser Capabilities File?

You can find a *browscap.ini* file at » http://browscap.org/.

What does thread safety mean when downloading PHP?

Thread Safety means that binary can work in a multithreaded webserver context, such as Apache 2 on Windows. Thread Safety works by creating a local storage copy in each thread, so that the data won't collide with another thread.

So what do I choose? If you choose to run PHP as a CGI binary, then you won't need thread safety, because the binary is invoked at each request. For multithreaded webservers, such as IIS5 and IIS6, you should use the threaded version of PHP.

Database issues

This section holds common questions about relation between PHP and databases. Yes, PHP can access virtually any database available today.

1. I heard it's possible to access Microsoft SQL Server from PHP. How?
2. Can I access Microsoft Access databases?

I heard it's possible to access Microsoft SQL Server from PHP. How?

On Unix machines you can use PDO_ODBC or the Unified ODBC API.

On Windows machines you can also use PDO_SQLSRV or SQLSRV.

Also see the answer to the next question.

Can I access Microsoft Access databases?

If you are running PHP on a Unix box and want to talk to MS Access on a Windows box you will need Unix ODBC drivers. » OpenLink Software has Unix-based ODBC drivers that can do this.

Another alternative is to use an SQL server that has Windows ODBC drivers and use that to store the data, which you can then access from Microsoft Access (using ODBC) and PHP (using the built in drivers), or to use an intermediary file format that Access and PHP both understand, such as flat files or dBase databases. On this point Tim Hayes from OpenLink software writes:

> Using another database as an intermediary is not a good idea, when you can use ODBC from PHP straight to your database - i.e. with OpenLink's drivers. If you do need to use an intermediary file format, OpenLink have now released Virtuoso (a virtual database engine) for NT, Linux and other Unix platforms. Please visit our » website for a free download.

One option that has proved successful is to use MySQL and its MyODBC drivers on Windows and synchronizing the databases. Steve Lawrence writes:

- Install MySQL on your platform according to instructions with MySQL. Latest available from » http://www.mysql.com/ No special configuration required except when you set up a database, and configure the user account, you should put % in the host field, or the host name of the Windows computer you wish to access MySQL with. Make a note of your server name, username, and password.
- Download the MyODBC for Windows driver from the MySQL site. Install it on your Windows machine. You can test the operation with the utilities included with this program.
- Create a user or system dsn in your ODBC administrator, located in the control panel. Make up a dsn name, enter your hostname, user name, password, port, etc for you MySQL database configured in step 1.
- Install Access with a full install, this makes sure you get the proper add-ins... at the least you will need ODBC support and the linked table manager.
- Now the fun part! Create a new access database. In the table window right click and select Link Tables, or under the file menu option, select Get External Data and then Link Tables. When the file browser box comes up, select files of type: ODBC. Select System dsn and the name of your dsn created in step 3. Select the table to link, press OK, and presto! You can now open the table and add/delete/edit data on your MySQL server! You can also build queries, import/export tables to MySQL, build forms and reports, etc.

Tips and Tricks:

- You can construct your tables in Access and export them to MySQL, then link them back in. That makes table creation quick.
- When creating tables in Access, you must have a primary key defined in order to have write access to the table in access. Make sure you create a primary key in MySQL before linking in access
- If you change a table in MySQL, you have to re-link it in Access. Go to tools>add-ins>linked table manager, cruise to your ODBC DSN, and select the table to re-link from there. you can also move your dsn source around there, just hit the always prompt for new location checkbox before pressing OK.

Installation

This section holds common questions about the way to install PHP. PHP is available for almost any OS, and almost any web server.

To install PHP, follow the instructions in Installation and Configuration.

1. Why shouldn't I use Apache2 with a threaded MPM in a production environment?
2. Unix/Windows: Where should my php.ini file be located?
3. Unix: I installed PHP, but every time I load a document, I get the message 'Document Contains No Data'! What's going on here?
4. Unix: I installed PHP using RPMS, but Apache isn't processing the PHP pages! What's going on here?
5. Unix: I patched Apache with the FrontPage extensions patch, and suddenly PHP stopped working. Is PHP incompatible with the Apache FrontPage extensions?
6. Unix/Windows: I have installed PHP, but when I try to access a PHP script file via my browser, I get a blank screen.
7. Unix/Windows: I have installed PHP, but when try to access a PHP script file via my browser, I get a server 500 error.
8. Some operating systems: I have installed PHP without errors, but when I try to start Apache I get undefined symbol errors: [mybox:user /src/php5] root# apachectl configtest apachectl: /usr/local/apache/bin/httpd Undefined symbols: _compress _uncompress
9. Windows: I have installed PHP, but when I try to access a PHP script file via my browser, I get the error: cgi error: The specified CGI application misbehaved by not returning a complete set of HTTP headers. The headers it did return are:
10. Windows: I've followed all the instructions, but still can't get PHP and IIS to work together!
11. When running PHP as CGI with IIS, PWS, OmniHTTPD or Xitami, I get the following error: Security Alert! PHP CGI cannot be accessed directly..
12. How do I know if my php.ini is being found and read? It seems like it isn't as my changes aren't being implemented.
13. How do I add my PHP directory to the PATH on Windows?
14. How do I make the php.ini file available to PHP on Windows?
15. Windows: How do I verify that PHP can write to the temporary directory under IIS?
16. Is it possible to use Apache content negotiation (MultiViews option) with PHP?
17. Is PHP limited to process GET and POST request methods only?

Why shouldn't I use Apache2 with a threaded MPM in a production environment?

PHP is glue. It is the glue used to build cool web applications by sticking dozens of 3rd-party libraries together and making it all appear as one coherent entity through an intuitive and easy to learn language interface. The flexibility and power of PHP relies on the stability and robustness of the underlying platform. It needs a working OS, a working web server and working 3rd-party libraries to glue together. When any of these stop working PHP needs ways to identify the problems and fix them quickly. When you make the underlying framework more complex by not having completely separate execution threads, completely separate memory segments and a strong sandbox for each request to play in, further weaknesses are introduced into PHP's system.

If you want to use a threaded MPM, look at a FastCGI configuration where PHP is running in its own memory space.

Unix/Windows: Where should my *php.ini* file be located?

By default on Unix it should be in */usr/local/lib* which is *<install-path>/lib*. Most people will want to change this at compile-time with the --with-config-file-path flag. You would, for example, set it with something like:

```
--with-config-file-path=/etc
```

And then you would copy *php.ini-development* from the distribution to */etc/php.ini* and edit it to make any local changes you want.

```
--with-config-file-scan-dir=PATH
```

On Windows the default path for the *php.ini* file is the Windows directory. If you're using the Apache webserver, *php.ini* is first searched in the Apaches install directory, e.g. *c:\program files\apache group\apache*. This way you can have different *php.ini* files for different versions of Apache on the same machine.

See also the chapter about the configuration file.

Unix: I installed PHP, but every time I load a document, I get the message 'Document Contains No Data'! What's going on here?

This probably means that PHP is having some sort of problem and is core-dumping. Look in your server error log to see if this is the case, and then try to reproduce the problem with a small test case. If you know how to use 'gdb', it is very helpful when you can provide a backtrace with your bug report to help the developers pinpoint the problem. If you are using PHP as an Apache module try something like:

- Stop your httpd processes
- gdb httpd
- Stop your httpd processes
- > run -X -f /path/to/httpd.conf
- Then fetch the URL causing the problem with your browser
- > run -X -f /path/to/httpd.conf
- If you are getting a core dump, gdb should inform you of this now
- type: bt
- You should include your backtrace in your bug report. This should be submitted to » https://github.com/php/php-src/issues

If your script uses the regular expression functions (preg_match() and friends), you should make sure that you compiled PHP and Apache with the same regular expression package. This should happen automatically with PHP and Apache 1.3.x

Unix: I installed PHP using RPMS, but Apache isn't processing the PHP pages! What's going on here?

Assuming you installed both Apache and PHP from RPM packages, you need to uncomment or add some or all of the following lines in your *httpd.conf* file:

```
# Extra Modules
AddModule mod_php.c
AddModule mod_perl.c

# Extra Modules
LoadModule php_module         modules/mod_php.so
```

```
LoadModule php5_module        modules/libphp5.so
LoadModule perl_module        modules/libperl.so
```

And add:

```
AddType application/x-httpd-php .php
```

... to the global properties, or to the properties of the VirtualDomain you want to have PHP support added to.

Unix: I patched Apache with the FrontPage extensions patch, and suddenly PHP stopped working. Is PHP incompatible with the Apache FrontPage extensions?

No, PHP works fine with the FrontPage extensions. The problem is that the FrontPage patch modifies several Apache structures, that PHP relies on. Recompiling PHP (using 'make clean ; make') after the FP patch is applied would solve the problem.

Unix/Windows: I have installed PHP, but when I try to access a PHP script file via my browser, I get a blank screen.

Do a 'view source' in the web browser and you will probably find that you can see the source code of your PHP script. This means that the web server did not send the script to PHP for interpretation. Something is wrong with the server configuration - double check the server configuration against the PHP installation instructions.

Unix/Windows: I have installed PHP, but when try to access a PHP script file via my browser, I get a server 500 error.

Something went wrong when the server tried to run PHP. To get to see a sensible error message, from the command line, change to the directory containing the PHP executable (*php.exe* on Windows) and run **php -i**. If PHP has any problems running, then a suitable error message will be displayed which will give you a clue as to what needs to be done next. If you get a screen full of HTML codes (the output of the phpinfo() function) then PHP is working, and your problem may be related to your server configuration which you should double check.

Some operating systems: I have installed PHP without errors, but when I try to start Apache I get undefined symbol errors:

```
[mybox:user /src/php5] root# apachectl configtest
 apachectl: /usr/local/apache/bin/httpd Undefined symbols:
  _compress
  _uncompress
```

This has actually nothing to do with PHP, but with the MySQL client libraries. Some need **--with-zlib**, others do not. This is also covered in the MySQL FAQ.

Windows: I have installed PHP, but when I try to access a PHP script file via my browser, I get the error:

```
cgi error:
 The specified CGI application misbehaved by not
 returning a complete set of HTTP headers.
 The headers it did return are:
```

This error message means that PHP failed to output anything at all. To get to see a sensible error message, from the command line, change to the directory containing the PHP executable (*php.exe* on Windows) and run **php -i**. If PHP has any problems running, then a suitable error message will be displayed which will give you a clue as to what needs to be done next. If you get a screen full of HTML codes (the output of the phpinfo() function) then PHP is working.

Once PHP is working at the command line, try accessing the script via the browser again. If it still fails then it could be one of the following:

- File permissions on your PHP script, *php.exe*, *php5ts.dll*, *php.ini* or any PHP extensions you are trying to load are such that the anonymous internet user `ISUR_<machinename>` cannot access them.
- The script file does not exist (or possibly isn't where you think it is relative to your web root directory). Note that for IIS you can trap this error by ticking the 'check file exists' box when setting up the script mappings in the Internet Services Manager. If a script file does not exist then the server will return a 404 error instead. There is also the additional benefit that IIS will do any authentication required for you based on the NTLanMan permissions on your script file.

Windows: I've followed all the instructions, but still can't get PHP and IIS to work together!

Make sure any user who needs to run a PHP script has the rights to run *php.exe*! IIS uses an anonymous user which is added at the time IIS is installed. This user needs rights to *php.exe*. Also, any authenticated user will also need rights to execute *php.exe*. And for IIS4 you need to tell it that PHP is a script engine. Also, you will want to read this faq.

When running PHP as CGI with IIS, PWS, OmniHTTPD or Xitami, I get the following error: `Security Alert! PHP CGI cannot be accessed directly..`

You must set the cgi.force_redirect directive to `0`. It defaults to `1` so be sure the directive isn't commented out (with a `;`). Like all directives, this is set in *php.ini*

Because the default is `1`, it's critical that you're 100% sure that the correct *php.ini* file is being read. Read this faq for details.

How do I know if my *php.ini* is being found and read? It seems like it isn't as my changes aren't being implemented.

To be sure your *php.ini* is being read by PHP, make a call to phpinfo(). Near the top, there will be a listing called `Configuration File (php.ini)`. This will tell you where PHP is looking for *php.ini* and whether or not it's being read. If just a directory *PATH* exists, then it's not being read, and you should put your *php.ini* in that directory. If *php.ini* is included within the *PATH*, it is being read.

If *php.ini* is being read and you're running PHP as a module, then be sure to restart your web server after making changes to *php.ini*

See also php_ini_loaded_file().

How do I add my PHP directory to the *PATH* on Windows?

On Windows:

- Go to Control Panel and open the System icon (Start → Control Panel)

- Go to the Advanced tab
- Click on the 'Environment Variables' button
- Look into the 'System Variables' pane
- Find the Path entry (you may need to scroll to find it)
- Double click on the Path entry
- Enter your PHP directory at the end, including ';' before (e.g. `;C:\php`)
- Press OK

> **Note** :
>
> Be sure to reboot after following the steps above to ensure that the *PATH* changes are applied.

How do I make the *php.ini* file available to PHP on Windows?

There are several ways of doing this. If you are using Apache, refer to the Apache documentation, otherwise you must set the *PHPRC* environment variable.

Windows: How do I verify that PHP can write to the temporary directory under IIS?

1. Right-click the temporary directory (*%TEMP%*) in File Explorer to get the permissions. The temporary directory is available from the configuration or phpinfo().
2. For IIS, check that user `IIS_User` has `MODIFY` permission.

Is it possible to use Apache content negotiation (MultiViews option) with PHP?

If links to PHP files include extension, everything works perfect. This FAQ is only for the case when links to PHP files don't include extension and you want to use content negotiation to choose PHP files from URL with no extension. In this case, replace the line `AddType application/x-httpd-php .php` with:

```
AddHandler php5-script php
AddType text/html php
```

This solution doesn't work for Apache 1 as PHP module doesn't catch `php-script`.

Is PHP limited to process GET and POST request methods only?

No, it is possible to handle any request method, e.g. CONNECT. Proper response status can be sent with header(). If only GET and POST methods should be handled, it can be achieved with this Apache configuration:

```
<LimitExcept GET POST>
Deny from all
</LimitExcept>
```

Build Problems

This section gathers most common errors that occur at build time.

1. I got the latest version of PHP using the anonymous Git service, but there's no configure script!
2. I'm having problems configuring PHP to work with Apache. It says it can't find httpd.h, but it's right where I said it is!
3. While configuring PHP (./configure), you come across an error similar to the following: checking lex output file root... ./configure: lex: command not found configure: error: cannot find output from lex; giving up
4. When I try to start Apache, I get the following message: fatal: relocation error: file /path/to/libphp4.so: symbol ap_block_alarms: referenced symbol not found
5. When I run configure, it says that it can't find the include files or library for GD, gdbm, or some other package!
6. When it is compiling the file language-parser.tab.c, it gives me errors that say yytname undeclared.
7. When I run make, it seems to run fine but then fails when it tries to link the final application complaining that it can't find some files.
8. When linking PHP, it complains about a number of undefined references.
9. I have followed all the steps to install the Apache module version on Unix, and my PHP scripts show up in my browser or I am being asked to save the file.
10. It says to use: --activate-module=src/modules/php4/libphp4.a, but that file doesn't exist, so I changed it to --activate-module=src/modules/php4/libmodphp4.a and it doesn't work!? What's going on?
11. When I try to build Apache with PHP as a static module using --activate-module=src/modules/php4/libphp4.a it tells me that my compiler is not ANSI compliant.
12. When I try to build PHP using --with-apxs I get strange error messages.
13. During make, I get errors in microtime, and a lot of RUSAGE_ stuff.
14. When compiling PHP with MySQL, configure runs fine but during make I get an error similar to the following: ext/mysql/libmysqlclient/my_tempnam.o(.text+0x46): In function my_tempnam': /php4/ext/mysql/libmysqlclient/my_tempnam.c:103: the use of tempnam' is dangerous, better use mkstemp', what's wrong?
15. I want to upgrade my PHP. Where can I find the ./configure line that was used to build my current PHP installation?
16. When building PHP with the GD library it either gives strange compile errors or segfaults on execution.
17. When compiling PHP I seemingly get random errors, like it hangs. I'm using Solaris if that matters.

I got the latest version of PHP using the anonymous Git service, but there's no configure script!

> You have to have the GNU autoconf package installed so you can generate the configure script from *configure.in*. Just run **./buildconf** in the top-level directory after getting the sources from the Git server. (Also, unless you run **configure** with the `--enable-maintainer-mode` option, the configure script will not automatically get rebuilt when the *configure.in* file is updated, so you should make sure to do that manually when you notice *configure.in* has changed. One symptom of this is finding things like @VARIABLE@ in your Makefile after configure or *config.status* is run.)

I'm having problems configuring PHP to work with Apache. It says it can't find *httpd.h*, but it's right where I said it is!

> You need to tell the configure/setup script the location of the top-level of your Apache source tree. This means that you want to specify **--with-apache=/path/to/apache** and *not* **--with-apache=/path/to/apache/src**.

While configuring PHP (`./configure`), you come across an error similar to the following:

```
checking lex output file root... ./configure: lex: command not found

configure: error: cannot find output from lex; giving up
```

Be sure to read the installation instructions carefully and note that you need both flex and bison installed to compile PHP. Depending on your setup you will install bison and flex from either source or a package, such as a RPM.

When I try to start Apache, I get the following message:

```
fatal: relocation error: file /path/to/libphp4.so:

symbol ap_block_alarms: referenced symbol not found
```

This error usually comes up when one compiles the Apache core program as a DSO library for shared usage. Try to reconfigure apache, making sure to use at least the following flags:

```
--enable-shared=max --enable-rule=SHARED_CORE
```

For more information, read the top-level Apache *INSTALL* file or the Apache » DSO manual page.

When I run configure, it says that it can't find the include files or library for GD, gdbm, or some other package!

You can make the configure script look for header files and libraries in non-standard locations by specifying additional flags to pass to the C preprocessor and linker, such as:

```
CPPFLAGS=-I/path/to/include LDFLAGS=-L/path/to/library ./configure
```

If you're using a csh-variant for your login shell (why?), it would be:

```
env CPPFLAGS=-I/path/to/include LDFLAGS=-L/path/to/library ./configure
```

When it is compiling the file *language-parser.tab.c*, it gives me errors that say `yytname undeclared`.

You need to update your version of Bison. You can find the latest version at » http://www.gnu.org/software/bison/bison.html.

When I run make, it seems to run fine but then fails when it tries to link the final application complaining that it can't find some files.

Some old versions of make don't correctly put the compiled versions of the files in the functions directory into that same directory. Try running **cp *.o functions** and then re-running **make** to see if that helps. If it does, you should really upgrade to a recent version of GNU make.

When linking PHP, it complains about a number of undefined references.

Take a look at the link line and make sure that all of the appropriate libraries are being included at the end. Common ones that you might have missed are '-ldl' and any libraries required for any database support you included.

Some people have also reported that they had to add '-ldl' immediately following *libphp4.a* when linking with Apache.

I have followed all the steps to install the Apache module version on Unix, and my PHP scripts show up in my browser or I am being asked to save the file.

This means that the PHP module is not getting invoked for some reason. Three things to check before asking for further help:

- Make sure that the httpd binary you are running is the actual new httpd binary you just built. To do this, try running: `/path/to/binary/httpd -l`

  If you don't see *mod_php4.c* listed then you are not running the right binary. Find and install the correct binary.

- Make sure you have added the correct Mime Type to one of your `Apache .conf` files. It should be: `AddType application/x-httpd-php .php`

  Also make sure that this AddType line is not hidden away inside a <Virtualhost> or <Directory> block which would prevent it from applying to the location of your test script.

- Finally, the default location of the Apache configuration files changed between Apache 1.2 and Apache 1.3. You should check to make sure that the configuration file you are adding the AddType line to is actually being read. You can put an obvious syntax error into your *httpd.conf* file or some other obvious change that will tell you if the file is being read correctly.

It says to use: `--activate-module=src/modules/php4/libphp4.a`, but that file doesn't exist, so I changed it to `--activate-module=src/modules/php4/libmodphp4.a` and it doesn't work!? What's going on?

Note that the *libphp4.a* file is not supposed to exist. The apache process will create it!

When I try to build Apache with PHP as a static module using `--activate-module=src/modules/php4/libphp4.a` it tells me that my compiler is not ANSI compliant.

This is a misleading error message from Apache that has been fixed in more recent versions.

When I try to build PHP using --with-apxs I get strange error messages.

There are three things to check here. First, for some reason when Apache builds the apxs Perl script, it sometimes ends up getting built without the proper compiler and flags variables. Find your apxs script (try the command **which apxs**), it's sometimes found in */usr/local/apache/bin/apxs* or */usr/sbin/apxs*. Open it and check for lines similar to these:

```
my $CFG_CFLAGS_SHLIB  = ' ';          # substituted via Makefile.tmpl
my $CFG_LD_SHLIB      = ' ';          # substituted via Makefile.tmpl
my $CFG_LDFLAGS_SHLIB = ' ';          # substituted via Makefile.tmpl
```

If this is what you see, you have found your problem. They may contain just spaces or other incorrect values, such as 'q()'. Change these lines to say:

```
my $CFG_CFLAGS_SHLIB  = '-fpic -DSHARED_MODULE'; # substituted via Makefile.tmpl
my $CFG_LD_SHLIB      = 'gcc';                   # substituted via Makefile.tmpl
my $CFG_LDFLAGS_SHLIB = q(-shared);              # substituted via Makefile.tmpl
```

The second possible problem should only be an issue on Red Hat 6.1 and 6.2. The apxs script Red Hat ships is broken. Look for this line:

```
my $CFG_LIBEXECDIR    = 'modules';         # substituted via APACI install
```

If you see the above line, change it to this:

```
my $CFG_LIBEXECDIR    = '/usr/lib/apache'; # substituted via APACI install
```

Last, if you reconfigure/reinstall Apache, add a **make clean** to the process after **./configure** and before **make**.

During make, I get errors in microtime, and a lot of `RUSAGE_` stuff.

During the **make** portion of installation, if you encounter problems that look similar to this:

```
microtime.c: In function `php_if_getrusage':
microtime.c:94: storage size of `usg' isn't known
microtime.c:97: `RUSAGE_SELF' undeclared (first use in this function)
microtime.c:97: (Each undeclared identifier is reported only once
microtime.c:97: for each function it appears in.)
microtime.c:103: `RUSAGE_CHILDREN' undeclared (first use in this function)
make[3]: *** [microtime.lo] Error 1
make[3]: Leaving directory `/home/master/php-4.0.1/ext/standard'
make[2]: *** [all-recursive] Error 1
make[2]: Leaving directory `/home/master/php-4.0.1/ext/standard'
make[1]: *** [all-recursive] Error 1
make[1]: Leaving directory `/home/master/php-4.0.1/ext'
make: *** [all-recursive] Error 1
```

Your system is broken. You need to fix your */usr/include* files by installing a glibc-devel package that matches your glibc. This has absolutely nothing to do with PHP. To prove this to yourself, try this simple test:

```
$ cat >test.c <<X
#include <sys/resource.h>
X
$ gcc -E test.c >/dev/null
```

If that spews out errors, you know your include files are messed up.

When compiling PHP with MySQL, configure runs fine but during `make` I get an error similar to the following: *ext/mysql/libmysqlclient/my_tempnam.o(.text+0x46): In function my_tempnam':*

/php4/ext/mysql/libmysqlclient/my_tempnam.c:103: the use of tempnam' is dangerous, better use mkstemp', what's wrong?

First, it's important to realize that this is a `Warning` and not a fatal error. Because this is often the last output seen during `make`, it may seem like a fatal error but it's not. Of course, if you set your compiler to die on Warnings, it will. Also keep in mind that MySQL support is enabled by default.

> **Note** :
>
> As of PHP 4.3.2, you'll also see the following text after the build (make) completes:
>
> ```
> Build complete.
>
> (It is safe to ignore warnings about tempnam and tmpnam).
> ```

I want to upgrade my PHP. Where can I find the *./configure* line that was used to build my current PHP installation?

Either you look at config.nice file, in the source tree of your current PHP installation or, if this is not available, you simply run a

```
<?php phpinfo(); ?>
```

script. On top of the output the **./configure** line, that was used to build this PHP installation is shown.

When building PHP with the GD library it either gives strange compile errors or segfaults on execution.

Make sure your GD library and PHP are linked against the same depending libraries (e.g. libpng).

When compiling PHP I seemingly get random errors, like it hangs. I'm using Solaris if that matters.

Using non-GNU utilities while compiling PHP may cause problems. Be sure to use GNU tools in order to be certain that compiling PHP will work. For example, on Solaris, using either the SunOS BSD-compatible or Solaris versions of `sed` will not work, but using the GNU or Sun POSIX (xpg4) versions of `sed` will work. Links: » GNU sed, » GNU flex, and » GNU bison.

Using PHP

This section gathers many common errors that you may face while writing PHP scripts.

1. I cannot remember the parameter order of PHP functions, are they random?
2. I would like to write a generic PHP script that can handle data coming from any form. How do I know which POST method variables are available?
3. I need to convert all single-quotes (') to a backslash followed by a single-quote (\'). How can I do this with a regular expression? I'd also like to convert " to \" and \ to \\.
4. When I do the following, the output is printed in the wrong order: <?php function myfunc($argument) { echo $argument + 10; } $variable = 10; echo "myfunc($variable) = " . myfunc($variable); ?> what's going on?

5. Hey, what happened to my newlines? <pre> <?php echo "This should be the first line."; ?> <?php echo "This should show up after the new line above."; ?> </pre>
6. I get the message 'Warning: Cannot send session cookie - headers already sent...' or 'Cannot add header information - headers already sent...'.
7. I need to access information in the request header directly. How can I do this?
8. When I try to use authentication with IIS I get 'No Input file specified'.
9. Windows: I can't access files shared on another computer using IIS
10. How am I supposed to mix XML and PHP? It complains about my <?xml tags!
11. Where can I find a complete list of variables are available to me in PHP?
12. How can I generate PDF files without using the non-free and commercial libraries like PDFLib? I'd like something that's free and doesn't require external PDF libraries.
13. A few PHP directives may also take on shorthand byte values, as opposed to only int byte values. What are all the available shorthand byte options?

I cannot remember the parameter order of PHP functions, are they random?

PHP is a glue that brings together hundreds of external libraries, so sometimes this gets messy. However, a simple rule of thumb is as follows:

Array function parameters are ordered as "*needle, haystack*" whereas String functions are the opposite, so "*haystack, needle*".

As of PHP 8.0, named arguments allow passing arguments by parameter name, making parameter order less of a concern.

I would like to write a generic PHP script that can handle data coming from any form. How do I know which POST method variables are available?

PHP offers many predefined variables, like the superglobal *$_POST*. You may loop through *$_POST* as it's an associate array of all POSTed values. For example, let's simply loop through it with `foreach`, check for empty() values, and print them out.

```
<?php
$empty = $post = array();
foreach ($_POST as $varname => $varvalue) {
    if (empty($varvalue)) {
        $empty[$varname] = $varvalue;
    } else {
        $post[$varname] = $varvalue;
    }
}

print "<pre>";
if (empty($empty)) {
    print "None of the POSTed values are empty, posted:\n";
    var_dump($post);
} else {
    print "We have " . count($empty) . " empty values\n";
    print "Posted:\n"; var_dump($post);
    print "Empty:\n";  var_dump($empty);
    exit;
}
?>
```

I need to convert all single-quotes (') to a backslash followed by a single-quote (\'). How can I do this with a regular expression? I'd also like to convert " to \" and \ to \\.

> Assuming this is for a database, use the escaping mechanism that comes with the database. For example, use mysql_real_escape_string() with MySQL and pg_escape_string() with PostgreSQL. There is also the generic addslashes() and stripslashes() functions, that are more common with older PHP code.
>
> Manually escaping values is error-prone and depends on context. Prefer using database APIs that support prepared statements and parameter binding instead of constructing queries by concatenating escaped strings.

When I do the following, the output is printed in the wrong order:

```
<?php
function myfunc($argument)
{
    echo $argument + 10;
}
$variable = 10;
echo "myfunc($variable) = " . myfunc($variable);
?>
```

what's going on?

> To be able to use the results of your function in an expression (such as concatenating it with other strings in the example above), you need to return the value, not echo it.

Hey, what happened to my newlines?

```
<pre>
<?php echo "This should be the first line."; ?>
<?php echo "This should show up after the new line above."; ?>
</pre>
```

> In PHP, the ending for a block of code is either "?>" or "?>\n" (where \n means a newline). So in the example above, the echoed sentences will be on one line, because PHP omits the newlines after the block ending. This means that you need to insert an extra newline after each block of PHP code to make it print out one newline.
>
> Why does PHP do this? Because when formatting normal HTML, this usually makes your life easier because you don't want that newline, but you'd have to create extremely long lines or otherwise make the raw page source unreadable to achieve that effect.

I get the message 'Warning: Cannot send session cookie - headers already sent...' or 'Cannot add header information - headers already sent...'.

> The functions header(), setcookie(), and the session functions need to add headers to the output stream but headers can only be sent before all other content. There can be no output before using these functions, output such as HTML. The function headers_sent() will check if your script has already sent headers and see also the Output Control functions.

I need to access information in the request header directly. How can I do this?

> The getallheaders() function will do this if you are running PHP as an Apache module. So, the following bit of code will show you all the request headers:

```php
<?php
$headers = getallheaders();
foreach ($headers as $name => $content) {
    echo "headers[$name] = $content<br />\n";
}
?>
```

See also apache_lookup_uri(), apache_response_headers(), and fsockopen()

When I try to use authentication with IIS I get 'No Input file specified'.

The security model of IIS is at fault here. This is a problem common to all CGI programs running under IIS. A workaround is to create a plain HTML file (not parsed by PHP) as the entry page into an authenticated directory. Then use a META tag to redirect to the PHP page, or have a link to the PHP page. PHP will then recognize the authentication correctly. This should not affect other NT web servers. For more information, see: » http://support.microsoft.com/kb/q160422/ and the manual section on HTTP Authentication .

Windows: I can't access files shared on another computer using IIS

You have to change the `Go to Internet Information Services`. Locate your PHP file and go to its properties. Go to the `File Security` tab, `Edit -< Anonymous access and authentication control`.

You can fix the problem either by unticking `Anonymous Access` and leaving `Integrated Window Authentication` ticked, or, by ticking `Anonymous Access` and editing the user as he may not have the access right.

How am I supposed to mix XML and PHP? It complains about my <?xml tags!

In order to embed <?xml straight into your PHP code, you'll have to turn off short tags by having the PHP directive short_open_tags set to `0`. You cannot set this directive with ini_set(). Regardless of short_open_tags being on or off, you can do something like: `<?php echo '<?xml'; ?>`. The default for this directive is `On`.

Where can I find a complete list of variables are available to me in PHP?

Read the manual page on predefined variables as it includes a partial list of predefined variables available to your script. A complete list of available variables (and much more information) can be seen by calling the phpinfo() function. Be sure to read the manual section on variables from outside of PHP as it describes common scenarios for external variables, like from a HTML form, a Cookie, and the URL.

How can I generate PDF files without using the non-free and commercial libraries like PDFLib? I'd like something that's free and doesn't require external PDF libraries.

There are a few alternatives written in PHP such as » FPDF and » TCPDF.

A few PHP directives may also take on shorthand byte values, as opposed to only int byte values. What are all the available shorthand byte options?

The available options are K (for Kilobytes), M (for Megabytes) and G (for Gigabytes), and are all case-insensitive. Anything else assumes bytes. `1M` equals one Megabyte or `1048576` bytes. `1K` equals one Kilobyte or `1024` bytes. These shorthand notations may be used in *php.ini* and in the ini_set() function. Note that the numeric value is cast to int; for instance, `0.5M` is interpreted as `0`.

> **Note** : **kilobyte versus kibibyte**
>
> The PHP notation describes one kilobyte as equalling 1024 bytes, whereas the IEC standard considers this to be a kibibyte instead. Summary: k and K = 1024 bytes.

Password Hashing

This section explains the reasons behind using hashing functions to secure passwords, as well as how to do so effectively.

1. Why should passwords supplied by users be hashed?
2. Why are common hashing functions such as md5 and sha1 unsuitable for passwords?
3. How should passwords be hashed, if the common hash functions are not suitable?
4. What is a salt?
5. How are salts stored?

Why should passwords supplied by users be hashed?

Password hashing is one of the most basic security considerations that must be made when designing any application or service that accepts passwords from users. Without hashing, any passwords that are stored can be stolen if the data store is compromised, and then immediately used to compromise not only the application or service, but also the accounts of users on other services, if they do not use unique passwords.

By applying a hashing algorithm to the user's passwords before storing them, it becomes implausible for any attacker to determine the original password, while still being able to compare the resulting hash to the original password in the future.

It is important to note, however, that hashing passwords only protects them from being compromised in the data store, but does not necessarily protect them from being intercepted by malicious code injected into the application or service itself.

Why are common hashing functions such as md5() and sha1() unsuitable for passwords?

Hashing algorithms such as MD5, SHA1 and SHA256 are designed to be very fast and efficient. With modern techniques and computer equipment, it has become trivial to “brute force” the output of these algorithms, in order to determine the original input.

Because of how quickly a modern computer can “reverse” these hashing algorithms, many security professionals strongly suggest against their use for password hashing.

How should passwords be hashed, if the common hash functions are not suitable?

When hashing passwords, the two most important considerations are the computational expense, and the salt. The more computationally expensive the hashing algorithm, the longer it will take to brute force its output.

PHP provides a native password hashing API that safely handles both hashing and verifying passwords in a secure manner.

The suggested algorithm to use when hashing passwords is Blowfish, which is also the default used by the password hashing API, as it is significantly more computationally expensive than MD5 or SHA1, while still being scalable.

The crypt() function is also available for password hashing, but it is only recommended for interoperability with other systems. Instead, it is strongly encouraged to use the native password hashing API whenever possible.

What is a salt?

A cryptographic salt is data which is applied during the hashing process in order to eliminate the possibility of the output being looked up in a list of pre-calculated pairs of hashes and their input, known as a rainbow table.

In more simple terms, a salt is a bit of additional data which makes hashes significantly more difficult to crack. There are a number of services online which provide extensive lists of pre-computed hashes, as well as the original input for those hashes. The use of a salt makes it implausible or impossible to find the resulting hash in one of these lists.

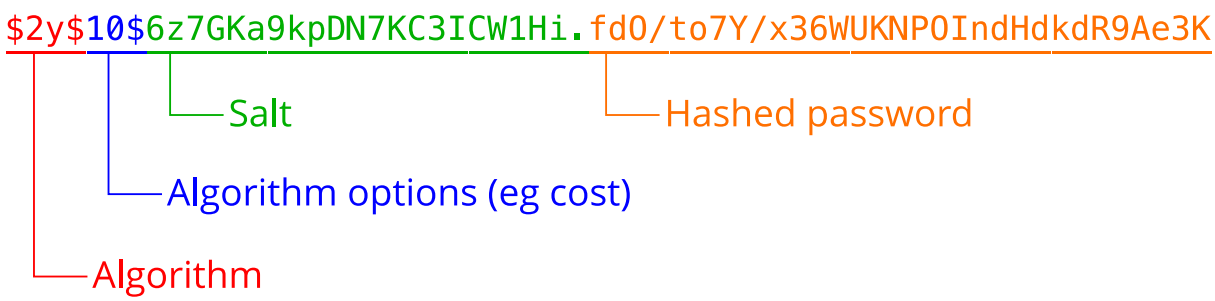
password_hash() will create a random salt if one isn't provided, and this is generally the easiest and most secure approach.

How are salts stored?

When using password_hash() or crypt(), the return value includes the salt as part of the generated hash. This value should be stored verbatim in the database, as it includes information about the hash function that was used and can then be given directly to password_verify() when verifying passwords.

> **Warning**
>
> password_verify() should always be used instead of re-hashing and comparing the result to a stored hash in order to avoid timing attacks.

The following diagram shows the format of a return value from crypt() or password_hash(). As can be seen, they are self-contained, with all the information on the algorithm and salt required for future password verification.



PHP and HTML

PHP and HTML interact a lot: PHP can generate HTML, and HTML can pass information to PHP. Before reading these faqs, it's important you learn how to retrieve variables from external sources. The manual page on this

topic includes many examples as well.

1. What encoding/decoding do I need when I pass a value through a form/URL?
2. I'm trying to use an <input type="image"> tag, but the $foo.x and $foo.y variables aren't available. $_GET['foo.x'] isn't existing either. Where are they?
3. How do I create arrays in a HTML <form>?
4. How do I get all the results from a select multiple HTML tag?
5. How can I pass a variable from Javascript to PHP?

What encoding/decoding do I need when I pass a value through a form/URL?

There are several stages for which encoding is important. Assuming that you have a string *$data*, which contains the string you want to pass on in a non-encoded way, these are the relevant stages:

- HTML interpretation. In order to specify a random string, you *must* include it in double quotes, and htmlspecialchars() the whole value.
- URL: A URL consists of several parts. If you want your data to be interpreted as one item, you *must* encode it with urlencode().

Example #1 A hidden HTML form element

```
<?php
    echo '<input type="hidden" value="' . htmlspecialchars($data) . '" />'."\n";
?>
```

> **Note** :
>
> It is wrong to urlencode() *$data*, because it's the browsers responsibility to urlencode() the data. All popular browsers do that correctly. Note that this will happen regardless of the method (i.e., GET or POST). You'll only notice this in case of GET request though, because POST requests are usually hidden.

Example #2 Data to be edited by the user

```
<?php
    echo "<textarea name='mydata'>\n";
    echo htmlspecialchars($data)."\n";
    echo "</textarea>";
?>
```

> **Note** :
>
> The data is shown in the browser as intended, because the browser will interpret the HTML escaped symbols.
>
> Upon submitting, either via GET or POST, the data will be urlencoded by the browser for transferring, and directly urldecoded by PHP. So in the end, you don't need to do any urlencoding/urldecoding yourself, everything is handled automagically.

Example #3 In a URL

```php
<?php
    echo '<a href="' . htmlspecialchars("/nextpage.php?stage=23&data=" .
        urlencode($data)) . '">'."\n";
?>
```

> **Note** :
>
> In fact you are faking a HTML GET request, therefore it's necessary to manually urlencode() the data.

> **Note** :
>
> You need to htmlspecialchars() the whole URL, because the URL occurs as value of an HTML-attribute. In this case, the browser will first un-htmlspecialchars() the value, and then pass the URL on. PHP will understand the URL correctly, because you urlencode()d the data.
>
> You'll notice that the `&` in the URL is replaced by `&`. Although most browsers will recover if you forget this, this isn't always possible. So even if your URL is not dynamic, you *need* to htmlspecialchars() the URL.

I'm trying to use an <input type="image"> tag, but the *$foo.x* and *$foo.y* variables aren't available. *$_GET['foo.x']* isn't existing either. Where are they?

When submitting a form, it is possible to use an image instead of the standard submit button with a tag like:

```
<input type="image" src="image.gif" name="foo" />
```

When the user clicks somewhere on the image, the accompanying form will be transmitted to the server with two additional variables: *foo.x* and *foo.y*.

Because *foo.x* and *foo.y* would make invalid variable names in PHP, they are automagically converted to *foo_x* and *foo_y*. That is, the periods are replaced with underscores. So, you'd access these variables like any other described within the section on retrieving variables from external sources. For example, *$_GET['foo_x']*.

> **Note** :
>
> Spaces in request variable names are converted to underscores.

How do I create arrays in a HTML <form>?

To get your <form> result sent as an array to your PHP script you name the <input>, <select> or <textarea> elements like this:

```
<input name="MyArray[]" />
<input name="MyArray[]" />
<input name="MyArray[]" />
<input name="MyArray[]" />
```

Notice the square brackets after the variable name, that's what makes it an array. You can group the elements into different arrays by assigning the same name to different elements:

```
<input name="MyArray[]" />
<input name="MyArray[]" />
<input name="MyOtherArray[]" />
<input name="MyOtherArray[]" />
```

This produces two arrays, MyArray and MyOtherArray, that gets sent to the PHP script. It's also possible to assign specific keys to your arrays:

```
<input name="AnotherArray[]" />
<input name="AnotherArray[]" />
<input name="AnotherArray[email]" />
<input name="AnotherArray[phone]" />
```

The AnotherArray array will now contain the keys 0, 1, email and phone.

> **Note** :
>
> Specifying array keys is optional in HTML. If you do not specify the keys, the array gets filled in the order the elements appear in the form. Our first example will contain keys 0, 1, 2 and 3.

See also Array Functions and Variables From External Sources.

How do I get all the results from a select multiple HTML tag?

The select multiple tag in an HTML construct allows users to select multiple items from a list. These items are then passed to the action handler for the form. The problem is that they are all passed with the same widget name. I.e.

```
<select name="var" multiple="yes">
```

Each selected option will arrive at the action handler as:

```
var=option1
var=option2
var=option3
```

Each option will overwrite the contents of the previous *$var* variable. The solution is to use PHP's "array from form element" feature. The following should be used:

```
<select name="var[]" multiple="yes">
```

This tells PHP to treat *$var* as an array and each assignment of a value to var[] adds an item to the array. The first item becomes *$var[0]*, the next *$var[1]*, etc. The count() function can be used to determine how many options were selected, and the sort() function can be used to sort the option array if necessary.

Note that if you are using JavaScript the `[]` on the element name might cause you problems when you try to refer to the element by name. Use it's numerical form element ID instead, or enclose the variable

name in single quotes and use that as the index to the elements array, for example:

```
variable = document.forms[0].elements['var[]'];
```

How can I pass a variable from Javascript to PHP?

Since Javascript is (usually) a client-side technology, and PHP is (usually) a server-side technology, and since HTTP is a "stateless" protocol, the two languages cannot directly share variables.

It is, however, possible to pass variables between the two. One way of accomplishing this is to generate Javascript code with PHP, and have the browser refresh itself, passing specific variables back to the PHP script. The example below shows precisely how to do this -- it allows PHP code to capture screen height and width, something that is normally only possible on the client side.

Example #4 Generating Javascript with PHP

```
<?php
if (isset($_GET['width']) AND isset($_GET['height'])) {
  // output the geometry variables
  echo "Screen width is: ". $_GET['width'] ."<br />\n";
  echo "Screen height is: ". $_GET['height'] ."<br />\n";
} else {
  // pass the geometry variables
  // (preserve the original query string
  //   -- post variables will need to handled differently)

  echo "<script language='javascript'>\n";
  echo "  location.href=\"{$_SERVER['SCRIPT_NAME']}?{$_SERVER['QUERY_STRING']}"
            . "&width=\" + screen.width + \"&height=\" + screen.height;\n";
  echo "</script>\n";
  exit();
}
?>
```

PHP and COM

PHP can be used to access COM and DCOM objects on Win32 platforms.

1. I have built a DLL to calculate something. Is there any way to run this DLL under PHP?
2. What does 'Unsupported variant type: xxxx (0xxxxx)' mean?
3. Is it possible manipulate visual objects in PHP?
4. Can I store a COM object in a session?
5. How can I trap COM errors?
6. Can I generate DLL files from PHP scripts like I can in Perl?
7. What does 'Unable to obtain IDispatch interface for CLSID {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}' mean?
8. How can I run COM object from remote server?
9. I get 'DCOM is disabled in C:\path...\scriptname.php on line 6', what can I do?
10. Is it possible to load/manipulate an ActiveX object in a page with PHP?
11. Is it possible to get a running instance of a component?
12. Is there a way to handle an event sent from COM object?

13. I'm having problems when trying to invoke a method of a COM object which exposes more than one interface. What can I do?
14. So PHP works with COM, how about COM+?
15. If PHP can manipulate COM objects, can we imagine to use MTS to manage components resources, in conjunction with PHP?

I have built a DLL to calculate something. Is there any way to run this DLL under PHP?

If this is a simple DLL there is no way yet to run it from PHP. If the DLL contains a COM server you may be able to access it if it implements the IDispatch interface.

What does 'Unsupported variant type: xxxx (0xxxxx)' mean?

There are dozens of VARIANT types and combinations of them. Most of them are already supported but a few still have to be implemented. Arrays are not completely supported. Only single dimensional indexed only arrays can be passed between PHP and COM. If you find other types that aren't supported, please report them as a bug (if not already reported) and provide as much information as available.

Is it possible manipulate visual objects in PHP?

Generally it is, but as PHP is mostly used as a web scripting language it runs in the web servers context, thus visual objects will never appear on the servers desktop. If you use PHP for application scripting e.g. in conjunction with PHP-GTK there is no limitation in accessing and manipulating visual objects through COM.

Can I store a COM object in a session?

No, you can't. COM instances are treated as resources and therefore they are only available in a single script's context.

How can I trap COM errors?

The COM extension throws `com_exception` exceptions, which you can catch and then inspect the `code` member to determine what to do next.

Can I generate DLL files from PHP scripts like I can in Perl?

No, unfortunately there is no such tool available for PHP.

What does 'Unable to obtain IDispatch interface for CLSID {xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}' mean?

This error can have multiple reasons:

- the CLSID is wrong
- the requested DLL is missing
- the requested component doesn't implement the IDispatch interface

How can I run COM object from remote server?

Exactly like you run local objects. You only have to pass the IP of the remote machine as second parameter to the COM constructor.

Make sure that you have set com.allow_dcom=**true** in your *php.ini*.

I get 'DCOM is disabled in C:\path...\scriptname.php on line 6', what can I do?

Edit your *php.ini* and set com.allow_dcom=**true**.

Is it possible to load/manipulate an ActiveX object in a page with PHP?

This has nothing to do with PHP. ActiveX objects are loaded on client side if they are requested by the HTML document. There is no relation to the PHP script and therefore there is no direct server side interaction possible.

Is it possible to get a running instance of a component?

This is possible with the help of monikers. If you want to get multiple references to the same word instance you can create that instance like shown:

```
<?php
$word = new COM("C:\docs\word.doc");
?>
```

This will create a new instance if there is no running instance available or it will return a handle to the running instance, if available.

Is there a way to handle an event sent from COM object?

You can define an event sink and bind it using com_event_sink(). You can use com_print_typeinfo() to have PHP generate a skeleton for the event sink class.

I'm having problems when trying to invoke a method of a COM object which exposes more than one interface. What can I do?

The answer is as simple as unsatisfying. We don't know exactly but we think you can do nothing.

So PHP works with COM, how about COM+?

COM+ extends COM by a framework for managing components through MTS and MSMQ but there is nothing special that PHP has to support to use such components.

If PHP can manipulate COM objects, can we imagine to use MTS to manage components resources, in conjunction with PHP?

PHP itself doesn't handle transactions yet. Thus if an error occurs no rollback is initiated. If you use components that support transactions you will have to implement the transaction management yourself.

Miscellaneous Questions

There can be some questions we can't put into other categories. Here you can find them.

1. How can I handle the bz2 compressed manuals on Windows?

How can I handle the bz2 compressed manuals on Windows?

If you don't have an archiver-tool to handle bz2 files » download the command line tool from Redhat (please find further information below).

If you would not like to use a command line tool, you can try free tools like » Stuffit Expander, » UltimateZip, » 7-Zip, or » Quick Zip. If you have tools like » WinRAR or » Power Archiver, you can easily decompress the bz2 files with it. If you use Total Commander (formerly Windows Commander), a bz2 plugin for that program is available freely from the » Total Commander site.

The bzip2 command line tool from Redhat:

Win2k Sp2 users grab the latest version 1.0.2, all other Windows user should grab version 1.00. After downloading rename the executable to bzip2.exe. For convenience put it into a directory in your path, e.g. C:\Windows where C represents your Windows installation drive.

Note: lang stands for your language and x for the desired format, e.g.: pdf. To uncompress the php_manual_lang.x.bz2 follow these simple instructions:

- open a command prompt window
- cd to the folder where you stored the downloaded php_manual_lang.x.bz2
- invoke bzip2 -d php_manual_lang.x.bz2, extracting php_manual_lang.x in the same folder

In case you downloaded the php_manual_lang.tar.bz2 with many html-files in it, the procedure is the same. The only difference is that you got a file php_manual_lang.tar. The tar format is known to be treated with most common archivers on Windows like e.g. » WinZip.